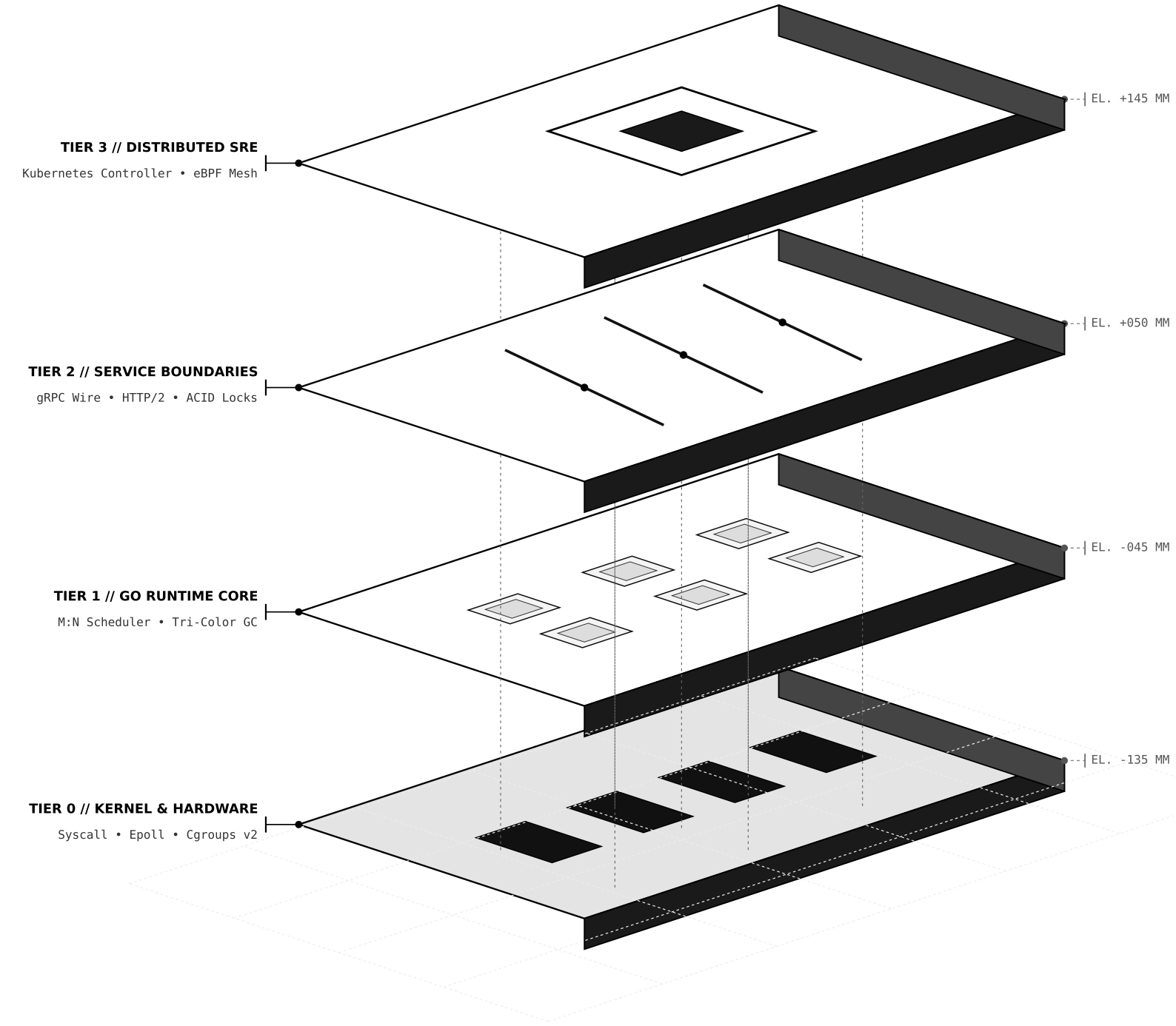
G O L A N G

GIÁO TRÌNH CẬP NHẬT LIÊN TỤC VỀ KỸ NGHỆ PHẦN MỀM VÀ DEVOPS/SRE

ĐOÀN NGỌC HOÀNG MINH



SYSTEM RUNTIME SPECIFICATION

M:N Work-Stealing Go Scheduler
Tri-Color Concurrent Mark-Sweep GC
Channel Rendezvous & Mutex Arena

DISTRIBUTED SRE INFRASTRUCTURE

Kubernetes Level-Triggered Reconcile
eBPF Kernel Tracing & Observability
Zero-Downtime Service Mesh Routing

PUBLICATION STANDARDS

Format: ISO A4 (210 x 297 mm)
Type: Source Serif 4 & JetBrains Mono
Print: High-Density Grayscale Monotone

MỤC LỤC

PHẦN I: NỀN TẢNG CƠ HỌC & BỘ NHỚ GO

Dẫn nhập	Trước khi viết dòng Go đầu tiên	3
Tổng quan	Mở cửa vào Go: Cài đặt và Tư duy Thực chiến	8
Chương 01	Đọc và viết một chương trình Go	14
Chương 02	Khi một bản sao vẫn chia sẻ dữ liệu	36
Chương 03	Mô hình dữ liệu và trách nhiệm thay đổi	50
Chương 04	Ranh giới xử lý lỗi: để bên gọi quyết định	79
Chương 05	Package là ranh giới	95

PHẦN II: TƯƠNG TRANH & ĐỘ ỔN ĐỊNH HỆ THỐNG

Chương 06	Thay đổi không sợ hãi	111
Chương 07	Dữ liệu đi vào và đi ra	135
Chương 08	Một race bắt đầu từ đâu	149
Chương 09	Dòng công việc có áp suất	159
Chương 10	Khi chương trình chậm hoặc phình	171
Chương 11	Lần theo một yêu cầu HTTP	193
Chương 12	Một service sống và tắt thế nào	203

PHẦN III: DỊCH VỤ PHÂN TÁN & CƠ SỞ DỮ LIỆU

Chương 13	Một thay đổi hoặc không có gì	219
Chương 14	Khi kiểu trở thành dữ liệu	241
Chương 15	Từ incident đến công cụ	257
Chương 16	Thấy được hệ thống	269
Chương 17	Đóng gói và điều phối	285
Chương 18	Đưa thay đổi ra production	305
Chương 19	Giữ type information khi abstraction lớn lên	325
Chương 20	Dự án tổng kết: opsprobe từ mã nguồn đến vận hành	338

PHẦN IV: KUBERNETES OPERATOR & eBPF NÂNG CAO

Chương 21	Vòng lặp điều hòa và Controller Pattern	356
Chương 22	Từ watch đến một controller Kubernetes thật	368
Chương 23	Từ controller đến operator: API riêng và vòng đời tài nguyên	387
Chương 24	Tự động hóa AWS bằng Go mà không biến credential thành bí mật dài hạn	408
Chương 25	Git và GitHub dưới góc nhìn của một hệ thống tự động hóa	425
Chương 26	Chuỗi cung ứng phần mềm có thể kiểm chứng	445
Chương 27	Quan sát Linux từ kernel bằng eBPF và Go	466
Chương 28	MCP và AIOps bằng Go: trao công cụ cho Agent mà không trao toàn quyền	489
Chương 29	Kỹ sư và bằng chứng trong kỷ nguyên Agent	511

BACK MATTER // TÀI LIỆU TRA CỨU

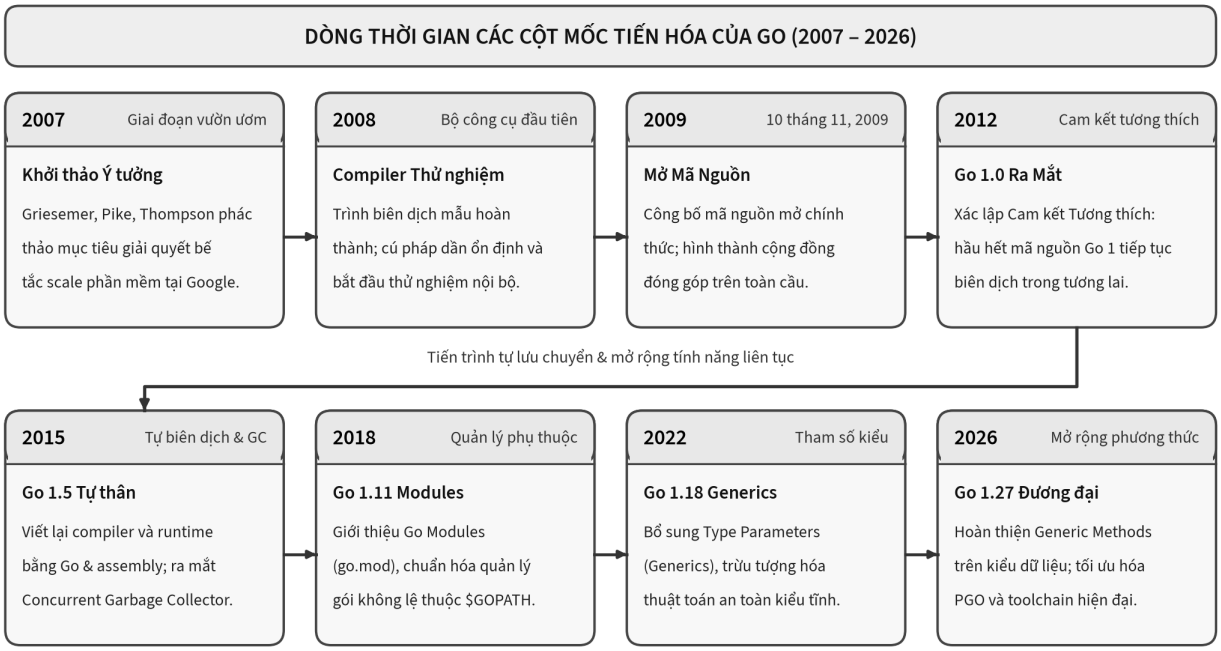
Atlas	Hồ sơ kiến trúc 50 thư viện DevOps & Cloud hàng đầu	524
Phụ lục A	Atlas Lỗi Thực Chiến: Chẩn đoán & Khắc phục nhanh	577

Trước khi viết dòng Go đầu tiên

Go không bắt đầu bằng câu hỏi “cú pháp nào còn thiếu?”. Ngôn ngữ bắt đầu từ một vấn đề kỹ thuật rất cụ thể: đến năm 2007, phần cứng máy tính đã phát triển nhanh hơn rất nhiều, nhưng công việc xây dựng và bảo trì một hệ sinh thái phần mềm quy mô lớn vẫn bị kéo chậm bởi ngôn ngữ, hệ thống đóng gói biên dịch và khối lượng công việc quản trị ghi chép mà lập trình viên phải mang theo.

Tại Google khi ấy, các kỹ sư Robert Griesemer, Rob Pike và Ken Thompson làm việc giữa những kho mã nguồn quy mô lớn, hệ thống nối mạng phân tán và phần cứng đang chuyển dịch mạnh mẽ sang kiến trúc đa lõi. C++ và Java là những lựa chọn phổ biến cho các dịch vụ máy chủ; chúng đem lại những năng lực quan trọng, đồng thời tạo ra một lượng phức tạp đáng kể trong quá trình đóng gói biên dịch, quản lý các gói phụ thuộc và tổ chức cấu trúc chương trình. Ở chiều ngược lại, các ngôn ngữ động mang lại cảm giác phát triển nhanh chóng hơn, nhưng phải đánh đổi một phần khâu kiểm tra kiểu dữ liệu tĩnh và hiệu năng thực thi. Bài toán không phải là lựa chọn một bên để lấn át bên còn lại. Bài toán là liệu một ngôn ngữ có thể kết hợp hiệu quả biên dịch, hiệu năng vận hành và tính an toàn kiểu vào cùng một quy trình làm việc tinh gọn hơn hay không.

Ngày 21 tháng 9 năm 2007, ba người bắt đầu phác thảo mục tiêu của một ngôn ngữ mới trên bảng trắng. Vài ngày sau, ngày 25 tháng 9, Pike đề xuất cái tên Go. Thiết kế ban đầu diễn ra song song với công việc khác; đầu năm 2008, Thompson đã có trình biên dịch thử nghiệm sinh mã C để thử các ý tưởng. Đến giữa năm, dự án đủ rõ để hướng tới trình biên dịch dùng trong thực tế. Ian Lance Taylor bắt đầu xây dựng phần đầu trình biên dịch GCC (`front end`) vào tháng 5 năm 2008; Russ Cox gia nhập vào cuối năm ấy và giúp đưa bản mẫu thử nghiệm (`prototype`), thư viện chuẩn cùng bộ công cụ tiến gần hơn tới một ngôn ngữ sẵn sàng ứng dụng vào thực tế sản xuất.



Hình 1 — Dòng thời gian không liệt kê mọi bản phát hành. Sơ đồ chỉ lưu giữ các bước làm thay đổi cam kết của Go với người phát triển phần mềm: công khai mã nguồn, ổn định lâu dài, tái thiết kế nền tảng thực thi, minh bạch phụ thuộc và hỗ trợ kiểu tổng quát tương thích ngược.

Go được công bố mã nguồn mở vào ngày 10 tháng 11 năm 2009. Ba năm đầu là giai đoạn ngôn ngữ biến chuyển nhanh chóng: nhóm thiết kế cùng cộng đồng thử nghiệm, loại bỏ và tinh chỉnh, trước khi chốt lại một nền tảng ổn định. Phiên bản Go 1 phát hành ngày 28 tháng 3 năm 2012. Từ thời điểm này, Lời hứa tương thích Go 1 (`Go 1 Compatibility Promise`) trở thành một phần bản sắc văn hóa của dự án: mã nguồn viết cho Go 1 được cam kết tiếp tục biên dịch và vận hành chính xác trong các bản Go 1 kế tiếp, với những ngoại lệ được nêu rõ. Cam kết đó làm thay đổi cách một tính năng mới được cân nhắc. Một bổ sung không chỉ cần hữu ích hôm nay; tính năng đó phải sống bền bỉ cùng hệ sinh thái trong nhiều năm dài.

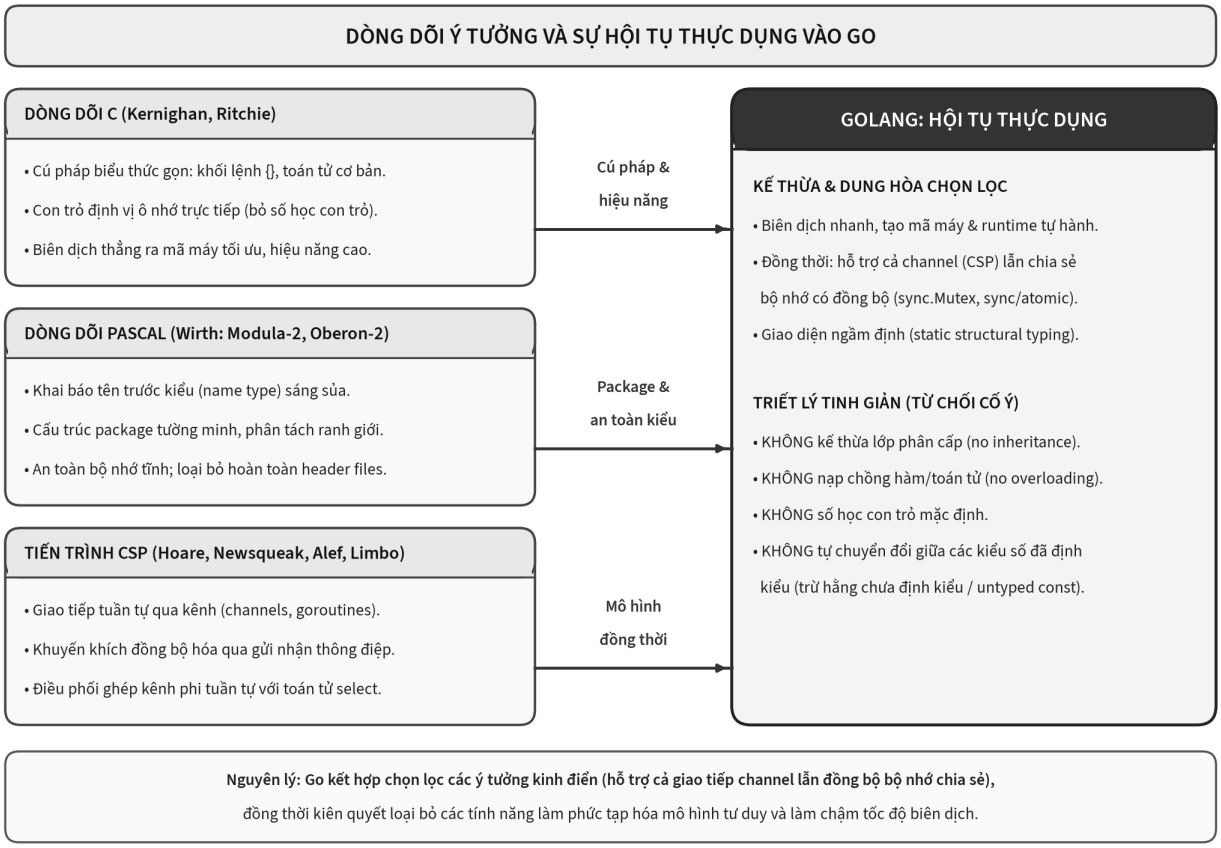
Phiên bản Go 1.5 là một cột mốc lịch sử về hiện thực hóa nền tảng. Bộ công cụ biên dịch chuyển dịch từ C sang Go, môi trường thực thi (`runtime`) cũng trở thành mã nguồn Go đi kèm một phần mã hợp ngữ (`assembly`). Bộ thu gom rác (`garbage collector`) được thiết kế lại để vận hành phần lớn công việc đồng thời với chương trình; bộ lập lịch (`scheduler`) cho phép giá trị mặc định của biến `GOMAXPROCS` chuyển từ 1 sang đúng số lượng vi xử lý luận lý của máy tính. Đó là những thay đổi nằm sâu dưới tầng bề mặt ngôn ngữ, làm sáng tỏ mục tiêu ban đầu: một chương trình Go cần vận hành hài

hòa với phần cứng đa lõi mà không ép buộc người viết phải tự gánh vác mọi chi tiết phức tạp của môi trường thực thi.

Những thay đổi lớn sau đó cũng đi theo nhịp tiến hóa thận trọng thay vì phá bỏ làm lại từ đầu. Phiên bản Go 1.11 giới thiệu mô-đun (`modules`) như một lựa chọn thay thế cho biến môi trường `GOPATH`, với cơ chế quản lý phiên bản và phân phối gói tích hợp trực tiếp vào bộ công cụ; mô hình này dần trở thành quy chuẩn quản lý phụ thuộc quen thuộc. Phiên bản Go 1.18, phát hành năm 2022, chính thức đưa tham số kiểu vào ngôn ngữ. Đến phiên bản Go 1.27 năm 2026, các phương thức tổng quát (`generic methods`) nối mảnh ghép còn thiếu vào bức tranh: phương thức có thể tự khai báo tham số kiểu riêng biệt. Vì vậy, "Go 2" không trở thành một nhánh viết lại tách rời; nhiều ý tưởng được thử nghiệm, thảo luận phản biện và đưa dần vào dòng phiên bản Go 1.x khi chứng minh được giá trị thực tiễn vững chắc.

Một dòng họ ý tưởng, không phải một cây sao chép

Ngôn ngữ Go kế thừa nhiều dòng tư tưởng học thuật cùng lúc. Cú pháp cơ bản đặt Go gần gũi với họ ngôn ngữ C. Cách nhìn nhận về việc khai báo, tổ chức gói mã nguồn và sự tường minh trong cấu trúc chịu ảnh hưởng đáng kể từ các ngôn ngữ Pascal, Modula và Oberon. Lĩnh vực xử lý đồng thời (`concurrency`) có một lịch sử phát triển dài hơn: mô hình tiến trình giao tiếp tuần tự (`CSP`) của Tony Hoare đi qua Newsqueak, Alef và Limbo trước khi các kênh truyền thông tin trở thành một phần của Go. Những cội nguồn này giải thích vì sao một số ý tưởng nghe có vẻ quen thuộc, chứ hoàn toàn không biến Go thành bản sao thụ động của bất kỳ ngôn ngữ nào trong số đó.



Hình 2 — Nhánh CSP giải thích lịch sử của các kênh truyền dữ liệu; nhánh C và Pascal/Modula/Oberon làm sáng tỏ những điểm xuất phát khác nhau. Thiết kế hoàn thiện vẫn phục vụ cho đúng bài toán kỹ thuật mà Go hướng tới giải quyết.

Một ngôn ngữ được sinh ra cho các kho mã nguồn quy mô lớn không chỉ cần cú pháp tinh gọn. Ngôn ngữ đòi hỏi một quy trình làm việc chuẩn mực bao quanh cú pháp: tốc độ biên dịch nhanh, công cụ định dạng tự động (`formatter`), cấu trúc gói và quản trị phụ thuộc minh bạch, trình biên dịch đưa ra phản hồi tức thời và bộ công cụ có khả năng phân tích mã nguồn. Lý do là vì Go thường được nói đến cùng `gofmt`, `go test` và công cụ dòng lệnh `go`, thay vì chỉ thuần túy là một danh sách các tính năng cú pháp.

Thiết kế bằng cách từ chối

Trong giai đoạn đầu, nhóm sáng lập đặt nặng sự đồng thuận: một tính năng mới chỉ được triển khai khi cả nhóm nhìn nhận rõ sự cần thiết. Câu hỏi không dừng ở việc “có thể thêm gì?”, mà còn là “có thể tinh giản gì để hệ thống vẫn vận hành hiệu quả?”. Kết quả mang lại là một tập hợp các quyết định đánh đổi (`trade-offs`) có chủ ý sâu sắc.

> **Một điều đáng nhớ:** Sự vắng mặt của cây phân cấp kế thừa, nạp chồng toán tử, cơ chế xử lý lỗi theo kiểu ném ngoại lệ hay kiểu tổng quát trong giai đoạn đầu không phải bản tuyên ngôn rằng các cơ chế ấy luôn tồi tệ. Mỗi cơ chế phát huy tác dụng trong những ngữ cảnh riêng biệt. Go ưu tiên một mô hình dễ đọc, dễ phân tích, biên dịch nhanh và duy trì sự ổn định dài hạn; cái giá phải trả là một số cách diễn đạt quen thuộc sẽ vắng mặt hoặc xuất hiện muộn hơn.

Việc không có cây phân cấp kiểu dữ liệu giải phóng người đọc khỏi việc lần theo cây kế thừa; thay vào đó, giao diện và cơ chế hợp thành (`composition`) sẽ được phân tích kỹ lưỡng khi chúng ta đã nắm vững mã nguồn cơ bản. Việc không nạp chồng toán tử bảo đảm ý nghĩa của các toán tử không bị thay đổi tùy tiện theo từng kiểu dữ liệu. Lỗi được trả về trực tiếp dưới dạng giá trị thông thường thay vì chuyển qua cơ chế ném ngoại lệ. Và kiểu tổng quát được chủ động trì hoãn cho đến khi thiết kế chứng minh mang lại giá trị thực tế lớn hơn hẳn so với mức độ phức tạp đưa vào hệ thống kiểu và bộ công cụ. Đây là các lựa chọn kỹ thuật hoàn toàn có thể thảo luận và đối chiếu, không phải là những giáo điều đạo đức.

Nhìn lịch sử theo cách này giúp lý giải cảm nhận đầu tiên khi tiếp cận Go. Ta sẽ nhận thấy nhiều câu lệnh ngắn gọn, ít từ khóa rườm rà, khai báo rõ ràng và bộ công cụ hiện diện từ rất sớm. Những đặc điểm ấy là dấu vết của một mục tiêu nhất quán: giảm thiểu ma sát cho các kỹ sư cùng hợp tác xây dựng, đọc hiểu và bảo trì phần mềm qua nhiều năm tháng.

Sau gần hai thập kỷ tiến hóa, cách tiếp cận các quyết định thiết kế này không phải là đọc thêm những trang lịch sử. Hãy đặt chúng ngay trước mắt trong một tệp mã nguồn `.go` tinh gọn.

Mở cửa vào Go

Một tệp nguồn Go chứa văn bản mã hóa UTF-8. Để chạy nó, bộ công cụ phải chọn các tệp thuộc chương trình, kiểm tra cú pháp và kiểu, biên dịch rồi liên kết thành tệp thực thi. Compiler có thể từ chối một phép cộng sai kiểu; nó không biết một endpoint có đúng với yêu cầu vận hành hay không. Ta sẽ dùng sự phân biệt ấy ngay từ chương trình đầu tiên.

Trước mắt, hãy đọc một tệp nhỏ và xác định ba điều: nó thuộc package nào, nó dùng tên nào từ package khác, và hàm nào sẽ chạy sau giai đoạn khởi tạo. Chưa cần biết allocator hay scheduler để trả lời đúng ba câu hỏi này.

Chương trình tối thiểu và Luồng thực thi

Lưu chương trình sau trong tệp `main.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("he thong san sang")
}
```

Để chỉ thị bộ công cụ biên dịch và chạy tệp mã nguồn trên môi trường dòng lệnh, ta thực thi:

```
go run main.go
```

Chương trình ghi dòng `he thong san sang` ra đầu ra tiêu chuẩn, thường là terminal đang chạy lệnh. Dòng này xuất hiện khi `main` gọi `fmt.Println`, không phải khi compiler đọc string trong source.

Đọc từng thành phần

`package` và `import` là khai báo; `func main()` khai báo một hàm; trong thân hàm có lời gọi `fmt.Println(...)`. Dấu ngoặc nhọn giới hạn thân hàm, còn dấu ngoặc kép tạo string literal. Những vai trò này giúp phân biệt phần mô tả chương trình với hành động xảy ra khi chương trình chạy.

Khai báo gói và Không gian tên: `package main`

Mỗi tệp nguồn Go có một package clause trước các import và khai báo khác; comment có thể đứng trước nó. `package main` nói tệp này thuộc package tên `main`. Các khai báo cấp package được dùng chung giữa các tệp của package; tên import và biến cục bộ có phạm vi khác, sẽ được lần cụ thể ở Chương 1.

Theo đặc tả chính thức của Go, một chương trình hoàn chỉnh có thể thực thi độc lập (complete program) được tạo thành bằng cách liên kết một package không được package nào khác import, gọi là main package, cùng toàn bộ các package phụ thuộc bắc cầu của nó. Main package này bắt buộc phải có tên package là `main` và phải chứa một khai báo hàm `main` không nhận tham số và không trả về kết quả (`func main()`). Khai báo `package main` đơn thuần chưa đủ để sinh ra file thực thi nếu thiếu định nghĩa hàm `main`. Ngược lại, một package không phải `main` đại diện cho một thư viện; khi chạy lệnh `go build` trên một thư viện như vậy, bộ công cụ Go sẽ biên dịch mã nguồn để kiểm tra tính hợp lệ về cú pháp và kiểu dữ liệu nhưng mặc định không phát sinh tệp thực thi nào trên đĩa.

Nhập thư viện chuẩn: `import "fmt"`

`import "fmt"` khai báo phụ thuộc vào package thư viện chuẩn có import path `"fmt"`. Trong tệp này, tên `fmt` cho phép truy cập các định danh được export của package, chẳng hạn `fmt.Println`. Import không phải câu lệnh gọi một hàm để "nạp công cụ" mỗi khi luồng thực thi đi qua nó.

Import thông thường phải được dùng. Để thấy ranh giới, xóa lời gọi `fmt.Println` nhưng giữ `import "fmt"`: compiler báo import không được sử dụng. Đổi thành `import . "fmt"` vẫn không làm chương trình đó hợp lệ. Dot import đưa các định danh được export vào phạm vi tệp để viết

`Println(...)` không có tiền tố; nó không miễn quy tắc sử dụng import. Chỉ dạng `import _ "fmt"` trong phép thử này khai báo phụ thuộc chỉ để lấy hiệu ứng khởi tạo, không tạo tên package để gọi. Trong chương trình đầu tiên, cách viết thông thường `fmt.Println` giữ nguồn gốc tên rõ hơn.

Hàm khởi điểm và khởi tạo package: `func main()`

Từ khóa `func` định nghĩa một hàm. Tên hàm `main` đại diện cho điểm nhập cuộc thực thi logic của người dùng. Hàm `main` khởi điểm không nhận bất kỳ tham số nào và không trả về giá trị.

Theo Go specification, chương trình khởi tạo các package trước khi gọi `main`. Package được import phải khởi tạo xong trước package phụ thuộc vào nó; trong mỗi package, khởi tạo biến cấp package đi trước các hàm `init`. Đó là khế ước kỹ thuật được dùng để dự đoán hành vi mã nguồn người dùng. Việc toolchain chuẩn dựng scheduler và allocator trước đó là tầng triển khai runtime, không phải danh sách thao tác mà ngôn ngữ bắt mọi implementation phải làm theo. Chương 5 sẽ đào sâu thứ tự khởi tạo; các chương runtime mới lần đường bootstrap đã pin theo phiên bản.

Truy xuất định danh công khai và Xuất dữ liệu: `fmt.Println`

Trong `fmt.Println`, `fmt` là tên package được import và `Println` là tên hàm được export của package đó. Dấu `.` nối hai tên trong một định danh có tiền tố package.

Một định danh được export khi bắt đầu bằng chữ hoa Unicode thuộc nhóm Lu và được khai báo ở cấp package, hoặc là tên field hay method. Vì vậy `Println` được dùng từ package khác; một biến cục bộ tên `Result` trong `main` không vì chữ hoa mà trở thành API. Theo contract của `fmt`, `Println` định dạng các argument, thêm newline và ghi ra standard output. Trong thư viện chuẩn Go 1.27.1, nó dùng `os.Stdout`; đường I/O tới OS phụ thuộc đối tượng đầu ra và nền tảng, không phải lời hứa về một file descriptor cụ thể của API `Println`.

Biên dịch Mã máy: Phân biệt `go run` và `go build`

Bộ công cụ Go cung cấp hai lệnh cơ bản với vai trò rõ ràng trong vòng đời phát triển:

Câu lệnh	Hợp đồng công cụ (Tool Contract)	Hành vi tệp trên đĩa	Bối cảnh sử dụng
`go run main.go`	Build rồi chạy main package được chỉ định.	Dùng executable tạm; không xuất binary vào working directory.	Thử nghiệm local.
`go build .`	Build package hiện tại; main package có thể tạo executable.	Với main package, mặc định xuất executable theo tên package/directory; `-o` chọn output khác.	Tạo artifact để phân phối.

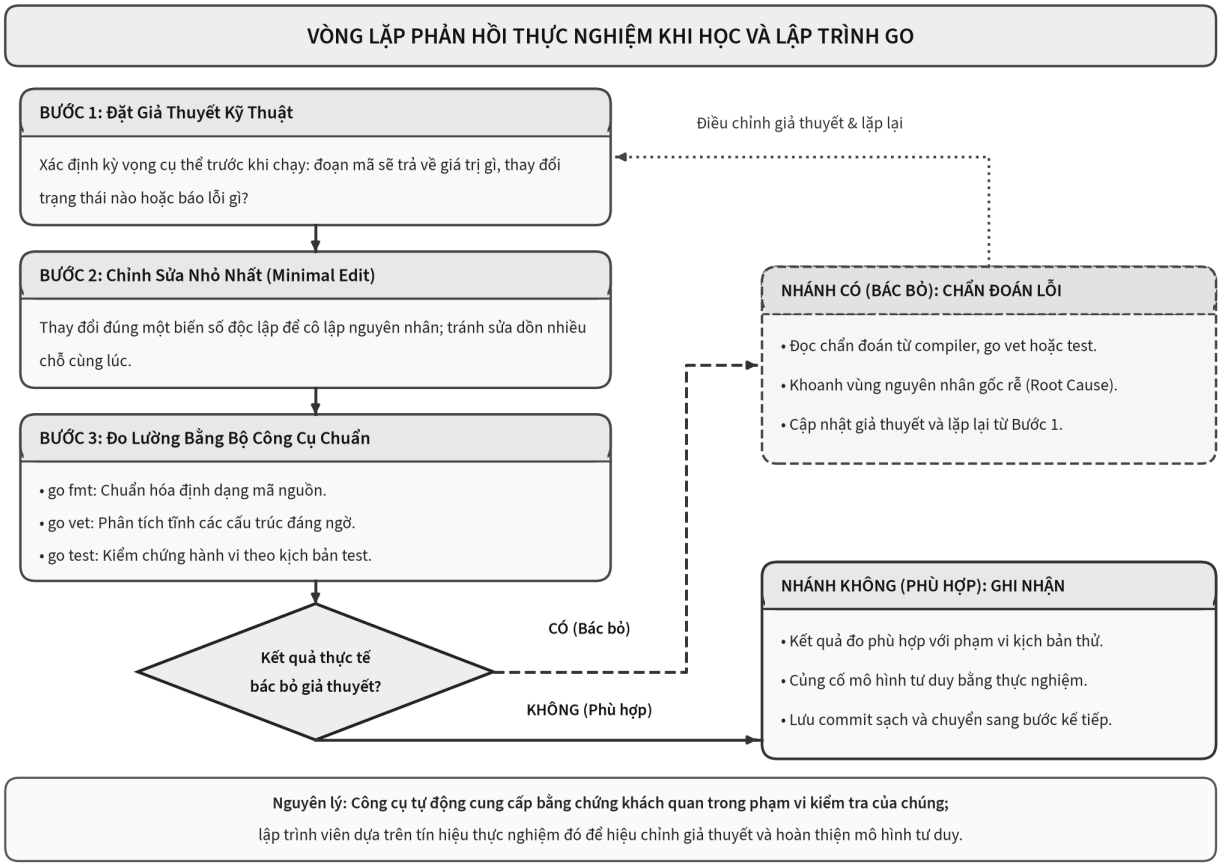
Trong lần gọi mặc định của toolchain Go 1.27.1, `go run` dùng tệp thực thi tạm và dọn work directory khi kết thúc; tùy chọn `-work` giữ lại thư mục đó để điều tra. Work directory khác `GOCACHE`, nơi toolchain giữ kết quả build có thể tái sử dụng giữa các lần gọi.

Tệp nhị phân sinh ra từ `go build` là mã máy thực thi trực tiếp trên vi kiến trúc CPU đích. Nó không chạy trên máy ảo như bytecode của Java và không dựa vào trình thông dịch như Python.

Tệp thực thi do toolchain chuẩn tạo ra có mã ứng dụng cùng phần hỗ trợ runtime cần cho chương trình, không phải toàn bộ source runtime hay mọi thư viện Go. Khả năng chạy trên máy khác còn phụ thuộc target OS, kiến trúc, cách liên kết và tài nguyên ngoài như tệp cấu hình. Chương 17 sẽ kiểm tra các phụ thuộc ấy trong container; ở đây build thành công chưa đủ để kết luận triển khai ở đâu cũng chạy.

Vòng lặp Phản hồi: Giả thuyết, Đo đạc và Giải thích

Trước khi sửa một dòng, hãy dự đoán: thay đổi này làm chương trình bị từ chối trước khi chạy, hay vẫn chạy nhưng cho kết quả khác? Sau đó chạy phép thử nhỏ để so dự đoán với bằng chứng. Cách làm này hữu ích hơn việc chỉ ghi nhớ một thông báo lỗi.



Hình 3 — Vòng lặp phản hồi giả thuyết – đo đạc – sửa mã nguồn. Trình biên dịch không phải là chướng ngại vật; trình biên dịch là công cụ kiểm định tĩnh sớm nhất cho các dự đoán của ta.

Bảng dưới đây minh họa ba thí nghiệm nhỏ về ranh giới kiểm tra tĩnh của Go compiler:

Thí nghiệm kiểm chứng	Thao tác thay đổi mã nguồn	Phần chính của diagnostic	Cơ chế phân tích của Compiler
Đổi tên gói khởi điểm	Đổi `package main` thành `package worker`, sau đó chạy `go run main.go`.	`is not a main package`	Gói đích không thỏa mãn điều kiện tạo tệp thực thi độc lập; đây là kiểm tra của `go run`.
Import không sử dụng	Thêm dòng `import "time"` nhưng không tham chiếu định danh được export nào của `time`.	`"time" imported and not used`	Compiler kiểm tra việc sử dụng import; không phải chứng minh toàn bộ dependency graph đã tối thiểu.
Khai báo biến cục bộ thừa	Khai báo `workerID := 1` trong `main` nhưng không dùng.	`workerID declared and not used`	Kiểm tra usage của biến cục bộ; không phải chứng minh toàn bộ dead code đã được phát hiện.

Compiler bắt một lớp lỗi cú pháp, kiểu và usage trước khi chạy; nó không chứng minh phần lớn lỗi vận hành đã bị loại bỏ. Deadline, quyền, resource budget và yêu cầu nghiệp vụ vẫn cần phép kiểm tra khác. Chương tiếp theo đi vào định danh, kiểu dữ liệu và phạm vi biến để người học hiểu chính xác trình biên dịch đang kiểm tra điều gì.

Chương 1 — Đọc và viết một chương trình Go

Chương trình dưới đây dùng một mã trạng thái HTTP đã có sẵn để tạo nhãn rồi in kết quả. Nó chưa gọi mạng, và quy tắc `status >= 500` chỉ là policy của ví dụ, không phải định nghĩa đầy đủ về sức khỏe dịch vụ. Ta sẽ lần từ từng tên và dấu trong source tới giá trị được tạo, nhánh được chọn và dữ liệu đi ra terminal.

```
package main

import "fmt"

const service = "checkout"

func classify(status int) string {
    if status >= 500 {
        return "failed"
    }
    return "healthy"
}

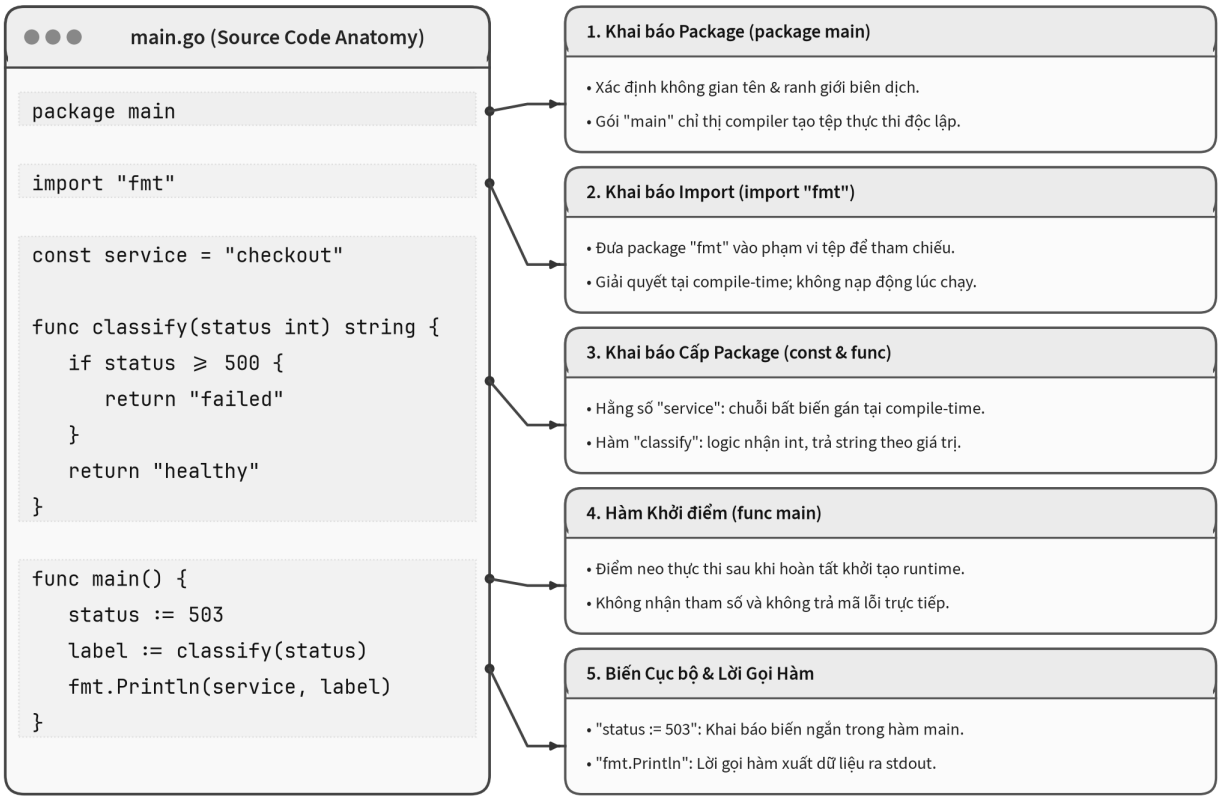
func main() {
    status := 503
    label := classify(status)
    fmt.Println(service, label)
}
```

Chạy chương trình trong một module nhỏ

Lưu đoạn mã trên thành tệp `main.go` trong một thư mục rỗng. Trước khi chạy, hãy khởi tạo một mô-đun cục bộ cho thư mục đó. Lệnh `go mod init` khởi tạo tệp `go.mod` và xác lập đường dẫn mô-đun (**module path**); một mô-đun quản lý một hoặc nhiều gói (**packages**) cùng các quan hệ phụ thuộc (**dependencies**) của chúng. Một mô-đun chưa đồng nghĩa với một gói đơn lẻ, và cũng không bắt buộc mã nguồn phải được phát hành công khai. Trong bài mở đầu này, điều cốt lõi là hiểu rằng thư mục đang nằm trong sự quản lý của một mô-đun và bộ công cụ Go dựa vào tệp `go.mod` để nhận diện mô-đun cùng các phụ thuộc liên quan.

```
go mod init example.com/first-go
go run .
```

Lệnh đầu tạo `go.mod`; `example.com/first-go` là tên định danh minh họa, không phải một địa chỉ tên miền thực tế cần sở hữu. Lệnh sau yêu cầu Go xây package `main` trong thư mục hiện tại rồi chạy executable tạm. Ta sẽ trở lại module, đường dẫn import, đồ thị phụ thuộc và cách một package trở thành ranh giới ở Chương 5. Hiện tại chỉ cần giữ một mốc thực hành: mỗi lab nằm trong một thư mục có `go.mod` riêng, và terminal phải đứng đúng thư mục của lab trước khi chạy lệnh được ghi trong bài.



Hình 4 — Giải phẫu trực quan cấu trúc một tệp nguồn Go. Tên gói, các khai báo cấp tệp và hàm khởi điểm có ranh giới rõ ràng.

Để thấu suốt chương trình trên, ta lần theo luồng chuyển dịch trạng thái qua ba câu hỏi: chương trình bắt đầu thực thi ở đâu, giá trị nào được tạo ra, và thông điệp nào được gửi ra ngoài? Câu trả lời lần lượt là: điểm khởi đầu nằm tại hàm `main`, giá trị số nguyên `503` được gán vào biến `status` rồi truyền qua hàm `classify` theo ngữ nghĩa giá trị để nhận lại chuỗi phân loại, và

cuối cùng lời gọi hàm `fmt.Println` gửi dữ liệu qua thiết bị đầu ra tiêu chuẩn.

Bản đồ Cấu trúc của một Tệp Nguồn

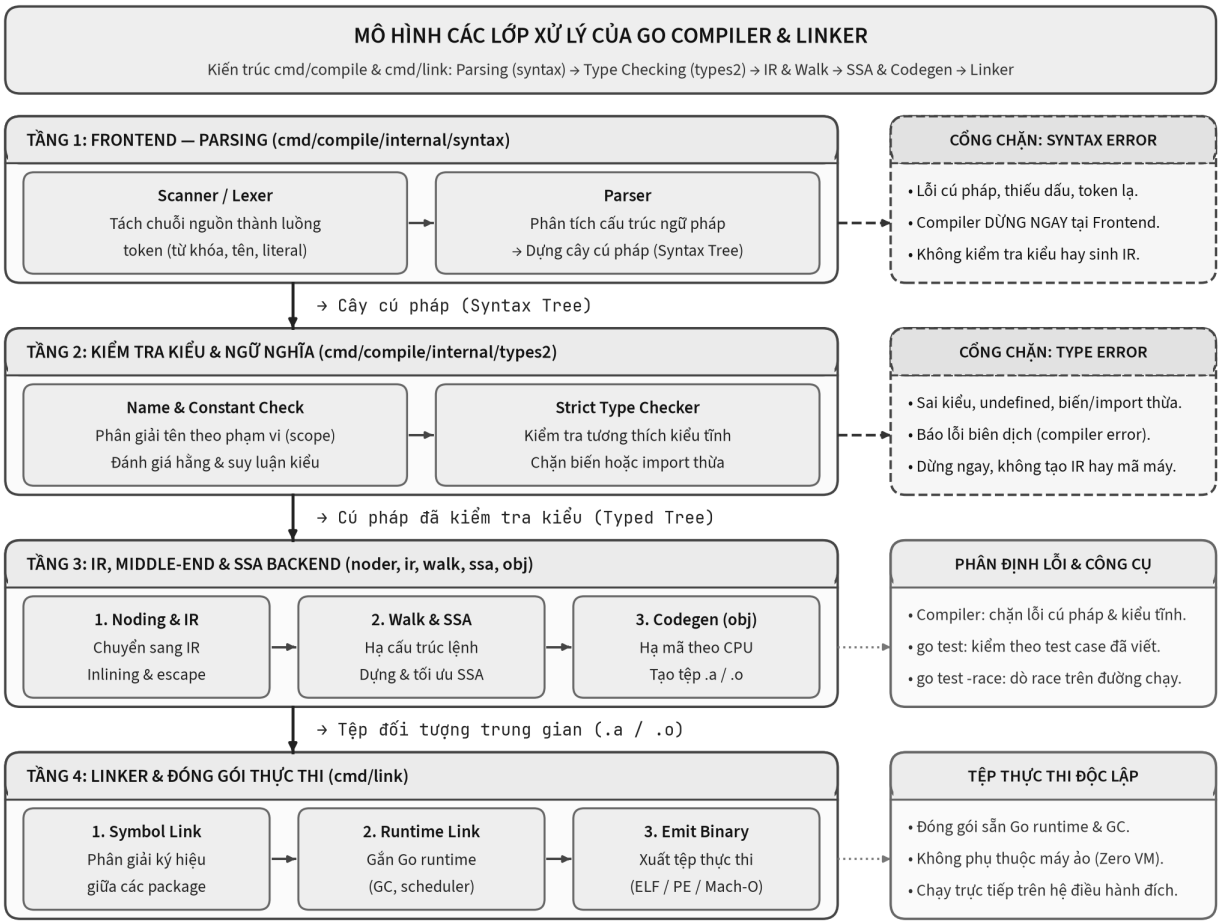
Từ trên xuống dưới, tệp mã nguồn được phân chia thành bốn vùng không gian cú pháp rõ rệt:

Một tệp mã nguồn Go được cấu trúc từ bốn thành phần ngữ pháp cơ bản: khai báo gói (`package`) xác định không gian tên và đơn vị biên dịch; khai báo import (`import`) thiết lập các gói phụ thuộc bên ngoài; khai báo cấp tệp định nghĩa hằng số (`const`), biến toàn cục (`var`), kiểu dữ liệu và hàm (`func`); và hàm khởi điểm (`func main`) là điểm neo thực thi logic khi chương trình khởi chạy. Bảng dưới đây đối chiếu chi tiết vai trò ngữ pháp và tác động thực thi của từng dòng mã trong tệp `main.go`:

Dòng mã	Vai trò ngữ pháp	Tác động thực thi
`package main`	Khai báo gói	Báo hiệu cho compiler tạo tệp thực thi độc lập khi có hàm `main`.
`import "fmt"`	Khai báo import	Đặt tên `fmt` trong phạm vi tệp; không phải một thao tác I/O lúc chạy.
`const service = "checkout"`	Khai báo hằng	Gắn định danh `service` với chuỗi bất biến tại lúc biên dịch.
`func classify(status int) string`	Khai báo hàm	Xác lập chữ ký hàm nhận `int` và trả về `string`.
`status >= 500`	Biểu thức so sánh	Đánh giá điều kiện nhị phân, sinh giá trị `bool`.
`return "failed"`	Câu lệnh trả về	Trả giá trị chuỗi cho bên gọi và chuyển giao quyền điều khiển.
`status := 503`	Khai báo biến ngắn	Giới thiệu biến `status` mang kiểu `int` và gán giá trị 503.
`fmt.Println(service, label)`	Lời gọi hàm	Định dạng các argument rồi ghi ra standard output.

Việc định vị chính xác vai trò ngữ pháp của từng dòng giúp ta đọc thông điệp lỗi của trình biên dịch một cách bình tĩnh: lỗi có thể bắt nguồn từ một biểu

thức sai cú pháp, một phép so sánh cấn kiểu dữ liệu, hay một biến bị gọi ngoài phạm vi sống, thay vì cảm giác hoang mang rằng toàn bộ chương trình đang bị hỏng.



Hình 5 — Mô hình các lớp xử lý của Go compiler. Trình biên dịch đọc tệp văn bản từ cấp độ từ vựng (token), dựng cây cú pháp trừu tượng (AST), kiểm tra chặt chẽ tính tương thích của kiểu dữ liệu, rồi mới sinh mã máy thực thi.

Quan hệ Giữa Khai báo, Biểu thức, Câu lệnh, Kiểu và Giá trị

Để xây dựng một mô hình tư duy vững chắc, ta cần phân biệt rạch ròi các khái niệm nền tảng thường bị đánh đồng trong lập trình:

Khai báo: Giới thiệu một tên và thứ mà tên ấy chỉ tới, như biến, hằng số, kiểu hoặc hàm, trong một phạm vi xác định. Với `const service = "checkout"`, tên `service` chỉ tới một hằng số; nó không phải một câu gán lại biến mỗi lần `main` chạy.

Biểu thức (Expression): Kết hợp toán hạng và toán tử để tính giá trị. Ví dụ `status >= 500` cho một giá trị kiểu `bool`. Lời gọi hàm cũng là biểu thức: có thể trả một hoặc nhiều kết quả; lời gọi không trả kết quả được dùng như

câu lệnh. Ngữ cảnh quyết định cách dùng các kết quả ấy. Đánh giá biểu thức có thể gọi hàm làm thay đổi dữ liệu hoặc gây hoảng loạn (`panic`), nên không đồng nghĩa với một phép tính không có tác động phụ.

Câu lệnh (Statement): Là đơn vị thực thi hoàn chỉnh chỉ dẫn máy tính thực hiện một hành động cụ thể, chẳng hạn như rẽ nhánh điều kiện (`if`), lặp vòng (`for`), gán giá trị (`=`), hoặc trả về từ hàm (`return`). Câu lệnh cấu thành luồng chảy động của chương trình.

Kiểu dữ liệu (Type): Là định nghĩa trừu tượng quy định tập hợp các giá trị hợp lệ và tập hợp các phép toán được phép thực hiện trên các giá trị đó. Kiểu cho compiler biết các operation hợp lệ; representation vật lý, kích thước và cách mã máy dùng giá trị còn phụ thuộc implementation và kiến trúc.

Giá trị (Value): Là dữ liệu hoặc kết quả cụ thể thuộc một kiểu. Kiểu quyết định những phép toán có nghĩa đối với giá trị đó. Việc trình biên dịch có phải hiện thực giá trị trong thanh ghi, bộ nhớ hay loại bỏ chỗ lưu trữ khi tối ưu là quyết định triển khai, không phải một phần của định nghĩa giá trị ở tầng ngôn ngữ.

Lời gọi hàm (**function call**): Là cơ chế chuyển giao quyền điều khiển từ hàm gọi sang hàm được gọi, đi kèm việc đánh giá các biểu thức đối số và truyền các giá trị đó theo đúng ngữ nghĩa giá trị của Go. Trình biên dịch có thể nội suy (**inline**) thân hàm hoặc truyền tham số qua thanh ghi/ngăn xếp theo giao diện nhị phân ứng dụng nội bộ (**internal ABI**), nhưng đó là quyết định tối ưu hóa của bộ công cụ thực tế (**toolchain**), không phải là khế ước trực tiếp của mã nguồn.

Định danh và Từ khóa

Trong ví dụ trên, `service`, `fmt`, `Println`, `classify`, `status` và `label` là các định danh (identifiers), tức những tên gọi do lập trình viên hoặc thư viện quy ước để đại diện cho biến, kiểu dữ liệu hoặc hàm số. Ngược lại, `package`, `import`, `const`, `func`, `if` và `return` là các từ khóa (keywords) bất biến của ngôn ngữ Go, được dành riêng cho trình phân tích cú pháp.

Định danh bắt đầu bằng một chữ cái Unicode hoặc ký tự gạch dưới _, theo sau bởi các chữ cái, chữ số Unicode hoặc ký tự gạch dưới. Phong cách quy chuẩn của Go thường ưu tiên kiểu lạc đà (như `MixedCaps` hoặc `mixedCaps`) cho các danh từ ghép. Cách viết `snake_case` vẫn hoàn toàn hợp lệ về mặt cú pháp; hãy chọn lối đặt tên nhất quán với toàn bộ giao diện lập trình và mã nguồn xung quanh thay vì nhầm lẫn quy ước phong cách thành lệnh cấm của trình biên dịch.

Quyền xuất khẩu (**exported identifier**) đòi hỏi hai điều kiện tiên quyết: định danh bắt đầu bằng một chữ cái in hoa Unicode (thuộc nhóm phân loại Lu), và được khai báo ở cấp độ gói hoặc là tên của một trường dữ liệu (**field**), phương thức (**method**). Hàm `Println` thỏa mãn cả hai tiêu chí; trong khi hàm `classify` thuộc phạm vi nội bộ. Một biến cục bộ `Label` nằm trong thân hàm vẫn chỉ là biến cục bộ, hoàn toàn không biến thành giao diện công khai chỉ vì được viết hoa chữ cái đầu.

Khai báo Biến và Khởi tạo Bộ nhớ

Biến giữ một giá trị thuộc kiểu xác định. Bốn cách viết dưới đây khác nhau ở việc ghi kiểu và giá trị khởi tạo, không quyết định biến nằm trên stack hay heap: khai báo tường minh với `var name Type = value` (ví dụ `var status int = 503`); khai báo suy luận kiểu với `var name = value`; khai báo không khởi tạo `var name Type` tuân theo giá trị mặc định; và khai báo ngắn `name := value` bên trong thân hàm. Bảng dưới đây tổng hợp các giá trị mặc định (**zero value**) chuẩn hóa theo đặc tả ngôn ngữ Go:

Kiểu dữ liệu	Giá trị Mặc định (Zero Value) theo Đặc tả	Ghi chú ngữ nghĩa
Số nguyên (`int`, `int64`, `byte`...)	`0`	Giá trị số không.
Số thực (`float32`, `float64`)	`0.0`	Số không dấu phẩy động.
Logic (`bool`)	`false`	Trạng thái sai mặc định.
Chuỗi ký tự (`string`)	`""`	Chuỗi rỗng với độ dài bằng 0.
Con trỏ, slice, map, channel, func, interface	`nil`	Giá trị rỗng có ý nghĩa và giới hạn thao tác riêng theo từng kiểu; Chương 2–3 phân tích rõ các cấu trúc dữ liệu này.

Với một biến được khai báo theo ngữ nghĩa Go thông thường mà không có giá trị khởi tạo tường minh, biến đó sẽ tự động nhận giá trị mặc định (**zero value**), tuyệt đối không nhận các byte rác ngẫu nhiên còn sót lại từ vùng nhớ trước đó. Điều này không áp dụng cho các khối bộ nhớ thô được can thiệp từ bên ngoài qua `unsafe` hoặc cgo. Giá trị mặc định cũng chưa chắc đã là một giá trị nghiệp vụ hợp lệ: khai báo `var port int` sẽ nhận giá trị 0, nhưng nếu giao diện API đòi hỏi dải cổng mạng từ 1 đến 65535 thì logic thẩm định vẫn phải từ chối giá trị này.

Cam kết ngữ nghĩa này không đồng nghĩa với việc mỗi biến đều bắt buộc phải chiếm một ô nhớ vật lý trong RAM và trình biên dịch luôn phải thực hiện thao tác xóa sạch các byte về số 0. Tùy thuộc vào chiến lược tối ưu hóa, trình biên dịch có thể xóa giá trị trên thanh ghi vi xử lý, gập hằng số tại thời

điểm biên dịch (**constant folding**), hoặc loại bỏ hoàn toàn việc cấp phát nếu biến không còn được sử dụng. Thao tác xóa byte bộ nhớ về 0 chỉ là cơ chế triển khai cụ thể khi vùng lưu trữ thực sự được cụ thể hóa (**materialized**) trên khung ngăn xếp (**stack frame**) hoặc vùng nhớ heap.

Khai báo ngắn cần ít nhất một biến mới không phải `_`. Nó có thể dùng lại biến đã khai báo trong cùng block nếu kiểu không đổi; khi ấy biến cũ được gán giá trị, không tạo bản thứ hai. Còn khai báo tên trùng ở block con tạo một biến khác.

> **Dừng để dự đoán:** Đọc đoạn mã sau và dự đoán chính xác hai dòng kết quả (**output**): biến `x` trong khối con (**block**) nhận giá trị nào? Sau khi thoát khỏi khối con, `x` có giá trị là 1, 2 hay 4?

```
x := 1
x, y := 2, 3
{
    x := 4
    fmt.Println(x, y)
}
fmt.Println(x, y)
```

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Kết quả in ra lần lượt là `4 3` rồi `2 3`. Tại câu lệnh `x, y := 2, 3`, do xuất hiện trong cùng block và có thêm biến mới `y`, Go áp dụng quy tắc redeclaration: biến `x` cũ nhận giá trị mới là 2 mà không tạo ra bản sao thứ hai. Khi bước vào block con, khai báo ngắn `x := 4` tạo ra một biến hoàn toàn mới nằm trong phạm vi từ vựng hẹp hơn, che khuất biến ngoài và khiến lệnh in đầu tiên xuất ra `4 3`. Sau khi thoát khỏi block con, biến nội bộ này hết phạm vi từ vựng; lệnh in cuối cùng đọc lại biến `x` của block cha (đang giữ giá trị 2) và biến `y` (3), cho kết quả `2 3`.

Phản ví dụ biên giới làm rõ bản chất: nếu bỏ `y` để viết tiếp `x := 2` trong cùng block, compiler lập tức báo lỗi `no new variables on left side of :=`. Ngược lại, phép gán `x = 2` thì hợp lệ vì nó chỉ cập nhật giá trị của

biến sẵn có. Phạm vi từ vựng, không phải hình thức của cú pháp `:=`, quyết định biến nào được dùng lại.

Hằng số: giá trị chưa cần một ô nhớ

Trước khi đọc tiếp, thử dự đoán vì sao `const small = 500` gán được cho `int16`, nhưng một variable `n := 500` không tự gán được cho variable `int16`. Hai bên cùng viết số 500; khác biệt nằm ở kiểu và thời điểm kiểm tra, không ở độ lớn của RAM.

```
const small = 500
var code int16 = small
n := small
// var other int16 = n // cần conversion tường minh
_ = code
_ = n
```

`small` là hằng số nguyên chưa có kiểu. Khi gán vào `code`, giá trị phải biểu diễn được bằng `int16`; khi dùng `:=` không có kiểu đích, kiểu mặc định của hằng số nguyên là `int`. Vì vậy `n` đã là variable kiểu `int`. Nếu thay 500 bằng 70000, phép gán vào `int16` bị compiler từ chối. Nếu viết `const small int = 500`, người lập trình đã ấn định kiểu ngay trong khai báo; hằng số có kiểu không còn được đối xử linh hoạt như hằng số chưa có kiểu ở phép gán ấy. Hằng số biểu diễn giá trị tại thời điểm biên dịch, không phải một biến bất biến có địa chỉ để lấy bằng &.

`iota` giúp diễn đạt một chuỗi hằng số trong cùng một nhóm khai báo. Nó bắt đầu từ 0, tăng theo từng mục khai báo hằng số trong nhóm, không theo số dòng văn bản; một mục khai báo nhiều tên vẫn dùng cùng giá trị `iota`. Mục bỏ biểu thức dùng lại biểu thức và kiểu của mục trước. Hãy tính ba giá trị rồi mới chạy thử:

```
const (
    Pending = iota
    Running
    Stopped
)
const Restarted = iota // nhóm mới, trở lại 0
```

Đây là tên cho các số nguyên, chưa phải một cơ chế enum tự kiểm tra giá trị hợp lệ. Variable `int` vẫn nhận được số không thuộc ba trạng thái. Nếu số được lưu vào database hay truyền qua giao thức, chèn một dòng giữa nhóm có thể đổi ý nghĩa các giá trị về sau; khi ấy giá trị tường minh và chính sách tương thích quan trọng hơn việc gõ ít ký tự. Chương 3 sẽ thêm một kiểu có tên để phân biệt ý nghĩa, nhưng kiểu ấy cũng không tự kiểm tra miền giá trị.

Trong `labs/edition-contracts`, ca kiểm thử `TestConstantsAndIdentity` lưu giữ vết thực nghiệm này thành một bài tập nhỏ. Hãy thử đổi hằng số chưa có kiểu thành hằng số mang kiểu `int` tường minh, rồi giải thích căn nguyên lỗi gán trước khi chỉnh sửa. Quy tắc chuẩn tắc bắt nguồn từ các mục Constants, Constant declarations và Representability trong đặc tả ngôn ngữ Go phiên bản Go 1.27; bài kiểm tra tập trung kiểm chứng các tình huống biên điển hình.

Phạm vi Từ vựng và Vòng đời Lưu trữ: Scope không đồng nghĩa với Lifetime

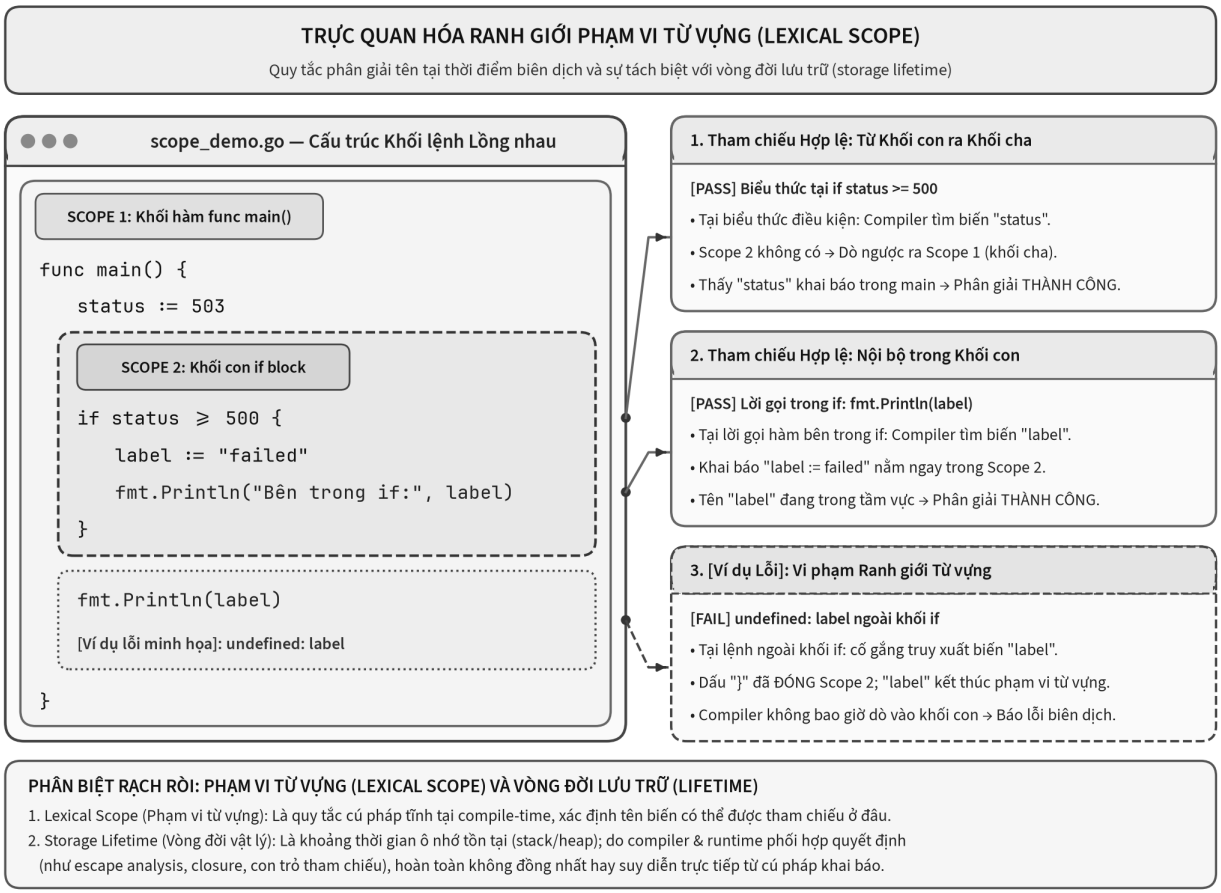
Một trong những nhầm lẫn phổ biến nhất của người mới học là đồng nhất phạm vi từ vựng của tên biến (scope) với vòng đời lưu trữ của giá trị trong bộ nhớ (lifetime). Đây là hai khái niệm độc lập:

Phạm vi từ vựng (**scope**) là vùng mã nguồn mà một định danh có thể được tham chiếu hợp lệ, bao gồm các khối lệnh ngoặc nhọn tường minh cũng như các khối lệnh ngầm định của tệp, `if`, `for`, `switch`; trong khi vòng đời lưu trữ (**storage lifetime**) là khoảng thời gian mà ô nhớ lưu trữ giá trị của biến thực sự tồn tại và hợp lệ trong không gian địa chỉ của tiến trình.

```
func main() {
    status := 503
    if status >= 500 {
        label := "failed"
        fmt.Println("Bên trong if:", label)
    }
    // Lệnh fmt.Println(label) ở đây sẽ gây lỗi biên dịch:
    // undefined: label
}
```

Biến `label` được khai báo bên trong khối lệnh `if`. Khi luồng đọc của trình biên dịch vượt ra ngoài dấu ngoặc nhọn đóng `}`, phạm vi từ vựng của tên `label` kết thúc; định danh này nằm ngoài lexical scope tại vị trí đó, nên quá trình phân giải tên (name resolution) của compiler không tìm thấy khai báo hợp lệ, từ chối câu lệnh với thông báo: `undefined: label`.

Tuy nhiên, việc một định danh kết thúc phạm vi từ vựng không đồng nghĩa với việc giá trị tương ứng bị hủy bỏ ngay lập tức. Nếu một con trỏ hoặc một bao đóng (**closure**) vẫn giữ khả năng quan sát giá trị đó, trình biên dịch sẽ lựa chọn vùng lưu trữ có vòng đời đủ dài; trong bản biên dịch thực tế đó có thể là bộ nhớ heap. Ngược lại, trình biên dịch có thể tận dụng thanh ghi, ngăn xếp (**stack**) hoặc lược bỏ hoàn toàn vị trí lưu trữ. Phạm vi từ vựng là quy tắc định danh ở tầng ngôn ngữ; vị trí lưu trữ vật lý là chi tiết tối ưu hóa của trình biên dịch, do đó không thể suy diễn vị trí bộ nhớ chỉ dựa vào cú pháp khai báo.



Hình 6 — Trực quan hóa ranh giới phạm vi từ vựng (lexical scope) của biến. Tên biến khai báo trong khối con không thể được tham chiếu từ khối cha bên ngoài.

Một cạm bẫy kỹ thuật nguy hiểm là hiện tượng che khuất biến (variable shadowing). Khi lập trình viên sử dụng toán tử `:=` bên trong một khối lệnh con với một tên biến trùng với biến đã có ở khối lệnh cha, Go sẽ tạo ra một biến hoàn toàn mới, che khuất biến bên ngoài trong suốt phạm vi của khối con:

```
func main() {
    retries := 0
    if retries == 0 {
        retries := 3
        fmt.Println("Gia tri trong if:", retries)
    }
    fmt.Println("Gia tri ngoai if:", retries)
}
```

Chương trình in ra giá trị 3 bên trong khối `if`, nhưng biến `retries` ban đầu ở ngoài vẫn giữ nguyên giá trị 0. Để cập nhật biến của khối cha, ta phải sử dụng phép gán = thay vì toán tử khai báo ngắn `:=`.

Hệ thống Kiểu Dữ liệu Cơ bản

Go kiểm tra kiểu trước khi chạy. Với hai biến số thuộc các kiểu khác nhau như `int` và `int64`, cùng giá trị hoặc cùng kích thước lưu trữ không làm phép cộng hay phép gán trực tiếp hợp lệ. Ta cần conversion tường minh trong ví dụ dưới đây. Quy tắc gán được rộng hơn riêng trường hợp số; Chương 3 sẽ phân biệt identity của kiểu, assignability và interface satisfaction.

Đặc tả ngôn ngữ quy định kiểu `int` và `uint` có cùng độ dài kích thước, 32 hoặc 64 bit; bộ công cụ Go 1.27.1 trên nền tảng mục tiêu amd64 quy định kích thước này là 64 bit. Các kiểu `int8`, `int16`, `int32`, `int64` cùng các kiểu không dấu tương ứng có độ rộng bit cố định. Trong đó, `byte` là biệt danh kiểu của `uint8`; `rune` là biệt danh của `int32`, thường dùng để biểu diễn một điểm mã Unicode (**code point**), dù bản thân kiểu số nguyên `int32` không tự động xác thực một giá trị có phải là điểm mã Unicode hợp lệ hay không.

Kiểu số thực gồm `float32` và `float64` tuân theo chuẩn IEEE-754. Toán tử so sánh bằng `==` hoàn toàn hợp lệ với số thực khi tính bằng nhau tuyệt đối (exact equality) chính là hợp đồng bài toán đòi hỏi (chẳng hạn so sánh với `0.0` hoặc hằng số cố định). Tuy nhiên, đối với các giá trị sinh ra từ chuỗi phép tính có sai số làm tròn (rounding error), ta thường cần tiêu chí gần bằng phù hợp với từng miền nghiệp vụ cụ thể, thay vì áp đặt máy móc một giá trị epsilon cố định cho mọi bài toán.

Kiểu logic `bool` chỉ nhận một trong hai giá trị `true` hoặc `false`, đồng hành cùng các toán tử logic gồm phép và &&, phép hoặc `||`, và phép phủ định `!`.

Kiểu `string` là chuỗi byte bất biến: chúng ta có thể gán một chuỗi mới cho biến, nhưng tuyệt đối không thể gán vào `text[0]` để sửa đổi từng byte của chuỗi hiện hành. Hàm `len` đếm số lượng byte, không đếm số lượng ký tự hiển thị. Chương 2 sẽ đi sâu vào định dạng UTF-8 và thao tác chuyển đổi sang lát cắt byte để kiểm chứng hai nguyên tắc này; ta không cần phải giả định bố cục bộ nhớ nội bộ của chuỗi để tuân thủ đúng khế ước ngôn ngữ.

Phép chuyển đổi kiểu bắt buộc phải thực hiện tường minh theo cú pháp `T(v)`, trong đó `T` là kiểu đích và `v` là giá trị cần chuyển:

```
var a int = 10
var b int64 = 20
var c int64 = int64(a) + b
```

Hằng số chưa có kiểu như `500` được xử lý theo quy tắc representability đã thử ở phần hằng số: `var code int16 = 500` hợp lệ vì giá trị biểu diễn được trong `int16`. Điều này khác với gán một biến đã có kiểu `int` sang `int16`; đừng gọi mọi phép gán hợp lệ là conversion ngầm hay xem hằng số như ngoại lệ duy nhất của hệ thống kiểu.

Toán tử và Biểu thức

Toán tử số học bao gồm cộng `+`, trừ `-`, nhân `*`, chia `/`, và chia lấy dư `%`. Phép chia giữa hai số nguyên `5 / 2` luôn cho kết quả nguyên `2` do phần thập phân bị cắt cụt; nếu muốn nhận kết quả số thực, ít nhất một toán hạng phải mang kiểu số thực như `5.0 / 2`.

Toán tử so sánh gồm ==, !=, <, <=, > và >=. Kết quả là giá trị logic; theo spec, kết quả so sánh là boolean chưa có kiểu và nhận kiểu phù hợp theo ngữ cảnh. Chẳng hạn `failed := status >= 500` tạo biến `failed` kiểu `bool`.

Trong hệ thống cú pháp Go, `count++` và `count--` là các câu lệnh độc lập chứ không phải biểu thức. Vì vậy `x = count++` không hợp lệ; Go cũng không có dạng prefix `++count`. Quy tắc này không cấm mọi biểu thức có side effect: một lời gọi hàm vẫn có thể thay đổi state.

Xuất Dữ liệu Định dạng với Gói `fmt`

Gói thư viện chuẩn `fmt` cung cấp ba cơ chế xuất dữ liệu chủ đạo với các đặc tính riêng biệt:

`fmt.Print`: Xuất các đối số nối tiếp nhau ra stdout mà không tự động chèn khoảng trắng phân cách (trừ khi hai đối số liên tiếp không phải là chuỗi) và không tự ngắt dòng.

`fmt.Println`: Xuất các đối số ra stdout, tự động thêm dấu cách giữa các đối số và luôn thêm ký tự xuống dòng ở cuối.

`fmt.Printf`: Định dạng chuỗi theo mẫu (format string) thông qua các ký hiệu động từ định dạng (verbs) bắt đầu bằng ký tự %.

Ký hiệu Verb	Ý nghĩa định dạng	Ví dụ biểu diễn
%v	Biểu diễn giá trị mặc định theo kiểu dữ liệu tự nhiên.	`fmt.Printf("%v", 503)` xuất `503`.
%+v	Mở rộng biểu diễn struct kèm theo tên từng trường dữ liệu.	`fmt.Printf("%+v", req)` xuất `{Path:/ ID:1}`.
%#v	Biểu diễn theo cú pháp Go của giá trị (Go-syntax representation); không phải mọi kết quả đều là biểu thức compile trực tiếp được.	`fmt.Printf("%#v", "ok")` xuất `"ok"`.
%T	Xuất tên định danh của kiểu dữ liệu.	`fmt.Printf("%T", 503)` xuất `int`.
%t	Xuất giá trị logic (`true` hoặc `false`).	`fmt.Printf("%t", true)` xuất `true`.

Ký hiệu Verb	Ý nghĩa định dạng	Ví dụ biểu diễn
%d	Xuất số nguyên theo hệ thập phân cơ số 10.	`fmt.Printf("%d", 255)` xuất `255`.
%x / %X	Xuất số nguyên hoặc mảng byte dưới dạng hệ thập lục phân.	`fmt.Printf("%x", 255)` xuất `ff`.
%f	Xuất số thực dấu phẩy động (có thể định dạng độ rộng như `%.2f`).	`fmt.Printf("%.2f", 3.14159)` xuất `3.14`.
%s	Xuất chuỗi ký tự hoặc mảng byte dưới dạng văn bản.	`fmt.Printf("%s", "ready")` xuất `ready`.
%q	Xuất chuỗi có dấu ngoặc kép và escape theo cú pháp Go; không thay validation hay lọc secret.	`fmt.Printf("%q", "ready")` xuất `"ready"`.
%p	Xuất giá trị con trỏ ở dạng thập lục phân có tiền tố 0x; không phải địa chỉ RAM vật lý.	`0xc000014088` là địa chỉ minh họa; `fmt.Printf("%p", &status)` có thể cho địa chỉ khác theo lần chạy.

Điều khiển Luồng: Rẽ nhánh và Lặp

Go tối giản hóa cấu trúc điều khiển luồng, loại bỏ các biến thể cú pháp dư thừa để giữ mã nguồn đồng nhất trên diện rộng.

Rẽ nhánh Điều kiện với `if / else`

Cú pháp `if` trong Go không yêu cầu cặp ngoặc đơn bọc điều kiện, nhưng bắt buộc phải có cặp ngoặc nhọn `{}` bao quanh khối lệnh, bất kể khối lệnh chỉ có một dòng đơn.

Go hỗ trợ câu lệnh khởi tạo ngắn đặt ngay trước mệnh đề điều kiện, phân cách bởi dấu chấm phẩy `;`:

```
if err := executeCheck(); err != nil {
    fmt.Println("Kiem tra that bai:", err)
}
```

Tên `err` dùng được trong điều kiện `err != nil`, thân `if` và các nhánh `else` của cùng câu lệnh. Nó không dùng được sau khi câu lệnh `if` kết thúc. Đây là phạm vi tên, không phải cơ chế tự đóng tài nguyên mà error có thể liên quan.

Lựa chọn Rời rạc với `switch`

Cấu trúc `switch` trong Go tự động kết thúc khi một nhánh `case` thỏa mãn, không đòi hỏi lập trình viên phải chèn lệnh `break` thủ công như các ngôn ngữ họ C. Nếu chủ đích muốn luồng chạy tiếp xuống nhánh kế tiếp, từ khóa `fallthrough` phải được khai báo tường minh.

Go cũng hỗ trợ `switch` không điều kiện (`switch {}`). Các `case` được kiểm tra theo thứ tự và nhánh đầu tiên đúng được chọn; dạng này có thể làm một chuỗi điều kiện dễ đọc hơn, không phải lúc nào cũng nên thay `if / else if`:

```
switch {
case status >= 500:
    return "failed"
case status >= 400:
    return "warning"
default:
    return "healthy"
}
```

Vòng lặp Duy nhất: `for`

Go chỉ có một từ khóa lặp duy nhất là `for`. Mọi nhu cầu lặp từ ba thành phần truyền thống, lặp theo điều kiện (tương đương `while`), lặp vô tận, cho đến duyệt tập hợp dữ liệu (`for range`) đều được biểu đạt thông qua `for`.

```
// Dang 1: Lap co bien dem truyen thong
for i := 0; i < 3; i++ {
    fmt.Println("Lan thu:", i)
}

// Dang 2: Lap theo dieu kien (tuong duong while)
for retries < 3 {
    retries++
}

// Dang 3: Lap vo han (cho den khi break hoac return)
for {
    if isDone() {
        break
    }
}

// Dang 4: Duyet mang hoac slice voi range
statuses := []int{200, 404, 500}
for index, code := range statuses {
    fmt.Printf("Chi so %d mang gia tri %d\n", index, code)
}
```

Trong ví dụ `[]int` này, `code` nhận bản sao giá trị phần tử; gán lại `code` không sửa phần tử của `statuses`. Muốn sửa phần tử gốc, dùng index rồi gán `statuses[index]`. Nếu phần tử về sau chứa pointer, slice hay map, bản sao vẫn có thể dẫn tới dữ liệu dùng chung; Chương 2–3 sẽ lần sự khác biệt đó. _ bỏ kết quả index khi không cần dùng.

Hàm và Ngữ nghĩa Truyền Giá trị

Hàm là đơn vị đóng gói logic độc lập, nhận vào các tham số hình thức và trả về kết quả cho bên gọi.

Go thực hiện truyền đối số theo giá trị (**pass-by-value**): tham số chỉ tiếp nhận bản sao giá trị của đối số truyền vào, không bao giờ trở thành chính biến gốc của bên gọi. Đây là ngữ nghĩa chuẩn tắc của lời gọi hàm, không phải một cam kết cứng nhắc về một lệnh sao chép từng bit hay vị trí lưu trữ ngăn xếp/thanh ghi; trình biên dịch hoàn toàn có thể tối ưu hóa miễn là bảo toàn hành vi quan sát được từ bên ngoài. Đối với kiểu số nguyên, việc gán lại

tham số bên trong hàm không làm thay đổi biến gốc. Đối với con trỏ hoặc các cấu trúc chứa con trỏ, bản sao giá trị con trỏ vẫn trỏ tới cùng một ô nhớ dữ liệu dùng chung.

Go cho phép hàm trả về cùng lúc nhiều giá trị độc lập, tạo tiền đề cho khuôn mẫu xử lý lỗi tiêu chuẩn của ngôn ngữ bằng cách trả về cặp giá trị gồm kết quả dữ liệu và đối tượng lỗi:

```
func parsePort(raw string) (int, error) {
    if raw == "" {
        return 0, fmt.Errorf("cong mang khong duoc de trong")
    }
    return 8080, nil
}
```

`parsePort` ở đây chỉ là stub để đọc chữ ký hai kết quả: nó chưa phân tích số và trả 8080 cho mọi input không rỗng. Không dùng nó để validation port; parser có kiểm tra range sẽ xuất hiện ở Chương 5. Nơi gọi nhận cả hai kết quả và kiểm tra lỗi trước khi dùng số:

```
port, err := parsePort("8080")
if err != nil {
    fmt.Println("Loi phan tich:", err)
    return
}
fmt.Println("Cong mang:", port)
```

Nhập môn Con trỏ: Địa chỉ Ô nhớ và Phép Giải Tham chiếu

Biến là tên gọi đại diện cho một vị trí lưu trữ giá trị. Con trỏ (pointer) là một giá trị chứa địa chỉ trong không gian địa chỉ của tiến trình trỏ tới một vị trí lưu trữ khác.

Toán tử lấy địa chỉ & trích xuất giá trị con trỏ tham chiếu tới biến đứng sau nó. Toán tử giải tham chiếu * đặt trước một biến con trỏ để đọc hoặc ghi đè giá trị tại vị trí mà con trỏ đang tham chiếu. Giá trị con trỏ in ra qua %p đại diện cho địa chỉ trong không gian địa chỉ ảo của tiến trình theo quy ước của nền tảng, không phải địa chỉ RAM vật lý.

```
func main() {
    x := 100
    var p *int = &x

    fmt.Printf("Gia tri goc: %d\n", x)     // 100
    fmt.Printf("Dia chi &x:  %p\n", &x)    // Vi du 0xc000014088
    fmt.Printf("Gia tri *p:  %d\n", *p)    // 100

    *p = 200
    fmt.Printf("Gia tri moi: %d\n", x)     // 200
}
```

Con trỏ cho phép các hàm khác nhau cùng thao tác và biến đổi một vùng dữ liệu mà không phải sao chép toàn bộ khối dữ liệu đó qua từng lời gọi hàm.

Thí nghiệm Nhỏ: Hợp ngữ Compiler Plan 9 đối chiếu với Mã máy objdump

Để hiểu rõ đường đi từ mã nguồn xuống phần cứng, ta cần phân biệt rạch ròi hai tầng biểu diễn thường bị nhầm lẫn: bản danh sách hợp ngữ trung gian của compiler (`-S`) và mã máy thực thi sau liên kết (`objdump`).

Thí nghiệm được thực hiện trên phiên bản Go 1.27.1, nền tảng mục tiêu `windows/amd64`, với mã nguồn tại tệp `labs/part1-reading-go/testdata/classify/main.go` tương tự ví dụ mở đầu chương. Hãy chạy lệnh từ thư mục `labs/part1-reading-go`. Cờ `-l` tạm thời vô hiệu hóa việc nội suy hàm (**inlining**) cho gói đang xây dựng để hàm `classify` hiển thị độc lập; đây là điều kiện để quan sát thực nghiệm, không phải cấu hình tối ưu hóa cho môi trường sản xuất. Câu lệnh dưới đây tạo tệp thực thi đồng thời xuất bản danh sách hợp ngữ của trình biên dịch:

```
go build -gcflags="-l -S" -o classify.exe ./testdata/classify
```

Biểu diễn trung gian Plan 9 từ `cmd/compile/internal/ssagen`:

```
main.classify STEXT nosplit size=34 align=0x0 args=0x8
    locals=0x0 funcid=0x0
    TEXT    main.classify(SB), NOSPLIT|NOFRAME|ABIInternal, $0-8
    CMPQ    AX, $500
    JLT 21
    LEAQ    go:string."failed"(SB), AX
    MOVL    $6, BX
    RET
    LEAQ    go:string."healthy"(SB), AX
    MOVL    $7, BX
    RET
```

Đoạn trích lọc lại các chỉ dẫn liên quan tới điều kiện và kết quả trả về, lược bỏ các dòng siêu dữ liệu; phần tiêu đề được định dạng lại cho vừa khung hiển thị. Hợp ngữ Plan 9 sử dụng các ký hiệu biểu tượng như `go:string."healthy"(SB)` khi chưa có địa chỉ liên kết nhị phân cuối cùng. Hãy đọc trực tiếp tệp nhị phân vừa tạo bằng công cụ `objdump`, thay vì biên dịch lại mã nguồn với các cờ khác rồi so sánh hai kết quả:

```
go tool objdump -s "main\.classify" classify.exe
```

Các cột địa chỉ, byte lệnh và instruction dưới đây được trích từ lần build local ấy; cột tên tệp/dòng và padding bị lược bỏ. Địa chỉ tuyệt đối có thể đổi khi môi trường hay artifact thay đổi, không phải giá trị cần học thuộc:

```
  0x1400a5260: 483df4010000    CMPQ AX, $0x1f4
  0x1400a5266: 7c0d            JL 0x1400a5275
  0x1400a5268: 488d053e100000  LEAQ runtime.rodata+685(SB), AX
  0x1400a526f: bb06000000      MOVL $0x6, BX
  0x1400a5274: c3              RET
  0x1400a5275: 488d057b110000  LEAQ runtime.rodata+1015(SB), AX
  0x1400a527c: bb07000000      MOVL $0x7, BX
  0x1400a5281: c3              RET
```

Sự đối chiếu giữa hai đầu ra làm nổi bật bản chất của toolchain:

Trong hàm `classify` của lượt biên dịch này, giao diện nhị phân nội bộ (**internal ABI**) phân bổ thanh ghi `AX` cho tham số `status`; hàm không cần dựng khung ngăn xếp (**stack frame**) riêng biệt. Điều đó không có nghĩa mọi hàm Go đều không sử dụng ngăn xếp, hay tên thanh ghi là một khế ước bất biến của mã nguồn.

Giá trị `$500` trở thành `$0x1f4` trong cách hiển thị của objdump, cùng giá trị số. Nhánh `JLT 21` của listing tương ứng `JL 0x1400a5275`, được mã hóa bằng `7c 0d`. Cả hai đều đi tới đường trả `"healthy"` khi `status < 500`; không có bằng chứng từ cặp đầu ra này rằng linker đã đảo điều kiện.

Các symbol string được giải quyết thành địa chỉ trong vùng mà objdump hiển thị là `runtime.rodata`. `LEAQ` đặt địa chỉ dữ liệu vào `AX`, `MOVL` đặt độ dài byte 6 hoặc 7 vào `BX`, rồi `RET` trả kết quả theo ABI của build này. Đây là biểu diễn cụ thể của string result, không thay định nghĩa string bất biến ở tầng ngôn ngữ.

Đọc Thông điệp Biên dịch: Phân tích Tĩnh Thay vì Phỏng đoán

Khi gặp lỗi biên dịch, hãy tìm tên, biểu thức hoặc khai báo mà thông báo chẩn đoán đang chỉ tới. Bảng dưới đây phân loại các dạng lỗi điển hình giúp chúng ta lựa chọn một phép thử có chủ đích:

Tình huống kiểm chứng	Thao tác tạo lỗi cú pháp	Thông báo thực tế từ Go 1.27.1 Compiler	Tầng phân tích của Compiler
Định danh chưa khai báo	Gán `status = 503` mà chưa từng khai báo biến.	`undefined: status`	Tầng phân giải tên (Symbol table lookup): Định danh không tồn tại trong bảng ký hiệu hiện hành.
Vượt ngoài tầm vực từ vựng	Gọi `fmt.Println(label)` ngoài khối `if`.	`undefined: label`	Tầng phân tích tầm vực (Lexical scoping): Tên biến nằm ngoài lexical scope tại điểm gọi.
Khai báo lại không có biến mới	Dùng `x := 10; x := 20` liên tiếp.	`no new variables on left side of :=`	Kiểm tra cú pháp khai báo ngắn: Vế trái của `:=` bắt buộc phải có ít nhất một biến mới.
Sai lệch kiểu tham số hàm	Truyền `int` vào hàm nhận `string`.	`cannot use status (variable of type int) as string value in argument to takesString`	Tầng kiểm tra kiểu tĩnh (`types2`): Chữ ký hàm từ chối đối số không tương thích kiểu.
Phép toán lệch kiểu số học	Thực hiện `a + b` giữa `int` và `int64`.	`invalid operation: a + b (mismatched types int and int64)`	Tầng kiểm tra kiểu tĩnh (`types2`): Cấm phép toán hai ngôi ngầm định giữa hai kiểu khác nhau.

Thông điệp biên dịch chỉ ra lỗi mà compiler phát hiện tại vị trí đang xét; nó không chứng minh chương trình đã đúng với yêu cầu hay an toàn khi chạy. Phân biệt lỗi phân giải tên, cú pháp và kiểu dữ liệu giúp ta đặt giả thuyết cụ thể rồi kiểm chứng bằng một thay đổi nhỏ. Chương 2 tiếp tục cách điều tra đó với slice: vì sao một bản sao giá trị vẫn có thể chia sẻ bộ nhớ nền.

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu ta đoán `a` và `b` khác nhau sau dòng `b[0] = 99`, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu "slice nguy hiểm". Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy `b := a`, và dữ liệu nào không được copy?

Một baseline để dự đoán

Với kiểu `int`, phép gán tạo ra một giá trị độc lập. Cả `a` và `b` đều có giá trị riêng.

```
a := 10
b := a
b = 99

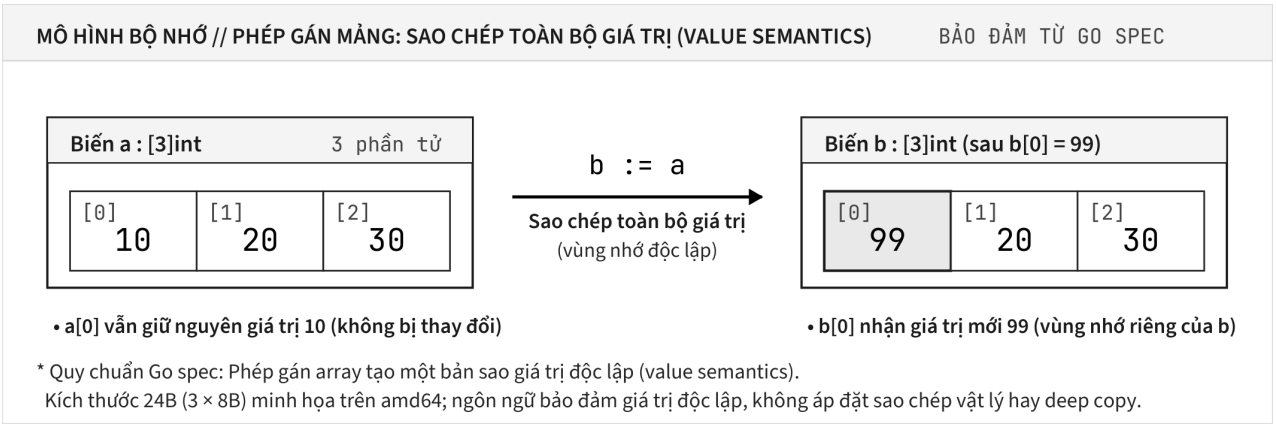
fmt.Println(a, b) // 10 99
```

Go đã sao chép giá trị của `a` sang `b`. Sau đó `b` nhận giá trị mới; `a` không đổi. Phép gán sao chép giá trị; khi gọi hàm, đối số cũng được truyền theo giá trị. Cả hai quy tắc đều không hứa rằng mọi dữ liệu được tham chiếu bên trong giá trị ấy sẽ được sao chép. Ta cần biết chính xác giá trị đang chứa gì.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên `[3]int` và `[4]int` là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```



Hình 7 — Phép gán của mảng sao chép toàn bộ các phần tử. Việc thay đổi `b[0]` chỉ tác động vào mảng `b` riêng biệt.

Đến đây, trực giác "copy nghĩa là độc lập" vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: `[3]int` trở thành `[]int`.

Khoảnh khắc slice đổi kết quả

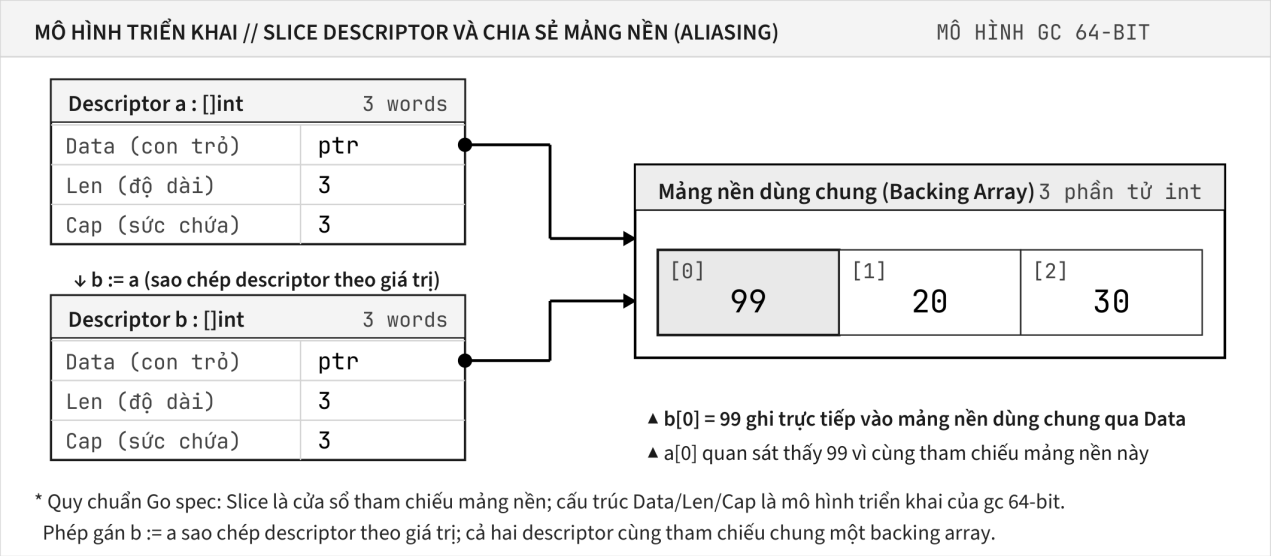
```
a := []int{10, 20, 30}
b := a
b[0] = 99
```

Kiểu `[]int` không phải là một mảng bỏ quên độ dài. Nó là kiểu slice: một giá trị slice mô tả một đoạn liên tục của mảng bên dưới.

Runtime Go 1.27.1 biểu diễn slice bằng ba trường: con trỏ dữ liệu, `len int` và `cap int` (`src/runtime/slice.go`). Với toolchain này trên amd64, giá trị slice chiếm 24 byte; số đó không bao gồm mảng được tham chiếu. Đây là chi tiết triển khai, không phải cam kết bố cục bộ nhớ của đặc tả Go. Đặc tả bảo đảm slice chia sẻ mảng bên dưới và có độ dài, dung lượng riêng; ta không cần một kích thước header cố định để suy luận về aliasing.

Chạy `TestSliceDescriptorImplementationMeasurement` trong `labs/part2-values` để đo bằng `unsafe.Sizeof` và ghi kèm Go version, hệ điều hành, kiến trúc. Thử nghiệm chỉ đo giá trị slice trên toolchain đang chạy, không đo toàn bộ bộ nhớ nó giữ sống và không chứng minh mọi compiler phải dùng cùng bố cục.

Khi phép gán `b := a` diễn ra, ngôn ngữ thực hiện sao chép giá trị theo ngữ nghĩa (copy by value). Trong mô hình ba từ máy, các thành phần của descriptor được sao chép sang `b` với cùng con trỏ dữ liệu, cùng `len` và cùng `cap`. Trình tối ưu hóa có thể giữ các trường này trên thanh ghi mà không bắt buộc phải phát sinh một lệnh chép 24 byte vật lý ở bộ nhớ. Biến `b` sở hữu một bản sao giá trị riêng biệt về cửa sổ nhìn (`len` và `cap`), nhưng con trỏ bên trong vẫn dẫn về cùng mảng nền ban đầu.



Hình 8 — Phép gán `b := a` sao chép giá trị lát cắt, bao gồm con trỏ dữ liệu, `len` và `cap`. Cả hai tiêu đề vẫn trỏ tới cùng mảng bên dưới, nên `b[0]` làm thay đổi phần tử mà `a` cũng quan sát được.

Vì vậy hai câu sau cùng đúng, và không mâu thuẫn:

Bảng 1 — Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Câu nói	Đúng ở đâu?
Assignment copy value.	`b` nhận một slice descriptor riêng: đổi `len` của `b` bằng reslice không tự đổi `len` của `a`.
Hai slice có thể cùng đổi dữ liệu.	Slice descriptor cùng tham chiếu một backing array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

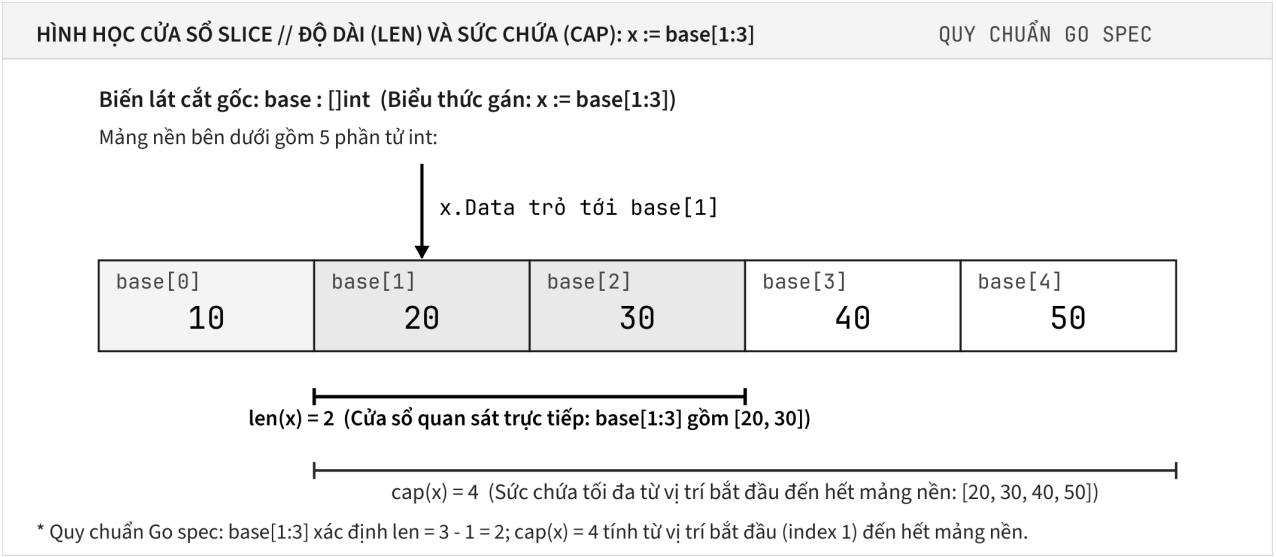
Cửa sổ, length và capacity

Đặt một slice lên một slice khác để thấy descriptor thực sự đang mô tả gì.

```
base := []int{10, 20, 30, 40, 50}
x := base[1:3]

fmt.Println(x)      // [20 30]
fmt.Println(len(x)) // 2
fmt.Println(cap(x)) // 4
```

`x` bắt đầu ở phần tử 20 của `base` và kết thúc trước 40. Vì thế length là 2. Capacity đếm số phần tử từ điểm bắt đầu của x đến cuối backing array mà x hiện có thể mở rộng tới; ở đây là 20, 30, 40, 50, nên là 4.



Hình 9 — `x` nhìn thấy hai phần tử đầu của cửa sổ. `cap(x)` không biểu thị `x` đang có bốn phần tử; nó chỉ ra rằng `x` có thể được cắt lát lại (`reslice`) hoặc nối thêm (`append`) trong phạm vi mảng bên dưới (**backing array**) hiện tại.

Hãy tự dự đoán trước khi chạy:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[1:3]
a[1] = 99

fmt.Println(base)
fmt.Println(a)
fmt.Println(b)
```

Kết quả là [1 99 3 4], [1 99], [99 3]. `a[1]` và `b[0]` đều chỉ vào phần tử `base[1]`. Hình vẽ không thay thế được việc lần index: nó chỉ cho biết vì sao có thể có một phần tử chung để lần.

append là một phép thử về capacity

Hàm `append` trả về lát cắt kết quả. Đặc tả Go phân biệt hai trường hợp: nếu dung lượng (**capacity**) hiện tại đủ chỗ cho các phần tử được thêm vào, `append` sẽ tái sử dụng mảng bên dưới (**underlying array**); nếu không đủ chỗ, `append` sẽ cấp phát một mảng bên dưới mới đủ lớn cho kết quả. Vì lát cắt trả về có thể thay đổi độ dài hoặc trỏ sang mảng nền mới, cách viết thông thường luôn là `s = append(s, value)`. Quy tắc tái sử dụng mảng nền là ngữ nghĩa cốt lõi của ngôn ngữ; trong khi cách thức tính toán dung lượng mới khi phải mở rộng mảng là chi tiết triển khai nội bộ.

```
s := make([]int, 2, 4)
s[0], s[1] = 10, 20
old := s

s = append(s, 30)
s[0] = 99

fmt.Println(old) // [99 20]
fmt.Println(s)   // [99 20 30]
```

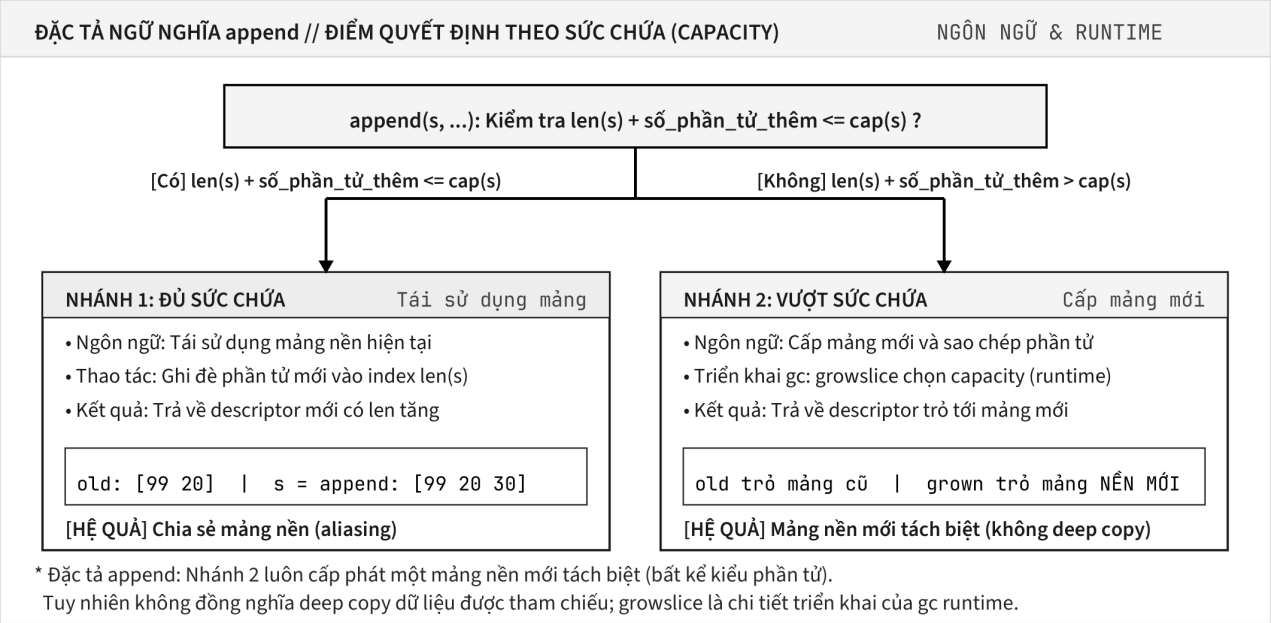
Ở đây `s` vẫn còn dung lượng dự phòng. Hai biến `old` và `s` có độ dài khác nhau, nhưng vẫn cùng trỏ vào hai phần tử đầu tiên của mảng nền (**backing array**). Không bao giờ dựa vào chuỗi dung lượng tình cờ đo được để thiết kế

quyền sở hữu. Trong Go 1.27.1, hàm `nextslicecap` tại `runtime/slice.go` tính toán dung lượng mới dựa trên cả độ dài cần có và dung lượng cũ; nếu độ dài mới vượt quá hai lần dung lượng cũ, nó có thể chọn thẳng độ dài mới. Nhánh thông thường sẽ tăng gấp đôi khi dưới ngưỡng 256 phần tử, rồi chuyển dần về mức tăng thoải hơn; hàm `growslice` còn tính toán kích thước phần tử và làm tròn kích thước cấp phát bộ nhớ. Đây là thuật toán triển khai, không phải quy tắc bất biến để dự đoán mọi lời gọi `append` hay vội vã kết luận dữ liệu nhất thiết phải nằm trên heap.

Để chủ động phòng vệ, Go cung cấp cú pháp lát cắt ba chỉ số đầy đủ `s[low:high:max]` (Full Slice Expression). Cú pháp này giới hạn sức chứa của lát cắt mới ở mức `max - low`. Khi ta đặt `max = high` (như `s[:n:n]`), capacity của lát cắt mới bị giới hạn đúng bằng độ dài nhìn thấy; thao tác `append` thêm phần tử vào lát cắt này sẽ không thể dùng tiếp phần capacity nằm ngoài phạm vi quan sát của lát cắt cũ, loại bỏ nguy cơ vô tình ghi đè lên các phần tử phía sau của mảng nền chung:

```
old := []int{10, 20}
s := old[:2:2]
grown := append(s, 30)
grown[0] = 99

fmt.Println(old)   // [10 20]
fmt.Println(grown) // [99 20 30]
```



Hình 10 — Hình mô tả hai nhánh ngữ nghĩa mà mã nguồn phải đáp ứng. Đừng suy đoán việc cấp phát bộ nhớ từ một dung lượng tình cờ; nếu mã nguồn đòi hỏi một kết quả độc lập, hãy chủ động sao chép dữ liệu.

Một aliasing bug có người dùng thật

Một function “chỉ chuẩn bị preview” đôi khi phá dữ liệu của caller.

```
func redactPreview(fields []string) {
    fields[0] = "***"
}

requestFields := []string{"api-key", "region"}
redactPreview(requestFields)
fmt.Println(requestFields) // [*** region]
```

Tham số `fields` tiếp nhận một giá trị lát cắt mới, nhưng tiêu đề lát cắt ấy vẫn tham chiếu tới vùng nhớ của `requestFields`. Việc sửa đổi phần tử là thao tác làm biến đổi vùng lưu trữ dùng chung. Nếu mục tiêu thực sự là tạo ra bản xem trước độc lập, hãy sao chép các phần tử sang một lát cắt mới trước khi chỉnh sửa:

```
func redactedCopy(fields []string) []string {
    preview := make([]string, len(fields))
    copy(preview, fields)
    if len(preview) > 0 {
        preview[0] = "***"
    }
    return preview
}
```

Hàm `copy` sao chép tối đa số phần tử vừa với đích và trả về số phần tử đã chép. Với `[]string` này, `make` tạo vùng chứa phần tử riêng; gán lại `preview[i]` không sửa `fields[i]`. Đầu vào `nil` hoặc rỗng trả về slice có độ dài 0; contract không yêu cầu giữ nguyên tính `nil`. Kiểm tra độ dài trước khi truy cập phần tử 0 tránh panic. Chi phí cấp phát và sao chép là cái giá của quyền sửa dữ liệu riêng, không phải lý do để bỏ contract.

Bài thực hành: Phát hiện việc sửa vùng nhớ dùng chung

Mở thư mục `labs/part2-values` trong trình soạn thảo và chạy riêng ca kiểm thử `TestRedactedCopyDoesNotMutateInput` trước khi sửa mã nguồn. Sau đó tạm thay phần tạo `preview` bằng phép gán `preview := fields`. Đừng suy đoán cảm tính bằng mắt: hãy chạy lại kiểm thử để thấy công cụ báo lỗi dữ liệu đầu vào đã bị biến đổi ngoài ý muốn. Khôi phục lại mã nguồn, xóa phần thân của `redactedCopy`, rồi tự hoàn thiện lại hàm từ giao ước của ca kiểm thử mà không nhìn vào bản cài đặt cũ. Đây là bước thực hành nhỏ nhưng bắt buộc mô hình tư duy "sao chép giá trị lát cắt vẫn có thể chia sẻ vùng nhớ nền" phải trải qua một thất bại thực tế.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Phép gán `preview := fields` chỉ sao chép tiêu đề lát cắt (**slice header / descriptor**), nên thao tác `preview[0]` vẫn chỉnh sửa phần tử chung với `requestFields`. Bản sửa đúng đắn bắt buộc phải khởi tạo một mảng nền mới, chẳng hạn kết hợp giữa `make` và `copy`. Ca kiểm thử phải duy trì đồng thời cả hai khẳng định kiểm thử: bản xem trước đã được che dấu thông tin và dữ liệu đầu vào không bị biến đổi; việc bỏ qua khẳng định thứ hai sẽ làm đánh mất hoàn toàn giao ước về quyền sở hữu dữ liệu.

Đừng chỉ thử đầu vào có hai phần tử. `TestRedactedCopyEmptyInput` kiểm tra `nil` và slice rỗng; `TestRedactedCopyOwnsElements` sửa phần tử từ cả hai phía để kiểm tra tính độc lập. Một bản sửa chỉ che phần tử đầu nhưng làm mất phần còn lại cũng chưa đáp ứng contract.

Một cách tránh copy dễ bị hiểu nhầm là giới hạn capacity. Hãy dự đoán hai dòng in sau trước khi chạy `TestCapacityLimitDoesNotIsolateExistingElements`:

```
base := []int{10, 20, 30}
view := base[:2:2]
view[0] = 99
fmt.Println(base) // [99 20 30]: phần tử cũ vẫn dùng chung

grown := append(view, 40)
grown[0] = 77
fmt.Println(base) // [99 20 30]: append đã cần mảng mới
```

Giới hạn capacity ngăn lần `append` này ghi vào phần đuôi của `base`; nó không cô lập các phần tử đã nằm trong cửa sổ. Ngược lại, `copy` cô lập vùng chứa phần tử nhưng không sao chép đệ quy đối tượng được trỏ tới. `TestPointerElementsRemainSharedAfterCopy` chép một `[]*int`: sửa `*clone[0]` vẫn đổi số nguyên chung, còn gán một con trỏ khác vào `clone[0]` chỉ đổi phần tử của bản sao. Quyền sở hữu phải được xét ở đúng tầng dữ liệu, không suy ra từ chữ "copy".

Một cửa sổ nhỏ giữ cả căn phòng

Tính độc lập khi chỉnh sửa dữ liệu chưa phải là câu hỏi cuối cùng về quyền sở hữu (**ownership**). Giả sử dịch vụ nhận một vùng đệm (**buffer**) lớn 4 MB, trích xuất 16 byte đầu làm mã định danh rồi lưu giữ trong nhật ký vận hành. Vùng đệm gốc không còn cần cho nghiệp vụ nữa.

> **Dừng để dự đoán:** Liệu thao tác giới hạn cả chiều dài (**length**) lẫn dung lượng (**capacity**) xuống 16 có giúp bộ thu gom rác (**garbage collector - GC**) giải phóng phần vùng nhớ còn lại không?

```
buf := make([]byte, 4<<20)
preview := buf[:16:16]
fmt.Println(len(preview), cap(preview)) // 16 16
```

Cần phân biệt rõ ba câu hỏi kỹ thuật. Độ dài `len` cho biết số lượng phần tử hiện hữu trong góc nhìn; dung lượng `cap` giới hạn phạm vi góc nhìn có thể mở rộng thông qua thao tác cắt lại lát cắt (**reslice**) hoặc tái sử dụng vùng lưu trữ khi gọi `append`. Trong khi đó, bộ thu gom rác GC chỉ quan tâm đối tượng nào trên bộ nhớ heap còn vết tham chiếu. Lát cắt `preview` vẫn tham chiếu tới toàn bộ mảng nền (**underlying array**) của `buf`. Đối với vùng nhớ được cấp phát lớn trong thí nghiệm này, việc còn giữ góc nhìn đồng nghĩa với việc giữ lại toàn bộ khối bộ nhớ gốc, chứ không riêng 16 byte đầu. Biểu thức cắt ba chỉ số (**full slice expression**) chỉ khống chế quyền mở rộng dung lượng, hoàn toàn không chuyển quyền sở hữu sang một vùng cấp phát độc lập.

Ngay cả khi gán `buf = nil`, chừng nào chương trình còn sử dụng `preview`, nhánh tham chiếu tới mảng nền vẫn tồn tại nguyên vẹn. Ngược lại, việc một biến còn nằm trong phạm vi từ vựng (**lexical scope**) cũng không đảm bảo trình biên dịch (**compiler**) coi nó còn sống đến cuối khối lệnh. Khi đo đạc, chúng ta cần bảo đảm kết quả thực sự được sử dụng; bài thực hành ở Chương 10 dùng `runtime.KeepAlive` để bảo vệ mục đích thí nghiệm. Đừng suy diễn số byte heap chỉ dựa trên cú pháp `make`: vị trí cấp phát bộ nhớ và khả năng tối ưu hóa của trình biên dịch luôn đòi hỏi quan sát trên chính bộ công cụ (**toolchain**) thực tế.

Hãy mở `labs/part10-resource-retention/baseline/retention.go`, đọc hàm `Prefix`, nhưng chưa xem bản sửa lỗi. Viết ca kiểm thử với hai ràng buộc: chỉnh sửa tiền tố không được làm đổi dữ liệu đầu vào (**input**), và tiền tố không được dùng chung byte đầu với dữ liệu gốc khi độ dài khác 0. Cách cắt ba chỉ số sẽ thất bại trước cả hai yêu cầu dù `cap(prefix) == len(prefix)`. Ca kiểm thử này chứng minh hiện tượng chia sẻ bộ nhớ; chỉ có hồ sơ định lượng tài nguyên (**memory profile**) sau nhiều chu kỳ chạy mới phơi bày toàn bộ hệ quả rò rỉ dung lượng.

Nếu contract là trả về dữ liệu độc lập, lời giải trực tiếp đã nằm trong thao tác copy mà ta vừa học:

```
func Prefix(buf []byte, n int) []byte {
    out := make([]byte, n)
    copy(out, buf[:n])
    return out
}
```

Tiền điều kiện ở đây là `0 <= n && n <= len(buf)`. Với tiền tố 16 byte, hàm giữ 16 phần tử byte trong vùng lưu trữ riêng biệt thay vì kéo dài tuổi thọ của toàn bộ vùng đệm đầu vào. Đây là hành vi sao chép các byte dữ liệu, không phải lời hứa giải phóng tức thì vùng nhớ cũ: nếu một biệt danh (**alias**) khác vẫn giữ tham chiếu tới vùng đệm, mảng nền đó vẫn tồn tại cho tới khi GC hoạt động. Chi phí cấp phát bộ nhớ cũng thay đổi tương ứng, do đó chỉ nên sao chép tại các ranh giới kiến trúc thật sự đòi hỏi quyền sở hữu độc lập, tránh sao chép tùy tiện để che giấu các khiếm khuyết thiết kế.

Hàm `bytes.Clone(buf[:n])` là lựa chọn chuẩn tắc khi giao diện làm việc với lát cắt byte. Trong khi đó, `slices.Clone` áp dụng cho lát cắt tổng quát nhưng là thao tác sao chép nông (**shallow copy**): việc nhân bản một `[]*Record` chỉ tạo vùng nhớ mới cho danh sách con trỏ (**pointers**), chứ không hề sao chép các cấu trúc `Record` mà con trỏ tham chiếu tới. Cả hai hàm đều có giao ước rõ ràng về bản sao; không nên dùng dung lượng quan sát được làm bằng chứng cho kích thước cấp phát thực tế. Cách viết kết hợp `make` và `copy` trong bài thực hành giữ cho mô hình tư duy tập trung vào quyền sở hữu mà không bị phân tâm bởi cú pháp tổng quát (**generics**).

Một góc nhìn (**view**) lớn có vòng đời dài hoàn toàn hợp lệ nếu bên gọi (**caller**) thực sự cần toàn bộ dữ liệu. Tình huống lỗi nguy hiểm là khi vòng đời thực tế vượt xa nhu cầu nghiệp vụ: hệ thống chỉ cần lưu một chuỗi mã ngắn nhưng bản hiện thực lại vô tình giữ trọn cả vùng đệm megabyte. Hiện tượng này được gọi là lưu giữ bộ nhớ ngoài ý muốn (**memory retention**). Bộ thu gom rác GC không gặp lỗi; chính giao ước về thời gian tồn tại của dữ liệu đã bị bản hiện thực nới rộng ngoài tầm kiểm soát.

Chuỗi ký tự: byte trước, văn bản sau (string)

Kiểu chuỗi (**string**) trong Go cũng là một giá trị, nhưng tuân theo quy tắc đặc thù: chuỗi là một chuỗi byte bất biến (**immutable sequence of bytes**). Hàm `len` đếm số lượng byte; chỉ số truy cập trả về giá trị từng byte. Văn bản thông thường được mã hóa bằng chuẩn UTF-8, song Go không bắt buộc mọi chuỗi đều phải chứa dữ liệu UTF-8 hợp lệ.

```
text := "Việt"
fmt.Println(len(text)) // 6

for index, value := range text {
    fmt.Println(index, value)
}
```

Lab chạy đoạn này với kết quả:

```
6
0 86
1 105
2 7879
5 116
```

Hai ký tự `V` và `i` mỗi ký tự chiếm đúng 1 byte. Ký tự `ệ` được mã hóa bằng 3 byte UTF-8, do đó điểm mã tiếp theo bắt đầu tại chỉ số byte 5. Vòng lặp `range` duyệt chuỗi sẽ giải mã UTF-8: giá trị thứ hai nhận được là một `rune`, dùng để biểu diễn một điểm mã Unicode (**code point**). Trong Go, `byte` là biệt danh kiểu của `uint8`, còn `rune` là biệt danh của `int32`. Cần lưu ý một điểm mã vẫn không đồng nghĩa với "một ký tự đồ họa hiển thị trước mắt người dùng": một cụm ký tự hiển thị (**grapheme cluster**) có thể kết hợp từ nhiều điểm mã. Chúng ta chưa cần xử lý phân đoạn văn bản phức tạp ở đây; điều tối quan trọng là không đánh đồng hàm `len` với số lượng ký tự hiển thị.

Chuỗi ký tự không cho phép thao tác gán trực tiếp như `text[0] = 'v'`. Với các thao tác theo byte, chuyển đổi chuỗi sang `[]byte`, biến đổi nội dung lát cắt rồi khởi tạo chuỗi mới là cách tiếp cận tường minh. Đây không phải con đường duy nhất: biến đổi theo điểm mã có thể dùng `[]rune`, còn nối

ghép chuỗi tuần tự nên dùng `strings.Builder`. Hãy xác định đúng đơn vị dữ liệu trước khi lựa chọn công cụ biến đổi.

Thao tác chuyển đổi kiểu (**conversion**) giữa chuỗi và lát cắt byte phải bảo toàn trọn vẹn ngữ nghĩa giá trị (**value semantics**): biến đổi lát cắt byte tạo ra từ chuỗi không được phép làm thay đổi chuỗi ban đầu; ngược lại, chuỗi sinh ra từ lát cắt phải bất biến dù lát cắt sau đó tiếp tục bị sửa đổi. Đặc tả ngôn ngữ Go không bắt buộc mọi phép chuyển đổi đều phải kích hoạt một lần sao chép vật lý. Trình biên dịch có thể lược bỏ cấp phát bộ nhớ khi chứng minh được hành vi quan sát bên ngoài không thay đổi. Khi muốn khẳng định một biểu thức có tiêu tốn bộ nhớ heap hay không, hãy đo lường và kiểm tra chỉ dẫn tối ưu của trình biên dịch trên đúng phiên bản đang dùng thay vì suy đoán cảm tính từ cú pháp.

Con trỏ là một bài toán khác (pointer)

Sau lát cắt, con trỏ (**pointer**) thường bị ngộ nhận như một dạng "lát cắt tường minh hơn". Con trỏ là một giá trị tham chiếu trực tiếp tới một biến khác (**variable**); trong khi lát cắt là một tiêu đề mô tả một phân đoạn trên mảng nền. Cả hai đều có thể dẫn tới hành vi chia sẻ dữ liệu, nhưng phục vụ hai bài toán kiến trúc hoàn toàn khác nhau.

```
count := 10
p := &count
*p = 99
fmt.Println(count) // 99
```

Cú pháp `&count` tạo ra một con trỏ tham chiếu tới biến `count`, còn `*p` dùng để giải tham chiếu (**dereference**) và thao tác trực tiếp trên biến đó. Con trỏ sẽ được phân tích toàn diện trong chương riêng khi chúng ta thiết kế giao diện lập trình xoay quanh việc biến đổi trạng thái trực tiếp và xử lý giá trị `nil`. Trước khi đi tới đó, hãy ghi nhớ mô hình tư duy cốt lõi: luôn xác định rõ giá trị nào đang được sao chép, và giá trị đó có tham chiếu tới vùng nhớ hoặc biến nào khác hay không.

Nếu câu trả lời là "không", vẫn cần xét các thành phần bên trong giá trị được sao chép: một mảng chứa slice hay con trỏ vẫn có thể chia sẻ dữ liệu được

tham chiếu. Phép gán sao chép giá trị, không mặc nhiên sao chép sâu toàn bộ dữ liệu mà nó dẫn tới. Với `int` hoặc mảng chỉ chứa các giá trị như `int`, sửa bản sao không đổi bản gốc. Khi nhiều giá trị cùng dẫn tới dữ liệu có thể thay đổi, đó là chia sẻ vùng nhớ (**aliasing**); tính đúng đắn phụ thuộc vào quyền sở hữu và quyền thay đổi dữ liệu được quy định rõ ràng.

Quyết định thiết kế tại ranh giới API

Khi một hàm tiếp nhận hoặc trả về lát cắt, câu hỏi thiết kế không phải là "Go có sao chép hay không?" mà là: thực thể nào được quyền thay đổi vùng lưu trữ này, và trong khoảng thời gian nào? Một hàm lọc dữ liệu có thể trả về một góc nhìn (**view**) để triệt tiêu chi phí cấp phát bộ nhớ; khi đó bên gọi (**caller**) phải được thông báo rằng góc nhìn này vẫn dùng chung vùng đệm dữ liệu. Một hàm tạo phản hồi (**response**) hoặc bản xem trước (**preview**) cần trả về một khối dữ liệu độc lập, bởi bên gọi không có lý do để dự đoán một thay đổi tiếp theo lại làm biến dạng dữ liệu đầu vào.

Không có lựa chọn nào là miễn phí. Thao tác sao chép tiêu tốn tài nguyên cấp phát bộ nhớ và thời gian tính toán; chia sẻ vùng nhớ tiết kiệm chi phí nhưng đòi hỏi khế ước về quyền sở hữu phải nghiêm ngặt. Trước khi tối ưu hóa, hãy thiết lập ca kiểm thử khẳng định rõ hành vi biến đổi dữ liệu mà giao diện API cam kết. Sau đó, khi điều tra một lỗi bất thường, hãy tuần tự lần theo ba câu hỏi: giá trị nào được gán, giá trị đó tham chiếu tới vùng lưu trữ nào, và những luồng thực thi nào còn có thể chạm tới vùng nhớ ấy. Đó là phương pháp biến hiện tượng "lát cắt khó lường" thành một quy trình điều tra kỹ thuật có thể kiểm chứng được.

Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi

`opsprobe` sẽ dần trở thành một CLI kiểm tra trạng thái dependency. Trước khi nó gọi HTTP hay đọc file config, nó đã có một bài toán nhỏ hơn: làm sao để biểu diễn trạng thái của một service mà người đọc không phải tự ghép bốn biến rời rạc trong đầu?

```
serviceName := "billing"
port := 8080
healthy := true
retries := 0
```

Bốn biến này đi cùng nhau. Chúng mô tả một dịch vụ, nhưng trình biên dịch chưa thể nhận biết mối liên kết đó. Càng nhiều hàm nhận các biến này một cách rời rạc, lập trình viên càng dễ đổi nhầm thứ tự, quên truyền một trường dữ liệu (**field**), hoặc cập nhật trên một bản sao cục bộ mà lầm tưởng đã thay đổi trạng thái chung. Chúng ta sẽ từng bước tái cấu trúc mô hình này. Mỗi lần thay đổi, ta dừng lại ở cùng một câu hỏi cốt lõi: hàm tiếp nhận giá trị nào, và giá trị ấy mở đường tới vùng dữ liệu nào?

Bug đầu tiên: bonus đi đâu mất?

Hãy tạm quên service. Một ví dụ nhỏ hơn cho thấy vấn đề rõ nhất:

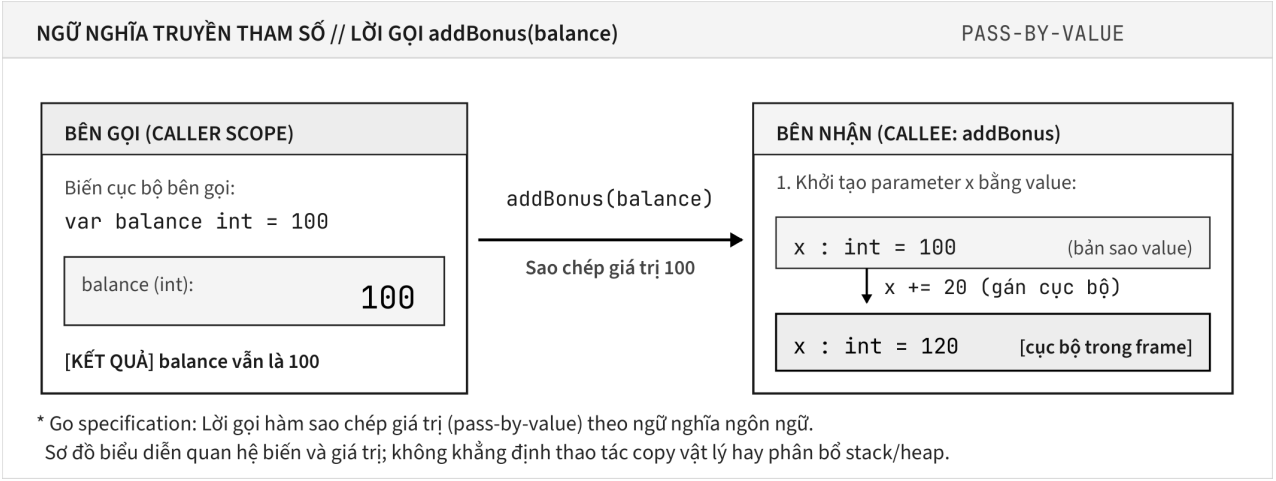
```
balance := 100

func addBonus(x int) {
    x += 20
}

addBonus(balance)
fmt.Println(balance) // 100
```

Biến `balance` không đổi vì lời gọi hàm trong Go tuân thủ ngữ nghĩa truyền giá trị (**pass-by-value**). Khi gọi `addBonus(balance)`, hàm tiếp nhận một tham số `x` mang giá trị bản sao `100`. Phép tính `x += 20` chỉ thay đổi giá trị

của biến cục bộ `x`; nó không thể tác động ngược lại biến `balance` ở phía bên gọi. Đây không phải là ngoại lệ của riêng kiểu `int`, mà là nền tảng chi phối mọi chữ ký hàm xuyên suốt chương này.



Hình 11 — Lời gọi `addBonus(balance)` sao chép giá trị 100 vào tham số `x`. Sơ đồ biểu diễn ngữ nghĩa của biến và giá trị, không khẳng định vị trí vật lý của chúng trong bộ nhớ RAM.

Nếu hàm cần tạo một giá trị mới, cách tiếp cận trên rất rõ ràng: giá trị đầu vào được tiếp nhận, kết quả mới trả ra qua lệnh `return`. Ví dụ, `withBonus(balance)` có thể trả về `120`, và bên gọi tự quyết định việc có gán kết quả đó trở lại `balance` hay không. Tuy nhiên, một số thao tác có mục đích trực tiếp biến đổi biến gốc do bên gọi cung cấp. Khi đó, mã nguồn cần thể hiện rõ ràng con đường dẫn tới biến đó.

Con trỏ: con đường dẫn tới một biến

Để làm chủ cơ chế quản lý bộ nhớ và ranh giới biến đổi dữ liệu trong Go, kỹ sư cần phân biệt chính xác bốn khái niệm nền tảng sau:

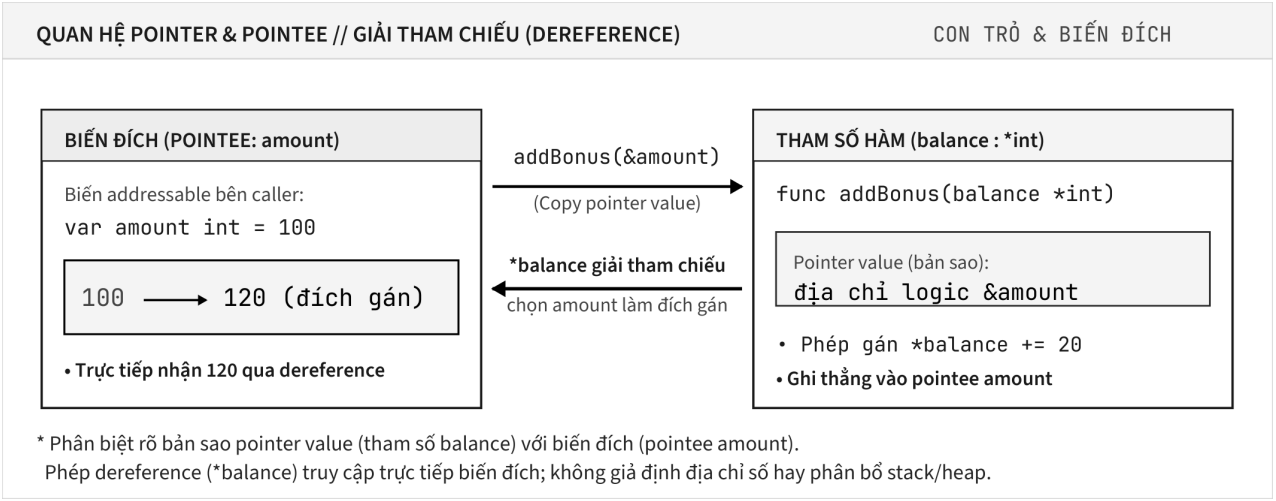
Khái niệm trong Go Semantics	Định nghĩa kỹ thuật chuẩn xác	Ví dụ tương ứng
Variable	Vị trí lưu trữ dữ liệu được cấp phát, có thể đọc hoặc gán giá trị mới.	Biến `balance`
Value	Dữ liệu thuộc một kiểu; chưa cần giả định representation vật lý.	Giá trị số nguyên `100`
Address	Kết quả của phép toán trích xuất địa chỉ ô nhớ bằng toán tử &.	Biểu thức `&balance`
Pointer value	Giá trị mang kiểu con trỏ (như `*int`), chứa địa chỉ của một biến hoặc `nil`.	Biến con trỏ `p`
Pointee	Biến đích tại ô nhớ mà con trỏ đang trỏ tới.	Ô nhớ chứa `balance`

Ta không cần suy đoán biến đó nằm cố định trên vùng nhớ ngăn xếp (**stack**) hay vùng nhớ động (**heap**). Đó là chi tiết hiện thực của trình biên dịch có thể thay đổi qua từng phiên bản. Điều cốt lõi khi đọc mã nguồn là mối quan hệ ngôn ngữ: giá trị con trỏ có đang trỏ tới một biến hợp lệ hay không.

```
func addBonus(balance *int) {
    *balance += 20
}

amount := 100
addBonus(&amount)
fmt.Println(amount) // 120
```

Ở phía bên gọi, `&amount` trích xuất con trỏ dẫn tới biến `amount`. Tham số `balance` vẫn là một giá trị được sao chép vào hàm, chỉ có điều giá trị sao chép đó là một địa chỉ con trỏ. Khi biểu thức `*balance += 20` thực thi, thao tác giải tham chiếu (**dereference**) `*balance` chọn chính biến `amount` làm đích gán dữ liệu. Bản sao của con trỏ không hề tạo ra bản sao của biến đích (**pointee**).



Hình 12 — Tham số `balance` trong hàm là giá trị con trỏ được truyền theo cơ chế sao chép giá trị. Thao tác `*balance` truy cập biến đích (**pointee**); sơ đồ cố ý không dùng địa chỉ số hay phân chia ngăn xếp/vùng nhớ heap để giữ đúng mô hình tinh thần ở tầng ngữ nghĩa.

Phép lấy địa chỉ & không áp dụng được cho mọi biểu thức. Thông thường, toán hạng phải lấy được địa chỉ: một biến, biểu thức giải tham chiếu con trỏ, phần tử lát cắt, trường của struct lấy được địa chỉ hoặc phần tử mảng lấy được địa chỉ. Go có ngoại lệ cho biểu thức tạo giá trị phức hợp viết trực tiếp (**composite literal**), như `&Service{Name: "billing"}`. Đây không phải hằng số: biểu thức ấy tạo một biến mới chứa giá trị `Service` và trả con trỏ tới biến đó. Ta có thể sửa các trường qua con trỏ. Ngược lại, `&100` và `&services["billing"]` không hợp lệ. Khả năng gán và khả năng lấy địa chỉ không đồng nhất: phần tử map có thể là đích của phép gán nhưng không thể là toán hạng của &.

Giá trị nil là thiếu biến đích, không phải "con trỏ rỗng vô hại"

Giá trị mặc định (**zero value**) của kiểu con trỏ là `nil`. Giá trị `nil` không trỏ tới bất kỳ biến nào, do đó thao tác giải tham chiếu trên nó sẽ lập tức gây hoảng loạn khi chạy (`panic`).

```
var bonus *int

if bonus == nil {
    fmt.Println("chưa có bonus")
    return
}
*bonus += 20
```

Thao tác giải tham chiếu `*bonus` đòi hỏi một biến đích để làm nơi gán giá trị; giá trị `nil` không có biến đích nào, do đó thao tác giải tham chiếu sẽ gây hoảng loạn (`panic`). Việc kiểm tra `nil` là một phần trong giao ước của hàm: hàm này có chấp nhận tham số rỗng hay bên gọi bắt buộc phải luôn truyền vào một con trỏ hợp lệ? Không nên thêm các phép kiểm tra `nil` một cách máy móc theo quán tính. Hãy xác lập rõ ràng và thể hiện điều đó qua giao diện lập trình, ca kiểm thử hoặc mã lỗi trả về khi có lý do chính đáng.

Gom trạng thái thành một giá trị có tên

Quay lại bốn biến rời rạc. Một struct gom chúng thành một value mà domain có thể gọi tên:

```
type Service struct {
    Name    string
    Port    int
    Healthy bool
    Retries int
}

billing := Service{
    Name:    "billing",
    Port:    8080,
    Healthy: true,
}
```

Cú pháp `type Service struct { ... }` định nghĩa kiểu `Service` với các trường đã nêu. Biểu thức `Service{...}` tạo một giá trị struct; trường `Retries` không được liệt kê nhận giá trị mặc định `0`. Go không cho khai báo một struct làm hằng bằng `const`. Giá trị mặc định cũng không thay thế kiểm tra tính hợp lệ: `Service{}` hợp lệ về kiểu nhưng chưa chắc hợp lệ cho dịch vụ thật. Trong nghiệp vụ này, cổng `0` có thể mang nghĩa “chưa được cấu hình”.

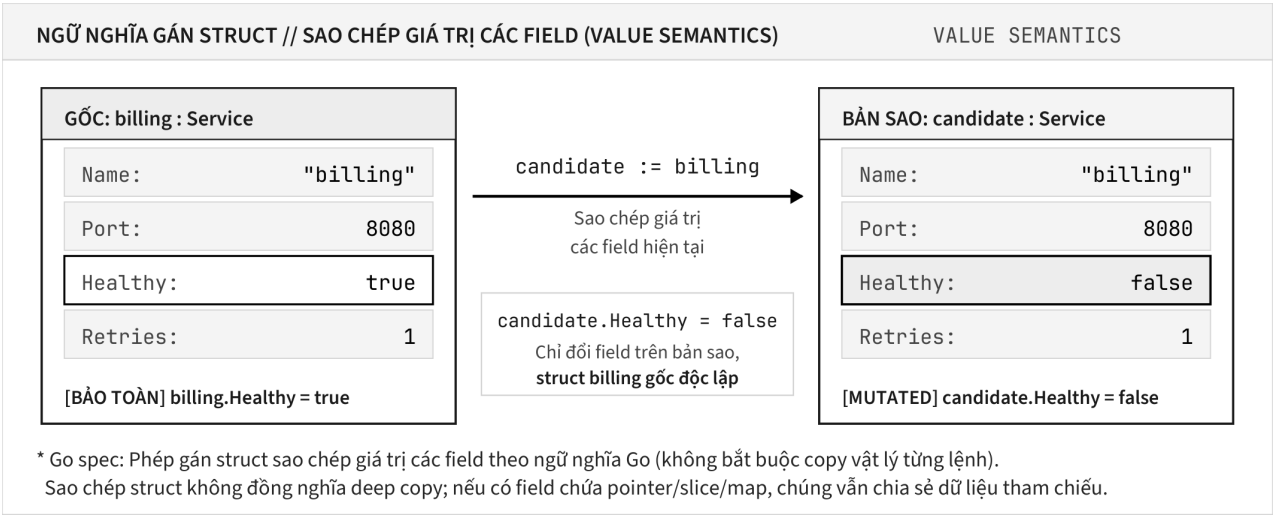
Cấu trúc (**struct**) không phải là một lớp đối tượng thiếu phương thức. Trước hết, nó là một giá trị phức hợp gồm nhiều thành phần mang chung một ý nghĩa nghiệp vụ. Phép truy xuất trường chỉ đơn thuần đọc hoặc gán một phần giá trị bên trong cấu trúc:

```
billing.Retries++
fmt.Println(billing.Name, billing.Retries) // billing 1
```

Một struct cũng được copy khi gán hoặc truyền làm argument. Với các field trong `Service` hiện tại, copy tạo các field độc lập:

```
candidate := billing
candidate.Healthy = false

fmt.Println(billing.Healthy)   // true
fmt.Println(candidate.Healthy) // false
```



Hình 13 — Phép gán sao chép giá trị cấu trúc hiện tại theo từng trường dữ liệu. Nếu một trường về sau là lát cắt, bảng băm hoặc con trỏ, cấu trúc vẫn được sao chép nhưng trường đó có thể mang theo một tham chiếu chia sẻ; khi ấy lập trình viên phải phân tích tiếp giá trị của trường thay vì nhận định phiến diện rằng “cấu trúc luôn độc lập dữ liệu”.

Cấu trúc lồng nhau (**nested struct**) phát huy hiệu quả khi một phần của miền nghiệp vụ tự thân mang một ý nghĩa hoàn chỉnh. Ví dụ, máy chủ và cổng mạng thường đi liền với nhau thay vì khai báo thành hai trường độc lập trên cùng một mặt phẳng:

```
type Endpoint struct {
    Host string
    Port int
}

type Service struct {
    Name     string
    Endpoint Endpoint
    Healthy  bool
    Retries  int
}
```

Việc nesting không tự động làm thiết kế tốt hơn. Nó có ích khi tạo ra một khái niệm đọc được, như `Endpoint`; gom chỉ để tránh thấy nhiều field thường làm model khó tìm hơn.

Khi hàm biến đổi một bản sao cục bộ

Ta muốn `opsprobe` đánh dấu service không khỏe sau một lần probe thất bại. Phiên bản đầu tiên trông rất có lý:

```
// Ví dụ mới, không dùng billing đã tăng Retries ở trên.
billing := Service{Name: "billing", Healthy: true}

func markUnhealthy(service Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(billing)
fmt.Println(billing.Healthy, billing.Retries) // true 0
```

Không có lỗi ẩn giấu nào ở đây. Biến `billing` được sao chép nguyên vẹn vào tham số `service`; hàm thay đổi trên bản sao cấu trúc đó rồi kết thúc. Phía bên gọi vẫn giữ nguyên giá trị cũ. Nếu đây là một thao tác chuyển đổi thuần túy (**pure transform**), hàm có thể trả về một đối tượng `Service` mới

qua lệnh `return`. Nếu ý định là thay đổi trực tiếp trạng thái mà bên gọi đang quản lý, chữ ký hàm bắt buộc phải thể hiện rõ điều đó:

```
func markUnhealthy(service *Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(&billing)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

Kiểu giá trị `T` và kiểu con trỏ `*T` không đồng nghĩa với khái niệm "bất biến" và "có thể thay đổi". Một biến cục bộ kiểu `Service` vẫn có thể được gán lại các trường; một tham số con trỏ hoàn toàn có thể chỉ được dùng để đọc dữ liệu. Khác biệt cốt lõi là tham số kiểu `T` tiếp nhận một bản sao giá trị độc lập, còn tham số kiểu `*T` mở ra khả năng truy cập trực tiếp biến đích (**pointee**) của bên gọi. Hãy chọn con trỏ khi hàm có nhiệm vụ biến đổi biến đích, khi việc biểu diễn trạng thái vắng mặt bằng `nil` là một phần tự nhiên của nghiệp vụ, hoặc khi thiết kế giao diện lập trình có lý do kỹ thuật rõ ràng. Hãy chọn truyền giá trị khi hàm chỉ cần ảnh chụp dữ liệu cục bộ, chuyển đổi rồi trả về bản mới, hoặc khi việc chia sẻ ô nhớ làm giao ước kỹ thuật trở nên phức tạp.

Cùng một lời gọi, bảy value khác nhau

Đến đây chúng ta có thể đọc các chữ ký hàm bằng một quy tắc nhất quán: Go luôn truyền đối số theo giá trị (**pass-by-value**). Dù vậy, "theo giá trị" chưa giải đáp trọn vẹn bài toán; người đọc mã nguồn còn phải phân tích xem giá trị đó trực tiếp chứa dữ liệu hay đang giữ một tham chiếu trỏ tới vùng nhớ khác.

Bảng 2 — Mỗi lời gọi đều copy argument value vào parameter; khả năng chia sẻ nằm trong value được copy.

Parameter	Value được copy khi gọi	Thay đổi từ function có hiện ra ở caller?
`x int`	số nguyên	Không, trừ khi return và caller gán kết quả.
`a [3]int`	toàn bộ array value	Không, các element của bản copy bị sửa riêng.
`s []int`	slice value, gồm reference tới array nền	Sửa element có thể hiện ra; reslice parameter không tự đổi len caller.
`m map[string]int`	map value, có reference tới map data	Ghi/xóa entry hiện ra qua map value cùng map data.
`p *int`	pointer value	`*p = ...` sửa pointee của caller nếu pointer hợp lệ.
`v Service`	các field của struct value	Không với các field value; xem từng field nếu chúng chứa reference.
`p *Service`	pointer value	Sửa field qua `p` sửa pointee của caller nếu pointer hợp lệ.

Bảng này không nhằm mục đích học thuộc lòng các kiểu dữ liệu. Nó cung cấp phương pháp luận để lần theo một lời gọi hàm: tham số mới tiếp nhận bản sao nào; bản sao ấy có dẫn tới vùng dữ liệu chung hay biến số chung hay không; biểu thức nào là đích gán cuối cùng? Đó cũng là lý do biến nhận giá trị (**value receiver**) và biến nhận con trỏ (**pointer receiver**) được giới thiệu song hành cùng phương thức, thay vì biến dấu hoa thị `*` thành một nghi thức cú pháp máy móc.

Bài thực hành: Sửa đúng khế ước kỹ thuật

Viết `recordFailure(service Service)` sao cho bên gọi nhận được `Healthy == false` và `Retries` tăng một. Trước khi đối chiếu đáp án, hãy chọn một trong hai khế ước: hàm trả về một cấu trúc `Service` mới, hoặc hàm nhận `*Service` và trực tiếp biến đổi giá trị được trỏ tới.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Nếu bên gọi cùng quản lý một trạng thái và hàm được đặt tên như một mệnh lệnh thực thi, con trỏ giúp thể hiện ý định kỹ thuật trực tiếp:

```
func recordFailure(service *Service) {
    service.Healthy = false
    service.Retries++
}

recordFailure(&billing)
```

Một biến thể value vẫn đúng khi API diễn đạt transform:

```
func failedCopy(service Service) Service {
    service.Healthy = false
    service.Retries++
    return service
}

billing = failedCopy(billing)
```

Hai phiên bản khác biệt ở sự phân định trách nhiệm của bên gọi. Phiên bản truyền con trỏ cho phép hàm tác động trực tiếp lên trạng thái đã tồn tại; phiên bản trả về giá trị giúp việc thay đổi trạng thái được thể hiện minh bạch qua câu lệnh gán của bên gọi. Không có "phiên bản chạy nhanh hơn" mặc định nào quan trọng hơn một giao ước kỹ thuật rõ ràng.

Bài thực hành: Lỗi logic khi mã nguồn vẫn biên dịch thành công

Trong thư mục `labs/part3-data-models`, hãy mở tệp `main_test.go` trước. Trong hàm `recordProbe`, tạm thời lược bỏ câu lệnh `registry[name] = service`, rồi chạy riêng ca kiểm thử `TestRecordProbeUpdatesExistingMapEntry`. Mã nguồn vẫn biên dịch thành công vì bản sao cấu trúc cục bộ được sửa đổi hoàn toàn hợp lệ; tuy nhiên ca kiểm thử sẽ chỉ ra rằng phần tử trong bảng băm không hề nhận giá trị mới. Hãy khôi phục lại mã bằng đúng đường biến đổi dữ liệu mà anh đã phân tích, thay vì sửa đổi kỳ vọng của bài kiểm thử. Sau đó, hãy xóa phần thân hàm `recordFailure` và tự viết lại dựa trên yêu cầu của `TestRecordFailureMutatesPointee`.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Thao tác tra cứu bảng băm trả về một giá trị `Service` độc lập. Việc chỉnh sửa biến cục bộ `service` chưa hề tác động đến dữ liệu của bảng băm; phép gán ngược trở lại `registry[name]` mới là thao tác biến đổi dữ liệu thực sự trên bảng băm. Hàm `recordFailure` cần tiếp nhận con trỏ `*Service` và chỉnh sửa trực tiếp biến đích, nhằm đáp ứng giao ước kiểm thử đòi hỏi bên gọi phải quan sát thấy sự thay đổi trên cùng một trạng thái.

Quản lý nhiều dịch vụ qua bảng băm, không tạo thêm biến rời rạc

Cấu trúc `Service` mô hình hóa một dịch vụ đơn lẻ. Khi công cụ `opsprobe` cần quản lý đồng thời nhiều dịch vụ theo tên, ta không thể để `billing`, `search`, `checkout` thành các biến số rời rạc. Kiểu bảng băm (**map**) thể hiện mối quan hệ ánh xạ giữa khóa (**key**) và giá trị (**value**):

```
services := map[string]Service{
    "billing": {
        Name:    "billing",
        Port:    8080,
        Healthy: true,
    },
}

billing, found := services["billing"]
fmt.Println(found, billing.Port) // true 8080
```

Kiểu `string` là kiểu của khóa; `Service` là kiểu của phần tử. Biến `found` là kết quả thứ hai của phép tra cứu bảng băm, thường được gọi là cơ chế **comma-ok**. Nó giúp phân biệt rõ giữa trường hợp khóa không tồn tại với trường hợp phần tử mang giá trị mặc định. Phép tra cứu nhận một kết quả vẫn hợp lệ về cú pháp: biểu thức `services["missing"].Retries` sẽ đọc trường `Retries` của cấu trúc rỗng `Service{}`, và cho kết quả `0`. Tuy nhiên con số `0` lúc đó có thể mang ý nghĩa “chưa có dịch vụ” hoặc “dịch vụ đã có nhưng số lần thử lại bằng 0”; đó là lý do bắt buộc phải sử dụng cơ chế **comma-ok** khi sự hiện diện thực tế của phần tử mang tính quyết định.

```
attempts := map[string]int{"billing": 0}

count, found := attempts["billing"]
fmt.Println(count, found) // 0 true

count, found = attempts["search"]
fmt.Println(count, found) // 0 false
```

Hằng bảng băm (**map literal**) tạo ra bảng băm đã sẵn sàng ghi dữ liệu. Lệnh `make(map[string]Service)` cũng khởi tạo một bảng băm sẵn sàng tiếp nhận phần tử. Ngược lại, giá trị mặc định của bảng băm là `nil`: thao tác đọc từ bảng băm `nil` sẽ trả về giá trị mặc định của kiểu phần tử, duyệt qua nó bằng vòng lặp `for range` sẽ lặp 0 lần mà không báo lỗi, nhưng việc gán phần tử mới vào bảng băm `nil` sẽ lập tức gây hoảng loạn (`panic`). Do đó, một bảng băm tiếp nhận từ bên gọi đòi hỏi một giao ước kỹ thuật rõ ràng tương tự như con trỏ: bên gọi có luôn khởi tạo bảng băm trước hay hàm có trách nhiệm tự khởi tạo nó trước khi ghi?

```
var services map[string]Service
fmt.Println(services["billing"].Name) // ""

// services["billing"] = billing
// panic: assignment to entry in nil map
services = make(map[string]Service)
services["billing"] = billing
```

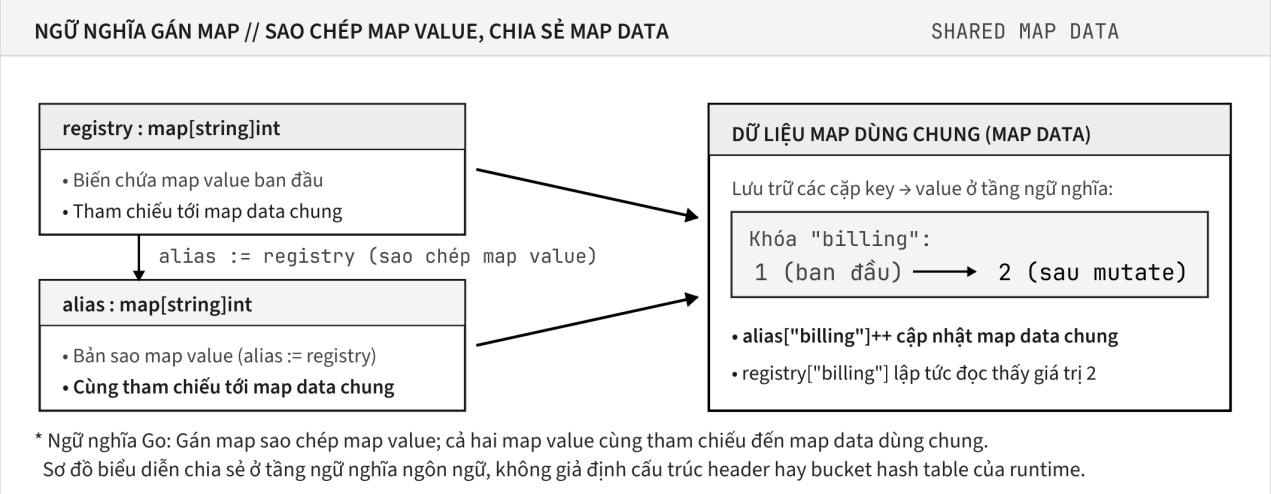
Lệnh `delete(services, "billing")` xóa một phần tử khỏi bảng băm; việc xóa một khóa không tồn tại hoàn toàn an toàn và không gây lỗi. Tuy nhiên, vòng lặp `for range` trên bảng băm không có thứ tự lặp cố định theo đặc tả ngôn ngữ. Nếu công cụ dòng lệnh cần xuất kết quả theo thứ tự ổn định, lập trình viên cần thu thập danh sách các khóa, sắp xếp chúng theo quy tắc định sẵn rồi mới in ra; tuyệt đối không xem thứ tự ngẫu nhiên trong một lần chạy là giao ước cố định của bảng băm.

Sao chép giá trị map, chia sẻ các phần tử

Ở tầng ngôn ngữ, `map[string]int` và `*int` là hai kiểu khác nhau. Phép gán và truyền đối số sao chép giá trị map; bản sao vẫn tham chiếu tới cùng dữ liệu map. Không cần giả định có một "tiêu đề map" với bố cục cố định để giải thích hành vi này. Trong đoạn chạy tuần tự dưới đây, cập nhật phần tử qua `alias` được đọc lại qua `registry`:

```
registry := map[string]int{"billing": 1}
alias := registry
alias["billing"]++

fmt.Println(registry["billing"]) // 2
```



Hình 14 — Phép gán sao chép giá trị map, không sao chép các phần tử. Hai giá trị dùng chung dữ liệu map; hình không khẳng định bố cục triển khai trong bộ nhớ.

Hãy phân biệt sửa phần tử với thay chính biến map. Nếu một hàm gán `registry = make(map[string]int)`, nó chỉ thay tham số cục bộ; biến map của bên gọi vẫn giữ giá trị cũ. Con trỏ cũng vậy: gán `p = &other` không đổi con trỏ của bên gọi, còn `*p = 7` sửa biến đích. Go vẫn truyền cả hai đối số theo giá trị, không tự tạo cơ chế truyền biến bằng tham chiếu. `TestPointerAndMapRebindingStayLocal` trong lab đặt hai thao tác cạnh nhau để anh kiểm chứng sự khác biệt.

Một đặc điểm khiến bảng băm khác biệt với lát cắt và mảng khi cập nhật giá trị cấu trúc: phần tử truy xuất qua chỉ mục bảng băm (**map index**) không thể lấy địa chỉ ô nhớ (**not addressable**). Vì vậy đoạn mã sau sẽ không thể biên dịch:

```
// services["billing"].Healthy = false // không hợp lệ
```

Biểu thức `services["billing"]` trả về một giá trị bản sao `Service` để đọc; nó không trả về một biến ô nhớ để ta có thể lấy địa chỉ của trường dữ liệu hay gán trực tiếp vào trường đó. Giải pháp theo định hướng truyền giá trị là: đọc giá trị ra biến cục bộ, chỉnh sửa trường dữ liệu, rồi gán ngược bản sao đã sửa trở lại bảng băm:

```
service := services["billing"]
service.Healthy = false
services["billing"] = service
```

Nếu miền nghiệp vụ đòi hỏi sổ đăng ký phải quản lý các con trỏ `*Service`, việc biến đổi dữ liệu qua con trỏ đích có thể trực tiếp hơn, nhưng đổi lại sổ đăng ký có nguy cơ chứa con trỏ `nil` và quyền sở hữu tài nguyên cần được quy định rõ ràng:

```
services := map[string]*Service{
    "billing": {
        Name:    "billing",
        Port:    8080,
        Healthy: true,
    },
}

services["billing"].Healthy = false
```

Đây không phải là lời khuyến khích chọn `map[string]*Service` theo quán tính. Với cấu trúc dung lượng nhỏ như `Service` hiện tại, mô hình `map[string]Service` kết hợp quy trình "lấy ra, sửa đổi, gán lại" giúp các điểm biến đổi trạng thái trở nên minh bạch và an toàn hơn. Con trỏ bên trong bảng băm chỉ thực sự đáng dùng khi giải quyết một giao ước kỹ thuật

xác thực: danh tính đối tượng dùng chung, biến đổi cần phản ánh đồng thời ở nhiều nơi, hoặc dữ liệu có dung lượng lớn đi kèm số liệu đo đạc thực tế.

Bài thực hành: Nhận diện giá trị mặc định của bảng băm

Một lần thăm dò trả về bảng băm độ trễ `latencyByService map[string]int`. Viết đoạn mã phân biệt giữa hai trường hợp: dịch vụ `billing` có độ trễ bằng `0` và `billing` chưa hề có kết quả thăm dò. Sau đó, thực hiện xóa kết quả của `billing` mà không cần kiểm tra khóa có tồn tại hay không.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Cơ chế kiểm tra hai giá trị trả về (**comma-ok**) cung cấp thông tin về sự hiện diện thực tế của phần tử; hàm tích hợp sẵn `delete` tự động xử lý an toàn khi khóa cần xóa không tồn tại trong bảng băm:

```
latency, found := latencyByService["billing"]
if !found {
    fmt.Println("chưa có kết quả")
} else {
    fmt.Println(latency)
}

delete(latencyByService, "billing")
```

Quyền hạn và trách nhiệm lưu giữ phần tử trong bộ nhớ

Sổ đăng ký vừa giúp tra cứu dịch vụ theo tên. Nhưng nếu tên không còn thuộc một tập hữu hạn mà mỗi yêu cầu gửi đến lại thêm một khóa mới, quyền chỉnh sửa bảng băm sẽ đi kèm một trách nhiệm hệ trọng: quyết định thời điểm loại bỏ dữ liệu. Hãy xét một bộ nhớ đệm (**cache**) tồn tại suốt vòng đời tiến trình. Mỗi lần trượt bộ đệm, hệ thống thêm `cache[key] = payload`; không có lệnh `delete`, không giới hạn số lượng khóa và không có thời hạn hết hạn (**TTL**). Vì bên sở hữu vẫn giữ bảng băm, các giá trị bên trong vẫn luôn trong trạng thái có thể tiếp cận được. Bộ thu gom rác (GC) không thể tự đoán rằng kết quả của yêu cầu từ ngày hôm qua đã hết giá trị nghiệp vụ để tự động xóa giúp.

Chưa thể vội vã kết luận rằng “bảng băm toàn cục luôn là rò rỉ bộ nhớ”. Một bảng cấu hình có số phần tử hữu hạn sống suốt vòng đời tiến trình là một thiết kế hoàn toàn bình thường. Một kho lưu trữ lịch sử có chính sách lưu giữ (**retention policy**), có dự báo tăng trưởng và hạn mức tài nguyên tương ứng cũng có thể tăng trưởng một cách hợp lệ. Bộ nhớ đệm hứa hẹn tiết kiệm tài nguyên nhưng nếu lưu giữ mọi khóa từng gặp mà không có điểm dừng sẽ trở thành thảm họa. Sai lầm nằm ở giao ước lưu giữ dữ liệu, chứ không nằm ở vị trí khai báo bảng băm. Rò rỉ bộ nhớ theo nghĩa vận hành là tài nguyên tiếp tục bị chiếm dụng sau khi nhu cầu sử dụng đã chấm dứt; trong Go, vùng nhớ đó thường vẫn ở trạng thái tiếp cận được (**reachable**), chứ không nhất thiết là việc mất dấu toàn bộ con trỏ như khái niệm rò rỉ trong các ngôn ngữ quản lý bộ nhớ thủ công.

Trước khi tham khảo thư mục `fixed` của bài thực hành `part10-resource-retention`, anh hãy thiết kế giao ước cho một bộ nhớ đệm đơn chủ sở hữu: tối đa hai phần tử, mỗi giá trị tối đa 4 MiB, mỗi khóa tối đa 128 byte; khi thêm khóa thứ ba thì tự động loại bỏ phần tử được thêm sớm nhất. Việc cập nhật khóa cũ không làm mới tuổi thọ của nó. Dữ liệu đầu vào vượt quá giới hạn phải bị từ chối ngay trước khi thay đổi trạng thái bộ đệm. Phía bên gọi được phép chỉnh sửa lát cắt byte của mình sau khi gọi `Put` mà không làm biến đổi bản sao bộ đệm đã lưu trữ. Đây là chiến lược vào trước ra trước (FIFO), không phải loại bỏ theo tần suất truy cập gần nhất (LRU), và hoàn toàn không có cơ chế thời hạn ngầm định.

Chính sách ấy dẫn tới những ca kiểm thử không cần đo RAM. Sau chuỗi `a, b, a, c`, bộ nhớ đệm (**cache**) phải còn `b` và `c`; cập nhật `a` không tạo phần tử (**entry**) thứ ba và không cứu nó khỏi bị loại. Gửi giá trị (**value**) quá lớn phải trả về lỗi (**error**), không được đẩy một phần tử hợp lệ ra ngoài. Chạy nhiều vòng khóa (**key**) mới, `Len()` không vượt hai. Sửa dữ liệu đầu vào (**input**) sau `Put` không được đổi giá trị đã lưu. Các ca kiểm thử nằm trong `fixed/retention_test.go`; hãy làm bản `exercise` chạy được trước khi đối chiếu phần triển khai (**implementation**).

Giới hạn từng phần tử (**entry**) riêng lẻ chưa phải giới hạn byte: hai phần tử mỗi cái hàng gigabyte vẫn quá lớn. Ngược lại, kiểm tra `len(value)` rồi lưu

thẳng một lát cắt (**slice**) nhỏ cũng chưa đủ, vì nó có thể giữ mảng bên dưới (**backing array**) lớn như Chương 2. Bản sửa sao chép các byte thực sự được nhận; nhân bản chuỗi khóa (**key string**) để một chuỗi con nhỏ không vô tình giữ vùng lưu trữ (**storage**) của chuỗi lớn. Với chính sách bài thực hành, tổng độ dài tải trọng (**payload**) được bộ nhớ đệm giữ không vượt `maxEntries * maxValueBytes`; khóa và siêu dữ liệu (**metadata**) có chi phí riêng. Đây không phải mức trần RSS hay lời hứa rằng bộ cấp phát (**allocator**) trả bộ nhớ ngay sau khi thu hồi (**eviction**).

Khi map mất phần tử, đường tham chiếu từ map tới giá trị cũ bị bỏ; một tham chiếu khác tới cùng dữ liệu vẫn có thể giữ nó sống. Các ô trong hàng đợi FIFO cũng phải bỏ tham chiếu sau khi dịch chuyển, thay vì chỉ giảm độ dài rồi để chuỗi cũ ở phần dung lượng chưa dùng. Lab có đúng một chủ sở hữu nên không cần mutex; nếu nhiều goroutine gọi `Put`, ta phải thiết kế đồng bộ hóa trước. Giới hạn dung lượng không làm map an toàn khi truy cập đồng thời.

Trong ca đo ở Chương 10, cùng chuỗi khóa mới làm phiên bản không giới hạn tăng từ hai lên tám phần tử, trong khi phiên bản sửa dừng lại ở hai. Bằng chứng kết thúc không phải "RAM trông đẹp hơn", mà là giao ước (**contract**) hữu hạn được kiểm thử bảo toàn đúng đắn, cộng với hồ sơ phân tích (**profile**) xác nhận tải trọng bị loại bỏ không còn bị bộ nhớ đệm giữ lại. Nếu sản phẩm thật sự cần lưu tất cả khóa, hãy thay đổi yêu cầu và nơi lưu dữ liệu; đừng gọi việc mất lịch sử vì bị dọn dẹp (**eviction**) là một bản sửa thành công.

Một thao tác cập nhật có trách nhiệm trong sổ đăng ký

`opsprobe` chưa cần đến mạng để hưởng lợi từ mô hình này. Một thao tác cập nhật hướng giá trị (**value-oriented update**) cho sổ đăng ký (**registry**) không cần con trỏ (**pointer**): lấy giá trị cấu trúc (**struct value**) ra khỏi map, sửa bản cục bộ, rồi gán nó trở lại phần tử trong map. Tham số map (**map parameter**) được truyền theo giá trị, nhưng giá trị map ấy vẫn dẫn tới dữ liệu map của bên gọi (**caller**); chính phép gán cuối là thao tác biến đổi có chủ ý lên sổ đăng ký chung.

```
func recordProbe(
    registry map[string]Service,
    name string,
    healthy bool,
) bool {
    service, found := registry[name]
    if !found {
        return false
    }

    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
    registry[name] = service
    return true
}
```

Signature này nói rõ ba vai: `registry` là giá trị map dùng để tra cứu (**lookup**) và gán phần tử; `name` là ảnh chụp chuỗi ký tự (**string snapshot**); `healthy` là ảnh chụp kiểu boolean (**bool snapshot**). Biến `service` là bản sao cấu trúc (**struct copy**) lấy từ phần tử. Nếu khóa (**key**) không tồn tại, hàm không tự tiện tạo ra dịch vụ mới mà trả về `false`; nếu có, nó ghi giá trị đã sửa trở lại map. Khi công cụ bắt đầu đọc cấu hình (**config**) và thăm dò điểm cuối (**probe endpoint**) ở phần sau, cách đọc chữ ký hàm này sẽ giúp ta hiểu rõ dữ liệu nào được chia sẻ, dữ liệu nào được chụp lại, và thao tác biến đổi nào có thể truy vết.

Sau chương này, một chữ ký hàm (**signature**) không còn chỉ đơn thuần là tên kiểu dữ liệu. Nó cho biết ta đang đưa vào hàm một ảnh chụp dữ liệu (**snapshot**), một cửa sổ mở tới vùng lưu trữ (**storage**) dùng chung, một giá trị map dẫn tới dữ liệu map, hay một con trỏ mở đường tới biến của bên gọi (**caller variable**). Khi cần gỡ lỗi thao tác biến đổi, hãy viết rõ trạng thái trước và sau lời gọi: biến nào tồn tại ở bên gọi, tham số nào xuất hiện ở hàm, và biểu thức nào thực sự là đích tiếp nhận phép gán.

Kiểu là ranh giới ý nghĩa, không chỉ cách chứa byte

Một cổng mạng (**port**) và một số lần thử lại đều có thể nằm trong `uint16`, nhưng đổi chỗ chúng là lỗi nghiệp vụ. Khai báo kiểu có tên đặt một ranh giới mà trình biên dịch (**compiler**) có thể kiểm tra:

```
type Port uint16
type Attempts uint16
type PortAlias = uint16

var port Port = 80
// var attempts Attempts = port // không gán trực tiếp
var raw uint16 = uint16(port)
var alias PortAlias = raw
```

`Port` và `Attempts` là hai kiểu được định nghĩa (**defined type**) khác nhau; việc có cùng kiểu nền tảng (**underlying type**) `uint16` không làm chúng có cùng danh tính (**identity**). Kiểu `PortAlias` lại là bí danh khác của chính `uint16`, không tạo ra một kiểu mới. Bí danh (**alias**) hữu ích khi tái cấu trúc giao diện lập trình (**API**) sang gói (**package**) khác mà vẫn giữ nguyên danh tính cho bên gọi (**caller**), nhưng không có tác dụng nếu mục tiêu là ngăn chặn nhầm lẫn giữa hai đại lượng nghiệp vụ. Đừng đồng nhất khả năng gán (**assignability**) với khả năng chuyển đổi (**convertibility**): chuyển đổi kiểu tường minh (**conversion**) có thể hợp lệ khi phép gán trực tiếp bị cấm; sự thỏa mãn giao diện (**interface satisfaction**) còn dựa trên tập phương thức (**method set**) chứ không chỉ hình thức biểu diễn bộ nhớ.

Quy tắc không phải "khác tên thì luôn phải chuyển đổi". Một giá trị có thể gán giữa hai kiểu có cùng kiểu nền nếu ít nhất một kiểu không có tên, chẳng hạn `type PortBox struct{ Port int }; var box PortBox; var plain struct{ Port int } = box`. Với hai kiểu được định nghĩa `Port` và `Attempts`, điều kiện ấy không thỏa. Hằng chưa định kiểu như `80` lại gán được cho `Port` khi biểu diễn được; đó là quy tắc về hằng, không phải Go tự đổi mọi biến số sang kiểu đích. Test biên dịch trong `part3-data-models` kiểm tra cả ví dụ được chấp nhận lẫn bị từ chối, bao gồm gọi method trên phần tử map và truyền `Service` vào `ProbeRecorder`.

Chuyển đổi kiểu (**conversion**) không phải là kiểm tra tính hợp lệ (**validation**). Với biến `wide := uint32(65537)`, biểu thức `Port(wide)` cho giá trị 1 do quy tắc cắt ngắn số nguyên sang kiểu có kích thước hẹp hơn. Ngược lại, cú pháp `Port(65537)` với hằng số bị trình biên dịch từ chối vì giá trị vượt quá phạm vi biểu diễn của kiểu đích. Hàm khởi tạo (**constructor**) tiếp nhận dữ liệu từ bên ngoài bắt buộc phải kiểm tra phạm vi trước khi chuyển đổi kiểu nếu giao ước (**contract**) quy định cổng hợp lệ nằm trong khoảng 1–65535. Việc đặt tên kiểu giúp phát hiện nhầm lẫn vai trò; nó không tự động ngăn chặn giá trị khởi tạo mặc định (**zero value**) hoặc giá trị nằm ngoài miền nghiệp vụ.

Để tự kiểm chứng, hãy mở `TestConstantsAndIdentity` và `testdata/incompatible` trong thư mục `labs/edition-contracts`. Ca kiểm thử biên dịch yêu cầu phép gán giữa hai kiểu được định nghĩa phải bị từ chối; không được chỉnh sửa mã mẫu (**fixture**) chỉ để dự án biên dịch thành công. Sau đó, hãy tự thiết kế hàm `NewPort(n uint32) (Port, error)` và viết ca kiểm thử cho các giá trị biên 0, 1, 65535, 65536 trước khi viết mã xử lý. Đây là bài học về giao ước tại ranh giới hệ thống, không phải bài học thuộc lòng cú pháp từ khóa `type`.

Khi hành vi thuộc về service

`recordProbe(registry, name, healthy)` đã làm đúng việc, nhưng khi `opsprobe` có thêm nhiều thao tác quanh một dịch vụ (**service**), chữ ký hàm bắt đầu lặp lại cùng một chủ thể. Một hàm cần tạo nhãn cho dịch vụ, một hàm khác ghi kết quả thăm dò (**probe**), hàm nữa kiểm tra cấu hình. Lúc này phương thức (**method**) giúp đặt hành vi cạnh kiểu dữ liệu mà nó đọc hoặc thay đổi; nó không thay đổi quy tắc ngữ nghĩa giá trị (**value semantics**) đã xây.

```
// Trích đoạn sư phạm tối giản
// (bản lab mở rộng xuất đủ %s healthy=%t retries=%d)
func (service Service) Summary() string {
    return service.Name
}

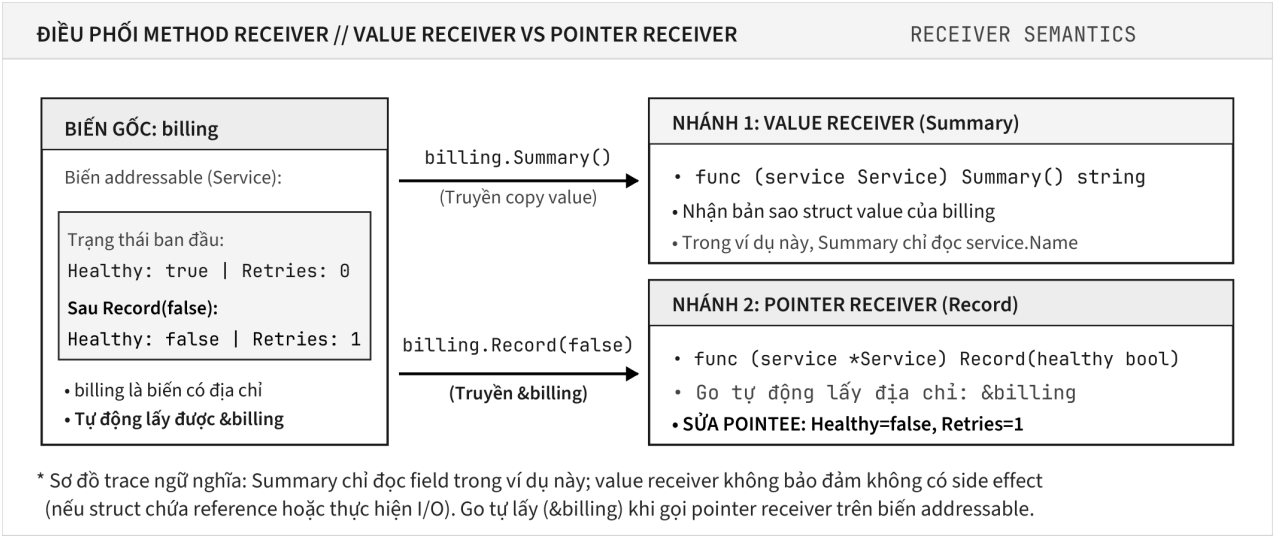
func (service *Service) Record(healthy bool) {
    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
}
```

Một phương thức (**method**) là hàm có biến nhận (**receiver**): phần tham số đặc biệt đứng trước tên phương thức. `Summary` nhận giá trị `Service`, nên biến nhận là một giá trị cấu trúc (**struct value**) được dùng để tạo kết quả. Phương thức `Record` nhận `*Service`, vì hành vi này có ý định sửa đổi đối tượng trỏ tới (**pointee**). Tên biến nhận không phải từ khóa và không phải một biến toàn cục bí mật; nó chỉ là tham số được viết ở vị trí riêng để gắn phương thức với `Service`.

```
billing := Service{Name: "billing", Healthy: true}

fmt.Println(billing.Summary()) // billing
billing.Record(false)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

Biểu thức `billing.Record(false)` hợp lệ vì `billing` là biến có thể lấy địa chỉ (**addressable variable**). Trong trường hợp này, Go cho phép cách viết ngắn gọn thay cho `(&billing).Record(false)`. Sự tiện lợi ấy không biến biến nhận giá trị thành biến nhận con trỏ: tập phương thức (**method set**) của `Service` chỉ chứa các phương thức khai báo với biến nhận `Service`; trong khi tập phương thức của `*Service` chứa cả các phương thức có biến nhận `Service` lẫn `*Service`.



Hình 15 — Phương thức vẫn dựa trên giá trị biến nhận. `Summary` dùng giá trị Service; `Record` cần `*Service` để chọn biến được trỏ tới làm đích biến đổi (**mutation**). Mũi tên là ngữ nghĩa truy vết (**trace semantics**), không mô tả cách bố trí bộ nhớ vật lý.

Phần tử được truy xuất theo chỉ số map (**map index**) không có tính chất lấy được địa chỉ (**not addressable**) theo đặc tả ngôn ngữ, nên cú pháp viết tắt lấy địa chỉ để gọi phương thức có biến nhận con trỏ (**pointer-receiver method**) không áp dụng được cho nó. Đây là giới hạn ở tầng mã nguồn, không phải kết luận rằng một địa chỉ vật lý "không ổn định" trong mọi bản cài đặt:

```
// services["billing"].Record(false) // không hợp lệ

service := services["billing"]
service.Record(false)
services["billing"] = service
```

Đoạn mã sửa dùng đúng mô hình cũ: tra cứu map để lấy giá trị cấu trúc, biến cục bộ `service` có thể lấy địa chỉ, phương thức nhận con trỏ biến đổi giá trị cục bộ, rồi bên gọi gán giá trị cấu trúc đã đổi trở lại map. Nếu mã nguồn thực tế cần nhiều nơi cùng giữ danh tính (**identity**) của một dịch vụ, kiểu `map[string]*Service` có thể hợp lý; nhưng không nên dùng con trỏ chỉ để né tránh phép gán lại khi chưa làm rõ quyền sở hữu (**ownership**).

Kết hợp thành phần trước, nhúng cấu trúc sau

Một dịch vụ có điểm cuối (**endpoint**) và trạng thái (**state**); hai phần đó là các khái niệm riêng biệt. Kết hợp thành phần (**composition**) đơn giản nhất là đặt chúng vào những trường có tên, để người đọc thấy rõ mối quan hệ chứa đựng:

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeState struct {
    Healthy bool
    Retries int
}

type Service struct {
    Name     string
    Endpoint Endpoint
    State    ProbeState
}
```

`Service` có `Endpoint` và `ProbeState`; không có quan hệ kế thừa nào được ngụ ý. Đây là cách kết hợp thành phần (**composition**) thường rõ ràng nhất khi miền nghiệp vụ (**domain**) có các phần độc lập. Nhúng cấu trúc (**embedding**) thay đổi đường dẫn truy xuất trường (**selector**), chứ không biến đối tượng ngoài thành kiểu con (**subtype**) của đối tượng trong:

```
type Named struct { Name string }
func (n Named) Label() string { return n.Name }
type Tagged struct { Named }

tag := Tagged{Named: Named{Name: "api"}}
fmt.Println(tag.Label()) // phương thức được thăng cấp
// var n Named = tag // Tagged không phải Named
```

Phương thức `Label` vẫn nhận biến nhận `Named`; lời gọi ngắn gọn đi xuyên qua trường được nhúng (**embedded field**). Hai trường nhúng có cùng tên bộ chọn ở cùng độ sâu có thể làm bộ chọn trở nên mơ hồ và bị trình biên dịch từ chối. Phương thức được thăng cấp (**promoted method**) cũng có thể khiến

kiểu bên ngoài vô tình thỏa mãn một giao diện ngoài dự kiến. Vì vậy, nhúng cấu trúc là một quyết định về giao diện lập trình công khai (**public API**), không phải mẹo tiết kiệm thao tác gõ tên trường. Khi bên gọi cần nhìn rõ quyền sở hữu, trường có tên tường minh vẫn là lựa chọn an toàn và dễ duyệt mã hơn. Những quy tắc này thuộc đặc tả Go về kiểu cấu trúc, bộ chọn và tập phương thức, hoàn toàn không phụ thuộc vào cách sắp đặt bộ nhớ vật lý của cấu trúc.

Interface xuất hiện ở nơi cần một hành vi

Một giao diện (**interface**) không phải bản thiết kế cha để mọi cấu trúc phải rập khuôn theo. Nó là một giao ước (**contract**) tinh gọn: mã nguồn bên tiêu dùng (**consumer**) tuyên bố mình cần phương thức nào, còn bất kỳ kiểu dữ liệu nào sở hữu tập phương thức (**method set**) phù hợp đều có thể dùng được. Với `opsprobe`, phần hiển thị kết quả chưa cần toàn bộ thông tin của `Service`; nó chỉ cần một dòng tóm tắt (**summary**).

```
type SummarySource interface {
    Summary() string
}

func renderSummary(source SummarySource) string {
    return "target: " + source.Summary()
}
```

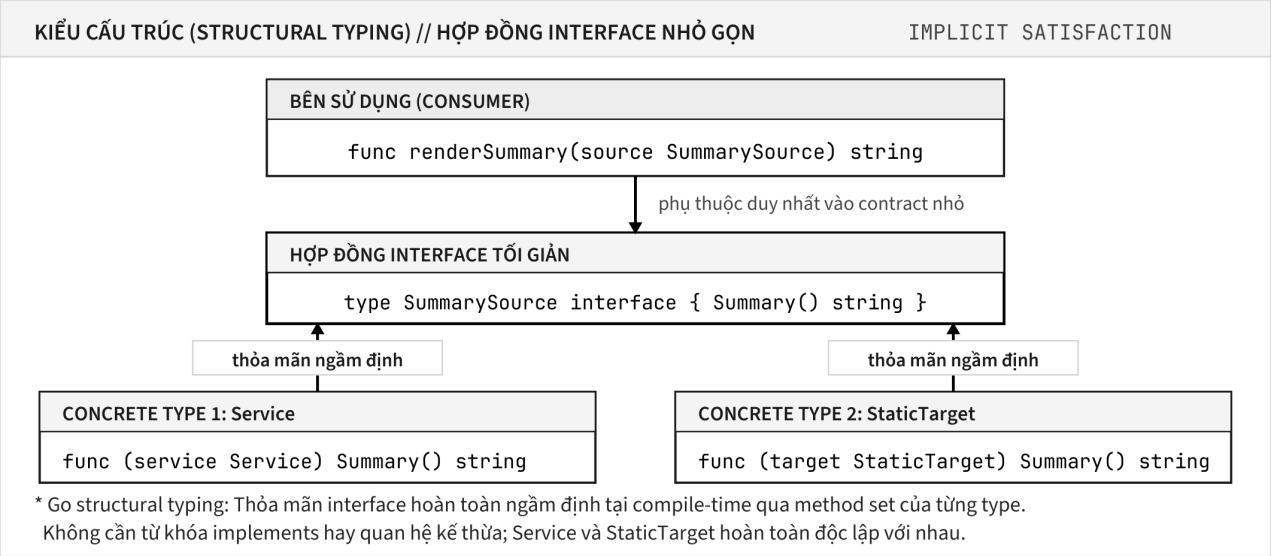
Hàm `renderSummary` không đòi hỏi bên gọi phải là `Service`, `StaticTarget`, hay một kiểu dữ liệu cụ thể nào trong tương lai. Nó chỉ phát lời gọi `Summary`. Ngôn ngữ Go không có từ khóa `implements`: phép gán hoặc đối số truyền vào hoàn toàn hợp lệ miễn là tập phương thức của kiểu thỏa mãn giao diện. Vì `Summary` có biến nhận giá trị, cả `Service` lẫn `*Service` đều sở hữu phương thức đó trong tập phương thức của mình.

```
type StaticTarget string

func (target StaticTarget) Summary() string {
    return string(target)
}

fmt.Println(renderSummary(Service{Name: "billing"}))
fmt.Println(renderSummary(
    StaticTarget("search (maintenance)"),
))
```

Kết quả là hai dòng có cùng định dạng dù hai kiểu dữ liệu không hề có tổ tiên kế thừa chung. Giao diện ở đây được đúc kết tự nhiên sau khi đã xác định rõ bên tiêu dùng và hai bên cung cấp (**provider**) có ý nghĩa thực tế. Nếu hệ thống chỉ có một `Service` duy nhất và không có ranh giới cần hoán đổi, việc tiếp nhận trực tiếp `Service` thường mạch lạc và dễ đọc hơn nhiều so với việc cố tình bổ sung giao diện chỉ vì suy đoán tương lai.



Hình 16 — `renderSummary` chỉ cần `Summary() string`. Sơ đồ cho thấy mối phụ thuộc đi từ bên tiêu dùng (**consumer**) tới giao ước (**contract**) nhỏ; `Service` và `StaticTarget` không cần khai báo bất kỳ quan hệ kế thừa hay đăng ký triển khai tường minh nào.

Biến nhận con trỏ (**pointer receiver**) thay đổi sự thỏa mãn giao diện (**interface satisfaction**) theo đúng quy tắc của tập phương thức (**method set**). Nếu giao ước đòi hỏi `Record(bool)`, giá trị `Service` sẽ không thỏa mãn vì phương thức đó chỉ thuộc về tập phương thức của `*Service`; chỉ có con trỏ `*Service` mới đáp ứng được.

```
type ProbeRecorder interface {
    Record(bool)
}

func markFailed(recorder ProbeRecorder) {
    recorder.Record(false)
}

billing := Service{Name: "billing", Healthy: true}
markFailed(&billing) // *Service thỏa ProbeRecorder

// markFailed(billing) // sai: Service thiếu Record
```

Typed nil: có kiểu động nhưng giá trị pointer là nil

Để dự đoán một giá trị giao diện (**interface**) có bằng `nil` hay không, hãy tách biệt kiểu động (**dynamic type**) khỏi giá trị động (**dynamic value**). Kiểu động là kiểu của giá trị được gán vào giao diện; giá trị động là giá trị cụ thể ấy. Giao diện `nil` chưa có cả hai thông tin này. Đây là mô hình ngữ nghĩa đủ để giải thích ví dụ dưới đây, không cần đi sâu vào địa chỉ hay bảng con trỏ hàm.

Nếu muốn đối chiếu tầng triển khai, mã nguồn Go 1.27.1 ở `runtime/runtime2.go` định nghĩa `iface` với `tab *itab` và `data unsafe.Pointer`, còn `eface` với `_type *_type` và `data`. Trên kiến trúc amd64, đó là hai trường con trỏ 64 bit. Cách giá trị cụ thể (**concrete value**) được đưa vào trường dữ liệu còn phụ thuộc vào kiểu; không nên coi nó như một đối tượng luôn được cấp phát riêng trên vùng nhớ heap. Biểu diễn này không phải giao ước của đặc tả ngôn ngữ và không cần dùng để quyết định nhánh kiểm tra `src == nil`.

Hãy dự đoán kết quả phép so sánh trước khi nhìn vào hình thức biểu diễn bộ nhớ:

```
var s *Service = nil
var src SummarySource = s

if src == nil {
    fmt.Println("Interface la nil")
} else {
    fmt.Println("Interface KHONG nil") // Ket qua thuc te
}
```

Một giá trị giao diện chỉ bằng `nil` khi chưa có cả kiểu động lẫn giá trị động. Khi gán con trỏ `s` kiểu `*Service` có giá trị `nil` vào `src`, giao diện vẫn mang kiểu động `*Service`, do đó `src == nil` trả về `false`. Trong ví dụ này, `Summary` sở hữu biến nhận giá trị `Service`: việc gọi `src.Summary()` bắt buộc phải giải tham chiếu (**dereference**) con trỏ `nil` để tạo giá trị cho biến nhận, vì vậy lỗi hoảng loạn (`panic`) xảy ra ngay trước khi bước vào thân phương thức. Với một phương thức có biến nhận con trỏ, biến nhận `nil` có thể đi thẳng vào thân phương thức; phương thức ấy chỉ an toàn nếu giao ước và bản cài đặt chủ động kiểm tra `nil`. Đừng bao giờ suy diễn từ "giao diện không `nil`" thành "lời gọi phương thức chắc chắn an toàn".

So sánh được chưa chắc so sánh an toàn

Khóa của bảng ánh xạ (**map key**) đòi hỏi kiểu có thể so sánh (**comparable**); lát cắt, map và hàm không hỗ trợ phép so sánh bằng giữa hai giá trị cùng loại, dù chúng có thể so sánh với `nil`. Mảng có thể so sánh khi từng phần tử so sánh được; cấu trúc so sánh được khi tất cả các trường đều có thể so sánh. Điều này giải thích vì sao việc bổ sung một trường lát cắt vào một cấu trúc có thể khiến giao diện lập trình dùng cấu trúc ấy làm khóa map lập tức bị lỗi biên dịch.

Kiểu giao diện cần thêm một bước suy luận. Kiểu giao diện cho phép viết toán tử ==, nhưng giá trị động thực tế bên trong có thể không thuộc loại so sánh được (**not comparable**):

```
var a any = []int{1}
var b any = []int{1}
// a == b sẽ panic: kiểu động là []int
```

Đây không phải lỗi của bộ thu gom rác (**garbage collector**). Trình biên dịch chỉ biết kiểu tĩnh `any`; khi hai kiểu động giống nhau, phép so sánh phải áp dụng trực tiếp cho giá trị động, mà lát cắt thì không hỗ trợ giao ước so sánh đó. Ca kiểm thử `TestDynamicComparabilityPanics` kiểm chứng chính xác phản ví dụ này. Nếu muốn so sánh nội dung lát cắt, hãy định nghĩa rõ ràng thế nào là "bằng nhau" rồi so sánh từng phần tử; đừng ép nó vào giao diện nhằm né tránh kiểm tra của trình biên dịch. Chương 19 sẽ quay lại giới hạn này khi thiết kế giao diện lập trình tổng quát (**generic API**) sử dụng ràng buộc `comparable`.

Một API nhỏ gọn, Đủ để Đọc bằng Ngữ nghĩa

Ở cuối mạch thiết kế này, công cụ `opsprobe` đã có thể diễn đạt ba ý định kiến trúc khác nhau một cách mạch lạc mà không cần dựa vào những danh xưng mơ hồ:

Phương thức `Summary` nhận bản sao biến nhận `Service`; với các trường giá trị hiện có, nó chỉ đọc dữ liệu để tạo chuỗi. Nếu cấu trúc được bổ sung trường con trỏ, map hoặc lát cắt, bản sao biến nhận vẫn có thể chia sẻ dữ liệu nền bên dưới. Biến nhận giá trị không thể bảo đảm mọi phương thức trong tương lai đều không gây tác dụng phụ (**side effect**).

Hàm `func (service *Service) Record(bool)` tiếp nhận biến nhận dưới dạng giá trị con trỏ, cho phép cập nhật trực tiếp biến trạng thái của bên gọi mà không cần sao chép toàn bộ cấu trúc dữ liệu.

Hàm `func renderSummary(SummarySource) string` tiếp nhận giao diện mang hành vi mà bên tiêu dùng cần. Nó gọi `Summary` thay vì đọc

trường của một cấu trúc cụ thể; đây chính là ranh giới trừu tượng cho phép thay thế bên cung cấp (**provider**) mà không cần sửa đổi mã nguồn hiển thị.

Mô hình kết hợp thành phần (**composition**) giúp dữ liệu có đường đi rõ ràng; phương thức đặt hành vi cạnh kiểu dữ liệu; giao diện xác lập ranh giới lỏng lẻo khi xuất hiện nhu cầu hoán đổi bên cung cấp. Từ nền tảng này, chương tiếp theo sẽ trang bị cho các API này một cơ chế báo cáo thất bại tường minh: lỗi kiểm tra probe không được phép chỉ âm thầm đổi trường `Healthy` rồi biến mất, mà phải vượt qua ranh giới hàm với đầy đủ ngữ cảnh để bên gọi chủ động ra quyết định xử lý.

Chương 4 — Ranh giới xử lý lỗi: để bên gọi quyết định

Công cụ `opsprobe` vừa bổ sung cấu trúc `Service`, trường trạng thái hoạt động (**state**) và một hàm ghi nhận kết quả thăm dò (**probe**). Phiên bản đầu tiên của hàm trả về kiểu boolean (`bool`): `true` nếu tên dịch vụ tồn tại, `false` nếu không tìm thấy. Đó là một giao diện API hợp lệ nếu bên gọi (**caller**) chỉ cần biết một mục ghi (**entry**) có được lưu lại hay không.

Nhưng hãy đặt hàm đó vào một tình huống vận hành thực tế. Màn hình giám sát báo dịch vụ `billing` không khỏe mạnh (**healthy**). Khi đó xuất hiện ít nhất hai câu hỏi kỹ thuật khác nhau: dịch vụ chưa từng được khai báo trong cấu hình, hay dịch vụ đã cấu hình nhưng thao tác thăm dò tới điểm cuối (**endpoint**) bị thất bại? Một giá trị `false` đơn thuần không mang bất kỳ nguyên nhân nào vượt qua ranh giới hàm. Bên gọi buộc phải suy đoán, hoặc đọc lại trạng thái đã bị biến đổi sau sự cố.

Chương này không bắt đầu bằng việc liệt kê các hàm trong gói `errors`. Nó bắt đầu bằng quyền quyết định kiến trúc: bên gọi cần tiếp nhận dạng thất bại nào để lựa chọn hiển thị thông báo, kích hoạt thử lại (**retry**), hay dừng tác vụ; còn nơi phát hiện lỗi cần bổ sung ngữ cảnh (**context**) nào mà không làm đánh mất định danh (**identity**) của nguyên nhân gốc.

Thất bại chưa phải là một quyết định kiến trúc

Chúng ta tách hành vi thăm dò thành một giá trị hàm (**function value**). Mã nguồn chưa mở kết nối mạng thực tế; điều đó sẽ được giải quyết ở chương về nhập xuất (I/O). Hiện tại nó thiết lập một ranh giới kiểm thử: thao tác có thể thành công hoặc trả về nguyên nhân thất bại:

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeFunc func(Endpoint) error
```

Kiểu `ProbeFunc` tiếp nhận giá trị điểm cuối và trả về giá trị `error`. Nếu không có lỗi, giá trị trả về là `nil`. Nếu xuất hiện sự cố, bên gọi nhận được một giá trị lỗi mang tính mô tả. Kiểu `error` là một giao diện (**interface**) chuẩn tắc của Go; điều trọng yếu ở đây không phải giao diện đó chỉ có một phương thức, mà là mọi thất bại đều đi ra theo đường trả về chính thức với một cấu trúc đồng nhất. Tuyệt đối không dùng `panic` để báo điểm cuối không thể kết nối, và cũng không tự ý ghi nhật ký (**log**) bên trong `ProbeFunc` rồi giả vờ thao tác thành công.

Giao diện ghi nhận kết quả của `opsprobe` chuyển từ câu hỏi "có mục dữ liệu không?" sang "thao tác thăm dò và cập nhật trạng thái đã hoàn tất thành công chưa?":

```
func applyProbe(
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error
```

Chữ ký hàm (**function signature**) khẳng định một nguyên tắc trước khi chúng ta đọc bản hiện thực (**implementation**): bên gọi không nhận về trạng thái mơ hồ hay một giá trị boolean thiếu ngữ nghĩa. Nó nhận `nil` khi thao tác hoàn tất trọn vẹn, hoặc nhận một giá trị lỗi để chủ động phân nhánh xử lý tiếp theo. Biến `registry` vẫn là giá trị bảng băm dẫn tới vùng dữ liệu dùng chung; vì bảng băm lưu trữ bản sao cấu trúc (**struct value**), hàm vẫn trích xuất phần tử, cập nhật biến cục bộ `Service`, rồi gán ngược lại bảng băm như bài học ở Chương 3.

Phân loại lỗi để quyết định, thêm ngữ cảnh để điều tra

Không phải chuỗi thông báo lỗi nào cũng là một khế ước kỹ thuật (**contract**). Một công cụ dòng lệnh (CLI) có thể muốn xử lý riêng biệt tình huống "tên dịch vụ không tồn tại trong cấu hình", nhưng tuyệt đối không nên phân tích chuỗi ký tự từ `err.Error()` để phục vụ logic điều hướng. Chúng ta khai báo một lỗi lính canh (**sentinel error**) cho các điều kiện mà bên gọi cần có hành vi ứng xử chuyên biệt:

```
var ErrUnknownService = errors.New("service is not configured")
```

Khi định danh dịch vụ không tồn tại, hàm `applyProbe` không trả về lỗi lính canh trần trụi. Nó bổ sung tên thao tác đồng thời bảo toàn định danh nguyên nhân bằng động từ bọc lỗi `%w`:

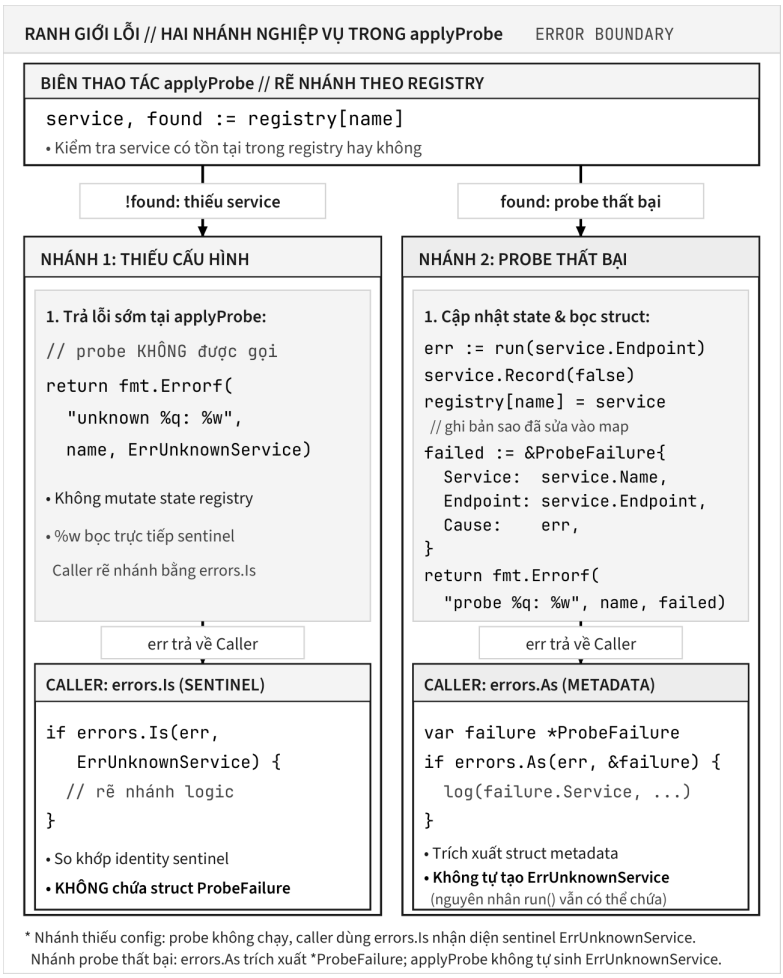
```
return fmt.Errorf("unknown %q: %w", name, ErrUnknownService)
```

`%w` không đơn thuần là thao tác nối chuỗi ký tự. Nó tạo ra một giá trị lỗi có khả năng mở gói (**unwrap**) để truy ngược về nguyên nhân gốc. Nhờ vậy bên gọi có thể kiểm tra ngữ nghĩa thông qua `errors.Is` thay vì so khớp câu chữ cảm tính:

```
if errors.Is(err, ErrUnknownService) {
    fmt.Println("thêm service vào config trước khi probe")
    return
}
```

Trong câu lệnh trên, `errors.Is` tìm qua các lớp bọc nên bên gọi không cần đếm chúng. Đổi `%w` thành `%v` trong chính `fmt.Errorf` này giữ phần mô tả nhưng bỏ đường mở gói tới `ErrUnknownService`; lỗi mới không còn nhận diện được sentinel ấy. Đây là mất liên kết với giá trị nguyên nhân, không phải một phép "xóa kiểu" chung cho mọi lỗi.

Hai lần `errors.New("connection refused")` tạo hai giá trị lỗi khác nhau dù thông báo giống hệt. `errors.Is` mặc định so sánh giá trị khi kiểu của đích so sánh được; lỗi cũng có thể cung cấp method `Is(error) bool` để định nghĩa cách khớp. Nó không tự suy ra hai lỗi cùng nhóm từ câu chữ. Tương tự, `errors.As` không chỉ hỏi "kiểu có giống nhau không": nó tìm lỗi gán được vào biến đích hoặc để method `As(any) bool` tùy biến việc khớp.



Hình 17 — Ranh giới lỗi không được nuốt chửng nguyên nhân gốc. Mỗi tầng kiến trúc chỉ bổ sung ngữ cảnh mà nó thực sự nắm giữ; bên gọi sử dụng `errors.Is` khi cần ra quyết định phân nhánh và dùng `errors.As` khi cần trích xuất dữ liệu có cấu trúc từ một kiểu lỗi cụ thể.

Kiểu lỗi tùy biến mang dữ liệu cấu trúc (custom error)

Lỗi lính canh trả lời câu hỏi “đây có phải nhóm lỗi đó không?”. Khi thao tác thăm dò tới một điểm cuối đã cấu hình bị thất bại, công cụ dòng lệnh cần biết thêm tên dịch vụ và thông tin điểm cuối liên quan. Chúng ta sử dụng kiểu lỗi tùy biến (**custom error type**) để đóng gói ngữ cảnh có cấu trúc:

```
type ProbeFailure struct {
    Service  string
    Endpoint Endpoint
    Cause    error
}

func (failure *ProbeFailure) Error() string {
    return fmt.Sprintf(
        "probe %s at %s:%d: %v",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
        failure.Cause,
    )
}

func (failure *ProbeFailure) Unwrap() error {
    return failure.Cause
}
```

Phương thức `Error` sinh chuỗi thông báo cho người vận hành. Phương thức `Unwrap` khẳng định trường `Cause` vẫn là nguyên nhân gốc nằm dưới lớp vỏ bọc này. Giá trị lỗi cần tìm kiếm trong chuỗi là `*ProbeFailure`, bởi kiểu `ProbeFailure` triển khai phương thức nhận con trỏ (**pointer receiver**). Đối số `target` truyền vào `errors.As` bắt buộc phải là `&failure`: con trỏ tham chiếu tới biến tiếp nhận kết quả. Hàm `errors.As` sẽ gán con trỏ lỗi tìm được vào biến đó:

```
var failure *ProbeFailure
if errors.As(err, &failure) {
    fmt.Printf("%s đang lỗi tại %s:%d\n",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
    )
}
```

Hàm `errors.As` không bao giờ kiểm tra "thông báo lỗi có chứa chuỗi máy chủ hay không". Nó tìm kiếm một giá trị lỗi có thể gán được (**assignable**) vào kiểu đích mà bên gọi chỉ định. Đây chính là lý do các kiểu lỗi tùy biến nên

lưu trữ dữ liệu nghiệp vụ có cấu trúc mà bên gọi thực sự cần đến, thay vì chỉ là một cấu trúc bọc lại một chuỗi văn bản vô hồn.

`&failure` ở đây có kiểu `**ProbeFailure`: `As` cần địa chỉ của biến sẽ nhận con trỏ tìm được, không phải một đối tượng lỗi đã khởi tạo sẵn. Đích phải là con trỏ khác `nil` tới kiểu triển khai `error` hoặc tới một interface; truyền sai đích có thể gây `panic`. Nếu một API trả typed nil `(*ProbeFailure)(nil)` dưới kiểu `error`, lỗi interface vẫn khác `nil` như Chương 3; `As` có thể khớp và gán `failure == nil`. Test `TestErrorMatchingBoundaries` kiểm chứng điều đó. Hợp đồng của lab trả `nil` thật khi thành công, không dựa vào typed nil.

Bọc lỗi còn là quyết định về API. Nếu package bọc một lỗi của database bằng `%w`, bên gọi có thể dùng `Is` hoặc `As` để phụ thuộc vào lỗi đó; thay database sau này có thể phá hợp đồng. Chỉ bọc khi muốn công khai khả năng nhận diện nguyên nhân. Nếu ranh giới cần che implementation, hãy trả lỗi thuộc miền nghiệp vụ và lưu chẩn đoán nội bộ ở nơi thích hợp, không để caller phải parse text. Khi dùng `errors.Join` hoặc nhiều `%w`, cấu trúc là cây lỗi: `Is` và `As` duyệt sâu theo thứ tự từ trước ra sau, còn `errors.Unwrap` chỉ xử lý method `Unwrap() error`, không mở nhánh `Unwrap() []error` của Join.

Phân tích thất bại: trạng thái nào cần ghi nhận, lỗi nào cần trả về?

Dưới đây là bản hiện thực của ranh giới lỗi. Hãy đánh giá đoạn mã như một quy trình phản biện thiết kế: mỗi lệnh trả về return mang theo thông tin gì, và trạng thái hệ thống nào còn lưu lại sau khi hàm kết thúc?

```
func applyProbe(
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error {
    service, found := registry[name]
    if !found {
        return fmt.Errorf(
            "unknown %q: %w",
            name,
            ErrUnknownService,
        )
    }

    if err := run(service.Endpoint); err != nil {
```

```
        service.Record(false)
        registry[name] = service
        failed := &ProbeFailure{
            Service:  service.Name,
            Endpoint: service.Endpoint,
            Cause:    err,
        }
        return fmt.Errorf("probe %q: %w", name, failed)
    }

    service.Record(true)
    registry[name] = service
    return nil
}
```

Nhánh thiếu cấu hình trả về lỗi sớm: không có cấu trúc `Service` hợp lệ để biến đổi trạng thái, do đó bảng băm đăng ký registry được giữ nguyên vẹn. Nhánh thăm dò thất bại vẫn ghi nhận trạng thái `Healthy=false` và tăng bộ đếm số lần thử lại, rồi trả về đối tượng lỗi chứa nguyên nhân gốc. Đây không phải là hai cơ chế báo lỗi xung đột; trạng thái trả lời câu hỏi “kết quả quan

sát gần nhất ra sao", còn lỗi giải thích "thao tác vừa thực thi không thể hoàn tất vì lý do gì". Nhánh thành công cập nhật trạng thái hoạt động rồi trả về `nil`.

Khi kiểm thử mã nguồn này, hệ thống tập trung chứng minh ba cam kết hợp đồng cụ thể thay vì chỉ so khớp chuỗi thông báo:

Tình huống kiểm thử	Điều kiện xác minh trạng thái	Điều kiện xác minh lỗi (Error Contract)
Dịch vụ không tồn tại trong cấu hình	Hàm kiểm tra probe tuyệt đối không được gọi.	`errors.Is(err, ErrUnknownService)` trả về `true`.
Thao tác probe thất bại	Trạng thái dịch vụ cập nhật `Healthy=false` và tăng số lần thử lại.	`errors.As` trích xuất thành công `*ProbeFailure`, đồng thời `errors.Is` bảo toàn nguyên nhân gốc.
Thao tác probe thành công	Trạng thái dịch vụ cập nhật `Healthy=true` tại đúng endpoint cấu hình.	Giá trị lỗi trả về là `nil`.

Bài thực hành `part4-error-boundaries` hiện thực hóa ba khế ước trên thành các ca kiểm thử tự động. Kiểu `ProbeFunc` được thay thế bằng một hàm giả lập nhỏ gọn trong kiểm thử; chúng ta không cần mở socket mạng thực tế để chứng minh tính đúng đắn của ranh giới lỗi. Hãy mở `main_test.go` trước khi đọc `main.go`, chủ động đổi riêng `%w` thành `%v` ở câu lệnh return cuối cùng của nhánh thăm dò thất bại, rồi chạy kiểm thử `TestApplyProbePreservesFailureCauseAndContext`. Thông báo lỗi hiển thị gần như tương đồng, nhưng ca kiểm thử lập tức báo đỏ vì `errors.Is` không còn nhận diện được nguyên nhân gốc. Khôi phục lại `%w`, sau đó tạm thời vô hiệu hóa nhánh xử lý tín hiệu hủy bỏ (**cancellation**) và chạy `TestApplyProbeCancellationDoesNotChangeHealthState`: lúc này kết quả kiểm thử sẽ cảnh báo việc nhầm lẫn tín hiệu hủy tác vụ thành lỗi sức khỏe dịch vụ. Hai thao tác chủ động tiêm lỗi (**failure injection**) này mang lại bài học sâu sắc hơn việc chỉ sao chép lại chuỗi lỗi có sẵn.

> **Dừng để dự đoán:** Một lệnh `run` trả về `errors.New("connection refused")`. Nếu `applyProbe` đổi `%w` ở lệnh trả về cuối cùng thành `%v`, thông báo của công cụ dòng lệnh có thể vẫn hiển thị "connection refused". Khế ước nào trong ba khế ước trên đã bị phá vỡ? Viết một biểu thức kiểm chứng (**assertion**) để phát hiện lỗi này.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Khế ước nhận diện nguyên nhân gốc bị phá vỡ: biểu thức `errors.Is(err, errConnectionRefused)` sẽ trả về `false` vì chuỗi lỗi không còn mở gói (**unwrap**) tới lỗi ban đầu. Ngữ cảnh dạng chuỗi văn bản cho con người đọc không đủ để thay thế định danh kiểu cho mã nguồn. Biểu thức kiểm chứng cần khởi tạo `errConnectionRefused`, gọi `applyProbe`, rồi khẳng định `errors.Is(got, errConnectionRefused)` phải là `true`.

Điểm dừng của ranh giới lỗi

Chúng ta chưa vội đưa cơ chế thử lại (**retry**), thời hạn quá giờ (**timeout**) hay thu hồi tài nguyên vào đây; việc dồn quá nhiều trách nhiệm sẽ làm mờ ranh giới kiến trúc. Hiện tại giao diện lập trình đã xác lập một quy ước rành mạch: giá trị `nil` khẳng định thao tác đã hoàn tất trọn vẹn, giá trị `error` bảo toàn nguyên nhân gốc và ngữ cảnh điều tra, còn bên gọi hoàn toàn làm chủ quyết định rẽ nhánh thông qua `errors.Is` hoặc `errors.As`.

Phần tiếp theo của Chương 4 sẽ thử thách ranh giới lỗi này dưới áp lực quản lý tài nguyên và tín hiệu hủy bỏ (**cancellation**): lệnh `defer` cần được đặt ở đâu, đối tượng `context` truyền tải qua giao diện lập trình như thế nào, và vì sao `panic` không bao giờ là giải pháp thay thế cho một thất bại có thể dự đoán được.

Tín hiệu hủy bỏ không phải là chỉ dấu sức khỏe dịch vụ

Một thao tác thăm dò có thể bị hủy bỏ do người vận hành chủ động ngắt tiến trình dòng lệnh, do yêu cầu cấp cha bị quá hạn thời gian, hoặc do dịch vụ quản lý đang trong quá trình tắt dần (**shutdown**). Không tình huống nào trong số đó chứng minh điểm cuối của `billing` đã ngừng hoạt động. Nếu hàm `applyProbe` quy kết mọi lỗi vào nhánh `Record(false)`, tín hiệu hủy bỏ từ bên gọi sẽ vô tình tạo ra một chỉ dấu sai lệch về tình trạng sức khỏe dịch vụ.

Đối tượng `context.Context` chịu trách nhiệm mang theo thời hạn (**deadline**) và tín hiệu hủy bỏ xuyên qua các ranh giới giao diện. Nó luôn đứng ở vị trí tham số đầu tiên trong chữ ký hàm, tuyệt đối không được cất giữ bên trong cấu trúc `Service`, và được truyền tuần tự xuống các hàm có khả năng gây nghẽn:

```
type ProbeFunc func(context.Context, Endpoint) error

func applyProbe(
    ctx context.Context,
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error
```

Trước khi thực thi thao tác thăm dò, ranh giới lỗi chủ động kiểm tra trạng thái hủy để tránh kích hoạt các tác vụ nhập xuất vô ích. Nếu tín hiệu hủy xuất hiện trong lúc `run` đang xử lý, hàm `run` phải tôn trọng `ctx` và trả về một giá trị lỗi có thể nhận diện được thông qua `errors.Is(err, context.Canceled)` hoặc `errors.Is(err, context.DeadlineExceeded)`. Trả về trực tiếp `ctx.Err()` là cách tiếp cận đơn giản; bọc thêm ngữ cảnh cũng hoàn toàn chuẩn mực miễn là không làm mất định danh lỗi gốc. Hàm `applyProbe` nhận diện hai nguyên nhân chuẩn hóa này, bảo toàn chúng trong chuỗi bọc lỗi, nhưng tuyệt đối không cập nhật trạng thái sức khỏe dịch vụ:

```
if err := ctx.Err(); err != nil {
    return fmt.Errorf("probe canceled: %w", err)
}

if err := run(ctx, service.Endpoint); err != nil {
    if errors.Is(err, context.Canceled) ||
        errors.Is(err, context.DeadlineExceeded) {
        return fmt.Errorf("probe incomplete: %w", err)
    }

    service.Record(false)
    registry[name] = service
    failed := &ProbeFailure{
        Service: service.Name,
        Endpoint: service.Endpoint,
        Cause: err,
    }
    return fmt.Errorf("probe %q: %w", name, failed)
}
```

Đây là một chính sách thiết kế (**policy**) của `opsprobe`, chứ không phải quy luật áp đặt rằng tín hiệu hủy không bao giờ mang ý nghĩa vận hành. Tại ranh giới lỗi này, trạng thái `Healthy` chỉ được ghi nhận khi một lượt quan sát diễn ra trọn vẹn. Một thao tác bị hủy bỏ dở dang chưa thể mang lại bằng chứng quan sát đó. Bên gọi vẫn có thể sử dụng `errors.Is(err, context.Canceled)` để kết thúc tiến trình trong yên lặng, hoặc dùng `errors.Is(err, context.DeadlineExceeded)` để phát cảnh báo quá giờ.

Lệnh `defer` phải đi liền với khâu chiếm dụng tài nguyên

Tín hiệu hủy bỏ không tự động giải phóng các tài nguyên mà một hàm đã cấp phát. Quyền sở hữu (**ownership**) gắn liền với vị trí chiếm dụng tài nguyên (**acquisition**) thành công: hàm nào mở phiên làm việc (**session**) thì hàm đó phải lập tức lên lịch dọn dẹp (**cleanup**) trước khi chuyển sang các lệnh có thể trả về lỗi. Chúng ta tách biệt phần này khỏi trạng thái sức khỏe để phân định rõ ràng giữa chính sách thu hồi tài nguyên và chính sách kiểm tra điểm cuối:

```
type ProbeSession interface {
    Close() error
}

type OpenSession func(Endpoint) (ProbeSession, error)
type SessionProbe func(
    context.Context,
    Endpoint,
    ProbeSession,
) error

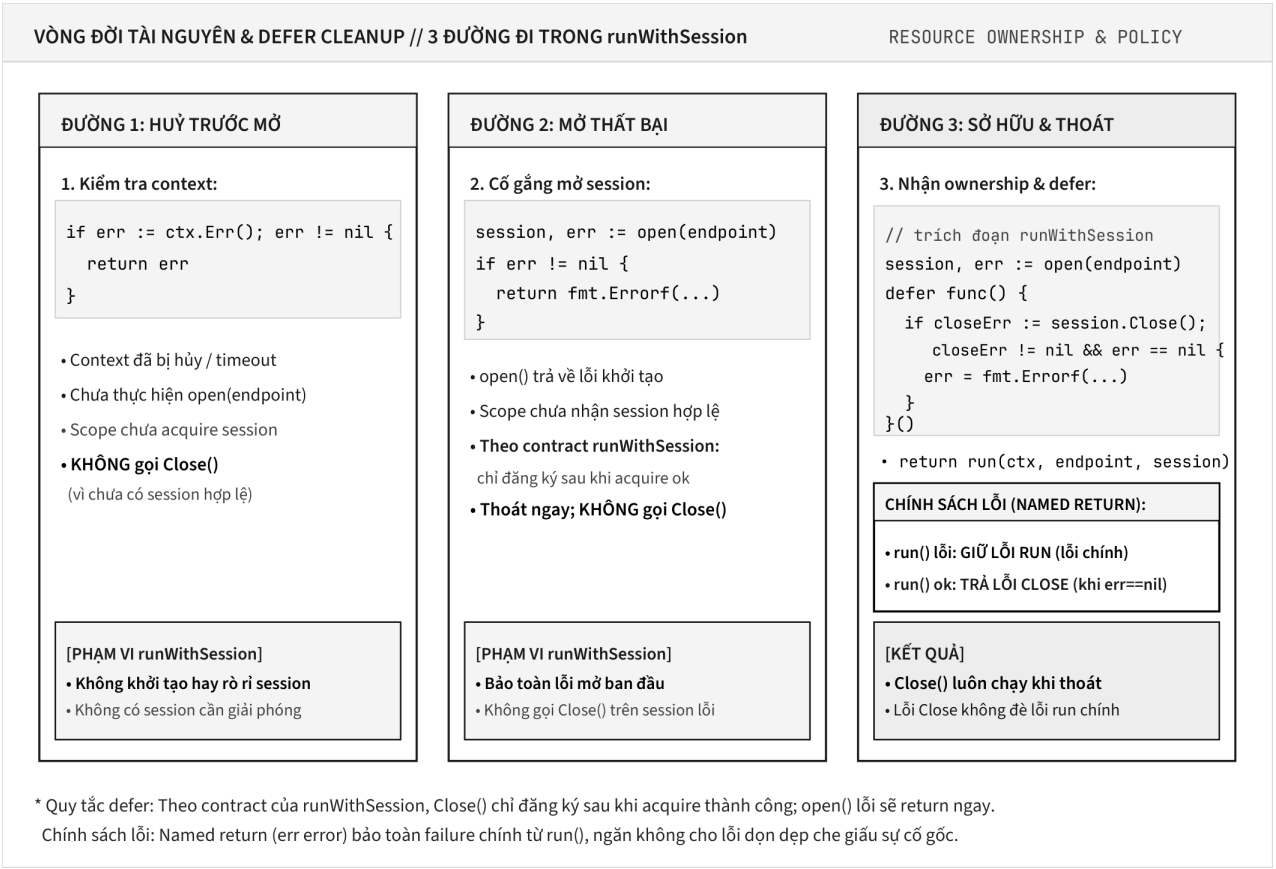
func runWithSession(
    ctx context.Context,
    endpoint Endpoint,
    open OpenSession,
    run SessionProbe,
) (err error) {
    if err := ctx.Err(); err != nil {
        return err
    }

    session, err := open(endpoint)
    if err != nil {
        return fmt.Errorf("open session: %w", err)
    }
    defer func() {
        if closeErr := session.Close(); closeErr != nil {
            if err == nil {
                err = fmt.Errorf("close session: %w", closeErr)
            }
        }
    }()

    return run(ctx, endpoint, session)
}
```

Lệnh `defer` được đăng ký ngay sau khi hàm `open` hoàn tất thành công. Nếu `open` thất bại, chưa có phiên làm việc nào được mở để phải đóng. Nếu `run` trả về theo bất kỳ nhánh nào, phương thức `Close` luôn thực thi trước khi `runWithSession` trả về kết quả cuối cùng. Giá trị trả về có tên `err` ở đây phục vụ mục đích thiết kế cụ thể: lỗi dọn dẹp chỉ trở thành lỗi trả về khi thao tác chính diễn ra thành công. Nếu `run` đã phát sinh lỗi, mã nguồn ưu tiên

bảo toàn thất bại nguyên phát thay vì thay thế nó bằng một lỗi thứ phát trong quá trình đóng phiên. Một hệ thống có yêu cầu lưu giữ đồng thời cả hai lỗi có thể sử dụng cấu trúc gom lỗi đa tầng (như `errors.Join`), nhưng điều đó phải được thực hiện với chủ đích rõ ràng.



Hình 18 — `defer` đăng ký dọn dẹp khi hàm `runWithSession` thoát, không phải khi một khối `if` kết thúc. Lỗi `Close` chỉ thay kết quả khi thao tác chính chưa có lỗi.

Điểm đăng ký và điểm thực thi khác nhau. Với `defer session.Close()`, receiver được tính ngay khi đăng ký; lời gọi chạy lúc hàm bao quanh kết thúc, theo thứ tự ngược với lúc đăng ký. Đặt defer trong vòng lặp không đóng tài nguyên ở cuối mỗi vòng: tất cả đợi tới cuối hàm. Nếu cần đóng từng phiên trước vòng kế tiếp, hãy đưa một lượt xử lý vào hàm riêng hoặc đóng tường minh. `TestDeferIsFunctionScopedAndCapturesArguments` truy vết cả khối lệnh và giá trị đối số để tách hai cơ chế này.

Defer cũng chạy khi panic tháo dỡ stack của cùng goroutine, nhưng không phải cơ chế bảo đảm trước mọi kiểu kết thúc tiến trình: `os.Exit` không chạy defer, và hàm đang nghẽn chưa thoát thì defer chưa thực thi. Giao ước `open` trong ví dụ yêu cầu trả phiên hợp lệ khi `err == nil` và không chuyển

quyền sở hữu phiên cho caller khi có lỗi. Nếu API mở tài nguyên cho phép trả cả tài nguyên lẫn lỗi, phải đọc hợp đồng riêng để biết ai dọn nó.

Cần lưu ý một ranh giới kiến trúc khác: bên gọi khởi tạo đối tượng context dẫn xuất bắt buộc phải gọi hàm `cancel`, kể cả khi thời hạn deadline có thể tự động hết hạn. Trong các đoạn mã ngắn, mẫu thiết kế chuẩn tắc là khai báo `ctx, cancel := context.WithTimeout(parent, timeout)` và ngay lập tức tiếp nối bằng `defer cancel()`. Hàm `cancel` giải phóng các tài nguyên bên trong bộ định thời của context; trong khi `session.Close` giải phóng tài nguyên mạng mà hàm `runWithSession` đã chiếm dụng. Hai thao tác dọn dẹp này thuộc về hai chủ thể sở hữu hoàn toàn khác nhau.

Đánh giá mã nguồn: đừng dùng `recover` để che giấu thất bại có thể dự đoán

Một review có thể gặp đoạn sau và thấy nó "tránh crash":

```
defer func() {
    if recovered := recover(); recovered != nil {
        err = fmt.Errorf("probe failed: %v", recovered)
    }
}()
```

Cách viết trên không thể thay thế cho câu lệnh `return error` chuẩn tắc. Tín hiệu hủy bỏ, thời gian quá hạn, thiếu cấu hình hay kết nối bị từ chối đều là các kết quả đầu ra (**outcomes**) mà giao diện lập trình đã lường trước; chúng bắt buộc phải lưu thông qua chuỗi bọc lỗi để bên gọi có thể phân loại theo chương trình. Việc lạm dụng `recover` ở đây sẽ biến các lỗi lập trình nghiêm trọng (như giải tham chiếu con trỏ `nil` hay phá vỡ điều kiện bất biến nội bộ) thành một chuỗi ký tự mơ hồ, trong khi trạng thái dữ liệu có thể đã bị biến dạng dở dang.

Cơ chế `panic` chỉ phù hợp với lỗi lập trình bất khả kháng, điều kiện bất biến nội bộ bị vi phạm nghiêm trọng, hoặc tại các ranh giới phục hồi chuyên biệt được thiết kế để cách ly mã nguồn không tin cậy. Hàm `recover` chỉ có vị trí hợp lý khi ranh giới đó có kế hoạch khôi phục trạng thái an toàn, ghi nhận nhật ký đầy đủ và sở hữu khế ước rõ ràng sau phục hồi. Công cụ `opsprobe`

hiện tại không chứa ranh giới như vậy; để `panic` xuất hiện minh bạch trong kiểm thử sẽ trung thực hơn nhiều so với việc ngụy trang nó thành một lỗi thăm dò thông thường.

Ở tầng ngôn ngữ, `recover` chỉ chặn panic khi được gọi trực tiếp từ hàm deferred đang xử lý panic trong cùng goroutine. Gọi nó qua một helper thông thường không chặn được; defer ở goroutine cha cũng không bắt panic của goroutine con. Khi phục hồi thành công, hàm đang có defer trở về caller, không tiếp tục ở câu lệnh ngay sau chỗ panic. Các test cleanup và recover của lab kiểm tra phạm vi này bằng panic có kiểm soát; không đổi panic thành kết quả probe thành công.

Bài thực hành: Lần theo đường dọn dẹp tài nguyên

Ngữ cảnh `ctx` bị hủy sau khi `open` trả về một phiên làm việc, còn `run` trả về `context.Canceled`. Hàm `Close` có được gọi không? Lỗi cuối cùng giữ định danh nào? Trạng thái `registry["billing"]` có nên thay đổi `Healthy` hay `Retries` không?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Hàm `Close` được gọi đúng một lần vì lệnh `defer` đã được đăng ký ngay sau khi cấp phát tài nguyên thành công. Hàm `runWithSession` giữ `context.Canceled` làm lỗi chính nếu `Close` cũng phát sinh lỗi; `applyProbe` bọc nguyên nhân đó để phép thử `errors.Is(err, context.Canceled)` vẫn trả về `true`. Vì thao tác không hoàn tất một quan sát điểm cuối, chính sách hiện tại giữ nguyên `Healthy` và `Retries`.

Kết chương: Mọi thất bại đều có đường lưu thông chuẩn tắc

`ErrUnknownService` cho caller điều kiện để xử lý cấu hình; `ProbeFailure` giữ ngữ cảnh của quan sát; các lớp bọc công khai nguyên nhân theo hợp đồng. Chính sách cancellation của lab giữ nguyên sức khỏe khi quan sát bị hủy, còn defer dọn phiên khi hàm bao quanh thoát. Panic có phạm vi phục hồi riêng và không thay thế đường trả lỗi đã thiết kế.

Công cụ `opsprobe` lúc này đã phân định minh bạch điều nó sở hữu, điều nó quan sát và trao toàn quyền quyết định cho bên gọi. Chương tiếp theo: ranh

giới gói (**package boundary**), phạm vi xuất khẩu và nguyên lý khép kín phụ thuộc.

Bốn câu hỏi trước khi merge một error boundary

Trước khi hợp nhất một ranh giới xử lý lỗi vào mã nguồn chính, hãy đối chiếu quy trình kiểm định qua bốn tiêu chí kỹ thuật cốt lõi sau:

Tiêu chí kiểm định ranh giới lỗi	Yêu cầu kỹ thuật bắt buộc
Nhận diện định danh lỗi (Error Identity)	Bên gọi có cần rẽ nhánh quyết định theo từng loại lỗi cụ thể không? Nếu có, phải bảo toàn identity bằng sentinel error hoặc custom type có cấu trúc; cấm bắt caller parse chuỗi văn bản.
Bản chất quan sát miền nghiệp vụ	Lỗi này có phản ánh trạng thái thực của dịch vụ đích không? Timeout hoặc cancellation từ Context của caller không tự động đồng nghĩa với việc endpoint bị unhealthy.
Quyền sở hữu tài nguyên	Hàm nào nhận quyền sở hữu? Defer đăng ký ở đâu và hàm đó thoát khi nào? Đừng nhầm kết thúc khối lệnh với thực thi dọn dẹp.
Chính sách bảo toàn lỗi gốc (Primary Error Policy)	Nếu thao tác dọn dẹp trong defer cũng phát sinh lỗi, lỗi ban đầu có bị nuốt mất không? Cần bảo toàn lỗi gốc để không làm gián đoạn việc điều tra sự cố.

Khi bốn câu trả lời hiện ngay trong chữ ký hàm, chuỗi bọc lỗi, chuyển dịch trạng thái và kiểm thử tự động, người đọc sau không cần phỏng đoán lỗi có bị che lấp, số lần thử lại có bị tăng sai, hay tài nguyên hệ thống có bị rò rỉ hay không. Đó là sự chuẩn bị cần thiết trước khi tách mã nguồn thành các package độc lập.

Chương 5 — Package là ranh giới

Sau Chương 4, công cụ `opsprobe` đã sở hữu một giao diện lập trình (**API**) cho một thao tác thăm dò. Nhưng bài thực hành vẫn nằm gọn trong một gói `main` chứa lẫn lộn mô hình dữ liệu, chính sách lỗi, cấu hình giả lập và cách thức in ra thiết bị đầu cuối. Khi mọi thứ còn ngắn gọn, một tệp duy nhất là lựa chọn tốt. Vấn đề xuất hiện không phải vì số lượng tệp tăng lên, mà vì một thay đổi trong cách hiển thị dòng lệnh có thể nhìn thấy mọi chi tiết nội bộ của gói thăm dò, và một thay đổi cấu hình có thể kéo theo toàn bộ mô hình nghiệp vụ lõi.

Chúng ta không mở đầu bằng một cấu trúc thư mục "chuẩn mẫu" mang tính giáo điều. Chúng ta bắt đầu bằng việc rà soát trách nhiệm: đoạn mã nào cần biết lệnh `fmt.Println`, đoạn mã nào bắt buộc phải biết điểm cuối (**endpoint**), và đoạn mã nào nắm giữ quyền quyết định cấu hình được nạp từ đâu? Khi câu trả lời khác nhau, ranh giới kiến trúc (**boundary**) bắt buộc phải phân tách độc lập.

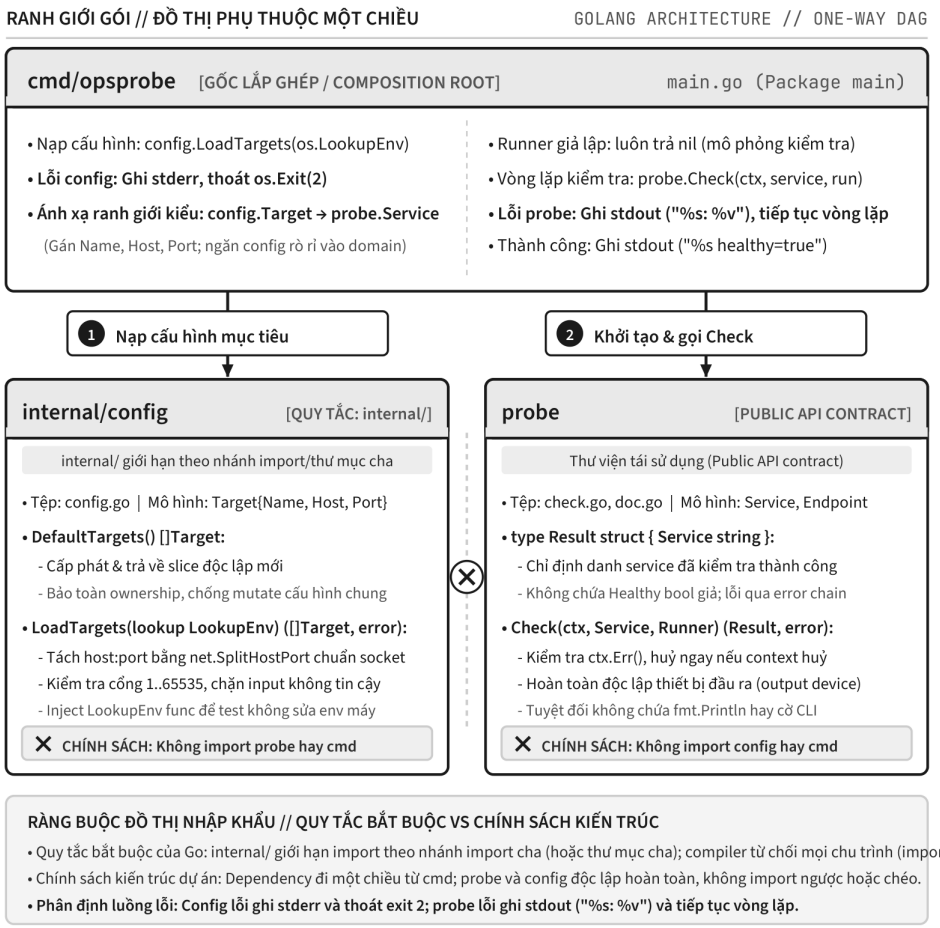
Một dependency graph có thể đọc từ ngoài vào trong

Refactor đầu tiên của `opsprobe` phân định ba vai trò kiến trúc rõ rệt:

Package `cmd/opsprobe` là điểm khởi đầu dòng lệnh (command entrypoint): tiếp nhận các phụ thuộc, điều phối ca sử dụng và trình bày kết quả ra màn hình dòng lệnh.

Package `probe` là thư viện có thể tái sử dụng (importable package): sở hữu mô hình nghiệp vụ cho một lần kiểm tra và hoàn toàn độc lập với thiết bị đầu ra.

Package `internal/config` là chi tiết triển khai nội bộ của ứng dụng: quản lý cấu hình mặc định, với phạm vi import được lệnh `go` kiểm tra theo ranh giới parent của `internal`, không phải một quy tắc chung "cùng module". Phần dưới sẽ phân biệt đường dẫn import với đường dẫn thư mục theo chế độ build.



Hình 19 — Hướng phụ thuộc (**dependency direction**) là công cụ để rà soát thiết kế. Gói `cmd/opsprobe` là điểm cấu thành (**composition root**); nó biết rõ các gói cụ thể. Ngược lại, gói `probe` hoàn toàn không biết đến giao diện dòng lệnh hay nguồn cấu hình, do đó nó có thể được đọc hiểu, kiểm thử độc lập và thay thế hàm thực thi mà không kéo theo tầng hiển thị.

Tệp `go.mod` xác lập đường dẫn mô-đun (**module path**), còn mỗi thư mục trong Go là một gói riêng biệt. Vì bài thực hành sử dụng mô-đun `example.com/golang-master/part5-package-design`, đường dẫn nhập khẩu của gói `probe` là `example.com/golang-master/part5-package-design/probe`. Một gói có thể gồm nhiều tệp trong cùng một thư mục; việc chia nhỏ tệp không tạo thêm ranh giới mới nếu chúng vẫn cùng khai báo `package probe`.

```
part5-package-design/
├── go.mod
├── cmd/opsprobe/main.go
├── probe/doc.go
├── probe/check.go
└── internal/config/config.go
```

Thư mục mang tên `cmd` là một quy ước hữu ích khi kho mã nguồn chứa nhiều tệp thực thi; bản thân trình biên dịch không gán ý nghĩa cú pháp đặc biệt nào cho nó. Tệp `main.go` cũng chỉ là tên tệp theo quy ước cộng đồng. Ranh giới kiến trúc thực sự được xác lập từ mệnh đề khai báo gói, đường dẫn nhập khẩu và chiều hướng của các quan hệ phụ thuộc.

Export là lời hứa, không phải cách sửa compile error

Trong gói `probe`, bên gọi chỉ cần khởi tạo dịch vụ, truyền hàm thực thi và nhận kết quả. Chỉ những khái niệm mang tính giao ước đó mới cần được xuất bản (**export**):

```
package probe

type Endpoint struct { Host string; Port int }
type Service struct { Name string; Endpoint Endpoint }

type Runner func(context.Context, Endpoint) error
type Result struct { Service string }

func Check(
    ctx context.Context,
    service Service,
    run Runner,
) (Result, error)
```

Chữ cái đầu viết hoa không làm cho kiểu dữ liệu “quan trọng hơn”; nó mở ra một giao ước công khai (**contract**) cho các gói khác sử dụng. Hàm `Check` được xuất bản vì công cụ dòng lệnh thực sự cần kích hoạt nó. Hàm trợ giúp bổ sung ngữ cảnh lỗi ở bên trong có thể giữ kín nội bộ (**unexported**) vì chưa có bên gọi nào ngoài `probe` cần đến nó. Việc xuất bản vội vàng rồi hy vọng một bên tiêu dùng sẽ xuất hiện là con đường ngắn nhất tạo ra một giao diện lập trình cồng kềnh với gánh nặng tương thích ngược không đáng có.

Tài liệu hướng dẫn của gói cũng là một phần không thể tách rời của giao ước. Tệp `probe/doc.go` bắt đầu bằng lời chú thích `Package probe ...`, làm rõ gói chịu trách nhiệm cho những gì và tuyệt đối không đảm nhận những gì. Lời chú thích cho các định danh xuất bản nên mô tả hành vi mà

bên gọi có thể an tâm dựa vào, thay vì chỉ lặp lại một cách sáo rỗng tên của kiểu dữ liệu.

`internal` là boundary do go command kiểm tra

Target mặc định là policy của application hiện tại. Ta để nó trong `internal/config`:

```
package config

type Target struct {
    Name string
    Host string
    Port int
}

func DefaultTargets() []Target {
    return []Target{
        {
            Name: "billing",
            Host: "billing.internal",
            Port: 8443,
        },
    }
}
```

`internal` được lệnh `go` kiểm tra, không chỉ là thỏa thuận đặt tên. Với lab dùng module, gói `example.com/golang-master/part5-package-design/internal/config` chỉ được import từ cây đường dẫn import có gốc `example.com/golang-master/part5-package-design`. Vì thế `cmd/opsprobe` hợp lệ. Đừng chỉ xét vị trí thư mục trên đĩa hoặc số lượng module: một module con mang path không liên quan vẫn có thể bị từ chối dù đặt ngay trong repo. Trong Go 1.27.1, `cmd/go/internal/load/pkg.go` dùng đường dẫn import cho package thuộc module; package ngoài module dùng cây thư mục tương ứng. Đây là hành vi toolchain, không phải quy tắc access modifier của Go Specification.

`TestInternalBoundaryUsesModuleImportPaths` trong `labs/edition-contracts` dựng hai consumer với cùng mã import và cùng `replace` cục bộ. Consumer có path dưới cây được phép build được;

consumer có path khác bị từ chối với `use of internal package ... not allowed`. Các thư mục vật lý đều nằm trong một thư mục tạm. Phép thử không biến `internal` thành ranh giới bảo mật trước người có quyền sửa source hay chạy chương trình; nó bảo vệ ranh giới import trong quy trình build.

Đây không phải lý do để nhét mọi gói vào `internal` theo phản xạ. Với tệp nhị phân tự chứa, đó thường là giá trị mặc định (**default**) tốt vì không công bố giao diện lập trình (**API**) ra ngoài. Với `probe` trong bài thực hành này, chúng ta cố tình để gói này có thể nhập khẩu (**importable**) để ranh giới giữa thao tác nghiệp vụ (**domain operation**) và công cụ dòng lệnh (**CLI**) hiển lộ rõ ràng. Nếu sau này không còn bên tiêu dùng (**consumer**) nào bên ngoài ứng dụng, nó cũng có thể chuyển thành nội bộ (`internal`); vị trí phải dựa trên bên tiêu dùng thực tế, không theo cấu trúc thư mục hình thức.

Điểm cấu thành (Composition root) chuyển dữ liệu, không giấu phụ thuộc

`config.Target` và `probe.Service` không cùng kiểu dữ liệu. Chúng thuộc hai ranh giới kiến trúc khác nhau: gói cấu hình xác định dữ liệu được khai báo như thế nào; trong khi gói thăm dò xác định một thao tác cần những gì. Tệp `cmd/opsprobe` là vị trí phù hợp nhất để ánh xạ giữa hai mô hình dữ liệu đó:

```
for _, target := range config.DefaultTargets() {
    service := probe.Service{
        Name: target.Name,
        Endpoint: probe.Endpoint{
            Host: target.Host,
            Port: target.Port,
        },
    }

    result, err := probe.Check(ctx, service, run)
    if err != nil {
        fmt.Printf("%s: %v\n", service.Name, err)
        continue
    }
    fmt.Printf("%s healthy=true\n", result.Service)
}
```

Phép ánh xạ này thoạt nhìn như đoạn mã lặp lại thông thường, nhưng nó bảo toàn hướng phụ thuộc (**dependency direction**) một cách trung thực: `probe` không cần nhập khẩu gói cấu hình, gói cấu hình không cần biết tới cấu trúc kết quả, và mã in ấn ra thiết bị đầu cuối không rò rỉ vào gói nghiệp vụ lõi. Khi nguồn cấu hình chuyển từ giá trị mặc định sang tệp tin hoặc biến môi trường ở chương sau, phương thức `probe.Check` không hề bị xáo trộn chỉ vì cách nạp cấu hình đã đổi.

Tái cấu trúc có kiểm chứng, không phải di chuyển tệp đơn thuần

Bài thực hành kiểm tra ba cam kết kỹ thuật trọng yếu khi phân tách gói nhằm bảo vệ ranh giới kiến trúc:

Thành phần kiểm thử	Hành vi kỳ vọng của hệ thống	Rủi ro nếu thiết kế sai ranh giới
Điều phối hủy bỏ (`probe.Check`)	Chuyển `ctx.Err()` thành lỗi nhận diện được bằng `errors.Is`, không kích hoạt runner khi context đã bị hủy.	Tiếp tục tiêu tốn tài nguyên mạng dù bên gọi đã phát tín hiệu dừng.
Toàn vẹn kết quả	Đối tượng kết quả mang chính xác tên định danh dịch vụ đã cấu hình.	Trộn lẫn trạng thái giữa các dịch vụ trong quá trình duyệt danh sách.
Độc lập dữ liệu (`internal/config`)	Trả về một slice độc lập theo từng lần gọi factory.	Caller sửa slice nhận được làm đột biến giá trị cấu hình mặc định dùng chung.

Ca kiểm thử cuối cùng không nhằm khẳng định "mọi lát cắt đều nguy hiểm". Nó chứng minh quyền sở hữu (**ownership**) của hàm tạo cấu hình: `DefaultTargets` tạo dữ liệu hoàn toàn mới cho bên gọi thay vì phát tán một lát cắt dùng chung. Đây là câu hỏi về ngữ nghĩa giá trị (**value semantics**) của Chương 2, nay được đặt vào ranh giới của một gói.

Khởi tạo Gói và Ràng buộc Đồ thị Phụ thuộc (Initialization DAG)

Đặc tả Go không cho phép một gói nhập khẩu chính nó, dù là trực tiếp hay gián tiếp. Vì vậy đồ thị nhập khẩu (**import graph**) tuyệt đối không được có chu trình; quá trình biên dịch phát hiện chu trình phụ thuộc (**cyclic dependency**) sẽ bị từ chối ngay với chẩn đoán `import cycle not allowed`. Quy tắc này giúp xác định thứ tự khởi tạo gói một cách tất định, chứ không chứng minh các dịch vụ phụ thuộc bên ngoài tiến trình đã sẵn sàng.

Nếu gói A nhập khẩu gói B, thì B bắt buộc phải được khởi tạo hoàn tất trước khi A bắt đầu. Trong phạm vi từng gói, các biến cấp gói được khởi tạo theo thứ tự phụ thuộc tĩnh, tiếp theo là các hàm `init()` chạy tuần tự trong luồng nhẹ khởi tạo trước khi hàm `main` thực thi. Tuy nhiên, hàm `init()` hoàn toàn có thể khởi tạo các luồng nhẹ khác; đồ thị nhập khẩu không thể bảo đảm các luồng đó đã kết thúc hoặc cấu hình bên ngoài đã sẵn sàng. Những

tác vụ có thể phát sinh lỗi, cần thời hạn xử lý (**deadline**) hoặc cần tài nguyên bên ngoài nên được gọi một cách tường minh từ điểm khởi tạo ứng dụng, nơi bên gọi có thể trực tiếp quan sát và xử lý lỗi.

Lần này bài học không bắt đầu từ mã nguồn đã được phân tách sẵn. Mở thư mục `labs/part5-package-refactor` trong trình soạn thảo: điểm xuất phát là một tệp `main.go` nguyên khối đang chạy. Chạy `go run .` để ghi nhận hành vi hiện tại làm mốc kiểm chuẩn, rồi chạy `go test -tags exercise ./...`. Lỗi biên dịch do gói `probe` chưa tồn tại chính là tín hiệu bắt đầu, không phải lỗi cần bỏ qua: các ca kiểm thử đang đòi hỏi một ranh giới công khai mà mã nguồn chưa thiết lập. Từ yêu cầu nghiệp vụ và các ca kiểm thử, người học tự quyết định tệp nào đưa vào `probe`, tệp nào thuộc `internal/config`, và công cụ dòng lệnh ánh xạ hai mô hình dữ liệu ở đâu. Khi hoàn tất, lệnh `go run ./cmd/opsprobe` phải bảo toàn kết quả xuất ra, đồng thời các lệnh `go test -tags exercise ./...` và `go vet ./...` đều phải báo xanh.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Bản triển khai tham chiếu nằm ở `labs/part5-package-design`, nhưng chỉ nên tham khảo sau khi các ca kiểm thử chấp nhận (**acceptance test**) đã đưa ra phản hồi đầu tiên. Ranh giới kiến trúc cần bảo toàn là: gói `probe` công bố các thao tác nghiệp vụ mà kiểm thử kích hoạt, gói `internal/config` duy trì chính sách mặc định và trả về lát cắt mới độc lập, còn `cmd/opsprobe` là nơi ánh xạ dữ liệu rồi hiển thị kết quả. Tuyệt đối không có lý do gì để `probe` nhập khẩu ngược lại gói cấu hình hoặc thiết bị đầu cuối.

Bài thực hành: Xác định vị trí đặt mã nguồn trong kiến trúc gói

Một kỹ sư muốn thêm lệnh `fmt.Println` vào `probe.Check` để in tiến trình xử lý, và muốn `probe` gọi trực tiếp `config.DefaultTargets` để giảm bớt bước ánh xạ ở `main`. Với đồ thị phụ thuộc trên, hai thay đổi đó gây ra rủi ro gì? Hãy đề xuất vị trí thay thế phù hợp.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Việc gọi `fmt.Println` kéo tầng hiển thị vào một gói có thể được tái sử dụng; công cụ dòng lệnh mới là nơi quyết định quy cách hiển thị và đích xuất dữ liệu. Việc để `probe` gọi gói cấu hình sẽ đảo ngược chiều phụ thuộc từ thao tác nghiệp vụ sang chính sách ứng dụng, khiến cho kiểm thử của `probe` bị ràng buộc chặt chẽ với cấu hình. Thông tin tiến trình nên được chuyển giao dưới dạng sự kiện hoặc kết quả để bên gọi tự hiển thị; việc ánh xạ dữ liệu nên nằm ở lệnh hoặc một dịch vụ ứng dụng nằm ngoài cả `probe` lẫn `config`.

Nguồn cấu hình là dữ liệu không đáng tin cậy, không phải điểm cuối

Yêu cầu nghiệp vụ tiếp theo đến từ người vận hành: trong một lượt chạy, họ muốn thay đổi điểm cuối mặc định mà không cần sửa đổi mã nguồn. Chúng ta chọn một biến môi trường có định dạng rõ ràng: `OPS_PROBE_TARGET=payments.internal:9443`. Đây là năng lực của ứng dụng, không phải trách nhiệm của gói `probe`. Hàm `probe.Check` chỉ tiếp nhận đối tượng `Endpoint` đã hợp lệ; nó không có lý do gì để bận tâm chuỗi ký tự đó bắt nguồn từ biến môi trường, tệp cấu hình hay một cờ lệnh trong tương lai.

Điều này xác lập một ranh giới kiến trúc quan trọng. Môi trường luôn cung cấp chuỗi ký tự thô; trong khi điểm cuối đòi hỏi tên máy chủ và cổng mạng có ý nghĩa. Thao tác phân tách, chuyển đổi kiểu và kiểm tra phạm vi thuộc về trách nhiệm của `internal/config`. Công cụ dòng lệnh chuyển giao hàm đọc thực tế `os.LookupEnv` vào ranh giới đó, rồi chỉ nhận về `[]Target` hoặc lỗi. Hình thức của giao diện lập trình đủ nhỏ gọn để kiểm thử độc lập mà không cần can thiệp biến môi trường của hệ điều hành:

```
type LookupEnv func(string) (string, bool)

func LoadTargets(lookup LookupEnv) ([]Target, error)
```

Đây không phải là một khung tiêm phụ thuộc (**dependency injection framework**) phức tạp. `LookupEnv` chỉ đơn thuần là một kiểu hàm mô tả chính xác phụ thuộc mà gói cấu hình cần: nhận một khóa, trả về giá trị

chuỗi và báo hiệu khóa có tồn tại hay không. Trong kiểm thử, một hàm đóng kín nhỏ thay thế cho `os.LookupEnv`; trong lệnh chính, hàm `os.LookupEnv` đáp ứng trọn vẹn chữ ký hàm đó. Nhờ vậy, ca kiểm thử mô tả dữ liệu đầu vào tường minh mà không có ca kiểm thử nào phải thay đổi môi trường tiến trình dùng chung.

Chuỗi `host:port` cũng không nên được tách bằng lệnh thô sơ `strings.Split`. Ký tự dấu hai chấm `:` xuất hiện phổ biến trong địa chỉ IPv6, do đó `net.SplitHostPort` là bộ phân tích cú pháp phù hợp với ngữ pháp socket mạng. Chuỗi `payments.internal:9443` là đầu vào hợp lệ; địa chỉ IPv6 cần đặt trong dấu ngoặc vuông như `[2001:db8::9]:9443`. Sau khi phân tách, cổng mạng vẫn ở dạng chuỗi: hàm `strconv.Atoi` chuyển đổi thành số nguyên `int`, rồi gói cấu hình kiểm tra phạm vi 1–65535 trước khi khởi tạo mục tiêu. Mọi dữ liệu lỗi đều trả về lỗi có ngữ cảnh để công cụ dòng lệnh in ra và thoát với mã khác 0.

Ở đây không cần khai báo lỗi lính gác (**sentinel error**) mới. Bên gọi hiện chỉ có một chính sách xử lý duy nhất cho cấu hình lỗi: thông báo cho người dùng và dừng chương trình trước khi quá trình thăm dò bắt đầu. Nếu sau này công cụ dòng lệnh cần phân biệt lỗi cú pháp với lỗi thiếu thông tin bảo mật để đưa ra giải pháp khắc phục khác nhau, đó mới là thời điểm danh tính lỗi xứng đáng được đưa vào giao ước. Chương 4 không yêu cầu "mọi lỗi phải có kiểu riêng"; nó nhấn mạnh chỉ giữ lại danh tính khi bên gọi có các quyết định ứng xử khác nhau.

Bài thực hành: Yêu cầu thay đổi sau khi tái cấu trúc

Sau khi hoàn thành lần tách package đầu tiên trong `labs/part5-package-refactor`, chạy `go test -tags configexercise ./...`. Test đòi `internal/config.LoadTargets` nhận một hàm đọc biến môi trường và đổi endpoint khi có `OPS_PROBE_TARGET`. Tự chọn file, API và thông báo lỗi; ràng buộc kỹ thuật là dùng `net.SplitHostPort`, từ chối host rỗng và port ngoài 1–65535, còn `cmd/opsprobe` mới được gọi `os.LookupEnv`. Khi test xanh, chạy lệnh với `OPS_PROBE_TARGET=payments.internal:9443` để kiểm tra đường đi thời gian chạy.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Bản hiện thực tham chiếu giữ `LoadTargets` trong `internal/config`, bắt đầu từ `DefaultTargets` để quyền sở hữu của slice không đổi, rồi chỉ ghi đè host và port của target đầu. Lệnh xử lý lỗi ở biên tiến trình bằng stderr và mã thoát 2; `probe` không thay đổi. Đây là dấu hiệu tái cấu trúc tốt: yêu cầu về nguồn cấu hình đi qua một package, trong khi thao tác kiểm tra endpoint không cần biết nguồn dữ liệu vừa đổi.

Code review: một field không có trạng thái thứ hai

Sau khi thay đổi cấu hình, chúng ta rà soát lại giao diện lập trình công khai thay vì vội vàng bổ sung thêm tính năng mới. Hàm `Check` có hai kết quả đầu ra: trả về `error` khi thao tác không hoàn tất, hoặc trả về `Result` khi hàm thực thi thành công. Trong giao ước này, trường `Result.Healthy` luôn mang giá trị `true`. Trường dữ liệu đó trông có vẻ vô hại, nhưng nó gợi ý cho bên gọi về một trạng thái thứ hai mà hàm không bao giờ trả về.

Đây là một khiếm khuyết trong thiết kế, không phải lỗi của trình biên dịch. Một kỹ sư mới tiếp nhận mã nguồn có thể viết `if !result.Healthy { retry() }`, rồi tin rằng nhánh rẽ đó có ý nghĩa thực tế. Trong thực tế, sự cố đã được truyền tải qua giá trị `error`; giá trị mặc định của `Result` đi kèm với lỗi không phải là một quan sát mang ý nghĩa “không lành mạnh”. Việc cố giữ trường boolean chỉ để phục vụ câu in ra màn hình có chữ `true` đã khiến ranh giới giao diện rộng hơn dữ liệu mà thao tác thực sự cung cấp.

Chúng ta thu hẹp `Result` chỉ còn định danh dịch vụ đã hoàn tất. Tệp `cmd/opsprobe` đã sở hữu nhánh kiểm tra `err == nil`, do đó chính công cụ dòng lệnh, nơi sở hữu tầng hiển thị, sẽ in `healthy=true`. Hoàn toàn không có thông tin nào bị thất thoát: thành công đã là bằng chứng của trạng thái lành mạnh; thất bại vẫn là lỗi đi kèm ngữ cảnh. Đây là một lần tái cấu trúc đặc biệt giá trị vì kiểm thử không hề báo đỏ trước khi sửa. Kiểm thử cũ vẫn xanh, nhưng giao ước cũ có thể khiến người đọc suy luận sai lệch.

```
// Result identifies the service whose check
// completed successfully.
type Result struct {
    Service string
}
```

Một lời ghi chú giải thích tốt giờ đây có thể phát biểu chính xác hơn: `Result` không "chứa trạng thái sức khỏe", mà định danh dịch vụ đã kiểm tra thành công. Khi giao diện thực sự cần diễn đạt nhiều trạng thái quan sát (chẳng hạn như lành mạnh, suy giảm, không thể kết nối cùng mốc thời gian và độ trễ), đó sẽ là một mô hình dữ liệu khác, được thiết kế cùng yêu cầu nghiệp vụ và kiểm thử riêng. Đừng bao giờ chừa chỗ cho một tính năng chưa tồn tại chỉ để giao diện trông có vẻ linh hoạt.

Bài thực hành: Đánh giá mã nguồn dưới các ràng buộc

Trong `labs/part5-package-design`, tạm giữ `Healthy bool` trong `Result` rồi viết ra hai kết quả mà `Check` có thể trả về. Sau đó bỏ trường này, sửa kiểm thử và chỉ sửa phần hiển thị ở tầng lệnh cho đến khi cả `go test ./...` cùng `go run ./cmd/opsprobe` đều đạt kết quả tốt. Tuyệt đối không chuyển lỗi thất bại thành `Result{Healthy: false}`: điều đó làm mất chuỗi lỗi mà Chương 4 đã xây dựng. Trong bản tái cấu trúc độc lập, `public_boundary_test.go` là khế ước tối thiểu; hãy sửa bản hiện thực theo kiểm thử, không tự ý thêm trường dữ liệu chỉ để dự đoán tương lai.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Giá trị `Result{Service: service.Name}` là đủ cho nhánh thành công. Sau khi kiểm tra `err == nil`, lệnh tự in chuỗi ký tự `healthy=true`. Nếu sau này cần trạng thái sức khỏe mà vẫn giữ việc phân biệt lỗi thao tác riêng, API sẽ cần một khế ước mới quy định rõ kết quả nào có thể xuất hiện cùng lỗi nào; đó không phải việc của cờ boolean hiện tại.

Module chọn source nào để build

Một ranh giới gói được phân chia chuẩn mực vẫn có thể biên dịch với mã nguồn khác với dự tính của người lập trình. Gói (**package**) là đơn vị tổ chức mã nguồn; trong khi mô-đun (**module**) là tập hợp các gói đi kèm phiên bản và các yêu cầu phụ thuộc. Tệp `go.mod` xác lập đường dẫn mô-đun, phiên bản ngôn ngữ và các yêu cầu; tệp `go.sum` lưu bảng băm kiểm tra để xác minh tính toàn vẹn của mã phụ thuộc đã tải. Nó không phải tệp khóa phiên bản liệt kê cứng mọi phiên bản sẽ dùng trong mọi chế độ biên dịch.

Go sử dụng thuật toán chọn phiên bản tối thiểu (**minimal version selection**) cho các yêu cầu trong đồ thị mô-đun: với mỗi đường dẫn mô-đun, Go chọn phiên bản cao nhất được yêu cầu trong đồ thị liên quan, chứ không tự ý tải bản mới nhất trên mạng Internet. Thuật ngữ "tối thiểu" ở đây chỉ tập hợp phiên bản thỏa mãn yêu cầu, không có nghĩa là chọn phiên bản thấp nhất trong mọi phụ thuộc. Chỉ thị `replace` có thể hoán đổi mã nguồn cục bộ hoặc chỉ định phiên bản; một bản biên dịch sử dụng chỉ thị thay thế cục bộ

chỉ chứng minh mã nguồn trên máy đó. Chỉ thị thay thế trong các gói phụ thuộc không tự động trở thành chính sách của mô-đun chính tiêu dùng nó. Khi điều tra sự cố, hãy đọc kỹ kết quả `go list -m all`, `go mod graph` và `go env GOMOD GOWORK` trước khi kết luận kiểm thử đang chạy đúng phụ thuộc mong muốn.

Thí nghiệm `TestMVSSelectsHighestRequiredVersion` trong `edition-contracts` đặt yêu cầu trực tiếp B ở `v1.0.0`, còn phụ thuộc C yêu cầu B ở `v1.2.0`. Trước khi chạy, hãy dự đoán `go list -m all` báo phiên bản B nào. Test dựng proxy tệp cục bộ với cả B `v1.3.0` nhưng không cho phép mạng: kết quả phải là `v1.2.0`, không phải phiên bản trực tiếp thấp hơn hay phiên bản có sẵn mới nhất. Đây là kiểm chứng lựa chọn phiên bản của toolchain, không chứng minh các phiên bản đó tương thích về API. Sau đó đổi yêu cầu của C và quan sát lựa chọn thay đổi; không cần sửa `go.sum` bằng tay để "ép" phiên bản.

Không gian làm việc (**workspace**) bổ sung một lựa chọn cục bộ: tệp `go.work` có thể đưa nhiều mô-đun vào vai trò mô-đun chính để phát triển song song. Nó không hợp nhất chúng thành một mô-đun duy nhất để phát hành. Một dịch vụ trong không gian làm việc có thể dùng thư viện vừa sửa đổi trên máy, trong khi người dùng tải về vẫn nhận phiên bản đã phát hành chính thức. Vì vậy, việc kiểm tra ngoài không gian làm việc là một phép thử độc lập bắt buộc, không phải sự lặp lại thừa thãi:

```
go env GOMOD GOWORK
go list -m all
# Chạy trong module muốn kiểm tra độc lập
$env:GOWORK = "off"
go test ./...
Remove-Item Env:GOWORK
```

Nếu đã có biến `GOWORK` trong môi trường, hãy lưu và khôi phục giá trị thay vì xóa bỏ nó; đoạn trên minh họa phiên làm việc chưa đặt biến đó. Bài thực hành không cần phụ thuộc thực tế trên mạng: tạo hai mô-đun cục bộ, cho mô-đun A nhập khẩu mô-đun B, dùng không gian làm việc để kiểm thử, rồi tắt không gian làm việc. Hãy giải thích vì sao kết quả thay đổi, và điều gì bắt

buộc phải xuất hiện trong `go.mod` hoặc phiên bản phát hành để bên tiêu dùng độc lập có thể biên dịch thành công. Đừng bao giờ sửa lỗi bằng chỉ thị `replace` trỏ vào đường dẫn máy tính cá nhân rồi coi sản phẩm đó là có thể tái lập.

Bộ công cụ đang chạy và phiên bản ngôn ngữ của mô-đun là hai khái niệm độc lập. Trong Go 1.27, lệnh `go test` mặc định kích hoạt kiểm tra vet `stdversion`, giúp phát hiện các biểu tượng thư viện chuẩn quá mới so với phiên bản có hiệu lực của tệp, sau khi đối chiếu cả `go.mod` và các thẻ chỉ thị biên dịch. Việc cài đặt bộ công cụ mới không tự động hợp thức hóa việc sử dụng các API mới trong một mô-đun vốn cam kết tương thích với phiên bản cũ. Lệnh `go mod tidy` của mô-đun `go 1.27` còn gom các khối khai báo `require` trùng lặp thành nhóm trực tiếp và gián tiếp; khác biệt đó là thay đổi về hình thức biểu diễn phụ thuộc, không phải bằng chứng đã nâng cấp hay tích hợp phụ thuộc thành công.

Build là phép chọn tệp trước khi compile

Hai tệp trong cùng một thư mục chưa chắc cùng được biên dịch vào một tệp nhị phân. Quy ước đặt tên như `probe_linux.go`, hậu tố kiến trúc và chỉ thị dòng `//go:build` điều khiển việc chọn lựa mã nguồn. Ràng buộc biên dịch (**build constraint**) bắt buộc phải đặt ở đầu tệp, đứng trước khai báo gói và cách biệt bằng một dòng trống. Đây là bước chọn lọc tệp của bộ công cụ; trình biên dịch vẫn kiểm tra các tệp đã chọn theo ngữ nghĩa ngôn ngữ chuẩn mực.

Với một hàm trợ giúp thuần Go, người lập trình có thể đặt hai bản cài đặt có cùng chữ ký hàm trong các tệp phân tách theo hệ điều hành. Lệnh `go list -json .` hiển thị rõ các trường `GoFiles`, `IgnoredGoFiles` và `CgoFiles`; thông tin này hữu ích hơn nhiều so với việc phán đoán cảm tính từ cây thư mục. Biên dịch chéo (**cross-compilation**) thay đổi mục tiêu biên dịch, chứ không giúp máy tính Windows thực thi hay kiểm chứng các lời gọi hệ thống (**syscall**) của Linux:

```
$env:GOOS = "linux"
$env:GOARCH = "amd64"
$env:CGO_ENABLED = "0"
go build -o probe-linux ./cmd/opsprobe
```

Thực thi trong một phiên lệnh tạm thời hoặc khôi phục ba biến môi trường về giá trị trước đó. Việc biên dịch thành công mới chỉ chứng minh trình biên dịch và trình liên kết (**linker**) chấp nhận tập hợp mã nguồn cho mục tiêu đó, chứ chưa chứng minh tệp nhị phân sẽ thực thi chính xác trên môi trường đích. Khi sử dụng cgo, hệ thống còn đòi hỏi trình biên dịch C và các thư viện phù hợp với nền tảng đích; việc tắt cgo không tạo ra một bản biên dịch chéo tương đương nếu các chức năng dựa trên C bị loại bỏ khỏi mã nguồn. Chương 14 phân tích giao ước ở ranh giới C, Chương 17 kiểm tra tiến trình trong môi trường triển khai thực tế, còn Chương 26 xác minh nguồn gốc (**provenance**) và lỗ hổng bảo mật của sản phẩm đóng gói. Mỗi bước giải đáp một câu hỏi kỹ thuật riêng biệt.

Điểm dừng: package đủ nhỏ để thay đổi

Chưa có một “kiến trúc hoàn chỉnh” phức tạp nào ở đây: chưa phân tích tệp cấu hình, chưa dùng kiểu tổng quát (**generics**), chưa tạo gói riêng cho từng danh từ. Chúng ta chỉ sở hữu một đồ thị phụ thuộc mà mỗi mũi tên đều giải thích rõ ràng “vì sao gói này cần biết đến gói kia?”. Đó là cấu trúc tinh gọn vừa đủ để tiếp tục thay đổi mã nguồn mà không biến mọi sửa đổi thành việc chắp vá hàm `main`.

Chương 6 — Thay đổi không sợ hãi

Ở cuối Chương 5, công cụ `opsprobe` đã biết lấy một điểm cuối (**endpoint**) từ cấu hình và gọi `probe.Check`. Nhưng một yêu cầu rất bình thường vẫn khó chứng minh: khi người vận hành đặt `OPS_PROBE_TARGET=payments.internal:9443`, điểm cuối nào thực sự đi tới hàm thực thi (**runner**)? Chạy lệnh dòng lệnh và nhìn dòng `billing healthy=true` không trả lời được câu hỏi đó. Dòng đó không in tên máy chủ hay cổng; kể cả có in, một lần chạy thử thủ công vẫn không phải giao ước (**contract**) mà mã nguồn bảo toàn được khi tiếp tục thay đổi.

Đây là lúc nhiều dự án đi sai hướng. Có người viết kiểm thử gọi trực tiếp hàm `main`, sửa biến môi trường của tiến trình chung, chụp luồng xuất chuẩn, rồi hy vọng không chạm vào lệnh `os.Exit`. Có người khác bỏ qua kiểm thử vì nghĩ rằng "mã dòng lệnh chỉ có vài dòng". Cả hai đều né tránh câu hỏi thiết kế: phần nào là logic điều phối (**orchestration**) có thể kiểm chứng, phần nào là ranh giới tiến trình (**process boundary**) chỉ nên giữ thật mỏng và rõ ràng?

Một kiểm thử khói không biết điều ta cần biết

Hàm `main` hiện đang gánh quá nhiều trách nhiệm: đọc môi trường thực tế, nạp cấu hình, gọi hàm thăm dò, quyết định mã thoát và in ra thiết bị đầu cuối. Một ca kiểm thử gọi nó chỉ quan sát được một nhánh luồng chạy thử. Với thiết kế hiện tại, kiểm thử rất khó truyền môi trường giả lập vào một cách an toàn, không thể quan sát điểm cuối mà hàm thực thi nhận được, hoặc không thể bóc tách lỗi cấu hình khỏi lỗi thăm dò mà không kích hoạt các tác dụng phụ của tiến trình.

Vấn đề không phải bản thân `main` được viết tệ. Các lệnh `os.LookupEnv`, xuất ra luồng chuẩn và `os.Exit` đều có vị trí phù hợp: chúng là ranh giới nơi chương trình tiếp xúc với tiến trình hệ điều hành. Vấn đề là toàn bộ logic cần xử lý trước khi chạm tới ranh giới ấy chưa được tổ chức thành một đơn vị độc lập để kiểm thử có thể kích hoạt.

Chúng ta tạo gói nội bộ `internal/app`. Gói này không thay thế `probe` hay `config`, mà đảm nhiệm việc điều phối hai gói đó. Hàm `Run` tiếp nhận đúng

hai quan hệ phụ thuộc (**dependency**) có thể hoán đổi trong kiểm thử: hàm đọc môi trường và hàm thực thi thăm dò. Nó trả về kết quả cấu trúc (**outcome**) thay vì in trực tiếp ra màn hình:

```
type Outcome struct {
    Service string
    Err     error
}

func Run(
    ctx context.Context,
    lookup config.LookupEnv,
    runner probe.Runner,
) ([]Outcome, error)
```

Trường `Outcome.Err` đại diện cho sự cố của một mục tiêu đã được cấu hình; công cụ dòng lệnh có thể in lỗi đó ra rồi tiếp tục thăm dò mục tiêu kế tiếp. Ngược lại, giá trị `error` trả về độc lập đại diện cho thất bại nghiêm trọng không thể tạo ra danh sách kết quả có ý nghĩa, chẳng hạn như cấu hình bất hợp lệ hoặc toàn bộ lượt chạy bị hủy bỏ. Việc phân tách hai nhánh xử lý này không nhằm mục đích tạo kiểu cho đẹp mắt: bên gọi có các chính sách xử lý hoàn toàn khác nhau. Chương 4 từng đặt câu hỏi "bên gọi cần quyết định điều gì?"; ở đây câu trả lời sẽ định hình trực tiếp kiểu dữ liệu trả về.

Tái cấu trúc đến khi kiểm thử có điểm tựa vững chắc

Hàm `Run` nạp mục tiêu, ánh xạ cấu hình `config.Target` sang `probe.Service`, gọi `probe.Check`, rồi lưu giữ từng kết quả trong một giá trị `Outcome`. Nó không trực tiếp đọc `os.LookupEnv`, không gọi `fmt.Printf` và tuyệt đối không gọi `os.Exit`. Các tác dụng phụ đó thuộc về tệp `cmd/opsprobe/main.go`:

```
outcomes, err := app.Run(
    context.Background(),
    os.LookupEnv,
    runner,
)
if err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(2)
}

for _, outcome := range outcomes {
    if outcome.Err != nil {
        fmt.Printf("%s: %v\n", outcome.Service, outcome.Err)
        continue
    }
    fmt.Printf("%s healthy=true\n", outcome.Service)
}
```

Đoạn mã `main` vẫn duy trì logic tối thiểu: chính sách trình bày và mã thoát tiến trình hiển lộ rõ ràng, rất dễ duyệt xét. Nhưng kiểm thử không cần phải gọi tới nó. Kiểm thử nằm ngay cạnh `Run`, truyền một hàm đóng kín (**closure**) thay thế cho `LookupEnv` và một hàm `probe.Runner` ghi nhận điểm cuối được gọi. Không hề có socket mạng, biến môi trường toàn cục hay thiết bị đầu cuối nào bị can thiệp.

Trước khi viết lời khẳng định kiểm thử (**assertion**), hãy phân định rạch ròi ba phạm vi bằng mắt. Điều này ngăn việc biến ca kiểm thử thành bản sao chép thụ động của mã triển khai:

Bảng 3 — Phạm vi của unit test quanh `Run`

Test kiểm soát	Test quan sát	Test không cần chạm
`LookupEnv` trả target nào; runner thành công hay lỗi	endpoint runner nhận; `Outcome` hoặc error trả về	environment process, stdout, stderr, `os.Exit`

Nếu một yêu cầu nghiệp vụ chỉ xuất hiện ở cột cuối cùng, hàm `Run` chưa phải đơn vị phù hợp để kiểm thử; đó là nhiệm vụ của kiểm thử tích hợp (**integration test**) vào thời điểm cần thiết. Nếu nó nằm ở hai cột đầu, kiểm thử đơn vị (**unit test**) có thể cung cấp phản hồi cực nhanh mà không cần giả lập toàn bộ tiến trình hệ điều hành.

```
func TestRunMapsEnvironmentTargetToRunner(t *testing.T) {
    var got probe.Endpoint

    outcomes, err := Run(context.Background(), lookup(
        "OPS_PROBE_TARGET", "payments.internal:9443",
    ), func(_ context.Context, endpoint probe.Endpoint) error {
        got = endpoint
        return nil
    })

    if err != nil {
        t.Fatalf("Run() error = %v", err)
    }
    if got != (probe.Endpoint{
```

```
        Host: "payments.internal",
        Port: 9443,
    }) {
        t.Fatalf("runner endpoint = %+v", got)
    }
    if len(outcomes) != 1 {
        t.Fatalf("outcomes = %+v", outcomes)
    }
    if outcomes[0].Err != nil ||
        outcomes[0].Service != "billing" {
        t.Fatalf("outcome = %+v", outcomes[0])
    }
}
```

Đây là một ca kiểm thử đơn vị chuẩn mực không phải vì nó ngắn gọn, mà vì nó kiểm chứng một đơn vị có ranh giới khoanh vùng hẹp. Ca kiểm thử đặt ra một câu hỏi hoàn toàn có thể quan sát được: dữ liệu cấu hình đầu vào có truyền tới hàm thực thi chính xác hay không, và kết quả có gắn đúng với dịch vụ tương ứng hay không. Nó không săm soi chi tiết triển khai nội bộ như việc hàm `Run` khai báo bao nhiêu biến cục bộ hay viết vòng lặp bằng cú pháp nào.

Hai loại lỗi, hai chính sách ứng xử

Hãy bổ sung thêm hai ca kiểm thử trước khi đọc toàn bộ phần triển khai. Với cấu hình `OPS_PROBE_TARGET=payments.internal:0`, gói cấu hình phải báo lỗi ngay trước khi hàm thực thi được kích hoạt. Đây là dữ liệu đầu vào bất hợp lệ của toàn bộ lượt chạy, do đó `Run` trả về lỗi và không tạo ra bất kỳ kết quả dở dang nào. Trong khi đó, với hàm thực thi trả về `errors.New("connection refused")`, mục tiêu đã tồn tại và thao tác đã bắt đầu; hàm `Run` sẽ lưu giữ `Outcome{Service: "billing", Err: ...}` để công cụ có thể báo một dòng lỗi rồi tiếp tục thăm dò các mục tiêu kế tiếp khi danh sách sau này mở rộng.

Tín hiệu hủy bỏ (**cancellation**) lại khác biệt hoàn toàn với cả hai trường hợp trên. Phương thức `probe.Check` bảo toàn danh tính lỗi `context.Canceled`, trong khi hàm `Run` dừng vòng lặp thay vì biến việc hủy bỏ toàn bộ lượt chạy thành kết quả thất bại cục bộ của từng dịch vụ. Nhìn từ bên ngoài, ba chính sách phân định rõ ràng:

Bảng 4 — Policy cho lỗi cấu hình, lỗi probe và cancellation

Tình huống	`Run` trả gì	Command nên làm gì?
Config không hợp lệ	`error`	In stderr, exit 2, không gọi runner.
Một probe thất bại	`Outcome.Err`	Báo service đó; policy hiện tại cho phép đi tiếp.
Context bị cancel	`error` giữ identity	Dừng run; không ghi health signal giả.

Không phải ca kiểm thử nào cũng cần dùng đối tượng giả lập. Bộ phân tích cấu hình đã sở hữu các bài kiểm thử đầu vào/đầu ra chuyên biệt ở Chương 5; phương thức `probe.Check` cũng đã có kiểm thử cho cơ chế hủy bỏ và bọc lỗi. Hàm `Run` chỉ cần kiểm thử phần liên kết ba giao ước đó lại với nhau. Nếu một ca kiểm thử chỉ lặp lại y hệt các khẳng định của gói tầng dưới, nó sẽ gây cản trở cho quá trình tái cấu trúc mà không hề gia tăng độ tin cậy của hệ thống.

Bài thực hành: Tách bỏ và tái thiết lập

Mở `labs/part6-testable-command`, đọc `internal/app/run_test.go` trước. Đừng chạy lệnh vội; hãy dự đoán mỗi kiểm thử đang bảo vệ chính sách nào. Sau đó tạm đổi tên `Run` để trình biên dịch báo các điểm mà kiểm thử đang đòi hỏi khế ước, rồi tự dựng lại hàm từ kiểm thử và các kiểu dữ liệu đã có. Khi kiểm thử đơn vị thành công, chạy `go run ./cmd/opsprobe`. Cuối cùng đổi runner trong `main` thành một runner trả lỗi và xác minh CLI in tên dịch vụ thay vì nuốt chửng lỗi thất bại. Khôi phục runner thành công trước khi rời bài lab.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Hàm `Run` cần gọi `config.LoadTargets` đúng một lần trước vòng lặp, ánh xạ từng `Target` sang `probe.Service`, rồi nối kết quả thành công hoặc thất bại của từng kiểm tra. Chỉ lỗi cấu hình và tín hiệu hủy bỏ mới thoát khỏi `Run` qua lỗi trả về. Hàm `main` giữ môi trường thực tế, dữ liệu xuất ra và mã thoát. Lời giải không phải là "dùng khung giả lập mock"; hai giá trị hàm hiện có đã là điểm ghép nối vừa đủ.

Khi một test case che mất các policy khác

Ca kiểm thử đầu tiên của `Run` cố ý chỉ dùng một trường hợp ghi đè hợp lệ. Nó trả lời câu hỏi khi tái cấu trúc (**refactor**): điểm cuối (**endpoint**) có thực sự truyền tới runner hay không? Nhưng nếu cứ nhân bản kiểu kiểm thử đó cho từng dữ liệu đầu vào xấu, bộ kiểm thử sẽ dần biến thành một chuỗi dài các hàm gần như giống hệt nhau. Lúc ấy người đọc không còn nhìn thấy các quy tắc của cấu hình; họ chỉ thấy rất nhiều lần gọi lặp lại `LoadTargets`.

Với `OPS_PROBE_TARGET`, chính sách của lab là: biến vắng mặt hoặc có chuỗi rỗng dùng cấu hình mặc định; chuỗi không rỗng phải tách được `host:port`, có host và port trong khoảng 1–65535. Một giá trị `found == true, raw == ""` không bị từ chối trong code hiện tại. Bảng kiểm thử phải có hàng riêng cho trường hợp đó, thay vì gọi mọi biến "có mặt" là một override hợp lệ. Những hàng khác nhau bảo vệ quyết định khác nhau, không chỉ làm tăng coverage.

Đây là lúc kỹ thuật kiểm thử hướng bảng (**table-driven test**) phát huy giá trị. Không phải vì cộng đồng Go áp đặt một nghi thức cố định, mà vì bảng cấu trúc làm hiển lộ rõ ràng phần dữ liệu thay đổi và phần logic giữ nguyên. Các giá trị `raw`, `found` và `want` thay đổi theo từng ca kiểm thử; trong khi cách thức gọi bộ phân tích cú pháp (**parser**) và lời khẳng định kiểm thử (**assertion**) về một mục tiêu duy nhất được dùng chung:

```
for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        targets, err := LoadTargets(
            func(string) (string, bool) {
                return tt.raw, tt.found
            },
        )
        if err != nil {
            t.Fatalf("LoadTargets() error = %v", err)
        }
        if len(targets) != 1 || targets[0] != tt.want {
            t.Fatalf("LoadTargets() = %+v", targets)
        }
    })
}
```

Hàm `t.Run` không chỉ giúp kết quả in ra trông có tổ chức. Khi một ca kiểm thử thất bại, lệnh `go test -v` sẽ chỉ rõ chính xác ca nào bị lỗi, thay vì buộc người kiểm tra phải đếm số thứ tự từng hàng trong lát cắt (**slice**). Do đó, trường tên ca kiểm thử cần mô tả hành vi rõ ràng: đặt tên “port zero” tốt hơn “case 4”, còn “uses the documented default” giúp nhắc nhở người bảo trì rằng giá trị mặc định là một phần trong giao ước (**contract**).

Đánh giá một bảng kiểm thử trước khi đặt niềm tin

Thêm các dữ liệu đầu vào bị từ chối vào một bảng kiểm thử thứ hai trong tệp `internal/config/config_test.go`:

```
tests := []struct {
    name string
    raw  string
}{
    {name: "missing port", raw: "payments.internal"},
    {name: "empty host", raw: ":8443"},
    {name: "non-numeric port", raw: "payments.internal:https"},
    {name: "port zero", raw: "payments.internal:0"},
    {
        name: "port above TCP range",
        raw:  "payments.internal:65536",
    },
}
```

Điểm dễ bỏ sót không nằm ở vòng lặp `for`, mà ở lời khẳng định kiểm thử (**assertion**). Ca kiểm thử này chỉ bảo đảm rằng từng dữ liệu không hợp lệ đều bị từ chối; nó không bảo đảm nguyên văn thông điệp lỗi (**error message**). Thông điệp lỗi là chi tiết kỹ thuật có thể hiệu chỉnh để hỗ trợ tốt hơn cho người vận hành. Quy tắc "dữ liệu đầu vào này không được phép tạo ra mục tiêu" mới chính là giao ước cốt lõi. Nếu giao diện dòng lệnh (**CLI**) hoặc giao diện lập trình (**API**) cần phân loại lỗi tự động bằng chương trình, khi đó mới phát sinh lý do thiết kế kiểu lỗi chuyên biệt (**custom error type**) hoặc lỗi lính gác (**sentinel error**); không nên dùng phép so sánh chuỗi thô sơ để thay thế cho thiết kế kiến trúc chuẩn mực.

Với phiên bản Go 1.27.1 này, biến vòng lặp khai báo bằng `:=` sở hữu một biến riêng biệt ở mỗi lần lặp (ngữ nghĩa bắt đầu từ Go 1.22), nên hàm bao đóng (**closure**) không còn vô tình giữ chung một biến `tt` như tập quán Go cũ. Tuy nhiên, một cạm bẫy con trỏ khác vẫn tồn tại: biểu thức `&tt` trỏ tới bản sao của lần lặp đó, chứ không phải phần tử thật trong lát cắt `tests`. Nếu mục tiêu là biến đổi trực tiếp phần tử gốc, hãy lặp lấy chỉ số rồi dùng `&tests[i]`. Ở đây hàm `t.Run` chạy tuần tự đồng bộ và hàm bao đóng chỉ đọc giá trị `tt`, nên con trỏ và vòng đời của goroutine đều không phải vấn đề mà chính sách bộ phân tích cần bận tâm.

Quy tắc biến vòng lặp còn phụ thuộc phiên bản ngôn ngữ có hiệu lực của module hoặc tệp, không chỉ phiên bản toolchain được cài. Lab khai báo `go 1.27`, nên dùng ngữ nghĩa mới. Ngay cả với biến riêng mỗi vòng, `t.Parallel()` không nhân bản map, pointer hay phụ thuộc toàn cục mà các case cùng sử dụng. Chỉ bật nó khi từng case có dữ liệu và tài nguyên độc lập; `t.Setenv` không được dùng trong test parallel hoặc có tổ tiên parallel vì môi trường thuộc cả tiến trình.

Có một điểm dễ làm test xanh giả: `t.Run` chỉ chờ tới khi subtest gọi `t.Parallel`, không đợi phần còn lại của subtest lúc ấy. Subtest parallel tiếp tục sau khi hàm cha trở về. Nếu cần kiểm tra tổng hợp sau tất cả case, hãy đặt chúng trong một nhóm `t.Run("group", ...)` rồi kiểm tra sau khi nhóm trả về; dùng `t.Cleanup` của nhóm để dọn tài nguyên, không dùng defer của cha làm tài nguyên biến mất trước lúc con chạy. Test `TestParallelSubtestsWaitForTheirGroup` ghi nhận thứ tự bằng primitive đồng bộ, không bằng sleep hay phỏng đoán scheduler.

Bài thực hành: Ôn tập phân loại dữ liệu đầu vào

Trước khi chạy kiểm thử, hãy tự phân loại năm hàng trên: hàng nào bị `net.SplitHostPort` từ chối, hàng nào đi qua bộ phân tích nhưng bị bước xác thực (**validation**) của sách từ chối? Sau đó thêm một ca hợp lệ cho IPv6 theo dạng `[2001:db8::10]:443`. Khẳng định kiểm thử của nó phải xác minh `Host` không còn dấu ngoặc vuông và `Port == 443`, thay vì chỉ kiểm tra sơ sài `err == nil`.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Trường hợp thiếu cổng và `:8443` không tạo được mục tiêu hợp lệ ở bước phân tích cú pháp hoặc kiểm tra tên máy chủ. Các giá trị `https`, `0` và `65536` đi xa hơn: chúng đòi hỏi chuyển đổi kiểu hoặc xác thực phạm vi số. Với địa chỉ IPv6, hàm `net.SplitHostPort` tách chuỗi có ngoặc thành máy chủ `2001:db8::10` và cổng `443`; do đó ca kiểm thử nên ghi nhận điều đó thành giao ước rõ ràng. Khi bảng kiểm thử bắt đầu chứa các cột không cùng chung một quy tắc, hoặc mỗi hàng đòi hỏi thiết lập quá khác biệt, việc tách thành các hàm kiểm thử riêng biệt sẽ sáng sủa hơn là cố nhồi nhét thêm cột.

Chạy đúng bằng chứng

Khi bộ kiểm thử báo xanh, lệnh `go test ./...` trả lời câu hỏi có giá trị bao quát: có gói nào vi phạm giao ước (**contract**) hay không? Tuy nhiên, khi điều tra một báo cáo sự cố cụ thể, việc chạy lặp đi lặp lại toàn bộ kiểm thử sẽ làm phân tán sự tập trung. Nếu người vận hành phản ánh rằng cổng `0` từng lọt qua bước xác thực (**validation**), ta cần khoanh vùng kiểm tra đúng quy tắc đó trước, rồi mới mở rộng phạm vi.

Từ thư mục `labs/part6-testable-command`, lệnh sau chọn một ca kiểm thử con bằng đường dẫn định danh của nó:

```
$case = 'TestLoadTargetsRejectsMalformedOverrides/port_zero'
go test -run $case -v ./internal/config
```

Đây không phải đường tắt để né tránh việc kiểm thử đầy đủ trước khi lưu mã (**commit**). Nó là chiếc kính lúp trong quá trình gỡ lỗi: kết quả in ra giữ nguyên tên ca kiểm thử cha và ca con `port_zero`, giúp người đọc nhận biết chính xác chính sách nào đang được đánh giá. Sau khi sửa lỗi, hãy chạy lại kiểm thử của gói rồi mới chạy toàn bộ hệ thống; lỗi từ bộ phân tích cú pháp có thể khiến ứng dụng dừng hoạt động trước cả khi hàm thực thi được gọi.

Hãy thử thay đổi tạm thời điều kiện phạm vi trong `LoadTargets` để cổng `0` đi qua, chỉ nhằm quan sát ca kiểm thử con bị phá vỡ như thế nào. Đừng lưu thay đổi mang tính thử nghiệm đó. Bài học không phải là học thuộc cờ lệnh `-run`; nó là việc phân biệt giữa vòng lặp điều tra ngắn gọn với bộ bằng chứng toàn diện cần có trước khi sáp nhập một thay đổi vào mã nguồn.

Từ đây Chương 6 đã phân định hai loại kiểm thử: hàm `Run` kiểm chứng việc liên kết các ranh giới (**boundary**), trong khi các ca kiểm thử con của cấu hình duy trì một ma trận đầu vào nhỏ gọn nhưng mang giá trị nghiệp vụ thiết thực. Phần tiếp theo sẽ tạo ra một thất bại mà kiểm thử đơn vị tuần tự báo xanh vẫn hoàn toàn bỏ lọt: hai luồng nhẹ (**goroutine**) cùng chạm vào một vùng dữ liệu mà không có cơ chế đồng bộ.

Một test xanh vẫn có thể bỏ sót race

`opsprobe` hiện chưa chạy các tác vụ thăm dò song song; đó là lựa chọn có chủ đích khi chúng ta còn đang tập trung vào ranh giới và chính sách xử lý lỗi. Vì vậy sẽ không thực tế nếu giả định rằng vừa phát hiện một cuộc đua dữ liệu (**data race**) trong mã nguồn của nó. Thay vào đó, hãy cô lập đúng thất bại cần học: nhiều tiến trình xử lý (**worker**) cùng ghi vào một bộ đếm hoàn thành. Đây là loại lỗi có thể trông vô hại khi chạy thử một lần, vì trình biên dịch không báo lỗi và kiểm thử đơn vị tuần tự vẫn vượt qua bình thường.

Cuộc đua dữ liệu (**data race**) xảy ra khi nhiều goroutine truy cập cùng một biến đồng thời, có ít nhất một thao tác ghi, và hoàn toàn thiếu cơ chế đồng bộ hóa phù hợp. Đây là định nghĩa chặt chẽ về hành vi, không phải phán đoán cảm tính dựa trên việc kết quả đếm cuối cùng "có vẻ đúng". Thao tác `successes++` không phải là một phép toán nguyên tử (**atomic operation**) ở cấp mã nguồn; nó đọc giá trị cũ, tạo giá trị mới rồi ghi đè lại. Hai goroutine hoàn toàn có thể xen ngang vào giữa các bước đó.

Bài thực hành `labs/part6-race-detector/broken` chứa một bộ đếm cố ý viết sai. Ca kiểm thử tạo 128 goroutine, mỗi goroutine gọi đúng một lần `RecordSuccess`:

```
type Counter struct {
    successes int
}

func (c *Counter) RecordSuccess() {
    c.successes++
}
```

Chạy kiểm thử thông thường có thể không phát hiện dấu hiệu đáng ngờ nào. Để đưa hành vi truy cập bộ nhớ vào phạm vi quan sát, hãy chạy mã mẫu kiểm thử (**fixture**) cùng bộ phát hiện tương tranh (**race detector**):

```
go test -race -tags raceexercise ./broken
```

Lệnh này được dự kiến là thất bại và in ra cảnh báo `WARNING: DATA RACE`. Đó là thất bại có chủ ý của bài thực hành, không phải một bước kiểm tra

được phép báo xanh. Bộ phát hiện tương tranh ghi lại vết ngăn xếp (**stack trace**) tại các điểm truy cập xung đột và vị trí các goroutine liên quan được khởi tạo. Khi đọc báo cáo chẩn đoán, hãy lần theo ba câu hỏi: biến nào được dùng chung, thao tác nào là ghi dữ liệu, và cơ chế đồng bộ hóa nào đáng lẽ phải xác lập thứ tự giữa chúng?

Bảng 5 — Ba phần hữu ích nhất của một race report

Phần report	Nó trả lời gì	Đừng kết luận quá mức
`Read by goroutine` hoặc `Write by goroutine`	Access hiện tại nằm ở dòng nào?	Dòng đó chưa chắc là nguồn gốc policy sai.
`Previous ... by goroutine`	Access xung đột nào chồng lên nó?	Thứ tự in ra không phải timeline đầy đủ của chương trình.
`created at`	Goroutine nào mở đường cho access này?	Một report không chứng minh mọi đường chạy đều racy.

Bộ phát hiện tương tranh (**race detector**) là công cụ thu thập bằng chứng khi mã đã chạy thực tế, không phải chứng minh toán học tuyệt đối rằng chương trình hoàn toàn không có tương tranh. Một nhánh luồng chưa được kiểm thử sẽ không kích hoạt truy cập bộ nhớ để công cụ quan sát. Do đó, cờ lệnh `go test -race` phải song hành cùng các ca kiểm thử chặt chẽ và khối lượng công việc có ý nghĩa; nó không thể thay thế cho tư duy thiết kế đồng thời đúng đắn.

Sửa chữa bất biến, không chỉ làm bộ phát hiện im lặng

Trước khi thêm `sync.Mutex`, hãy phát biểu đặc tính bất biến (**invariant**) bằng một câu duy nhất: `Successes` phải trả về đúng số lần `RecordSuccess` đã hoàn tất, kể cả khi nhiều worker đang chạy đồng thời. Cấu trúc `Counter` vì thế mang thêm một trường `mu sync.Mutex` cạnh `successes`. Khóa bảo vệ (**lock**) không phải một nghi thức huyền bí; nó là ranh giới phân định bảo vệ hai thao tác truy cập vào trạng thái dùng chung:

```
func (c *Counter) RecordSuccess() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.successes++
}

func (c *Counter) Successes() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.successes
}
```

Phiên bản này nằm trong thư mục `labs/part6-race-detector/fixed`. Ca kiểm thử của nó chờ mọi worker kết thúc bằng `sync.WaitGroup`, sau đó kiểm tra số lượng đếm đạt đúng 128; lệnh `go test -race ./fixed` phải báo xanh. `WaitGroup` ở đây chỉ có nhiệm vụ giúp ca kiểm thử nhận biết khi nào các worker hoàn tất. Bản thân nó không thể tự bảo vệ biến `successes`; khóa loại trừ tương hỗ (**mutex**) mới đảm nhận trọng trách đó. Phân biệt rõ hai trách nhiệm này ngay từ đầu sẽ giúp tránh được câu hỏi bối rối sau này: "Đã gọi `Wait` rồi, sao vẫn xảy ra xung đột dữ liệu?"

Khóa loại trừ tương hỗ (`sync.Mutex`) và thứ tự bộ nhớ (**memory ordering**) sẽ được đào sâu ở phần xử lý đồng thời (**concurrency**). Ở Chương 6, trọng tâm là: một ca kiểm thử sở hữu khẳng định kiểm thử (**assertion**) đúng đắn vẫn có thể chưa quan sát được cách thức trạng thái bị xung đột truy cập. Nếu yêu cầu bài toán đưa luồng nhẹ vào mã nguồn, việc bổ sung cờ `-race` vào bộ bằng chứng chấp nhận thay đổi là một quyết định thiết kế vì khả năng kiểm thử (**testability**), không phải hình thức trang trí.

Bài thực hành: Điều tra cuộc đua truy cập bộ nhớ (race condition)

Chạy mã mẫu `broken` một lần với cờ `-race`, rồi mở tệp `fixed/counter_test.go`. Trước khi chạy phiên bản đã sửa, hãy khoanh vùng hai phương thức cùng truy cập vào `successes` và giải thích vì sao ngay cả phương thức đọc dữ liệu cũng phải khóa bảo vệ. Sau đó thử bỏ thao tác khóa riêng trong `Successes`; ca kiểm thử hiện tại vẫn có thể báo xanh vì nó đọc kết quả sau lời gọi `wg.Wait`, nhưng phương thức công khai đã đánh mất giao ước an toàn dành cho bên gọi đồng thời.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Phương thức `RecordSuccess` thực hiện ghi còn `Successes` thực hiện đọc trên cùng một trường dữ liệu, do đó cả hai bắt buộc phải đi qua cùng một khóa mutex nếu giao diện cam kết an toàn khi sử dụng đồng thời. Lời gọi `wg.Wait` chỉ tạo điểm chờ đồng bộ trong nội bộ ca kiểm thử hiện tại; nó hoàn toàn không thay thế được cơ chế đồng bộ hóa trong giao diện lập trình. Mã mẫu bị hỏng được gắn thẻ phân loại riêng để kho mã nguồn vẫn duy trì bộ kiểm thử mặc định báo xanh, trong khi lệnh kiểm tra cuộc đua dữ liệu được thực thi như một thử nghiệm có kiểm soát mà thất bại chính là kết quả mong đợi.

Kiểm thử mờ (Fuzzing) khi đặc tính bất biến đã rõ ràng

Các ca kiểm thử con của cấu hình đã định danh rõ những giá trị biên mà chúng ta chủ động lựa chọn: cổng `0`, tên máy chủ rỗng, cổng vượt quá giới hạn. Tuy nhiên, bộ phân tích cú pháp tiếp nhận một chuỗi `string`; không ai có thể lường trước mọi tổ hợp dấu ngoặc, ký tự Unicode, dấu hai chấm hay các byte dị thường mà dữ liệu đầu vào từ môi trường có thể mang theo. Kỹ thuật kiểm thử mờ (**fuzzing**) hỗ trợ sinh thêm dữ liệu đầu vào tự động, nhưng chỉ thực sự có giá trị khi mục tiêu kiểm thử biết rõ cần đòi hỏi điều kiện gì ở mọi dữ liệu sinh ra đó.

Đặc tính bất biến (**property**) của `LoadTargets` không phải là "mọi chuỗi ký tự đều được phân tích cú pháp thành công". Phần lớn chuỗi bất hợp lệ bắt buộc phải bị từ chối. Đặc tính đúng đắn là: nếu `LoadTargets` trả về thành công, nó phải trả về đúng một mục tiêu có máy chủ không rỗng và cổng nằm trong khoảng 1–65535. Điều này kết hợp rất chặt chẽ với kiểm thử hướng bảng: bảng kiểm thử xác lập các trường hợp then chốt nào phải bị từ chối hoặc ánh xạ chính xác; trong khi kỹ thuật kiểm thử mờ tìm kiếm xem còn dữ liệu đầu vào nào khiến mã nguồn tự mâu thuẫn với bất biến đó hay không.

Đừng thay thế đặc tính ấy bằng khẳng định mơ hồ như "hàm không phát sinh hoảng loạn (`panic`)". Không hoảng loạn là tiêu chuẩn vệ sinh mã nguồn cơ bản cần có, nhưng một bộ phân tích cú pháp hoàn toàn có thể bình thản trả về `Target{Host: "", Port: 0}` mà không `panic`, để rồi

phá vỡ hoạt động của lệnh ở tầng sâu hơn. Một đặc tính kiểm thử mạnh mẽ buộc ca kiểm thử phải soi xét kỹ kết quả khi thành công. Ngược lại, đừng ép bộ kiểm thử mờ phải đoán đúng từng câu chữ của thông điệp lỗi hay bắt mọi dữ liệu đầu vào xấu phải rơi vào đúng một nhánh mã cụ thể; những đòi hỏi ấy chỉ khiến ca kiểm thử trở nên mong manh mà không nâng cao sự thấu hiểu về giao ước.

Trong thực tế có nhiều họ đặc tính kiểm thử: kết quả luôn bảo toàn bất biến nghiệp vụ, quy trình mã hóa rồi giải mã (**encode/decode**) bảo toàn dữ liệu tương đương (**round-trip**), hoặc hai cách biểu diễn khác nhau phải luôn đồng thuận kết quả. Ở đây chúng ta áp dụng họ đặc tính đầu tiên vì bộ phân tích cú pháp này chưa có định dạng xuất dữ liệu để thực hiện đối chiếu hai chiều. Việc chọn lựa đặc tính bám sát giao diện lập trình hiện có giúp kiểm thử mờ phục vụ đắc lực cho thiết kế, thay vì biến tướng thành việc chạy thử dữ liệu ngẫu nhiên vô nghĩa.

```
func FuzzLoadTargetsNeverReturnsInvalidTarget(f *testing.F) {
    for _, seed := range []string{
        "",
        "billing.internal:8443",
        "[2001:db8::10]:443",
        "payments.internal:0",
        "not a target",
    } {
        f.Add(seed)
    }

    f.Fuzz(func(t *testing.T, raw string) {
        targets, err := LoadTargets(lookup(raw))
        if err != nil {
```

```
            return
        }
        if len(targets) != 1 {
            t.Fatalf("got %d targets", len(targets))
        }
        target := targets[0]
        if target.Host == "" ||
            target.Port < 1 ||
            target.Port > 65535 {
            t.Fatalf("invalid target %+v", target)
        }
    })
}
```

`f.Add` không nhằm cung cấp danh sách kiểm thử đầy đủ. Nó là tập hạt giống đầu vào (**seed corpus**) có chủ đích: một giá trị mặc định, một tên máy thông thường, một địa chỉ IPv6, một giá trị vượt ngưỡng phạm vi và một chuỗi sai hoàn toàn định dạng mục tiêu. Khi chạy `go test`, tập hạt giống này được thực thi như các ca kiểm thử thông thường. Khi kích hoạt cờ `-fuzz`, bộ công cụ sẽ biến đổi (**mutate**) tập dữ liệu đầu vào nhằm tìm kiếm những nhánh phủ mã mới. Nếu phát hiện trường hợp thất bại, Go sẽ tự động ghi lại dữ liệu đầu vào tái lập được để lần chạy sau kiểm chứng ngay mà không phụ thuộc vào yếu tố may rủi.

Từ `labs/part6-testable-command`, chạy một lượt có giới hạn thời gian:

```
go test -run=^$ `
  -fuzz=FuzzLoadTargetsNeverReturnsInvalidTarget `
  -fuzztime=2s ./internal/config
```

Tùy chọn `-run=^$` giúp bỏ qua các ca kiểm thử thông thường trong lượt chạy fuzz này; chúng vẫn được kiểm tra trong bộ kiểm thử định kỳ. Thời lượng `-fuzztime=2s` là hạn mức thời gian cho thử nghiệm cục bộ, không phải chứng chỉ cam kết bộ phân tích an toàn tuyệt đối. Mục tiêu fuzz (**fuzz target**) báo xanh chỉ xác nhận các dữ liệu đã thử nghiệm chưa phá vỡ đặc tính bất biến; nó không thay thế việc duyệt mã (**code review**), các ca kiểm thử biên có ý nghĩa hay việc xác thực giới hạn tài nguyên.

Bài thực hành: Thiết kế đặc tính bất biến cho kiểm thử fuzzer

Một đề xuất nghe có vẻ hợp lý là "nếu `err == nil`, trường `target.Name` luôn là `billing`". Hãy quyết định có nên đưa điều kiện đó vào mục tiêu fuzz này hay không. Nếu sau này cấu hình hỗ trợ nhiều dịch vụ khác nhau, khẳng định kiểm thử đó sẽ cản trở việc tái cấu trúc dù bộ phân tích cú pháp vẫn hoạt động hoàn toàn chính xác. Hãy giữ cho đặc tính kiểm thử bám sát nhất với giao ước cốt lõi hiện tại: cấu trúc hợp lệ của mục tiêu, không phải một chi tiết ngẫu nhiên của ca kiểm thử mẫu.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Điều kiện `Name == "billing"` là giao ước tạm thời của mẫu kiểm thử nhỏ, chứ không phải đặc tính bất biến mà bộ phân tích cần duy trì khi mô hình dữ liệu đầu vào mở rộng. Đặc tính kiểm thử mờ nên đủ chặt chẽ để bắt lỗi mục tiêu không hợp lệ và đủ tinh gọn để không đóng băng một chi tiết thiết kế chưa từng được cam kết. Khi kiểm thử mờ phát hiện thất bại thực sự, trước hết hãy chuyển dữ liệu đầu vào đó thành một ca kiểm thử hồi quy (**regression test**) dễ đọc; tập dữ liệu sinh ra là bằng chứng, còn ca kiểm thử với tên gọi tường minh là lời giải thích cho những kỹ sư tiếp quản sau này.

Một giới hạn khác nằm ngay trong property đang dùng: parser bỏ qua mọi override và luôn trả default vẫn thỏa host/port hợp lệ. Vì vậy, property này cần đi cùng bảng kiểm thử ánh xạ chính xác, không thay thế nó. Đối chứng âm trong evidence giữ nguyên fuzz target nhưng cố ý bỏ override: seed test vẫn xanh, còn case `maps a host and port from the override` đỏ. Đó là bằng chứng cụ thể về khoảng trống của test, không phải lý do thêm một assertion tùy tiện để khóa mọi thiết kế tương lai. Fake runner cũng chỉ chứng minh đối số và chính sách điều phối; nó không chứng minh DNS, kết nối hay dịch vụ thật đã hoạt động.

Đo câu hỏi trước khi tối ưu câu trả lời

Sau khi thấy bộ phân tích cú pháp có nhiều bước xác thực, một phản xạ quen thuộc là đề xuất thêm bộ nhớ đệm (**cache**) lưu kết quả: "lệnh `net.SplitHostPort` có thể chậm". Nhưng hàm `LoadTargets` hiện chỉ thực thi một lần duy nhất khi lệnh khởi tạo. Kể cả khi một bài kiểm chuẩn hiệu năng cho thấy việc ghi đè gây tốn bộ nhớ cấp phát, điều đó vẫn chưa biến nó thành điểm nghẽn cổ chai (**bottleneck**) của một công cụ dòng lệnh chỉ chạy vài thao tác thăm dò. Một bài kiểm chuẩn hiệu năng tốt bắt đầu bằng câu hỏi đủ cụ thể để kết quả có thể trực tiếp làm thay đổi một quyết định thiết kế.

Câu hỏi cần kiểm chứng ở đây là: một lượt nạp giá trị mặc định, một lượt ghi đè tên máy chủ và một lượt ghi đè địa chỉ IPv6 có chi phí tài nguyên tương đối ra sao trên máy đang chạy? Khởi tạo kiểm thử nằm trước vòng lặp `b.Loop()`, nên bộ đếm thời gian của phép đo kiểm chuẩn không tính phần chuẩn bị đó; phương thức `b.ReportAllocs` yêu cầu kết quả xuất ra thêm thông tin cấp phát bộ nhớ:

```
func BenchmarkLoadTargets(b *testing.B) {
    tests := []struct {
        name string
        raw  string
    }{
        {name: "default"},
        {
            name: "hostname override",
            raw:  "payments.internal:9443",
        },
        {name: "IPv6 override", raw: "[2001:db8::10]:443"},
    }
```

```
    for _, tt := range tests {
        b.Run(tt.name, func(b *testing.B) {
            lookup := func(string) (string, bool) {
                return tt.raw, tt.raw != ""
            }
            b.ReportAllocs()
            for b.Loop() {
                _, _ = LoadTargets(lookup)
            }
        })
    }
}
```

Chạy từ thư mục bài thực hành với `go test -run=^$ -bench=BenchmarkLoadTargets -benchmem ./internal/config`. Tùy chọn `-run=^$` giúp tránh trộn các ca kiểm thử thông thường vào lượt đo hiệu năng; cờ `-benchmem` cung cấp các chỉ số `allocs/op` (số lần cấp phát bộ nhớ trên mỗi thao tác) và `B/op` (số byte tiêu tốn trên mỗi thao tác). Đừng sao chép số đo `ns/op` của máy tính cá nhân vào tệp README như một chân lý phổ quát. Tốc độ vi xử lý, phiên bản Go, chế độ tiết kiệm năng lượng và các tiến trình nền của hệ điều hành đều làm kết quả dao động. Nếu một đề xuất tối ưu hiệu năng sắp được sáp nhập, hãy đo lường trước và sau thay đổi trên cùng một môi trường cố định, lặp lại nhiều lần và ghi nhận chính xác khối lượng công việc mà quyết định kỹ thuật đang phục vụ.

Trong trường hợp này, kết luận hoàn toàn có thể là “không cần tối ưu”. Đó là một kết quả kỹ thuật chuẩn mực: việc nạp cấu hình một lần duy nhất chưa xứng đáng để đánh đổi bằng việc làm xáo trộn giao diện lập trình, bổ sung tầng nhớ đệm phức tạp hay khiến mã nguồn trở nên khó đọc. Bài kiểm chuẩn hiệu năng đã hoàn thành xuất sắc nhiệm vụ khi giúp chúng ta từ chối một tối ưu hóa sớm khi chưa có áp lực thực tế, chứ không chỉ hữu ích khi nó tạo ra một bản vá chạy nhanh hơn.

Bài thực hành: Đánh giá kết quả đo lường hiệu năng

Chạy benchmark hai lần độc lập. Chỉ đề xuất tối ưu khi xuất hiện áp lực khối lượng công việc thực tế (như nạp lại cấu hình trong vòng lặp cao điểm hoặc có hồ sơ đo lường thực tế); việc "một con số trông lớn hơn" là chưa đủ căn cứ.

Giao ước ở ranh giới tiến trình

Kiểm thử đơn vị của `app.Run` cố ý không phụ thuộc vào thiết bị đầu cuối (**terminal**) hay mã thoát của tiến trình (**exit code**). Điều đó giúp hàm duy trì ranh giới tinh gọn, nhưng cũng tạo ra một khoảng trống cần kiểm chứng ở đúng vị trí: lệnh vẫn phải ghi nhận kết quả thành công ra luồng xuất chuẩn `stdout`, đưa lỗi cấu hình ra luồng lỗi chuẩn `stderr` và trả mã thoát 2. Đây là giao ước công khai của tiến trình; việc tự ý thay đổi nó có thể làm gãy đổ hàng loạt tập lệnh tự động hóa của người vận hành dù mã nguồn bên trong vẫn vượt qua kiểm thử.

Lời gọi trực tiếp `main` trong kiểm thử thường khiến khoảng trống ấy rất khó kiểm chứng, vì hàm `os.Exit` sẽ chấm dứt tiến trình chạy kiểm thử. Thay vì cố gắng giả lập toàn bộ tiến trình hệ điều hành, chúng ta bóc tách phần hiển thị kết quả thành một hàm nhỏ tiếp nhận các giao diện ghi dữ liệu (**writer**) và trả về mã thoát. Hàm `main` chỉ đảm nhận nhiệm vụ gắn kết các tài nguyên thực tế của hệ thống vào hàm đó:

```
func main() {
    runner := func(context.Context, probe.Endpoint) error {
        return nil
    }
    os.Exit(run(
        context.Background(),
        os.Stdout,
        os.Stderr,
        os.LookupEnv,
        runner,
    ))
}
```

Hàm `run` vẫn ủy thác cho `app.Run`; nó chỉ bổ sung chính sách xử lý tiến trình ở mức tối thiểu. Nhờ vậy, ca kiểm thử có thể truyền đối tượng `bytes.Buffer` làm đại diện cho `stdout` và `stderr`, rồi quan sát đúng ba yếu tố mà vỏ hệ điều hành nhìn thấy: mã thoát, dữ liệu thành công và dữ liệu báo lỗi.

```
func TestRunWritesSuccessContract(t *testing.T) {
    var out, errOut bytes.Buffer
    code := run(
        context.Background(),
        &out,
        &errOut,
        func(string) (string, bool) { return "", false },
        func(
            context.Context,
            probe.Endpoint,
        ) error { return nil },
    )

    if code != 0 {
        t.Fatalf("code = %d", code)
    }
    if out.String() != "billing healthy=true\n" {
        t.Fatalf("stdout = %q", out.String())
    }
    if errOut.Len() != 0 {
        t.Fatalf("stderr = %q", errOut.String())
    }
}
```

Ca kiểm thử này không kiểm chứng xem vòng lặp trả về kết quả ra sao hay `app.Run` phân tích cú pháp cấu hình như thế nào. Các giao ước đó đã thuộc về trách nhiệm của các gói nội bộ riêng biệt. Nó chỉ bảo vệ lời hứa của lệnh: kết quả thành công có thể đọc tự động bằng chương trình trên `stdout` và không làm rò rỉ thông điệp lỗi.

Kiểm thử phía thất bại cũng cần được khoanh vùng chặt chẽ tương tự. Truyền vào một cổng không hợp lệ, để hàm thực thi phát hiện lỗi nếu bị gọi ngoài ý muốn, sau đó khẳng định mã thoát là 2, luồng `stdout` rỗng và thông tin chẩn đoán xuất hiện ở `stderr`. Đừng so khớp toàn bộ thông điệp:

câu chữ mô tả lỗi có thể được biên tập lại, trong khi nơi xuất hiện và chính sách mã thoát mới là giao ước vận hành bất biến.

Bài thực hành: Đánh giá mã nguồn tại ranh giới tiến trình

Chuyển một lệnh `fmt.Fprintf` từ `out` sang `errOut` rồi quyết định đó là việc tinh chỉnh hiển thị vô hại hay là thay đổi phá vỡ tương thích (**breaking change**) cho đường ống lệnh vỏ hệ thống (**shell pipeline**). Sau đó đặt tình huống có thêm cờ `--json`: kiểm thử khế ước cần có ca kiểm thử riêng cho chế độ được cam kết rõ ràng này, không được nới lỏng khế ước chế độ văn bản đến mức chấp nhận bất kỳ dữ liệu đầu ra nào.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Trong đường ống vỏ lệnh, stdout thường mang dữ liệu mà lệnh tiếp theo sẽ tiếp nhận xử lý; việc chuyển kết quả lành mạnh sang stderr có thể là thay đổi phá vỡ tương thích. Kiểm thử không nhằm đóng băng từng dấu câu mãi mãi. Nó bảo vệ ranh giới mà con người và hệ thống tự động hóa dựa vào: thành công thoát với mã `0` và có kết quả trên stdout; lỗi cấu hình thoát với mã 2, không in dữ liệu thành công, đồng thời đưa thông tin chẩn đoán ra stderr.

Kiểm thử giao ước này không thay thế cho kiểm thử đầu cuối (**end-to-end test**) khởi chạy tệp nhị phân thực tế. Sau này khi lệnh tiếp nhận thêm cờ lệnh, tệp cấu hình, tín hiệu hệ điều hành hoặc kết nối mạng thực sự, một số luồng bắt buộc phải được kiểm chứng qua tiến trình thực. Nhưng việc lạm dụng kiểm thử nhị phân để so khớp từng khoảng trắng hay mọi nhánh xác thực sẽ rất chậm chạp, cồng kềnh và trùng lặp với kiểm thử đơn vị. Ranh giới kiến trúc lành mạnh là: logic chứa các phụ thuộc có thể thay thế được kiểm thử ở gói nội bộ; các cam kết mà tiến trình công bố ra ngoài được giữ ở hàm `run`; và chỉ các luồng tích hợp then chốt mới cần chuyển giao cho kiểm thử tích hợp.

Đây là điểm mấu chốt của mốc thiết kế này: khả năng kiểm thử không đồng nghĩa với việc phủ một lớp giả lập (**mock**) nặng nề lên mọi gói mã nguồn. Nó là năng lực đặt từng chính sách nghiệp vụ vào một ranh giới đủ nhỏ để chúng ta kiến tạo dữ liệu đầu vào, quan sát kết quả và nhận biết chính xác thay đổi nào đang được bảo vệ.

Kết quả kiểm thử xanh chưa trả lời câu hỏi nào?

Với các bài thực hành vừa hoàn tất, kết quả báo xanh đồng nghĩa các khẳng định kiểm thử (**assertion**) đã được thỏa mãn trên tập dữ liệu và lịch trình thực thi cụ thể đó. Một khẳng định kiểm thử bị bỏ sót vẫn có thể để lọt lỗi trên chính dữ liệu đầu vào ấy. Kiểm thử mờ (**fuzzing**) mở rộng không gian tìm kiếm đầu vào dựa trên thời gian và độ bao phủ mã nguồn; số lượt chạy không cố định, và bộ fuzzer không thể tự sáng tạo thêm một đặc tính bất biến nếu tác giả chưa định nghĩa. Khi phát hiện lỗi sập (**crasher**), hãy lưu trữ mẫu dữ liệu đó cùng một ca kiểm thử hồi quy (**regression test**) có tên rõ ràng để người sửa đổi sau này biết rõ giao ước từng bị phá vỡ ở đâu.

Kiểm chuẩn hiệu năng (**benchmark**) lại giải đáp một câu hỏi hoàn toàn khác. Cấu trúc `b.Loop()` xuất hiện từ Go 1.24; trong phiên bản Go 1.27.1, lần gọi đầu tiên thiết lập lại bộ đếm thời gian và lần trả về `false` sẽ dừng bộ đếm. Với dạng chuẩn `for b.Loop() { ... }`, trình biên dịch bảo đảm duy trì sự sống của các đối số, kết quả lời gọi và biến gán trong thân vòng lặp theo cơ chế được mô tả tại `testing/benchmark.go`. Không nên suy diễn cơ chế tối ưu ấy sang một hàm bọc tùy tiện hoặc ngộ nhận rằng bài đo lường phản ánh đầy đủ môi trường sản xuất. Cách viết truyền thống `b.N` vẫn hoàn toàn hợp lệ; nếu sử dụng, hãy loại trừ phần khởi tạo khỏi bộ đếm thời gian khi cần và bảo đảm phép toán cần đo không bị trình biên dịch loại bỏ do không sử dụng kết quả. Không bao giờ trộn lẫn hai cách viết vòng lặp trong cùng một bài đo lường.

Bộ phát hiện tương tranh (**race detector**) cũng đi kèm chi phí tài nguyên nhất định. Tài liệu kỹ thuật chính thức nêu mức tiêu hao thường gặp khoảng 5–10 lần dung lượng bộ nhớ và 2–20 lần thời gian thực thi tùy cấu trúc chương trình; đây không phải số đo riêng của các bài thực hành trong sách. Bộ phát hiện chỉ quan sát các truy cập bộ nhớ thực sự xảy ra trong lượt chạy, do đó một nhánh mã chưa được kích hoạt sẽ không thể tạo ra báo cáo. Độ trễ mạng có thể làm bộc lộ những lịch trình thực thi đồng thời mới, nhưng điều đó không có nghĩa công cụ bất lực trước các tác vụ mạng. Để nâng cao xác suất phát hiện, hãy thiết kế các kịch bản có giá trị thực tế và bao quát đầy đủ các luồng tương tác then chốt, thay vì chỉ lặp đi lặp lại một kịch bản thuận lợi.

Lệnh `go test -cover` báo cáo độ bao phủ câu lệnh (**statement coverage**) dựa trên cơ chế gắn mã chẩn đoán (**instrumentation**) của Go, không phản ánh tỷ lệ mọi dòng mã nguồn và cũng không phải độ bao phủ nhánh (**branch coverage**) hay độ bao phủ đường dẫn (**path coverage**). Dù đạt mức 100%, con số đó chưa thể chứng minh mọi tổ hợp điều kiện, dữ liệu biên hay tương tác đồng thời đều chính xác. Ví dụ, một cấu trúc điều kiện `if` đã được kiểm thử ở nhánh đúng vẫn có thể chưa từng được kiểm tra ở nhánh sai. Khi đọc báo cáo độ bao phủ, hãy luôn đặt câu hỏi: khẳng định kiểm thử nào đang bảo vệ chính sách nghiệp vụ, nhánh luồng nào chưa từng chạy qua, và công cụ nào có thể phát hiện loại lỗi tiềm ẩn đó. Đó là giới hạn tất yếu của các công cụ thu thập bằng chứng, không phải lý do để buông lỏng việc kiểm thử.

Chương 7 — Dữ liệu đi vào và đi ra

Một lệnh vừa in ra một dòng `billing healthy=true` có vẻ như đã giao tiếp được với thế giới bên ngoài. Nhưng terminal chỉ là một `io.Writer` rất đặc biệt. Khi dữ liệu vào chuyển thành tệp cấu hình, pipe từ lệnh khác, body HTTP hay một stream nén, trực giác "đọc file rồi có dữ liệu" bắt đầu che một sự thật quan trọng: dữ liệu không nhất thiết đến cùng lúc, và tài nguyên không tự hết hạn đúng lúc ta muốn.

Mô hình tinh thần của chương này là stream cung cấp byte theo từng lượt, không phải một slice đã có đủ dữ liệu. Caller phải xử lý byte nhận được, xác nhận điểm kết thúc và biết ai chịu trách nhiệm đóng tài nguyên. `io.Reader` không tự hứa một deadline hay tiến độ trong thời gian hữu hạn; những chính sách ấy thuộc nguồn dữ liệu và application.

Một lần `Read` không có nghĩa là toàn bộ input

Hãy bắt đầu với một lỗi nhỏ hơn `opsprobe`. Một cấu hình JSON trông như thế này:

```
[
  {
    "name": "billing",
    "endpoint": "https://billing.internal/health"
  }
]
```

Một cách nghĩ sai lầm nhưng phổ biến là cấp phát vùng đệm (**buffer**) đủ lớn cho ca kiểm thử, gọi phương thức `Read` một lần duy nhất, rồi suy luận số byte nhận được. Cách làm này thường vượt qua kiểm thử với `strings.NewReader` và dữ liệu đầu vào ngắn. Nhưng `io.Reader` không cam kết sẽ lấp đầy vùng đệm. Nguồn đọc (**reader**) có thể là tệp, ổ cắm mạng (**socket**), đường ống dẫn (**pipe**) hay bộ giải mã khác; mỗi lần gọi phương thức chỉ tiếp nhận một phần lượng byte sẵn có.

Giao ước (**contract**) tối thiểu của `Read(p)` quy định: hàm trả về `n` byte nằm trong lát cắt `p[:n]`, và có thể kèm theo một lỗi. Giá trị `n` hoàn toàn có thể

nhỏ hơn `len(p)`. Đặc biệt, một nguồn đọc được phép trả về các byte dữ liệu hữu ích đi kèm mã lỗi lính canh `io.EOF` ngay trong một lượt gọi; bên gọi (**caller**) bắt buộc phải tiêu thụ dữ liệu `p[:n]` trước khi xử lý lỗi. Tín hiệu `io.EOF` biểu thị luồng dữ liệu đã kết thúc hoàn toàn, không đồng nghĩa với việc lần đọc hiện tại không chứa dữ liệu.

Bảng 6 — Cách đọc kết quả `Read`

Kết quả	Caller phải làm gì	Diễn giải đúng
`n > 0`, `err == nil`	Xử lý `p[:n]`, rồi đọc tiếp	Stream đang tiến lên nhưng chưa kết thúc.
`n > 0`, `err == io.EOF`	Vẫn xử lý `p[:n]`, rồi kết thúc	Lượt cuối vừa có dữ liệu vừa báo hết stream.
`n > 0`, error khác `nil`	Xử lý `p[:n]`, rồi trả hoặc phân loại lỗi	Dữ liệu có thể tới cùng lỗi khác EOF; không được bỏ phần đã đọc.
`n == 0`, `err == io.EOF`	Kết thúc	Không còn byte để tiêu thụ.
`n == 0`, error khác `nil`	Trả hoặc phân loại lỗi	Lượt này không trả byte và có lỗi.
`n == 0`, `err == nil`	Không coi là EOF; tránh vòng busy-loop vô hạn	Không có tiến độ trong lượt này; contract không khuyến khích kết quả ấy với buffer không rỗng nhưng caller vẫn phải phân biệt nó.

Đừng tự viết vòng lặp `Read` thủ công chỉ để chứng minh mình nắm vững giao ước nếu thư viện chuẩn đã cung cấp sẵn công cụ nguyên thủy (**primitive**) phù hợp. Hàm `io.ReadAll` là lựa chọn xác đáng khi dữ liệu đầu vào có giới hạn kích thước tin cậy và việc lưu trữ toàn bộ dữ liệu trong bộ nhớ là chấp nhận được. Khi dữ liệu đầu vào có khả năng tăng đột biến hoặc vô hạn, thiết kế bắt buộc phải chuyển sang kỹ thuật truyền dữ liệu phân đoạn (**streaming**). Sự khác biệt này không đơn thuần là tối ưu vi mô: nó quyết định chương trình có bị áp lực cấp phát bộ nhớ cưỡng bức bởi dữ liệu đầu vào không kiểm chứng hay không.

Giới hạn byte trước khi gọi parser

"Tệp cấu hình chỉ nhỏ" không phải là một giới hạn. Nếu chương trình nhận một đường dẫn hoặc reader từ bên ngoài, nó cần đặt giới hạn ngay tại ranh giới mình kiểm soát. Chỉ đặt giới hạn sau `json.Decode` là quá muộn: bộ phân tích đã có quyền đọc bất cứ lượng byte nào trước khi chính sách được áp dụng.

Trước khi mở `bounded.go`, chỉ đọc hai test `TestReadBounded...`. Chúng đòi `ReadBounded` nhận tối đa `max` byte; input lớn hơn đúng một byte phải trả error mà caller nhận diện được bằng `errors.Is`. Hãy tự viết phần thân theo ba bước: bọc reader để chỉ cho phép tối đa `max + 1` byte, đọc phần đã bọc, rồi so độ dài với `max`. Byte thứ `max + 1` không phải dữ liệu cần giữ; nó là bằng chứng phân biệt tệp vừa chạm ngưỡng với tệp thật sự vượt ngưỡng.

Lab từ chối `max < 0` và `max == math.MaxInt64` trước khi đọc: giá trị sau không cho phép tính `max + 1` trong `int64`. Không sửa tràn số bằng cách bỏ lớp giới hạn rồi gọi `io.ReadAll(r)` không giới hạn. Test dùng reader ghi nhận lời gọi để kiểm tra đầu vào bị từ chối mà chưa đọc; không cần tạo tệp khổng lồ. Giới hạn byte cũng không phải trần heap chính xác: buffer, parser và metadata có chi phí riêng.

`io.LimitReader` chỉ giới hạn số byte mà reader bọc ngoài sẽ trả; nó không tự nói JSON có hợp lệ hay không, và cũng không làm input nhỏ trở nên đáng tin. Vì thế giới hạn byte là một chính sách tài nguyên riêng; `Decode` vẫn chịu trách nhiệm cho cấu trúc JSON và số tài liệu. Hai test ngắn bảo vệ hai câu hỏi khác nhau, không gộp chúng thành một assertion mơ hồ kiểu "cấu hình lỗi".

Đây cũng là điểm cần tránh một cam kết sai lầm về cơ chế hủy bỏ (**cancellation**). Giao diện `io.Reader` chỉ công bố phương thức `Read`; nó không tiếp nhận `context.Context`. Một lớp bọc (**wrapper**) như `ReadBounded` có thể dừng sau khi đọc đủ ngưỡng byte, nhưng không thể tự đánh thức một nguồn đọc cơ sở bên dưới đang bị nghẽn thao tác vào/ra (**blocking I/O**). Với tệp cục bộ, điều đó thường không phải chính sách bắt buộc. Với thân thông điệp HTTP (**HTTP body**), thời hạn chót (**deadline**) và tín hiệu hủy bỏ gắn liền với vòng đời của yêu cầu mạng sẽ được phân tích ở

chương mạng, nơi ta xác định rõ quyền sở hữu kết nối và cách tầng vận chuyển phản hồi.

Failing test trước parser

Lab `labs/part7-stream-boundaries` không gắn ngay tệp JSON vào `opsprobe`. Project xuyên suốt hiện chưa cần tệp cấu hình; nhét nó vào lúc này chỉ khiến trực giác mới bị lẫn với chính sách endpoint cũ. Ta dùng một chương trình tái hiện tối thiểu để thấy ranh giới stream trước.

Hãy mở tệp `targets/targets_test.go` và quan sát ca kiểm thử `TestDecodeConsumesChunkedStream`. Cấu trúc `chunkReader` cố ý chỉ giải phóng ba byte dữ liệu trong mỗi lần gọi. Trước khi đọc phần triển khai của hàm `Decode`, hãy trả lời: nếu mã nguồn giả định ngây thơ rằng một lượt `Read` là đủ, ca kiểm thử sẽ đánh mất đoạn nào của tài liệu? Sau đó, hãy tạm đổi tên hàm `Decode`, chạy kiểm thử và tự xây dựng lại hàm theo đúng giao ước mà ca kiểm thử đòi hỏi.

Function kết thúc có thể nhỏ:

```
func Decode(r io.Reader) ([]Target, error) {
    dec := json.NewDecoder(r)
    var targets []Target
    if err := dec.Decode(&targets); err != nil {
        return nil, fmt.Errorf(
            "decode target document: %w", err,
        )
    }

    var extra any
    if err := dec.Decode(&extra); !errors.Is(err, io.EOF) {
        if err == nil {
            return nil, errors.New(
                "target config must contain one JSON value",
            )
        }
        return nil, fmt.Errorf(
            "read after target document: %w", err,
        )
    }
    return targets, nil
}
```

`json.Decoder` tiếp nhận một `io.Reader`, do đó nó tự động đọc qua nhiều khối dữ liệu (**chunk**); bên gọi không cần giả định sai lầm rằng ổ cắm mạng hay tệp tin là một lát cắt bộ nhớ có sẵn. Ca kiểm thử thứ hai đặt thêm một giá trị JSON phía sau tài liệu đầu tiên. Việc chỉ gọi `Decode` một lần duy nhất chưa đáp ứng đầy đủ giao ước: chương trình sẽ âm thầm bỏ qua phần dữ liệu dư thừa còn lại. Lần gọi `Decode` thứ hai không nhằm mục đích lấy cấu hình mới, mà để kiểm chứng rằng sau tài liệu hợp lệ chỉ còn khoảng trắng và ký hiệu kết thúc luồng `io.EOF`.

Điều này không biến bộ giải mã JSON thành lời giải vạn năng cho mọi định dạng. Tệp CSV có quy tắc phân tách bản ghi (**record**) và dấu trích dẫn (**quoting**) riêng; tệp nhật ký dòng lệnh (**line-oriented log**) có ranh giới phân tách theo ký tự xuống dòng; thân thông điệp HTTP có vòng đời mạng và giới hạn kích thước khác biệt. Các cơ chế đó sẽ được phân tích trong ngữ cảnh

phù hợp. Ở đây, trọng tâm là câu hỏi kỹ thuật cốt lõi: dữ liệu đầu vào kết thúc ở đâu, và đoạn mã nào đã xác minh điều đó?

Đủ một document chưa có nghĩa mọi decoder hiểu giống nhau

Dữ liệu đầu vào `{"port":80,"port":443}` là một tài liệu có khóa bị trùng lặp. Giao diện `encoding/json` mà phòng thực hành sử dụng giữ nguyên ngữ nghĩa v1: với trường số trong ví dụ này, giá trị khai báo sau sẽ ghi đè lên giá trị trước. Go 1.27 chính thức đưa `encoding/json/v2` và `encoding/json/jsontext` thành giao diện tiêu chuẩn; theo mặc định, phiên bản v2 sẽ từ chối các khóa trùng tên và chuỗi UTF-8 không hợp lệ. Giao diện v1 vẫn tiếp tục được hỗ trợ đầy đủ, mặc dù cơ chế cài đặt bên dưới có thể thay đổi và câu chữ chẩn đoán lỗi có thể khác biệt. Không được tùy tiện thay đổi đường dẫn nhập khẩu (**import path**) rồi xem đó là một đợt chuyển đổi mã nguồn (**migration**) tương đương.

Trước khi thực thi ca kiểm thử `TestJSONVersionBoundary` trong thư mục `labs/edition-contracts`, hãy dự đoán bộ giải mã nào chấp nhận tài liệu trên và giá trị `port` cuối cùng sẽ là bao nhiêu. Bài kiểm thử chỉ kiểm chứng hai trường hợp khóa trùng lặp và chuỗi UTF-8 không hợp lệ; nó không chứng minh bộ giải mã là công cụ kiểm định toàn bộ chính sách cấu hình. Giới hạn dung lượng byte, tính duy nhất của một tài liệu, lược đồ hợp lệ và ngữ nghĩa lựa chọn giao diện lập trình là bốn bài toán kỹ thuật hoàn toàn độc lập.

Đóng output cũng là một phần của result

Khi đọc tệp, đăng ký `defer file.Close()` sau khi mở thành công giúp dọn file lúc hàm thoát. Khi ghi, phải quan sát cả lỗi ghi và lỗi đóng; nhưng `io.WriteCloser` không cam kết `Close` flush hay lưu dữ liệu bền vững xuống đĩa. Ý nghĩa ấy thuộc kiểu cụ thể. `bufio.Writer` không có `Close`: phải gọi và kiểm tra `Flush` trước khi đóng file bên dưới. `os.File.Close` không thay thế `File.Sync` nếu hợp đồng yêu cầu commit dữ liệu ổn định. Cả flush lẫn close thành công cũng không tự chứng minh cập nhật tệp là nguyên tử hoặc chống mọi sự cố mất điện.

Mã nguồn đầy đủ mở tài nguyên rồi thiết lập `defer Close` ngay sau điểm tiếp nhận. Để làm nổi bật chính sách chọn lọc kết quả, phần lõi có thể biểu diễn tinh gọn như sau:

```
func save(w io.WriteCloser, targets []Target) error {
    writeErr := Encode(w, targets)
    closeErr := w.Close()
    if writeErr != nil {
        return writeErr
    }
    return closeErr
}
```

Mã nguồn thực tế vẫn giải phóng tài nguyên trên mọi nhánh trả về. Đoạn mã trên làm rõ chính sách kỹ thuật: ưu tiên giữ lại lỗi ghi trước, rồi mới phản hồi lỗi đóng tệp. Nguồn ghi giả lập bảo vệ giao ước đó mà không đòi hỏi phải phụ thuộc vào hệ thống tệp vật lý.

Ranh giới Dữ liệu: Short Write, Bộ đệm và Ngộ nhận về Syscall

Khi làm việc với các dòng nhập xuất dữ liệu, trực giác bề mặt thường dẫn tới hai ngộ nhận tai hại: đánh đồng hành vi của Reader với Writer, và cho rằng mỗi thao tác đọc ghi đều kích hoạt một lời gọi hệ thống xuống nhân hệ điều hành.

Ràng buộc Khắt khe của `io.Writer` và Khái niệm Short Write

Nếu như `io.Reader` cho phép trả về số byte đọc được `n < len(p)` kèm theo `err == nil` trong các tình huống bình thường (chẳng hạn khi đường ống mạng chưa nhận đủ dữ liệu), thì giao diện `io.Writer` lại áp đặt một hợp đồng nghiêm ngặt hơn nhiều. Theo đặc tả của thư viện chuẩn Go, một hàm `Write(p)` bắt buộc phải trả về một lỗi khác `nil` nếu nó không thể ghi trọn vẹn toàn bộ lát cắt dữ liệu (`n < len(p)`).

Biến lỗi `io.ErrShortWrite` là lỗi lính canh (**sentinel error**) mô tả một lượt ghi thiếu dữ liệu (**short write**) mà không có lỗi cụ thể đi kèm. Bộ ghi có thể trả về lỗi chi tiết khác; giao ước không bắt buộc mọi lượt ghi ngắn phải mang chính lỗi lính canh này. Nếu bộ ghi trả về `n < len(p)` kèm lỗi `nil`, hành vi đó vi phạm nghiêm trọng giao ước của giao diện `io.Writer`. Bên gọi sử

dụng giao diện cần chủ động kiểm tra kết quả, không được suy diễn rằng dữ liệu đã tới đích cuối cùng chỉ vì thao tác ghi ở tầng trung gian báo thành công.

Bộ đệm User-space với `bufio`

Một lời gọi hệ thống (**syscall**) đưa luồng thực thi vượt qua ranh giới giữa không gian người dùng (**user space**) và không gian nhân (**kernel space**). Thao tác này phát sinh chi phí tính toán nhất định, nhưng không đồng nghĩa với việc bộ lập lịch (**scheduler**) phải thực hiện chuyển đổi ngữ cảnh (**context switch**) sang luồng hệ điều hành (**OS thread**) khác; việc tạm hoãn (**blocking**) và tái lập lịch còn phụ thuộc vào bản chất thao tác và trạng thái sẵn sàng của tài nguyên. Ghi từng byte riêng lẻ qua tệp không có bộ đệm thường làm tăng đột biến số lượt thao tác vào/ra (**I/O**) so với việc gom thành khối lớn. Không thể suy đoán máy móc số lượng syscall hay mức sụt giảm thông lượng (**throughput**) chỉ từ số lần gọi phương thức `Write`; kỹ sư phải xác định rõ kiểu bộ ghi cụ thể (**concrete writer**) và tiến hành đo đạc trực tiếp trên hệ điều hành mục tiêu.

`bufio.Reader` và `bufio.Writer` thực hiện gom cụm dữ liệu trong vùng đệm thuộc không gian người dùng (**user space buffer**). Trong mã nguồn Go 1.27.1, hàm khởi tạo mặc định phân bổ kích thước vùng đệm 4096 byte; con số này không phải quy tắc bắt buộc của `io.Reader` hay cam kết rằng mọi vùng đệm đều phải có dung lượng tương tự. Bộ ghi chỉ chuyển dữ liệu xuống nguồn ghi cơ sở khi cần giải phóng dung lượng hoặc khi được gọi phương thức `Flush` tường minh; bên gọi bắt buộc phải kiểm tra lỗi trả về từ `Flush`. Vùng đệm hỗ trợ giảm số lượt gọi tầng dưới, chứ không bảo đảm một tỷ lệ cắt giảm syscall hay mức tăng thông lượng cố định.

Lời gọi `Read` không đồng nghĩa với Syscall

Một ngộ nhận kỹ thuật phổ biến là mặc định rằng mọi lệnh `r.Read(p)` trong mã nguồn Go đều tương ứng với một chỉ thị CPU `SYSCALL` trực tiếp xuống kernel.

Đường dẫn thực thi đi qua vùng nhớ người dùng hay lời gọi hệ thống phụ thuộc vào bản cài đặt cụ thể, hệ điều hành và trạng thái thao tác. Bảng dưới đây phân định rành mạch các trường hợp thường gặp:

Kiểu cụ thể của Reader	Bản chất cơ chế khi gọi `Read(p)`	Có phát sinh Syscall xuống OS không?
`*bytes.Buffer` hoặc `*strings.Reader`	Sao chép byte vào buffer của caller từ dữ liệu trong bộ nhớ.	Không cần syscall I/O cho phần sao chép; cách compiler hiện thực `copy` không phải contract của Reader.
`*bufio.Reader` (khi còn dữ liệu trong đệm)	`Read` sao chép byte từ đệm vào buffer của caller; khác với các API trả một slice nhìn vào đệm.	Không cần gọi underlying Reader trong lượt được phục vụ đủ từ đệm.
`*bufio.Reader` (khi bộ đệm đã cạn)	Gọi `Read` lên đối tượng `io.Reader` bên dưới để nạp lại bộ đệm nội bộ.	Phụ thuộc underlying Reader. Nếu bọc in-memory buffer thì không có syscall; nếu bọc file hoặc socket thì do reader bên dưới quyết định.
`*os.File` chưa đệm	Gửi yêu cầu I/O trực tiếp tới kernel qua bảng mô tả tệp (OS file I/O path).	Có. Thực hiện syscall đọc tệp của hệ điều hành (`read` trên Linux hoặc `ReadFile` trên Windows).
`*net.TCPConn` trong đường đọc Linux đã pin bên dưới	Thử non-blocking I/O; kết hợp poller nếu chưa sẵn sàng.	Có OS I/O; không có quan hệ một-một giữa số lần gọi `Read` và syscall. Windows dùng đường completion khác.

Với Go 1.27.1 trên Linux, đường dẫn đọc ổ cắm mạng trong `internal/poll/fd_unix.go` có thể thử thao tác đọc không chặn (**non-blocking read**), gặp mã lỗi `EAGAIN`, chuyển quyền chờ cho bộ thăm dò mạng (**network poller**) rồi thử lại sau. Đây là mô hình triển khai cụ thể theo phiên bản, không phải cam kết bất biến của giao diện `net.Conn`. Không được áp đặt vết kiểm tra tính sẵn sàng (**readiness trace**) của Linux sang Windows: nền tảng IOCP sử dụng mô hình hoàn tất thao tác (**completion-based**) và cấu trúc xử lý khác biệt. Giao diện lập trình chung vẫn xoay quanh việc tiếp nhận byte, xử lý lỗi và kiểm soát thời hạn chót; bộ thăm dò mạng hỗ trợ điều phối goroutine hiệu quả chứ không thể triệt tiêu hoàn toàn thao tác vào/ra xuống hệ điều hành. Chương 11 sẽ tích hợp cơ chế này vào một yêu cầu mạng thực tế.

Một filesystem là một khả năng, không phải một đường dẫn

Bộ phân tích cú pháp ở trên chỉ cần dữ liệu byte, hoàn toàn không đòi hỏi quyền mở mọi tệp tin trên máy chủ. Khi một ứng dụng có nhiều nguồn cung cấp tệp, giao diện `fs.FS` mô tả năng lực `Open(name)` với tên logic phân tách bằng dấu gạch chéo `/`. Bên tiêu dùng (**consumer**) có thể tiếp nhận `embed.FS` chứa tài nguyên nhúng tại thời điểm biên dịch, `os.DirFS` ánh xạ vào thư mục tại thời điểm thực thi, hoặc một hệ thống tệp giả lập phục vụ kiểm thử. Bên tiêu dùng không cần phải thay đổi bộ phân tích cú pháp theo từng nguồn cung cấp tệp.

```
// Đặt trong package có thư mục fixtures tại build time.
//go:embed fixtures/*.txt
var files embed.FS

data, err := fs.ReadFile(files, "fixtures/message.txt")
```

Các gói nhập khẩu của đoạn mã này bao gồm `embed` và `io/fs`; bản chạy được đặt tại `labs/edition-contracts`. Chỉ thị tiền biên dịch (**compiler directive**) bắt buộc phải đặt tại phạm vi gói (**package scope**) và gắn liền với biến dữ liệu phù hợp. Dữ liệu byte được đóng gói trực tiếp vào tệp nhị phân tại thời điểm biên dịch, không tự động cập nhật khi tệp bên ngoài thay đổi. Giao diện `embed.FS` chỉ cung cấp năng lực đọc dữ liệu, không phải là nơi lưu trữ khóa bí mật (**secret**) hoặc cấu hình cần điều chỉnh sau khi triển khai. Các byte dữ liệu được đóng gói hoàn toàn có thể bị trích xuất từ tệp nhị phân; việc biến mất tệp tin rời rạc trên đĩa không đồng nghĩa với an ninh bảo mật.

Hàm `fs.ValidPath` không cho phép các thành phần đường dẫn `..` hoặc đường dẫn bắt đầu bằng dấu `/`; những tên như `a..b` vẫn có thể hợp lệ, còn `.` là trường hợp đặc biệt chỉ định thư mục gốc. Một đường dẫn hợp lệ không tự tạo ra vùng cách ly an toàn (**sandbox**). Hàm `os.DirFS` và `fs.Sub` không thể ngăn chặn các liên kết mềm bên dưới trỏ ra ngoài thư mục cơ sở. Nếu tiếp nhận tên tệp từ nguồn không tin cậy và đòi hỏi giới hạn truy cập nghiêm ngặt, hãy tham khảo giao ước của `os.Root` cùng các giới hạn theo hệ điều hành trong Go 1.27.1 thay vì tin tưởng mù quáng rằng hàm `Clean` chặn được

mọi đường thoát. Quyền mở tệp và quyền phân tích cú pháp dữ liệu là hai ranh giới hoàn toàn độc lập.

`os.OpenRoot(base)` mở một khả năng truy cập có gốc; các thao tác như `root.Open(name)` không cho tên hoặc symlink dẫn ra ngoài cây đó. Nó vẫn đi theo symlink hợp lệ bên trong, không phải cấm mọi symlink. Với Go 1.27.1 trên Windows và Linux, đây là primitive chống traversal, nhưng không phải sandbox tiến trình: không chặn mọi truy cập thiết bị, mount hay hard link có sẵn trong root. Tài liệu còn nêu ngoại lệ TOCTOU trên `GOOS=js`; không suy rộng bảo đảm Windows sang mọi nền tảng.

Test `TestRealFilesystemRootConfinement` của lab tạo tệp và symlink trên filesystem thật trong thư mục tạm. Nó đối chiếu `DirFS` đọc được link ra ngoài với `Root` từ chối, đồng thời kiểm tra link nội bộ vẫn đọc được. Nếu hệ điều hành không cho tạo symlink, nhánh đó ghi SKIP với nguyên nhân, không chuyển thành PASS bằng `fstest.MapFS`. Test traversal `../outside.txt` chạy riêng dù thiếu quyền tạo link. Những phép thử này không mô phỏng kẻ tấn công đổi symlink đồng thời, nên chỉ là kiểm chứng các trường hợp đã chạy của API.

Bài thực hành: Đọc tệp qua trừu tượng hóa fs.FS

Viết một hàm tiếp nhận `fs.FS` và tên logic, đọc một tệp rồi chuyển các byte dữ liệu cho bộ phân tích cú pháp đã có sẵn. Kiểm thử với hệ thống tệp giả lập; kiểm thử riêng cho các trường hợp tên tệp không hợp lệ và tệp bị thiếu. Sau đó thay thế bằng `embed.FS` mà không cần sửa đổi bộ phân tích cú pháp. Đừng để kiểm thử trên hệ thống tệp giả lập bị ngộ nhận thành bằng chứng khẳng định liên kết mềm (**symlink**) hoặc quyền truy cập thực tế trên hệ điều hành đã được bảo đảm an toàn.

Parser cũng có ngân sách

Quy ước nhật ký "mỗi dòng là một bản ghi" khiến `bufio.Scanner` trở nên tiện lợi hơn vòng lặp đọc thủ công. Tuy nhiên, `Scanner` có giới hạn độ dài mã thông báo (**token**): một dòng nhật ký quá dài có thể khiến phương thức `Scan()` trả về `false` trước khi chạm tới ký hiệu kết thúc luồng `EOF`. Bên gọi bắt buộc phải kiểm tra `Err()`, và nếu miền nghiệp vụ cho phép độ dài dòng lớn hơn, hãy gọi phương thức `Buffer` trước khi quét với hạn mức trần thích hợp. Không bao giờ nâng mức trần lên vô hạn chỉ để cứu vãn một dữ liệu đầu vào bị hỏng. Ca kiểm thử `TestScannerLimitAndValidation` sử dụng cùng một dữ liệu đầu vào với hai hạn mức khác nhau để phân biệt trường hợp kết thúc hợp lệ với trường hợp vượt quá ngân sách tài nguyên.

Giới hạn dung lượng byte không thể thay thế cho thao tác kiểm tra miền giá trị số học. Hàm `strconv.ParseUint(text, 10, 16)` trả về lỗi nếu chuỗi văn bản không phải là số nguyên không dấu (**unsigned integer**) có thể biểu diễn trong 16 bit; sau đó quy tắc nghiệp vụ vẫn phải đưa ra quyết định liệu cổng mạng 0 có được phép hay không. Hàm `Atoi` chọn kiểu `int` phụ thuộc kiến trúc máy tính, không định nghĩa được một khuôn dạng truyền tải (**wire format**) ổn định. Khi định dạng đòi hỏi bề rộng bit chính xác, hãy chỉ định rõ bề rộng bit trong bộ phân tích cú pháp và kiểm thử kỹ lưỡng các giá trị biên.

Với biểu thức chính quy do người dùng nhập vào, hãy sử dụng `regexp.Compile` và xử lý lỗi cẩn trọng. Hàm `MustCompile` chỉ phù hợp với các mẫu biểu thức cố định trong mã nguồn mà sự cố cú pháp được coi là lỗi lập trình, không phải phương thức báo lỗi cho người vận hành. Gói `regexp` cam kết thời gian so khớp tuyến tính theo độ dài chuỗi đầu vào; điều đó không đồng nghĩa với việc chi phí biên dịch mẫu, dung lượng bộ nhớ hay tần suất so khớp không cần đặt giới hạn. Bộ phân tích cú pháp, số lượng bản ghi, mẫu biểu thức và vòng đời của luồng dữ liệu đều phải có ngân sách tài nguyên rõ ràng trong ứng dụng. Hãy ưu tiên các hàm trong gói `strings` hoặc `bytes` khi thao tác chỉ đơn giản là tìm kiếm ký tự phân tách hoặc tiền tố; chỉ bổ sung biểu thức chính quy khi ngữ pháp cú pháp thực sự đòi hỏi.

Tự kiểm tra trước khi tích hợp

Chạy từ `labs/part7-stream-boundaries`:

```
go test ./...
go vet ./...
```

> **Dừng để dự đoán:** Thử lần lượt: bỏ lượt `Decode` thứ hai; cho `chunkReader` nhả một byte; rồi cho writer giả fail ở `Write` trước `Close`. Hãy dự đoán ca kiểm thử nào sẽ thất bại.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Bỏ lượt giải mã thứ hai làm `TestDecodeRejectsAdditionalDocument` thất bại; khối dữ liệu nhỏ hơn không làm thay đổi khế ước. Khi ghi và đóng tệp cùng gặp lỗi, phải ưu tiên giữ lại lỗi từ thao tác ghi.

Với limit, input dài đúng `max` byte được trả nguyên vẹn; input dài `max + 1` trả error giữ identity `ErrDocumentTooLarge`. Đó là lý do lab không dùng một `if len(data) == max` mơ hồ: bằng chứng cần phân biệt đúng ngưỡng với vượt ngưỡng.

Chương này không bổ sung tệp cấu hình vào `opsprobe`: hiện tại chưa có yêu cầu vận hành thực tế nào đòi hỏi điều đó. Đây là chủ đích bảo toàn công cụ giảng dạy phục vụ trọng tâm bài học. Khi xuất hiện yêu cầu thực tế đòi hỏi nhập xuất dữ liệu mục tiêu, các ranh giới luồng dữ liệu và chính sách đóng tài nguyên trong phòng thực hành này sẽ trở thành nền tảng vững chắc để tích hợp có chủ đích.

Trước khi tiếp cận một giao diện vào/ra mới, hãy tự đặt ra ba câu hỏi: Các byte dữ liệu truyền đến theo từng lượt phân đoạn nào? Đâu là dấu hiệu kết thúc luồng được chấp thuận? Và thực thể nào sẽ quan sát lỗi thất bại của tài nguyên sau cùng? Ba câu hỏi này không thay thế cho tài liệu đặc tả của định dạng hay giao thức cụ thể, nhưng chúng triệt tiêu một quan niệm sai lầm phổ biến: xem nhẹ thao tác nhập/xuất như những đường ống phụ trợ nằm ngoài giao ước kỹ thuật của chương trình.

Phần tiếp theo sẽ đặt thêm một áp lực vận hành mới lên chương trình: nhiều goroutine cùng đồng thời hoạt động trong một tiến trình. Trước khi lựa chọn kênh truyền (**channel**) hay khóa loại trừ tương hỗ (**mutex**), ta cần nhìn nhận hiện tượng chạy đua dữ liệu (**data race**) không phải như một câu khẩu hiệu mơ hồ về tính an toàn luồng (**thread safety**), mà là hai thao tác truy cập dữ liệu không tồn tại trật tự xảy ra trước (**happens-before**) an toàn.

Chương 8 — Một race bắt đầu từ đâu

Một chương trình kiểm tra nhiều endpoint thường có một người thu thập kết quả và một người trình bày. Bản chạy thử có thể luôn in đúng trên laptop, nên ta rất dễ gọi nó là “chạy song song được”. Nhưng kết quả đúng trong một lượt không tạo ra một lời hứa đúng cho lượt kế tiếp.

Mô hình tinh thần của chương này: hai goroutine cùng tồn tại chưa tạo ra data race; phải có truy cập cùng vị trí nhớ, ít nhất một lần ghi, ít nhất một thao tác không phải đồng bộ, và thiếu quan hệ thứ tự thích hợp. Hai bên chỉ đọc dữ liệu bất biến không cần khóa chỉ vì cùng tồn tại. Trước khi chọn channel, mutex hay atomic, ta phải lần được những lần truy cập đó và hỏi: thứ tự nào thực sự được thiết lập?

Một lá cờ không tự công bố kết quả

Xét tình huống một tiến trình thu thập (collector) chuẩn bị bản báo cáo, sau đó gán cờ `ready` để báo cho bên hiển thị biết dữ liệu đã sẵn sàng:

```
type summary struct {
    completed int
    last      string
}

var latest summary
var ready bool

func collect() {
    latest = summary{completed: 1, last: "billing"}
    ready = true
}

func show() summary {
    for !ready {
    }
    return latest
}
```

Nguồn trông có một thứ tự hiển nhiên: gán `latest`, rồi mới gán `ready`. Nhưng đó chỉ là thứ tự **bên trong** `collect`. `show` là một luồng thực thi khác.

Nó vừa đọc `ready` khi `collect` có thể ghi `ready`, vừa đọc `latest` khi `collect` có thể ghi `latest`. Không có thao tác đồng bộ nào nối hai bên.

Vì vậy ở đây không chỉ có một vấn đề "đôi khi in cũ". Có hai cặp truy cập tranh chấp: đọc/ghi `ready` và đọc/ghi `latest`. Việc vòng lặp có tình cờ kết thúc ở một máy không biến các cặp ấy thành có thứ tự. Đặc biệt, nhìn thấy `ready == true` không là bằng chứng ngôn ngữ rằng người đọc cũng nhìn thấy bản `latest` đã chuẩn bị xong.

Bốn tầng không được trộn lẫn

Bảng 7 — Bốn loại bằng chứng khi điều tra race

Tầng	Câu hỏi nó trả lời	Không được suy ra thêm
Nghĩa của ngôn ngữ	Hai lần truy cập thường, cùng vị trí, có ít nhất một lần ghi và không có quan hệ happens-before là race.	Bộ chạy của máy này sẽ xen kẽ ở đúng chỗ nào.
Hành vi được tài liệu cam kết	Lệnh `go` khởi động goroutine mới; nhưng goroutine kết thúc không tự tạo quan hệ đồng bộ cho phần còn lại của chương trình.	Một cờ bool tự công bố mọi lần ghi xảy ra trước nó.
Chi tiết hiện thực	Bộ lập lịch, số P và thời điểm ngắt để chuyển việc quyết định một lần chạy thực tế xen kẽ ra sao.	Đó là hợp đồng ổn định của Go hay của code.
Hành vi đo được	`go test -race` báo những lần truy cập mà lượt chạy đã thực sự đi qua.	Không có báo cáo chứng minh mọi đường chạy đều không có race.

Hai tầng đầu là lời hứa để thiết kế. Tầng thứ ba chỉ hữu ích khi giải thích một vết chạy hoặc một vấn đề hiệu năng; đừng dùng nó để biện minh rằng race "khó xảy ra". Tầng cuối là bằng chứng thực nghiệm mạnh cho đường đã chạy, nhưng nó không thay thế lập luận về thứ tự. Một test xanh không cho phép ta đổi "chưa quan sát thấy" thành "được bảo đảm".

Vẽ chứng minh thay vì đoán lịch chạy

Chương 6 đã dùng race detector để tìm một lần truy cập thực sự xung đột. Ở đây bước kế tiếp không phải chạy lại cùng báo cáo, mà là viết được lời giải thích cho phiên bản đúng trước khi chạy. Ta phân tích trật tự bộ nhớ thông qua ba quan hệ định chuẩn được xác định trong Go Memory Model:

Quan hệ thứ tự	Bản chất ngữ nghĩa trong Go	Ví dụ áp dụng thực tế
Sequenced-before	Thứ tự bộ phận giữa các thao tác trong cùng goroutine, theo luồng điều khiển và quy tắc đánh giá biểu thức của Go; không phải mọi biểu thức đều có thứ tự trái sang phải.	`registry.Add(report)` trả về trước khi `defer` gọi `wg.Done()` trong worker của lab.
Synchronized-before	Cạnh đồng bộ do ngôn ngữ hoặc API công bố.	`mu.Unlock()` xảy ra trước sự trở về của lượt `mu.Lock()` về sau trên cùng mutex; `wg.Done()` xảy ra trước sự trở về của lượt `Wait` mà nó giải phóng.
Happens-before	Quan hệ bao đóng bắc cầu giữa sequenced-before và synchronized-before.	Khi các access được sắp thứ tự và không có race trên vị trí ấy, một read quan sát write gần nhất đứng trước nó trong thứ tự này; cần xét cả những write khác, không chỉ vẽ một cạnh từ write mong muốn.

Cần phân biệt rạch ròi hai cặp khái niệm thường bị nhầm lẫn trong môi trường đồng thời:

Xử lý đồng thời (**concurrency**) là cấu trúc phân tách chương trình thành nhiều phần việc có thể tiến triển trong các khoảng thời gian đan xen chồng chéo; tính toán song song (**parallelism**) là việc thực thi thực tế cùng một thời điểm. Xử lý song song không bắt buộc mỗi luồng phải chạy trên một nhân vi xử lý vật lý riêng biệt, bởi CPU logic (**logical CPU**) cũng có thể là một luồng phần cứng (**hardware thread**). Trong Go, số lượng bộ xử lý logic `P` được cấu hình thông qua `GOMAXPROCS` giới hạn số goroutine thực thi mã nguồn Go đồng thời; các luồng hệ điều hành và cơ chế lập lịch của nhân hệ điều hành tác động trực tiếp đến hành vi quan sát được.

Data race là cặp truy cập vào cùng vị trí nhớ, có ít nhất một lần ghi, không được sắp thứ tự bởi happens-before, và có ít nhất một thao tác không phải đồng bộ. Nếu tất cả truy cập liên quan đều là atomic đúng contract, đó không phải data race; trộn atomic với truy cập thường không tự được miễn trừ. Go vẫn đặt giới hạn cho hành vi của bản cài đặt khi có race, khác với undefined behavior không giới hạn kiểu C/C++. Race vẫn là lỗi nghiêm trọng; tranh chấp trên dữ liệu nhiều từ máy như slice hoặc interface có thể gây hỏng bộ nhớ. Lỗi chạy đua ở tầng logic lại có thể tồn tại dù mọi truy cập

đã dùng mutex đúng cách: thứ tự hợp lệ về bộ nhớ chưa chắc hợp lệ với bất biến nghiệp vụ.

Đừng biến ba tên thành câu thần chú. Với mỗi kết quả cần đọc, hãy vẽ đường cụ thể. Trong ví dụ `ready`, không có cạnh nào từ `latest = ...` sang việc `show` đọc `latest`; vì vậy lời giải không thể chỉ là thêm một vòng lặp chờ.

Một khóa bảo vệ một bất biến, không bảo vệ một dòng

Khóa `sync.Mutex` phù hợp với dữ liệu thực sự thuộc quyền sở hữu chung, khi nhiều goroutine cần luân phiên biến đổi hoặc chụp ảnh trạng thái cùng một vùng nhớ. Bất biến kỹ thuật của phòng thực hành mới là: sau khi tiến trình điều phối đã chờ toàn bộ công nhân xử lý hoàn tất, bản chụp dữ liệu phải chứa chính xác một báo cáo từ mỗi công nhân. Khóa bảo vệ lát cắt `reports` trong suốt quá trình bổ sung và sao chép; `sync.WaitGroup` cung cấp bằng chứng cho tiến trình điều phối biết chính xác thời điểm có thể đánh giá bất biến.

Trước khi tham khảo `fixed/registry.go`, hãy đọc tệp `exercise/registry_test.go`. Bài kiểm thử đòi hỏi kiểu cấu trúc `Registry`, phương thức `Add(Report)` và phương thức `Snapshot() []Report`. Hãy tự quyết định trường dữ liệu nào cần khai báo, phương thức nào sử dụng biến nhận con trỏ (**pointer receiver**), và phạm vi khóa (**critical section**) cần bao trùm tới ranh giới nào. Lệnh thực thi ban đầu sẽ báo lỗi đỏ vì mã nguồn chưa được lập trình:

```
go test -tags exercise ./exercise
```

Tuyệt đối không giải quyết bằng cách trả về trực tiếp lát cắt nội bộ. Bài học về hiện tượng trùng lặp tham chiếu (**aliasing**) ở Chương 2 đã chứng minh lý do: bên gọi nắm giữ lát cắt trả về có thể biến đổi các phần tử bên trong nó. Phương thức `Snapshot` cam kết cung cấp một bản chụp độc lập của thùng chứa dữ liệu, do đó phòng thực hành sử dụng hàm `copy` để ngăn bên gọi ghi đè phần tử bên trong sổ đăng ký (**registry**). Kiểu `Report` chỉ chứa giá trị chuỗi thuần túy nhằm giữ trọng tâm bài học vào trật tự thực thi; quyền sở

hữu phức tạp của các đối tượng lồng nhau sẽ được phân tích khi xuất hiện nhu cầu thực tế.

Một bản cài đặt đạt chuẩn không cần bận tâm bộ lập lịch sẽ kích hoạt luồng công nhân nào trước. Nó chỉ cần bảo đảm mỗi phương thức `Add` xác lập khóa trước khi nối phần tử và giải phóng khóa ngay sau đó, đồng thời `Snapshot` giữ khóa trong suốt quá trình nhân bản dữ liệu. Tài liệu gói `sync` cam kết thao tác `Unlock` xảy ra trước một thao tác `Lock` kế tiếp trên cùng một khóa. Đây là giao ước kỹ thuật của giao diện lập trình, không phải suy diễn cảm tính từ số lượng nhân CPU hay một lần kiểm thử vượt qua ngẫu nhiên.

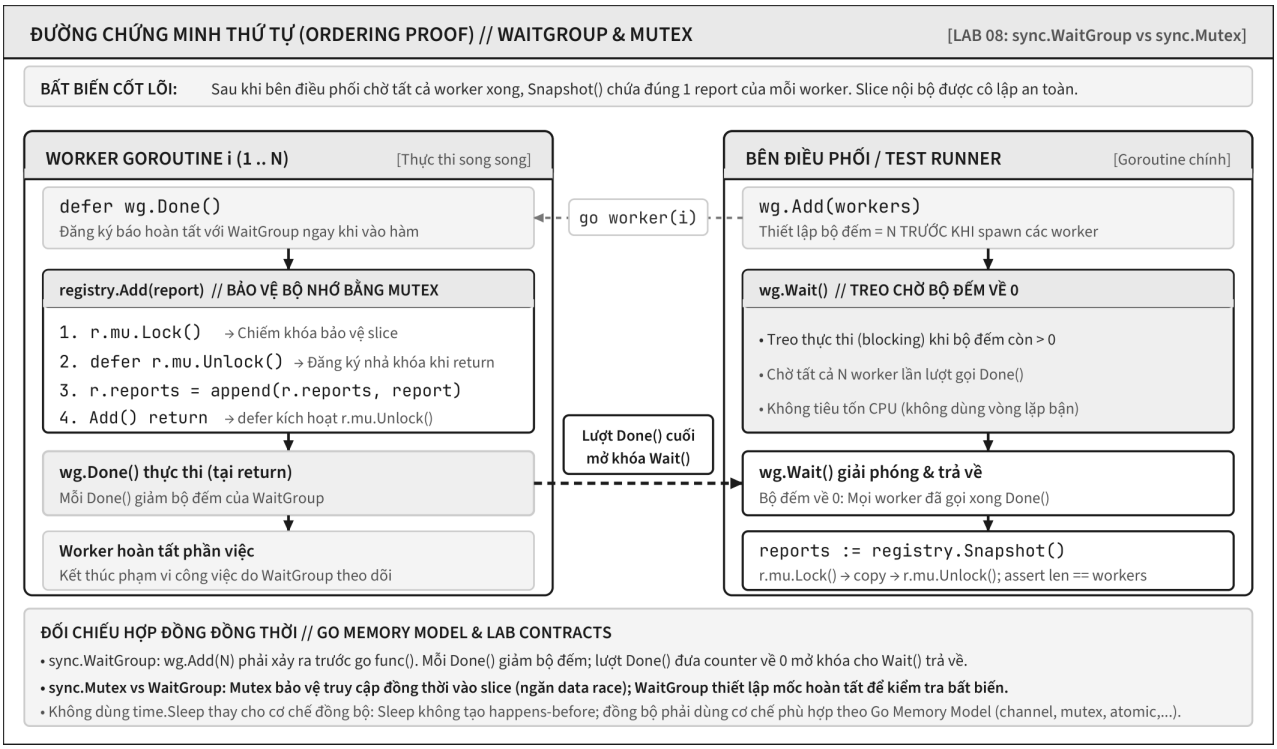
Thử một bất biến khác: `balance + reserved == 100`. Bên ghi đổi cặp `(100, 0)` thành `(60, 40)` dưới một khóa. Nếu bên đọc khóa để lấy `balance`, mở khóa, rồi khóa lại để lấy `reserved`, nó có thể ghép `100` của bản cũ với `40` của bản mới. Không truy cập nào thiếu khóa, nhưng tổng `140` chưa từng là trạng thái hợp lệ. `TestFieldLocksDoNotMakeAnInvariantSnapshot` trong lab dùng hai tín hiệu để buộc đúng lịch xen kẽ này, không chờ bằng `Sleep`. Bản đọc giữ khóa xuyên suốt cả hai trường chỉ có thể nhận một cặp nhất quán. Race detector sạch không thay được assertion về bất biến.

`sync.RWMutex` cũng phải bảo vệ toàn bộ bất biến ấy. Nhiều bên đọc có thể cùng giữ `RLock`, nhưng trong vùng này không được ghi dữ liệu chung chỉ vì gọi đó là thao tác “đọc”. `Unlock` của bên ghi đồng bộ với sự trở về của `RLock` về sau; `RUnlock` tương ứng đồng bộ với sự trở về của lượt `Lock` kế tiếp. Không nâng `RLock` thành `Lock` ngay trong vùng khóa, không hạ `Lock` thành `RLock`, và không lấy khóa đọc đệ quy: một bên ghi đang chờ có thể khiến lượt `RLock` tiếp theo bị chặn. Đây là contract API, không phải lời hứa RWMutex luôn nhanh hơn Mutex.

WaitGroup tạo mốc hoàn tất, không thay Mutex

Test đặt `wg.Add(workers)` trước khi tạo goroutine. Mỗi worker gọi `registry.Add(report)` rồi mới `wg.Done()` qua `defer`. `Wait` chỉ trả về khi bộ đếm về zero; không phải từng lượt `Done` đều tự mở khóa bên chờ. Tài liệu `sync` quy định `Done` đồng bộ trước sự trở về của lượt `Wait` mà nó giải phóng. Trong lab, mutex bảo vệ việc thêm và sao chép báo cáo; WaitGroup bổ sung mốc để biết mọi lượt thêm được đăng ký đã hoàn tất. Nó không bảo đảm worker thành công, không dừng worker, và không chờ công việc nằm sau `Done` hoặc công việc không được đăng ký. Không được gọi `Add` dương trên bộ đếm zero sau khi đã bắt đầu `Wait` cho đợt việc đó.

Đường chứng minh cho một worker có thể đọc thành lời:



Hình 20 — Mốc hoàn tất và bảo vệ dữ liệu là hai quan hệ khác nhau: bộ đếm WaitGroup xác định khi nào đợt việc đã xong; mutex giữ bất biến của reports. Snapshot chỉ chạy sau khi Wait trả về và vẫn phải giữ mutex.

Các mũi tên nét liền thể hiện trật tự tuần tự trong mã nguồn; quan hệ `Done → Wait returns` là cam kết mà `sync.WaitGroup` xác lập. Khóa loại trừ tương hỗ bảo đảm các thao tác truy cập vào lát cắt không bị chồng lấn thời gian. `WaitGroup` không thể thay thế cho cơ chế khóa: nếu các tiến trình công nhân cùng đồng thời bổ sung phần tử vào một lát cắt mà không có khóa

bảo vệ, sự cố chạy đua dữ liệu vẫn sẽ bùng phát cho dù luồng chính có kiên nhẫn chờ đến khi tất cả kết thúc mới thống kê kết quả.

Sau khi tự làm, chạy bản tham chiếu độc lập:

```
go test -race ./fixed
```

Kết quả kiểm thử thành công cung cấp đồng thời hai loại bằng chứng kỹ thuật: bài kiểm thử xác nhận bất biến nghiệp vụ cụ thể, trong khi công cụ phát hiện chạy đua không tìm thấy bất kỳ truy cập xung đột nào trên nhánh thực thi đó. Tuy nhiên, điều này không đồng nghĩa với lời khẳng định rằng mọi giao diện lập trình bổ sung trong tương lai vào `Registry` đều tự động an toàn. Bất kỳ phương thức mới nào thao tác đọc hoặc ghi lên `reports` đều bắt buộc phải quay lại rà soát bất biến và phạm vi bao bọc của khóa.

Điều `go` cho phép, điều nó không hứa

Câu lệnh `go f()` thiết lập quan hệ xảy ra trước từ thời điểm kích hoạt đến lúc hàm `f` bắt đầu chạy. Cơ chế này bảo đảm goroutine mới nhìn thấy đầy đủ dữ liệu đã chuẩn bị trước lệnh `go`; nhưng nó không cung cấp chiều ngược lại. Việc hàm `f` kết thúc hay goroutine hoàn tất không tự động phát tín hiệu cho các goroutine khác biết công việc đã xong. Đó là lý do `sync.WaitGroup` trong phòng thực hành là một mắt xích chứng minh hình thức, không phải là thủ thuật tạm bợ để giảm thiểu tình trạng kiểm thử chập chờn (**flaky test**).

Tuyệt đối không sử dụng `time.Sleep` để lấp liếm mũi tên quan hệ còn thiếu. Việc tạm dừng thời gian chỉ là phỏng đoán may rủi rằng tiến trình công nhân có đủ thời gian hoàn thành trên một lượt chạy; nó hoàn toàn không xác lập bất kỳ trật tự xảy ra trước nào và khiến bộ kiểm thử vừa chậm chạp vừa bất định. Kênh truyền ở Chương 9 sẽ giải quyết trọn vẹn bài toán bàn giao công việc và vòng đời; trước thời điểm đó, ta cần phân biệt rạch ròi giữa việc công bố trạng thái an toàn với việc bảo vệ trạng thái đang liên tục bị biến đổi.

Công bố state và chờ điều kiện

`sync.Once` duy trì giao ước kỹ thuật "thực thi đúng một lần duy nhất", chứ không phải "thử lại cho đến khi thành công". Khi nhiều bên gọi cùng thực thi `Do(f)` trên cùng một đối tượng `Once`, sự kiện kết thúc của hàm `f` được bảo đảm xảy ra trước sự trở về của toàn bộ các lời gọi `Do` liên quan. Tuy nhiên, nếu hàm `f` phát sinh lỗi hoảng loạn (`panic`), `Once` vẫn ghi nhận lượt thực thi đó đã hoàn tất; bên gọi dù có phục hồi lỗi (`recover`) rồi gọi lại `Do` cũng không thể kích hoạt hàm `f` chạy lại lần thứ hai. Ca kiểm thử `TestOncePanicIsNotRetry` trong `labs/edition-contracts` bảo vệ chính phản ví dụ này. Quá trình khởi tạo tài nguyên phụ thuộc có nguy cơ lỗi tạm thời đòi hỏi chính sách thử lại (**retry policy**) tường minh; không được che giấu rủi ro bên trong `Once` với hy vọng hệ thống sẽ tự khôi phục kỳ diệu.

Cấu trúc `sync.Cond` phục vụ bài toán hoàn toàn khác: chờ đợi cho đến khi vị từ điều kiện (**predicate**) được bảo vệ bởi khóa chuyển sang trạng thái hợp lệ. Phương thức `Wait` giải phóng khóa, tạm hoãn thực thi chờ tín hiệu, rồi tự động tái xác lập khóa trước khi trở về. Tín hiệu thông báo (**notification**) không phải là thẻ trạng thái có thể tích lũy cho bên gọi trong tương lai; chính điều kiện vị từ mới là trạng thái cốt lõi cần được kiểm tra cẩn trọng dưới sự bảo vệ của khóa:

```
mu.Lock()
for !ready {
    cond.Wait()
}
// Đọc state được bảo vệ khi vẫn giữ lock.
mu.Unlock()
```

Vòng lặp `for` bao quanh không hàm ý Go cho phép `Wait` tự động thức tỉnh vô cớ: tài liệu chính thức quy định rõ ràng cần có lời gọi `Signal` hoặc `Broadcast` để phương thức `Wait` trở về. Vòng lặp này tối quan trọng vì tại thời điểm goroutine tái đoạt được khóa, một goroutine khác có thể đã biến đổi dữ liệu khiến vị từ điều kiện không còn thỏa mãn. Bên sản xuất (**producer**) biến đổi vị từ dưới cùng một khóa bảo vệ, sau đó phát tín hiệu thông báo theo chính sách xác định. Nếu bên sản xuất đã thiết lập `ready = true` trước khi bên tiêu thụ tiếp cận, bên tiêu thụ sẽ trực tiếp đọc vị từ mà

không cần truy vết tín hiệu thông báo trong lịch sử. Đây là nền tảng cốt lõi cho mô hình hàng đợi ở Chương 21, chứ không phải lý do lạm dụng `Cond` thay thế cho kênh truyền trong mọi tình huống.

Kỹ thuật công bố nguyên tử (**atomic publication**) phát huy tối đa hiệu quả khi bên đọc chỉ tiếp nhận một bản chụp dữ liệu (**snapshot**) đã hoàn tất toàn vẹn, trong khi bên ghi tiến hành thay thế toàn bộ bản chụp mới thay vì đột biến trực tiếp đối tượng đang lưu hành. Kiểu dữ liệu `atomic.Pointer[Config]` cho phép ghi nhận (**Store**) con trỏ mới và tải về (**Load**) con trỏ hiện hành; các thao tác nguyên tử này tuân thủ trật tự nhất quán tuần tự (**sequentially consistent**) theo đặc tả của gói `sync/atomic`. Tuy nhiên, thao tác nguyên tử trên con trỏ không tự động bảo vệ an toàn cho các trường dữ liệu nằm sau con trỏ đó. Nếu sau khi `Store` mà mã nguồn lại biến đổi bảng ánh xạ (**map**), lát cắt hay trường dữ liệu của chính đối tượng đó trong lúc bên đọc đang truy xuất, hiện tượng chạy đua dữ liệu vẫn sẽ bùng phát.

Trong ca kiểm thử `TestCondPredicateAndAtomicPublication`, mỗi cấu hình được chuẩn bị trước `Store` và không bị sửa về sau. Source thể hiện chính sách bất biến; test kiểm tra những giá trị và truy cập đã chạy, không chứng minh mọi người dùng tương lai tuân thủ chính sách đó. Atomic không tự tạo giao dịch nhiều trường, không giải quyết quyền sở hữu lồng nhau và không luôn nhanh hơn Mutex.

Một cờ atomic có thể công bố dữ liệu thường nếu giao thức là một lần chuẩn bị rồi chỉ đọc. Thay `ready bool` trong ví dụ đầu bằng `ready atomic.Bool`, gọi `ready.Store(true)` sau khi ghi `latest`, và chỉ đọc `latest` sau khi `ready.Load()` quan sát `true`: lần Store đó đồng bộ trước Load quan sát nó; kết hợp thứ tự trong hai goroutine, lần ghi `latest` đứng trước lần đọc. Không được tiếp tục ghi `latest` sau công bố trong khi người đọc còn dùng nó. Reset cờ rồi dùng lại cùng ô nhớ cũng không tự chờ người đọc cũ kết thúc. Nếu một vị trí được truy cập bằng cả atomic lẫn thao tác thường mà không có quan hệ thứ tự thích hợp, nó vẫn có thể race; atomic không "bao bọc" mọi truy cập khác. `TestAtomicFlagPublishesImmutablePayload` kiểm tra giao thức một

lần, còn đối chứng sửa payload đã công bố được chạy riêng với race detector. Vòng quay chờ trong phép thử chỉ cô lập quan hệ bộ nhớ; chương trình thực tế thường nên chờ bằng cơ chế không quay CPU.

Chương này kết thúc khi người học quan sát hai goroutine không còn hỏi “có chạy cùng lúc không?”, mà hỏi “lần truy cập nào cùng dữ liệu, bất biến nào cần giữ, và đường happens-before nào làm điều đó đúng?”. Câu hỏi ấy sẽ tiếp tục đồng hành qua channel, bộ xử lý HTTP, bộ nhớ đệm và mã nguồn runtime; cơ chế kỹ thuật có thể thay đổi, nhưng trách nhiệm chứng minh thứ tự luôn bất biến.

Chương 9 — Dòng công việc có áp suất

Mô hình tinh thần của chương này là một send trên channel là một lần bàn giao còn dang dở cho tới khi bên tiêu thụ nhận nó, bộ đệm lưu giữ nó, hoặc người gửi chọn dừng. Channel không biến công việc thành "song song an toàn" chỉ bằng một dấu `<-`. Nó buộc ta viết rõ ai còn nợ ai một giá trị, ai báo không còn giá trị nào nữa, và điều gì xảy ra khi người nhận không còn chờ.

Hãy bắt đầu bằng một tiện ích có vẻ hợp lý. `firstResult` chạy nhiều probe, nhận kết quả đầu tiên rồi return. Tên này không hứa kết quả thành công: kết quả đến đầu tiên vẫn có thể chứa lỗi. Đoạn trích tập trung vào điểm gửi, chưa phải một hàm hoàn chỉnh:

```
results := make(chan Result)

for _, probe := range probes {
    go func(probe Probe) {
        result := probe(ctx)
        results <- result
    }(probe)
}

select {
case result := <-results:
    return result, nil
case <-ctx.Done():
    return Result{}, ctx.Err()
}
```

Nếu một tác vụ kiểm tra hoàn tất trước, bên gọi tiếp nhận một `Result` và lập tức thoát khỏi hàm. Những tác vụ kiểm tra hoàn tất sau vẫn cố gắng gửi dữ liệu. Do `results` là kênh truyền không có bộ đệm (**unbuffered channel**) và không còn bên nhận nào túc trực, chúng bị phong tỏa vĩnh viễn tại lệnh `results <- result`. Đây chính là hiện tượng rò rỉ goroutine (**goroutine leak**): công việc đã kết thúc nhưng goroutine không thể giải phóng vì nó đang nắm giữ một giao dịch bàn giao không còn ai tiếp nhận. Đối tượng `context` không thể tự giải cứu lệnh gửi này; nó chỉ là một tín hiệu mà mã nguồn phải chủ động lắng nghe.

Dừng đúng chỗ send

Một worker gửi kết quả phải nhìn đồng thời hai khả năng: consumer sẵn sàng nhận, hoặc request đã hết lý do để tiếp tục. `select` đặt hai quyết định ấy cạnh nhau:

```
select {
case out <- result:
    // Giá trị đã được nhận hoặc nằm trong buffer.
case <-ctx.Done():
    // Không có lần bàn giao nào nữa.
    // Worker được phép kết thúc.
}
```

Đây không phải là một khoảng thời gian chờ (**timeout**) chung chung. Nếu kênh `out` chưa có bên nhận và ngữ cảnh chưa bị hủy bỏ, tiến trình công nhân sẽ chờ đợi có chủ đích: áp lực nghẽn từ bên tiêu thụ (**consumer backpressure**) đã truyền ngược trở lại công nhân. Nếu ngữ cảnh bị hủy, công nhân sẽ từ bỏ kết quả chưa bàn giao nhằm tránh bị kẹt cứng. Việc hàm `Work` đang chạy bên trong công nhân có lập tức dừng lại hay không lại thuộc về một giao ước độc lập: `ctx` là tín hiệu yêu cầu hủy bỏ mang tính hợp tác, không thể cưỡng bức một hàm hoặc lời gọi hệ thống bất kỳ phải lập tức chấm dứt.

Cấu trúc `select` chỉ lựa chọn một nhánh giao tiếp đang ở trạng thái có thể tiến triển. Khi nhiều nhánh `case` cùng sẵn sàng đồng thời, đặc tả ngôn ngữ Go yêu cầu lựa chọn ngẫu nhiên giả (**pseudo-random**) với xác suất đồng đều; nó hoàn toàn không cam kết thứ tự ưu tiên, tính công bằng dài hạn hay trật tự logic nghiệp vụ. Nhánh `default` khiến `select` không bao giờ bị nghẽn (**non-blocking**), do đó rất dễ vô tình đánh tráo trạng thái "bên tiêu thụ xử lý chậm" thành hành vi "âm thầm bỏ rơi dữ liệu". Chỉ nên sử dụng `default` khi việc loại bỏ dữ liệu hoặc chuyển hướng sang cơ chế dự phòng là chính sách vận hành đã được quy định tường minh.

Buffer là khoảng nợ hữu hạn

Với kênh truyền không có bộ đệm, bên gửi và bên nhận bắt buộc phải đồng bộ trực tiếp tại cùng một thời điểm giao tiếp (**rendezvous**). Với `make(chan Result, 8)`, bên gửi có thể tích lũy trước tối đa tám kết quả chưa được bên tiêu thụ tiếp nhận. Con số dung lượng 8 không giúp công việc tính toán nhanh hơn, không phản ánh số lượng luồng công nhân, và không phải bằng chứng xác nhận hệ thống có thể chịu tải tám yêu cầu mạng đồng thời. Nó đơn thuần là hạn mức công việc ứ đọng (**backlog quota**) mà chương trình cho phép lưu giữ tạm thời trong bộ nhớ trước khi áp lực truyền ngược trở lại bên gửi.

Bảng 8 — Buffer là backlog hữu hạn, không phải số worker

Thiết kế	Sender được đi trước bao xa?	Câu hỏi phải trả lời
`make(chan T)`	Không có chỗ đệm; cần receiver sẵn sàng.	Người gửi có được phép chờ consumer không?
`make(chan T, n)`	Tối đa `n` value chưa nhận.	`n` đại diện quota bộ nhớ hay burst nào? Khi đầy thì policy là gì?
`select` có `default`	Có thể không gửi dù channel chưa bị đóng.	Bỏ, log, retry hay trả lỗi có được hứa không?

Đây là ngữ nghĩa của đặc tả ngôn ngữ, không phải mô hình lập lịch nội bộ: kênh truyền có bộ đệm chỉ điều chỉnh thời điểm các thao tác gửi hoặc nhận được phép tiếp tục tiến triển. Mô hình bộ nhớ Go còn cam kết thao tác gửi xảy ra trước thời điểm thao tác nhận tương ứng hoàn tất. Giao ước này bảo đảm chuyển giao an toàn một giá trị cùng toàn bộ các bước tính toán đã được chuẩn bị trước thao tác gửi; nó hoàn toàn không cho phép nhiều goroutine cùng đồng thời biến đổi trạng thái tùy tiện.

Chiều ngược lại cần phân biệt buffer. Với kênh không đệm, lần nhận đồng bộ trước khi lần gửi tương ứng hoàn tất; bên gửi vì thế biết đã có người nhận, không biết người đó đã xử lý xong. Với capacity `C`, lần nhận thứ `k` đồng bộ trước khi lần gửi thứ `k+C` hoàn tất: muốn gửi thêm khi buffer đầy phải có chỗ được giải phóng. Lần gửi đầu vào kênh có đệm có thể hoàn tất trước bất

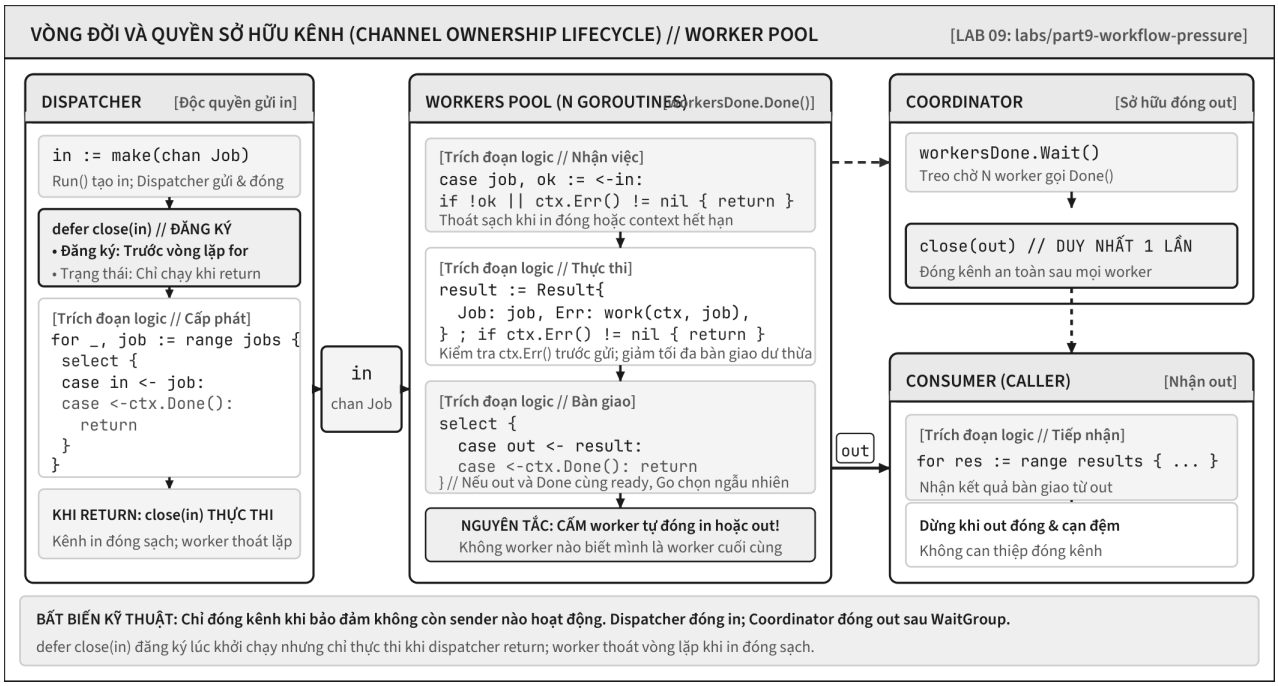
kỳ lần nhận nào. Nếu cần chờ xử lý xong, hãy thiết kế tín hiệu xác nhận riêng.

Các test `TestBufferedAdmissionIsNotCompletion`, `TestUnbufferedReceivePublishesBeforeSendReturns` và `TestFullBufferNeedsAReceive` trong lab buộc từng trạng thái bằng tín hiệu hoặc `testing/synctest`, không dự đoán lịch chạy. Dung lượng channel giới hạn số giá trị trong buffer, không giới hạn số goroutine đang chờ gửi hay tổng byte của dữ liệu được tham chiếu. Gửi một slice hay con trỏ vẫn sao chép giá trị dẫn tới dữ liệu chung; quyền thay đổi dữ liệu sau bàn giao là chính sách phải thiết kế, không phải phép sao chép sâu của channel.

Close nói về người gửi, không nói "mọi thứ ổn"

`close(results)` chấm dứt việc gửi, không chứng nhận mọi job thành công hay mọi kết quả đã được xử lý. Gửi vào channel đã đóng gây panic; đóng channel khi có bên đang chờ gửi cũng không phải đường hủy an toàn cho bên gửi đó. Giá trị đã nằm trong buffer vẫn được nhận trước, rồi phép nhận trả zero value với `ok == false`. Một giá trị zero đã được gửi hợp lệ vẫn có `ok == true`. `range` dừng sau khi channel đã đóng và buffer đã cạn. Việc đóng đồng bộ trước lần nhận trả zero value do channel đã đóng, không thay cho tín hiệu xác nhận xử lý xong từng kết quả.

Trong thiết kế này, quyền đóng thuộc bên điều phối việc gửi: dispatcher đóng đầu vào, coordinator chờ mọi worker rồi đóng đầu ra đúng một lần. Đây là quy ước sở hữu của chương trình, không phải quy tắc compiler rằng "chỉ sender được đóng". Một biến `chan T` hoặc `chan<- T` vẫn cho phép gọi `close`; kiểu chỉ nhận thì không. Không để worker đầu tiên hoàn tất tự đóng đầu ra trong khi các worker khác còn có thể gửi.



Hình 21 — Sơ đồ này là mô hình tư duy về quyền sở hữu (**ownership**), không phải dòng thời gian của bộ lập lịch. Mỗi kênh truyền sở hữu duy nhất một chủ thể phát tín hiệu kết thúc, và mọi nhánh xử lý của công nhân đều phải có đường thoát an toàn khi bên tiêu thụ không còn tiếp nhận.

Đọc một worker pool như một hợp đồng đóng kênh

Mã nguồn phòng thực hành `labs/part9-workflow-pressure` không kết nối trực tiếp vào `opsprobe`: chúng ta chưa có yêu cầu vận hành thực tế nào đòi hỏi công cụ dòng lệnh đó phải xử lý nhiều điểm cuối đồng thời. Việc sử dụng một mô hình tái hiện độc lập giúp giữ sắc bén một câu hỏi kỹ thuật cốt lõi: một bên sản xuất, nhiều công nhân và một bên tiêu thụ sẽ hoàn tất với bao nhiêu kết quả, và tại thời điểm nào?

Mở `exercise/pool_test.go` trước. Test không cho sẵn hiện thực; nó chỉ đòi các type `Job`, `Result`, `Work` và function `Run`. Bốn cam kết kỹ thuật cấu thành hợp đồng thực thi của `Run` bao gồm:

Điều khoản hợp đồng	Mục tiêu kiểm chứng	Rủi ro hệ thống nếu vi phạm
Bảo toàn kết quả khi không hủy	Nếu không hủy và consumer tiếp tục nhận, mỗi job tạo đúng một `Result`, kể cả khi Work trả lỗi. Khi hủy, kết quả chưa bàn giao có thể bị bỏ theo policy.	Mất kết quả hoặc lặp kết quả ngoài contract.
Giới hạn tài nguyên song song	Số lượng lời gọi hàm `Work` thực thi đồng thời không bao giờ vượt quá ngưỡng `workers`.	Tràn bộ đệm, quá tải kết nối cơ sở dữ liệu hoặc cạn kiệt CPU.
Cơ chế giải phóng khi hủy bỏ	Với `Work` của fixture có thể hoàn tất trả về, context bị hủy cho worker thoát lệnh gửi dù kết quả đầu ra không được đọc; tiến trình điều phối đóng kết quả đầu ra sau khi worker trả về.	Worker giữ một lần bàn giao không còn người nhận. Nếu `Work` không trả về, nhóm worker không có quyền cưỡng bức nó dừng.
Chu kỳ đóng kênh duy nhất	Coordinator chờ cả dispatcher lẫn worker kết thúc rồi đóng output đúng một lần. Hết output là mốc kết thúc đợt việc của lab, không chứng nhận mọi job thành công.	Panic do đóng nhiều lần hoặc goroutine con chưa được join.

Có một ranh giới nhỏ nhưng đáng lưu tâm ngay từ đầu: nếu `ctx` đã bị hủy trước khi bộ phân phối kịp bàn giao công việc, `Work` không được phép khởi chạy chỉ vì một worker vừa được tạo. Hủy bỏ tác vụ (**cancellation**) là chính sách dừng tiếp nhận công việc mới; việc đưa một `Job` vào `Work` với ngữ cảnh đã hết hạn chỉ tạo thêm một kết quả không còn bên tiếp nhận.

Chạy lệnh khởi đầu; nó phải đỏ vì API chưa tồn tại:

```
go test -tags exercise ./exercise
```

Tránh tăng kích thước bộ đệm (**buffer**) nhằm che giấu thất bại trong kiểm thử. Ca kiểm thử hủy bỏ tác vụ chủ đích không tiêu thụ kết quả đầu ra; một giải pháp cài đặt chỉ gồm thao tác gửi đơn thuần `out <- result` sẽ làm lộ goroutine bị tắc nghẽn. Đồng thời, không đóng kênh `results` từ bên trong worker. Hãy phác thảo trên giấy ba chủ thể sở hữu (**owner**) trước khi viết

mã: bộ phân phối sở hữu thao tác gửi và gọi `close(jobs)`; các worker chỉ nhận công việc rồi gửi kết quả; tiến trình điều phối sở hữu thao tác `close(results)` sau khi gọi `WaitGroup.Wait`.

Một khung đủ nhỏ cho điểm send quan trọng là:

```
result := Result{Job: job, Err: work(ctx, job)}

if ctx.Err() != nil {
    return
}

select {
case out <- result:
case <-ctx.Done():
    return
}
```

`ctx.Err()` đặt trước mệnh đề `select` là chính sách có chủ đích của bài thực hành: nếu tín hiệu hủy đã được quan sát sau khi hoàn thành `Work`, kết quả đó không còn được bàn giao, kể cả khi bên tiêu thụ (**consumer**) vừa sẵn sàng trở lại. Bên trong `select`, tín hiệu hủy vẫn mở đường thoát cho tình huống bên tiêu thụ biến mất trong lúc worker đang chờ. Phần còn lại là thiết kế do người học chủ động triển khai: bộ phân phối đưa `jobs` vào kênh như thế nào để đồng thời lắng nghe `ctx.Done`; worker tiếp nhận công việc đến khi đóng kênh `jobs` ra sao; goroutine nào đảm trách việc chờ `WaitGroup`; và trường hợp `workers <= 0` sẽ trả về kết quả đóng ngay hay bị từ chối tại ranh giới giao diện lập trình (API). Ca kiểm thử hiện tại ấn định chính sách thứ nhất nhằm tập trung câu hỏi vào việc kiểm soát vòng đời tác vụ.

Đừng chỉ kiểm tra việc hủy bỏ ở bộ điều phối (**dispatcher**). Một công việc có thể đã được bàn giao đúng lúc tín hiệu hủy xảy ra; worker cần kiểm tra lại trước khi gọi `Work`. Điều này không cưỡng bức một lệnh `Work` đang chạy phải dừng ngay lập tức (bởi chính `Work` còn phải tôn trọng ngữ cảnh context của nó), nhưng nó giữ được cam kết hẹp hơn và kiểm chứng được: sau khi tín hiệu hủy đã được quan sát, pool không khởi động thêm một công việc mới.

Sau khi tự làm, chạy bản tham chiếu với race detector:

```
go test -race ./fixed
```

Hướng của channel cho thấy quyền hạn

Trong hàm nội bộ, `chan Job` có cả quyền gửi và nhận. Ranh giới giao diện lập trình có thể thu hẹp quyền hạn đó:

```
func dispatch(ctx context.Context, jobs []Job) <-chan Job
func consume(in <-chan Job, out chan<- Result)
```

Đây là chữ ký API, chưa có thân hàm để chạy riêng. Giá trị `dispatch` trả cho bên gọi chỉ cho phép nhận. Bên trong `consume`, tham số `in` chỉ cho phép nhận, còn `out` chỉ cho phép gửi và vẫn cho phép `close`; quyền này thuộc biểu thức trong hàm, không thu hẹp biến channel hai chiều mà bên gọi đang giữ. Direction ngăn một số thao tác sai kiểu, nhưng không chứng minh ai là chủ thể duy nhất được đóng channel.

Khi đọc mã nguồn xử lý kênh, đừng bắt đầu bằng câu hỏi kênh này có bộ đệm (**buffered**) hay không. Hãy đặt câu hỏi theo thứ tự chặt chẽ hơn: giá trị nào đang được bàn giao, bên nào phải còn hoạt động để tiếp nhận giá trị đó, ai nắm quyền tuyên bố kết thúc việc gửi dữ liệu, và tín hiệu hủy bỏ tác vụ có mở đường thoát cho mọi điểm nghẽn chặn (**blocking**) hay không. Trả lời thỏa đáng bốn câu hỏi này trước khi lựa chọn dung lượng bộ đệm (**capacity**) thường ngăn chặn được hiện tượng rò rỉ, khóa chết (**deadlock**) và hàng đợi phình to vô hạn.

Đã hủy công việc, vì sao goroutine vẫn còn?

Ví dụ đầu chương bị nghẽn ở thao tác gửi; việc chuyển sang thao tác nhận không tự động loại bỏ vấn đề. Trong `baseline.Wait` của bài thực hành `part10-resource-retention`, bên gọi truyền ngữ cảnh `context` rồi hủy nó, nhưng goroutine chỉ thực hiện thao tác nhận `<-in`. Khi không có bên gửi (**sender**), goroutine tiếp tục chờ đợi. Ngữ cảnh là một tín hiệu phối hợp, không phải thẩm quyền để bên gọi yêu cầu môi trường thực thi (**runtime**) cưỡng bức chấm dứt một goroutine khác. Bản sửa đặt thao tác nhận và `ctx.Done()` trong cùng mệnh đề `select`, sau đó đóng kênh `done` khi goroutine thực sự kết thúc trả về. Việc tiếp nhận tín hiệu từ `done` là bằng chứng xác nhận hoàn tất; việc gọi `cancel()` mới chỉ là phát đi yêu cầu dừng.

Ca kiểm thử sử dụng tín hiệu `started` nhằm bảo đảm goroutine đã thực sự khởi chạy, sau đó hủy ngữ cảnh và chờ tín hiệu từ `done`. Bản phát hiện lỗi bổ sung kênh giải phóng dự phòng do ca kiểm thử quản lý để mở đường thoát sau khi ghi nhận lỗi, tránh để chính mã kiểm thử gây rò rỉ goroutine. Thao tác đếm số lượng goroutine hỗ trợ quá trình điều tra, nhưng kiểm thử hồi quy cần chờ đợi chính xác công việc mà nó sở hữu thay vì đo đạc toàn bộ tiến trình hệ điều hành (**process**) rồi kỳ vọng một con số cố định.

Vòng lặp định kỳ đặt ra một bẫy khác. Hãy đọc đoạn mã dưới đây và chọn thao tác có thể kết thúc goroutine: hủy context bên ngoài, gọi `ticker.Stop()`, hay tạo một đường return riêng?

```
for range ticker.C {
    refresh()
}
```

Phương thức `Ticker.Stop` ngừng phát nhịp định kỳ (**tick**) nhưng không đóng kênh `ticker.C`. Do đó vòng lặp `range` không được đánh thức để kết thúc chỉ nhờ thao tác dừng này; tiến trình có thể chuyển từ trạng thái chạy liên tục sang trạng thái chờ đợi vô hạn. Việc hủy một ngữ cảnh mà vòng lặp không lắng nghe cũng không đem lại hiệu quả. Bài thực hành chuẩn hóa vòng đời quản lý theo cấu trúc sau; khi có nhịp phát sinh, hàm `work` vẫn

phải tuân thủ giao ước tôn trọng ngữ cảnh nếu bên gọi yêu cầu kết thúc hệ thống trong khoảng thời gian hữu hạn (**graceful shutdown**):

```
for {
    select {
    case <-ctx.Done():
        return
    case <-ticker.C:
        if ctx.Err() != nil {
            return
        }
        work(ctx)
    }
}
```

Bên sở hữu gọi `defer ticker.Stop()` và đóng kênh `done` khi vòng lặp kết thúc. Việc kiểm tra `ctx.Err()` trước khi thực thi `work` giúp tránh bắt đầu công việc khi đã ghi nhận tín hiệu hủy; thao tác này không bảo đảm tín hiệu hủy không xuất hiện ngay sau bước kiểm tra. Nếu hàm phản hồi (**callback**) đang bị nghẽn trong một thao tác nhận dữ liệu không kiểm tra ngữ cảnh, vòng lặp chưa thể quay lại mệnh đề `select`. Ca kiểm thử `TestCancellationDoesNotKillNoncooperativeCallback` chủ ý sử dụng hàm phản hồi như vậy: việc hủy ngữ cảnh chưa thể đóng kênh `done`, đòi hỏi phải giải phóng hàm phản hồi trước. Ca kiểm thử kế tiếp thay thế hàm phản hồi bằng tác vụ lắng nghe đồng thời cả kênh `release` và `ctx.Done()`, qua đó bảo đảm khả năng kết thúc. Việc truyền đối số ngữ cảnh là chưa đủ; tác vụ bắt buộc phải kiểm tra ngữ cảnh tại mọi điểm có khả năng gây nghẽn chặn (**blocking**).

Tránh khái quát hóa sai lầm về vòng đời này thành nhận định: quên gọi `Stop` luôn dẫn đến rò rỉ bộ nhớ (**memory leak**). Tài liệu gói `time` từ phiên bản Go 1.23 xác nhận bộ thu gom rác (**Garbage Collector - GC**) có thể thu hồi đối tượng định thời không còn tham chiếu dù chưa gọi `Stop`. Trong các phiên bản Go 1.23–1.26, cấu hình phiên bản module và thiết lập `asynctimerchan` vẫn có thể kích hoạt cơ chế cũ. Go 1.27 đã loại bỏ hoàn toàn cờ thiết lập này và mặc định sử dụng kênh định thời đồng bộ; môi trường bài thực hành chạy trên Go 1.27.1 không phụ thuộc vào cờ trên để

suy đoán hành vi cũ. Đây là điều kiện phiên bản cần được ghi nhận cụ thể khi đo đạc. Việc bộ định thời được giải phóng và việc goroutine đọc bộ định thời kết thúc là hai bài toán hoàn toàn tách biệt; chừng nào worker còn chạy và duy trì tham chiếu tới bộ định thời, tiền đề thu hồi bộ nhớ vẫn chưa được thỏa mãn.

Một worker chạy ngầm có thể là tác vụ hợp lệ duy trì suốt vòng đời dịch vụ. Tác vụ đó chỉ trở thành rò rỉ goroutine (**goroutine leak**) khi bên sở hữu không còn nhu cầu xử lý nhưng không thiết lập cơ chế kết thúc, bất kể worker đang bị nghẽn chặn hay thỉnh thoảng vẫn thức giấc. Khái niệm rò rỉ tài nguyên (**resource leak**) có phạm vi rộng hơn: tệp tin, kết nối mạng hoặc thao tác đăng ký hàm phản hồi không được giải phóng đúng quy trình có thể làm cạn kiệt tài nguyên hệ thống dù dung lượng vùng nhớ ảo (**heap**) không tăng đáng kể. Vì vậy, giao ước dành cho nhóm worker cần được bảo toàn: bên nào nhận việc, bên nào phát tín hiệu hủy, bên nào đóng kênh và bên nào xác nhận toàn bộ tác vụ con đã chấm dứt.

Mô hình Điều phối G/M/P và Giới hạn Thực tế của Concurrency

Khi vết thực thi (**trace**) cho thấy worker chờ được lập lịch hoặc dung lượng bộ nhớ tăng tỷ lệ thuận theo số lượng goroutine, mô hình điều phối của môi trường thực thi (**runtime**) sẽ giúp người kỹ sư đặt đúng câu hỏi chẩn đoán. Mô hình này bổ sung cho giao ước của nhóm worker, không phải điều kiện tiên quyết để người học viết đúng cú pháp kênh.

Trong môi trường thực thi Go 1.27.1, tệp `src/runtime/proc.go` mô tả mô hình điều phối M:N thông qua ba thực thể `G`, `M` và `P`: đại diện lần lượt cho goroutine, luồng hệ điều hành và tài nguyên điều phối gắn với `GOMAXPROCS`. Thực thể `P` sở hữu một hàng đợi thực thi cục bộ; nó không đồng nhất với một lõi xử lý vật lý dành riêng. Đây là mô hình phục vụ việc phân tích vết thực thi hoặc đọc mã nguồn gốc, không phải giao ước ngữ nghĩa của ngôn ngữ. Tệp `runtime/stack.go` thiết lập hằng số `stackMin = 2048` byte cho phần ngăn xếp của Go, nhưng kích thước cấp phát thực tế còn cộng thêm phần phụ thuộc hệ điều hành rồi làm tròn; không thể suy đoán rằng mỗi

goroutine trên môi trường Windows của bài thực hành chỉ tiêu tốn 2 KiB tổng bộ nhớ hay luôn bắt đầu với dung lượng cấp phát đó.

Goroutine chia sẻ tài nguyên điều phối nhưng vẫn giữ ngăn xếp, siêu dữ liệu và có thể giữ đối tượng sống; việc tạo thêm goroutine còn làm tăng gánh nặng lập lịch và gốc quét thu gom rác (GC root). Giới hạn xử lý đồng thời phải xuất phát từ tài nguyên thực tế của khối lượng công việc (kết nối mạng, bộ mô tả tệp, dung lượng RAM, tải CPU hoặc năng lực của hệ thống hạ nguồn), chứ không xuất phát từ một con số kích thước ngăn xếp hay số lượng goroutine truyền miệng.

Phiên bản môi trường thực thi này cài đặt kênh thông qua cấu trúc `hchan`, bao gồm khóa bảo vệ và hàng đợi chờ nội bộ; đó cũng không phải một giao diện lập trình hay cam kết về chi phí vận hành của ngôn ngữ. Dưới áp lực tranh chấp tài nguyên (**contention**), với các mức dung lượng bộ đệm khác nhau, hoặc khi goroutine phải chuyển sang trạng thái ngủ rồi được đánh thức, chi phí vận hành kênh có thể trở nên đáng kể. Tuy nhiên, không tồn tại quy tắc phổ quát khẳng định `Mutex`, thao tác nguyên tử `atomic` hay kênh luôn có tốc độ cao hơn: chúng mô tả các giao ước đồng bộ hóa khác nhau. Hãy sử dụng kênh khi cần bàn giao quyền sở hữu công việc hoặc phối hợp vòng đời; nếu một đường dẫn thực thi trọng yếu (**hot path**) phát sinh vấn đề hiệu năng, hãy đo kiểm chuẩn (**benchmark**) trên khối lượng công việc thực tế và kiểm tra xung đột dữ liệu (**race condition**) trước khi thay đổi nguyên hàm đồng bộ hóa (**synchronization primitive**).

Chương 10 — Khi chương trình chậm hoặc phình

Một chương trình bị phán xét là "chậm" hoặc "tốn bộ nhớ" thường kéo theo những nỗ lực tối ưu hóa mù quáng: thay thế gói `fmt` bằng phép cộng chuỗi thủ công, mở thêm hàng loạt goroutine không kiểm soát, hoặc viết lại thuật toán theo những mẹo vặt phức tạp. Những can thiệp này thường làm mã nguồn suy giảm độ trong sáng nghiêm trọng mà không giải quyết được nguyên nhân gốc rễ khiến người dùng hoặc hệ thống phải chờ đợi.

Mô hình tư duy nền tảng của chương này xác lập: tối ưu hóa là việc kiểm chứng một giả thuyết khoa học dưới một khối lượng công việc (workload) mang tính đại diện; một phép đo chỉ trả lời câu hỏi mà nó được thiết kế để đo lường. Benchmark không bảo đảm một dịch vụ mạng sẽ phản hồi nhanh trong mọi tình huống thực tế. Bản ghi CPU profile không phản ánh việc bộ nhớ đang phình to. Dấu vết thực thi (Execution trace) không phải là bản chụp toàn cảnh ngữ nghĩa của mã nguồn. Mỗi công cụ là một dụng cụ đo lường chuyên biệt nhằm thu hẹp không gian suy đoán.

Giả sử `opsprobe` phải dựng báo cáo cho hàng nghìn endpoint và thao tác này chậm trên CI. Câu hỏi đo được là: với cùng đầu vào, `Render` mất bao nhiêu thời gian trôi qua và cấp phát bao nhiêu; CPU profile tập trung ở đâu; bản thay thế có giữ từng byte output không? Thời gian benchmark và mẫu CPU trả lời hai câu hỏi khác nhau, không được gọi `ns/op` là thời gian CPU.

Bản chất Chuỗi Tối ưu hóa: Từ Source đến Mã máy

Để đặt câu hỏi về hiệu năng, tách quyết định lúc compile khỏi công việc lúc chạy:

Tầng	Cơ chế cần điều tra	Bằng chứng phù hợp
Compiler của toolchain đã pin	Inlining, escape analysis, SSA và loại bỏ kiểm tra biên có thể đổi cách thực hiện cùng semantics.	Diagnostic compiler; assembly của cùng source và cờ build.
Mã máy của artifact	Instruction, thanh ghi và lời gọi còn lại sau build.	Objdump của đúng binary, không trộn với listing từ build khác.
Runtime khi workload chạy	Allocation, GC, lập lịch và I/O tiêu tốn tài nguyên theo đường thực thi.	Benchmark, profile và trace, kèm môi trường và giới hạn phép đo.

Các tầng này giúp đặt câu hỏi, không phải một pipeline cố định của ngôn ngữ. Inlining là đưa thân hàm vào nơi gọi, không phải "nội suy hàm"; nó có thể làm thay đổi thông tin mà escape analysis nhận được. Compiler có thể dùng thanh ghi, stack, heap hoặc loại bỏ giá trị. Diagnostic cho biết quyết định lúc biên dịch; phép đo cho biết chi phí lúc chạy trên đúng artifact và workload, không thay thế lẫn nhau.

Phân tích Thoát Bộ nhớ: Ngăn xếp đối đầu Vùng nhớ Động

Escape analysis theo dõi dòng con trỏ và vòng đời vùng nhớ để tránh đặt dữ liệu trên stack khi tham chiếu tới nó có thể sống lâu hơn vùng lưu trữ ấy. Phạm vi tên biến trong khối lệnh không phải vòng đời của dữ liệu: một biến hết scope chưa có nghĩa vùng nhớ phải được chuyển lên heap; ngược lại, con trỏ tới dữ liệu trong frame đã kết thúc không được tồn tại nếu frame ấy bị thu hồi. Kích thước hoặc các giới hạn cấp phát trên stack cũng có thể khiến compiler chọn heap. Đây là quyết định của toolchain, không phải quy luật cú pháp. Mã nguồn `cmd/compile/internal/escape` của bản pin giải thích các ràng buộc này.

Một giá trị `does not escape` có thể nằm trên stack, trong thanh ghi hoặc bị loại bỏ; kết luận này không nói toàn hàm không cấp phát. Trả con trỏ, lưu

vào biến toàn cục hay đóng gói vào interface là những đường mà compiler phải phân tích, không phải luật "cứ có pointer là heap". Kết quả phụ thuộc thân hàm, inlining, kiến trúc và phiên bản. Diagnostic mô tả quyết định build; `allocs/op` mô tả cấp phát bình quân trên workload đã chạy.

Chạy `go build -gcflags=all=-m=2 ./fixed` từ `labs/part10-measure-first` với Go 1.27.1. Trích đoạn dưới rút gọn prefix đường dẫn `fixed/`, lược dòng không liên quan và ngắt dòng cho vừa khung; không phải toàn bộ diagnostic:

```
./render.go:13:6: cannot inline Render:
    function too complex: cost 231 exceeds budget 80
./render.go:16:18: inlining call to
    strings.(*Builder).WriteString
./render.go:18:36: inlining call to strconv.FormatInt
./render.go:13:13: readings does not escape
./render.go:16:18: append(strings.b.buf, strings.s...)
    escapes to heap in Render:
  flow: {heap} ← &{storage for append(...)}:
    from append(strings.b.buf, strings.s...) (spill)
    from strings.b.buf = append(...) (assign)
```

Kết quả chẩn đoán này ghi nhận hai quan sát thực nghiệm đối với mã nguồn và chuỗi công cụ của thí nghiệm:

Thứ nhất, tham số `readings []Reading` được xác nhận không thoát (`does not escape`). Bản thân lát cắt không tự quyết định nơi lưu trữ dữ liệu hay cấu trúc mô tả; kết quả này chỉ khẳng định trình biên dịch trong lượt phân tích này nhận thấy không cần đưa giá trị đó ra vùng nhớ heap dựa trên cách thức sử dụng bên trong hàm.

Thứ hai, vùng lưu trữ được tạo bởi lệnh `append` bên trong `strings.Builder` được ghi nhận thoát ra vùng nhớ heap. Đó là dữ liệu thực nghiệm hữu ích giúp giải thích chỉ số `allocs/op`, không phải quy tắc khẳng định mọi lệnh `append` hay mọi thao tác của `Builder` đều bắt buộc phải cấp phát trên heap.

Tối ưu hóa SSA và Loại bỏ Kiểm tra Biên (Bounds Check Elimination)

Một bước tối ưu hóa quan trọng của tầng sinh mã trung gian SSA (Static Single Assignment) là loại bỏ việc kiểm tra biên truy cập (**bounds check elimination**) khi trình biên dịch chứng minh toán học được chỉ số mảng luôn nằm trong phạm vi hợp lệ. Theo ngữ nghĩa chuẩn của Go, truy cập ngoài biên bắt buộc phải kích hoạt `panic`; lệnh mã máy cụ thể dùng để bảo đảm điều kiện an toàn đó phụ thuộc vào trình biên dịch và kiến trúc vi xử lý. Dữ liệu đầu ra của `ssa/prove` dưới đây là quan sát thực nghiệm trên Go 1.27.1, không đại diện cho cấu trúc mã máy mà mọi bản dựng đều bắt buộc phải tạo ra.

Thực thi cùng lệnh biên dịch với cờ `-gcflags=all=-d=ssa/prove/debug=1`. Cờ `all` sẽ phát sinh thông tin chẩn đoán cho toàn bộ thư viện chuẩn; tiền tố đường dẫn `GOROOT` được rút gọn trong trích đoạn dưới đây nhằm tránh nhầm lẫn các dòng xuất xứ từ `strings.go` với kết quả của thân hàm `Render`:

```
./render.go:15:20: Induction variable: limits [0,?), increment 1
./render.go:15:20: Inverted loop iteration
./render.go:21:19: Disproved Less64
strings/strings.go:987:27: Proved IsInBounds
strings/strings.go:988:38: Proved IsSliceInBounds
```

Các dòng tại `render.go` nhận diện biến quy nạp và việc chuyển bộ đếm nội bộ sang đếm giảm. `downward_counting_loop.go` của Go 1.27.1 chỉ thực hiện bước này khi thân vòng lặp không phụ thuộc biến đếm đó; nó không đảo thứ tự các Reading trong output. Các dòng `strings.go` chứng minh điều kiện biên tại chính vị trí thư viện được nêu. Muốn kết luận một bounds check cụ thể của `Render` đã biến mất, phải đối chiếu vị trí source với diagnostic và assembly của cùng artifact; sự xuất hiện của `ssa/prove` không đủ. Giữ bất biến rõ ràng rồi đo, đừng làm source khó đọc để săn một thông báo compiler.

Cơ chế Thu gom Rác và Đánh đổi Không gian - Thời gian

Bộ thu gom rác trong môi trường thực thi chuẩn của Go 1.27.1 vận hành theo cơ chế đánh dấu - dọn dẹp đồng thời (**concurrent mark-sweep**), kết hợp hàng rào ghi (**write barrier**) và các khoảng dừng toàn hệ thống (**stop-the-world**) được mô tả chi tiết tại `runtime/mgc.go`. Đây là giải pháp kỹ thuật nội bộ của bản cài đặt, không phải đặc tả ngôn ngữ bắt buộc mọi môi trường thực thi của Go phải áp dụng. Tài liệu hướng dẫn GC (Go GC Guide) phác thảo mô hình đánh đổi cơ bản giữa tài nguyên CPU và bộ nhớ; để giải thích chính xác các chi tiết trong phiên bản này, kỹ sư cần đối chiếu thêm mã nguồn gốc và tài liệu phát hành, tuyệt đối không sử dụng tài liệu hướng dẫn như một thước đo thay thế cho bài kiểm thử dịch vụ thực tế.

Tại tệp `runtime/mgcpacer.go` của Go 1.27.1, hằng số `gcBackgroundUtilization = 0.25` thiết lập mục tiêu dành khoảng 25% năng lực của `GOMAXPROCS` cho các tác vụ đánh dấu chạy ngầm trong pha đánh dấu. Điều này không đồng nghĩa bộ thu gom rác luôn chiếm dụng đúng 25% tổng năng lực CPU của toàn bộ tiến trình hay chiếm giữ riêng biệt một bộ xử lý ảo P: các worker chuyên trách, worker bán thời gian và worker nhàn rỗi sẽ cùng tham gia điều phối, trong khi cơ chế hỗ trợ trực tiếp (**mark assist**) phát sinh chi phí riêng. Muốn xác định chính xác khối lượng công việc thực tế tiêu hao bao nhiêu phần trăm CPU cho việc dọn rác, kỹ sư phải trích xuất các chỉ số và hồ sơ hiệu năng của chính lượt chạy đó.

Môi trường thực thi phát sinh các điểm dừng tạm thời tại thời điểm kết thúc dọn dẹp (**sweep termination**) và kết thúc đánh dấu (**mark termination**); khoảng thời gian dừng thực tế cần được đo đạc từ môi trường vận hành thực, tránh ngộ nhận mục tiêu thiết kế lý thuyết thành cam kết bảo đảm độ trễ. Trong pha đánh dấu, mỗi thao tác cấp phát bộ nhớ có thể tạo ra một khoản nợ công việc đánh dấu theo tỷ lệ do bộ điều phối nhịp độ (**pacer**) tính toán. Khi một goroutine phát sinh nợ, luồng hỗ trợ đánh dấu (**mark assist**) có thể buộc chính goroutine đó phải tham gia thực hiện công việc dọn dẹp hoặc chờ cấp tín chỉ. Đây là hướng điều tra quan trọng khi nhận thấy tốc độ cấp phát và độ trễ tăng đồng thời, chứ không đơn thuần là phép so sánh trực giác giữa tốc độ cấp phát và tốc độ đánh dấu hay kết luận võ đoán rằng độ trễ

đuôi (**tail latency**) chắc chắn sẽ tăng. Kỹ sư cần đối chiếu dữ liệu vết thực thi, các chỉ số từ `runtime/metrics` và luồng hỗ trợ đánh dấu trong mã nguồn thực tế.

Green Tea GC là bối cảnh triển khai, không phải contract

GC bắt đầu từ các gốc như stack goroutine và biến toàn cục rồi lần theo con trỏ tới đối tượng còn sống. Write barrier phối hợp việc cập nhật con trỏ với pha đánh dấu để GC không bỏ sót đối tượng cần giữ; nó không thay mutex hoặc công bố dữ liệu an toàn cho goroutine khác. Tăng goroutine hay cấu trúc nhiều con trỏ có thể tăng công việc quét, nhưng phải đo mới biết chi phí đáng kể ở workload nào. Thu hồi đối tượng không đồng nghĩa trả ngay từng byte về hệ điều hành.

Phiên bản Go 1.27.1 tích hợp cơ chế Green Tea GC trong môi trường thực thi. Ý tưởng trọng tâm của thiết kế này là ưu tiên quản lý theo từng trang bộ nhớ hoặc khối liên tục (**page/span**) thay vì coi từng đối tượng đơn lẻ là đơn vị xử lý duy nhất, nhằm tối ưu hóa tính cục bộ truy cập bộ nhớ (**cache locality**). Đây là chi tiết hiện thực nội bộ có thể tiếp tục được tinh chỉnh giữa các phiên bản Go và không phải là thước đo bảo đảm hiệu năng cho một dịch vụ cụ thể. Khi chi phí thu gom rác xuất hiện trong hồ sơ hiệu năng, câu hỏi vận hành cốt lõi vẫn là: tốc độ cấp phát, dung lượng heap đang hoạt động, tập gốc tham chiếu (**root set**), mật độ con trỏ và độ trễ thực tế của khối lượng công việc đang ở mức nào? Hãy đo đạc thực nghiệm trước, rồi mới phân tích xem việc nâng cấp phiên bản Go, cắt giảm thao tác cấp phát hay tái cấu trúc dữ liệu có thực sự đem lại lợi ích.

Hai biến số môi trường chi phối trực tiếp hành vi đánh đổi không gian và thời gian này:

Tham số `GOGC`: Thiết lập điểm cân bằng trong bài toán đánh đổi giữa chi phí CPU và dung lượng bộ nhớ. Trong các phiên bản Go hiện đại, ngưỡng kích hoạt heap mục tiêu còn tính toán cả các gốc tham chiếu: ước lượng gần đúng theo công thức `live heap + (live heap + roots) * GOGC / 100`. Giá trị `GOGC=100` không đồng nghĩa tổng dung lượng heap luôn bằng chính xác hai lần lượng heap đang hoạt động. Việc tăng chỉ số này thường làm giảm tần suất kích hoạt chu kỳ thu gom rác nhưng lại gia tăng phụ phí tiêu hao bộ nhớ; đây là xu hướng định tính cần được kiểm chứng cụ thể trên khối lượng công việc thực tế.

`GOMEMLIMIT`, từ Go 1.19, là giới hạn mềm cho đại lượng bộ nhớ runtime quản lý: `MemStats.Sys - MemStats.HeapReleased`, không chỉ live heap và không phải RSS hay trần cgroup. GC điều chỉnh mục tiêu và nhịp chạy để cố giữ giới hạn, nhưng có thể vượt nó nhằm tránh dành quá nhiều CPU cho GC. Không có cam kết chỉ vượt "tạm thời": nếu dữ liệu cần giữ đã lớn hơn ngân sách, GC không thể thu hồi nó. Cần đo thêm bộ nhớ ngoài runtime, chừa khoảng dự phòng và sửa budget/workload; setting này không bảo đảm tránh OOM.

Đo lường Hiệu năng: Benchmark là Hợp đồng Thực nghiệm

Giao ước đo chuẩn (**benchmark contract**) quyết định công việc nào phải được tính giờ. Nếu bài toán chỉ đo riêng hàm `Render`, hãy khởi tạo danh sách `readings` bên ngoài vòng lặp đo; nếu bài toán đo toàn bộ chuỗi đường ống tiếp nhận và xử lý dữ liệu đầu vào, chi phí chuẩn bị tương ứng có thể chính là thành phần cần tính toán. Tránh lược bỏ phần thiết lập theo quán tính máy móc mà vô tình bỏ qua khối lượng công việc thực tế mà hệ thống phải gánh chịu.

Phương thức `b.Loop` xuất hiện từ Go 1.24 và là cú pháp quy chuẩn cho các bài đo chuẩn mới trong phiên bản này; vòng lặp `b.N` truyền thống vẫn tiếp tục được hỗ trợ. Lần gọi đầu tiên sẽ tái lập bộ đếm thời gian (**reset timer**), lần

trả về `false` sẽ dừng bộ đếm, nhờ đó các bước thiết lập đặt trước vòng lặp không bị tính vào kết quả đo:

```
func BenchmarkRender(b *testing.B) {
    readings := representativeReadings(1_000)

    for b.Loop() {
        Render(readings)
    }
}
```

Lệnh thực thi đo đạc thống kê nhiều lần:

```
go test -bench BenchmarkRender -benchmem -count=6 ./fixed
```

`ns/op` là thời gian trôi qua mà timer benchmark ghi nhận chia cho số thao tác, không phải riêng thời gian CPU. `B/op` và `allocs/op` là các giá trị cấp phát bình quân runner ghi nhận, không phải live heap hay RSS; một số bình quân làm tròn không chứng minh mọi đường chạy đều không cấp phát. Sáu lượt giúp nhìn độ biến thiên, nhưng không loại bỏ tác động của warmup, tần số CPU hay công việc nền. Tách phép đo có profiling khỏi A/B thời gian để overhead của profiler không nghiêng về một phía.

Công cụ chẩn đoán	Câu hỏi kỹ thuật được giải đáp	Giới hạn phân tích
Kiểm thử đơn vị (Unit test)	Đầu ra và mã lỗi có bảo toàn đúng cam kết logic?	Không chứng minh được mã chạy nhanh hơn hay tốn ít RAM hơn.
Đo lường chuẩn (Benchmark)	Thao tác này tiêu tốn thời gian và phân bổ bộ nhớ ra sao dưới input chuẩn?	Không phản ánh trực tiếp độ trễ mạng hay tải tương tranh thực tế.
Phân tích CPU (CPU profile)	Sample CPU tập trung ở stack/hàm nào?	Không đo trực tiếp thời gian chờ khi goroutine đã block; CPU dùng cho syscall, spin hay đường khóa vẫn có thể xuất hiện.
Phân tích Bộ nhớ (Heap profile)	Đường dẫn mã nguồn nào chịu trách nhiệm cho các khối cấp phát trên heap?	Dữ liệu mang tính lấy mẫu thống kê, không ghi nhận từng biến riêng lẻ.

Công cụ chẩn đoán	Câu hỏi kỹ thuật được giải đáp	Giới hạn phân tích
Dấu vết thực thi (Execution trace)	Sự kiện runtime và chuyển trạng thái goroutine nào xảy ra trong khoảng đã ghi?	Kích thước và overhead tùy workload, thời gian ghi và version; cần ngân sách, không coi trace là diễn giải mọi semantics của source.

Phân tích CPU và Bộ nhớ (pprof)

Khi benchmark phát hiện điểm nghẽn hiệu năng, công cụ trích xuất profile (`pprof`) giúp định vị chính xác vị trí tích lũy chi phí trong cây gọi hàm:

```
go test -run '^$' -bench BenchmarkRender `
  -cpuprofile cpu.out -memprofile mem.out ./fixed
go tool pprof -top cpu.out
```

Chỉ số `flat` ghi nhận chi phí tiêu tốn trực tiếp bên trong thân hàm, trong khi chỉ số `cum` (cumulative) ghi nhận tổng chi phí tích lũy của hàm đó cùng toàn bộ các hàm con mà nó triệu gọi. Một hàm có `flat` nhỏ nhưng `cum` lớn là dấu hiệu cho thấy nó đang dẫn lối vào một chuỗi thao tác tiêu tốn nhiều tài nguyên.

Nếu nghi ngờ goroutine bị giữ lại ngoài dự kiến, Go 1.27 bổ sung hồ sơ hiệu năng `goroutineleak` trong gói `runtime/pprof` cùng điểm cuối tương ứng của `net/http/pprof`, không còn cần cờ thử nghiệm như ở Go 1.26. Cơ chế này rà soát nhóm trạng thái chờ mà môi trường thực thi xác định không còn khả năng được đánh thức; nó không thể phát hiện mọi dạng rò rỉ và không tự động thu hồi goroutine. Một hồ sơ hiệu năng không ghi nhận điểm nghẽn vẫn đòi hỏi kỹ sư phải đối chiếu bản trích xuất trạng thái goroutine (**goroutine dump**), quyền sở hữu kênh và đường dẫn hủy bỏ tác vụ như đã thảo luận ở Chương 09. Khi mở giao diện quan sát vết thực thi, lệnh `go tool trace -http=:6060` từ Go 1.27 mặc định chỉ lắng nghe trên giao diện nội bộ `localhost`; nếu muốn kết nối từ xa, kỹ sư phải chỉ định địa chỉ tường minh. Tránh mở cổng công khai cho các hồ sơ hay vết thực thi chứa dữ liệu nhạy cảm chỉ nhằm mục đích thuận tiện.

Ca điều tra: tăng bộ nhớ nhưng chưa biết ai đang giữ

Giả sử một tiến trình tiếp nhận khối dữ liệu tải trọng (**payload**) 4 MiB cho mỗi phần tử. Sau mỗi chu kỳ xử lý hai phần tử, chỉ số `TotalAlloc` tăng lên rõ rệt; người trực ca vận hành có xu hướng muốn gọi cưỡng bức bộ thu gom rác hoặc vội vàng kết luận rằng hệ thống bị rò rỉ bộ nhớ. Kỹ sư cần một câu hỏi chẩn đoán chặt chẽ hơn: các lượt cấp phát bộ nhớ đó chỉ đơn thuần đã diễn ra trong quá khứ, hay các đối tượng vẫn đang tồn tại sau khi công việc đã hoàn tất, và thành phần nào đang duy trì tham chiếu tới chúng? Bài thực hành độc lập `labs/part10-resource-retention` chủ ý cô lập đường dẫn lưu giữ dữ liệu khỏi giao thức HTTP, cơ sở dữ liệu và công cụ kiểm thử nhằm bảo đảm câu trả lời không bị nhiễu bởi nhóm kết nối (**connection pool**) hay bộ nhớ đệm (**cache**) của các thư viện.

Khởi chạy từng chế độ khảo sát trong một tiến trình riêng biệt, giữ nguyên cấu hình phần cứng, chuỗi công cụ biên dịch và các tham số đo đạc. Từ thư mục của bài thực hành, lệnh sau thực thi trên PowerShell; bổ sung cờ `-fixed` rồi thay đổi thư mục đích để ghi nhận dữ liệu bản khắc phục, tránh ghi đè lên hồ sơ hiệu năng của bản lỗi ban đầu:

```
go build -o artifacts/investigate.exe ./cmd/investigate
./artifacts/investigate.exe -mode=slice `
  -cycles=4 -items=2 -bytes=4194304 -limit=2 `
  -out=artifacts/slice-baseline
go tool pprof -inuse_space -top `
  artifacts/slice-baseline/04.heap.pprof
go tool pprof -alloc_space -top `
  artifacts/slice-baseline/04.heap.pprof
```

Thay đổi chế độ sang `cache`, `churn`, `goroutine` hoặc `ticker` nhằm duy trì cùng nhịp độ công việc nhưng kiểm chứng các cơ chế hoạt động khác nhau. Chế độ `churn` cấp phát rồi lập tức giải phóng tải trọng mà không lưu giữ kết quả; cờ `-fixed` không làm thay đổi luồng thực thi của chế độ đối chứng này. Các chế độ `goroutine` và `ticker` không cấp phát tải trọng theo cờ `-bytes`: chúng khởi tạo các worker để điều tra vòng đời quản lý tác vụ thay vì mô phỏng việc xử lý 4 MiB dữ liệu. Tài liệu hướng dẫn ghi nhận đầy

đủ giới hạn của khối lượng công việc cũng như cách thức chạy riêng biệt cho chế độ ngữ cảnh `context`.

Trước khi phân tích mã nguồn, hãy đối chiếu bảng số liệu thực nghiệm được đo đạc ngày 09/10/2026 trên môi trường `go1.27.1 windows/amd64`. Các dãy số thể hiện kết quả quan sát sau chu kỳ 1 đến 4; tại mỗi chu kỳ, môi trường bài thực hành chủ động kích hoạt hai lượt thu gom rác trước khi ghi nhận số liệu. Người kỹ sư cần xác định nên mở hồ sơ vùng nhớ heap hay bản trích xuất goroutine cho từng dòng dữ liệu, và giả thuyết nào hiện vẫn còn thiếu bằng chứng xác thực?

Bảng 9 — Các triệu chứng dẫn tới những phép đo khác nhau

Mode	Quan sát bản lỗi hoặc đối chứng	Quan sát bản sửa
Slice	Entry 2, 4, 6, 8; `HeapAlloc` 8.35, 16.35, 24.35, 32.35 MiB	Cùng số entry; `HeapAlloc` xấp xỉ 0.35 MiB
Cache	Entry 2, 4, 6, 8; `HeapAlloc` 8.35, 16.35, 24.35, 32.36 MiB	Entry luôn 2; `HeapAlloc` xấp xỉ 8.35–8.36 MiB
Churn	Entry 0; `HeapAlloc` xấp xỉ 0.35 MiB	Không có đường sửa riêng; đây là đối chứng
Receive	`NumGoroutine`: 3, 5, 7, 9 sau khi bên gọi hủy ngữ cảnh	Luôn 1, đã chờ `done` từng worker
Ticker	`NumGoroutine`: 3, 5, 7, 9 sau khi bên gọi hủy ngữ cảnh	Luôn 1, đã chờ `done` từng worker

Các con số làm tròn ở trên không phải là ngưỡng kiểm thử tuyệt đối và không đại diện cho ngân sách bộ nhớ phổ quát. Bài thực hành thiết lập `MemProfileRate = 1` nhằm theo dõi chi tiết từng thao tác cấp phát trong một khối lượng công việc quy mô nhỏ; bản thân việc ghi chép hồ sơ hiệu năng và trích xuất thông tin goroutine cũng tự tiêu tốn bộ nhớ. Do đó, chỉ số `TotalAlloc` bao gồm cả chi phí quan sát chứ không đồng nhất với tổng dung lượng tải trọng xử lý. Trong điều kiện vận hành thông thường, môi trường thực thi chỉ lấy mẫu thống kê; hồ sơ hiệu năng luôn có độ trễ nhất định khi ghi nhận việc cấp phát hoặc giải phóng ô nhớ như lưu ý trong tài liệu `runtime.MemProfile`. Việc kích hoạt hai lượt thu gom rác là lựa chọn có chủ đích để chuẩn hóa môi trường thí nghiệm, tuyệt đối không phải phương thức vận hành hệ thống thực tế nhằm che đậy hiện tượng rò rỉ.

Từ vị trí cấp phát tới đường giữ dữ liệu

Tại chế độ lát cắt (`slice`), chỉ số dung lượng đang sử dụng (`inuse_space`) sau chu kỳ 4 ghi nhận 32 MiB tại vị trí `main.payload`. Với cùng khối lượng công việc trên bản khắc phục, việc lọc ngăn xếp theo `main.payload|fixed.Prefix` cho thấy chỉ còn 128 byte tại `fixed.Prefix`. Tuy nhiên, chỉ số tổng dung lượng cấp phát tích lũy (`alloc_space`) vẫn ghi nhận 32 MiB tại `main.payload` trên cả hai phiên bản: cả hai đều đã tiếp nhận tám tải trọng, điểm khác biệt duy nhất nằm ở khối lượng dữ liệu bị giữ lại sau khi hoàn tất xử lý. Chế độ `churn` cũng ghi nhận 32 MiB đã cấp phát tại vị trí này nhưng không còn tải trọng nào tồn tại trong hồ sơ quan sát. Chỉ số `alloc_space` phản ánh tổng chi phí cấp phát lũy kế, không biểu thị lượng dữ liệu còn sống và không phải là công cụ phát hiện rò rỉ bộ nhớ.

Hồ sơ hiệu năng chỉ dẫn tới vị trí đối tượng được khởi tạo trên mã nguồn, chứ không tự động phác họa chuỗi tham chiếu đang lưu giữ đối tượng đó. Chỉ khi phân tích chi tiết hàm `Prefix`, kỹ sư mới xác định được đường dẫn từ danh sách `views` tới lát cắt tiền tố, rồi dẫn tới mảng nền kích thước lớn; biểu thức cắt lát ba chỉ số không hề thực hiện sao chép dữ liệu. Việc chủ động sao chép tại ranh giới chuyển giao sẽ loại bỏ hoàn toàn đường dẫn lưu giữ không mong muốn này. Đây chính là giải pháp khắc phục dựa trên bài kiểm thử chia sẻ mảng nền (**aliasing**) đã được thiết lập ở Chương 02, chứ không phải can thiệp vào bộ thu gom rác. Nếu một tham chiếu khác vẫn tiếp tục tồn tại, kết quả đo đạc sẽ thay đổi; do đó kỹ sư cần bảo toàn khối lượng công việc và quyền sở hữu trong bộ tạo mẫu cùng các bằng chứng thực nghiệm.

Tại chế độ bộ đệm (`cache`), hồ sơ hiệu năng của bản lỗi cũng định vị tới `main.payload`: 32 MiB vẫn sống sau bốn chu kỳ hoạt động. Bản khắc phục theo cơ chế FIFO chỉ còn giữ 8 MiB tải trọng tại `fixed.(*Cache).Put`, bởi bộ đệm chỉ sở hữu bản sao độc lập của hai mục nhập gần nhất. Trong chỉ số `alloc_space`, riêng hàm `Put` đã cấp phát 32 MiB phục vụ việc sao chép; bản khắc phục có thể phát sinh nhiều lượt cấp phát hơn nhưng lại duy trì ít ô nhớ sống hơn. Tuyệt đối không lựa chọn một phương án cài đặt chỉ vì quan

sát thấy chỉ số `B/op` nhỏ hơn. Việc đối chiếu các mục nhập trong bảng ánh xạ với giao ước thiết kế ở Chương 03 giúp phân định mức tăng trưởng nào là hợp thức: tám khóa dữ liệu trong kho lưu trữ có chủ đích và tám khóa trong bộ đệm cam kết giới hạn tối đa hai phần tử có thể tạo ra cùng một biểu đồ vùng nhớ, nhưng mang bản chất kỹ thuật hoàn toàn đối lập.

Goroutine không cần nhiều heap để làm lỗi vòng đời

Dãy số 3, 5, 7, 9 gợi mở giả thuyết rằng mỗi chu kỳ hoạt động đang lưu giữ lại hai worker, nhưng chưa đủ để khẳng định chúng đã bị rò rỉ. Mở tệp trích xuất trạng thái `04.goroutines.txt`: chế độ nhận ghi nhận tám ngăn xếp trong hàm `baseline.Wait.func1` đang ở trạng thái chờ kênh; chế độ định thời ghi nhận tám ngăn xếp trong `baseline.RunTicker.func1` đang bị phong tỏa tại mệnh đề `select`. Bước tiếp theo là phân tích mã nguồn và giao ước: bên sở hữu đã phát tín hiệu hủy, thao tác nhận không lắng nghe ngữ cảnh `context`, trong khi vòng lặp định thời chỉ chờ đợi nhịp đồng hồ hoặc kênh giải phóng dự phòng của bài thực hành. Hệ thống hoàn toàn thiếu đường dẫn hủy tác vụ nghiệp vụ để cho phép hàm kết thúc trở về. Số lượng worker gia tăng ổn định qua từng chu kỳ kết hợp cùng các điểm chờ vô hạn đó mới giúp kỹ sư kết nối chuẩn xác triệu chứng quan sát với nguyên nhân gốc rễ.

Bản khắc phục chủ động chờ tín hiệu từ kênh `done` sau khi phát lệnh hủy trước khi tiến hành đo đạc, bảo đảm các worker đã thực sự kết thúc thay vì chỉ phỏng đoán hy vọng bộ điều phối sẽ sớm bố trí tài nguyên. Khi kết thúc thí nghiệm, khung điều phối giải phóng toàn bộ worker của bản lỗi rồi đồng bộ hóa; số lượng goroutine trở về mức 1 trong lượt chạy thực tế. Đây là minh chứng rõ ràng cho việc kiểm soát trọn vẹn quy trình dọn dẹp tài nguyên, chứ không dựa dẫm vào việc môi trường thực thi tự động chấm dứt goroutine. Bài kiểm thử với hàm phản hồi không hợp tác vẫn đòi hỏi cơ chế giải phóng chuyên biệt như đã phân tích ở Chương 09: nếu dữ liệu trích xuất cho thấy tiến trình tắc nghẽn bên trong hàm phản hồi, việc chỉ sửa đổi mệnh đề `select` ở vòng lặp bên ngoài là hoàn toàn không đủ.

Chế độ ngữ cảnh làm sáng tỏ giới hạn của bộ đếm số lượng goroutine. Với 512 ngữ cảnh con được thiết lập thời hạn kết thúc (**deadline**) một giờ, phiên

bản bỏ quên thao tác gọi `cancel` vẫn chỉ ghi nhận duy nhất một goroutine trong suốt lượt đo. Tuy nhiên, hồ sơ số lượng đối tượng đang sử dụng (`inuse_objects`) lại lưu giữ 1024 đối tượng trực tiếp tại `context.WithDeadlineCause` ở chu kỳ 4; bản khắc phục không còn bất kỳ đối tượng trực tiếp nào tại vị trí đó trong hồ sơ đo đạc. Cần tránh suy diễn con số 1024 đồng nghĩa với 1024 ngữ cảnh: một lời gọi hàm có thể khởi tạo nhiều cấu trúc đối tượng nội bộ, đồng thời một số điểm cấp phát của bộ định thời hoặc ngữ cảnh cha vẫn có thể xuất hiện trong hồ sơ. Bài kiểm thử vòng đời và mã nguồn của gói `context` ở Chương 12 sẽ làm rõ nguyên lý ngữ cảnh cha duy trì tham chiếu tới ngữ cảnh con cho đến khi bị hủy bỏ hoặc thời hạn kết thúc chạm ngưỡng. Sau khi khung điều phối hủy ngữ cảnh cha, các khoản cấp phát trực tiếp tại vị trí này biến mất khỏi hồ sơ quan sát. Thiếu sót trong việc gọi `cancel` ở trường hợp này là hành vi lưu giữ thừa thãi ngoài vòng đời yêu cầu, chứ không đồng nghĩa với nhận định sai lầm rằng mọi ngữ cảnh bị lỗi đều tồn tại vĩnh viễn.

Heap đã giảm, vì sao bộ nhớ process chưa giảm tương ứng?

Kỹ sư cần phân định rạch ròi giữa số liệu đo đạc thực tế và các nhận định mang tính suy diễn. Chỉ số `HeapAlloc` phản ánh tổng số byte của các đối tượng trên heap mà môi trường thực thi tính toán đang hoạt động; `HeapInuse` biểu thị dung lượng byte trong các khối bộ nhớ (**span**) đang được sử dụng, có thể bao gồm cả những khoảng trống chưa chứa đối tượng. Chỉ số `HeapSys` là tổng bộ nhớ heap mà môi trường thực thi đã yêu cầu cấp phát từ hệ điều hành, trong khi `HeapReleased` ghi nhận dung lượng đã hoàn trả lại cho hệ thống. Các chỉ số này không đồng nhất với kích thước bộ nhớ thực tế (Resident Set Size - RSS), và bài thực hành không tiến hành đo lường RSS. Chỉ số RSS hay tập làm việc (**working set**) còn chịu ảnh hưởng sâu sắc từ hệ điều hành, dung lượng ngăn xếp, chi phí nội bộ của môi trường thực thi và các vùng nhớ bổ trợ khác; việc dung lượng đối tượng giảm không bảo đảm biểu đồ tiêu thụ bộ nhớ của tiến trình sẽ suy giảm ngay lập tức một lượng tương ứng.

Trong chế độ lát cắt bị lỗi, sau khi khung điều phối loại bỏ toàn bộ các góc nhìn (**views**), chỉ số `HeapAlloc` giảm mạnh từ 33,924,560 xuống 369,936

byte nhưng `HeapSys` vẫn giữ nguyên ở mức 41,680,896 byte trong suốt lượt chạy. Đây là bằng chứng thực nghiệm cụ thể nhắc nhở rằng không thể vội vàng kết luận bộ thu gom rác chưa hoạt động chỉ dựa trên phần dung lượng heap đã xin từ hệ điều hành. Đồng thời cũng không thể quy chụp toàn bộ phần chênh lệch đó là RSS: đó là một đại lượng hoàn toàn khác biệt chưa được đo đạc. Nếu nghi ngờ nhu cầu sử dụng bộ nhớ cao là hợp lệ, hãy kiểm tra quy mô dữ liệu đang phục vụ cùng hạn mức ngân sách; nếu nghi ngờ phát sinh nhiều cấp phát tạm thời hoặc chu kỳ thu gom rác chưa hoàn tất, hãy so sánh giữa các chu kỳ sau khi khối lượng công việc đã lắng xuống; nếu đối tượng vẫn tồn tại ngoài phạm vi cần thiết, hãy truy vết thành phần sở hữu hoặc các tham chiếu đang giữ đối tượng; nếu số lượng tệp tin mở hoặc kết nối cạn kiệt trong khi heap vẫn ổn định, hãy đo lường trực tiếp tài nguyên đó thay vì tiếp tục nhìn vào vùng nhớ heap.

Một kết luận kỹ thuật có giá trị bảo vệ khoa học đòi hỏi phải xâu chuỗi một cách logic: khối lượng công việc đại diện, vị trí cấp phát hoặc ngăn xếp chờ đợi, mã nguồn làm rõ đường dẫn duy trì tham chiếu, cùng bài kiểm thử giao ước và kết quả đo đạc lặp lại sau khi sửa đổi. Mỗi bằng chứng thực nghiệm chỉ trả lời cho một câu hỏi riêng biệt. Một bản chụp tức thời có giá trị cao, một lần chỉ số RSS không suy giảm hay một báo cáo kiểm tra tĩnh hoàn toàn sạch lỗi đều không thể thay thế cho toàn bộ chuỗi bằng chứng khoa học nói trên.

Đối chiếu G/M/P với Execution Trace: Nhìn vào nhịp đập của Runtime

Chương 09 giới thiệu G là goroutine, M là luồng hệ điều hành, P là tài nguyên để chạy Go gắn với `GOMAXPROCS`. Đọc `findRunnable` và `stealWork` trong `runtime/proc.go` của Go 1.27.1: việc sẵn sàng chạy có thể nằm trong hàng đợi cục bộ của P hoặc hàng đợi chung; một P thiếu việc có thể lấy việc từ P khác. Netpoll đưa công việc chờ mạng trở lại trạng thái có thể chạy. Preemption tạo cơ hội nhường thực thi, nhưng không cam kết một goroutine sẽ chạy trước hạn nghiệp vụ hay theo FIFO. Đây là implementation giúp giải thích trace, không phải phép chứng minh công bằng hoặc tính đúng đắn của chương trình. Chi tiết syscall được phân biệt ở dưới.

CPU profile lấy mẫu thời gian CPU, nên không trực tiếp đo thời gian goroutine đã bị chặn. Không vì thế mà `pprof` "bất lực": block/mutex profile có thể giúp điều tra một số khoảng chờ nếu được bật, còn trace đặt các chuyển trạng thái lên cùng dòng thời gian. CPU thấp chưa chứng minh mọi goroutine đều đang chờ hoặc xác định nguyên nhân dịch vụ chậm; phải chọn bằng chứng phù hợp.

Bản chất của Execution Trace là ghi lại một tập rộng các sự kiện runtime phát sinh trong quá trình thực thi: thời điểm goroutine chuyển đổi trạng thái, thời điểm P được gắn kết hay tách rời khỏi M, các pha thu gom rác GC, và hoạt động của các điểm dừng đồng bộ hay lời gọi hệ thống. Trace không phải là một bản ghi toàn tri ghi lại mọi hành vi phần cứng. Trong mô hình trace của Go (đối chiếu `internal/trace.GoState`), runtime quản lý các trạng thái như `GoUndetermined`, `GoNotExist`, `GoRunnable`, `GoRunning`, `GoWaiting`, và `GoSyscall`. Để phục vụ việc phân tích triệu chứng vận hành, ta thường tập trung quan sát một tập con các trạng thái chủ đạo:

Một là, `GoRunning`: Goroutine đang trực tiếp chiếm giữ một luồng M trên một bộ xử lý ảo P để thực thi mã máy.

Hai là, `GoRunnable`: Goroutine đã sẵn sàng chạy (vừa được tạo ra, vừa nhận được dữ liệu, hoặc vừa được đánh thức) nhưng đang nằm trong hàng đợi chờ P tiếp nhận. Thời gian `Runnable` cao (scheduler latency) là tín hiệu chỉ báo để điều tra áp lực lập lịch hoặc tranh chấp tài nguyên khi lượng công việc sẵn sàng lớn hơn số bộ xử lý ảo (`GOMAXPROCS`) hiện có, chứ không tự nó kết luận nguyên nhân gốc rễ.

Ba là, `GoWaiting`: Goroutine chủ động nhường quyền thực thi và chờ đợi một điều kiện đồng bộ hóa: chờ kênh truyền, chờ khóa `sync.Mutex`, chờ `sync.WaitGroup`, hoặc chờ sự kiện mạng từ bộ thăm dò mạng (**network poller**). Khi chờ kênh hay khóa mutex, runtime chuyển trạng thái goroutine sang chờ trong bộ điều phối người dùng mà không nhất thiết phải kéo theo việc chuyển ngữ cảnh luồng hệ điều hành ngay lập tức.

Bốn là, `GoSyscall`: runtime đã chuyển G sang trạng thái gọi hệ thống; trạng thái này không tự chứng minh syscall đã chặn lâu. `entersyscall` cho phép P được thu hồi để M khác chạy Go, còn đường `entersyscallblock` dành cho lời gọi biết sẽ chặn có thể bàn giao P ngay. Với I/O mạng do netpoll hỗ trợ, G thường chờ theo mô hình `GoWaiting` mà không giữ riêng một M bị chặn cho cả thời gian chờ. Không phải mọi syscall hay mọi thiết bị đều đi qua netpoll. Mark assist là một hướng điều tra khác khi cấp phát tạo nợ đánh dấu; phải đọc sự kiện và stack thực tế thay vì suy nguyên nhân từ một tên trạng thái.

Để phân tích định lượng các điểm nghẽn mà không suy diễn cảm tính từ một trạng thái đơn lẻ, công cụ `go tool trace` hỗ trợ trích xuất bốn hồ sơ pprof chuyên biệt từ dữ liệu trace: `sync` (độ trễ do rào cản đồng bộ như mutex, waitgroup, kênh truyền), `sched` (độ trễ điều phối từ lúc goroutine sẵn sàng đến lúc được chạy), `syscall` (độ trễ chặn tại các lời gọi hệ thống), và `net` (độ trễ chờ I/O mạng từ bộ thăm dò mạng).

Trong `labs/part10-measure-first`, ta có thể thu thập trace thực tế khi chạy kiểm thử xử lý theo lô đồng thời (`TestRenderConcurrentBatchWorkload`):

```
go test -run TestRenderConcurrentBatchWorkload `
  -trace trace.out ./fixed
go tool trace trace.out
```

Giao diện đồ họa trên trình duyệt (mở qua lệnh `go tool trace trace.out`, mặc định chỉ lắng nghe trên localhost) hiển thị các dòng thời gian của từng bộ xử lý ảo P, cho thấy thời điểm các worker goroutine chạy song song trên các bộ xử lý và các khoảng dừng của hệ thống.

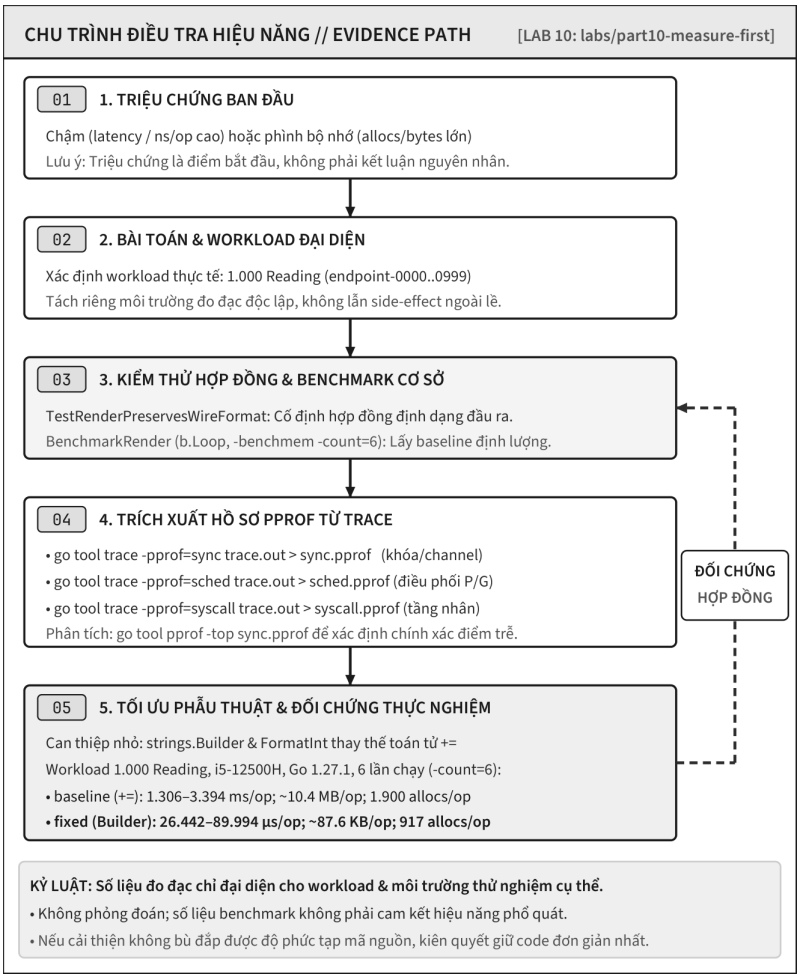
Để trích xuất các hồ sơ định lượng phục vụ phân tích, ta sử dụng cờ `-pprof` của `go tool trace`:

```
go tool trace -pprof=sync trace.out > sync.pprof
go tool trace -pprof=sched trace.out > sched.pprof
go tool trace -pprof=syscall trace.out > syscall.pprof
go tool pprof -top sync.pprof
```

Các lệnh > ghi profile nhị phân ở trên yêu cầu PowerShell 7.4 trở lên hoặc shell chuyển hướng byte nguyên vẹn. Windows PowerShell cũ có thể chuyển output native thành text và làm hỏng profile; không dùng Out-File cho dữ liệu này. Khung kiểm chứng của lab ghi trực tiếp stdout bytes bằng subprocess.

Lượt trace ngày 11/10/2026 trên Go 1.27.1 `windows/amd64` ghi nhận `sync.(*WaitGroup).Wait` và `runtime.chanrecv1` trong hồ sơ `sync`. Hồ sơ `sched` có stack đánh thức bên chờ qua `WaitGroup.Add/Done`, ngoài chi phí khởi động trace và khung test; không được gán toàn bộ cho worker mới tạo. Hồ sơ `syscall` có đường `syscall.WriteFile` từ bộ ghi trace và output test. Hồ sơ `net` không có sample vì fixture không chạy workload mạng. Đây là các khoảng chờ quan sát được của chính harness, không phải bằng chứng Render có lỗi CPU, I/O hoặc lập lịch trong production.

Dấu vết thực thi không phải là bằng chứng xác nhận ngữ nghĩa đúng đắn của logic nguồn hay cam kết về độ trễ trên môi trường sản xuất. Trace là công cụ phân loại triệu chứng: khi gặp vấn đề về hiệu năng, các hồ sơ pprof trích xuất từ trace giúp kỹ sư định hướng bước can thiệp tiếp theo dựa trên dữ liệu đo đạc thực nghiệm thay vì phỏng đoán.



Hình 22 — Từ triệu chứng đến giả thuyết có thể bác bỏ: chọn benchmark, profile hoặc trace theo câu hỏi, thay đổi nhỏ rồi đo lại A/B cùng workload. Kiểm tra đầu ra là một gate riêng, không phải chứng minh mọi hành vi đều đúng.

Điển cứu thực tế: Phân tích bài toán nối chuỗi

Thư mục `labs/part10-measure-first` duy trì hai cách hiện thực cho cùng một yêu cầu định dạng văn bản: phương án cơ sở (`baseline`) nối chuỗi bằng toán tử `+=` lặp lại; phương án tối ưu (`fixed`) sử dụng `strings.Builder` và `strconv.FormatInt`.

Khối lượng công việc gồm 1.000 phần tử `Reading`, từ `endpoint-0000` đến `endpoint-0999`, được khởi tạo hoàn toàn bên ngoài vòng lặp đo. Lượt kiểm chứng cục bộ ngày 01/10/2026 sử dụng Go 1.27.1, nền tảng `windows/amd64`, vi xử lý CPU 12th Gen Intel Core i5-12500H, công cụ đo ghi nhận hậu tố mặc định `-16`, thực thi sáu lần lặp lại trên cùng cấu hình máy, phiên bản Go, khối lượng công việc và giao ước đo lường. Khoảng giá trị dưới đây thể hiện mức tối thiểu và tối đa (min–max) của sáu kết quả `ns/op`, không phải khoảng tin cậy thống kê (**confidence interval**). Độ phân

tán số liệu cho thấy kỹ sư cần thận trọng khi suy luận tỷ lệ tăng tốc (**speedup**); đây không phải là phép đo độ trễ vận hành của toàn bộ dịch vụ:

Phương án hiện thực	Khoảng thời gian mỗi thao tác	Cấp phát do công cụ đo ghi nhận
`baseline` (Toán tử +=)	1.306–3.394 ms/op	10,443,876–10,443,996 B/op; 1,900–1,901 allocs/op
`fixed` (`strings.Builder`)	26.442–89.994 µs/op	87,600–87,608 B/op; 917 allocs/op

Trong mã nguồn của phương án cơ sở `baseline`, mỗi thao tác cộng chuỗi đều lưu giữ lại chuỗi kết quả dài hơn cho lượt lặp tiếp theo. Mã nguồn tầng thực thi của Go 1.27.1 tại `runtime/string.go`, kết hợp cùng số liệu cấp phát từ bài đo chuẩn, giúp làm rõ chi phí tái cấp phát liên tục khi dựng dữ liệu chuỗi; phương án `fixed` gom toàn bộ kết quả vào cấu trúc `strings.Builder`. Không thể vội vã suy đoán từ kết quả này rằng mọi phép cộng chuỗi + đều gây cấp phát bộ nhớ hay áp đặt một con số thông lượng cố định. Khi trích xuất hồ sơ CPU bằng lệnh nêu trên, hãy đối chiếu trực tiếp các mẫu thu thập của chính lượt chạy với đường dẫn nối hoặc sao chép chuỗi, tránh sử dụng lại tỷ lệ phần trăm từ một hồ sơ không còn tạo tác lưu trữ để kiểm chứng.

Kết quả kiểm thử đầu ra và bài đo chuẩn củng cố cho quyết định lựa chọn `strings.Builder` đối với khối lượng công việc này: định dạng văn bản được bảo toàn nguyên vẹn và số lượng cấp phát quan sát được giảm rõ rệt. Tuy nhiên, nó chưa phải bằng chứng xác nhận dịch vụ trên môi trường sản xuất sẽ phản hồi nhanh hơn, bởi bài đo chưa tính đến yếu tố mạng, tải tương tranh đồng thời cũng như trải nghiệm người dùng cuối.

Tối ưu hóa Dựa trên Hồ sơ Vận hành (PGO)

Từ Go 1.20 và hoàn thiện ở các phiên bản gần đây, Go hỗ trợ kỹ thuật Tối ưu hóa Dựa trên Hồ sơ (Profile-Guided Optimization - PGO).

Trong quy trình biên dịch thông thường, compiler chỉ có cái nhìn tĩnh về mã nguồn, không biết nhánh rẽ nào trong câu lệnh `if` hay phương thức nào của `interface` được gọi thường xuyên nhất trong thực tế. Với PGO, kỹ sư thu

thập một tệp CPU profile đại diện (`default.pgo`) từ hệ thống production đang chịu tải thực. Trình biên dịch nạp hồ sơ này vào pass phân tích để:

Một là, profile có thể giúp compiler ưu tiên inlining ở đường nóng. Ngưỡng và heuristic không phải API ổn định; cần diagnostic và benchmark của đúng bản build, không mặc định một hàm chắc chắn được inline.

Hai là, profile có thể mở ra cơ hội devirtualization khi compiler chứng minh được điều kiện phù hợp. Đó là tối ưu hóa có thể có, không phải lời hứa rằng mọi interface call nóng sẽ thành lời gọi tĩnh hay luôn inline xuyên package.

Tuy nhiên, kỹ thuật PGO chỉ thực sự đáng tin cậy khi hồ sơ hiệu năng phản ánh đúng khối lượng công việc đại diện của môi trường mục tiêu. Cần ưu tiên thu thập hồ sơ từ môi trường sản xuất; trong trường hợp bất khả thi, môi trường thử nghiệm (**staging**) hoặc các bài đo chuẩn có độ bao phủ đại diện cũng có thể được cân nhắc. Các bài đo vi mô (**microbenchmarks**) với phạm vi hẹp thường mô phỏng quá ít hành vi thực tế của chương trình để tạo ra một hồ sơ PGO chất lượng; hãy chỉ sử dụng chúng để kiểm chứng các giao ước nhỏ. Dù sử dụng nguồn dữ liệu nào, việc đo đạc lại kết quả kiểm chứng toàn trình (**end-to-end**) sau khi áp dụng tối ưu hóa vẫn là yêu cầu bắt buộc.

Thử A/B: profile có giúp chính workload này không?

Để kiểm tra PGO bằng số liệu thay vì chỉ mô tả khả năng của nó, trong `labs/part10-measure-first` hãy chạy phép thử bằng Python standard library và đúng Go đã pin:

```
python pgo_ab.py --go 'D:/Golang/.tools/go1.27.1/bin/go.exe' `
  --out artifacts/pgo-ab-new
```

Đổi đường dẫn Go theo máy; thư mục output phải chưa tồn tại để không ghi đè evidence cũ. Đây là A/B **cùng source** `fixed.Render` khi tắt và bật PGO, không phải so hai thuật toán `baseline` và `fixed`. Script thu CPU profile ba giây bằng `-pgo=off` trên 1.000 Reading đã dựng ngoài vòng đo, build hai test binary với `-pgo=off` và `-pgo=<profile>`, rồi kiểm tra output cả hai. Profile chỉ đại diện cho bài toán Render này, không cho toàn bộ service.

Lệnh compiler và SHA của source, profile, binary được lưu để xác định artifact thực sự đã dùng.

Sau một lượt warmup mỗi phía không đưa vào thống kê, script chạy sáu cặp luân phiên thứ tự A/B, mỗi mẫu 400 ms, `GOMAXPROCS=2`. Khi đo A/B không bật profiler. Lượt thực chạy ngày 11/10/2026, Go 1.27.1 `windows/amd64`, ghi nhận:

Build cùng Render	Min–max ns/op, sáu mẫu	Median ns/op
PGO tắt	44,514–49,137	47,083
PGO dùng profile vừa thu	42,463–50,011	46,737.5

Hai khoảng chồng lấn; chênh lệch median nhỏ không đủ để nhận PGO là một cải thiện có ý nghĩa. Đây chỉ là thống kê mô tả, chưa có kiểm định ý nghĩa thống kê; máy vẫn có tải nền và chưa đo toàn service. Từng mẫu cùng raw stdout/stderr được giữ trong evidence của checkpoint, không thay bảng số liệu lịch sử phía trên. Phép thử có giá trị vì cho phép bác bỏ kỳ vọng “bật PGO là chắc chắn nhanh hơn”, chứ không vì buộc phải tạo một tỷ lệ tăng tốc đẹp.

Kỷ luật Tối ưu hóa Kỹ thuật

Với một đề xuất tối ưu hóa, bốn bước sau giữ câu hỏi và bằng chứng đi cùng nhau:

Một là, xác định khối lượng công việc đại diện phản ánh đúng bài toán thực tế.

Hai là, thiết lập kiểm thử đơn vị để cố định hợp đồng kết quả đầu ra không bị biến dạng.

Ba là, kết hợp bài đo chuẩn, phân tích thoát bộ nhớ và công cụ đo hồ sơ hiệu năng để thu thập số liệu định lượng trước khi can thiệp mã nguồn.

Bốn là, đối chiếu lại số liệu sau khi sửa đổi; nếu mức cải thiện không đủ lớn để bù đắp cho độ phức tạp gia tăng của mã nguồn, kiên quyết giữ lại giải pháp đơn giản và dễ bảo trì nhất.

Chương 11 — Lần theo một yêu cầu HTTP

Trong nhật ký vận hành (**log**), một lệnh gọi HTTP thường chỉ còn lại một dòng tóm tắt: `GET https://api.example/... 1.8s`. Dòng nhật ký đó hữu ích, nhưng lại che giấu gần như toàn bộ câu chuyện bên dưới. Khoảng thời gian 1.8 giây có thể là tra cứu DNS, chờ một kết nối rảnh trong bể kết nối, kết nối TCP (**TCP dial**), bắt tay bảo mật (**TLS handshake**), máy chủ xử lý, đọc thân phản hồi, trễ qua máy chủ ủy quyền (**proxy**), hoặc thời hạn chót (**deadline**) do chính bên gọi đặt ra. Nếu chỉ thiết lập một thời gian chờ tổng thể duy nhất và một dòng nhật ký sau cùng, ta chỉ biết yêu cầu thất bại nhưng hoàn toàn không biết cần khắc phục ở tầng nào.

Mô hình tinh thần của chương này: một yêu cầu HTTP đi qua những chặng do địa chỉ đích, trạng thái kết nối và chính sách vận hành quyết định. Tái sử dụng kết nối có thể tránh DNS, TCP và TLS mới. Ngay cả khi phải tạo kết nối, địa chỉ IP trực tiếp không cần phân giải tên, HTTP không có TLS, còn proxy có thể thay đổi nơi quay số và cách phân giải. HTTP/2 còn cho nhiều yêu cầu dùng chung kết nối đang hoạt động. Vì vậy sơ đồ dưới đây giới hạn ở đường HTTP/1.x trực tiếp thông thường, không phải thứ tự bắt buộc của mọi lần gọi.

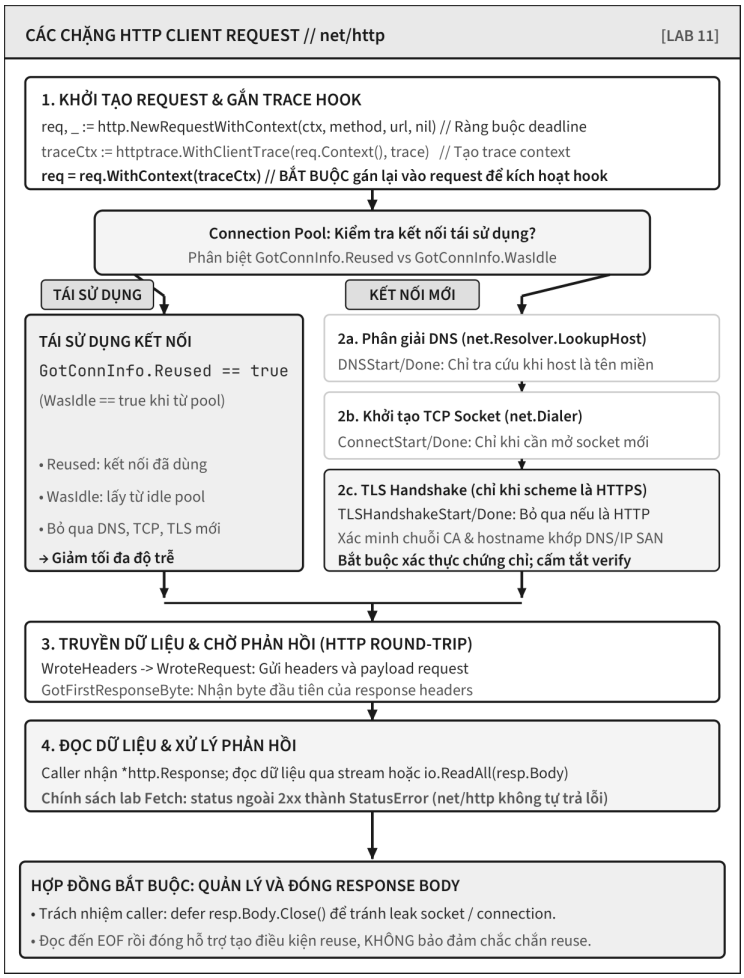
Bắt đầu bằng dấu vết thay vì API

Gói `net/http/httptrace` đính kèm các điểm móc (**hook**) vào ngữ cảnh của một yêu cầu gửi ra ngoài. Nó cho phép người điều tra đánh dấu các sự kiện quan trọng như bắt đầu hoặc kết thúc tra cứu DNS, quay số kết nối TCP, bắt tay TLS, tiếp nhận kết nối mạng, gửi yêu cầu và nhận byte phản hồi đầu tiên. Các điểm móc này có thể được kích hoạt đồng thời từ nhiều goroutine và một số có thể đến sau khi yêu cầu đã hoàn tất hoặc thất bại; do đó, bộ ghi nhận (**recorder**) phải được đồng bộ hóa an toàn nếu gom các sự kiện vào cùng một lát cắt (**slice**).

```
type event struct {
    name string
    at   time.Duration
}

started := time.Now()
trace := &httptrace.ClientTrace{
    DNSStart: func(httptrace.DNSStartInfo) {
        record("dns start", time.Since(started))
    },
    GotConn: func(info httptrace.GotConnInfo) {
        name := fmt.Sprintf("connection reused=%t", info.Reused)
        record(name, time.Since(started))
    },
    GotFirstResponseByte: func() {
        record("first response byte", time.Since(started))
    },
}
traceCtx := httptrace.WithClientTrace(req.Context(), trace)
req = req.WithContext(traceCtx)
```

Trường `GotConnInfo.Reused` mang ý nghĩa quyết định, chứ không chỉ là mốc thời gian thông thường. Thông tin này phân biệt rõ một yêu cầu tái sử dụng kết nối sẵn có trong bể kết nối (**connection pool**) với một yêu cầu phải thiết lập kết nối mạng mới. Dấu vết quan sát (**trace**) là dữ liệu của **một vòng khứ hồi (round trip)**; nó không tự động bao phủ toàn bộ chuỗi chuyển hướng (**redirect**) và không thay thế nhật ký ngữ cảnh nghiệp vụ. Hãy gắn định danh yêu cầu (**request ID**), đích đến đã chuẩn hóa và kết quả vào nhật ký vận hành; chỉ dùng dấu vết truy vết khi cần bóc tách độ trễ (**latency**) chi tiết theo từng chặng kỹ thuật.



Hình 23 — Đường HTTP/1.x trực tiếp: DNS chỉ cần khi phải phân giải tên, TLS chỉ có với HTTPS. Đọc và đóng thân phản hồi tạo điều kiện tái sử dụng nhưng không bảo đảm socket sẽ được giữ lại; HTTP/2 và proxy nằm ngoài hình.

Tên miền, địa chỉ mạng, kết nối và danh tính không phải là một

Tên miền là dữ liệu đầu vào ở tầng ứng dụng. Phương thức `net.Resolver.LookupHost` truy vấn bộ phân giải (**resolver**) cục bộ và trả về danh sách địa chỉ mạng cho máy chủ đích. Bộ phân giải thực tế thay đổi tùy theo hệ điều hành, cấu hình mạng và cờ biên dịch; trên Unix, tài liệu gói `net` mô tả cả bộ phân giải thuần Go lẫn nhánh gọi qua thư viện C trong một số điều kiện. Đừng mặc định rằng "Go luôn tự gửi một gói tin DNS ra ngoài mạng". Hãy ghi nhận lỗi DNS là lỗi của tầng phân giải tên miền, gắn liền với máy chủ đích và thời hạn chót, đồng thời xem cấu hình mạng là một ranh giới độc lập.

Một địa chỉ mạng khả dụng chưa phải là danh tính đã được chứng thực. Khi địa chỉ là HTTPS và cần một kết nối mới, quá trình bắt tay TLS sẽ thương lượng kênh truyền bảo mật. Thuộc tính `tls.Config.ServerName` được

dùng để xác minh tên máy chủ (**hostname**) trên chứng chỉ, đồng thời được gửi trong quá trình bắt tay để hỗ trợ máy chủ ảo (**virtual hosting**) khi không dùng địa chỉ IP trực tiếp. Việc đặt `InsecureSkipVerify=true` để "sửa lỗi chứng chỉ" chính là hành vi vô hiệu hóa việc kiểm tra chuỗi chứng chỉ và tên miền mặc định, chứ không phải giải pháp sửa lỗi mạng. Giá trị mặc định `false` bảo toàn bước kiểm tra an ninh cốt lõi này; nếu sử dụng hàm gọi lại để kiểm tra tùy biến thì phải tự bảo đảm đúng giao ước an toàn. Với cơ quan cấp chứng chỉ (CA) nội bộ, hãy cấu hình chứng chỉ gốc (**root CA**) và chính sách tên máy chủ có chủ đích thay vì tắt bỏ khâu xác thực.

TCP, TLS và HTTP là các tầng giao thức hoàn toàn tách biệt. Kết nối TCP được thiết lập tới địa chỉ và cổng mạng; tầng TLS xác minh danh tính đối tác và thương lượng mã hóa; tầng HTTP thực thi chu trình gửi yêu cầu và nhận phản hồi bên trên. Với HTTP/2, nhiều yêu cầu có thể cùng chia sẻ một kết nối mạng duy nhất (**multiplexing**). Vì thế nhận định "mỗi yêu cầu tương ứng một kết nối TCP riêng biệt" là mô hình tinh thần sai lệch ngay từ đầu.

Chứng chỉ hợp lệ, vì sao thao tác vẫn phải bị từ chối?

Một bên gọi có chứng chỉ máy khách do CA nội bộ ký và hoàn tất bắt tay TLS thành công. Nó gửi yêu cầu `POST /write` kèm tiêu đề `X-Role: admin`. Dịch vụ có được phép thực thi không? Nếu câu trả lời chỉ dựa vào việc "đã kết nối qua mTLS" thì ta vừa đồng nhất quyền được tham gia vào kênh truyền với quyền thực hiện mọi thao tác bên trong kênh truyền đó.

Bài thực hành `labs/part11-request-path/identity` khởi tạo CA, chứng chỉ máy chủ và chứng chỉ máy khách mới trong thời gian chạy bằng gói `crypto/x509`. Trình lắng nghe chỉ mở trên giao diện cục bộ (**loopback**); không có khóa riêng tư nào ghi vào kho mã nguồn, không phụ thuộc vào kho tin cậy của hệ điều hành và tuyệt đối không bỏ qua khâu kiểm thực. Máy khách kết nối tới địa chỉ cục bộ nhưng thiết lập `ServerName=service.test`, khớp chính xác với trường DNS SAN của chứng chỉ máy chủ. Địa chỉ mạng để quay số kết nối và tên máy chủ cần xác minh là hai dữ liệu đầu vào riêng biệt; trường Common Name (CN) cũ không thể thay thế cho SAN trong phép kiểm tra tên máy chủ này.

Hãy mở tệp kiểm thử `TestCertificateBoundaries` và dự đoán từng trường hợp bị từ chối trước khi chạy lệnh. Máy khách bắt buộc phải từ chối chuỗi chứng chỉ không thuộc CA tin cậy hoặc trường SAN không khớp. Máy chủ sử dụng cờ `RequireAndVerifyClientCert`, từ chối khi thiếu chứng chỉ, CA máy khách lạ, chứng chỉ hết hạn hoặc chứng chỉ có mục đích sử dụng không phù hợp. Ca kiểm thử đi qua quy trình bắt tay TLS thực tế của thư viện chuẩn, sau đó xác nhận không có biến đổi dữ liệu trái phép. Kiểm thử không dùng cấu trúc giả để tự gán nhãn đã xác thực.

```
cd labs/part11-request-path
go test -count=1 -v ./identity
go test -race ./identity
```

Sau quá trình bắt tay, hàm `Authorize` chỉ trích xuất định danh URI SAN từ chứng chỉ lá (**leaf certificate**) trong `VerifiedChains`, không lấy vai trò từ tiêu đề HTTP hay từ chứng chỉ chưa được kiểm chứng. Chính sách mẫu cho phép `urn:go-book:reader` đọc dữ liệu tại `/read`; và chỉ tài khoản `urn:go-book:operator` mới được phép ghi dữ liệu tại `/write`. Chứng chỉ do CA tin cậy ký nhưng mang danh tính lạ hoặc thiếu thông tin danh tính đều nhận mã từ chối 403 Forbidden. Tầng TLS không thể thay tầng ứng dụng quyết định chính sách phân quyền này. Chương 12 đặt ranh giới này trước hàm xử lý có tác dụng phụ (**side effects**); Chương 28 tiếp tục ràng buộc thẩm quyền với hành động, đối tượng và quy trình phê duyệt.

> **Thử nghiệm đối chứng:** Kích hoạt biến môi trường `RELIABILITY_MUTANT=trust_equals_role` rồi chạy kiểm thử `TestAuthorizationContract`. Biến thể lỗi sẽ cấp quyền thực hiện mọi hành động cho bất kỳ chuỗi chứng chỉ hợp lệ nào; cùng ca kiểm thử đó sẽ thất bại (báo đỏ) vì tài khoản đọc ghi được dữ liệu và bộ đếm đột biến tăng lên. Không được khắc phục bằng cách tắt khâu kiểm thực chứng chỉ: điều đó sẽ phá vỡ ranh giới danh tính và không giải quyết được bài toán phân quyền.

Bài thực hành chọn TLS 1.3 làm chính sách cục bộ, không áp đặt đây là yêu cầu cho mọi môi trường triển khai thực tế. Nó chưa đi sâu vào cơ chế xoay vòng chứng chỉ, thu hồi chứng chỉ (**revocation**), kết thúc TLS tại cổng trung gian hay nhà cung cấp danh tính tập trung. Cơ chế mTLS là một giải pháp khi có hạ tầng phân phối và quản lý chứng chỉ tương thích; đây không phải điều kiện bắt buộc cho mọi tầng truyền tải giao thức MCP.

Thời hạn chót có phạm vi, không phải một con số trang trí

Hàm `http.NewRequestWithContext` ràng buộc ngữ cảnh vào toàn bộ vòng đời của yêu cầu gửi ra ngoài: tìm kết nối mạng, gửi yêu cầu, đọc tiêu đề và đọc toàn bộ thân phản hồi. Đó là thời hạn chót sát nhất với mục tiêu của bên gọi: khi bên gọi mất kiên nhẫn, toàn bộ yêu cầu cần nhận biết lý do để dừng lại.

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(
    ctx,
    http.MethodGet,
    rawURL,
    nil,
)
if err != nil {
    return err
}
resp, err := client.Do(req)
```

Thuộc tính `http.Client.Timeout` thiết lập giới hạn toàn diện từ đầu đến cuối (**end-to-end**) cho các yêu cầu qua máy khách đó, bao gồm cả thời gian đọc thân phản hồi. Cấu hình `http.Transport` cung cấp các chốt chặn hẹp hơn: `TLSHandshakeTimeout`, `ResponseHeaderTimeout`, giới hạn số lượng kết nối và thời hạn quay số. Mỗi chốt chặn giải quyết một yêu cầu chính sách riêng biệt, vì vậy không nên áp đặt máy móc một con số “5 giây” chung chung cho mọi trường cấu hình.

Bảng 10 — Timeout phải gắn với một khả năng chờ cụ thể

Chốt	Phạm vi	Câu hỏi thiết kế
Context của request	Toàn bộ ý nghĩa của request với caller	Caller còn cần kết quả sau bao lâu?
`Client.Timeout`	End-to-end cho call qua client	Client dùng chung có một ngân sách chung hợp lý không?
`TLSHandshakeTimeout`	Handshake TLS mới	Có cần chặn handshake treo mà vẫn cho body dài?
`ResponseHeaderTimeout`	Sau khi gửi đủ request, chờ response headers	Server phải bắt đầu phản hồi trong bao lâu?
`MaxConnsPerHost`	Tổng connection theo host; dial có thể block khi đầy	Bao nhiêu concurrency là quota có chủ đích?

Thời gian chờ không thể tự trả lời câu hỏi liệu việc thử lại (**retry**) có an toàn hay không. Bộ điều vận `Transport` có cơ chế thử lại nội bộ có giới hạn cho một số lỗi mạng trên các kết nối đã từng dùng thành công, khi yêu cầu có tính lũy đẳng (**idempotent**) và phần thân có thể gửi lại; đó là hành vi được ghi trong tài liệu của thư viện, không phải cam kết của tầng nghiệp vụ. Nếu mã nguồn tự động thử lại yêu cầu `POST`, lập trình viên bắt buộc phải định nghĩa khóa lũy đẳng (**idempotency key**), ngân sách số lần thử lại, thời hạn chót chung và tiên liệu hậu quả khi máy chủ đã xử lý xong nhưng máy khách chưa kịp nhận phản hồi. “Lỗi mạng” không đồng nghĩa với việc thao tác nghiệp vụ chưa từng diễn ra.

Thân phản hồi là khoản nợ tài nguyên của bên gọi

Phương thức `Client.Do` trả về cấu trúc `Response` với trường thân `Body` khác `nil` khi không có lỗi. Bên gọi bắt buộc phải đóng thân phản hồi này. Việc đọc hết đến ký tự kết thúc `io.EOF` rồi mới đóng sẽ tạo điều kiện để tái sử dụng kết nối HTTP/1.x duy trì (**keep-alive**), dù không cam đoan yêu cầu tiếp theo chắc chắn sẽ dùng lại kết nối đó. Từ Go 1.27, khi gọi lệnh `Close`, tầng hiện thực HTTP/1 sẽ tự động giải phóng phần dữ liệu chưa đọc tới một hạn mức thận trọng để hỗ trợ tái sử dụng kết nối. Điều đó không đồng nghĩa với việc đóng một phần thân bất kỳ đều sẽ đọc hết dữ liệu hoặc giữ lại được kết nối mạng; không nên biến chi tiết cài đặt này thành chính sách phân định quyền sở hữu tài nguyên. Hàm tiện ích hoặc trả về `*http.Response` và ghi rõ tài liệu yêu cầu bên gọi phải đóng thân phản hồi, hoặc chủ động đọc hết, đóng lại rồi chỉ trả về giá trị dữ liệu độc lập.

Bài thực hành lựa chọn chính sách thứ hai cho phản hồi nhỏ, đáng tin. Hàm `Fetch` gắn ngữ cảnh bên gọi vào yêu cầu và đóng thân trên cả nhánh thành công lẫn lỗi đọc. Nó đọc thân trước rồi mới kiểm tra mã trạng thái cuối cùng sau chính sách chuyển hướng của `Client`; nếu đọc thành công mà mã ngoài dải 2xx, nó trả `StatusError`. Lỗi đọc thân được ưu tiên, không tự biến thành `StatusError`. `Client.Do` không coi mã ngoài 2xx là lỗi truyền tải. Bộ kiểm thử truyền máy khách và bộ điều vận giả để quan sát contract mà không cần mạng ngoài; `Fetch` hiện không có trần số byte, nên không phải hàm an toàn cho phản hồi lớn hoặc nguồn không đáng tin.

```
cd labs/part11-request-path
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test -tags traceexercise ./exercise
go test -race -tags traceexercise ./exercise
go test ./fixed
```

Sau hàm `Fetch`, còn một nhiệm vụ độc lập thứ hai: chuyển các điểm móc truy vết thành dữ liệu có thể đọc mà không phát sinh xung đột dữ liệu (**data race**) mới. Mở tệp `exercise/trace_test.go`; kiểm thử không đòi hỏi kết nối DNS hay máy chủ thực tế. Bộ kiểm thử kết nối các điểm móc `DNSStart`, `GotConn` và `GotFirstResponseByte` vào một bộ ghi nhận, rồi gọi phương

thức `GotConn` đồng thời từ nhiều goroutine. Ta có thể tự chọn cấu trúc dữ liệu nội bộ và cơ chế đồng bộ hóa; các tên sự kiện chuẩn hóa theo bài kiểm thử gồm "dns start", "connection reused=true" và "first response byte", và phương thức `Snapshot` bắt buộc phải trả về bản sao độc lập để bên gọi không can thiệp vào lịch sử của bộ ghi. Đây là bài kiểm tra tối thiểu cho đúng lời cảnh báo ở đầu chương: `httptrace` kích hoạt các điểm móc đồng thời; dấu vết dùng để điều tra yêu cầu không được phép tự gây ra xung đột dữ liệu ngay trong công cụ điều tra.

Thứ tự sự kiện trong một ảnh chụp trạng thái tuần tự có thể là bằng chứng hữu ích cho một yêu cầu cụ thể. Khi các điểm móc kích hoạt đồng thời, bộ ghi nhận chỉ cam kết lưu giữ các sự kiện đã tiếp nhận mà không tự suy diễn thứ tự nhân quả thiếu cơ sở. Hãy dùng mốc thời gian để hỗ trợ điều tra, đối chiếu với trường `GotConnInfo.Reused`, thời hạn chót và kết quả cuối cùng; không nên vội vã kết luận từ danh sách sự kiện rằng tra cứu DNS, quay số hay máy chủ luôn là chặng gây chậm trễ nhất.

Khi bên gọi cần truyền phát (**stream**) một phản hồi dung lượng lớn, chính sách của bài thực hành sẽ không còn phù hợp: không nên dùng `io.ReadAll` chỉ để “cho đơn giản”. Khi ấy giao diện lập trình bắt buộc phải trả về luồng dữ liệu và chuyển giao trách nhiệm gọi lệnh `Close` lại cho bên gọi, tương tự như bài toán vào/ra dữ liệu ở Chương 7. Một giao diện lập trình tinh gọn nhưng mơ hồ về việc đóng thân phản hồi thường gây rò rỉ kết nối mạng âm thầm và nguy hiểm hơn nhiều so với một hàm có thêm vài dòng tài liệu giải thích rõ ràng.

Cấu trúc `http.Client` và `http.Transport` an toàn khi sử dụng đồng thời từ nhiều luồng; tài liệu chuẩn `net/http` khuyến nghị nên khởi tạo và tái sử dụng chúng vì bộ điều vận lưu giữ trạng thái nội bộ như bộ đệm kết nối (**connection cache**). Việc tái sử dụng không đồng nghĩa với việc lạm dụng một biến toàn cục vô danh. Hãy đóng gói một máy khách đã được cấu hình chặt chẽ theo đúng chính sách vận hành của dịch vụ, truyền nó qua tham số vào ranh giới cần thực hiện lệnh gọi mạng, đồng thời duy trì khả năng kiểm thử độc lập cho cả bộ điều vận lẫn thời hạn chót.

Thí nghiệm cục bộ: yêu cầu thứ hai có thực sự tái sử dụng kết nối cũ?

Tệp `labs/part11-request-path/fixed/reuse_test.go` sử dụng `httptest.Server`, một đối tượng `http.Transport` mới và gói `httptrace`. Thí nghiệm này hoàn toàn không có bước tra cứu DNS hay mã hóa TLS; đó là chủ ý thiết kế nhằm cô lập hoàn toàn bể kết nối rảnh. Hai yêu cầu tuần tự đọc trọn vẹn dữ liệu rồi đóng thân phản hồi. Ca kiểm thử kiểm tra giá trị `GotConnInfo.Reused` mang giá trị `false` cho yêu cầu đầu tiên, và `true` cho yêu cầu tiếp theo. Điều này không khẳng định mọi yêu cầu trên môi trường thực tế đều sẽ được tái sử dụng: máy chủ có thể chủ động ngắt kết nối, thân phản hồi có thể chưa được hoàn trả về bể, hạn ngạch hoặc cơ chế ghép kênh HTTP/2 có thể làm thay đổi đường đi. Thí nghiệm chỉ mang lại bằng chứng có thể tái lập cho một mệnh đề xác thực: khi bộ điều vận đã sở hữu một kết nối rảnh tương thích, vòng khứ hồi tiếp theo có thể bỏ qua việc khởi tạo kết nối mạng mới.

```
cd labs/part11-request-path
go test -run TestTransportReusesIdleConnection ./fixed
go test -race ./fixed
```

Kiểm thử `TestLiteralIPNeedsNoDNSAndServerMayPreventReuse` đặt thêm một phản ví dụ: máy chủ cục bộ dùng địa chỉ IP và gửi `Connection: close`. Hai yêu cầu đều đọc hết và đóng thân nhưng không có điểm móc DNS nào và không tái sử dụng kết nối. Điều này tách trách nhiệm đóng tài nguyên của bên gọi khỏi quyền quyết định giữ kết nối của hai phía.

Khi điều tra yêu cầu chậm, hãy kiểm tra kết nối có được tái sử dụng, giới hạn nào đã kích hoạt, các mốc DNS/TLS/byte đầu tiên nằm ở đâu, thân phản hồi có được đóng và việc thử lại có an toàn cho nghiệp vụ hay không. Một dòng nhật ký 1.8 giây là điểm khởi đầu; kết luận cần dữ liệu ở đúng chặng đang nghi ngờ.

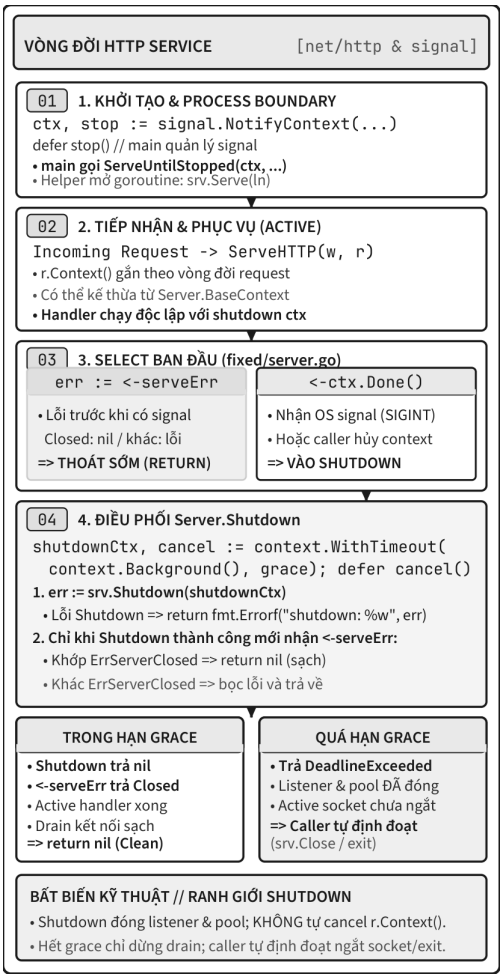
Chương 12 — Một service sống và tắt thế nào

Một dịch vụ nhỏ thường bắt đầu bằng `http.ListenAndServe(":8080", handler)`. Dòng lệnh đó đủ để mở cổng mạng (**port**), nhưng chưa đủ để khẳng định dịch vụ đang cam kết điều gì với bên gọi, với hệ thống triển khai (**deployment system**), hay với người vận hành. Khi tiến trình nhận tín hiệu dừng, nếu hàm `main` kết thúc ngay thì các yêu cầu đang xử lý sẽ bị cắt đứt. Khi một máy khách gửi tiêu đề quá chậm, nếu máy chủ thiếu chính sách thời gian chờ đọc tiêu đề thì kết nối mạng có thể chiếm dụng tài nguyên lâu hơn dự kiến. Khi hàm xử lý tiếp nhận JSON lạ và cố suy đoán ngữ nghĩa, ranh giới giữa dữ liệu đầu vào chưa được kiểm chứng và trạng thái nội bộ sẽ hoàn toàn biến mất.

Mô hình tinh thần của chương này: dịch vụ là một ranh giới biến dữ liệu đầu vào chưa được kiểm chứng thành các đơn vị công việc có giới hạn tài nguyên, sau đó rút lui an toàn theo một vòng đời có thể quan sát được. Việc định tuyến, phần mềm trung gian, kiểm tra tính hợp lệ và tắt êm dịu chỉ thực sự đáng đưa vào khi chúng làm sáng tỏ ranh giới này. Chúng không phải một danh mục tính năng để phô diễn cấu trúc khung làm việc (**framework**).

Một lần triển khai bị cắt ngang cho thấy sự thiếu hụt giao ước vận hành

Xét trường hợp `/v1/checks` tiếp nhận một mục tiêu kiểm tra, lưu yêu cầu, rồi trả về mã `201`. Trong một lần triển khai phiên bản mới, bộ cân bằng tải (**load balancer**) chuyển hướng lưu lượng mạng sang phiên bản mới nhưng tiến trình cũ nhận tín hiệu dừng ngay giữa lúc đang ghi phản hồi. Có ba trạng thái cần phân biệt rõ ràng: dịch vụ còn nhận kết nối mới, đang để các yêu cầu đã tiếp nhận chạy đến khi hoàn tất, hay đã dừng hẳn. Phương thức `Server.Shutdown` của `net/http` đóng trình lắng nghe (**listener**), đóng các kết nối rảnh (**idle connection**), rồi kiên nhẫn chờ các kết nối đang hoạt động hoàn tất để trở về trạng thái rảnh rồi mới đóng lại. Lệnh này không tự hủy ngữ cảnh của hàm xử lý đang chạy. Ngữ cảnh `r.Context()` của yêu cầu gửi đến có vòng đời độc lập: nó bị hủy khi máy khách ngắt kết nối, luồng HTTP/2 bị hủy, hoặc khi `ServeHTTP` trả về, chứ không phải chỉ vì `Shutdown` vừa được gọi. Nó không tự chờ các kết nối đã chuyển giao quyền kiểm soát (**hijacked connection**), chẳng hạn như WebSocket; những kết nối tồn tại lâu dài luôn cần chính sách vòng đời chuyên biệt.



Hình 24 — Tách kết quả `Shutdown` khỏi vòng đời của handler. Hết ngân sách chờ trả về lỗi context, không chứng minh handler đã kết thúc; DB, logger và kết nối hijack vẫn cần bên sở hữu riêng.

Một hàm chạy server có thể giữ contract ấy trong một nơi thay vì để `main` thoát vì mọi error đều trông giống nhau. Hãy đọc phần đầu và phần shutdown dưới đây như hai đoạn liên tiếp của cùng một function; tách chúng ra cũng giúp lifecycle hiện rõ hơn trên trang:

```
func ServeUntilStopped(
    ctx context.Context,
    srv *http.Server,
    ln net.Listener,
    grace time.Duration,
) error {
    if srv == nil {
        return errors.New("HTTP server is required")
    }
    if ln == nil {
        return errors.New("listener is required")
    }
    if grace <= 0 {
        return errors.New("shutdown grace must be positive")
```

```
    }

    serveErr := make(chan error, 1)
    go func() { serveErr <- srv.Serve(ln) }()

    select {
    case err := <-serveErr:
        if errors.Is(err, http.ErrServerClosed) {
            return nil
        }
        return fmt.Errorf("serve: %w", err)
    case <-ctx.Done():
    }
```

Đến đây máy chủ đã nhận yêu cầu dừng, nhưng `ctx` không nên dùng trực tiếp làm ngữ cảnh tắt dịch vụ: ngữ cảnh đó đã bị hủy. Ta tạo một ngân sách thời gian mới để `Shutdown` có cơ hội hoàn tất các yêu cầu đang xử lý dở dang (**drain**):

```
    base := context.Background()
    shutdownCtx, cancel := context.WithTimeout(base, grace)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil {
        return fmt.Errorf("shutdown: %w", err)
    }
    err := <-serveErr
    if err == nil {
        return errors.New("server stopped without an error")
    }
    if !errors.Is(err, http.ErrServerClosed) {
        return fmt.Errorf("serve after shutdown: %w", err)
    }
    return nil
}
```

`srv`, `ln` và `grace` là ba tiền điều kiện mà helper kiểm tra; bên gọi cũng phải cung cấp context khác `nil` theo contract của gói `context`. `shutdownCtx` được tạo từ `context.Background()` vì context phát tín hiệu dừng đã bị hủy. Ngân sách `grace` cần phù hợp với giới hạn triển khai và thời lượng yêu cầu, không lấy context đã hủy làm cơ hội chờ drain.

Sau khi hết ngân sách, `Shutdown` trả lỗi context nhưng handler có thể vẫn chạy. Ngay cả `Serve` trả `ErrServerClosed` cũng chỉ xác nhận vòng nhận kết nối đã dừng, không phải mọi công việc đã kết thúc. Test `TestShutdownExpiredBudgetDoesNotFinishActiveHandler` giữ handler tại một cổng chờ, dùng deadline đã quá hạn rồi xác nhận context request chưa bị hủy và handler chưa hoàn tất. Fixture tự mở cổng chờ và join khi dọn dẹp. `Server.Close` có thể cưỡng bức đóng kết nối HTTP do server quản lý, nhưng không giết goroutine đang tính toán hoặc tự dọn DB/logger. Chỉ đóng các phụ thuộc sau khi những tác vụ còn dùng chúng đã kết thúc theo policy đã chọn.

`RegisterOnShutdown` dùng để phát thông báo bắt đầu shutdown cho giao thức riêng; `Shutdown` không chờ các callback này hoàn tất. Test `TestShutdownCallbackIsNotificationNotJoin` giữ một callback đang chờ ngay cả khi `Shutdown` đã trả `nil`. Nếu WebSocket hoặc tác vụ nền cần join, chủ sở hữu phải tạo và chờ tín hiệu hoàn tất riêng.

Từ signal của process sang context của server

`ServeUntilStopped` đã nhận một `context.Context`, nên hàm `main` chỉ cần nối vòng đời của tiến trình vào đầu vào đó. Trên các máy chủ tương thích Unix, `signal.NotifyContext` chuyển tín hiệu `SIGINT` hoặc `SIGTERM` thành thao tác hủy ngữ cảnh; lệnh `defer stop()` hủy đăng ký bắt tín hiệu khi `main` thoát ra. Cho đến khi hàm `stop()` được kích hoạt, tín hiệu hệ điều hành vẫn được điều hướng theo chính sách này; nếu hệ thống muốn ngắt lần thứ hai quay về hành vi mặc định hoặc thoát cưỡng bức, đó phải là một quyết định thiết kế có chủ đích. Sau tín hiệu đầu tiên, `ServeUntilStopped` bước vào giai đoạn `Shutdown` với khoảng thời gian `grace` đã chọn thay vì lập tức cắt ngang các yêu cầu đang xử lý.

```
ctx, stop := signal.NotifyContext(
    context.Background(),
    os.Interrupt,
    syscall.SIGTERM,
)
defer stop()

if err := ServeUntilStopped(
    ctx,
    srv,
    ln,
    15*time.Second,
); err != nil {
    return err
}
```

Đây là cầu nối tại ranh giới tiến trình, không phải sự cho phép để hàm xử lý (**handler**) bỏ qua `r.Context()`: `Shutdown` chỉ ngừng tiếp nhận yêu cầu mới và chờ các yêu cầu đang chạy hoàn tất; nó không tự hủy `r.Context()` của các hàm xử lý đó. Hàm xử lý vẫn bắt buộc phải truyền `r.Context()`

vào cơ sở dữ liệu hoặc các lệnh gọi HTTP ra bên ngoài (**outbound**) để thích ứng với vòng đời riêng của từng yêu cầu gửi đến.

Nếu chính sách của dịch vụ quy định tín hiệu dừng cũng phải hủy các hàm xử lý đang chạy, việc đó cần được đấu nối tường minh, chẳng hạn dùng ngữ cảnh ứng dụng làm `Server.BaseContext` rồi hủy ngữ cảnh đó ngay khi nhận tín hiệu:

```
appCtx, cancelApp := context.WithCancel(context.Background())
srv.BaseContext = func(net.Listener) context.Context {
    return appCtx
}
// Policy đã chọn khi nhận signal:
cancelApp()
```

Đó là một chính sách khác với việc rút lui êm dịu thuần túy (**graceful drain**): hàm xử lý nào theo dõi ngữ cảnh đều có thể dừng trước khi hoàn tất toàn bộ công việc đang phục vụ. Lựa chọn này phù hợp khi việc giới hạn thời gian tắt tiến trình quan trọng hơn việc chờ mọi yêu cầu hoàn thành trọn vẹn. Trên Windows, `os.Interrupt` là tín hiệu khả chuyển; máy chủ dịch vụ và trình quản lý tiến trình cần được kiểm chứng trên đúng môi trường thực tế trước khi giả định `SIGTERM` có cùng hành vi tiếp nhận.

Thao tác đã xong nhưng ngữ cảnh con vẫn còn sống

Một hàm xử lý đã trả kết quả không đồng nghĩa mọi tài nguyên do nó tạo ra đều được giải phóng đúng lúc. Trong bài thực hành `part10-resource-retention`, hàm `DoTimeout` tạo ngữ cảnh con (**child context**) có thời hạn một giờ trực thuộc ngữ cảnh cha (**parent context**) sống lâu nhưng có thể hủy. Hàm gọi lại (**callback**) trả về ngay. Ngữ cảnh con sẽ hết hạn cùng hàm gọi lại, cùng hàm xử lý, hay theo lộ trình hủy của chính ngữ cảnh? Ngữ cảnh không thể tự biết hàm gọi lại đã hoàn tất công việc. Nếu bên sở hữu bỏ quên hàm hủy `CancelFunc`, ngữ cảnh con vẫn không bị hủy chỉ vì hàm vừa kết thúc.

Theo giao ước (**contract**) của gói `context`, gọi lệnh hủy sẽ hủy ngữ cảnh con cùng toàn bộ các thế hệ con cháu kế thừa (**descendants**), gỡ bỏ quan hệ liên

kết từ ngữ cảnh cha và dừng bộ đếm thời gian (**timer**) liên quan. Trong mã nguồn Go 1.27.1, phương thức `cancelCtx.propagateCancel` đăng ký ngữ cảnh con vào tập `children` của ngữ cảnh cha; phương thức `timerCtx.cancel` tháo gỡ liên kết đó và dừng bộ đếm. Đây là chi tiết hiện thực được kiểm chứng trên phiên bản này, không phải trường dữ liệu để tầng ứng dụng truy cập trực tiếp. Ngữ cảnh cha còn sống, thời hạn còn xa và không ai gọi lệnh hủy là điều kiện đủ để giữ các ngữ cảnh con tồn tại vượt quá thời gian thao tác thực sự cần đến. Bộ thu gom rác (GC) không thể thay thế cho quyết định kết thúc chủ động này.

Đặt trách nhiệm dọn dẹp ngay tại nơi sở hữu một lần gọi:

```
func DoTimeout(
    parent context.Context,
    budget time.Duration,
    work func(context.Context) error,
) error {
    ctx, cancel := context.WithTimeout(parent, budget)
    defer cancel()
    return work(ctx)
}
```

Lệnh `defer` bảo vệ cả nhánh thực thi thành công lẫn nhánh phát sinh lỗi. Nếu tạo nhiều ngữ cảnh con bên trong vòng lặp của một hàm sống suốt vòng đời dịch vụ rồi `defer` mọi thao tác hủy ở hàm bao bọc ngoài cùng, việc dọn dẹp tài nguyên sẽ bị trì hoãn tới tận khi dịch vụ ngừng hẳn. Hãy tách mỗi thao tác thành một hàm có phạm vi cục bộ ngắn như trên, hoặc chủ động gọi hủy ở cuối mỗi vòng lặp và trên mọi nhánh thoát. Đồng thời, không trả ngữ cảnh con ra cho tác vụ đang chạy nền rồi lập tức hủy ngay khi hàm tạo kết thúc: cần bàn giao hàm hủy `CancelFunc` cùng trách nhiệm chờ đợi hoàn tất cho bên tiếp nhận mới, tránh áp dụng `defer` một cách máy móc.

Kiểm thử của bài thực hành ghi nhận ngữ cảnh con mà hàm gọi lại tiếp nhận. Với bản sửa lỗi trả về, `child.Err()` mang giá trị `context.Canceled`; với bản còn lỗi, ngữ cảnh con vẫn chưa có lỗi, rồi chỉ bị hủy khi kiểm thử chủ động hủy ngữ cảnh cha. Một ca kiểm thử khác dùng thời hạn chót đã quá hạn để chứng minh ngữ cảnh con có thể tự kết thúc dù

bên sở hữu quên hủy. Các ca kiểm thử này tập trung kiểm chứng vòng đời công khai, không xâm phạm trường nội bộ hay kỳ vọng một con số suy giảm RAM thiếu căn cứ.

Công cụ `go vet` phát hiện trường hợp phổ biến như bỏ qua `CancelFunc` bằng ký tự gạch dưới `_`; dữ liệu mẫu `testdata/lostcancel` cố ý tạo ra cảnh báo chẩn đoán đó. Tuy nhiên, bản lỗi chuyển hàm hủy cho hàm `ForgetCancel` rỗng: công cụ phân tích tĩnh cục bộ có thể nhận định trách nhiệm đã được bàn giao trong khi thực tế không ai thực thi việc dọn dẹp. Vì vậy, kết quả `go vet` sạch không chứng minh quyền sở hữu tài nguyên đã chuẩn xác. Đọc kỹ luồng gọi mã nguồn và kiểm thử thời điểm kết thúc của ngữ cảnh con là hai bằng chứng bổ sung bắt buộc.

Không phải mọi trường hợp quên gọi hủy đều dẫn đến rò rỉ bộ nhớ vĩnh viễn. Ngữ cảnh cha bị hủy hoặc thời hạn chót trôi qua thì ngữ cảnh con cũng sẽ chấm dứt; với `WithCancel(context.Background())` không có bộ đếm hay quan hệ cha con giữ ngữ cảnh, việc bỏ mọi tham chiếu tới ngữ cảnh con sẽ không tự hình thành chuỗi giữ từ gốc. Dù vậy, một ngữ cảnh con mang theo dữ liệu giá trị lớn, có nhánh con cháu, bộ đếm thời gian hoặc tác vụ đang chờ sẽ gây tiêu tốn tài nguyên khác biệt so với một ngữ cảnh con rỗng. Cần xác định rõ nhánh tham chiếu giữ lại và thời gian lưu giữ dư thừa. Lệnh `CancelFunc` cũng không chờ hàm gọi lại kết thúc: bài toán chờ tín hiệu hoàn tất `done` từ Chương 9 vẫn giữ nguyên giá trị.

Ngữ cảnh của yêu cầu gửi đến `r.Context()` do máy chủ HTTP quản lý; hàm xử lý không nhận `CancelFunc` để tự hủy ngữ cảnh gốc đó. Hàm xử lý chỉ chịu trách nhiệm đối với các ngữ cảnh con do chính nó khởi tạo để gọi các dịch vụ hạ nguồn (**downstream**). Chính sách hủy các yêu cầu đang hoạt động khi tắt tiến trình là quyết định kiến trúc riêng biệt đã nêu ở trên, không được nhầm lẫn giữa việc quên gọi hủy trong một thao tác với mô hình tinh thần về `Server.Shutdown`.

Hàm xử lý là ranh giới kiểm tra, không phải nơi "cố hiểu" dữ liệu đầu vào

Trong bài thực hành, `POST /v1/checks` là một ranh giới nhỏ. Điểm cuối tiếp nhận dữ liệu JSON chỉ chứa trường `target`, chấp nhận giao thức `http` hoặc `https` với máy chủ không rỗng, chuyển giá trị hợp lệ sang kho lưu trữ `Store`, và không để lộ lỗi nội bộ của kho lưu trữ ra phía máy khách. Bài thực hành chưa thực hiện lệnh thăm dò gửi ra ngoài: đó là quyết định tiềm ẩn rủi ro giả mạo yêu cầu phía máy chủ (SSRF) và cạn kiệt hạn ngạch (**quota**), do đó không nên đưa tùy tiện vào một hàm xử lý mẫu.

> **Dừng để dự đoán:** Xét một hàm xử lý HTTP tiếp nhận gói tin (**payload**) thông qua `json.Decoder`:

```
decoder := json.NewDecoder(r.Body)
var input Check
err := decoder.Decode(&input)
```

> Nếu client gửi body gồm một document hợp lệ rồi nối thêm dữ liệu thừa như `{"target":"https://example.com"}{"debug":true}`, lệnh `decoder.Decode(&input)` có báo lỗi không, và phần dữ liệu thừa phía sau sẽ đi về đâu? Làm thế nào để chứng minh stream đã thực sự kết thúc trước khi chuyển giao dữ liệu sang `Store`?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Lệnh `decoder.Decode(&input)` đọc và giải mã thành công object đầu tiên vào biến `input` mà không trả về lỗi nào, bởi vì `json.Decoder` được thiết kế để xử lý luồng stream và dừng lại ngay khi hoàn tất một JSON value hợp lệ. Lúc này, phần dữ liệu thừa phía sau không nhất thiết còn nguyên vẹn trong `r.Body`. Do `json.Decoder` duy trì một bộ đệm đọc trước (read buffer) nội bộ, các byte của object thứ hai có thể đã được nạp sẵn vào bộ đệm của decoder hoặc vẫn còn nằm lại trên stream của request. Nếu handler dừng lại ở lần gọi đầu tiên rồi tiếp tục xử lý nghiệp vụ, dữ liệu thừa sẽ bị bỏ qua và ranh giới kiểm soát input bị phá vỡ.

Để chứng minh luồng dữ liệu (**stream**) không còn dữ liệu thừa ngoài khoảng trắng, hàm xử lý thực hiện thêm một lần giải mã thứ hai vào một cấu trúc rỗng (`var extra struct{}`). Khi luồng dữ liệu đã kết thúc đúng chuẩn, bộ giải mã bắt buộc phải trả về chính xác lỗi `io.EOF`. Mọi dữ liệu còn sót lại trong bộ đệm giải mã hoặc trên luồng mạng đều khiến lần gọi thứ hai trả về một kết quả khác `io.EOF`, tạo bằng chứng rõ ràng để hàm xử lý từ chối yêu cầu bằng mã 400 Bad Request trước khi chuyển giao việc cho `Store`.

Dưới đây là handler của lab; các helper về lỗi và ghi JSON được định nghĩa trong cùng package:

```
func (a app) createCheck(
    w http.ResponseWriter,
    r *http.Request,
) {
    if r.Method != http.MethodPost {
        methodNotAllowed(w)
        return
    }
    if a.store == nil {
        internalError(w)
        return
    }

    r.Body = http.MaxBytesReader(w, r.Body, maxRequestBody)
    decoder := json.NewDecoder(r.Body)
    decoder.DisallowUnknownFields()

    var input Check
```

```
    err := decoder.Decode(&input)
    if err != nil || validateTarget(input.Target) != nil {
        badRequest(w)
        return
    }
    var extra struct{}
    if err := decoder.Decode(&extra); !errors.Is(err, io.EOF) {
        badRequest(w)
        return
    }
    check, err := a.store.Create(r.Context(), input)
    if err != nil {
        internalError(w)
        return
    }
    writeJSON(w, http.StatusCreated, check)
}
```

Lần gọi `Decode` thứ hai không nhằm đọc một bản ghi kiểm tra khác. Nó chỉ kiểm chứng rằng sau giá trị đầu tiên chỉ còn khoảng trắng và ký hiệu kết thúc tệp `io.EOF`; nếu vẫn còn giá trị JSON dư thừa, yêu cầu sẽ bị từ chối trước khi chuyển giao cho `Store`. Đoạn mã trên không phải bộ giải mã dùng chung cho mọi giao diện lập trình (API): chẳng hạn điểm tiếp nhận tải tệp lên không thể áp dụng mức trần 4 KiB cố định này. Nguyên tắc cốt lõi là giới hạn dung lượng, lược đồ dữ liệu và kiểm tra tính hợp lệ phải được thực thi ngay trước khi dữ liệu đầu vào biến thành công việc nội bộ; không nên giải mã vô hạn rồi mới kiểm tra tính hợp lệ của yêu cầu.

`http.MaxBytesReader` giới hạn số byte được đọc từ thân; lab trả 400 khi việc giải mã hoặc kiểm tra phần dư gặp lỗi, kể cả vượt 4 KiB. `DisallowUnknownFields` từ chối tên trường không ánh xạ vào struct, không phải bộ kiểm tra lược đồ hoàn chỉnh. Với `encoding/json` đang dùng, tên trường được đối chiếu không phân biệt hoa/thường và trường trùng được xử lý theo thứ tự: hai trường `target` kiểu string khiến giá trị sau thay giá trị trước. Test `TestJSONBoundaryDocumentsDuplicateCaseAndSizePolicy` ghi rõ hành vi này và chứng minh thân quá lớn chưa chạm tới `Store`. Nếu chữ ký

hay phân quyền đòi một biểu diễn không nhập nhằng, cần policy từ chối trường trùng và quy tắc tên chính xác riêng; không suy ra chúng từ cờ này.

Bài thực hành bắt đầu bằng các ca kiểm thử thất bại (báo đỏ) đối với yêu cầu hợp lệ, sai phương thức HTTP, JSON chứa trường lạ, chứa tài liệu JSON thứ hai và sai mục tiêu kiểm tra. Hãy tự hoàn thiện hàm xử lý trước khi tham khảo thư mục `fixed/`:

```
cd labs/part12-service-lifecycle
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test -tags lifecycleexercise ./exercise
go test -race -tags lifecycleexercise ./exercise
go test ./fixed
```

Kiểm thử của hàm xử lý còn đặt hai giá trị JSON liên tiếp để phân biệt hai loại lỗi dễ nhầm: tài liệu hợp lệ đầu tiên không được phép khiến phần dữ liệu thừa phía sau bị âm thầm bỏ qua. Sau khi hàm xử lý vượt qua kiểm thử (báo xanh), bộ kiểm thử `lifecycleexercise` truyền trình lắng nghe cục bộ và ngữ cảnh hủy vào `ServeUntilStopped`; giao ước đặt ra là trình lắng nghe phục vụ yêu cầu trước khi có tín hiệu hủy, máy chủ trả về sau khi tắt êm dịu, và các tiền điều kiện sai phải trả về lỗi tường minh thay vì gây sụp đổ (`panic`). Kiểm thử dùng `httptest` hoặc trình lắng nghe cục bộ, nhờ đó kiểm chứng trọn vẹn giao ước HTTP mà không cần mở cổng ra mạng. Phương thức `ResponseRecorder.Result()` ghi nhận ảnh chụp trạng thái phản hồi sau khi hàm xử lý thực thi; không nên dùng `DeepEqual` trên toàn bộ cấu trúc `http.Response`, mà chỉ so khớp mã trạng thái, tiêu đề (**header**) và phần thân mà dịch vụ đã cam kết.

Định tuyến và phần mềm trung gian là cách đặt luật ở đúng ranh giới

Một bộ định tuyến (**router**) tinh gọn phát huy giá trị khi người bảo trì nhìn vào bảng định tuyến là biết phương thức nào dẫn đến hàm xử lý nào. Với dịch vụ có ít điểm cuối tiếp nhận, `http.ServeMux` kết hợp kiểm tra phương thức tường minh thường hiệu quả hơn một thư viện phụ thuộc cồng kềnh. Mẫu định tuyến, yêu cầu xác thực và giới hạn thân yêu cầu nên được thể hiện rõ ràng ngay tại ranh giới tiếp nhận; hàm nghiệp vụ bên trong chỉ nhận giá trị đã được làm sạch và ngữ cảnh của yêu cầu.

Phần mềm trung gian (**middleware**) hợp lý là nơi thực thi các quy tắc chung cho nhiều yêu cầu: đính kèm định danh yêu cầu do máy chủ tạo, ghi nhật ký kết quả, truy vết (**tracing**), xác thực người dùng hoặc kiểm soát tần suất gửi yêu cầu (**rate limiting**). Đây không phải nơi để nhồi nhét rẽ nhánh nghiệp vụ. Nếu phần mềm trung gian cần ghi nhận mã trạng thái và số byte đã truyền, bộ bọc `ResponseWriter` phải tuyệt đối tôn trọng giao ước của `net/http`; một lớp bọc cẩu thả có thể phá vỡ các giao diện tùy chọn hoặc ghi tiêu đề sau khi thân phản hồi đã gửi. Hãy bắt đầu với các lớp trung gian nhỏ gọn có thể kiểm thử độc lập, thay vì sao chép một chuỗi bộ bọc phức tạp mà không ai còn hiểu rõ.

Nhật ký vận hành (**logging**) phải giải đáp được câu hỏi giám sát: định danh yêu cầu, tuyến đường và phương thức, mã trạng thái, thời lượng xử lý, kết quả và phân loại lỗi. Không được ghi mật khẩu, mã xác thực quyền truy cập (**bearer token**), tệp cookie hay toàn bộ phần thân yêu cầu theo mặc định. Lỗi trả về cho máy khách là giao ước công khai; nhật ký nội bộ có thể lưu vết nguyên nhân kỹ thuật chi tiết với quyền truy cập phù hợp. Khi hai thông điệp này trùng lặp nguyên văn, đó là dấu hiệu ranh giới kỹ thuật chưa được phân định rõ ràng.

Thời hạn chờ phía máy chủ phòng ngừa một kiểu áp lực khác

Thời gian chờ phía máy khách ở chương trước bảo vệ bên **gọi**. Phía máy chủ cũng phải thiết lập các giới hạn để bảo vệ tài nguyên của chính nó. `ReadHeaderTimeout` giới hạn thời gian đọc tiêu đề yêu cầu rồi tái lập thời hạn chót để hàm xử lý tự quyết định tốc độ và thời hạn đọc phần thân. `ReadTimeout` bao trùm toàn bộ thời gian đọc thân yêu cầu nhưng không cho phép hàm xử lý áp dụng chính sách riêng cho từng tác vụ tải tệp. `WriteTimeout` giới hạn thời gian ghi phản hồi; `IdleTimeout` giới hạn thời gian chờ yêu cầu kế tiếp trên kết nối duy trì (**keep-alive**). `MaxHeaderBytes` đặt trần bảo vệ bộ phân tích cú pháp tiêu đề.

Bảng 11 — Server timeout là các policy khác nhau

Field	Bảo vệ điều gì	Cần quyết định cùng nó
`ReadHeaderTimeout`	Client gửi header quá chậm	Header hợp lệ cần bao lâu trong mạng thực?
`ReadTimeout`	Toàn bộ đọc request, gồm body	Endpoint upload có rate/lifetime riêng không?
`WriteTimeout`	Response write kéo dài	Streaming response có cần policy khác?
`IdleTimeout`	Keep-alive không gửi request tiếp	Bao lâu connection rảnh vẫn đáng giữ?
`MaxHeaderBytes`	Header bất thường lớn	Cookie/proxy hợp lệ chiếm bao nhiêu?

Giá trị zero không có cùng nghĩa ở mọi trường. `ReadTimeout` và `WriteTimeout` không dương nghĩa là không áp giới hạn ấy. Nhưng `ReadHeaderTimeout=0` và `IdleTimeout=0` lấy giá trị `ReadTimeout`; muốn hiểu cấu hình phải đọc cả quan hệ kế thừa này. `MaxHeaderBytes=0` dùng mức mặc định riêng. Với HTTP/1 trong Go 1.27.1, deadline ghi được đặt sau khi đọc header, nên thời gian handler xử lý trước lần ghi cũng có thể tiêu ngân sách `WriteTimeout`; đây không phải đồng hồ chỉ bắt đầu ở lệnh `Write` đầu tiên hay cơ chế cưỡng chế hủy handler. Không suy từ cấu hình socket rằng computation đã dừng. Hãy kiểm chứng policy trên giao thức và môi trường triển khai thật, nhất là với streaming hoặc proxy.

Phương thức `Server.Shutdown` không ngắt các yêu cầu đang hoạt động và không tự hủy `r.Context()` của hàm xử lý. Hàm xử lý vẫn phải tôn trọng

`r.Context()` khi tương tác với cơ sở dữ liệu, máy khách gọi ra ngoài hoặc luồng xử lý nền, bởi vì yêu cầu gửi đến có vòng đời độc lập. Chương 9 và Chương 11 đã cung cấp hai mảnh ghép còn lại: goroutine phải có đường thoát khi ngữ cảnh bị hủy; yêu cầu của máy khách phải tiếp nhận ngữ cảnh từ bên gọi. Việc tắt êm dịu chỉ thực sự đạt được khi các ranh giới bên trong sẵn sàng trả lại quyền điều khiển.

Một dịch vụ đáng tin cậy được đo lường tại các ranh giới: đầu vào bị giới hạn ở đâu, mã nghiệp vụ tiếp nhận dữ liệu gì, phản hồi và nhật ký có phân tách rõ ràng giữa thông tin công khai và nội bộ hay không, máy chủ ngừng nhận việc khi nào và yêu cầu cũ được bao nhiêu thời gian để kết thúc. Kết nối WebSocket cần một bên sở hữu riêng biệt. Trả lời trọn vẹn những câu hỏi đó, dịch vụ mới có một vòng đời hoàn chỉnh để vận hành thay vì chỉ gọi `ListenAndServe` một cách phó mặc.

Chương 13 — Một thay đổi hoặc không có gì

Một buổi trực, dịch vụ trả mã `500` cho thao tác tắt một điểm kiểm tra đang gặp lỗi. Người trực ca tin rằng yêu cầu thất bại nên thực hiện gọi lại. Lần đọc tiếp theo lại cho thấy điểm kiểm tra đã bị tắt, nhưng nhật ký kiểm toán (**audit log**) không ghi nhận dòng nào về danh tính người thực hiện. Đây không phải lỗi bỏ quên nhật ký đơn thuần. Từ ngoài nhìn vào, hệ thống đang kể hai câu chuyện mâu thuẫn về cùng một thay đổi.

Trạng thái bền vững sẽ được yêu cầu kế tiếp, tiến trình khác hoặc lần khởi động sau sử dụng. Câu hỏi đầu tiên là: sau thao tác nhiều bước, những trạng thái nào được phép tồn tại? Mô hình tư duy của chương là ranh giới xác nhận: các thay đổi cùng ý nghĩa nghiệp vụ phải được xác nhận hoặc hoàn nguyên cùng nhau. Tính nguyên tử bảo vệ kết cục lưu trữ; việc người đọc đồng thời thấy gì còn phụ thuộc mức cô lập, snapshot và cách họ nhóm các lần đọc. Hai truy vấn độc lập vẫn có thể đọc một phần trước và một phần sau commit, dù giao dịch ghi nguyên tử.

Một `if err != nil` không tự tạo atomicity

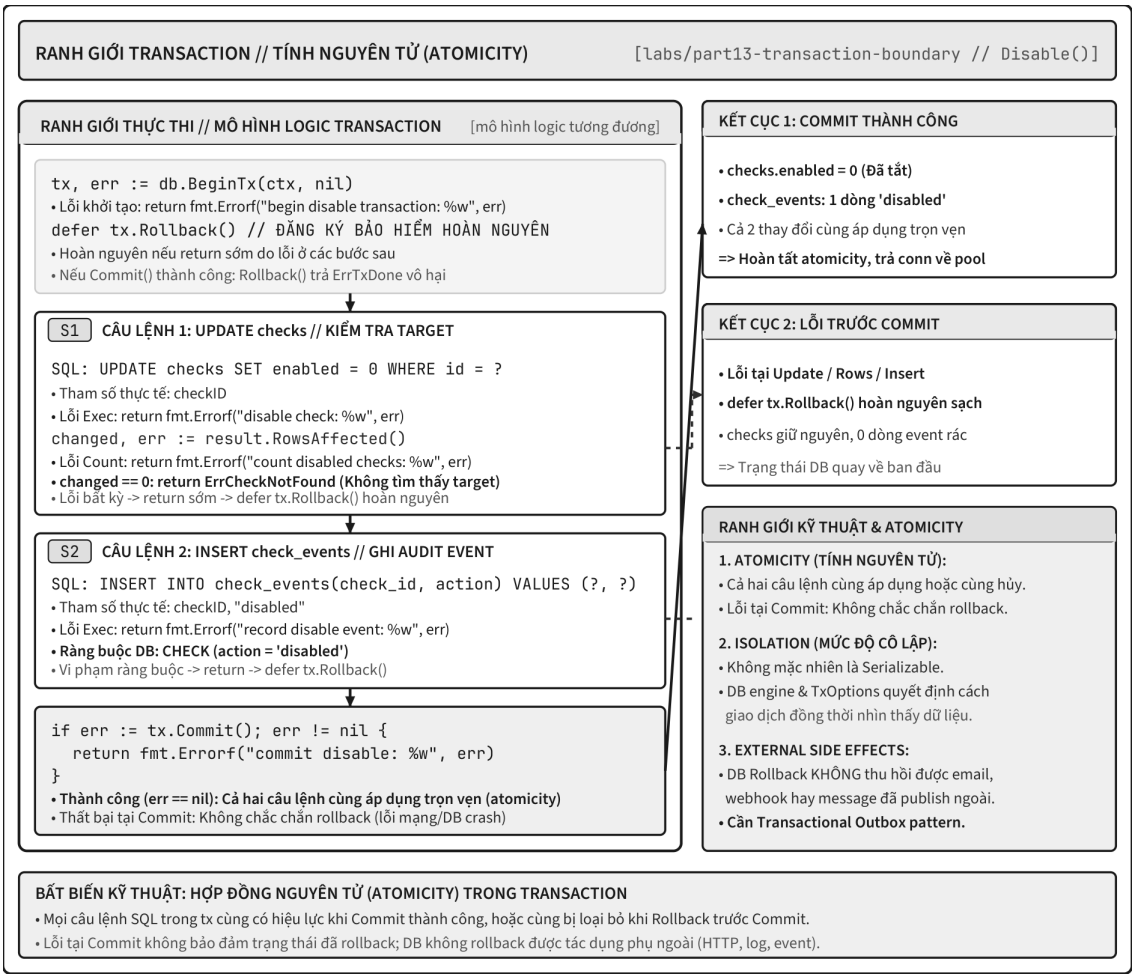
Giả sử API `Disable` cần làm hai việc: đổi `checks.enabled` thành `false`, rồi ghi event `disabled`. Đoạn code ngây thơ có vẻ cẩn thận vì đã kiểm tra error sau từng câu SQL:

```
if _, err := db.ExecContext(ctx,
    "UPDATE checks SET enabled = 0 WHERE id = ?", checkID,
); err != nil {
    return err
}

if _, err := db.ExecContext(ctx,
    "INSERT INTO check_events(check_id, action) VALUES (?, ?)",
    checkID, "disabled",
); err != nil {
    return err
}
```

Nhưng hai lời gọi độc lập là hai ranh giới thực thi tách biệt. Nếu câu lệnh `UPDATE` thành công còn `INSERT` lỗi vì vi phạm ràng buộc toàn vẹn (**constraint**), hàm trả về lỗi thật, song cơ sở dữ liệu vẫn giữ điểm kiểm tra ở trạng thái tắt. Bên gọi có lý do để thực hiện thử lại; người điều tra sự cố đối mặt với một trạng thái thiếu nguyên nhân phát sinh. Không có dòng `if` nào tự động đảo ngược câu lệnh đầu tiên chỉ vì câu lệnh kế tiếp thất bại.

Giao dịch thông báo cho cơ sở dữ liệu rằng tập câu lệnh thuộc một đơn vị công việc: các thay đổi được xác nhận hoặc hoàn nguyên cùng nhau theo contract của hệ quản trị. Tính nguyên tử không có nghĩa "hàm Go trả lỗi thì chắc chắn chưa commit". Nếu phản hồi xác nhận bị mất, bên gọi có thể không biết kết cục dù phía lưu trữ đã quyết định. Khi `Commit` trả lỗi, tài liệu Go yêu cầu bỏ các kết quả của giao dịch vì không còn xác nhận được chúng là hợp lệ; việc truy vấn lại hoặc thử lại cần định danh thao tác và policy của database/driver.



Hình 25 — Sơ đồ khái niệm về tính nguyên tử (**atomicity**) của thao tác `Disable`. Hình không khẳng định mức độ cô lập (**isolation level**) nào; cơ sở dữ liệu, trình điều khiển (**driver**) và `TxOptions` mới quyết định chi tiết mức cô lập được hỗ trợ.

> **Dừng để dự đoán:** Nếu câu lệnh ghi sự kiện bị từ chối, ca kiểm thử nào chứng minh điểm kiểm tra vẫn giữ trạng thái `enabled`? Việc chỉ kiểm tra hàm trả về lỗi chưa đủ; trạng thái của dữ liệu sau sự cố mới là điều hợp đồng hành vi phải khóa chặt.

Bài thực hành: Làm cho thất bại không để lại dấu vết nửa chừng

Bài thực hành dùng SQLite trong bộ nhớ (**in-memory**) qua gói `modernc.org/sqlite`, một trình điều khiển thuần Go. Lý do chọn công cụ này là để kiểm thử chạy cục bộ độc lập, không cần Docker, thông tin xác thực hay kết nối mạng. Gói `database/sql` vẫn là giao diện lập trình chính; trình điều khiển là bộ chuyển đổi thích ứng theo phương ngữ (**dialect**) và giao thức của cơ sở dữ liệu cụ thể. Việc dùng SQLite không phải lời khuyên thay thế PostgreSQL cho mọi dịch vụ: cơ chế giữ chỗ tham số (**placeholder**), khả năng xử lý đồng thời (**concurrency**) và hành vi vận hành giữa hai hệ thống là hoàn toàn khác biệt.

Mở `exercise/disable_test.go` trước. Test đã dựng schema và nêu API, nhưng chưa cho implementation. Cần viết hàm `Disable` sao cho thỏa mãn đủ bốn giao ước:

Một là, `check` tồn tại được tắt và có đúng một event `disabled`.

Hai là, không có `check` thì trả `ErrCheckNotFound`, không tự tạo event.

Ba là, nếu event không thể ghi, `check` vẫn giữ `enabled = true`.

Bốn là, `nil` database là lỗi ở API boundary, không phải một lỗi `panic` muộn trong goroutine hay driver.

```
cd labs/part13-transaction-boundary
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go test -race ./fixed
```

Đừng mở `fixed/` trước lượt tự làm đầu tiên. Contract đã nói rõ điều gì phải đúng, còn thiết kế do người học chủ động quyết định: transaction được mở ở đâu, cleanup đặt thế nào, statement nào phải chạy qua `tx` thay vì `db`, và `RowsAffected` được dùng ra sao để phân biệt không tìm thấy check với cập nhật thành công.

```
// Đáp án tham chiếu: một transaction, một commit.
func Disable(
    ctx context.Context,
    db *sql.DB,
    checkID int64,
) error {
    if db == nil {
        return errors.New("database is required")
    }
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return fmt.Errorf("begin disable transaction: %w", err)
    }
    defer tx.Rollback()

    result, err := tx.ExecContext(ctx,
        "UPDATE checks SET enabled = 0 WHERE id = ?", checkID,
    )
    if err != nil {
        return fmt.Errorf("disable check: %w", err)
    }
    changed, err := result.RowsAffected()
```

```
    if err != nil {
        return fmt.Errorf("count disabled checks: %w", err)
    }
    if changed == 0 {
        return ErrCheckNotFound
    }

    if _, err := tx.ExecContext(
        ctx,
        "INSERT INTO check_events(check_id, action) " +
            "VALUES (?, ?)",
        checkID,
        "disabled",
    ); err != nil {
        return fmt.Errorf("record disable event: %w", err)
    }
    if err := tx.Commit(); err != nil {
        return fmt.Errorf("commit disable: %w", err)
    }
    return nil
}
```

Sau khi tự làm, hãy đối chiếu `fixed/disable.go`. `defer tx.Rollback()` đặt ngay sau `BeginTx` bảo vệ những đường thoát trước khi giao dịch kết thúc; mọi statement của đơn vị phải chạy qua `tx`, không chen `db.ExecContext` ở ngoài. Sau `Commit` thành công, rollback trả `sql.ErrTxDone`. Trong Go 1.27.1, nếu đã gọi `driver.Tx.Commit` rồi driver trả lỗi, `sql.Tx` cũng đã đánh dấu kết thúc: deferred rollback không gọi ngược xuống driver để đảo commit. Test `TestCommitErrorDoesNotMakeDeferredRollbackUndoCommit` dùng driver giả cố ý trả lỗi xác nhận để kiểm chứng vòng đời thật của `database/sql`; nó không giả làm thí nghiệm mất gói tin với database thật. Cleanup của Go và kết cục lưu trữ là hai ranh giới khác nhau.

Phương thức `RowsAffected` trả về số dòng bị tác động do trình điều khiển cung cấp; một số hệ cơ sở dữ liệu hoặc câu lệnh đặc thù có thể không hỗ trợ thông tin này. Bài thực hành dùng SQLite, nơi hợp đồng này được kiểm chứng nhất quán. Trong dịch vụ thực tế, kỹ sư cần chọn cách phân biệt trường hợp không tìm thấy bản ghi dựa trên ngữ nghĩa của hệ quản trị và truy vấn đang dùng, thay vì xem một phương thức tiện tay là quy chuẩn chung của SQL.

Ký tự `?` cũng không phải cú pháp giữ chỗ phổ quát. Trình điều khiển SQLite dùng ký tự đó, trong khi nhiều trình điều khiển PostgreSQL lại dùng `$1`, `$2`. Tuy nhiên nguyên tắc cốt lõi không thay đổi: luôn truyền giá trị qua đối số của trình điều khiển, không ghép chuỗi đầu vào trực tiếp vào câu lệnh SQL. Cơ chế ràng buộc tham số (**parameter binding**) tách biệt dữ liệu khỏi cấu trúc cú pháp; cơ chế này không thay thế cho kiểm tra hợp lệ nghiệp vụ, nhưng ngăn ngừa việc biến `target` hay `checkID` thành một mảnh mã lệnh ngoài ý muốn.

`*sql.DB` là một handle dùng chung, không phải "một connection"

Gói `database/sql` là thư viện chuẩn cung cấp giao diện lập trình chung cho các cơ sở dữ liệu quan hệ; nó luôn cần một trình điều khiển cụ thể để hoạt động. Theo tài liệu của thư viện, kiểu `*sql.DB` là một đối tượng xử lý (**handle**) an toàn khi dùng đồng thời và quản lý một tập kết nối dùng chung (**connection pool**). Vì thế, hãy giữ đối tượng này tại ranh giới vòng đời ứng dụng và đóng lại khi tiến trình dừng hoạt động; tránh tạo mới rồi đóng `*sql.DB` theo từng yêu cầu HTTP riêng lẻ.

Một hiểu lầm phổ biến là xem `sql.Open` như bằng chứng cho thấy cơ sở dữ liệu đang hoạt động thông suốt. Việc mở đối tượng xử lý thường không thiết lập kết nối mạng ngay lập tức. Khi quá trình khởi động hoặc chính sách kiểm tra sức khỏe cần xác nhận khả năng tiếp cận máy chủ, hãy dùng `PingContext` kèm thời hạn xử lý (**deadline**) phù hợp; tránh biến lệnh kiểm tra này thành thủ tục bắt buộc trước mỗi truy vấn.

```
func OpenAndCheck(
    ctx context.Context,
    dsn string,
) (*sql.DB, error) {
    db, err := sql.Open("sqlite", dsn)
    if err != nil {
        return nil, fmt.Errorf("open database handle: %w", err)
    }
    if err := db.PingContext(ctx); err != nil {
        db.Close()
        return nil, fmt.Errorf("ping database: %w", err)
    }
    return db, nil
}
```

Ví dụ này chỉ minh họa boundary. Nó không chép sẵn `SetMaxOpenConns(50)` vì một con số đẹp không phải policy. Quản lý connection pool trong `database/sql` xoay quanh bốn chính sách cốt lõi và một giao diện quan sát:

Một là, `SetMaxOpenConns` đặt trần cho tổng số kết nối mở tại một thời điểm, bao gồm cả kết nối đang bận thực thi lẫn kết nối rảnh trong pool. Mặc định giá trị này bằng 0 (không giới hạn). Khi chạm trần, các truy vấn tiếp theo bị chặn và xếp hàng chờ cho đến khi có kết nối được giải phóng hoặc context bị hủy. Nếu một goroutine đang giữ một kết nối trong transaction lại cố gắng thực hiện một truy vấn độc lập từ cùng pool mà không còn kết nối trống, việc chạm trần sẽ trực tiếp dẫn đến bế tắc đồng thời (deadlock).

Hai là, `SetMaxIdleConns` khống chế số lượng kết nối rảnh tối đa được giữ lại trong pool. Giá trị mặc định trong tài liệu hiện hành của thư viện chuẩn là 2 (tài liệu lưu ý giá trị mặc định này có thể thay đổi trong các phiên bản tương lai). Nếu ứng dụng đặt `SetMaxOpenConns` rất cao nhưng giữ nguyên `SetMaxIdleConns` bằng 2, sau mỗi đợt tăng tải đột biến, toàn bộ các kết nối dư thừa sẽ bị đóng ngay lập tức, dẫn đến hiện tượng mở và đóng kết nối liên tục (connection churn), tiêu tốn chi phí bắt tay kết nối và xác thực tài khoản.

Ba là, `SetConnMaxIdleTime` xác định thời gian rảnh rỗi tối đa mà một kết nối được phép nằm chờ trong pool trước khi bị thu hồi (đóng lười khi rảnh), giúp thu nhỏ kích thước pool về mức tối thiểu trong những khung giờ thấp điểm.

Bốn là, `SetConnMaxLifetime` quy định tuổi thọ tối đa mà một kết nối được phép tiếp tục tái sử dụng (reuse) tính từ thời điểm khởi tạo; các kết nối quá hạn có thể được đóng một cách lười (lazily) trước khi tái sử dụng. Đây không phải cơ chế ngắt cưỡng chế (hard kill) khi kết nối đang bận phục vụ truy vấn. Việc chủ động đặt giới hạn tuổi thọ kết nối giúp làm mới tài nguyên kết nối định kỳ theo yêu cầu của hạ tầng hoặc máy chủ cơ sở dữ liệu.

Năm là, phương thức `DB.Stats()` cung cấp dữ liệu quan sát trực tiếp tại thời gian chạy qua cấu trúc `sql.DBStats`. Các trường `OpenConnections`,

`InUse`, `Idle`, `WaitCount`, `WaitDuration`, `MaxIdleClosed`, `MaxIdleTimeClosed` và `MaxLifetimeClosed` phục vụ việc chẩn đoán xem ứng dụng có đang gặp tình trạng thiếu hụt kết nối hay phải xếp hàng chờ đợi kéo dài hay không; đây là số liệu quan sát hỗ trợ điều tra, không phải lời khẳng định nguyên nhân gốc rễ duy nhất. Việc lựa chọn thông số pool phải dựa trên năng lực chịu tải của máy chủ cơ sở dữ liệu phía sau, số lượng bản sao tiến trình cùng chia sẻ tài nguyên, và số liệu thực nghiệm từ `DB.Stats()`, tránh sao chép những con số cảm tính.

Cơ chế hủy bỏ (**cancellation**) cũng dừng lại tại ranh giới của trình điều khiển. Phương thức `QueryContext` và `BeginTx` nhận đối số ngữ cảnh (`context.Context`), nhưng tài liệu `database/sql` ghi rõ rằng nếu trình điều khiển không hỗ trợ hủy bỏ ngữ cảnh thì phương thức có thể chỉ trả về kết quả sau khi truy vấn đã hoàn tất trên máy chủ. Ngữ cảnh vẫn phải được truyền xuyên suốt; đồng thời, thời hạn thực tế của truy vấn cần được kiểm chứng với đúng trình điều khiển và máy chủ cơ sở dữ liệu trước khi đưa vào mục tiêu mức dịch vụ (**SLO**).

Đọc cũng mượn tài nguyên

Phương thức `QueryRowContext` phù hợp khi truy vấn được kỳ vọng trả về tối đa một dòng dữ liệu; lỗi phát sinh sẽ xuất hiện trong bước quét dữ liệu `Scan`. Phương thức này không kiểm tra tính duy nhất: nếu truy vấn trả về nhiều dòng, `Scan` chỉ đọc dòng đầu tiên và bỏ qua phần còn lại; nếu không có dòng nào, nó trả về lỗi `sql.ErrNoRows`. Bất biến nghiệp vụ rằng chỉ có duy nhất một bản ghi phải do câu truy vấn hoặc lược đồ dữ liệu bảo đảm, không thể tự suy diễn từ tên hàm. Khi làm việc với nhiều dòng, `QueryContext` trả về `*Rows`, đại diện cho dòng kết quả và tài nguyên của trình điều khiển. Hãy đóng đối tượng này trên mọi nhánh trả về, duyệt qua toàn bộ dữ liệu rồi kiểm tra lỗi lặp qua phương thức `rows.Err()`.

```
rows, err := db.QueryContext(ctx,
    "SELECT id, target FROM checks WHERE enabled = ?", true,
)
if err != nil {
    return nil, fmt.Errorf("list enabled checks: %w", err)
}
defer rows.Close()

var checks []Check
for rows.Next() {
    var check Check
    if err := rows.Scan(&check.ID, &check.Target); err != nil {
        return nil, fmt.Errorf("scan check: %w", err)
    }
    checks = append(checks, check)
}
if err := rows.Err(); err != nil {
    return nil, fmt.Errorf("iterate checks: %w", err)
}
return checks, nil
```

Đoạn code này chưa đề cập đến mức cô lập (isolation level): việc thao tác đọc nhận diện được hàng dữ liệu nào khi có luồng ghi đồng thời đang chạy phụ thuộc hoàn toàn vào chính sách của giao dịch và cơ sở dữ liệu. Nó chỉ chốt vòng đời ở phía Go: truy vấn đã mượn tài nguyên thì hàm phải hoàn trả. Đây là cùng một trách nhiệm ta đã gặp với response body ở Chương 11 và listener ở Chương 12, chỉ ở một ranh giới khác.

Câu lệnh chuẩn bị sẵn (Prepared Statement) và ranh giới tài nguyên

`PrepareContext` tạo một statement có thể thực thi nhiều lần với các đối số khác nhau. Driver và hệ quản trị quyết định nó chuẩn bị, lập kế hoạch và tái sử dụng tài nguyên thế nào; `database/sql` không hứa một lần biên dịch SQL duy nhất cho cả pool. Statement tạo từ `DB` có thể được chuẩn bị lại trên kết nối khác, còn statement từ `Tx` gắn với kết nối của giao dịch. Ví dụ sau chuẩn bị một mẫu chèn sự kiện rồi dùng lại trong cùng `tx`:

```
stmt, err := tx.PrepareContext(ctx,
    "INSERT INTO check_events(check_id, action) VALUES (?, ?)",
)
if err != nil {
    return fmt.Errorf("prepare insert event: %w", err)
}
defer stmt.Close()

for _, action := range actions {
    _, err := stmt.ExecContext(ctx, checkID, action)
    if err != nil {
        return fmt.Errorf("insert event %s: %w", action, err)
    }
}
```

Quy tắc sở hữu tài nguyên ở đây rất rõ ràng: một prepared statement tạo từ `db.PrepareContext` sống cùng với vòng đời của database handle và có thể tự động chuẩn bị lại trên các kết nối vật lý khác nhau trong pool khi cần thiết. Ngược lại, statement được tạo bên trong một transaction qua `tx.PrepareContext` gắn chặt với vòng đời của chính transaction đó; khi transaction kết thúc bằng `Commit` hoặc `Rollback`, statement này sẽ tự động đóng và không thể tiếp tục sử dụng. Lệnh `stmt.Close()` tường minh vẫn hữu ích khi muốn giải phóng tài nguyên sớm trước khi transaction hoàn tất, nhưng không phải điều kiện duy nhất để ngăn ngừa rò rỉ trong transaction.

Prepared statement không mặc định đồng nghĩa với việc mọi câu lệnh đều chạy nhanh hơn. Lợi ích tối ưu của prepared statement phát huy rõ rệt khi cùng một mẫu câu lệnh được thực thi lặp lại nhiều lần với các tham số khác nhau (chẳng hạn trong các vòng lặp chèn dữ liệu hàng loạt như dự án `opsprobe` ở Chương 20). Nếu một câu lệnh chỉ được gọi duy nhất một lần trong toàn bộ vòng đời của request, chi phí phụ trội để chuẩn bị câu lệnh cần được cân nhắc và đo đạc thực nghiệm với driver và hệ quản trị cơ sở dữ liệu cụ thể trước khi quyết định áp dụng.

Việc truyền giá trị qua tham số (parameter binding / placeholder) tách biệt rành mạch dữ liệu khỏi cấu trúc câu lệnh SQL, giúp phòng ngừa lớp lỗ hổng tấn công chèn mã SQL (SQL injection) do nối trực tiếp giá trị vào chuỗi truy

vấn. Tuy nhiên, theo tài liệu hướng dẫn bảo mật chính thức của Go (`go.dev/doc/database/sql-injection`), parameter binding chỉ áp dụng cho các giá trị tham số; các thành phần động trong câu lệnh như tên bảng, tên cột, mệnh đề `ORDER BY` hoặc các đoạn SQL ghép nối vẫn cần ranh giới kiểm tra và danh sách trắng (allowlist) riêng.

Transaction giải quyết được gì, và không giải quyết được gì

Giao dịch giúp gom nhóm các thao tác ghi trên cùng một cơ sở dữ liệu. Cơ chế này không tự biến lời gọi HTTP, việc phát thông điệp (**publish message**), vô hiệu hóa bộ nhớ đệm hay gửi thư điện tử thành một khối nguyên tử cùng với lệnh xác nhận (**commit**). Nếu hàm `Disable` vừa xác nhận giao dịch vừa gọi webhook, sự cố sau khi xác nhận có thể khiến cơ sở dữ liệu lưu dữ liệu mới nhưng webhook chưa từng được phát đi. Đó là một bài toán ranh giới khác, đòi hỏi định danh phân biệt, cơ chế thử lại có kiểm soát và mô hình hộp thư gửi đi (**outbox pattern**) khi yêu cầu nghiệp vụ đặt ra; một lệnh `defer tx.Rollback()` không thể giải quyết được ranh giới liên hệ thống này.

Atomicity không làm retry tự nhiên an toàn

Nhìn lại sự cố mở đầu chương: phản hồi từ dịch vụ có thể bị thất lạc ngay sau khi lệnh `Commit` đã hoàn tất. Bên gọi chỉ quan sát thấy lỗi đường truyền mạng nhưng cơ sở dữ liệu đã thay đổi trạng thái bền vững. Giao dịch đã giữ đúng cam kết phạm vi hẹp của nó: câu lệnh `UPDATE` và `INSERT` cùng có hiệu lực hoặc cùng bị hủy bỏ. Tuy nhiên giao dịch chưa thể phân định lần gọi lại tiếp theo là cùng một thao tác cần lặp lại hay là một yêu cầu hoàn toàn mới.

Với hàm `Disable` trong bài thực hành, thao tác `UPDATE checks SET enabled = 0` thoạt nhìn có vẻ mang tính lũy đẳng (**idempotent**): chạy lại nhiều lần thì trường trạng thái vẫn giữ nguyên giá trị `false`. Tuy nhiên lệnh `INSERT` sự kiện kiểm toán sẽ ghi thêm một dòng bản ghi mới. Vì vậy, việc trạng thái đích đạt giá trị mong muốn chưa đủ để kết luận thao tác thử lại là an toàn; các tác dụng phụ (**side effects**) như nhật ký kiểm toán, thông báo hoặc hạn ngạch có thể biến đổi theo mỗi lượt thực thi. Kiểm thử

`TestDisableRepeatedCallRecordsAnotherAuditEvent` cố ý chứng minh hợp đồng hiện tại của bài thực hành: việc gọi `Disable` hai lần sẽ tạo ra hai sự kiện độc lập. Kiểm thử thành công không phải bằng chứng khẳng định giao diện lập trình an toàn khi gọi lại; đó là thực nghiệm để tránh ngộ nhận rằng tính nguyên tử đương nhiên đồng nghĩa với tính lũy đẳng.

Khi yêu cầu nghiệp vụ quy định bên gọi có thể thử lại cùng một thao tác nhưng hệ thống chỉ được sinh ra duy nhất một tác dụng phụ, giao diện lập trình bắt buộc phải có định danh cho thao tác đó: chẳng hạn khóa lũy đẳng (**idempotency key**) được bên gọi lưu giữ qua các lần gửi lại. Dịch vụ cần nhận diện xem khóa này đã được xử lý hay chưa để trả về kết quả tương ứng, thường bằng ràng buộc duy nhất (**unique constraint**) hoặc bản ghi theo dõi trạng thái nằm trong cùng giao dịch cơ sở dữ liệu. Các chi tiết như cấu trúc phản hồi khi phát hiện khóa trùng lặp, thời gian lưu giữ khóa và những tác dụng phụ nào thuộc về cùng thao tác đều là thành phần trong cam kết của dịch vụ. Chúng không thể tự động hình thành chỉ nhờ lời gọi `BeginTx`.

> **Dừng để dự đoán:** Một bên gọi gửi yêu cầu `Disable` nhưng gặp lỗi quá hạn thời gian chờ (**timeout**) ở tầng truyền tải mạng và phản hồi từ dịch vụ bị thất lạc. Tại thời điểm nhận lỗi quá hạn, bên gọi chỉ biết mình chưa nhận được kết quả, trong khi phía máy chủ có thể đã xác nhận thành công hoặc chưa từng hoàn tất giao dịch. Nếu bên gọi tự ý gửi lại lệnh `Disable` mà không mang theo khóa lũy đẳng, việc quan sát thấy trường `enabled = false` cùng số lượng bản ghi sự kiện kiểm toán tăng lên 2 sau đó nói lên điều gì về sự khác biệt giữa sự thật phía máy chủ và bằng chứng bên gọi sở hữu, cũng như ranh giới an toàn của việc thử lại?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Sự cố mạng làm đứt gãy kênh truyền khiến bên gọi chỉ sở hữu một bằng chứng duy nhất: yêu cầu đã quá hạn thời gian chờ, hoàn toàn không có bằng chứng xác nhận máy chủ đã kịp hoàn tất giao dịch hay chưa. Trong thực tế, máy chủ có thể đã xác nhận giao dịch thành công trước khi gói tin phản hồi bị rơi rụng trên đường truyền. Khi bên gọi tự động gửi lại lệnh mà không có khóa lũy đẳng, trạng thái `enabled = false` trên bảng `checks` không chứng minh được lần gọi nào đã tạo nên kết quả, bởi tài nguyên có thể đã tắt từ lần gọi đầu tiên hoặc do một tiến trình khác can thiệp.

Bằng chứng rõ ràng nhất nằm ở bảng `check_events`: số lượng bản ghi tăng lên 2 khẳng định giao dịch thứ hai đã thực thi độc lập và ghi thêm tác dụng phụ. Tình huống này chứng minh tính nguyên tử (**atomicity**) của giao dịch chỉ bảo đảm các câu lệnh trong một phiên cùng thành công hoặc cùng thất bại, chứ không tự động mang lại tính an toàn khi gọi lại (**idempotency**). Để bên gọi có thể thử lại an toàn sau các lỗi mạng không chắc chắn, hệ thống bắt buộc phải cung cấp hợp đồng thử lại tường minh thông qua khóa lũy đẳng được kiểm tra và lưu vết nguyên tử cùng giao dịch nghiệp vụ.

Tương tự, giao dịch không tự động biến mọi thao tác đồng thời thành tuần tự. Cấu trúc `sql.TxOptions` cho phép bên gọi yêu cầu mức độ cô lập (**isolation level**); khi bên gọi yêu cầu mức cô lập khác mặc định mà trình điều khiển không hỗ trợ, `BeginTx` sẽ trả về lỗi. Hệ quản trị và trình điều khiển thực tế mới là nơi quyết định các hiện tượng dị thường đọc dữ liệu (**read anomalies**), cơ chế khóa và độ trễ phản hồi có ngữ nghĩa ra sao. Do đó, trước khi dùng mức cô lập để bảo vệ một bất biến nghiệp vụ thực tế, kỹ sư cần đọc kỹ tài liệu cơ sở dữ liệu, thiết kế kiểm thử tranh chấp cho bất biến đó và đo lường lỗi cũng như thời gian chờ khóa. Trong chương này, ta tập trung khóa chặt một hợp đồng hành vi có thể kiểm chứng cục bộ: nếu thao tác ghi sự kiện gặp lỗi thì thao tác cập nhật dữ liệu không được phép tồn tại.

Giết process ở khe giữa hai lời hứa

Một yêu cầu cộng 7 đơn vị đã xác nhận thành công trên cơ sở dữ liệu, nhưng bên gọi chưa kịp nhận phản hồi thì tiến trình bị dừng đột ngột. Khi khởi động lại, dịch vụ có được phép cộng thêm 7 đơn vị nữa hay không? Nếu hệ thống còn phải phát đi một sự kiện thông báo, làm sao tiến trình mới biết được sự kiện ấy chưa từng được gửi? Hai câu hỏi này không có cùng lời giải. Cơ sở dữ liệu có thể ghi nhận thao tác đã hoàn tất trong khi nghĩa vụ phát tán thông điệp ra bên ngoài vẫn còn dang dở.

Thực hành tại thư mục `labs/part13-transaction-boundary/outbox` thu gọn bài toán vào ba thành phần: một giá trị số dư, một khóa thao tác (**operation key**) và một sự kiện tương ứng. Dữ liệu tải trọng (**payload**) là số nguyên `delta` có giá trị từ 1 đến 100; việc lặp lại cùng khóa và cùng giá trị gia số được coi là thao tác thử lại của cùng một lệnh, trong khi việc thay đổi giá trị gia số trên cùng khóa sẽ bị từ chối bằng lỗi `ErrConflict`. Đây là quy ước dữ liệu của bộ kiểm thử, không phải thuật toán chuẩn hóa JSON hay chính sách lưu trữ áp dụng cho mọi hệ thống. Hãy mở tập tin `TestOutboxContract` trước khi đọc `store.go`: yêu cầu của bài thực hành là sau khi xác nhận giao dịch, số dư chỉ được tăng đúng một lần và nghĩa vụ gửi sự kiện bắt buộc phải tồn tại bền vững.

Phiên bản lỗi `BrokenApply` ghi nhận số dư cùng bản ghi thao tác, xác nhận giao dịch, rồi mới cố gắng chèn sự kiện vào bảng hàng đợi (**outbox**). Tiến trình con kết thúc đột ngột bằng lệnh `os.Exit(77)` ngay sau bước xác nhận: các hàm dọn dẹp trì hoãn (**deferred cleanup**) hoàn toàn không có cơ hội thực thi. Khi mở cơ sở dữ liệu bằng kết nối mới, kiểm thử ghi nhận số dư đã tăng nhưng không hề tồn tại sự kiện nào ở trạng thái chờ gửi (**pending**). Thao tác gửi lại với cùng khóa thao tác cũng không thể khắc phục: bản ghi thao tác đã tồn tại từ trước khiến nhánh tạo sự kiện bị bỏ qua. Một tác vụ quét bộ nhớ khi khởi động lại ứng dụng không thể khôi phục một nghĩa vụ chưa từng được ghi xuống đĩa cứng.

Phiên bản khắc phục đưa toàn bộ ba thao tác ghi vào trong cùng một giao dịch: đặt chỗ trước cho khóa bằng ràng buộc duy nhất (**unique constraint**), cập nhật số dư, và lưu sự kiện ở trạng thái chờ gửi. Câu lệnh đầu tiên là thao

tác ghi trực tiếp; bài thực hành không đọc trước rồi mới nâng mức khóa trong hai giao dịch cạnh tranh nhau. Hai kết nối gửi cùng một khóa thao tác được kiểm thử độc lập: chỉ duy nhất một thao tác nghiệp vụ và một sự kiện tương ứng được phép tồn tại. Thao tác gửi trùng lặp sẽ đối chiếu lại dữ liệu tải trọng trước khi hoàn tất; hệ thống không mặc định coi một khóa hợp lệ là quyền được thay đổi dữ liệu đã được chấp nhận trước đó.

Bảng 12 — Trạng thái phải nhìn thấy khi mở lại database sau process death

Checkpoint chết	Dữ liệu nghiệp vụ	Nghĩa vụ gửi	Hành động khi khôi phục
Trước xác nhận giao dịch	Chưa cộng; không có bản ghi thao tác.	Không có sự kiện.	Gửi lại cùng khóa để thực hiện.
Sau xác nhận, trước phản hồi	Đã cộng đúng một lần.	Chờ xử lý (**pending**).	Thử lại không cộng thêm; bộ phân phối đọc bản ghi chờ.
Phía nhận đã xác nhận, trước khi đánh dấu hoàn tất	Đã cộng ở cả hai phía.	Phía gửi vẫn chờ xử lý.	Gửi lại cùng định danh; phía nhận khử trùng lặp.

Chạy bộ kiểm thử từ mô-đun Chương 13; mỗi lượt thực thi sử dụng tập tin tạm riêng biệt do kiểm thử quản lý, không tác động đến cơ sở dữ liệu của dự án:

```
go test -count=1 -v ./outbox
go test -race ./outbox
```

Kiểm thử `TestRecoveryWindows` là tên bộ kiểm thử các khoảng thời gian xảy ra lỗi (**failure windows**), không phải bài kiểm thử chỉ dành riêng cho hệ điều hành Windows. Kịch bản này kích hoạt một tiến trình con chạy đến điểm kiểm tra (**checkpoint**), yêu cầu mã thoát 77, sau đó mở lại tập tin cơ sở dữ liệu để xác minh trạng thái. Lượt điều phối phục hồi sự kiện cũng được khởi chạy trong một tiến trình mới độc lập. Trường hợp sau khi gửi dùng tập tin cơ sở dữ liệu riêng của phía nhận: không hề có giao dịch phân tán xuyên qua hai tập tin, nhưng phía nhận lưu định danh hộp thư đến (**inbox identity**) cùng tác dụng phụ tương ứng trong cùng một giao dịch cục bộ. Tình huống mất gói tin xác nhận (**acknowledgment**) dẫn đến việc gửi lặp hai lần nhưng

chỉ tạo ra một hiệu ứng duy nhất; đó là hợp đồng lũy đẳng của phía nhận, không phải cơ chế chuyển phát chính xác một lần (**exactly-once delivery**).

Bộ điều phối (**dispatcher**) trong bài thực hành chỉ có một bên sở hữu duy nhất, kể cả giữa các tiến trình khác nhau. Thành phần này duy trì sự kiện ở trạng thái chờ gửi cho đến khi bộ truyền tin báo thành công rồi mới đánh dấu đã hoàn tất. Nhờ đó, hệ thống không gặp tình trạng cờ xử lý bị treo kẹt sau khi tiến trình dừng đột ngột; đổi lại, cơ chế chưa hỗ trợ thuê bao tài nguyên (**lease**) để nhiều bộ điều phối cùng chia sẻ khối lượng công việc. Việc thử lại sau các lỗi không rõ kết quả có thể dẫn đến việc phát lại sự kiện. Nếu phía nhận không có cơ chế khử trùng lặp bền vững gắn nguyên tử với tác dụng phụ, hoặc xóa định danh quá sớm, mô hình hộp thư gửi đi không thể ngăn chặn hoàn toàn hiệu ứng lặp lại. Khi tích hợp dịch vụ bên ngoài, kỹ sư cần tìm hiểu kỹ hợp đồng lũy đẳng và thời gian lưu giữ khóa của nhà cung cấp dịch vụ, thay vì giả định dựa trên hành vi cục bộ của SQLite.

> **Dừng để dự đoán:** Đặt `RELIABILITY_MUTANT=split_commit` rồi chạy chỉ `TestOutboxContract`; ca kiểm thử sẽ báo đỏ với số dư 7 nhưng pending 0. Bỏ biến môi trường ấy và chạy lại. Vì sao đổi thứ tự thành "đánh dấu completed rồi send" chỉ chuyển từ nguy cơ gửi lặp sang nguy cơ mất event?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Tiến trình (**process**) có thể dừng đột ngột sau `completed` nhưng trước khi kịp gọi `send`, khiến lần khởi động lại không còn nhận diện được tác vụ cần hoàn tất. Giữ trạng thái `pending` cho tới khi nhận xác nhận (**acknowledgment**) sẽ đảm bảo nghĩa vụ gửi có thể phục hồi, nhưng không khép kín được khoảng trống sau `send`. Tính nguyên tử cục bộ chỉ bảo vệ tính nhất quán của dữ liệu và nghĩa vụ; chính định danh tại phía nhận (**receiver identity**) mới triệt tiêu nguy cơ trùng lặp khi tác vụ được gửi lại.

Bài thực hành cố định phiên bản trình điều khiển `modernc.org/sqlite v1.59.0`, thiết lập nhật ký hoàn nguyên dạng `DELETE`, cờ đồng bộ `synchronous=FULL` và thời gian chờ bận 5 giây trên mỗi kết nối. Bằng

chứng thực nghiệm đã kiểm chứng là kịch bản tiến trình dừng đột ngột tại điểm kiểm tra, không bao gồm các sự cố mất điện nguồn, hỏng hóc đĩa vật lý hay lỗi hệ thống tệp mạng. Các giả định về cơ chế khóa và đồng bộ dữ liệu xuống đĩa của SQLite vẫn phải được thỏa mãn; một lệnh thoát tiến trình `os.Exit` không thể chứng minh toàn bộ độ bền vững của toàn bộ ngăn xếp lưu trữ.

Tiến hóa lược đồ dữ liệu (Schema Migration) như một trạng thái bền vững

Cấu trúc bảng và các ràng buộc dữ liệu (schema) cũng là trạng thái bền vững. Mã nguồn ứng dụng có thể được rollback về một commit Git cũ chỉ trong vài giây, nhưng dữ liệu trên đĩa cứng đã bị thay đổi thì không thể đảo ngược một cách tự động. Khi dịch vụ bước vào môi trường sản xuất có dữ liệu người dùng thật, việc gọi các câu lệnh tạo bảng tùy tiện trong mã khởi động không còn là giải pháp khả thi. Quản lý schema đòi hỏi sáu nguyên tắc kỷ luật:

Một là, thứ tự di chuyển có phiên bản (**versioned migration ordering**). Mọi thay đổi lược đồ dữ liệu phải được biểu diễn thành các bước tuần tự có đánh số phiên bản hoặc nhãn thời gian tăng dần, được lưu trữ trong một bảng quản lý (như `schema_migrations`). Mỗi bước chỉ được thực thi đúng một lần và theo đúng thứ tự đã định.

Hai là, quyền sở hữu di chuyển lược đồ (**migration ownership**). Một phương án triển khai phổ biến là giao quyền hạn thực thi cho một quy trình phát hành chuyên trách (như Kubernetes Job hoặc bước chạy đường ống CI/CD trước khi cập nhật khối lượng công việc). Tuyệt đối không để hàng chục thể hiện của ứng dụng cùng khởi động và cùng tranh chấp chạy các câu lệnh DDL mà không có cơ chế điều phối, vì điều này có thể dẫn đến xung đột khóa và gây dừng hoạt động hệ thống.

Ba là, tính tương thích khi triển khai (**deployment compatibility**). Trong các chiến lược triển khai không gián đoạn (như cập nhật cuốn chiếu **rolling update** hay triển khai song song **blue-green**), phiên bản mã nguồn cũ và phiên bản mã nguồn mới sẽ cùng chạy đồng thời trong một khoảng thời

gian. Một chiến lược thường dùng là Mở rộng - Di chuyển - Thu hẹp (**Expand - Migrate - Contract**): khi cần thêm một cột mới, cột đó cho phép giá trị NULL hoặc có giá trị mặc định để mã nguồn cũ chưa biết đến cột này vẫn hoạt động bình thường; sau khi toàn bộ thể hiện mới đã vận hành ổn định, mới tiến hành bước dọn dẹp các trường lỗi thời.

Bốn là, chiến lược xử lý sự cố khi nâng cấp dữ liệu. Phương án sửa tiến (**forward-fix**) thường được ưu tiên trong thực tế vận hành để tránh nguy cơ mất mát dữ liệu mới phát sinh trong giai đoạn chuyển tiếp nếu chạy tập lệnh đảo ngược (**down migration**). Tuy nhiên, lựa chọn giữa sửa tiến và hoàn nguyên (**rollback**) phụ thuộc vào mô hình lỗi, tính toàn vẹn của dữ liệu và chính sách vận hành của từng hệ thống.

Năm là, khả năng đặt các câu lệnh định nghĩa dữ liệu (**DDL**) trong giao dịch phụ thuộc vào hệ quản trị và phiên bản. PostgreSQL cho phép nhiều thao tác như `CREATE TABLE`, `ALTER TABLE` nằm trong giao dịch, nhưng vẫn phải xét khóa và ngoại lệ như `CREATE INDEX CONCURRENTLY`. SQLite cho phép giao dịch chứa DDL; lab kiểm chứng `ALTER TABLE ADD COLUMN`. MySQL 8.0 có DDL nguyên tử cho những statement được hỗ trợ; với thao tác trên bảng, khả năng này còn phụ thuộc storage engine, cụ thể là InnoDB. DDL nguyên tử từng statement không phải giao dịch DDL do ứng dụng quản lý: phần lớn DDL gây xác nhận ngầm định (**implicit commit**), nên không thể gom tùy ý với những thay đổi khác rồi hoàn nguyên chung. `CREATE TEMPORARY TABLE` là ngoại lệ không gây implicit commit, nhưng bảng được tạo vẫn không bị xóa bởi `ROLLBACK`. Phải đọc contract của đúng statement, không áp một quy tắc chung cho mọi DDL.

Sáu là, kiểm thử tiến hóa lược đồ dữ liệu (**schema evolution**). Bài thực hành của chương chứng minh cơ chế này qua kiểm thử `TestSchemaMigrationOrderingAndRollback`. Việc nâng cấp lược đồ từ phiên bản 1 lên phiên bản 2 được thực thi trong giao dịch nguyên tử. Kiểm thử chủ động tạo kịch bản di chuyển lỗi (thêm cột `failed_col` nhưng hoàn nguyên giao dịch trước khi hoàn tất ghi nhận phiên bản) và kiểm tra qua `PRAGMA table_info` cùng bảng `schema_migrations` để xác nhận

cột hủy không tồn tại và số hiệu phiên bản giữ nguyên ở mức 1; sau đó mới thực thi bước nâng cấp v2 hợp lệ và xác nhận thành công.

Ranh giới bộ nhớ đệm (Cache Consistency) và nguồn chân lý

Khi lưu lượng đọc tăng cao, việc đưa thêm một lớp bộ nhớ đệm (**cache**) vào kiến trúc là giải pháp phổ biến để giảm tải cho cơ sở dữ liệu. Tuy nhiên, bộ nhớ đệm là một ranh giới hoàn toàn tách biệt khỏi cơ sở dữ liệu. Trong thiết kế của chương này, cơ sở dữ liệu là nguồn chân lý duy nhất (**single source of truth**).

Mô hình đệm thông dụng nhất là đệm bên cạnh (**Cache-Aside**): Khi đọc, ứng dụng kiểm tra bộ nhớ đệm trước; nếu không có dữ liệu (**cache miss**), nó đọc từ cơ sở dữ liệu rồi lưu ngược lại vào bộ nhớ đệm. Khi cập nhật trạng thái: ứng dụng ghi thay đổi vào cơ sở dữ liệu trước, rồi mới tiến hành xóa hoặc vô hiệu hóa (**invalidate**) bản ghi tương ứng trong bộ nhớ đệm.

Vấn đề cốt lõi của tính nhất quán nằm ở ranh giới thất bại: nếu thao tác cập nhật cơ sở dữ liệu thành công nhưng lời gọi mạng xóa bộ nhớ đệm phía sau gặp sự cố (đứt kết nối mạng, quá hạn thời gian chờ, hoặc máy chủ đệm gặp sự cố dừng đột ngột), cơ sở dữ liệu đã mang trạng thái mới nhưng bộ nhớ đệm vẫn giữ nguyên giá trị cũ. Lượt đọc tiếp theo từ một bên gọi khác sẽ lấy dữ liệu lỗi thời từ bộ nhớ đệm (**stale read**), tạo ra tình huống hệ thống kể hai câu chuyện mâu thuẫn nhau.

Nguyên nhân căn bản là vì giao dịch cơ sở dữ liệu không thể bảo đảm tính nguyên tử cho một hệ thống bộ nhớ đệm bên ngoài. Giao dịch của gói `database/sql` chỉ có phạm vi hiệu lực bên trong công cụ lưu trữ (**storage engine**) của cơ sở dữ liệu; nó hoàn toàn không có khả năng hoàn nguyên một gói tin mạng đã gửi sang Redis hay Memcached.

Cơ chế hết hạn theo thời gian (**TTL - Time-To-Live**) chỉ là một biện pháp giới hạn khoảng thời gian tối đa mà dữ liệu lỗi thời có thể tồn tại nếu các giả định về chính sách làm mới dữ liệu được thỏa mãn; TTL không phải một cam kết đúng đắn tuyệt đối. Kỹ sư phải luôn phân định rõ: với những dữ liệu cho phép tính nhất quán cuối cùng (**eventual consistency** như số lượt xem bài viết, thông tin hiển thị hồ sơ), sự chênh lệch ngắn hạn giữa bộ nhớ đệm và

cơ sở dữ liệu là chấp nhận được; nhưng với các bất biến nghiệp vụ quan trọng, giải pháp phụ thuộc vào yêu cầu của hệ thống: đọc trực tiếp cơ sở dữ liệu bỏ qua đệm (**bypass cache**), sử dụng khóa có phiên bản, mô hình ghi xuyên (**write-through**) hoặc ghi sau (**write-behind**) với hợp đồng phù hợp, vô hiệu hóa qua sự kiện, hoặc cơ chế kiểm tra phiên bản dữ liệu.

Kiểm thử `TestCacheAsideStaleReadOnInvalidationFailure` trong bài thực hành cố ý tái lập kịch bản này: cơ sở dữ liệu đã cập nhật thành công cờ `enabled = false`, nhưng thao tác xóa bộ nhớ đệm gặp lỗi khiến đệm vẫn trả về giá trị `true`. Đây là bằng chứng thực nghiệm rõ ràng nhất để người học không nhầm lẫn giữa tính nguyên tử của giao dịch cục bộ với tính nhất quán của toàn bộ hệ thống.

Biểu diễn tuần tự hóa và tương thích tiến hóa dữ liệu

Như đã phân tích ở Chương 07, việc tuần tự hóa dữ liệu (serialization) biến các cấu trúc dữ liệu trong bộ nhớ thành dòng byte để truyền qua mạng hoặc ghi xuống đĩa. Trong tầng lưu trữ, việc lưu các đối tượng phức tạp dưới dạng chuỗi JSON hoặc mảng byte nhị phân (BLOB) vào một cột của cơ sở dữ liệu là một hợp đồng biểu diễn giữa người ghi (writer) và người đọc (reader) qua các mốc thời gian khác nhau.

Tính tương thích của dữ liệu đã lưu trữ phụ thuộc vào định dạng tuần tự hóa, lược đồ phiên bản, chiến lược di chuyển dữ liệu, ngữ nghĩa giá trị mặc định, khả năng phân biệt giữa việc vắng mặt trường dữ liệu với giá trị zero tường minh, thời gian lưu giữ (retention period), và khả năng viết lại dữ liệu lịch sử nếu cần thiết. Các thẻ tag như `omitempty` (hoặc `omitzero` từ Go 1.24) là các tùy chọn định dạng tuần tự hóa, không phải cơ chế tương thích phổ quát cho mọi trường hợp tiến hóa dữ liệu. Đối chiếu Chương 07 để nắm vững cách thức xử lý dòng byte và ánh xạ dữ liệu.

Điểm dừng: Giữ vững ranh giới trạng thái bền vững

Điểm dừng của chương không phải là việc thuộc lòng các câu lệnh SQL hay cấu hình máy chủ. Đó là thói quen phân định ranh giới trạng thái một cách có kỷ luật:

Một là, gom các thay đổi cùng ý nghĩa nghiệp vụ vào một transaction; phân biệt kết cục nguyên tử với bằng chứng bên gọi nhận được và mức cô lập của các lần đọc.

Hai là, quản lý kết nối như một tài nguyên dùng chung có giới hạn qua connection pool, đo lường bằng `DB.Stats()` thay vì suy đoán thông số.

Ba là, tận dụng prepared statement cho các thao tác lặp lại theo lô, đồng thời giải phóng tài nguyên với `Close()` và bảo vệ câu lệnh bằng parameter binding.

Bốn là, coi schema như một trạng thái bền vững cần tiến hóa có phiên bản, kiểm soát quyền sở hữu và bảo đảm tương thích hai chiều khi triển khai.

Năm là, nhận thức rõ ranh giới giữa cơ sở dữ liệu và bộ nhớ đệm, luôn xác định nguồn chân lý và chấp nhận đánh đổi tính nhất quán dựa trên yêu cầu bất biến của nghiệp vụ.

Khi các ranh giới này được phân định rõ ràng, hệ thống giảm thiểu các trạng thái dữ liệu mơ hồ, cho phép kỹ sư xác lập và kiểm chứng được các ranh giới thất bại từ mã nguồn ứng dụng đến tầng lưu trữ bền vững.

Chương 14 — Khi kiểu trở thành dữ liệu

Một hàm nhận `any` có thể tiếp nhận nhiều kiểu cụ thể, nhưng chữ ký ấy không cho phần thân hàm biết đích có phải con trỏ tới cấu trúc cấu hình hay không. Trình biên dịch vẫn kiểm tra biểu thức ở nơi gọi; điều còn thiếu là hợp đồng hẹp hơn bên trong hàm. Đoạn mã nào sẽ kiểm tra hình thái của giá trị, và đoạn mã nào được trao quyền thay đổi giá trị đó?

Hãy xét một bộ tải cấu hình nhỏ. Ứng dụng `opsprobe` có thể tiếp nhận giá trị từ các biến môi trường, nhưng mô-đun đọc môi trường không nên phụ thuộc hay biết trước mọi cấu trúc cấu hình của từng lệnh con sau này. Bộ tải tiếp nhận một đích đến (**destination**) cùng một bảng ánh xạ các giá trị đã đọc; đối tượng đích quyết định trường dữ liệu nào được phép nhận giá trị. Nếu bên gọi truyền `Config{}` theo dạng giá trị thay vì con trỏ `&Config{}`, bộ tải không được phép âm thầm tạo bản sao rồi khiến bên gọi ngộ nhận cấu hình đã được áp dụng. Nếu một trường có gắn thẻ dữ liệu (**struct tag**) nhưng kiểu dữ liệu không phải `string`, bộ tải không được tùy tiện suy đoán cách phân tích cú pháp.

Mô hình tư duy của chương là: cơ chế soi chiếu (**reflection**) biến kiểu dữ liệu và giá trị thành các đối tượng dữ liệu có thể khảo sát tại thời gian chạy (**runtime**); cơ chế này không hề xóa bỏ hợp đồng hành vi, mà chuyển giao một phần việc thực thi hợp đồng từ trình biên dịch sang các khối kiểm tra phòng vệ (**guard clauses**) tường minh trong mã nguồn. Gói `unsafe` nằm ở rìa của mô hình này: nó vượt qua các chốt chặn an toàn của hệ thống kiểu dữ liệu, vì vậy chỉ có chỗ đứng khi biểu diễn bộ nhớ và vòng đời dữ liệu đã được chứng minh chặt chẽ tại một ranh giới cục bộ hẹp.

Một value có shape mà compiler chưa biết

Trong lời gọi hàm tĩnh thông thường, trình biên dịch nhận biết rõ `config.Endpoint` mang kiểu `string`; thao tác gán `config.Endpoint = raw` được kiểm tra tính hợp lệ trước khi chương trình khởi chạy. Tuy nhiên với hàm `func ApplyEnv(dst any, values map[string]string) error`, trình biên dịch chỉ biết đối số `dst` là một giá trị giao diện (**interface value**). Kiểu động (**dynamic type**) và giá trị động (**dynamic value**) đi cùng `dst` tới thời gian chạy, và hàm `reflect.ValueOf` cung cấp một góc nhìn phản chiếu để ta truy vấn thông tin của chúng.

```
config := Config{}

value := reflect.ValueOf(config)
fmt.Println(value.Kind())   // struct
fmt.Println(value.CanSet()) // false

pointer := reflect.ValueOf(&config)
target := pointer.Elem()
fmt.Println(target.Kind())   // struct
fmt.Println(target.CanSet()) // true
```

Trong mã nguồn của Go 1.27.1, cấu trúc `reflect.Value` hiện chứa các trường nội bộ sau. Chúng chỉ nhằm làm sáng tỏ một quan sát cơ chế của chuỗi công cụ; hợp đồng mà mã ứng dụng được phép dựa vào là các phương thức đã công bố trong tài liệu chuẩn như `CanSet`, `Kind` và `Elem`, tuyệt đối không phải tên trường hay các cờ bit nội bộ:

```
type Value struct {
    typ_ *abi.Type
    ptr unsafe.Pointer
    flag
}
```

Cờ nội bộ hiện lưu giữ thông tin về danh mục loại kiểu `Kind`, trạng thái chỉ đọc và khả năng định vị địa chỉ bộ nhớ (**addressability**). Ở tầng giao diện lập trình, điều cần ghi nhớ rất rõ ràng: hàm `reflect.ValueOf(config)` tiếp nhận một bản sao giá trị nên không cho phép ghi đè ngược lại vào biến của bên gọi; phương thức `CanSet()` vì thế trả về `false`.

Ngược lại, biểu thức `reflect.ValueOf(&config).Elem()` quan sát biến gốc thông qua con trỏ. Đối với trường cấu trúc, phương thức `CanSet()` còn phụ thuộc vào việc trường đó có thể chỉnh sửa theo quy tắc hiển thị công khai (**exported**) hay không. Đây là chốt chặn phòng vệ cấp giao diện cần sử dụng thay vì suy diễn từ các cờ bit nội bộ.

`CanAddr` và `CanSet` trả lời hai câu hỏi khác nhau. Một trường không công khai lấy từ `reflect.ValueOf(&config).Elem()` có thể có địa chỉ nhưng vẫn không được gán hoặc lấy ra qua `Interface`. Ngược lại, việc sao chép giá trị vào interface không sao chép sâu dữ liệu mà giá trị ấy tham chiếu: `reflect.ValueOf(slice).CanSet()` là `false`, nhưng phần tử của slice vẫn có thể được gán qua `Index`. Trực giác đúng là xét quyền của từng `Value`, không suy rằng mọi dữ liệu nằm sau `any` đều là bản sao độc lập.

Lời gọi `ApplyEnv(nil, nil)` còn tiềm ẩn một cạm bẫy khác. Biểu thức `reflect.ValueOf(nil)` trả về giá trị rỗng của `reflect.Value`, có loại kiểu `Kind()` là `Invalid`; nhiều thao tác tiếp theo trên một giá trị không hợp lệ sẽ gây sập tiến trình (`panic`). Do đó, ranh giới xử lý bắt buộc phải kiểm tra phương thức `IsValid()` trước khi thực hiện các phép toán phụ thuộc hình thái. Đây là chốt chặn phòng vệ cho một giá trị thời gian chạy thực sự không tồn tại, không phải trường hợp con trỏ `nil` thông thường.

Cơ chế thực thi	Đặc tính điều phối lời gọi	Xu hướng đóng gói và cấp phát	Khả năng Compiler Tối ưu
Lời gọi hàm hoặc phương thức tĩnh	Compiler biết target từ type tĩnh	Không cần cơ chế reflection riêng	Có thể inline hay tối ưu thêm tùy toolchain.
Lời gọi qua interface	Target phụ thuộc dynamic type	Representation và allocation là quyết định triển khai	Có thể được devirtualize khi compiler chứng minh được điều kiện; không có bảo đảm.
Lời gọi qua reflection (`Value.Call`)	Metadata và kiểm tra động là một phần thao tác	API nhận/trả `reflect.Value`	Thường đắt hơn đường tĩnh; nếu chi phí quan trọng, phải benchmark contract cụ thể.

Bảng 13 — Compiler và reflection chia việc khác nhau

Câu hỏi	Code có type tĩnh	Code reflection có `any`
Value có phải `*Config` không?	Compiler đã biết ở call site	Kiểm tra pointer, `nil` và `Elem().Kind()` khi chạy.
Field có đúng builtin `string` không?	Compiler kiểm tra assignment	So exact `StructField.Type` với `reflect.TypeFor[string]()`.
Có quyền ghi field không?	Quy tắc visibility và addressability của Go	`CanSet` và exported field vẫn giữ ranh giới đó.
Sai thì xảy ra khi nào?	Thường compile error	Phải là `error` có context, không phải panic do đoán shape.

Reflection vì thế không thể thay thế cho kiểu chung (**generics**). Nếu một hàm chỉ cần thực thi một thao tác trên tập hợp các kiểu dữ liệu đã xác định từ trước, tham số kiểu (**type parameter**) sẽ giúp trình biên dịch duy trì hợp đồng tĩnh một cách hiệu quả hơn. Cơ chế soi chiếu chỉ thực sự hợp lý khi thao tác phụ thuộc hoàn toàn vào siêu dữ liệu (**metadata**) tại thời gian chạy: các bộ tuần tự hóa (**serializer**), giải mã (**decoder**), công cụ quản lý lược đồ dữ liệu, ranh giới phần mở rộng (**plugin**), hoặc bộ nạp đọc thẻ cấu trúc. Chính gói `encoding/json` là minh chứng quen thuộc về giao diện lập trình tiếp nhận giá trị có hình thái do thời gian chạy quyết định; tuy nhiên điều đó không đồng nghĩa với việc mọi tiện ích chuyển đổi trong ứng dụng đều nên tự viết bằng reflection.

Interface, generics, reflection và unsafe giải quyết bốn bài toán khác nhau

Giao diện (**interface**) là cơ chế trừu tượng hóa theo hành vi: kiểu cụ thể (**concrete type**) có thể thay đổi linh hoạt lúc chạy, nhưng trình biên dịch vẫn kiểm tra chặt chẽ hợp đồng phương thức. Tham số kiểu (**generics**) là cơ chế trừu tượng hóa tại thời điểm biên dịch dựa trên tập hợp kiểu có ràng buộc (**constraints**); trình biên dịch vẫn nắm bắt đầy đủ thông tin kiểu cần thiết để tối ưu hóa. Cơ chế soi chiếu (**reflection**) khảo sát hoặc thao tác trên `Type` và `Value` tại thời gian chạy khi siêu dữ liệu hay hình thái động là đầu vào. Gói `unsafe` là lối thoát hiểm vượt ra ngoài phạm vi an toàn kiểu thông thường; công cụ này chỉ có chỗ đứng khi cách thức biểu diễn bộ nhớ và vòng đời dữ liệu đã có bằng chứng xác minh cụ thể. Bốn công cụ có thể cùng hiện diện trong một dự án, nhưng không phải là bốn phương án có thể hoán đổi tùy tiện cho nhau.

Đây chỉ là cầu nối nhằm định vị cơ chế soi chiếu vào đúng ranh giới trách nhiệm, chưa phải trọng tâm phân tích về tham số kiểu. Chương 19 sẽ bắt đầu lại từ một bài toán lặp cụ thể, sau đó xây dựng tham số kiểu, ràng buộc kiểu và phương thức bảo toàn thông tin kiểu mà không biến mọi giao diện lập trình thành `any`. Khi đó, câu hỏi không còn là việc soi chiếu có linh hoạt hơn hay không, mà là hợp đồng nào trình biên dịch vẫn có thể bảo đảm giúp ta.

Struct tag là schema nhỏ, không phải comment bí mật

Thẻ cấu trúc (**struct tag**) là siêu dữ liệu gắn liền với phần khai báo trường dữ liệu. Trình biên dịch lưu trữ thông tin này trong mô tả kiểu; phương thức `reflect.StructField.Tag.Lookup` có thể đọc giá trị thẻ tại thời gian chạy. Thẻ dữ liệu không tự phân tích biến môi trường hay kiểm tra tính hợp lệ của miền nghiệp vụ. Nó chỉ cung cấp chỉ dẫn cho bộ tải biết trường nào tham gia vào lược đồ cấu hình này.

```
type Config struct {
    Endpoint string `env:"OPS_PROBE_ENDPOINT"`
    Token    string `env:"OPS_PROBE_TOKEN"`
    Ignored  string `env:"-"`
}
```

Hai trường công khai (**exported**) được lựa chọn vì giao diện của bộ nạp chỉ được phép ghi vào phần trạng thái mà kiểu dữ liệu đã chủ động công khai. Giá trị thẻ `env:"-"` là cơ chế từ chối tham gia có chủ đích. Ngược lại, một trường nội bộ (**unexported**) lại khai báo một thẻ `env` thực sự sẽ bị xem là lỗi lược đồ: kiểu dữ liệu sở hữu vừa yêu cầu bộ nạp tham gia lược đồ, lại vừa cấm quyền ghi dữ liệu. Bộ nạp bắt buộc phải trả về lỗi tường minh thay vì âm thầm bỏ qua hoặc lạm dụng `unsafe` để biến trạng thái riêng tư thành cửa sau trái phép.

Đây là mã triển khai tham chiếu cho một ranh giới xử lý thu gọn. Hàm chỉ tiếp nhận dữ liệu chuỗi `string`; việc phân tích cổng mạng (**port**), khoảng thời gian (**duration**), địa chỉ URL hay chính sách bảo mật khóa bí mật thuộc về các hợp đồng chuyên biệt khác, không bị giấu sau một hàm tiện ích tự làm mọi việc.

```
var (
    ErrDestination = errors.New(
        "destination must be a non-nil pointer to struct",
    )
    ErrSchema      = errors.New("invalid env schema")
)

type assignment struct {
    field reflect.Value
    raw   string
}

func schemaError(field, rule string) error {
    return fmt.Errorf(
        "%w: env field %s %s",
        ErrSchema,
        field,
        rule,
    )
}
```

`ApplyEnv` đọc lược đồ và thu thập phép gán trước, sau đó mới ghi vào đích. Trường hợp lệ đứng trước trường sai vì thế không bị cập nhật nếu hàm trả lỗi lược đồ. Cam kết này chỉ dành cho lỗi mà helper kiểm tra; các lệnh gán vẫn diễn ra tuần tự, không phải giao dịch nguyên tử đối với người đọc đồng thời.

Bên gọi phải sở hữu đích và giữ map đầu vào ổn định trong lượt gọi, hoặc thiết kế đồng bộ riêng theo Ch08. Reflection không tự thêm khóa hay rollback khi chương trình bị dừng.

```
func ApplyEnv(dst any, values map[string]string) error {
    value := reflect.ValueOf(dst)
    if !value.IsValid() ||
        value.Kind() != reflect.Pointer ||
        value.IsNil() {
        return ErrDestination
    }
    target := value.Elem()
    if target.Kind() != reflect.Struct {
        return ErrDestination
    }
    typ := target.Type()
    stringType := reflect.TypeFor[string]()
    assignments := make([]assignment, 0, typ.NumField())
    for i := 0; i < typ.NumField(); i++ {
        field := typ.Field(i)
        name, tagged := field.Tag.Lookup("env")
        if !tagged || name == "-" {
            continue
        }
        if !field.IsExported() {
```

```
            return schemaError(field.Name, "is unexported")
        }
        if field.Type != stringType {
            return schemaError(field.Name, "must be string")
        }
        fieldValue := target.Field(i)
        if !fieldValue.CanSet() {
            return schemaError(field.Name, "cannot be set")
        }
        if raw, found := values[name]; found {
            assignments = append(assignments, assignment{
                field: fieldValue,
                raw:   raw,
            })
        }
    }
    for _, assignment := range assignments {
        assignment.field.SetString(assignment.raw)
    }
    return nil
}
```

Đối tượng `Type` giải đáp câu hỏi về phần khai báo: trường thứ `i` có tên là gì, mang thẻ dữ liệu nào, có được công khai hay không, và thuộc chính xác kiểu dữ liệu nào. Phương thức `Kind` chỉ phản ánh danh mục loại kiểu nền tảng tại thời gian chạy; vì một kiểu định danh như `type Token string` vẫn có `Kind() == reflect.String`, nó không đủ chặt chẽ cho một hợp đồng chỉ chấp nhận kiểu nguyên thủy `string`. Lời gọi `TypeFor[string]()` cung cấp định danh chính xác của kiểu dữ liệu cần tiếp nhận. Đối tượng `Value` giải đáp câu hỏi về thể hiện thực tế: giá trị hiện tại là gì và có quyền gán giá trị hay không. Quy tắc nhất quán là: đọc siêu dữ liệu từ `Type`, và đọc hoặc biến đổi dữ liệu thực thông qua `Value`.

Helper kiểm tra mọi trường có thẻ tham gia, kể cả khi map không chứa giá trị cho trường ấy. Nó chỉ duyệt trường trực tiếp, không tự đi sâu vào cấu trúc nhúng. Alias như `type Token = string` vẫn là chính kiểu `string` và được chấp nhận, khác với định nghĩa `type Token string`. Các thẻ phải có định dạng chuẩn: `StructTag.Lookup` không là bộ kiểm tra cú pháp, và tài liệu không quy định kết quả cho thẻ sai định dạng. Chạy `go vet` để tìm

lỗi thẻ khai báo; đừng đọc cam kết "lỗi lược đồ" của helper thành cam kết phát hiện mọi lỗi văn bản trong tag.

> **Dừng để dự đoán:** Nếu caller gọi `cfg := Config{}` rồi thực hiện `ApplyEnv(cfg, values)` bằng giá trị thay vì truyền con trỏ `&cfg`, liệu hàm có thể cập nhật các trường cấu hình không? Vì sao implementation của `ApplyEnv` chặn lỗi ngay từ guard kiểm tra con trỏ ở đầu hàm thay vì duyệt qua từng field rồi kiểm tra `CanSet()`?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Khi truyền struct bằng giá trị vào đối số interface `any`, giá trị được đóng gói thành một bản sao không có địa chỉ liên kết với biến gốc (unaddressable). Đối tượng `reflect.ValueOf(dst)` sinh ra từ một giá trị như vậy đại diện cho một bản sao độc lập, và phương thức `CanSet()` trên các trường của nó luôn trả về `false`. Nếu cố tình gọi `SetString()` lên một `reflect.Value` không có quyền gán, runtime sẽ lập tức gây `panic`. Kể cả khi hàm cố tình bỏ qua mà không `panic`, biến `cfg` ban đầu của caller cũng hoàn toàn không nhận được dữ liệu mới, biến giá trị trả về `nil` thành một tín hiệu thành công giả tạo.

Trong cài đặt thực tế của `ApplyEnv`, ta không đợi đến lúc duyệt từng trường mới kiểm tra `CanSet()`. Guard clause ở ngay đầu hàm kiểm tra trực tiếp `value.Kind() != reflect.Pointer` cùng điều kiện con trỏ không `nil` và trả về `ErrDestination` ngay lập tức. Đây là ranh giới phòng vệ fail-fast: một khi đích đến không phải là con trỏ tham chiếu tới vùng nhớ có quyền đột biến, hàm từ chối dứt khoát toàn bộ thao tác trước khi tốn chi phí duyệt metadata của struct.

Bài thực hành: Viết kiểm tra bảo vệ trước khi gán giá trị (setter)

Thực hành tại `labs/part14-reflection-boundary` bắt đầu bằng kịch bản kiểm thử không đạt. Hãy mở `exercise/apply_test.go`, chỉ đọc định nghĩa kiểu `Config` và các câu lệnh khẳng định (**assertion**) trước. Hãy tự quyết định thứ tự các điều kiện phòng vệ, nhưng tuyệt đối không dùng `unsafe`, không phân tích thẻ dữ liệu bằng cách cắt chuỗi thủ công, và không gây dừng đột ngột (`panic`) khi dữ liệu đầu vào sai lệch hình thái.

```
cd labs/part14-reflection-boundary
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go test -race ./fixed
```

Bộ kiểm thử đòi hỏi xử lý chuẩn xác các ranh giới dễ gây nhầm lẫn: con trỏ tham chiếu tới cấu trúc được phép cập nhật; biến cấu trúc dạng giá trị, con trỏ mang kiểu nhưng có giá trị `nil`, và giá trị `nil` không định kiểu đều phải bị từ chối; thẻ mang giá trị `-` không được ghi; trường kiểu `int`, kiểu chuỗi có định danh riêng, hoặc trường nội bộ có gắn thẻ đều bị coi là lỗi lược đồ. Một lỗi lược đồ phát hiện sau các trường hợp lệ vẫn không được phép làm biến đổi cục bộ đối tượng đích. Yêu cầu đặt ra không đòi hỏi bộ nạp phải trở thành một khung cấu hình phức tạp.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Bắt đầu từ `reflect.ValueOf(dst)`, kiểm tra tuần tự các điều kiện `IsValid`, kiểm tra con trỏ `Pointer`, con trỏ rỗng `IsNil` và kiểm tra đối tượng trỏ tới là cấu trúc `Elem().Kind() == reflect.Struct`. Tiếp theo, duyệt qua từng khai báo trường, bỏ qua thẻ `-`, từ chối thẻ thực trên trường nội bộ, so sánh chính xác kiểu dữ liệu và kiểm tra quyền gán `CanSet`. Chỉ thu thập danh sách phép gán trong giai đoạn A; sang giai đoạn B mới áp dụng các phép gán đó. Nhờ vậy, lược đồ sai định dạng sẽ trở thành lỗi có thể xử lý ở bên gọi thay vì gây sập chương trình hoặc làm biến đổi dở dang dữ liệu.

`unsafe` không phải nút "mở khóa" field private

Khi cơ chế soi chiếu từ chối ghi dữ liệu vào trường `debug`, một số lập trình viên có thể tìm cách dùng `unsafe.Pointer` để vượt qua ranh giới bảo vệ. Tuyệt đối không làm như vậy. Việc trường dữ liệu không được công khai là một phần của hợp đồng giao diện và quyền sở hữu; mã nguồn generic không thể lường trước bất biến nào của trạng thái riêng tư (**private state**) sẽ bị phá vỡ khi bị gán dữ liệu tùy tiện.

Gói `unsafe` chỉ tồn tại cho một số ranh giới đặc thù: tương tác với vùng nhớ do hệ điều hành hoặc mã nguồn ngôn ngữ khác quản lý, các thành phần tầng thấp có biểu diễn bộ nhớ cố định, hoặc các thao tác cơ bản mà thư viện chuẩn cung cấp. Công cụ này cho phép các phép toán vượt ra ngoài ranh giới an toàn kiểu thông thường, nhưng không biến mọi hành vi thành bất định. Lập trình viên chỉ được phép dựa trên các cam kết mà đặc tả ngôn ngữ và tài liệu gói `unsafe` công bố: các hàm `unsafe.Sizeof`, `unsafe.Alignof` và `unsafe.Offsetof` đều có ngữ nghĩa được tài liệu hóa rõ ràng.

Cái giá phải trả cho việc dùng `unsafe` rất cao: các giả định về cách thức biểu diễn bộ nhớ có thể phụ thuộc vào nền tảng; kiến trúc phần cứng khác nhau có thể sở hữu cách sắp xếp bố cục (**layout**) bộ nhớ khác nhau; các quy tắc về vòng đời, bộ thu gom rác (**GC**) và con trỏ vẫn phải được chứng minh chặt chẽ. Một số đo kiểm nghiệm trên hệ điều hành Windows kiến trúc amd64 chỉ là kết quả đo lường trên mục tiêu đó, không phải giao diện nhị phân (**ABI**) phổ quát hay cam kết chung cho mọi bản dựng.

```
type Header struct {
    Flag byte
    ID   uint64
}

fmt.Println(unsafe.Sizeof(Header{}))
fmt.Println(unsafe.Alignof(Header{}))
fmt.Println(unsafe.Offsetof(Header{}.ID))
// Ghi cùng Go version, GOOS và GOARCH của lần đo.
```

Tránh suy diễn bố cục bộ nhớ của giao diện, phần đầu chuỗi (**string header**), bảng ánh xạ hay đối tượng thời gian chạy từ các kết quả in ra. Đó là các chi tiết cài đặt nội bộ và hoàn toàn có thể thay đổi qua các phiên bản Go. Bài thực hành thực thi thí nghiệm này qua kiểm thử tự động để mã nguồn được trình biên dịch kiểm tra; nhật ký kết quả luôn ghi rõ phiên bản Go, hệ điều hành `GOOS` và kiến trúc `GOARCH`, tuyệt đối không xem một số đo cục bộ là quy chuẩn phổ quát.

Code review: đừng giữ địa chỉ trong `uintptr`

Đoạn dưới là mã nguồn tiêu cực để phân tích, không phải kỹ thuật nên áp dụng trong thực tế. Nó trích xuất địa chỉ mảng byte nền tảng từ kiểu cũ `reflect.StringHeader` rồi trả về `uintptr` ra ngoài phạm vi chuỗi:

```
func badDataAddress(s string) uintptr {
    header := (*reflect.StringHeader)(unsafe.Pointer(&s))
    return header.Data
}
```

Trước khi chấp nhận đoạn mã này, hãy phân tích bản chất cơ chế của Go Runtime:

Thứ nhất, phân biệt rạch ròi giữa việc di dời ngăn xếp (**stack movement**) và thu gom rác trên vùng nhớ động (**heap GC**). Cơ chế chạy Go hiện tại sử dụng vùng nhớ heap không di dời và có thể sao chép ngăn xếp khi kích thước ngăn xếp goroutine tăng lên; đó là chi tiết cài đặt nội bộ, không phải sự cho phép để tùy tiện lưu giữ địa chỉ bộ nhớ. Một con trỏ `unsafe.Pointer` vẫn được bộ thu gom rác nhận diện là con trỏ hợp lệ nếu tuân thủ các mẫu quy chuẩn mà gói `unsafe` công bố; hãy chỉ dựa vào các phép chuyển đổi đã được tài liệu hóa.

Thứ hai, kiểu `uintptr` đơn thuần là một số nguyên đủ độ rộng để chứa các bit địa chỉ trên kiến trúc hiện hành, hoàn toàn không phải một tham chiếu sống. Nếu tách địa chỉ khỏi con trỏ `unsafe.Pointer` gốc và lưu vào `uintptr`, bộ thu gom rác không có cơ sở nào để nhận diện số nguyên đó đang giữ đối tượng sống; việc mở rộng ngăn xếp cũng không thể tự động điều chỉnh giá trị của một số nguyên đã sao chép. Vì vậy đối tượng có thể bị

thu hồi ngay khi vòng đời con trỏ gốc chấm dứt, và địa chỉ lưu trong biến số nguyên sẽ trở nên lỗi thời (**stale**).

Thứ ba, với pattern chuyển pointer qua `uintptr` để cộng offset rồi chuyển ngược, tài liệu `unsafe.Pointer` yêu cầu conversion và phép tính nằm trong cùng một biểu thức:

```
p = unsafe.Pointer(uintptr(p) + offset)
```

Mẫu chuyển đổi trong cùng một biểu thức còn đòi hỏi kết quả ở trong đối tượng đã cấp phát; không được tính con trỏ tới ngay sau phần tử cuối theo quy ước đôi khi dùng trong C. `runtime.KeepAlive` không hợp thức hóa việc tách địa chỉ qua biến trung gian `uintptr`. Quy tắc chuyển đổi dựa trên tài liệu `unsafe`; phần heap không di dời và sao chép stack được đối chiếu source Go1.27.1 Windows/amd64. Ví dụ sai ở trên dùng để đọc và phản biện, không chạy nó rồi coi việc chưa crash là bằng chứng an toàn.

Trong thư viện Go 1.27.1, các kiểu `reflect.StringHeader` và `reflect.SliceHeader` đã bị đánh dấu lỗi thời (**deprecated**); tài liệu chính thức cảnh báo rằng trường `Data uintptr` không đủ năng lực giữ dữ liệu sống và cách biểu diễn này không an toàn cũng như không mang tính khả chuyển (**portable**). Khi thực sự cần thao tác tại ranh giới tầng thấp, hãy tuân thủ hợp đồng của các hàm hỗ trợ trong gói `unsafe` thay vì tự ý định nghĩa cấu trúc đầu mục; các hàm hỗ trợ này vẫn yêu cầu tuân thủ nghiêm ngặt các quy tắc về vòng đời và quyền biến đổi dữ liệu:

```
// Trích xuất con trỏ byte nền tảng
ptr := unsafe.StringData(s)

// Tái tạo chuỗi từ con trỏ byte và độ dài
str := unsafe.String(ptr, len(s))

// Lấy con trỏ phần tử đầu của slice
slicePtr := unsafe.SliceData(buf)

// Tạo slice từ con trỏ và độ dài
slice := unsafe.Slice(slicePtr, len(buf))
```

Với chuỗi rỗng, `StringData` trả con trỏ không được quy định, có thể là `nil`; không được giải tham chiếu. `SliceData(nil)` trả `nil`, nhưng slice khác `nil` có capacity bằng 0 trả con trỏ khác `nil` tới địa chỉ không được quy định. Dùng độ dài 0 với `unsafe.String` hoặc `unsafe.Slice` không cấp quyền đọc byte ở con trỏ ấy. Ví dụ giữ `len(buf)` để không dựng một vùng nhìn vượt khỏi dữ liệu đã có; phép dựng slice này chia sẻ mảng nền, không sao chép. Kiểm thử `TestUnsafeHelpersEmptyAndSharedStorage` chỉ chạy các trường hợp hợp lệ đó, không thử đọc địa chỉ tùy ý.

Cần bảo đảm tuyệt đối hợp đồng về vòng đời và quyền sửa đổi dữ liệu: con trỏ Go hợp lệ được bộ thu gom rác chủ động theo dõi, khác hoàn toàn với một địa chỉ đã bị biến thành số nguyên vô hồn. Hàm `unsafe.String` không cho phép chỉnh sửa mảng byte nền tảng trong khi chuỗi ký tự vẫn đang được sử dụng. Hàm `runtime.KeepAlive` phát huy tác dụng tại các ranh giới như tài nguyên gắn bộ dọn dẹp (**finalizer**) mà hệ điều hành chỉ còn quản lý qua bộ mô tả tệp; nó không hợp thức hóa các phép chuyển đổi nằm ngoài quy chuẩn của `unsafe`, không khóa cố định vùng nhớ (**pinning**), và không thể khôi phục một địa chỉ số nguyên đã lỗi thời.

Biên giới với C là một hợp đồng khác

Cơ chế soi chiếu vẫn vận hành bên trong khuôn khổ hệ thống kiểu của Go. Khi một thư viện chỉ cung cấp giao diện nhị phân ngôn ngữ C (**C ABI**), công cụ `cgo` sẽ đưa chương trình vượt qua một ranh giới hoàn toàn khác biệt: quy cách biểu diễn dữ liệu, vòng đời, bộ cấp phát bộ nhớ (**allocator**) và chuỗi công cụ của cả hai phía đều đòi hỏi nghiên cứu cẩn trọng. Một chuỗi Go quản lý độ dài tường minh và có thể chứa byte `NUL`; trong khi hàm `strlen` của C lại dừng ngay tại byte `NUL` đầu tiên. Hai thao tác đo độ dài này sở hữu hai hợp đồng ngữ nghĩa hoàn toàn khác nhau.

Thực hành tại `labs/edition-contracts/interop` có một hàm nhỏ dùng `C.CString`, gọi `C.strlen`, rồi giải phóng vùng nhớ đã cấp phát bằng `C.free`. Kiểm thử yêu cầu chuỗi `"a\x00b"` phải cho kết quả là 1 ở phía C, khác biệt với hàm `len` của chuỗi Go. Phép thử này không phải lời khuyên chuyển bộ phân tích sang C: nó chỉ làm sáng tỏ sự khác biệt về mặt dữ liệu.

Chuỗi byte không có ký tự kết thúc `NUL`, chuỗi ký tự Unicode và chuỗi định dạng C đòi hỏi chính sách chuyển đổi rõ ràng; nhận định cảm tính rằng cùng là văn bản là chưa đủ.

Vùng nhớ do `C.CString` cấp phát bắt buộc phải được giải phóng qua bộ cấp phát của C; bộ thu gom rác của Go không tự động thu hồi vùng nhớ này. Đối với con trỏ Go truyền sang C, tài liệu `cgo` quy định chặt chẽ phạm vi bộ nhớ được phép trỏ tới, cơ chế ghim vùng nhớ (**pinning**) và khoảng thời gian C được phép giữ con trỏ. Tránh suy diễn rằng việc có một địa chỉ hợp lệ tại thời điểm gọi hàm cho phép phía C lưu trữ địa chỉ đó vô thời hạn. Kiểu `runtime.Pinner` và `runtime/cgo.Handle` giải quyết các bài toán ranh giới khác nhau; `Handle` là định danh an toàn để duy trì giá trị Go, không phải quyền cho phép C giải tham chiếu tùy tiện. Chỉ áp dụng các cơ chế này sau khi đã thiết kế kỹ lưỡng vòng đời đối tượng và nắm vững hợp đồng tương ứng.

Công cụ `cgo` còn làm thay đổi hợp đồng triển khai: quá trình biên dịch đòi hỏi trình biên dịch C phù hợp, tệp nhị phân có thể phụ thuộc vào các thư viện liên kết động của hệ điều hành, và việc biên dịch chéo (**cross-compilation**) cần thiết lập chuỗi công cụ đích phức tạp. Vùng nhớ cấp phát phía C không nằm trong hạn mức quản lý bộ nhớ của Go, và các sự cố sập chương trình tầng native có thể vượt ra khỏi luồng xử lý lỗi mà mã nguồn Go kiểm soát được. Không tồn tại một con số chi phí vượt ranh giới cố định: nếu ranh giới này nằm trên luồng xử lý cao điểm (**hot path**), kỹ sư phải đo đạc với đúng kích thước tải trọng, tần suất gọi và môi trường thực tế, đồng thời xem xét khả năng gộp nhiều thao tác vào một lần gọi duy nhất. Mã nguồn Go thuần túy thường đem lại sự tinh gọn cho quy trình đóng gói tự chứa; việc tích hợp C chỉ hợp lý khi cần tận dụng các giao diện lập trình, trình điều khiển hoặc thành phần đã được kiểm chứng mà chi phí viết lại tiềm ẩn nhiều rủi ro hơn chi phí tương tác.

Chương này không hướng dẫn kỹ thuật lách khỏi các quy tắc an toàn của Go. Nó trang bị tư duy thiết lập ranh giới có trách nhiệm khi kiểu dữ liệu chỉ xuất hiện tại thời gian chạy. Cơ chế soi chiếu có thể giúp mã nguồn xử lý linh hoạt mà vẫn an toàn nếu các hợp đồng về hình thái dữ liệu, siêu dữ liệu và quyền biến đổi được thể hiện tường minh. Khi không có đủ bằng chứng xác nhận tính an toàn, việc quay trở về hệ thống kiểu tĩnh, các giao diện nhỏ hoặc giao diện lập trình cụ thể luôn là giải pháp thiết kế bền vững hơn.

Chương 15 — Từ incident đến công cụ

Một sự cố vận hành (**incident**) thông thường không thiếu các câu lệnh để gõ. Điều còn thiếu là một bản ghi đáng tin cậy về những gì câu lệnh đó đã thực sự thực hiện. Một kỹ sư vận hành có thể chạy `curl`, `nslookup`, `git`, hoặc một công cụ nội bộ rồi quan sát thấy một dòng báo lỗi. Nhưng dòng lỗi ấy không cho biết chính xác câu lệnh nào đã được kích hoạt, tiến trình có được cấp đủ thời gian xử lý hay không, có xuất dữ liệu chẩn đoán ra luồng lỗi chuẩn `stderr` hay không, và tiến trình đã kết thúc với mã trạng thái thoát nào. Việc gõ lại cùng một dòng lệnh nhiều lần chỉ làm lịch sử điều tra thêm mờ mịt.

Chương này bắt đầu từ một yêu cầu nghiệp vụ khiêm tốn: một lệnh điều tra sự cố sẽ kích hoạt một công cụ chẩn đoán đã được chọn trước, chờ đợi tiến trình theo một thời hạn xử lý (**deadline**), rồi thu thập toàn bộ dữ liệu đầu ra và trạng thái kết thúc có thể giải trình. Đây chưa phải một tính năng đồ sộ cho công cụ `opsprobe`; trước hết ta dựng một mô hình tái lập tối giản độc lập nhằm nắm vững ranh giới quản lý tiến trình con (**process boundary**). Khi hợp đồng hành vi đã được minh định, `opsprobe` mới có thể tích hợp thành phần này như một khối nền tảng vững chắc.

Mô hình tư duy của chương là: bộ chạy câu lệnh là một tiến trình giám sát thu nhỏ của tiến trình con. Nó không chỉ đơn thuần kích hoạt một tệp nhị phân. Thành phần này tiếp nhận ý định đã được kiểm tra hợp lệ, quản lý thời hạn thực thi, chờ tiến trình kết thúc, lưu giữ dữ liệu đầu ra tách bạch giữa các luồng, phân loại kết quả thoát, và để bên gọi quyết định phương thức hiển thị cũng như lưu vết. Nếu thiếu sự kiểm soát chặt chẽ các yếu tố đó, công cụ chẩn đoán có thể biến một sự cố rõ ràng thành một chuỗi lệnh vỏ hệ điều hành rời rạc và không thể tái lập.

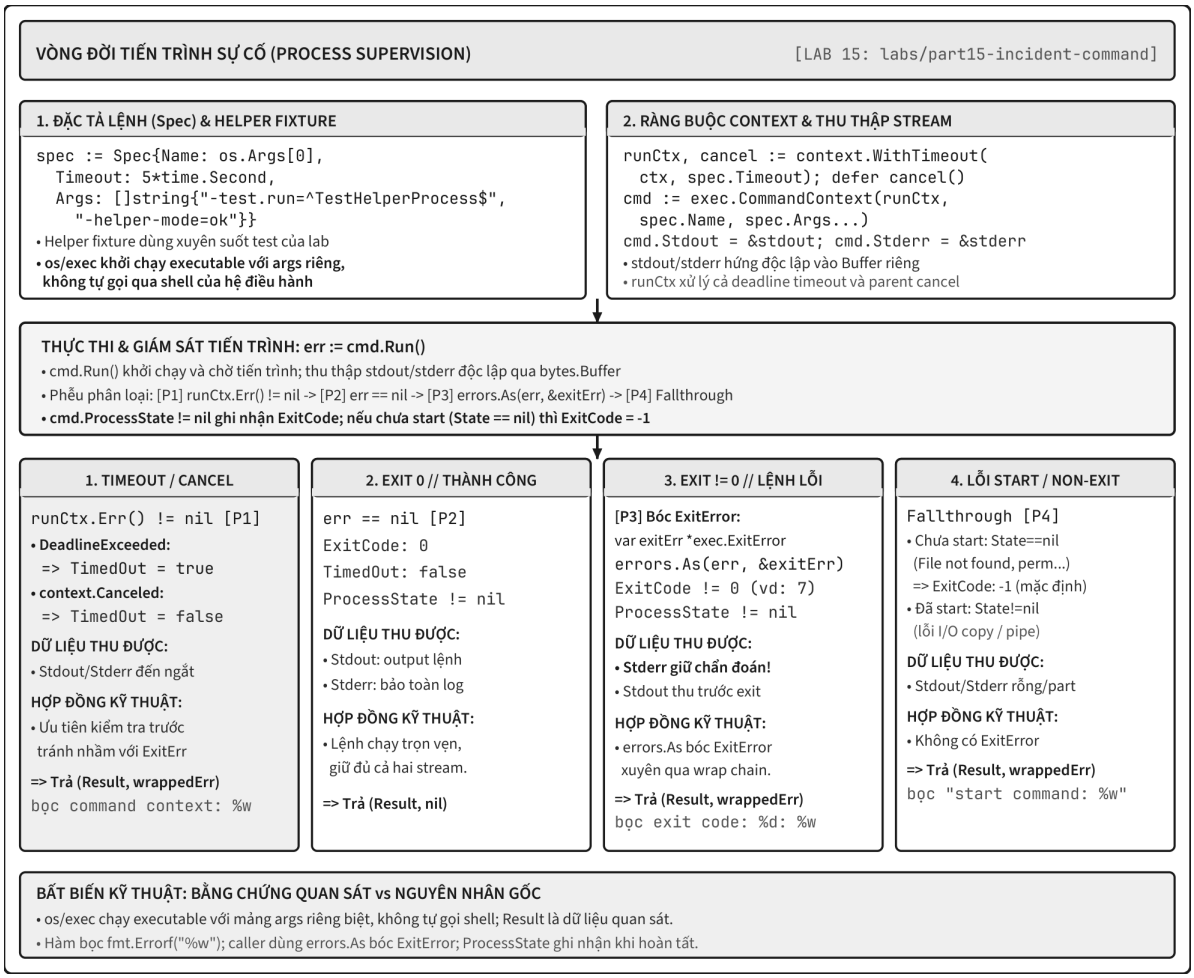
Một câu lệnh nhìn có vẻ vô hại

Giả sử alert báo dependency `catalog` trả lời chậm. Một command đầu tiên có thể chạy một probe nội bộ hoặc một program hệ điều hành để lấy thêm manh mối. Cách tắt nhất thường trông như sau:

```
exec.Command("sh", "-c", "probe " + target).Run()
```

Cách làm tắt này che giấu quá nhiều quyết định hệ thống. Giá trị `target` là dữ liệu thuần túy hay là một đoạn mã kịch bản vỏ lệnh (**shell script**)? Lỗi trả về xuất phát từ mã thoát khác 0, không tìm thấy tệp thực thi, hay do thời hạn xử lý đã cạn kiệt? Dữ liệu từ luồng đầu ra chuẩn `stdout` và luồng lỗi chuẩn `stderr` được chuyển hướng về đâu? Và nếu tiến trình cha của `opsprobe` đang trong quá trình dừng tắt theo vòng đời đã nêu ở Chương 12, liệu tiến trình con có tiếp tục chạy mồ côi hay không?

Gói `os/exec` cố ý không tự động kích hoạt vỏ lệnh hệ thống và không tự ý mở rộng ký tự đại diện (**globbing**), đường ống dẫn (**pipeline**) hay chuyển hướng luồng (**redirect**). Đó là một ranh giới bảo vệ hữu ích: tên lệnh thực thi và từng đối số là các phần tử dữ liệu độc lập. Nếu thực sự cần tới vỏ lệnh, kỹ sư phải chủ động lựa chọn trình thông dịch, cơ chế bao đóng chuỗi ký tự và mô hình đe dọa an ninh một cách tường minh; một công cụ xử lý sự cố không nên vô tình nhận quyền thực thi vỏ lệnh chỉ vì hành vi nối chuỗi thiếu kiểm soát.



Hình 26 — Mô hình khái niệm. `Result` là bằng chứng cục bộ về một lần chạy; nó không tự chứng minh nguyên nhân gốc của incident.

Trong chương này, bên gọi chỉ định tệp thực thi trực tiếp từ mã nguồn hoặc thông qua danh sách trắng cho phép (**allowlist**). Giá trị `target` chỉ đơn thuần là một đối số tham số. Đây không phải lời giải vạn năng cho mọi bộ điều phối công cụ, nhưng là ranh giới chuẩn xác cho yêu cầu hiện tại: người dùng không được phép truyền nguyên một chuỗi dòng lệnh tùy ý qua cờ cấu hình rồi buộc chương trình phải tự suy đoán ý nghĩa cú pháp.

Tách đối số ngăn việc shell diễn giải `;`, `*` hay `$HOME`, nhưng chương trình đích vẫn có thể hiểu đầu vào như một cờ, biểu thức hoặc đường dẫn nguy hiểm. Bên gọi phải kiểm tra theo hợp đồng của chính công cụ; `--` chỉ kết thúc cờ nếu công cụ hỗ trợ. Tên không chứa dấu phân cách đường dẫn còn được tìm qua cơ chế `LookPath`; kết quả phụ thuộc môi trường. `exec.ErrDot` chặn trường hợp tìm ra executable tương đối với thư mục hiện tại, không xác minh danh tính hay tính tin cậy của mọi executable

trong PATH. Công cụ đã chọn trước nên dùng đường dẫn và quyền truy cập được kiểm soát.

Bốn kết cục, không phải một `error`

Bộ chạy lệnh có thể trả về lỗi `error`, nhưng bên gọi bắt buộc phải phân biệt được dạng thất bại cụ thể. Thành công trọn vẹn là khi tiến trình được khởi chạy và kết thúc với mã trạng thái bằng 0. Một câu lệnh được kích hoạt thành công nhưng kết thúc với mã thoát 7 lại là một kết cục hoàn toàn khác: luồng lỗi chuẩn `stderr` của nó có thể chứa đựng chính xác manh mối chẩn đoán sự cố. Quá hạn thời gian chờ (**deadline exceeded**) là kết cục thứ ba. Không tìm thấy tệp thực thi hoặc không đủ quyền khởi chạy là kết cục thứ tư, thường xuất phát từ sai sót cấu hình môi trường trước khi quá trình chẩn đoán kịp bắt đầu.

Bảng 14 — Kết cục của một lần chạy process

Kết cục	Process đã start?	`Result` giữ gì	Caller nên biết gì
Exit 0	Có	stdout, stderr, exit code 0	Tool hoàn tất theo contract của nó.
Exit khác 0	Có	stdout, stderr, exit code	Tool đã chạy; output có thể giải thích failure.
Deadline/ cancel	Có thể đã start	output đã thu được, cờ timeout	Parent context hay ngân sách thời gian đã kết thúc.
Không start	Không	command và exit code chưa có	Sai executable, quyền hoặc môi trường.

Hai đối tượng `Result` và `error` song hành cùng nhau vì chúng giải đáp hai khía cạnh riêng biệt. Cấu trúc `Result` lưu giữ toàn bộ bằng chứng thực nghiệm thu thập được; giá trị `error` xác nhận lượt chạy có thỏa mãn hợp đồng hành vi hay không. Do đó, tránh thiết kế giao diện lập trình chỉ trả về lỗi đơn thuần rồi làm thất lạc nội dung `stderr` mỗi khi tiến trình kết thúc với mã thoát khác 0. Đồng thời, không nên quy chụp mọi mã thoát khác 0 đều là lỗi nghiêm trọng: một công cụ chẩn đoán có thể chủ động sử dụng mã thoát để thông báo một tình trạng nghiệp vụ đã được trù tính từ trước.

Viết supervisor nhỏ trước

Bài thực hành giới hạn cấu trúc `Spec` trong phạm vi rất hẹp: trường `Name` là tệp thực thi do bên gọi quyết định, mảng `Args` chứa các đối số phân tách rõ ràng, và `Timeout` quy định ngân sách thời gian của lượt chạy. Hàm `Run` không cần biết dịch vụ `catalog` là gì, không phân tích cú pháp JSON và không tự ý ghi nhật ký. Việc tách bạch các trách nhiệm này giúp kiểm thử cô lập và kiểm soát trọn vẹn ranh giới tiến trình. Hai đoạn mã `Run` dưới đây là các phần liên tiếp của cùng một hàm; mã nguồn hoàn chỉnh có thể chạy được lưu tại tập tin `fixed/run.go`.

```
type Spec struct {
    Name    string
    Args    []string
    Timeout time.Duration
}

type Result struct {
    Command  []string
    Stdout   string
    Stderr   string
    ExitCode int
    TimedOut bool
}
```

Đối số `context.Context` luôn là dữ liệu đầu vào tiên quyết. Nó liên kết bộ chạy lệnh vào chuỗi yêu cầu, dòng lệnh CLI, hoặc vòng đời của dịch vụ mà không buộc bộ chạy phải trực tiếp xử lý các tín hiệu hệ điều hành cụ thể. Ta thiết lập một thời hạn con; cơ chế hủy bỏ từ tiến trình cha sẽ lan truyền xuyên suốt qua cùng đường dẫn đó. Phương thức `exec.CommandContext` sử dụng ngữ cảnh để ngắt tiến trình khi ngữ cảnh hoàn tất; mặc định hàm hủy bỏ của câu lệnh sẽ gửi tín hiệu cưỡng chế dừng (**Kill**) lên tiến trình. Đây là hành vi đã được tài liệu hóa của thư viện chuẩn, không phải lời cam kết rằng toàn bộ cây tiến trình con trên mọi hệ điều hành đều được dọn dẹp sạch sẽ.

```
func Run(ctx context.Context, spec Spec) (Result, error) {
    if ctx == nil || strings.TrimSpace(spec.Name) == "" {
        return Result{}, ErrInvalidSpec
    }
    if spec.Timeout <= 0 {
        return Result{}, ErrInvalidSpec
    }

    runCtx, cancel := context.WithTimeout(ctx, spec.Timeout)
    defer cancel()

    cmd := exec.CommandContext(runCtx, spec.Name, spec.Args...)
    var stdout, stderr bytes.Buffer
    cmd.Stdout = &stdout
    cmd.Stderr = &stderr
    err := cmd.Run()
```

Đoạn mã đầu tiên kết thúc tại lệnh `Run`: nó khởi động tiến trình, chờ tiến trình cùng hai luồng nhập xuất hoàn tất, rồi lưu giữ lỗi gốc để phân loại. Bước kiểm tra tính hợp lệ bắt buộc phải diễn ra trước lời gọi `context.WithTimeout`, bởi việc truyền ngữ cảnh `nil` sẽ gây sập chương trình. Hai bộ đệm dữ liệu được gán trước khi chạy để bên gọi không phải suy đoán xem một dòng thông tin chẩn đoán xuất phát từ luồng chuẩn `stdout` hay luồng lỗi `stderr`.

Sau khi process đã trả, phần còn lại dựng `Result` kể cả khi có error. Đọc nó như đoạn tiếp theo của function ở trên:

```
    result := Result{
        Command:  append([]string{spec.Name}, spec.Args...),
        Stdout:   stdout.String(),
        Stderr:   stderr.String(),
        ExitCode: -1,
    }
    if cmd.ProcessState != nil {
        result.ExitCode = cmd.ProcessState.ExitCode()
    }
    if contextErr := runCtx.Err(); contextErr != nil {
        result.TimedOut = errors.Is(
            contextErr,
            context.DeadlineExceeded,
        )
        return result, fmt.Errorf(
            "command context: %w",
```

```
            contextErr,
        )
    }
    if err == nil {
        return result, nil
    }
    var exitErr *exec.ExitError
    if errors.As(err, &exitErr) {
        return result, fmt.Errorf(
            "command exited with code %d: %w",
            result.ExitCode,
            err,
        )
    }
    return result, fmt.Errorf("start command: %w", err)
}
```

`ProcessState` được điền khi tiến trình đã khởi chạy và được chờ kết thúc. Runner khởi tạo `ExitCode=-1` khi chưa có trạng thái ấy, nhưng `ProcessState.ExitCode()` cũng có thể trả `-1` nếu tiến trình kết thúc bởi signal trên hệ điều hành hỗ trợ. Trước bước chạy, lỗi đặc tả trả `Result{}`

với mã zero; vì vậy chỉ một số `ExitCode` không chứng minh tiến trình đã chạy hay thành công. Hãy xét cả `error` và hợp đồng của runner.

Runner ưu tiên `runCtx.Err()` ở thời điểm thu kết quả. Đây là chính sách phân loại, không phải bằng chứng context đã gây ra mọi mã thoát. Nếu context bị hủy sát lúc tiến trình kết thúc, lỗi context có thể được ưu tiên và `*exec.ExitError` không còn nằm trong error trả về. Cờ `TimedOut` chỉ chỉ ra `DeadlineExceeded`, không chỉ mọi lần hủy. Muốn phân biệt thêm nguyên nhân, cần hợp đồng kết quả khác; helper hiện tại không suy ra nhân quả từ thứ tự quan sát.

Giải pháp này chỉ phù hợp với dữ liệu đầu ra dung lượng nhỏ, được giới hạn theo hợp đồng của công cụ đang chạy. Cấu trúc `bytes.Buffer` không phải chính sách tiếp nhận đầu ra phù hợp cho một tiến trình có thể in dữ liệu vô tận. Một công cụ môi trường thực tế bắt buộc phải chọn một trong ba chiến lược rõ ràng: truyền dòng dữ liệu có cơ chế tạo áp lực ngược (**backpressure**), thu thập có giới hạn trần và đánh dấu cắt ngắn rõ ràng, hoặc lưu trữ dữ liệu đầu ra thành tệp tạo tác (**artifact**). Tuyệt đối không âm thầm sử dụng bộ đệm vô hạn trong bộ nhớ rồi ngộ nhận đó là năng lực quan sát hệ thống.

Cancel process không đồng nghĩa dọn cả cây process

`CommandContext` mặc định gọi `Process.Kill` cho tiến trình trực tiếp khi context kết thúc; việc khởi chạy có thể chưa xảy ra nếu context đã hủy. Nó không dọn cả cây tiến trình. Tiến trình hậu duệ có thể giữ mở pipe dù tiến trình trực tiếp đã thoát. `WaitDelay` bắt đầu đếm từ khi context kết thúc hoặc `Wait` quan sát tiến trình thoát, tùy điều kiện nào đến trước; khi hết khoảng chờ nó có thể kill tiến trình trực tiếp còn sống và đóng các pipe do `Cmd` quản lý. Đây không phải thời lượng tối đa kể từ `Start`, cũng không làm một `Read`/`Write` tùy ý đang block tự kết thúc. Nếu tiến trình thoát thành công nhưng pipe phải đóng bởi `WaitDelay` và không có lần gọi `Cancel`, `Wait` trả `exec.ErrWaitDelay`. Nhóm tiến trình và Windows job object cần chính sách hệ điều hành riêng.

Test deadline hiện có kiểm lỗi và cờ timeout, không chứng minh tiến trình đã khởi chạy trước hạn: 30ms có thể hết ngay trong giai đoạn start. Test

`TestRunCancelsAnAlreadyStartedChild` bổ sung điểm bắt tay qua TCP loopback; chỉ hủy sau khi helper báo đã chạy và đang chờ. Fixture chờ runner trả về, kiểm kết nối đã đóng và join goroutine do test sở hữu. Nó kiểm tiến trình trực tiếp trên môi trường chạy test, không đo latency cố định, không chứng minh cây hậu duệ được dọn. Runner vẫn để `WaitDelay=0`, nên các công cụ giữ pipe mở có thể khiến `Run` chờ lâu hơn ngân sách context. Đó là giới hạn cần giải quyết trước khi dùng cho loại công cụ ấy.

Từ runner sang CLI có thể kể lại

Giao diện dòng lệnh (**CLI**) là ranh giới tiếp xúc trực tiếp với con người. Mã nguồn không nên nhồi nhét việc phân tích cờ cấu hình, kiểm tra hợp lệ mục tiêu, khởi tạo tiến trình con, định dạng dữ liệu và ghi nhật ký vào trong một hàm `main` kéo dài. Kiểu `flag.FlagSet` là đủ cho câu lệnh đầu tiên: phân tích cờ `-target` và `-timeout`, kiểm tra mục tiêu theo đúng hợp đồng của công cụ thăm dò, rồi kích hoạt bộ chạy với tệp thực thi đã định. Khi hàm `Run` hoàn tất, giao diện có thể in báo cáo trực quan cho người dùng; nhật ký có cấu trúc chỉ ghi nhận các trường dữ liệu cần thiết phục vụ việc tái dựng lượt chạy.

```
if !errors.Is(err, ErrInvalidSpec) {
    logger.Info(
        "diagnostic returned",
        "tool", spec.Name,
        "exit_code", result.ExitCode,
        "timed_out", result.TimedOut,
        "ok", err == nil,
    )
}
```

Đoạn ghi nhận dùng `spec.Name` đã được chọn trước, không truy cập `result.Command[0]`: lỗi đặc tả trả slice rỗng và truy cập ấy sẽ panic. Nhãn “returned” cùng trường `ok` cũng tránh gọi mọi lượt chạy là hoàn tất thành công; không ghi toàn bộ error hay đối số vì chúng có thể chứa dữ liệu nhạy cảm.

Tuyệt đối không ghi nhật ký các thông tin mã thông báo bảo mật (**token**), tiêu đề ủy quyền (**authorization header**), toàn bộ biến môi trường hay chuỗi dòng lệnh thô chứa mật khẩu bí mật. Gói `slog` không tự động che giấu dữ liệu nhạy cảm; chính ranh giới thiết kế phải chọn lọc trường nào được phép lưu vào nhật ký. Chương 16 sẽ nâng tầm một lượt chẩn đoán đơn lẻ thành các tín hiệu quan sát toàn diện qua trạng thái sức khỏe, chỉ số đo lường, dấu vết thực thi và mục tiêu mức dịch vụ (**SLO**). Tại mốc này, điều cốt lõi là báo cáo không được đánh tráo giữa ba trạng thái: câu lệnh không thể khởi động, câu lệnh thực thi thất bại, và câu lệnh bị chủ động ngắt do cạn kiệt ngân sách thời gian.

> **Dừng để dự đoán:** Nếu hàm `Run` chỉ trả về `error`, báo cáo sự cố sẽ đánh mất thông tin gì khi một công cụ thăm dò kết thúc với mã thoát 2 nhưng đã kịp in lời giải thích chẩn đoán vào `stderr`? Ngược lại, nếu hàm `Run` chỉ trả về `Result` mà không trả về `error`, liệu bên gọi có còn phân biệt được lượt chạy thành công với lượt chạy bị cưỡng chế dừng do quá hạn thời gian?

Bài thực hành: Chứng minh trạng thái thoát và thời hạn

Hãy mở tập tin `labs/part15-incident-command/exercise/run_test.go` trước. Bài kiểm thử không gọi `ping`, `curl` hay bất kỳ tệp thực thi có sẵn nào của hệ điều hành. Nó sử dụng chính tệp nhị phân kiểm thử hiện hành làm tiến trình hỗ trợ, giúp các trường hợp thành công, mã thoát khác 0 và tạm dừng đều vận hành đồng nhất trên Windows, Linux và macOS. Đó là một nguyên lý thiết kế kiểm thử cốt lõi: thay vì phụ thuộc vào các công cụ ngoại vi trên máy chủ, ta chủ động kiểm soát tiến trình mục tiêu đang được quan sát.

```
cd labs/part15-incident-command
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

Hợp đồng hành vi đòi hỏi: đặc tả rỗng hoặc thời gian chờ không dương phải bị từ chối dứt khoát trước khi khởi chạy; trường hợp thành công lưu giữ hai luồng dữ liệu riêng biệt cùng mã thoát 0; trường hợp mã thoát khác 0 bảo toàn luồng `stderr`, mã thoát và vẫn cho phép `errors.As` nhận diện kiểu `*exec.ExitError`; trường hợp quá hạn thời gian trả về lỗi khớp với `context.DeadlineExceeded`, đánh dấu cờ `TimedOut` và không bị treo tiến trình. Việc hủy bỏ từ ngữ cảnh cha vẫn trả về `context.Canceled`, không bị đánh đồng với lỗi quá hạn thời gian. Bài tập ban đầu báo đỏ vì hành vi chưa được cài đặt, không phải do gói thư viện gặp lỗi.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Tạo ngữ cảnh con sau khi xác thực dữ liệu, dùng `exec.CommandContext` với tên lệnh và các tham số tách riêng, rồi gán hai bộ đệm `bytes.Buffer` trước khi gọi `Run`. Dựng đối tượng `Result` bất kể `Run` trả về lỗi, vì dữ liệu đầu ra và trạng thái thoát vẫn có thể cần thiết cho công tác điều tra sự cố. Kiểm tra `runCtx.Err()` trước lỗi `ExitError` để thời hạn quá giờ không bị che khuất bởi trạng thái của tiến trình vừa bị hủy. Bản sửa đổi không dùng vỏ lệnh hệ thống (**shell**) và không cam kết dọn dẹp toàn bộ cây tiến trình con.

Khi nào runner nhỏ này chưa đủ

Bộ chạy lệnh này chưa tiếp nhận luồng nhập chuẩn `stdin`, chưa thiết lập thư mục làm việc hay biến môi trường riêng biệt, chưa truyền dòng dữ liệu đầu ra, chưa che giấu dữ liệu bí mật, chưa hỗ trợ thử lại, và chưa được coi là bộ thực thi tổng quát cho dữ liệu người dùng tùy ý. Mỗi tính năng trên đều làm thay đổi bề mặt tấn công (**attack surface**) hoặc vòng đời tiến trình. Chẳng hạn, trường `Cmd.Env` sẽ thay thế hoàn toàn biến môi trường của tiến trình con chứ không tự động hợp nhất với môi trường hiện hành; trường `Cmd.Dir` biến thư mục làm việc thành một đầu vào khác cần kiểm tra hợp lệ. Tuyệt đối không thêm các trường này chỉ vì cấu trúc dữ liệu còn chỗ trống.

Điểm dừng của chương ghi nhận một bước tiến vững chắc: chuyển dịch từ thói quen gõ lệnh cảm tính khi hoảng loạn sang việc kiểm soát câu lệnh có hợp đồng hành vi chặt chẽ về tiến trình. Ứng dụng `opsprobe` sau này có thể tận dụng bộ chạy này để kích hoạt một công cụ chẩn đoán đã được lựa chọn, nhưng chỉ khi yêu cầu xử lý sự cố thực sự đòi hỏi. Trước khi nghĩ đến việc bổ sung chỉ số đo lường, bảng điều khiển hay Kubernetes, kỹ sư phải hiểu tường minh một lượt thực thi đơn lẻ đã bắt đầu, kết thúc và thất bại như thế nào.

Chương 16 — Thấy được hệ thống

Sau Chương 15, `opsprobe` đã ghi nhận được một lượt thực thi: lệnh đã gọi, mã thoát và việc hết thời gian chờ. Nhưng điều tra sự cố thường cần nhiều hơn một lượt chạy. Dịch vụ phụ thuộc có chậm dần từ sáng? Lỗi tập trung ở một đích hay nhiều đích? Một yêu cầu chậm đã chờ ở đâu? Chỉ giữ dòng log cuối cùng thì mất xu hướng; chỉ giữ biểu đồ tổng hợp thì mất chi tiết để điều tra từng lần thất bại.

Mô hình tư duy của chương là các phép chiếu: mỗi tín hiệu giữ lại một phần bằng chứng về hệ thống và bỏ qua phần khác. Log lưu chi tiết sự kiện; metric tổng hợp để thấy biến động theo thời gian; trace liên kết các chặng xử lý khi ngữ cảnh được truyền đúng; health check trả lời một câu hỏi hẹp tại thời điểm kiểm tra. Không tín hiệu nào tự ghi lại toàn bộ hệ thống.

Một biểu đồ xanh vẫn có thể nói dối

Giả sử dashboard hiển thị `probe_success=1` sau lần thăm dò gần nhất. Một lần thành công sau sự cố có thể làm biểu đồ xanh trở lại, nhưng không cho biết mười phút trước có bao nhiêu lần thất bại. Nó cũng không nói timeout được tính thế nào, hay giá trị còn sót lại từ một tiến trình đã ngừng thăm dò. Ta phải xác định ý nghĩa dữ liệu trước khi chọn cách vẽ.

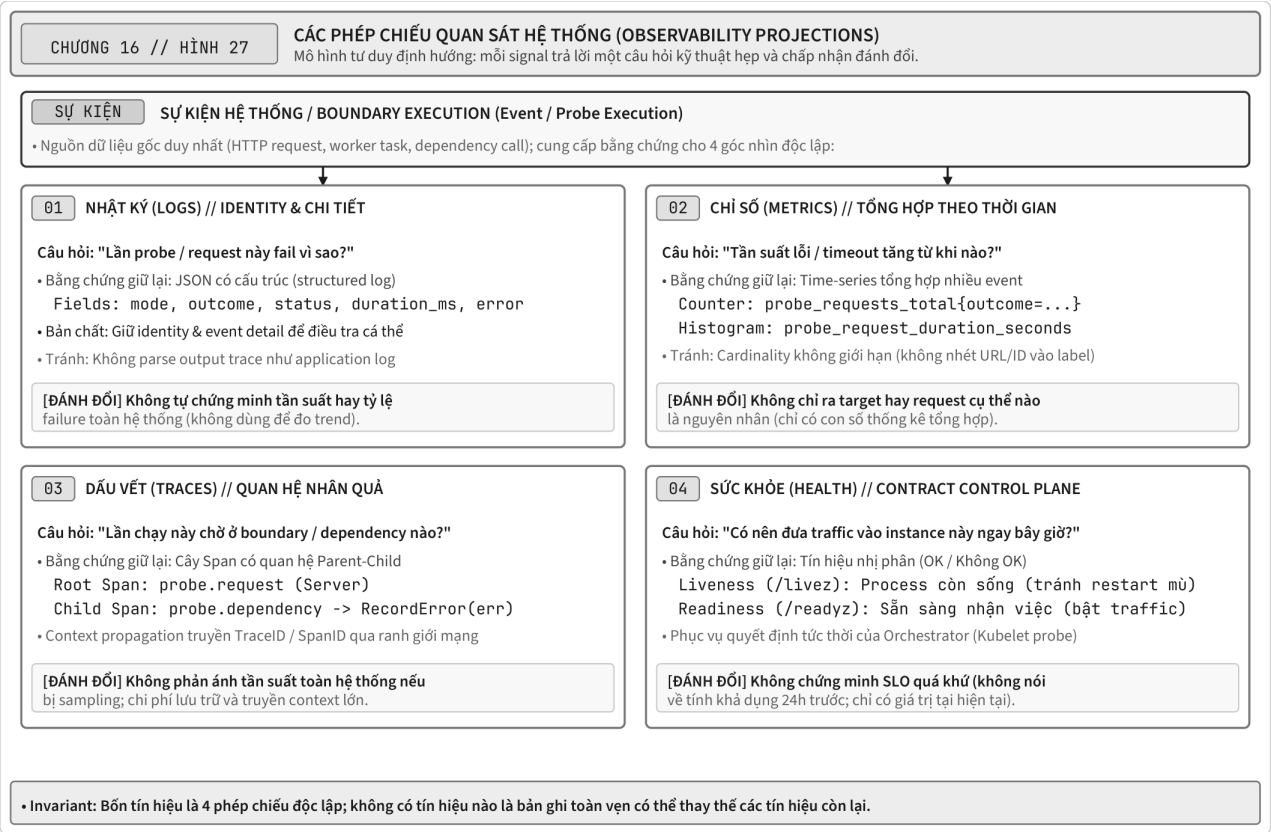
Một bộ đếm chỉ có nghĩa khi biết sự kiện nào làm nó tăng. Trong lab, mỗi lượt thăm dò hoàn tất nhận đúng một kết quả: `success`, `failure` hoặc `timeout`. SLI khả dụng được chọn là số lượt `success` chia cho tổng số lượt hoàn tất. Timeout tăng mẫu số nhưng không tăng tử số. Đây là chính sách của phép thăm dò này, không phải định nghĩa khả dụng áp dụng cho mọi sản phẩm.

```
availability = successes / completed probes
```

Nếu chưa có lượt thăm dò nào hoàn tất, phép chia tỷ lệ hoàn toàn vô nghĩa về mặt toán học. Tuyệt đối không tự ý trả về `1.0` chỉ để nhuộm xanh bảng điều khiển khi hệ thống hoàn toàn chưa thu thập được dữ liệu. Giao diện lập trình của bài thực hành cung cấp thêm một giá trị logic kiểu boolean nhằm

giúp bên gọi (**caller**) phân biệt rạch ròi giữa trạng thái “0% thành công” với trạng thái “chưa có mẫu thu thập (**sample**)”. Một chỉ số mức độ dịch vụ (SLI) chỉ đơn thuần là quy ước đo lường đã định hình; mục tiêu mức độ dịch vụ (SLO) mới chính là đích ngắm và khoảng thời gian quan sát (**time window**) mà đội ngũ kỹ thuật cam kết gắn liền với chỉ số đó. Con số “99.9%” sẽ không có giá trị vận hành thực tế chừng nào ta chưa xác định rõ đo lường đối tượng nào, trong khung thời gian bao lâu, và biện pháp can thiệp nào sẽ được kích hoạt khi làm cạn kiệt ngân sách lỗi (**error budget**).

Bốn câu hỏi, bốn loại bằng chứng



Hình 27 — Các tín hiệu đo lường giải đáp những câu hỏi kỹ thuật khác nhau. Sơ đồ là mô hình tư duy định hướng, không phải cấu trúc bắt buộc hay đường ống OpenTelemetry hoàn chỉnh.

Bảng 15 — Chọn signal từ câu hỏi trước

Câu hỏi	Signal chính	Điều nó không tự chứng minh
“Lần probe này fail vì sao?”	Log có field và error an toàn	Tần suất hay tỷ lệ failure toàn hệ thống.
“Timeout tăng từ khi nào?”	Metric theo thời gian	Target hay request cụ thể nào là nguyên nhân.
“Lần chạy này chờ ở boundary nào?”	Trace của một execution	Tần suất của mọi execution nếu trace bị sample.
“Có nên đưa traffic vào instance này ngay bây giờ?”	Readiness	SLO của một giờ vừa qua.

Log có thể lưu đích đến đã che dữ liệu nhạy cảm, mã thoát, thời lượng và request ID để điều tra một lượt cụ thể. Không nên đưa nguyên URL, user ID, thông điệp lỗi thô hay trace ID vào nhãn metric mặc định. Mỗi tổ hợp giá trị nhãn tạo một chuỗi thời gian riêng. Cardinality là số chuỗi phân biệt, không chỉ là số tên nhãn: một nhãn `request_id` cũng có thể sinh hàng triệu chuỗi. Chi phí lưu trữ và truy vấn có thể trở thành sự cố mới.

Dấu vết thực thi (**trace**) cũng không đơn thuần là dòng nhật ký gắn thêm mũi tên điều hướng. Nó khắc họa chuẩn xác mối quan hệ nhân quả giữa các tác vụ trong cùng một lượt xử lý; cơ chế truyền ngữ cảnh (**context propagation**) là mắt xích gắn kết các phân đoạn (**span**) xuyên qua ranh giới giao thức HTTP, hàng đợi thông điệp (**queue**) hoặc tiến trình con khi hệ thống được cài đặt cơ chế giám sát thích hợp. Chương 11 đã sử dụng `httptrace` để quan sát một ứng dụng máy khách cục bộ, nhưng kỹ thuật đó không tự động trở thành hệ thống truy vết phân tán (**distributed tracing**). Phần này tập trung chuẩn hóa ngữ nghĩa để khi áp dụng OpenTelemetry trong thực tế, ta nắm rõ một thuộc tính (**attribute**), phân đoạn thực thi (**span**) hay quyết định chọn mẫu (**sampling**) đang trực tiếp phục vụ câu hỏi kỹ thuật nào.

Tín hiệu kiểm tra sức khỏe (**health check**) là kênh thông tin dễ bị lạm dụng sai lệch nhất. Tín hiệu sống (**liveness**) trả lời việc tiến trình có còn năng lực tiếp tục hoạt động hay không để hệ thống giám sát (**supervisor**) không khởi động lại mù quáng; tín hiệu sẵn sàng (**readiness**) giải đáp xem thể hiện tiến trình hiện tại có nên đón nhận lưu lượng truy cập hay không. Cả hai đều là giao ước tức thời phục vụ mặt phẳng điều khiển (**control plane**) ở thời điểm

hiện tại. Một điểm cuối `/readyz` phản hồi mã `200` ở thời khắc này không thể chứng minh độ khả dụng trong 24 giờ đã qua, và ta cũng không được phép kích hoạt một dịch vụ phụ thuộc ở xa trong mọi lượt thăm dò liveness chỉ để khiến điểm cuối có vẻ tinh vi. Việc xác định dịch vụ phụ thuộc nào thuộc phạm vi kiểm tra readiness là một chính sách dựa trên năng lực phục vụ thực tế, không phải bảng danh mục sao chép máy móc từ các bộ khung mẫu (**framework**).

Gắn metric vào outcome, không gắn vào cảm giác

Đoạn mã mô phỏng thu nhỏ dưới đây không xuất định dạng Prometheus và không nhằm thay thế hệ thống thu thập (**collector**). Nó gói gọn ngữ nghĩa cốt lõi mà các bộ trích xuất dữ liệu sau này có thể đọc: kết quả là một tập đóng cố định, ảnh chụp trạng thái (**snapshot**) là bản sao chép nhất quán, và chỉ số SLI không mang giá trị khi chưa thu thập mẫu đo. Việc giữ bài thực hành hoàn toàn trong thư viện chuẩn giúp ta kiểm chứng chặt chẽ các ràng buộc hợp đồng trước khi thảo luận về bộ thư viện phát triển (SDK) hoặc hạ tầng lưu trữ phía sau.

```
type Outcome string

const (
    OutcomeSuccess Outcome = "success"
    OutcomeFailure Outcome = "failure"
    OutcomeTimeout Outcome = "timeout"
)

type Snapshot struct {
    Completed uint64
    Succeeded uint64
    Failed    uint64
    TimedOut  uint64
}

func (s Snapshot) SuccessRatio() (float64, bool) {
    if s.Completed == 0 {
        return 0, false
    }
    return float64(s.Succeeded) / float64(s.Completed), true
}
```

Trường dữ liệu `Completed` không phải là chi tiết dư thừa. Nó là mẫu số được lưu trữ độc lập để kiểm chứng tính bất biến toán học: `Completed == Succeeded + Failed + TimedOut`. Trong bài thực hành, một lệnh gọi `Observe` thành công sẽ gia tăng chính xác hai giá trị: trường `Completed` và nhóm kết quả tương ứng. Một kết quả không xác định bắt buộc phải trả về lỗi trước khi thực hiện biến đổi trạng thái; nếu mã nguồn chấp nhận chuỗi ký tự thô tùy ý rồi tự tạo nhóm mới, tập đóng sẽ lập tức biến thành lỗi rò rỉ bùng nổ số lượng nhãn (**cardinality leak**) ngay tại nguồn phát sinh dữ liệu.

```
type Recorder struct {
    mu       sync.Mutex
    snapshot Snapshot
}

func (r *Recorder) Observe(outcome Outcome) error {
    r.mu.Lock()
    defer r.mu.Unlock()

    switch outcome {
    case OutcomeSuccess:
        r.snapshot.Succeeded++
    case OutcomeFailure:
        r.snapshot.Failed++
```

```
    case OutcomeTimeout:
        r.snapshot.TimedOut++
    default:
        return ErrUnknownOutcome
    }
    r.snapshot.Completed++
    return nil
}

func (r *Recorder) Snapshot() Snapshot {
    r.mu.Lock()
    defer r.mu.Unlock()
    return r.snapshot
}
```

Khóa đồng bộ `sync.Mutex` ở đây không phải là nỗ lực tự viết lại một thư viện số liệu đo lường. Cơ chế này đơn thuần bảo toàn tính bất biến của bộ ghi nhận mẫu khi nhiều luồng thực thi đồng thời từ Chương 9 cùng thông báo kết quả. Phương thức `Snapshot` trả về bản sao giá trị độc lập, bởi bên gọi tuyệt đối không được phép chỉnh sửa trạng thái nội bộ làm bộ đếm bị đảo ngược giá trị. Nếu bộ xuất dữ liệu sau này đọc ảnh chụp trạng thái theo chu kỳ, nó vẫn phải tuân thủ giao ước riêng về hành vi tái lập khi tiến trình khởi động lại, mốc thời gian, bộ đếm tích lũy hoặc giá trị vi phân chênh lệch. Bản ghi trong bộ nhớ tạm thời này không có chính sách lưu trữ dài hạn (**retention**), không hỗ trợ cửa sổ trượt và không thể xem là bằng chứng lịch sử sau khi tiến trình kết thúc.

> **Dừng để dự đoán:** Một probe kết thúc với lỗi `context.DeadlineExceeded` (timeout). Sự kiện này có đồng nghĩa với việc máy chủ đích đã sập hoàn toàn không? Trong SLI đo độ khả dụng, kết quả timeout được phân loại vào đâu?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Lỗi quá thời gian (**timeout**) chỉ chứng minh bên gọi không nhận được phản hồi trong ngân sách thời gian đã định, hoàn toàn không đủ căn cứ để suy diễn dịch vụ đích đã sập hoàn toàn. Trong thực tế, dịch vụ đích có thể vẫn đang chạy nhưng bị nghẽn hàng đợi xử lý, hoặc phản hồi đã được gửi đi nhưng bị chậm trễ trên đường truyền mạng, thậm chí một tác vụ ghi dữ liệu đã được xác nhận thành công ở hệ thống phía sau trước khi bên gọi cạn kiệt thời gian chờ.

Theo contract của lab, timeout tăng `Completed` và `TimedOut`, không tăng `Succeeded`: mẫu số là mọi lượt hoàn tất, không phải chỉ các lượt thất bại. Nhãn `outcome="timeout"` giữ nguyên nhân quan sát để điều tra; nó không biến timeout thành thành công. Sản phẩm có thể chọn một SLI khác, nhưng phải công bố rõ đối tượng, công thức và cửa sổ đo.

Tính lũy đẳng (**idempotency**) trả lời câu hỏi khác: lặp lại thao tác có làm thay đổi tác động dự kiến so với thực hiện một lần không? Nó giúp thiết kế retry, không chứng minh lần gọi hết hạn đã thành công hay thất bại ở phía máy chủ. Chương 13 đã xét trường hợp ghi dữ liệu xong nhưng mất phản hồi; ở đây ta chỉ biết kết quả mà bên thăm dò quan sát được.

> **Dừng để dự đoán:** Bộ đếm đã ghi 1.000 lượt thành công. Mười lượt tiếp theo đều timeout. Tỷ lệ từ lúc khởi động và tỷ lệ trong mười lượt mới nhất là bao nhiêu?

`SuccessRatio()` trên snapshot mới trả về `1000/1010`, xấp xỉ 99,01%; trong cửa sổ mười lượt mới, tỷ lệ là `0/10 = 0%`. Không phép tính nào sai, nhưng chúng trả lời hai câu hỏi khác nhau.

`TestLifetimeRatioCanHideRecentOutage` tái hiện sự khác biệt này mà không cần máy chủ thật hay chờ thời gian thực.

Hai snapshot của cùng một recorder chưa reset cho phép tính chênh lệch `Succeeded` và `Completed`. Khi tiến trình khởi động lại, bộ đếm có thể giảm; trừ trực tiếp hai số `uint64` khi đó có thể tràn vòng và sinh số rất lớn. Một khoảng mất dữ liệu còn có thể làm cửa sổ không đủ độ bao phủ. Hệ thống thu thập phải xử lý reset, thời gian và dữ liệu thiếu; ví dụ Prometheus `increase()` xử lý reset và ngoại suy theo cửa sổ, nên kết quả không nhất thiết là số nguyên hay một bản đếm sự kiện chính xác từng lần. Lab snapshot không hiện thực những cơ chế đó.

SLO không phải một alert thật to

Một cam kết SLO thiết lập kỳ vọng dịch vụ có thể đo lường định lượng; trong khi cảnh báo (**alert**) là chính sách vận hành buộc kỹ sư phải can thiệp khi rủi ro đe dọa kỳ vọng đó vượt qua ngưỡng an toàn. Ta không nên kích hoạt cảnh báo cho từng sự kiện `failure` đơn lẻ: một máy chủ kiểm thử bị tắt có thể sinh ra tín hiệu nhiễu vô hại, trong khi một tỷ lệ lỗi ban đầu còn thấp nhưng tăng trưởng đột biến trên lưu lượng thực tế lại là tín hiệu khẩn cấp cần khoanh vùng. Khi đã xác lập đầy đủ chỉ số SLI, khung thời gian quan sát và mục tiêu cam kết, đội ngũ mới có thể cấu hình cảnh báo chuẩn xác dựa trên tốc độ tiêu hao ngân sách lỗi và tài liệu quy trình xử lý (**runbook**) tương ứng.

Điều này cũng lý giải vì sao trạng thái `Completed=0` không được phép quy kết thành mức độ khả dụng 100%. Khi hoàn toàn thiếu vắng mẫu đo, ta không thể suy diễn ra tỷ lệ phần trăm; trạng thái phản ánh trung thực có thể là `unknown`, một chú thích trên bảng điều khiển, hoặc một cảnh báo về sự gián đoạn dữ liệu đo từ xa (**telemetry gap**) tùy theo chính sách quy định. Tuyệt đối không chữa cháy bằng cách chia cho `max(1, completed)`: câu lệnh toán học sẽ vượt qua bước biên dịch, nhưng tính toàn vẹn của bằng chứng đã bị bóp méo.

Một cảnh báo chất lượng phải dẫn dắt trực tiếp tới hành động ứng phó cụ thể. Thông báo cần chỉ rõ định danh dịch vụ hoặc máy chủ đích, chỉ số SLI chịu ảnh hưởng, khung thời gian xảy ra sự cố, đội ngũ sở hữu chịu trách

nhiệm và địa chỉ tra cứu chi tiết một cách an toàn. Chỉ số đo lường khoanh vùng kỹ sư điều tra tới nhóm đối tượng (**cohort**) đang suy thoái; nhật ký hoặc dấu vết thực thi sẽ trả lời câu hỏi "điều gì bất thường đang diễn ra bên trong nhóm đối tượng đó?". Đó là một quy trình điều tra thống nhất và tuần tự, không phải phép ghép nối rời rạc bốn công cụ quan sát đặt cạnh nhau.

Bài thực hành: Giữ mẫu số và ranh giới nhãn quan sát

Mở tệp `labs/part16-observability-evidence/exercise/recorder_test.go` trước tiên. Bài tập không yêu cầu cài đặt Prometheus, OpenTelemetry hay dựng bảng điều khiển phức tạp. Bộ kiểm thử ràng buộc các giao ước thiết kế tinh gọn nhưng thường bị vi phạm: tình trạng thiếu dữ liệu không được ngụy tạo thành 100%; kết quả không xác định không được làm biến đổi trạng thái nội bộ; và nhiều luồng đồng thời ghi nhận kết quả không được làm xáo trộn tổng số tích lũy. Đây là điểm mà công cụ dò điều kiện chạy đua từ Chương 8 gắn liền trực tiếp với tính đúng đắn của dữ liệu giám sát, không đơn thuần là một câu lệnh kiểm tra hình thức.

```
cd labs/part16-observability-evidence
go test -tags exercise ./exercise
go test -race -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

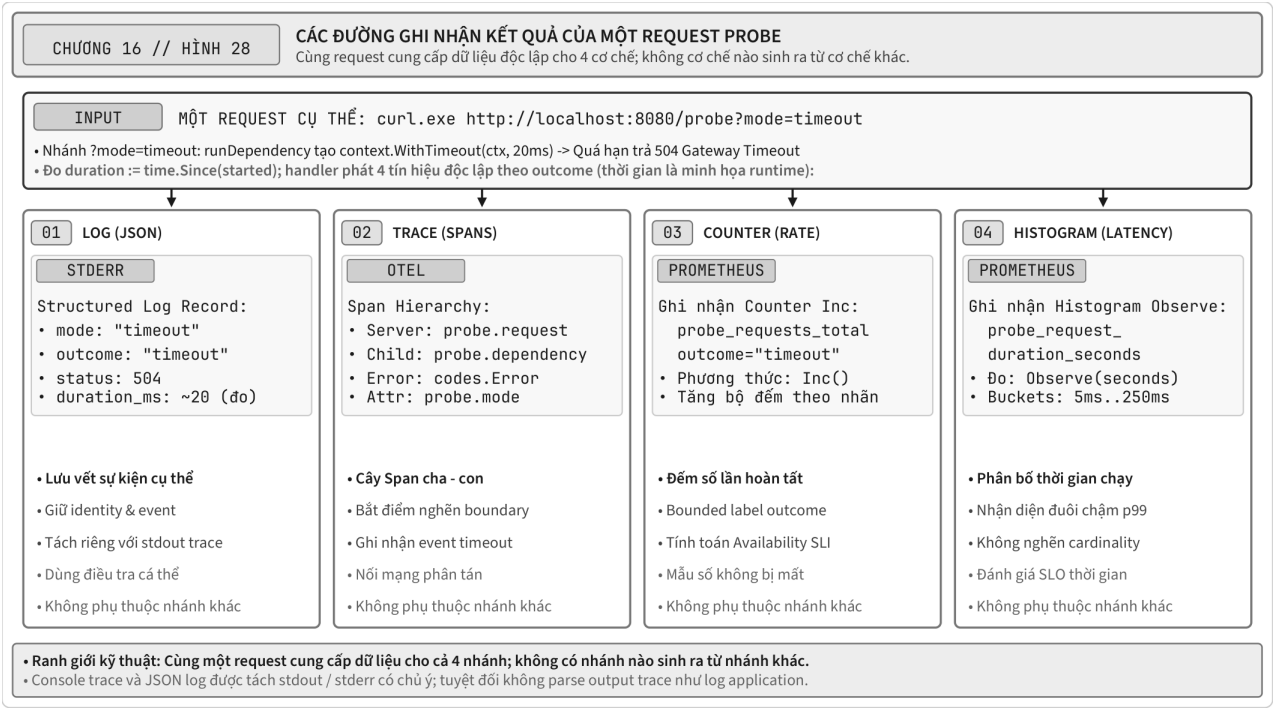
Giao ước thiết kế chủ ý không tiếp nhận `target string`, đường dẫn URL hay nội dung thông điệp lỗi. Những dữ liệu đó thuộc về nhật ký vận hành, dấu vết thực thi hoặc một mô hình dữ liệu riêng biệt có chính sách lưu trữ và khử thông tin nhạy cảm. Bộ ghi nhận `Recorder` chỉ lưu giữ chỉ số tổng hợp có giới hạn chiều nhãn. Khi các ca kiểm thử đạt kết quả tốt, hãy đối chiếu với thư mục `fixed/` và giải thích vì sao phương thức `Snapshot` trả về giá trị cấu trúc thay vì con trỏ `*Snapshot`, cũng như lý do vì sao một khóa `sync.Mutex` dùng chung là hoàn toàn đầy đủ cho tính đúng đắn trước khi phát sinh nhu cầu đo lường tranh chấp tài nguyên.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Khai báo kiểu `Outcome` là một tập đóng rồi dùng `switch` kiểm tra trước khi tăng biến đếm `Completed`. Đặt khóa đồng bộ bao quanh cả bước xác thực lẫn thao tác biến đổi dữ liệu để kết quả không hợp lệ không thể chạy đua với hàm chụp nhanh (**snapshot**) hoặc tạo ra trạng thái cập nhật dở dang. Phương thức `SuccessRatio` trả về `(float64, bool)`; giá trị boolean `false` mang ngữ nghĩa không có dữ liệu, không phải là cơ chế truyền tải lỗi. Bản sửa đổi ưu tiên một bất biến dễ kiểm chứng hơn là một API có vẻ "linh hoạt".

Giai đoạn hai: ba tín hiệu đi qua một service thật

Bộ ghi nhận vừa rồi chủ ý không phụ thuộc vào Prometheus hay OpenTelemetry. Khi câu hỏi nghiệp vụ đã rõ ràng, ta liên kết nó với các công cụ giám sát thực tế mà không cần xây dựng một nền tảng quan sát đồ sộ. Thư mục `labs/part16-real-signals` chứa một dịch vụ HTTP thu nhỏ. Một yêu cầu gửi tới điểm cuối `/probe` sẽ khởi tạo một phân đoạn máy chủ `probe.request`; tác vụ mô phỏng dịch vụ phụ thuộc tạo ra phân đoạn con `probe.dependency`; kết quả sau cùng được lưu thành nhật ký định dạng JSON, bộ đếm Prometheus và biểu đồ tần suất (**histogram**). Bộ xuất dữ liệu ra màn hình dòng lệnh in dấu vết thực thi ra cổng xuất chuẩn để người lập trình quan sát trực tiếp quan hệ cha con ngay trên môi trường phát triển cục bộ.



Hình 28 — Cùng một yêu cầu cung cấp dữ liệu cho nhật ký, dấu vết thực thi, bộ đếm và biểu đồ tần suất. Các đường ghi nhận thể hiện những góc nhìn khác nhau, không phải nhật ký sinh ra bộ đếm hay dấu vết sinh ra biểu đồ tần suất.

Đây là một lộ trình thu thập bằng chứng độc lập, không phải ba cách gọi tên khác nhau cho cùng một dòng dữ liệu. Bộ đếm trả lời câu hỏi **bao nhiêu lần đã hoàn tất theo từng nhóm kết quả.** Biểu đồ tần suất trả lời câu hỏi **thời lượng thực thi phân bố ra sao so với các khoảng phân chia** (bucket) **đã ấn định.** Dấu vết thực thi bảo toàn mối quan hệ của **một** yêu cầu cụ thể với các dịch vụ phụ thuộc mà nó triệu gọi. Nhật ký lưu giữ mã trạng thái và thời lượng của từng sự kiện để kỹ sư không phải suy diễn ngược từ dữ liệu tổng hợp về một yêu cầu riêng lẻ.

Mở tệp `labs/part16-real-signals/README.md`, theo dõi nhiệm vụ tự điều tra, rồi khởi chạy dịch vụ. Dấu vết trên giao diện dòng lệnh và nhật ký JSON được phân tách có chủ đích giữa hai luồng xuất chuẩn (**stdout**) và xuất lỗi (**stderr**): tuyệt đối không bóc tách dữ liệu dấu vết thực thi như thể nhật ký ứng dụng trong môi trường vận hành thực tế.

```
cd labs/part16-real-signals
go test ./...
go vet ./...
go test -race ./...
go run ./cmd/probe-api
```

Tại một cửa sổ dòng lệnh khác, lần lượt thực thi từng kịch bản lỗi (**failure mode**). Lệnh `curl.exe -i` giữ nguyên vẹn mã trạng thái HTTP thay vì để hệ vỏ biến đổi các phản hồi 503 hoặc 504 thành thông báo lỗi khó theo dõi.

```
curl.exe -i http://localhost:8080/probe?mode=ok
curl.exe -i http://localhost:8080/probe?mode=slow
curl.exe -i http://localhost:8080/probe?mode=fail
curl.exe -i http://localhost:8080/probe?mode=timeout
curl.exe -s http://localhost:8080/metrics
```

Các trạng thái `ok` và `slow` đều được ghi nhận là `success`; việc xử lý chậm chạp không đồng nghĩa với thất bại. Trạng thái `fail` xuất hiện khi dịch vụ phụ thuộc phản hồi lỗi lập tức và dịch vụ trả về mã `503`. Trạng thái `timeout` phản ánh trường hợp bên gọi cạn kiệt ngân sách 20 ms trước khi dịch vụ phụ thuộc mô phỏng kịp hoàn tất, khiến dịch vụ trả về mã `504`. Đây là cơ chế tiêm lỗi có ranh giới xác định, không phải độ trễ ngẫu nhiên không kiểm soát. Hãy so sánh nhật ký JSON, bộ đếm, biểu đồ tần suất và dấu vết thực thi trước khi mở mã nguồn: bằng chứng nào giúp phân biệt `fail` với `timeout` nhanh nhất? bằng chứng nào chứng minh `slow` vẫn thành công nhưng đã rơi vào khoảng phân loại thời gian khác?

Bảng 16 — Instrument của lab và contract của nó

Instrument	Event boundary	Tập dữ liệu cố ý giữ hẹp	Điều không được suy ra
`probe_requests_total`	Dependency trả kết quả, trước khi ghi HTTP response.	`outcome`: `success`, `failure`, `timeout`.	Không chứng minh client nhận được response.
`probe_request_duration_seconds`	Từ đầu handler đến sau dependency, trước ghi response.	Cùng label; bucket 5–250 ms.	Không đo toàn bộ thời gian phía client; bucket không là SLO.
JSON log	Sau dependency, trước ghi response.	mode, outcome, HTTP status dự định gửi, duration.	Không xác nhận response đã được bàn giao thành công.
Hai spans	Request và dependency.	mode/outcome; error status khi fail.	Một trace không là thống kê cho mọi request.

Việc lựa chọn các khoảng phân loại (**bucket**) cho biểu đồ tần suất là một quyết định đo lường có tính toán. Bài thực hành chọn các mốc 5, 10, 25, 50, 100 và 250 ms nhằm phân biệt rõ ràng trường hợp quá thời gian 20 ms với trường hợp chậm 40 ms trên máy tính cá nhân; cấu hình này không được sao chép nguyên xi thành ngưỡng đo lường thực tế. Nếu cần trả lời câu hỏi về độ trễ ở ngưỡng 2 giây, việc đặt mốc tối đa 250 ms sẽ làm mất hoàn toàn độ phân giải dữ liệu cần thiết. Nếu đưa từng đường dẫn định tuyến, URL thô, định danh người dùng, mã dấu vết hay thông báo lỗi vào làm nhãn, một chỉ số tổng hợp sẽ bị biến thành kho lưu trữ sự kiện vô tận. Bài tập chỉ gán kết quả `outcome` vào nhãn; tham số `mode` ít mang giá trị trong ngữ cảnh này và không cần thiết cho câu hỏi vận hành được đặt ra.

> **Dừng để dự đoán:** Sau khi chạy `ok`, `slow`, `fail`, `timeout` một lần, bộ đếm `success` sẽ mang giá trị bao nhiêu? Khởi động lại tiến trình rồi thu thập lại `/metrics`: vì sao việc các con số bị đặt lại từ đầu không chứng minh rằng những lượt probe cũ chưa từng diễn ra? Máy chủ Prometheus lưu trữ mẫu theo chuỗi thời gian là một ranh giới độc lập với tiến trình ứng dụng đang xuất dữ liệu tích lũy hiện thời.

Bộ kiểm thử của bài học không nhằm thay thế bộ thu thập cục bộ hay bảng điều khiển hiển thị. Nó xác thực ba giao ước kỹ thuật có thể tự động hóa: chỉ số đo lường chỉ chứa các nhãn có giới hạn chiều, nhật ký JSON ghi nhận trường `outcome`, và phân đoạn con thực sự nhận phân đoạn yêu cầu làm nút cha. Mã nguồn sử dụng thư viện Go client của Prometheus để xuất dữ liệu tại `/metrics`, cùng bộ phát triển OpenTelemetry Go SDK kèm bộ xuất dữ liệu dòng lệnh để phục vụ gỡ lỗi cục bộ. Khi chuyển dấu vết sang hệ thống thu thập tập trung hoặc dịch vụ lưu trữ chuyên dụng, các yếu tố về điểm cuối, tỷ lệ chọn mẫu, thời hạn lưu trữ, bảo mật quyền riêng tư và quyền truy cập là những chính sách mới cần được thiết kế; tuyệt đối không ghi dữ liệu bí mật vào thuộc tính chỉ để phục vụ tìm kiếm tiện lợi.

Có sample chưa có nghĩa tín hiệu đã trả lời sự cố

Trong thư mục `part16-real-signals`, các chế độ cục bộ giúp việc thiết lập đo lường được trực quan và sáng tỏ. Giai đoạn tiếp theo tại `projects/opsprobe/failurelab` tiếp tục vận hành với `slog`, một vùng đăng ký Prometheus riêng biệt và bộ xuất dữ liệu dòng lệnh OpenTelemetry, nhưng thay thế dịch vụ phụ thuộc giả lập bằng kết nối mạng nội bộ HTTP có điểm kiểm tra (**checkpoint**). Mục đích không phải là dựng thêm bảng điều khiển: cấu trúc này buộc cả ba tín hiệu đo lường phải phản ánh thống nhất về cùng một lượt thăm dò khi phát sinh sự cố quá thời gian hoặc phần thân phản hồi bị ngắt quãng.

Biểu đồ tần suất `opsprobe_probe_duration_seconds` đo lường thời gian thực thi nội tại của lượt thăm dò; chỉ số `failurelab_job_seconds` đo từ lúc tiếp nhận yêu cầu cho tới khi hoàn tất thăm dò, bao gồm cả thời

gian chờ trong hàng đợi nhưng không tính thời gian chờ tiến trình tiêu thụ lấy kết quả. Thước đo biến thiên `failurelab_running` chỉ bao quát các lượt thăm dò đang trong tiến trình xử lý, trong khi `failurelab_queue_depth` phản ánh số công việc chưa được công nhân (**worker**) tiếp nhận. Các chỉ số này không thể thay thế thước đo số lượng tiến trình công nhân còn sống của toàn dự án. Một luồng công nhân đang chờ gửi dữ liệu vẫn có thể duy trì hoạt động ngay cả khi chỉ số đang chạy bằng 0; chỉ khi ghi nhận kênh `Done` đóng lại mới xác nhận luồng đã thực sự kết thúc.

Hãy đọc Chương 20 để chủ động lựa chọn bằng chứng phân biệt trước khi thực thi. Bộ kiểm thử xác thực sự hiện diện của nhật ký kèm kết quả, mẫu đo biểu đồ tần suất và phân đoạn dấu vết, chứ không cam đoan về độ trễ môi trường vận hành thực tế hay một hạ tầng lưu trữ dấu vết bền vững. Khi thực hiện khối lượng công việc có giới hạn, hãy ghi nhận cả số lượng yêu cầu bị từ chối: thông lượng chỉ tính trên các yêu cầu được tiếp nhận mà bỏ qua lượng từ chối có thể khiến một dịch vụ đang chủ động cắt tải trông có vẻ rất khỏe mạnh. Không sử dụng mã định danh máy chủ đích tùy tiện làm nhãn đo lường; bài thực hành giữ nguyên tập kết quả hữu hạn, còn định danh cụ thể được lưu giữ trong nhật ký và phân đoạn dấu vết theo đúng chính sách dữ liệu đã định trước.

Khi dùng thư viện thật

Khi yêu cầu kỹ thuật đòi hỏi điểm cuối `/metrics`, biểu đồ tần suất đo thời lượng, mẫu điểm nổi bật (**exemplars**), lan truyền ngữ cảnh dấu vết, bộ thu thập hoặc truy vấn hạ tầng lưu trữ, hãy sử dụng thư viện máy khách chuẩn mực và quy ước ngữ nghĩa được duy trì ổn định thay vì tự sáng chế định dạng truyền tải trên đường truyền. Tuy nhiên, bộ công cụ phát triển phần mềm không thể lựa chọn chỉ số SLI thay cho đội ngũ kỹ thuật. Trước khi bổ sung một công cụ đo lường, hãy xác định rõ câu hỏi vận hành cần giải đáp, ranh giới phát sinh sự kiện, đơn vị đo, số lượng chiều nhãn, hành vi khi khởi động lại, đội ngũ sở hữu và hành động can thiệp khi tín hiệu suy thoái. Nếu chưa trả lời được những câu hỏi then chốt đó, việc bổ sung thêm chỉ số đo lường chỉ tạo thêm rác dữ liệu chứ không giúp cải thiện khả năng quan sát hệ thống.

Điểm dừng của chương đánh dấu một bước chuyển trong nhận thức: kết quả từ Chương 15 không còn là một giá trị `Result` bị in ra màn hình rồi lãng quên. Nó có thể chuyển hóa thành bằng chứng xác thực theo chuỗi thời gian, nhưng chỉ sau khi ta quy ước trung thực kết quả nào được thống kê, tín hiệu nào lưu giữ chi tiết bối cảnh, và những suy diễn nào mà số liệu hiện thời chưa đủ căn cứ chứng minh. Chương 17 sẽ chuyển dịch từ "thể hiện tiến trình có thể quan sát" sang "tạo phẩm được đóng gói và điều phối"; khi ấy tín hiệu sẵn sàng sẽ kết nối trực tiếp với một mặt phẳng điều khiển thực sự, thay vì chỉ dừng lại ở một điểm cuối hình thức.

Chương 17 — Đóng gói và điều phối

Một chương trình Go chạy được bằng lệnh `go run` thường tạo ra một ảo giác an tâm: mã nguồn (**source**), trình biên dịch (**compiler**), hệ vỏ dòng lệnh (**shell**), tệp cấu hình và tiến trình (**process**) đều hiện diện cận kề, khiến việc "chạy được" có vẻ là một kết quả hiển nhiên. Nhưng mọi sự không còn đơn giản khi chương trình phải được biên dịch một lần, triển khai sang máy chủ khác, tiếp nhận các cấu hình môi trường khác, bị dừng đột ngột giữa chừng, rồi tự động thay thế khi nút mạng (**node**) gặp sự cố biến mất. Lúc ấy, câu hỏi đặt ra không đơn thuần là "làm sao chạy Docker?" mà là: ai sở hữu trạng thái mà ta muốn hệ thống duy trì?

Mô hình tư duy của chương là: ảnh container đóng gói giao ước khởi động của một tiến trình; bộ điều phối duy trì trạng thái mong muốn bằng vòng lặp điều hòa, không lặp lại một lệnh khởi chạy. Ảnh container (**image**) xác định tiến trình mặc định nào, hệ thống tệp nào và tham số mặc định nào hiện diện. Một bản khai báo tải công việc (**workload declaration**) quy định bao nhiêu bản sao của tạo phẩm (**artifact**) đó cần tồn tại và chính sách nào được áp dụng. Bộ điều khiển (**controller**) quan sát trạng thái hiện tại, thực thi một hành động tối thiểu nhằm thu hẹp khoảng cách tới trạng thái mong muốn (**desired state**), rồi tiếp tục quan sát. Không bước nào trong quy trình này tự động chứng minh ứng dụng đã phục vụ chính xác các yêu cầu nghiệp vụ.

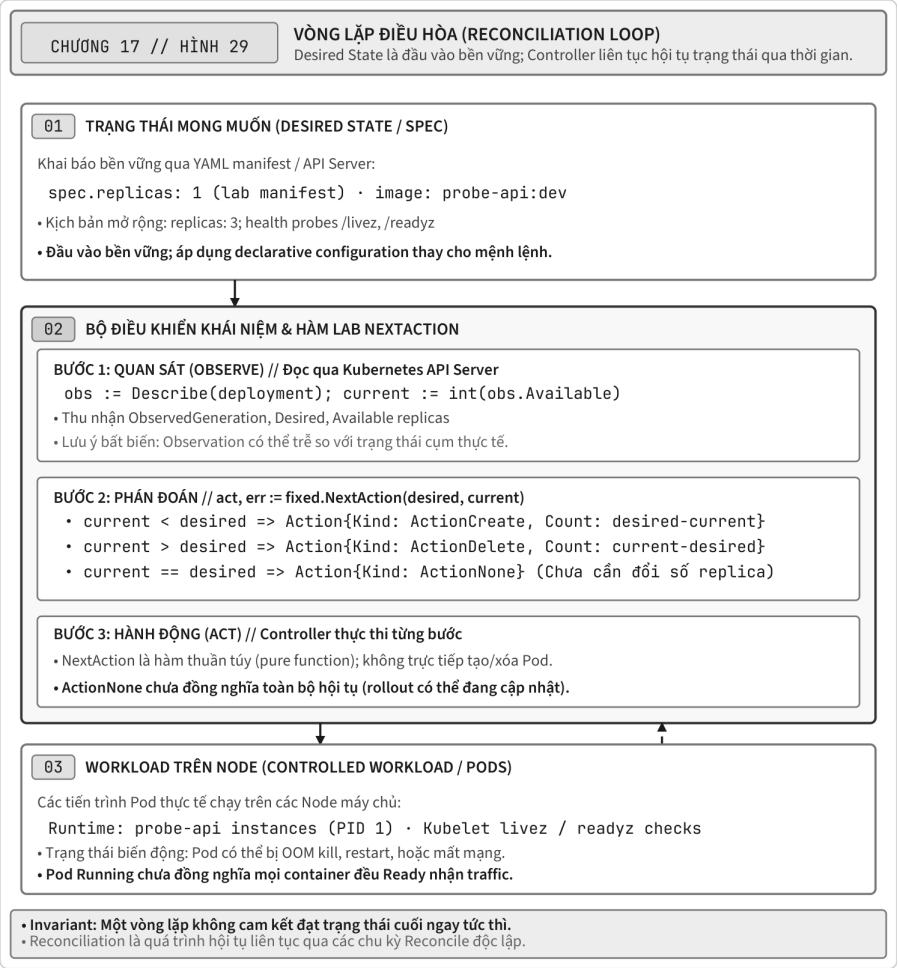
Một image chạy được chưa phải một deployment

Giả sử đội ngũ kỹ thuật đã đóng gói một ảnh container và khởi chạy thử nghiệm. Tệp nhị phân xuất nhật ký bình thường, cổng mạng mở, yêu cầu đầu tiên phản hồi mã `200`. Nhưng khi đưa cùng ảnh container đó vào cụm điều phối (**cluster**), một thể hiện bị khởi động lại liên tục do tiến trình kết thúc sớm, thể hiện khác chưa tiếp nhận lưu lượng vì chưa đạt trạng thái sẵn sàng, trong khi bản triển khai (**deployment**) vẫn đang luân chuyển thay thế Pod cũ bằng Pod mới. Nếu quy kết tất cả thành "Docker bị lỗi", ta đã đánh mất ba ranh giới kỹ thuật tách biệt:

Bảng 17 — Ba contract không nên trộn làm một

Boundary	Câu hỏi chính	Bằng chứng phù hợp
Artifact	Process nào, default args nào, file nào được mang theo?	Image config, digest, Dockerfile, SBOM nếu có.
Runtime	Process có start, nhận signal, mở listener và thoát theo contract không?	Exit status, log, health/readiness, resource observation.
Control plane	Desired state có đang được đưa gần current state không?	Spec, status, event và action của controller.

Vùng chứa (**container**) không phải là một máy ảo thu nhỏ, cũng không tự biến một tệp nhị phân thành một dịch vụ hoàn chỉnh. Đó là phương thức đóng gói hệ thống tệp và cấu hình thực thi cho một tiến trình độc lập. Lệnh `docker run` là một chỉ thị mang tính mệnh lệnh: nó yêu cầu môi trường thực thi khởi tạo một vùng chứa ngay tức thì. Ngược lại, Kubernetes vận hành ở một tầng trừu tượng khác: đối tượng có trường `spec` mô tả trạng thái mong muốn, trường `status` báo cáo trạng thái thực tế đã quan sát, và các bộ điều khiển liên tục thu hẹp khoảng cách giữa hai trạng thái đó. Vì lẽ đó, việc “đã áp dụng tệp YAML” không đồng nghĩa với việc “đã có lưu lượng được phục vụ”, cũng như trạng thái “Pod Running” hoàn toàn không bảo đảm “mọi vùng chứa bên trong đều đã sẵn sàng”.



Hình 29 — Trạng thái mong muốn là đầu vào bền vững hơn một lệnh khởi chạy tức thời. Bộ điều khiển thực thi qua nhiều vòng lặp; trạng thái cụm máy chủ liên tục biến động.

Image là process contract, không phải bản sao của laptop

Đặc tả cấu hình image OCI chứa đựng các giá trị thực thi mặc định như nền tảng phần cứng, thư mục làm việc, định danh người dùng, `Entrypoint` và `Cmd`. Với một ảnh container chạy tệp nhị phân Go, cách tiếp cận chuẩn mực là: `ENTRYPOINT` ấn định tệp thực thi cố định; `CMD` cung cấp danh sách tham số mặc định mà môi trường thực thi có thể ghi đè. Việc áp dụng cả hai ở định dạng mảng thực thi (**exec form**) giúp phân tách rõ ràng tệp thực thi và từng đối số, ngăn chặn tiến trình hệ vỏ làm sai lệch quy tắc trích dẫn chuỗi và cắt đứt đường truyền tín hiệu hệ điều hành.

```
FROM golang:1.27.1 AS build
WORKDIR /src

COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath \
  -o /out/probe-api ./cmd/probe-api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/probe-api /probe-api
USER nonroot:nonroot
ENTRYPOINT ["/probe-api"]
CMD ["--config", "/etc/probe-api/config.json"]
```

Đây là một mẫu cấu trúc tham khảo, không phải tệp Dockerfile có thể sao chép nguyên xi cho bài thực hành hiện hành: kho mã nguồn của cuốn sách không đặt `cmd/probe-api` hay `go.sum` tại thư mục gốc. Chi tiết đó quan trọng hơn vẻ ngoài trau chuốt của đoạn mã ví dụ. Một tệp Dockerfile chỉ phát huy giá trị khi đường dẫn gói phần mềm, các thành phần phụ thuộc, nền tảng mục tiêu và tài nguyên thực thi hoàn toàn khớp nối với mã nguồn thực tế. Đừng áp dụng `CGO_ENABLED=0`, ảnh distroless hay tài khoản `nonroot` như một công thức máy móc; hãy kiểm chứng xem tệp nhị phân có thực sự đòi hỏi thư viện ngôn ngữ C, chứng chỉ số CA, dữ liệu múi giờ, hệ vỏ dòng lệnh hoặc đường dẫn ghi tệp tin nào hay không.

Chỉ thị `RUN` thực thi tại thời điểm đóng gói ảnh container (**build time**) và tạo ra một lớp dữ liệu (**layer**); chỉ thị `CMD` ấn định giá trị mặc định khi vùng chứa bắt đầu khởi chạy (**container start**). Sự nhầm lẫn giữa hai thời điểm này có thể khiến các ca kiểm thử hoặc tác vụ di chuyển dữ liệu bị kích hoạt nhầm lúc đóng gói, trong khi ảnh container khi khởi chạy lại thiếu vắng tiến trình chính xác. Tương tự, thao tác sao chép mã nguồn rồi biên dịch bên trong ảnh container không tự động bảo đảm tính tái lập của mã nguồn: thẻ ảnh cơ sở (**base image tag**) có thể bị trôi phiên bản, việc tải mô-đun đòi hỏi đồ thị phụ thuộc được kiểm soát chặt chẽ, và ảnh container cuối cùng phải được định danh bằng mã băm cố định (**digest**) khi tính bất biến của danh tính quan trọng hơn một nhãn phiên bản dễ đọc.

> **Dừng để dự đoán:** Nếu môi trường thực thi ghi đè `CMD` thành `--config /run/config.json`, tệp thực thi nào vẫn được giữ nguyên? Nếu `ENTRYPOINT` được viết ở định dạng hệ vỏ (**shell form**), tiến trình nào thực sự nắm giữ mã định danh PID 1 và tín hiệu yêu cầu dừng sẽ được điều hướng qua đâu? Hãy kiểm chứng trực tiếp từ cấu hình image OCI hoặc quy tắc đặc tả của môi trường thực thi.

Container vẫn sống trên máy thật



Hình 30 — Dãy máy chủ Pleiades tại NASA Ames, ảnh chụp ngày 08/10/2008 của Marco Librero/NASA Ames Research Center, từ Wikimedia Commons. Tài nguyên thuộc phạm vi công cộng theo giấy phép PD-USGov-NASA; bản sách chuyển sang thang độ xám (**grayscale**). Bức ảnh phản ánh hạ tầng vật lý thực tế, không nhằm chứng minh NASA sử dụng Go hay Kubernetes trên hệ thống này.

Bức ảnh phòng máy chủ nhắc nhở một ranh giới thực tế dễ bị các tệp YAML che khuất: tiến trình sau cùng vẫn trực tiếp cạnh tranh tài nguyên CPU, RAM, băng thông I/O và mạng trên phần cứng vật lý. Không gian tên (**namespace**) trên Linux thiết lập những góc nhìn cô lập cho một số tài nguyên nhất định; nhóm điều khiển (**cgroup**) tổ chức và thiết lập hạn mức tài nguyên theo chính sách. Chúng không biến hạn ngạch CPU thành các

nhân vật lý bổ sung, cũng không biến giới hạn bộ nhớ thành một dung lượng vùng nhớ heap Go được bảo đảm cấp phát nguyên vẹn.

Hãy điều tra một tình huống giả định: Pod có hạn ngạch bộ nhớ (**memory limit**), dịch vụ không tăng số lượng yêu cầu, nhưng vẫn bị hệ điều hành tiêu diệt do cạn kiệt bộ nhớ (**OOM kill**) sau khi bổ sung thư viện bản địa (**native library**). Hồ sơ cấp phát vùng nhớ (**heap profile**) của Go chưa thể giải thích toàn bộ dung lượng bộ nhớ thực tế (RSS); ngoài bộ nhớ do Go trực tiếp quản lý, hệ thống còn có các khoản cấp phát bản địa, ánh xạ vùng nhớ (**mappings**) và các thành phần hệ điều hành hoạch toán theo chính sách của môi trường. Thiết lập `GOMEMLIMIT` chỉ là giới hạn mềm cho phần bộ nhớ do runtime Go kiểm soát, không phải trần giới hạn RSS hay quyền yêu cầu nhân hệ điều hành bỏ qua hạn mức cgroup. Tài liệu `runtime/debug.SetMemoryLimit` nêu rõ cả phần bị loại khỏi ngân sách, điển hình như vùng nhớ do mã nguồn bên ngoài runtime quản lý.

Vì vậy việc đọc hạn mức trong phần khai báo, lượng tài nguyên sử dụng thực tế và hồ sơ đo lường là ba bước khác nhau. Chương 10 cung cấp công cụ đo cấp phát và giải thích vai trò của `GOMAXPROCS`; số công việc Go có thể chạy song song không đồng nhất với ngân sách thời gian CPU do cgroup cấp. Tại đây, nhiệm vụ của khâu triển khai là thiết lập khoảng đệm tài nguyên phù hợp với tải và quan sát hiện tượng điều tiết (**throttling**), cạn bộ nhớ hoặc độ trễ gia tăng. Không chọn một tỷ lệ phần trăm bộ nhớ cố định cho mọi dịch vụ. Bài điều tra kết thúc khi người học chỉ rõ thành phần nào đã được đo, thành phần nào chưa được đo và cơ chế nào thực thi ngưỡng dừng, không phải sau khi thêm một biến môi trường vào tệp kê khai.

Một process làm gì khi control plane muốn dừng nó?

Chương 12 đã phân tách vòng đời máy chủ HTTP khỏi hàm `main`: tín hiệu ngắt chuyển tiếp tới ngữ cảnh (`context`) của ứng dụng, máy chủ dừng tiếp nhận kết nối mới, rồi gọi `Shutdown` để chờ các yêu cầu đang xử lý trong một khoảng thời gian ân hạn (**grace period**). Khế ước đó thuộc trách nhiệm của chương trình. Môi trường thực thi vùng chứa (**container runtime**) và tiến trình `kubelet` chỉ phát sinh thêm một sự kiện phía trên: khi đến thời điểm chấm dứt, tiến trình nhận tín hiệu theo chính sách của môi trường thực thi; Kubernetes cũng có thể chờ một khoảng thời gian ân hạn trước khi cưỡng chế dừng tiến trình. Không có lời hứa hẹn chung nào rằng mọi hàm xử lý đang chạy sẽ tự động bị hủy, mọi tiến trình con tự động biến mất, hay toàn bộ yêu cầu chắc chắn sẽ hoàn tất trọn vẹn.

Vì vậy, cú pháp `ENTRYPOINT ["/probe-api"]` mang ý nghĩa vận hành chuẩn tắc hơn `ENTRYPOINT /probe-api --config ...` khi mục đích là để tệp nhị phân trở thành tiến trình chính (PID 1). Dạng mảng thực thi (**exec form**) không tự động xử lý dừng êm dịu thay cho chương trình, nhưng nó tránh việc chèn một vỏ lệnh hệ thống (**shell**) nằm chắn giữa môi trường thực thi và tệp nhị phân. Tín hiệu `SIGTERM` vẫn chỉ là một thông điệp: ứng dụng phải chủ động quyết định cách chuyển nó thành thao tác làm cạn yêu cầu (**drain**), thiết lập hạn định thời gian (**deadline**) và trả về mã thoát phù hợp; hệ thống điều phối cần được cấu hình khoảng thời gian đủ lớn cho chính sách dừng đó.

Đây cũng là lúc xem xét lại cơ chế sẵn sàng (**readiness**) của Chương 16 theo đúng phạm vi. Đầu kiểm tra sẵn sàng trả lời câu hỏi: “Có nên đưa Pod này vào tập đích phục vụ của Service hay không?”. Nó không xác nhận image được đóng gói đúng, không thay liveness hay SLO, và không yêu cầu khởi động lại chỉ vì một phụ thuộc ở xa đang chậm. Khi Pod được đánh dấu chưa sẵn sàng, Service thông thường sẽ loại nó khỏi tập đích sau khi trạng thái được truyền tới các thành phần định tuyến. Đây không phải hàng rào tức thời cho mọi yêu cầu mới: cập nhật có độ trễ, kết nối đã thiết lập và khách truy cập thẳng Pod không tự bị chặn. Readiness thất bại không tự dừng tiến trình; chính sách khác vẫn có thể thay thế Pod.

Desired state không phải một script dài

Tệp kê khai (**manifest**) minh họa dưới đây chỉ xác lập ba ý niệm rõ ràng: định danh bất biến của gói phát hành, số lượng bản sao mong muốn và khế ước sẵn sàng phục vụ. Nó không tuyên bố rằng đây là toàn bộ cấu hình hoàn chỉnh của một dịch vụ trên môi trường sản xuất.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: probe-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: probe-api
  template:
    metadata:
      labels:
        app: probe-api
    spec:
      containers:
        - name: probe-api
          image: registry.example/probe-api@sha256:REPLACE_ME
          args: ["--config", "/etc/probe-api/config.json"]
          readinessProbe:
            httpGet:
              path: /readyz
              port: 8080
```

`replicas: 3` không có nghĩa "chạy lệnh khởi động đúng ba lần rồi xong". Đó là số lượng bản sao mong muốn (**desired count**). Bộ điều khiển (**controller**) có thể tạo Pod khi thiếu hụt, tạo bản sao thay thế (**replacement**) khi một Pod bị mất, hoặc từng bước thay thế các bản sao khi mẫu khai báo (**template**) thay đổi. Pod là đơn vị mang tính tạm thời (**ephemeral**); một Pod đã được ấn định lịch điều phối không tự động chuyển dịch sang máy chủ khác. Chính bộ điều khiển cấp cao hơn mới là thành phần khởi tạo Pod mới để khối lượng công việc liên tục tiến về trạng thái mong muốn.

Trường `image` sử dụng mã băm nội dung (**digest**) trong ví dụ vì mã băm là định danh bất biến của gói phát hành cụ thể. Nhãn phiên bản như `:latest`

chỉ là một tên gọi có thể trỏ đến ảnh container khác ở thời điểm khác; nó có thể tiện lợi cho quy trình phát triển, nhưng tuyệt đối không phải là bằng chứng khẳng định việc triển khai đang chạy đúng bản dựng nào. Ký hiệu giữ chỗ `REPLACE_ME` cố ý khiến tệp kê khai chưa thể áp dụng ngay: một mã băm giả mạo sẽ tạo ra cấu hình trông có vẻ hoàn chỉnh nhưng hoàn toàn không có gói phát hành thực tế để kéo về.

Đặc tả Pod (**pod spec**) cũng không phải là nơi để nhồi nhét mọi cấu hình trong mã nguồn. Biến môi trường, Secret, ConfigMap, ổ lưu trữ (**volume**), tài khoản dịch vụ (**service account**), yêu cầu và hạn ngạch tài nguyên cùng chính sách mạng đều có ngữ nghĩa, quyền truy cập và vòng đời riêng biệt. Khi một trường dữ liệu ảnh hưởng tới khả năng khởi động hay tính sẵn sàng phục vụ, nó là một phần của khế ước thời gian chạy bắt buộc phải được kiểm thử và quan sát; đó không phải là "chất keo YAML" để xem nhẹ khâu đánh giá mã nguồn.

Đọc reconciliation bằng một hàm nhỏ

Bộ điều khiển Kubernetes trong thực tế quan sát các đối tượng API, có bộ nhớ đệm, cơ chế thử lại, xử lý lỗi và điều phối nhiều loại tài nguyên khác nhau. Chúng ta không cần mô phỏng lại cả một cụm máy chủ trong bài thực hành. Thay vào đó, mô hình tái hiện tối thiểu giữ lại phần tư duy cốt lõi: một lần quan sát chỉ trả về hành động kế tiếp, tuyệt đối không cam kết hệ thống đã hội tụ ngay tức thì sau hành động đó.

```
type ActionKind string

const (
    ActionNone   ActionKind = "none"
    ActionCreate ActionKind = "create"
    ActionDelete ActionKind = "delete"
)

type Action struct {
    Kind  ActionKind
    Count int
}

func NextAction(desired, current int) (Action, error) {
    if desired < 0 || current < 0 {
        return Action{}, ErrNegativeReplicaCount
    }
    switch {
    case current < desired:
        return Action{
            Kind:  ActionCreate,
            Count: desired - current,
        }, nil
    case current > desired:
        return Action{
            Kind:  ActionDelete,
            Count: current - desired,
        }, nil
    default:
        return Action{Kind: ActionNone}, nil
    }
}
```

Hàm cố tình không lặp vòng `for` cho đến khi số lượng bằng nhau. Nếu nó tự gọi lệnh "khởi động" ba lần liên tiếp, nó sẽ che khuất những sự kiện thực tế diễn ra giữa các hành động: thao tác tạo mới có thể thất bại, phiên bản mới có thể chưa sẵn sàng, trạng thái hiện tại có thể bị một tác nhân khác thay đổi, hoặc bộ điều khiển cần phải áp dụng cơ chế điều tốc (**rate limit**). Trạng thái `ActionNone` không chứng minh ứng dụng đang khỏe mạnh; nó chỉ khẳng định bản chụp số lượng tức thời đưa vào hàm đang bằng với số lượng

mong muốn. Sự cẩn trọng này là một đặc tính kỹ thuật cốt lõi của mã nguồn điều phối.

Trong bộ điều khiển thực tế, trạng thái mong muốn có thể thay đổi liên tục, bộ nhớ đệm có thể bị trễ, và hành động có thể đã hoàn tất thành công trước khi bên gọi kịp nhận phản hồi. Phương thức `NextAction` trả về cùng một hành động cho cùng một đầu vào chỉ chứng minh tính tất định của thuật toán tính toán, hoàn toàn không chứng minh các hiệu ứng phụ (**side effects**) là an toàn khi lặp lại nhiều lần. Với bản chụp số lượng mong muốn 3 và hiện tại 1, việc thực thi chỉ thị "tạo thêm 2" hai lần liên tiếp có thể làm phát sinh tới 5 phiên bản hoạt động. Để lặp lại một cách an toàn, bộ điều khiển bắt buộc phải đọc lại trạng thái và thiết lập cơ chế định danh quyền sở hữu chặt chẽ cho thao tác tạo mới; sự cố quá thời gian không thể khẳng định thao tác trước đó đã diễn ra hay chưa. Chương 21 và Chương 23 sẽ kiểm chứng ranh giới này với hàng đợi công việc, dữ liệu quan sát cũ và khoảng thời gian tiến trình gặp sự cố đột ngột. Đây là bài toán về vòng lặp điều khiển và giao diện lập trình, không thuộc về cú pháp ngôn ngữ Go.

Bảng 18 — Điều một vòng reconciliation được phép kết luận

Dữ liệu quan sát	Hành động hợp lý của bộ điều hòa mẫu	Điều vẫn chưa biết
Mong muốn 3, hiện tại 1	Tạo thêm 2 phiên bản máy chủ.	Hai phiên bản đó có khởi động và sẵn sàng hay không.
Mong muốn 3, hiện tại 5	Dừng hoặc loại bỏ 2 phiên bản.	Phiên bản nào nên bị chọn, kết nối mạng có cần rút cạn (**drain**) hay không.
Mong muốn 3, hiện tại 3	Giữ nguyên số lượng bản sao.	Bản sửa đổi ảnh container, lưu lượng mạng, độ trễ, chỉ số SLO và sức khỏe tài nguyên.

Bài thực hành: Tự hoàn thành vòng lặp điều hòa một bước

Mở tệp `labs/part17-reconciliation-contract/exercise/reconcile_test.go` trước tiên. Bài tập không đòi hỏi dịch vụ nền Docker, cụm máy chủ, thông tin xác thực hay kết nối mạng. Bộ kiểm thử truyền số lượng mong muốn và hiện thời vào hàm rồi yêu cầu hành động đầu ra phải có giới hạn, tất định, và không biến đầu vào âm thành một hành động sai lệch. Bộ kiểm thử không thực thi hiệu ứng phụ, do đó không chứng minh tính lũy đẳng của một giao diện lập trình khởi tạo phiên bản. Đây là cách tách biệt phép tính toán chênh lệch khỏi hệ thống điều phối thực tế.

```
cd labs/part17-reconciliation-contract
go test -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

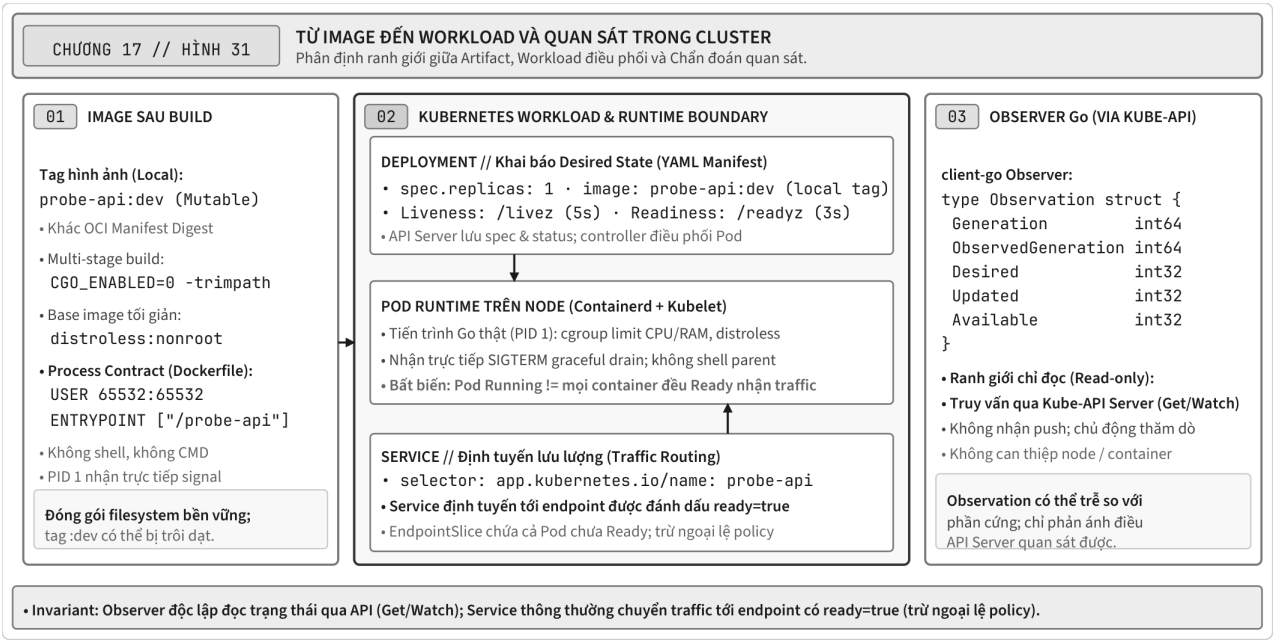
Hãy hoàn thiện phương thức `NextAction` dựa trên giao ước thiết kế, tuyệt đối không mở thư mục `fixed/` trước khi tự làm. Câu hỏi trước khi viết mã là: hành động nào phản ánh chính xác sự chênh lệch giữa số lượng mong muốn và hiện thời, và trường hợp hai số bằng nhau cần trả về giá trị gì để bên gọi không phải suy đoán mơ hồ? Sau khi các ca kiểm thử đạt kết quả tốt, hãy thử thay thế trạng thái hiện tại bằng kết quả dự kiến của hành động đó rồi gọi lại hàm. Hiện tượng gì sẽ xảy ra ở vòng lặp thứ hai? Đó là phép thử nghiệm tính hội tụ khi dữ liệu quan sát được cập nhật mới, chứ chưa phải bằng chứng về tính an toàn khi lặp lại hiệu ứng phụ.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Xác thực cả hai số lượng trước, vì phép trừ `desired - current` với số âm có thể biến một cấu hình lỗi thành hành động hợp lệ giả tạo. Sau đó trả về `create` hay `delete` với `Count` đúng bằng độ chênh lệch; trường hợp bằng nhau trả về `ActionNone` với `Count` bằng 0. Không làm biến đổi dữ liệu đầu vào, không dùng vòng lặp, không tạm dừng (**sleep**) và không cố "chờ sẵn sàng": mỗi việc đó thuộc về một ranh giới trách nhiệm khác.

Giai đoạn hai: đưa service thật vào container và cụm máy chủ

Vòng lặp `NextAction` ở trên giúp chúng ta hiểu bản chất của quy trình điều hòa trạng thái (**reconciliation**) mà chưa cần tới cụm máy chủ thực tế. Nhưng khi đưa một dịch vụ Go vào vận hành, mã nguồn không còn chạy trực tiếp trên máy của lập trình viên. Nó phải được đóng gói vào một ảnh container OCI (**OCI image**) và giao phó cho một hệ thống điều phối. Thư mục `labs/part17-container-kubernetes` dùng chính dịch vụ HTTP `probe-api` từ `labs/part16-real-signals` để thực hành các ranh giới này trên môi trường cục bộ có Docker và Kubernetes.



Hình 31 — Đối tượng Deployment khai báo khuôn mẫu Pod cùng các đầu kiểm tra, Service chọn luồng lưu lượng truy cập, môi trường thực thi trên máy chủ chạy ảnh container. Quan sát viên chỉ đọc dữ liệu thông qua API Server; sơ đồ lược bỏ các thành phần điều phối trung gian.

Trước khi bắt đầu, bài thực hành kiểm tra các công cụ bắt buộc bằng tệp kịch bản `scripts/verify-prereqs.ps1`. Nó yêu cầu các công cụ `docker`, `kubectl` và `kind` phải hiện diện trên máy tính cá nhân. Nếu môi trường hiện tại chưa được cài đặt đầy đủ các công cụ này, kịch bản sẽ dừng lại kèm thông báo chỉ dẫn rõ ràng; khi đó, các câu lệnh dưới đây là quy trình mẫu được chuẩn hóa để người học đối chiếu khi có đủ điều kiện, tuyệt đối không tự ý thay thế bằng một cụm máy chủ đám mây trả phí.

```
cd labs/part17-container-kubernetes
./scripts/verify-prereqs.ps1
```

Ảnh container là khế ước kỹ thuật của tiến trình

Tệp Dockerfile của bài thực hành áp dụng mô hình dựng nhiều giai đoạn (**multi-stage build**) để tách biệt môi trường biên dịch với môi trường thực thi. Giai đoạn đầu tiên sử dụng ảnh Go chính thức để tải mô-đun phụ thuộc và biên dịch tệp nhị phân với `CGO_ENABLED=0` cùng cờ `-trimpath` nhằm loại bỏ đường dẫn tệp hệ thống cục bộ khỏi tệp nhị phân. Giai đoạn cuối cùng chỉ sao chép duy nhất tệp nhị phân sang ảnh nền tối giản `distroless/static-debian12:nonroot`.

```
FROM golang:1.27.1 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath \
  -o /out/probe-api ./cmd/probe-api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/probe-api /probe-api
USER 65532:65532
ENTRYPOINT ["/probe-api"]
```

Với câu lệnh dưới đây, chỉ thị `ENTRYPOINT` ở định dạng mảng thực thi (**exec form**) sẽ khởi chạy tệp nhị phân trực tiếp làm tiến trình PID 1 bên trong vùng chứa; nếu bổ sung thêm cờ `--init`, tiến trình khởi tạo hệ thống sẽ đảm nhận vị trí đó. Máy chủ tiếp nhận cổng từ biến môi trường `PORT` tương tự như Chương 12 và Chương 16. Cờ `--read-only` thiết lập hệ thống tệp gốc ở chế độ chỉ đọc, không ngăn cấm thao tác ghi vào ổ lưu trữ (**volume**) hay vùng nhớ tạm (**tmpfs**) được gắn kết riêng biệt. Cờ `--cap-drop ALL` loại bỏ toàn bộ năng quyền Linux (**capabilities**), trong khi chỉ định `USER 65532:65532` ấn định danh tính tài khoản không mang đặc quyền quản trị (**root**). Những thiết lập này thu hẹp đáng kể bề mặt tấn công nhưng không thể thay thế toàn diện chính sách cô lập của vùng chứa.

```
# Chạy từ thư mục gốc của repository (minh họa)
docker build `
  -f labs/part17-container-kubernetes/Dockerfile `
  -t probe-api:dev `
  labs/part16-real-signals

docker run --rm --name probe-api `
  --read-only --cap-drop ALL `
  -p 18080:18080 -e PORT=18080 probe-api:dev
```

Ở một terminal khác, ta kiểm tra các contract vận hành:

```
curl.exe -i http://localhost:18080/readyz
curl.exe -i http://localhost:18080/probe?mode=slow
docker inspect --format '{{.Config.User}}' probe-api
docker stop -t 5 probe-api
```

Trong ảnh container của bài thực hành, lệnh `docker stop -t 5` chuyển tiếp tín hiệu `SIGTERM` tới tiến trình chính và ấn định thời gian chờ tối đa 5 giây; nếu tiến trình vẫn chưa thoát, Docker mới cưỡng chế dừng bằng `SIGKILL`. Khoảng thời gian chờ đó không cam đoan mọi yêu cầu mạng đều hoàn tất trọn vẹn: ta cần chủ động quan sát khâu làm cạn yêu cầu (**drain**) và mã trạng thái thoát của chương trình để kiểm chứng giao ước dừng êm dịu (**graceful shutdown**). Đánh đổi của ảnh tối giản (**distroless**) và hệ thống tệp chỉ đọc là gì? Vùng chứa sẽ hoàn toàn không có hệ vỏ dòng lệnh (`/bin/sh`), không chứa trình quản lý gói phần mềm, và tệp nhị phân không thể tùy tiện ghi tệp tạm thời nếu không được gắn kết một thư mục riêng biệt. Việc cắt giảm công cụ và các gói phụ thuộc giúp thu hẹp phạm vi quản lý, nhưng không đồng nghĩa tệp nhị phân, các thư viện liên kết hay cấu hình môi trường hoàn toàn miễn nhiễm với lỗ hổng bảo mật.

Cần lưu ý: nhãn `probe-api:dev` chỉ là định danh cục bộ trên máy phát triển. Nó không phải là một định danh bất biến dùng cho môi trường production; Chương 18 sẽ giải quyết bài toán định danh bằng digest.

Trạng thái mong muốn và điều tra sự cố trong cụm máy chủ

Khi chuyển từ vùng chứa đơn lẻ sang Kubernetes, ta không còn ra lệnh chạy một tiến trình mà khai báo trạng thái mong muốn (**desired state**) thông qua các tệp kê khai: `namespace.yaml`, `deployment.yaml` và `service.yaml`.

```
kind create cluster --name go-book
kind load docker-image probe-api:dev --name go-book
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl -n go-book rollout status deployment/probe-api
kubectl -n go-book get deploy,pod,service
kubectl -n go-book port-forward service/probe-api 18080:80
```

Trong `deployment.yaml`, hai probe dẫn tới hai quyết định khác nhau:

Thứ nhất là đầu kiểm tra sẵn sàng phục vụ (**readiness probe**, tại điểm cuối `/readyz`): Giải đáp câu hỏi "Liệu Pod này có sẵn sàng tiếp nhận lưu lượng truy cập từ đối tượng Dịch vụ (**Service**) ngay lúc này hay không?". Sau khi vượt quá ngưỡng thất bại cho phép, tiến trình `kubelet` sẽ thiết lập điều kiện sẵn sàng thành sai (`false`). Với đối tượng dịch vụ thông thường, điểm cuối chưa sẵn sàng sẽ bị loại bỏ khỏi danh sách phân phối lưu lượng; đối tượng lát cắt điểm cuối (**EndpointSlice**) vẫn có thể lưu giữ địa chỉ mạng nhưng đánh dấu trạng thái chưa sẵn sàng. Cấu hình dịch vụ bật cờ `publishNotReadyAddresses` là trường hợp ngoại lệ cần đối soát chính sách riêng biệt. Việc kiểm tra sẵn sàng thất bại không tự động kích hoạt thao tác khởi động lại vùng chứa.

Thứ hai là đầu kiểm tra sống còn (**liveness probe**, tại `/livez`): giám sát điều kiện mà chính sách vận hành chọn để khởi động lại container. `kubelet` đếm các lần thất bại liên tiếp theo `failureThreshold`; manifest của lab đặt ngưỡng 3, nên một lần thất bại chưa đủ. Ngưỡng có thể được đặt bằng 1: không có bảo đảm chung rằng một mẫu lỗi đơn lẻ không bao giờ dẫn tới restart. Khi đủ ngưỡng, kubelet chấm dứt container theo grace period và áp dụng restart policy. Tránh đưa lỗi phụ thuộc ở xa vào liveness nếu restart không giải quyết nguyên nhân gốc; nó có thể tạo chuỗi khởi động lại liên hoàn. Readiness và liveness phục vụ hai quyết định khác nhau.

Phần tài nguyên cũng phân định rạch ròi giữa mức yêu cầu `requests` (chỉ số mà bộ lập lịch sử dụng để tìm kiếm nút mạng phù hợp cho Pod) và mức trần `limits` (ngưỡng giới hạn tài nguyên được cưỡng chế thực thi theo cơ chế của môi trường vận hành). Với nhóm điều khiển (**cgroup**) trên Linux, khi hết ngân sách thời gian CPU trong một chu kỳ, tiến trình có thể phải chờ tới lần cấp ngân sách kế tiếp (**throttling**). Đây là giới hạn thời gian được chạy, không phải giảm xung nhịp phần cứng. Giới hạn bộ nhớ được xử lý theo cơ chế phản ứng thụ động và có thể dẫn tới việc tiến trình bị tiêu diệt do cạn bộ nhớ (**OOM kill**). Đây không phải lời cam đoan rằng mọi thời điểm vượt ngưỡng đều bị chấm dứt tức thì, cũng không phải là trần giới hạn dành riêng cho vùng nhớ heap của Go.

Để hiểu cách điều tra sự cố dựa trên bằng chứng thay vì suy đoán, lab cung cấp fixture `k8s/failure-wrong-image.yaml` với tên image không tồn tại:

```
kubectl apply -f k8s/failure-wrong-image.yaml
kubectl -n go-book rollout status `
  deployment/probe-api --timeout=45s
kubectl -n go-book get pods
kubectl -n go-book get events --sort-by=.lastTimestamp
kubectl -n go-book describe pod `
  -l app.kubernetes.io/name=probe-api
kubectl apply -f k8s/deployment.yaml
kubectl -n go-book rollout status deployment/probe-api
```

Nếu ảnh container vẫn không thể kéo về máy chủ và tệp triển khai chưa được điều chỉnh, câu lệnh `rollout status` dự kiến sẽ cạn kiệt thời gian chờ sau 45 giây. Người vận hành thiếu kinh nghiệm thường vội vã kết luận "hệ thống Kubernetes bị treo". Nhưng khi tra cứu sự kiện bằng `kubectl get events` và kiểm tra chi tiết bằng `kubectl describe pod`, mặt phẳng điều khiển (**control plane**) cung cấp các bằng chứng chẩn đoán cụ thể: lỗi không kéo được ảnh container `ErrImagePull` và cơ chế lùi bước chờ kéo lại `ImagePullBackOff`. Sau khi xác định chính xác nguyên nhân gốc, ta khôi phục trạng thái bằng cách áp dụng lại tệp `deployment.yaml` chuẩn ban đầu. Khi hoàn tất bài học, ta giải phóng cụm máy chủ cục bộ bằng lệnh: `kind delete cluster --name go-book`.

Quan sát API bằng client-go

Thay vì phụ thuộc hoàn toàn vào lệnh CLI `kubectl`, các kỹ sư Go thường cần viết các công cụ chẩn đoán hoặc tự động hóa bằng chính ngôn ngữ Go. Thư mục `labs/part17-container-kubernetes/client-observer` minh họa một chương trình chẩn đoán chỉ đọc (**read-only**) dùng thư viện `k8s.io/client-go`.

Chương trình nạp cấu hình cluster từ kubeconfig cục bộ, gọi API `Get` để lấy snapshot của Deployment `probe-api`, và mở một kênh `Watch` có giới hạn 15 giây để lắng nghe các thay đổi:

```
cd labs/part17-container-kubernetes/client-observer
go test ./...
go vet ./...
go test -race ./...
go run ./cmd/observe-deployment `
  --namespace go-book --name probe-api
```

Kết quả in ra tách biệt rõ ràng giữa trạng thái mong muốn (**desired state**) và quan sát thực tế (**current observation**):

```
get: generation=1 observed=1 desired=1 updated=1 available=1
```

Chương trình này cố ý giữ phạm vi hẹp: nó không tạo, sửa hay xóa bất kỳ tài nguyên nào. Nó minh họa sự khác biệt giữa `Generation` (số thế hệ phiên bản đặc tả mà người dùng muốn) và `ObservedGeneration` (thế hệ mà bộ điều khiển báo đã quan sát). Hai số lệch nhau cho biết trạng thái chưa theo kịp thế hệ hiện tại, không chứng minh bộ điều khiển đang tiến triển: nó có thể đang lỗi hoặc chưa chạy. Hai số bằng nhau cũng không chứng minh khối lượng công việc khỏe mạnh; còn phải đọc các điều kiện và trạng thái bản sao.

Lab chỉ in một snapshot rồi chờ sự kiện đầu tiên; nó không duy trì một dòng lịch sử liên tục. Watch hiện không truyền `ResourceVersion` từ kết quả Get, nên một thay đổi xảy ra giữa hai lời gọi có thể không được phản ánh trong dòng sự kiện mà người học thấy. Thời hạn 15 giây là ngân sách chung của cả Get và Watch, không phải thêm 15 giây sau Get. Không lấy việc watch

im lặng làm bằng chứng Deployment không thay đổi. Chương 22 sẽ dạy đầy đủ cách nối List/Watch và xử lý lịch sử đã hết hạn; không biến công cụ chẩn đoán này thành controller mới.

Bảng 19 — Các lớp kiểm soát từ binary đến cluster

Tầng kiểm soát	Nhiệm vụ chính	Bằng chứng kiểm chứng	Giới hạn không được suy diễn
Tệp nhị phân (**Binary**)	Xử lý yêu cầu, tín hiệu, cổng HTTP.	Kiểm thử đơn vị/chạy đua, nhật ký JSON.	Chưa nắm bắt hệ thống tệp hay hạn mức cgroup.
Vùng chứa (**Container**)	Đóng gói hệ thống tệp, định danh người dùng, PID 1.	Lệnh `docker inspect`, mã thoát.	Chạy được một vùng chứa không bảo đảm tính sẵn sàng.
Tải công việc (**Workload**)	Điều phối bản sao, cập nhật cuộn, đầu kiểm tra.	Trạng thái triển khai, sự kiện của Pod.	Trạng thái Pod khả dụng không chứng minh mã ứng dụng không lỗi.
Thư viện máy khách	Đọc và theo dõi trạng thái qua giao diện lập trình.	Thế hệ mong muốn, thế hệ đã quan sát.	Ảnh chụp tức thời không thay thế được bộ điều khiển.

> **Dừng để dự đoán:** Nếu một Pod có đầu kiểm tra sống còn thành công nhưng đầu kiểm tra sẵn sàng thất bại, Service có chuyển tiếp yêu cầu vào Pod đó không? Kubelet có khởi động lại Pod không? Hãy đối chiếu câu trả lời với bảng trên trước khi xem tiếp.

Khi nào phải dùng Docker, Kubernetes và client-go thật

Khi yêu cầu kỹ thuật đòi hỏi đóng gói và đẩy ảnh container, chạy kiểm thử tích hợp trong vùng chứa, triển khai khối lượng công việc, đọc trạng thái cụm máy chủ hoặc viết bộ điều khiển, lúc đó các công cụ thực tế là bắt buộc. Docker CLI/BuildKit và API Kubernetes đều có phiên bản, quyền hạn và chính sách cụm máy chủ riêng biệt; `client-go` không phải một thư viện tiện ích để nạp vào chỉ vì muốn phân tích cú pháp YAML. Trước khi tương tác với API, hãy xác định rõ ràng tài nguyên nào là nguồn chân lý duy nhất (**source of truth**), định danh nào gắn với quyền sở hữu, cơ chế thử lại có thể lặp lại hành động nào, trạng thái nào bên gọi được phép tin cậy, và thông tin xác thực nào chương trình tự động được phép sử dụng.

Điểm kết thúc của chương là một góc nhìn thoát khỏi tư duy giản lược "triển khai chỉ là chạy câu lệnh". Định danh bất biến của gói phát hành, vòng đời tiến trình và trạng thái mong muốn là ba khế ước kỹ thuật liên tiếp. Chương 16 cung cấp các tín hiệu đo lường để nắm bắt hiện trạng phiên bản; chương này cung cấp tư duy bộ điều khiển để hiểu thấu đáo vì sao phiên bản có thể bị tạo mới, thay thế hoặc rút khỏi luồng lưu lượng truy cập.

`NextAction` chỉ là một lần quan sát và một hành động kế tiếp; nó cố ý chưa có hàng đợi, cơ chế thử lại hay máy chủ API. Sau dự án tổng kết ở Chương 20, Chương 21 sẽ xây dựng các cơ chế điều phối đó trên một mô hình tái lập độc lập. Chương 22 mới gắn kết mô hình tư duy này vào `client-go`, bộ nhớ đệm và cơ chế `List/Watch` của Kubernetes. Trình tự này giúp cho Dockerfile, tệp YAML và các bộ khung công cụ không che khuất câu hỏi nền tảng: tại lần quan sát này, hành động nào là hợp lệ và còn điều gì vẫn chưa thể kết luận?

Chương 18 — Đưa thay đổi ra production

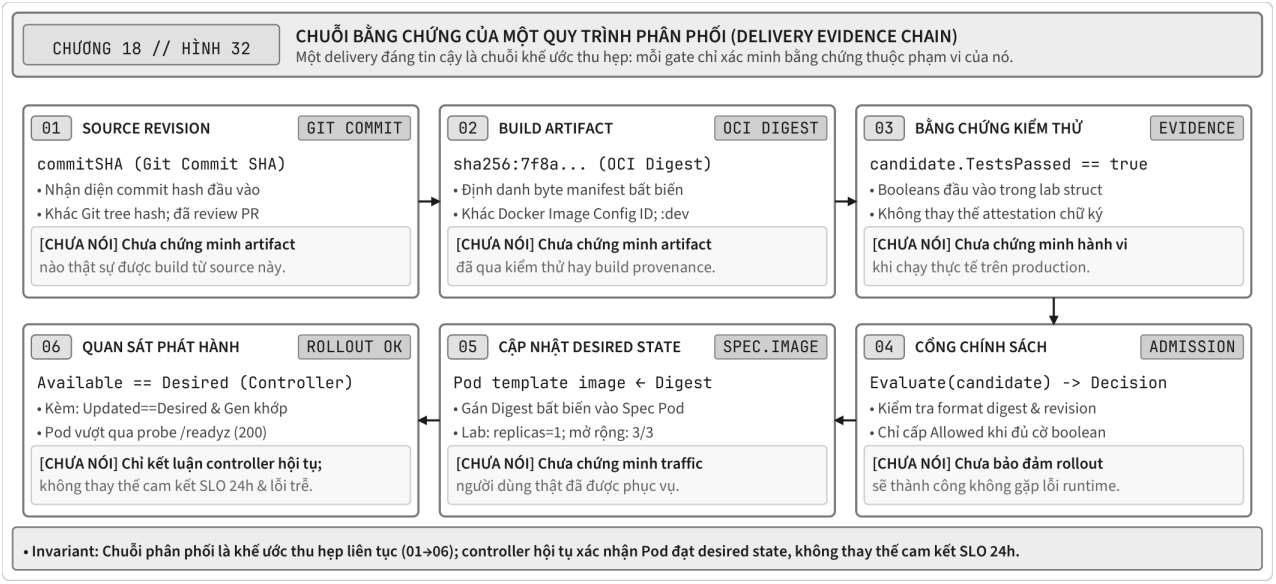
Sau Chương 17, ta có thể đặt một sản phẩm đóng gói (**artifact**) có mã băm nội dung (**digest**) vào trạng thái mong muốn (**desired state**). Tuy nhiên, một mã băm không tự trả lời sản phẩm ấy đến từ mã nguồn nào, đã vượt qua bài kiểm thử nào, tác vụ nào có thẩm quyền triển khai, hay quá trình luân chuyển hoàn tất mang ý nghĩa gì đối với người dùng cuối. Một đường ống tự động báo xanh chỉ là tín hiệu về những công đoạn nó đã thực thi; nếu các bước không xác định rõ khế ước kỹ thuật (**contract**), sắc xanh chỉ mang lại cảm giác yên tâm giả tạo thay vì cung cấp bằng chứng xác thực.

Mô hình tư duy cốt lõi của chương là: phân phối phần mềm là chuỗi bằng chứng kỹ thuật gắn kết một phiên bản mã nguồn với một sản phẩm đóng gói bất biến, sau đó cho phép thăng hạng (**promotion**) dựa trên chính sách kiểm soát nghiêm ngặt. Đường ống tự động không "đưa mã nguồn lên môi trường thực tế". Nó đóng gói sản phẩm, ghi lại định danh duy nhất của sản phẩm, thu thập bằng chứng cho các cổng kiểm soát đã xác định, và chỉ có tác vụ mang đầy đủ thẩm quyền mới được phép cập nhật trạng thái mong muốn bằng chính mã băm nội dung đó. Quy trình phục hồi (**rollback**) cũng là một hành động thăng hạng có chủ đích trở lại một sản phẩm đóng gói đã biết từ trước, hoàn toàn không phải chiếc nút hoàn tác kỳ diệu cho mọi hiệu ứng phụ trong toàn hệ thống.

Một commit xanh vẫn chưa là release

Giả sử một yêu cầu nhập mã (**pull request**) vượt qua bài kiểm thử đơn vị. Sau đó máy thực thi (**runner**) đóng gói ảnh container, tác vụ triển khai sử dụng nhãn `:main`, cụm máy chủ kéo ảnh về và quá trình luân chuyển hiển thị trạng thái `3/3 Sẵn sàng`. Chuỗi vận hành này còn rất nhiều câu hỏi cốt lõi chưa được trả lời: ảnh container đang vận hành có thực sự khớp với ảnh vừa đóng gói hay không; phiên bản mã nguồn nào đã tạo ra ảnh đó; thông tin xác thực dùng để triển khai có phạm vi phân quyền rộng đến mức nào; trạng thái `3/3` chỉ phản ánh số lượng bản sao khả dụng hay thực sự chứng minh các yêu cầu nghiệp vụ đang xử lý thành công; và nếu bước chuyển dịch cơ sở dữ liệu (**data migration**) đã diễn ra, việc phục hồi ảnh container cũ có tự động khôi phục lại dữ liệu hay không?

Đây không phải lý do để chối bỏ tự động hóa tích hợp và phân phối liên tục vì định kiến “quá phức tạp”. Đó là căn cứ xác đáng để mỗi công đoạn chỉ đưa ra một lời hứa kỹ thuật thu hẹp nhưng có thể kiểm chứng được. Phiên bản mã nguồn là định danh của dữ liệu đầu vào. Mã băm nội dung là định danh bất biến của sản phẩm đóng gói đầu ra. Bài kiểm thử là bằng chứng về hành vi trong phạm vi mà ca kiểm thử đó bao phủ. Bản ghi xuất xứ (**provenance**) là bằng chứng về mối quan hệ giữa mã nguồn và quy trình đóng gói do công cụ kiểm chứng xác nhận. Chính sách thăng hạng quyết định bằng chứng nào là đủ điều kiện để cập nhật trạng thái mong muốn. Trạng thái luân chuyển là quan sát tức thời về bộ điều khiển và các bản sao, không đồng nghĩa với cam kết mức dịch vụ (SLO).



Hình 32 — Một chuỗi phân phối đáng tin cậy được cấu thành từ các khế ước kỹ thuật thu hẹp. Mũi tên tiến trình không biến một bài kiểm thử thành công thành lời hứa bảo đảm trước mọi dạng sự cố.

Bảng 20 — Mỗi tín hiệu delivery cho phép kết luận đến đâu

Bằng chứng kỹ thuật	Điều có thể khẳng định	Điều chưa thể khẳng định
Phiên bản mã nguồn	Mã nguồn đầu vào đã được chỉ định rõ ràng.	Sản phẩm đóng gói nào thực sự được tạo ra.
Mã băm ảnh container	Một sản phẩm đóng gói bất biến xác định theo nội dung.	Sản phẩm đó đã vượt qua bài kiểm thử hay xuất xứ nào.
Kiểm thử thành công	Khế ước kỹ thuật của ca kiểm thử hiện tại đã đạt.	Toàn bộ hành vi trên môi trường thực tế, dữ liệu hay phụ thuộc ngoài.
Luân chuyển hoàn tất	Bộ điều khiển đã thỏa mãn điều kiện luân chuyển của khối công việc.	Chỉ số SLO, chi phí hạ tầng hay giao dịch nghiệp vụ của người dùng.

Nhãn phiên bản dễ đọc vẫn mang lại giá trị ghi nhớ cho con người, nhưng nhãn là một con trỏ có thể thay đổi và trỏ sang một sản phẩm đóng gói hoàn toàn khác theo thời gian. Việc thăng hạng lên môi trường thực tế bắt buộc phải mang theo mã băm nội dung mà cổng kiểm soát đã phê duyệt, không thể kích hoạt đóng gói lại "cùng một commit" ở tác vụ sau rồi hy vọng hai kết quả đầu ra sẽ giống hệt nhau. Nếu đòi hỏi khả năng tái lập cao hơn, chính sách đóng gói phải ghim chặt các dữ liệu đầu vào phù hợp, ghi nhận chính xác phiên bản công cụ và quản trị ảnh cơ sở cùng các gói phụ thuộc; mã băm

nội dung chỉ định danh kết quả đầu ra chứ không tự giải thích cách sản phẩm đó được tạo ra.

Gate không phải nghi thức, mà là policy có người chịu trách nhiệm

Một đường ống tự động được thiết kế tốt không bắt buộc mọi thay đổi mã nguồn phải chạy toàn bộ tập kiểm thử tốn kém chỉ để bảng theo dõi có thêm nhiều ô hiển thị màu xanh. Hệ thống cần thiết lập cổng kiểm soát dựa trên mức độ rủi ro và vòng phản hồi thông tin. Kiểm tra tĩnh hoặc kiểm thử đơn vị gọn nhẹ có thể thực thi sớm trên mọi thay đổi. Kiểm thử tích hợp phụ thuộc vào dịch vụ thực tế có thể chạy ở giai đoạn riêng biệt với thời hạn và bước dọn dẹp tài nguyên rõ ràng. Tác vụ đóng gói và tải lên kho chỉ kích hoạt khi mã nguồn đã vượt qua cổng kiểm soát cần thiết. Việc thăng hạng lên môi trường thực tế đòi hỏi định danh bất biến, bằng chứng đã được xác minh, phạm vi phân quyền thu hẹp và phương án phục hồi đã được diễn tập cẩn trọng.

Hạn ngạch chi phí (**cost guardrail**) cũng là một khế ước kỹ thuật bắt buộc. Một tác vụ tích hợp có thể khởi tạo cụm máy chủ tạm thời, dung lượng kho lưu trữ hoặc lưu lượng mạng đi ra ngoài; tất cả đều phải có chủ thể chịu trách nhiệm, thời hạn tối đa, giới hạn số lượng tác vụ đồng thời, quy trình thu hồi tài nguyên và giới hạn phạm vi truy cập. Chi phí tốn kém không tự động biến một tác vụ thành sai trái, nhưng một quy trình báo xanh cũng không thể hợp thức hóa những tài nguyên thiếu vòng đời quản lý. Hãy chủ động đo lường chi phí của từng giai đoạn nặng nề trước khi tối ưu hóa, tương tự như cách đặt phép đo chuẩn trước khi can thiệp vào mã nguồn ở Chương 10.

> **Dừng để dự đoán:** Nếu tác vụ đóng gói (**build job**) thành công nhưng không ghi mã băm nội dung (**digest**) vào bản ghi thăng hạng, tác vụ triển khai (**deploy job**) đang chứng minh điều gì khi nó chỉ kéo nhãn `:main`? Nếu công cụ quét bảo mật báo không có lỗ hổng nhưng ảnh container mang đi triển khai lại là kết quả của một lần dựng khác, liệu lượt quét đó đã đánh giá đúng gói phát hành chưa?

Không có cổng kiểm soát (**gate**) chung áp dụng cho mọi sản phẩm. Một dịch vụ thanh toán có thể đòi hỏi quy trình phê duyệt khắt khe và kiểm thử khế ước khác xa một công cụ dòng lệnh (CLI) nội bộ. Điều bắt buộc phải giữ vững là chính sách không bao giờ mở khi gặp lỗi (**fail-closed**, tức không cho phép vượt qua mặc định khi thiếu chứng cứ): bằng chứng thiếu hụt hoặc không hợp lệ phải dẫn tới quyết định từ chối thăng hạng môi trường kèm lý do giải thích minh bạch, chứ không âm thầm đổi nhãn phiên bản hay lạm dụng thông tin xác thực quyền cao để "chạy cho xong".

Quyền deploy là capability, không phải biến môi trường tiện tay

Tiến trình thực thi của hệ thống CI có thể biên dịch mã nguồn và chạy các hành động mà quy trình tự động (**workflow**) cho phép, do đó thông tin xác thực để triển khai tuyệt đối không nên hiện diện tràn lan ở mọi tác vụ. Với GitHub Actions, mã định danh `GITHUB_TOKEN` nên bắt đầu bằng đặc quyền đọc tối thiểu rồi mới nâng quyền cục bộ tại tác vụ thực sự cần thiết. Khóa bảo mật dài hạn của nhà cung cấp đám mây được lưu vào biến bí mật (**secret**) là một ranh giới rủi ro lớn: biến bí mật có thể bị lạm dụng ngoài ý muốn, cơ chế che giấu nhật ký không phải giải pháp bảo vệ tuyệt đối, và chính sách kiểm soát trên đám mây mới là nơi quyết định mã xác thực làm được những gì.

Xác thực liên kết OpenID Connect (OIDC) là giải pháp phân quyền năng lực (**capability**) chính xác cho tác vụ thực sự cần. Quy trình tự động xin token định danh (**identity token**); nhà cung cấp đám mây kiểm tra các khẳng định (**claims**) và điều kiện tin cậy rồi cấp mã truy cập ngắn hạn cho tác vụ. Cơ chế này không tự động trao quyền triển khai: chính sách tin cậy vẫn bắt buộc phải giới hạn chặt chẽ theo kho mã nguồn, nhánh phát triển, môi trường, vai trò (**role**) và tài nguyên cụ thể. Quyền `id-token: write` chỉ cho phép tác vụ xin mã OIDC, không đồng nghĩa được phép sửa đổi tài nguyên đám mây. Đó là lý do việc "bật OIDC" không thể thay thế cho khâu đánh giá cẩn trọng chính sách IAM.

```
permissions:
  contents: read

jobs:
  deploy-production:
    permissions:
      contents: read
      id-token: write
    environment: production
```

Đoạn YAML trên chỉ mô tả ý định phân định quyền hạn, chưa phải một quy trình triển khai hoàn chỉnh có thể vận hành: nó chưa định nghĩa vai trò trên đám mây, khẳng định đối tượng (**subject claim**), quy tắc phê duyệt, mã băm nội dung, phiên bản hành động hay câu lệnh thực thi. Tuyệt đối không sao chép đoạn mã rồi tùy tiện thêm khóa quản trị viên (**admin**) để vượt qua bước kiểm tra tin cậy. Thiết kế an toàn phải bắt đầu từ tài nguyên mà tác vụ được phép chỉnh sửa, danh tính quy trình được đám mây tin tưởng, và điều kiện cụ thể khiến yêu cầu cấp mã bị từ chối.

Promotion phải giữ identity của artifact

Bài lab không thực hiện ký ảnh container và không kết nối trực tiếp kho lưu trữ bên ngoài. Nó tập trung vào ranh giới ra quyết định: ứng viên phát hành (**candidate**) phải có mã băm nội dung hợp lệ, mã phiên bản để truy vết, bằng chứng kiểm thử và bằng chứng xuất xứ; thiếu bất kỳ điều kiện nào cổng kiểm soát sẽ trả về `Decision` từ chối, trong khi mã băm sai định dạng được coi là dữ liệu đầu vào không hợp lệ. Phân biệt rõ hai trường hợp này giúp bên gọi không nhầm lẫn giữa một cấu hình sai sót nghiêm trọng với một "bản phát hành bị từ chối theo quy trình thông thường".

```
type Candidate struct {
    Digest             string
    Revision           string
    TestsPassed        bool
    ProvenanceVerified bool
}

type Decision struct {
    Allowed bool
    Digest  string
    Reason  string
}

func Evaluate(candidate Candidate) (Decision, error) {
    if !validDigest(candidate.Digest) {
        return Decision{}, ErrInvalidDigest
```

```
    }
    if strings.TrimSpace(candidate.Revision) == "" {
        return Decision{}, ErrMissingRevision
    }
    if !candidate.TestsPassed {
        return Decision{Reason: ReasonTestsNotPassed}, nil
    }
    if !candidate.ProvenanceVerified {
        return Decision{Reason: ReasonProvenanceMissing}, nil
    }
    return Decision{
        Allowed: true,
        Digest:  candidate.Digest,
    }, nil
}
```

`ProvenanceVerified` ở đây là dữ liệu đầu vào đã được bộ kiểm chứng khác xác thực; trường boolean này **không** mô hình hóa chữ ký số, gốc tin cậy, định danh của bên tạo dựng hay chính sách tiếp nhận (**admission policy**). Đây là chủ ý sư phạm: trước khi tiếp cận các bộ khung chứng thực (**attestation framework**) hay giao diện API của kho lưu trữ, chúng ta cần thấy rõ quyết định thăng hạng môi trường phải bảo toàn mã băm nội dung và tuân thủ nguyên tắc đóng an toàn (**fail-closed**) khi thiếu chứng cứ. Một cấu

trúc nhỏ trong bài học không thể thay thế cho toàn bộ hệ thống chuỗi cung ứng thực tế.

Rollback là một release khác, không phải máy quay thời gian

Đối tượng Kubernetes Deployment lưu giữ phiên bản chỉnh sửa (**revision**) cho mỗi thay đổi trong khuôn mẫu Pod và hỗ trợ quay lui (**rollout undo**) về phiên bản trước. Tuy nhiên, phiên bản chỉnh sửa không thể bao quát toàn bộ trạng thái bên ngoài khuôn mẫu Pod: việc thay đổi số lượng bản sao `replicas` không tạo phiên bản mới, lịch sử triển khai có thể bị dọn dẹp theo hạn mức `revisionHistoryLimit`, cấu hình bên ngoài có thể đã bị sửa đổi, trong khi việc di chuyển dữ liệu (**database migration**) và các thông điệp đã gửi vào hàng đợi sẽ không tự động hoàn tác. Do đó, tuyên bố "có khả năng phục hồi về bản cũ" phải được định nghĩa bằng các điều kiện cụ thể: **phục hồi gói phát hành/khuôn mẫu nào, trên môi trường nào, với khế ước dữ liệu và tác dụng phụ ra sao?**

Một tài liệu hướng dẫn vận hành phục hồi (**rollback runbook**) cần tối thiểu: tiêu chí kích hoạt, mã băm/phiên bản đã được xác nhận ổn định, người chịu trách nhiệm có thẩm quyền phê duyệt, câu lệnh hoặc quy trình tự động hóa đã được kiểm nghiệm, bằng chứng giám sát sau phát hành, và phương án xử lý dữ liệu. Nếu bản phát hành mới chỉ thay đổi tệp nhị phân phi trạng thái (**stateless**), thao tác phục hồi có thể đơn giản là thăng hạng lại mã băm cũ rồi theo dõi tiến trình triển khai. Nếu bản phát hành thay đổi lược đồ dữ liệu theo hướng phá vỡ tính tương thích, phục hồi có thể đòi hỏi cờ tính năng (**feature flags**), chiến lược di chuyển dữ liệu hai pha (**expand/contract**) hoặc tạm dừng dịch vụ thay vì vội vã chạy lệnh đảo ngược (**undo**).

```
rollback request
  -> chọn artifact/revision đã biết
  -> kiểm tra data và blast radius policy
  -> promote digest cũ
  -> quan sát rollout + signal
  -> ghi lại quyết định và kết quả
```

Đừng thiết lập phục hồi tự động chỉ vì một chỉ số giám sát đột ngột báo đỏ nếu chính sách chưa xác định rõ cửa sổ thời gian quan sát, chủ thể sở hữu và

chi phí chấp nhận cảnh báo dương tính giả (**false-positive cost**). Nhưng cũng đừng biến "phục hồi thủ công" thành việc một cá nhân tự ý gõ nhãn phiên bản bất kỳ trong lúc xảy ra sự cố. Cùng định danh bất biến, cổng kiểm soát và ranh giới phân quyền của quy trình phát hành xuôi bắt buộc phải được duy trì nghiêm ngặt khi thực hiện quy trình quay lui.

Điển cứu: triển khai báo xanh nhưng người dùng vẫn lỗi

Một bản phát hành mang mã băm `sha256:abc...` đã vượt qua bài kiểm thử đơn vị và kiểm thử tích hợp, xuất xứ đã được xác minh đầy đủ, rồi được triển khai chính xác theo mã băm nội dung. Bộ điều khiển Deployment báo hoàn tất: các Pod mới đã ở trạng thái sẵn sàng phục vụ. Vài phút sau, chỉ số cam kết mức dịch vụ (SLI) về tính sẵn sàng tụt dốc vì cấu hình trên môi trường thực tế trỏ tới điểm cuối của dịch vụ phụ thuộc đã bị ngừng hoạt động. Không có bằng chứng nào ở đây mâu thuẫn với nhau:

Một là, bằng chứng đóng gói và thăng hạng chỉ ra chính xác sản phẩm nào và cổng kiểm soát nào đã được áp dụng.

Hai là, trạng thái triển khai chỉ khẳng định bộ điều khiển đã thay thế các bản sao đúng theo chính sách luân chuyển.

Ba là, các chỉ số đo lường, nhật ký ghi vết và phân tích phân tán từ Chương 16 phản ánh luồng tương tác của người dùng đang gặp lỗi và hỗ trợ khoanh vùng nhóm phiên bản hoặc cấu hình bị sai lệch.

Hành động xử lý đúng đắn không phải là vội vã kết luận "hệ thống CI vô dụng" hay "Kubernetes đánh lừa chúng ta". Đội ngũ kỹ sư cần kiểm tra bán kính ảnh hưởng sự cố (**blast radius**), cấu hình của ứng viên phát hành và khế ước dữ liệu; nếu việc phục hồi về mã băm cũ khôi phục được khả năng phục vụ mà không làm xung đột dữ liệu, họ sẽ thăng hạng mã băm cũ theo đúng quy trình hướng dẫn. Sau sự cố, cổng kiểm soát cần được bổ sung tại chính ranh giới còn thiếu hụt: cơ chế kiểm tra tính hợp lệ của cấu hình hoặc môi trường tích hợp sát với thực tế hơn, chứ không nhất thiết là chèn thêm một lượt quét bảo mật không liên quan.

Bài thực hành: Tự viết quyết định tiếp nhận đóng khi lỗi (fail-closed)

Mở tệp `labs/part18-promotion-evidence/exercise/admission_test.go` trước. Bài kiểm thử không cần kết nối mạng, kho lưu trữ ngoài, biến bí mật hay thư viện mật mã. Nó yêu cầu ta tự phân định rõ ràng sự khác biệt giữa một ứng viên có cấu trúc dữ liệu bị lỗi (**malformed**) với một ứng viên hợp lệ về hình thức nhưng chưa đủ bằng chứng phê chuẩn. Phần bài tập ban đầu sẽ báo lỗi đỏ vì hàm `Evaluate` chưa được viết; phần mã hoàn chỉnh giữ lại hiện thực cô đọng để kiểm chứng các bất biến trước khi phụ thuộc vào bất kỳ nhà cung cấp dịch vụ CI nào.

```
cd labs/part18-promotion-evidence
go test -tags exercise ./exercise
go test ./fixed
go vet ./fixed
go test -race ./fixed
```

Sau khi hoàn thành, hãy thử viết thêm một ca kiểm thử mới: nếu trường `TestsPassed` nhận giá trị phủ định nhưng xuất xứ hợp lệ, liệu biến quyết định `Decision` có được phép giữ lại mã băm nội dung hay không? Hãy lựa chọn chính sách kiểm soát và giải thích rõ ràng lý do kỹ thuật. Bài thực hành hiện tại chọn phương án không giữ lại mã băm trong quyết định bị từ chối để bên gọi không vô tình lấy định danh đó mang đi triển khai; một chính sách khác chỉ được coi là hợp lệ khi được ghi nhận minh bạch và có ca kiểm thử độc lập đi kèm.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Hãy thẩm định mã băm nội dung rồi đến phiên bản mã nguồn trước vì chúng là khế ước dữ liệu đầu vào. Sau đó kiểm tra từng cổng điều kiện và trả về quyết định từ chối bình thường mà không phát sinh lỗi; `error` chỉ dành riêng cho các ứng viên có dữ liệu bị lỗi hình thức (**malformed**). Chỉ quyết định được chấp thuận mới được phép mang theo mã băm nội dung. Hàm `validDigest` chấp nhận đúng tiền tố `sha256:` cùng 64 ký tự thập lục phân viết thường; đây là kiểm tra định dạng cú pháp, không phải xác thực nội dung thực tế có tồn tại hay kiểm tra chữ ký số.

Giai đoạn hai: chuỗi chuyển giao từ commit đến cổng phê duyệt

Thuật toán thẩm định `Evaluate` ở trên làm sáng tỏ các điều kiện cần thiết để phê duyệt một quyết định thăng hạng môi trường. Tuy nhiên trong thực tế vận hành, các bằng chứng ấy không xuất hiện đồng thời trong một hàm xử lý đơn lẻ; chúng được sinh ra qua từng chặng độc lập của một quy trình phân tán. Dự án `labs/part18-workflow-delivery` kết nối nền tảng lý thuyết này vào một quy trình hoàn chỉnh gồm ba cấu phần: tệp quy trình tự động mẫu có thể khảo sát (**specimen**), công cụ dòng lệnh `promote-gate` viết bằng Go vận hành trực tiếp trong đường ống CI, và cấu hình Terraform thiết lập cầu nối phân quyền OpenID Connect (OIDC) với hạ tầng đám mây.

Để tránh những ngộ nhận sai lệch trong vận hành, chuỗi phân phối kỹ thuật phải phân định rõ ràng bảy lớp thông tin:

Lớp thông tin	Bản chất kỹ thuật	Vai trò trong quy trình phân phối
Phiên bản mã nguồn	Mã băm commit Git của mã nguồn đầu vào (ví dụ: `bcb3fe0`).	Xác định chính xác phiên bản mã nguồn cần đóng gói.
Định danh ảnh cục bộ	Nhãn cục bộ hoặc mã băm cấu hình (`.Id`) trên máy đóng gói.	Mang tính tạm thời cục bộ, không dùng để ra quyết định trên môi trường thực tế.
Mã băm bản kê OCI	Mã băm nội dung bản kê ảnh container (`sha256:...`).	Ghim chặt nội dung bản kê thay vì nhãn phiên bản có thể bị trôi. Ảnh container đa nền tảng còn có mã băm chỉ mục (**index digest**); đường ống tự động phải phân định rõ đang ghim chỉ mục hay bản kê đơn lẻ.
Bản ghi xuất xứ	Tài liệu chứng thực ghi nhận cam kết mã nguồn, công cụ và quy trình.	Cung cấp bằng chứng chuỗi cung ứng cho cổng kiểm soát thẩm định.
Bằng chứng xác thực	Kết quả xác minh chữ ký số của bản ghi xuất xứ từ công cụ chuyên trách.	Căn cứ kỹ thuật xác thực, loại bỏ cờ điều kiện giả mạo.
Quyết định thăng hạng	Quyết định phê duyệt hoặc từ chối đóng khi gặp lỗi (**fail-closed**).	Cổng chặn ngăn ngừa việc triển khai khi thiếu hụt bằng chứng.
Triển khai thực tế	Cập nhật trạng thái mong muốn của khối công việc bằng mã băm đã duyệt.	Thực thi đưa phiên bản đã phê chuẩn vào vận hành chính thức.



Hình 33 — Quy trình tự động truyền định danh và bằng chứng qua các tác vụ; quyền OIDC và môi trường triển khai không tự động thay thế cho việc cổng kiểm soát thẩm định ứng viên phát hành. Chỉ quyết định được phê duyệt mới được phép cập nhật trạng thái mong muốn bằng mã băm nội dung.

Workflow GitHub Actions mẫu và kiểm tra repo multi-module

Tệp `labs/part18-workflow-delivery/workflows/delivery.yaml` là tài liệu nghiên cứu và kiểm tra mẫu (runnable specimen), không phải workflow đang kích hoạt trong `.github/workflows/`. Nó minh họa các nguyên tắc thiết kế cốt lõi:

Một là, kiểm tra chuẩn xác từng module Go: Kho mã nguồn của chúng ta bao gồm nhiều module độc lập và không sử dụng tệp `go.work` ở thư mục gốc. Việc thực thi `go test ./...` tại thư mục gốc sẽ thất bại hoặc không mang ý nghĩa. Tác vụ `verify` sử dụng chỉ thị `working-directory` để chạy kiểm thử, kiểm tra cú pháp (**vet**), và công cụ phát hiện tương tranh (**race detector**) độc lập cho từng module nằm trong chuỗi phân phối: `labs/part16-real-signals` (dịch vụ mẫu) và `labs/part18-workflow-delivery` (cổng kiểm soát).

Hai là, quyền mặc định chỉ đọc: Khai báo `permissions: { contents: read }` ở cấp cao nhất. Quyền `id-token: write` chỉ được mở duy nhất ở job deploy.

Ba là, ghim hành động tự động bằng mã băm commit bất biến: Thay vì dùng nhãn phiên bản có nguy cơ bị trôi lệch, quy trình tự động ghim chặt mã băm commit 40 ký tự đầy đủ của từng Action.

Bốn là, không đánh đồng mã định danh ảnh cục bộ với mã băm Manifest: Trường `.Id` mà lệnh kiểm tra (`inspect`) của bài lab lấy ra chính là mã băm cấu hình (**config digest**), không phải mã băm manifest dùng để kéo ảnh từ kho lưu trữ. Bản kê OCI Manifest và mã băm tương ứng hoàn toàn có thể được tạo cục bộ theo bố cục OCI; việc đẩy ảnh lên kho không phải là điều kiện tiên quyết để mã băm tồn tại. Đường ống CI/CD này trích xuất định danh phục vụ thăng hạng từ siêu dữ liệu Buildx/kho lưu trữ và bắt buộc phải phân biệt giữa bản kê của một nền tảng đơn lẻ với chỉ mục ảnh (**image index**) hỗ trợ đa nền tảng.

```
# Trích riêng verify.steps; workflow đầy đủ nằm trong lab.
# YAML scalar xuống dòng, SHA vẫn là một chuỗi liên tục.
- uses:
    actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
- uses:
    actions/setup-go@3041d5d36e89ee602123d24268e0ec2d978a6375
  with:
    go-version: '1.27.1'
- name: Verify Service Module
  working-directory: labs/part16-real-signals
  run: |
    go test -v ./...
    go vet ./...
    go test -race ./...
- name: Verify Gate Module
  working-directory: labs/part18-workflow-delivery
  run: |
    go test -v ./...
    go vet ./...
    go test -race ./...
```

Thực thi cổng phê duyệt bằng Go và nguyên tắc đóng khi lỗi

Để biến policy admission thành một chốt chặn tự động trong pipeline, thư mục `labs/part18-workflow-delivery/cmd/promote-gate` cung cấp một công cụ dòng lệnh nhỏ gọn viết bằng Go. Công cụ này nhận các tham số đầu vào và trả về mã thoát (exit code) chuẩn của hệ điều hành:

```
cd labs/part18-workflow-delivery
go test -v ./...
go vet ./...
go test -race ./...
```

Trong đường ống CI/CD, giá trị `tests-passed=true` được suy diễn hợp lệ vì tác vụ kiểm tra `verify` phía trước đã hoàn tất thành công trong đồ thị phụ thuộc (`needs: verify`). Tuy nhiên, `provenance-verified` chỉ có thể mang giá trị `true` khi hệ thống đã có một bước xác thực chữ ký số chuyên trách. Nếu giai đoạn này chưa có bước thẩm định chứng thực xuất xứ (**attestation**) thật, ta tuyệt đối không truyền cờ giả mạo `--provenance-verified=true` (tránh tạo ảo tưởng bằng chứng, tức **evidence theater**). Cổng kiểm soát bắt buộc phải từ chối theo nguyên tắc đóng an toàn (**fail-closed**):

```
# Định danh digest mẫu 64 ký tự hex
$DIGEST = "sha256:0123456789abcdef0123456789abcdef" + `
          "0123456789abcdef0123456789abcdef"
go build -o promote-gate.exe ./cmd/promote-gate

# 1. Thiếu provenance: gate từ chối fail-closed (Exit 1)
./promote-gate.exe `
  --digest $DIGEST `
  --revision "bcb3fe0" `
  --tests-passed=true `
  --provenance-verified=false

# 2. Đạt chuẩn khi có đủ cả test và provenance (Exit 0)
./promote-gate.exe `
  --digest $DIGEST `
```

```
  --revision "bcb3fe0" `
  --tests-passed=true `
  --provenance-verified=true

# 3. Trượt test: gate từ chối (Exit 1)
./promote-gate.exe `
  --digest $DIGEST `
  --revision "bcb3fe0" `
  --tests-passed=false `
  --provenance-verified=true

# 4. Định danh trôi nổi :latest: lỗi đầu vào (Exit 2)
./promote-gate.exe `
  --digest ":latest" `
  --revision "bcb3fe0"
```

Khi truyền `--digest ":latest"`, chương trình dừng ngay với thông báo: `invalid candidate: digest must be a lowercase sha256 digest` và trả mã lỗi 2. Các lệnh trên chạy binary vừa build, nên `$LASTEXITCODE` là mã của chương trình: 0 cho allow, 1 cho deny, 2 cho input lỗi. Nếu thay bằng `go run`, công cụ Go chỉ trả mã khác 0 khi chương trình lỗi; không giữ nguyên mã thoát 2 cho shell (chẩn đoán có thể ghi `exit status 2`). Hậu tố `.exe` và cách gọi trên dành cho PowerShell/Windows; trên Linux build tên không hậu tố rồi gọi `./promote-gate`.

Đặc biệt, trong đường ống CI/CD, khi công cụ `promote-gate` trả về mã lỗi (khác 0) do thiếu dữ liệu xuất xứ hoặc trượt kiểm thử, tác vụ thăng hạng phải dừng lại ngay lập tức. Mọi cơ chế nuốt lỗi (như `|| echo` hay `continue-on-error`) nhằm để bước triển khai tiếp tục chạy đều là hành vi "diễn kịch bằng chứng": dựng cổng kiểm soát cho có hình thức nhưng vẫn âm thầm phát hành mã nguồn chưa qua kiểm chứng. Ứng viên phát hành bị cổng từ chối đồng nghĩa với việc tuyệt đối không có bất kỳ lệnh triển khai nào được phép kích hoạt.

Cầu nối AWS OIDC và Terraform: nguyên tắc phân quyền tối thiểu nghiêm ngặt

Thư mục `labs/part18-workflow-delivery/terraform/` cung cấp một cấu hình Terraform minh họa mô hình liên kết danh tính OpenID Connect (OIDC) giữa GitHub Actions và AWS IAM trong đường ống này nhằm sử dụng thông tin xác thực tạm thời thay cho các khóa truy cập dài hạn lưu trữ trong hệ thống CI.

Cần hiểu đúng ranh giới của các cơ chế phân quyền trong cấu hình này:

Thứ nhất, khóa chặt danh tính: chính sách tin cậy (**trust policy**) sử dụng toán tử `StringEquals` với định danh đối tượng (**subject**) thực tế của tác vụ triển khai, truyền qua biến Terraform bắt buộc `github_oidc_subject`. Với đối tượng mặc định có định dạng `repo:OWNER/REPO:environment:production`, điều kiện `sub` kiểm tra kho mã nguồn và môi trường; nó không tự động khóa nhánh. Bài lab bổ sung điều kiện AWS riêng biệt trên khẳng định `ref`, mặc định khớp với `refs/heads/main`; quy trình tự động cũng ngăn chặn tác vụ triển khai ngoài nhánh `main`. Đây là hai cổng kiểm soát hoàn toàn độc lập. Môi trường triển khai trên GitHub vẫn cần thiết lập quy tắc nhánh/nhãn và phê duyệt phù hợp. Nếu hạ tầng đám mây không hỗ trợ khẳng định `ref` riêng, việc tùy biến chuỗi đối tượng chứa nhánh là một giải pháp khác nhưng bắt buộc phải xác thực khẳng định thực tế. Chuỗi đối tượng của kho mã nguồn có thể dùng định danh bất biến chứa mã số định danh; các kho mã nguồn lâu năm có thể chọn định dạng đó, vì vậy bài lab không tự suy đoán định danh đối tượng từ tên kho mã nguồn.

Thứ hai, phân định tài nguyên theo từng hành động cụ thể: trong chính sách của bài lab, mệnh đề `ecr:GetAuthorizationToken` bắt buộc phải dùng `Resource = ["*"]` vì hành động này không hỗ trợ định danh tài nguyên cụ thể (ARN ở cấp độ tài nguyên). Đó không phải trường hợp ngoại lệ duy nhất trong toàn bộ hệ thống phân quyền AWS IAM. Các hành động ECR còn lại trong chính sách đều hỗ trợ giới hạn theo ARN của kho lưu trữ; các hành động trên AWS App Runner được giới hạn bằng ARN của dịch vụ tương ứng. Luôn tra cứu tài liệu tham khảo phân quyền dịch vụ cho từng hành động

trước khi thiết lập tài nguyên và điều kiện kiểm tra; cấu hình ARN thu hẹp trong một ví dụ cụ thể không thể coi là quy tắc áp dụng cho mọi quyền hạn.

Thứ ba, tên kho ECR và tên IAM role có hợp đồng riêng. `ecr_repository_name` kiểm độ dài và từng thành phần đường dẫn theo pattern của API ECR, không chỉ xem các ký tự có viết thường hay không: một tên như `team//api` vẫn sai. Biến `deployer_role_name` giữ tên role riêng với giới hạn IAM, vì tên kho có dấu `/` không thể dùng nguyên xi làm tên role. Không suy từ việc Terraform nhận biến rằng tài nguyên đã tồn tại hay caller có quyền thao tác.

```
# Trích đoạn từ terraform/main.tf
data "aws_caller_identity" "current" {}

condition {
  test     = "StringEquals"
  variable = "token.actions.githubusercontent.com:sub"
  values   = [var.github_oidc_subject]
}

condition {
  test     = "StringEquals"
  variable = "token.actions.githubusercontent.com:ref"
  values   = [var.github_deploy_ref]
}
```

Lưu ý: cấu hình Terraform này là tài liệu và mã nguồn kiểm tra cú pháp mẫu. Nó không chứng minh vai trò IAM đã được đảm nhận (**assume role**) thành công, không chứng minh tài nguyên ECR/App Runner thực tế tồn tại, và không chứng minh lệnh triển khai đã hoàn tất. Máy hiện tại không cài `terraform` hay `aws`; ta giữ blocker này rõ ràng, không tự cài và không tạo tài nguyên cloud thật.

Bảng 21 — Bảy tầng bằng chứng trong quy trình delivery có trách nhiệm

Tầng bằng chứng	Bản chất kỹ thuật	Trách nhiệm kiểm chứng	Giới hạn không được suy diễn
Phiên bản mã nguồn	Mã băm cam kết Git.	Đồ thị cam kết Git.	Chưa chứng minh mã nguồn biên dịch thành công.
Kiểm tra hợp chuẩn	`go test`, `go vet`, `go test -race`.	Đồ thị phụ thuộc của máy thực thi CI.	Chỉ bao phủ phạm vi các ca kiểm thử hiện hành.
Định danh ảnh cục bộ	Mã băm cấu hình (`.Id`).	Tiến trình nền Docker cục bộ.	Không dùng làm định danh để kéo ảnh từ xa.
Mã băm bản kê OCI	Mã băm bản kê ảnh container (`sha256:`).	Siêu dữ liệu đóng gói hoặc kho lưu trữ.	Chưa chứng minh vùng chứa chạy đúng logic nghiệp vụ.
Bản ghi xuất xứ	Siêu dữ liệu chứng thực đóng gói.	Công cụ kiểm chứng chuyên trách (Cosign hoặc GitHub).	Thiếu khâu kiểm chứng thì không thể coi là đã xác thực.
Cổng phê duyệt	Công cụ dòng lệnh Go kiểm tra đóng an toàn (**fail-closed**).	Chính sách tiếp nhận thăng hạng.	Không thay thế được môi trường vận hành thực tế.
Triển khai môi trường	Cập nhật trạng thái mong muốn của cụm máy chủ.	Bộ điều khiển của hệ thống điều phối.	Bản sao sẵn sàng không bảo đảm cam kết mức dịch vụ (SLO) nghiệp vụ.

> **Dừng để dự đoán:** Nếu một đường ống CI tự động gán cờ `--provenance-verified=true` mà không xác thực chữ ký, điều gì sẽ xảy ra nếu kẻ tấn công tráo đổi ảnh container bằng một ảnh độc hại có cùng nhãn? Vì sao đó là tạo ảo tưởng bằng chứng (**evidence theater**)?

Điểm dừng: evidence không thay thế trách nhiệm

Đường ống CI/CD, liên kết OIDC, chứng thực nguồn gốc, đối tượng Deployment và cơ chế quay lui không làm phần mềm tự động an toàn. Chúng biến toàn bộ lộ trình của một thay đổi thành một chuỗi có thể kiểm chứng độc lập: mã nguồn nào, gói phát hành nào, cổng kiểm soát nào, đặc quyền nào, tiến trình phát hành nào và phương án ứng phó nào khi xảy ra sự cố. Đó chính là nền tảng kỹ thuật vững chắc để một dịch vụ Go có thể được phát hành liên tục mà không biến mỗi đợt triển khai thành một canh bạc của niềm tin mơ hồ.

Từ đây, cuốn sách bước vào một nghiên cứu điển hình lớn hơn: một dịch vụ tiếp nhận dữ liệu đầu vào, thực thi tác vụ có thời hạn kết thúc (**deadline**), phát tín hiệu giám sát, được đóng gói hoàn chỉnh, rồi được chuyển giao qua chính sách phân phối và sẵn sàng phục hồi dựa trên đầy đủ bằng chứng kỹ thuật. Mỗi công cụ chỉ được giới thiệu khi câu hỏi vận hành thực tế đã xuất hiện và đòi hỏi lời giải.

Chương 19 — Giữ type information khi abstraction lớn lên

Một dịch vụ có hai danh sách đích phục vụ: tên miền và cổng mạng. Cả hai đều đòi hỏi cùng một thao tác xử lý: giữ lại phần tử xuất hiện đầu tiên của mỗi giá trị theo đúng thứ tự ban đầu. Nếu viết trực diện, ta sẽ có hai hàm xử lý gần như giống hệt nhau. Sự trùng lặp này chưa gây nguy hiểm; nó làm nổi bật câu hỏi thiết kế cốt lõi: phần nào trong thao tác thực sự khác biệt theo kiểu dữ liệu, và phần nào tuân theo quy luật tính toán chung?

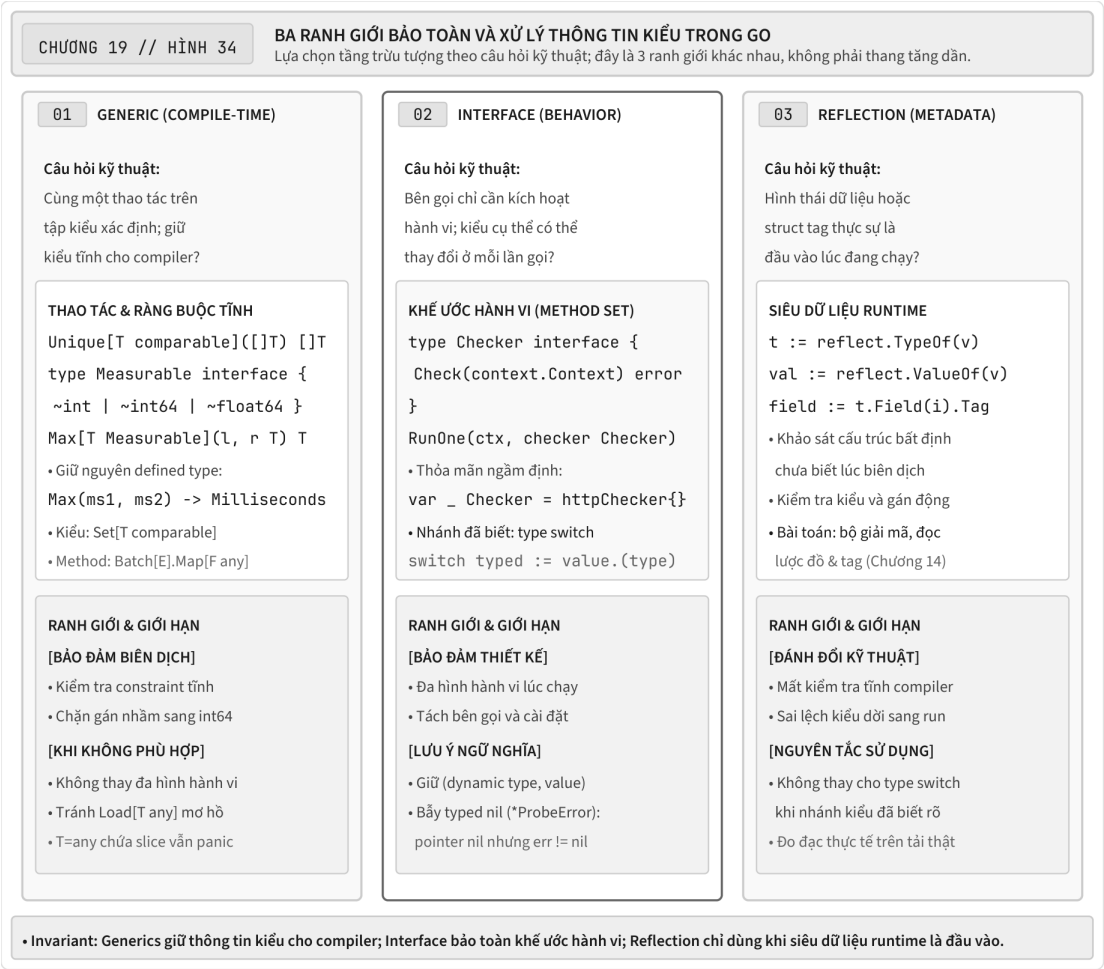
```
func uniqueNames(values []string) []string {
    seen := make(map[string]struct{}, len(values))
    result := make([]string, 0, len(values))
    for _, value := range values {
        if _, exists := seen[value]; !exists {
            seen[value] = struct{}{}
            result = append(result, value)
        }
    }
    return result
}

func uniquePorts(values []int) []int {
    seen := make(map[int]struct{}, len(values))
    result := make([]int, 0, len(values))
    for _, value := range values {
        if _, exists := seen[value]; !exists {
            seen[value] = struct{}{}
            result = append(result, value)
        }
    }
    return result
}
```

Một phản ứng thường thấy là chuyển đổi mọi thứ sang kiểu `any`. Hàm xử lý sẽ bớt lặp lại về mặt hình thức, nhưng bên gọi sẽ đánh mất thông tin kiểu mà trình biên dịch có thể tận dụng để bắt lỗi sớm. Phản ứng ngược lại là vội vã tạo một tầng trừu tượng tổng quát (**generic**) cho mọi đoạn mã trùng lặp. Điều đó cũng nguy hại không kém: một tham số kiểu (**type parameter**) chỉ thực sự

có ý nghĩa khi thao tác tính toán chung trở nên sáng tỏ và chặt chẽ hơn sau khi được tách biệt.

Mô hình tư duy cốt lõi của chương là: hàm tổng quát giữ lại thông tin kiểu cho trình biên dịch khi cùng một thao tác áp dụng lên một tập kiểu dữ liệu; giao diện (**interface**) bảo toàn khế ước hành vi khi kiểu cụ thể có thể thay đổi; trong khi cơ chế phản chiếu (**reflection**) chỉ phù hợp khi hình thái dữ liệu hoặc siêu dữ liệu thực sự là đầu vào lúc chương trình đang chạy. Đây là ba ranh giới kỹ thuật hoàn toàn tách biệt, không phải ba cấp độ linh hoạt tăng dần để tùy tiện sử dụng.



Hình 34 — Lựa chọn tầng trừu tượng phù hợp với câu hỏi kỹ thuật cần giải quyết. Sơ đồ minh họa mô hình tư duy, không mô tả cách biểu diễn bộ nhớ lúc vận hành của giao diện hay cách trình biên dịch sinh mã thực thi.

Bắt đầu bằng duplication, không bằng dấu ngoặc vuông

Cả hai hàm `uniqueNames` và `uniquePorts` đều chỉ cần một phép tính toán duy nhất: so sánh hai giá trị bằng toán tử bằng `==`. Điều kiện này loại bỏ các kiểu mảng động (**slice**), bảng băm (**map**) và hàm (**function**), nhưng hoàn toàn đáp ứng cho `string`, `int`, con trỏ, cấu trúc chứa các trường có thể so sánh, và kiểu định danh có kiểu nền tảng tương ứng. Ràng buộc `comparable` xác lập chính xác ranh giới kỹ thuật đó.

```
func Unique[T comparable](values []T) []T {
    seen := make(map[T]struct{}, len(values))
    result := make([]T, 0, len(values))
    for _, value := range values {
        if _, exists := seen[value]; !exists {
            seen[value] = struct{}{}
            result = append(result, value)
        }
    }
    return result
}

names := Unique([]string{"api", "db", "api"})
ports := Unique([]int{443, 8080, 443})
```

Trong khai báo trên, `T` là tham số kiểu (**type parameter**), còn `comparable` là ràng buộc (**constraint**): tập hợp các đối số kiểu mà `T` được phép tiếp nhận, đồng thời là giới hạn cho các thao tác được phép thực thi trong thân hàm. Do bên gọi truyền vào `[]string` hoặc `[]int`, trình biên dịch sẽ tự động suy luận đối số kiểu mà không bắt buộc lập trình viên phải viết `Unique[string](...)`. Việc viết tường minh vẫn hoàn toàn hợp lệ khi ngữ cảnh chưa đủ dữ liệu để suy luận, nhưng đó không phải là nghi thức bắt buộc cho mọi lời gọi hàm.

Nếu thử nghiệm với `Unique([][]byte{{1}, {1}})`, mã nguồn sẽ không thể biên dịch. Đó là một kết quả bảo vệ có giá trị thiết thực: mảng động không thể so sánh bằng toán tử bằng, do đó khóa của bảng băm và phép so sánh `==` không có ngữ nghĩa tương thích với những gì hàm đã cam kết. Việc chuyển đổi đầu vào thành `[]any` không sửa chữa được khế ước kỹ thuật; hai giá trị giao diện mang kiểu động là mảng còn có thể gây hoảng loạn (`panic`)

ngay trong lúc chạy khi đem so sánh. Khi phép so sánh bằng cho các mảng byte đòi hỏi một ngữ nghĩa khác (như `bytes.Equal`), giao diện lập trình mới bắt buộc phải tuyên bố rõ ràng ngữ nghĩa đó thay vì giả vờ rằng hàm `Unique` có thể áp dụng phổ quát cho mọi loại dữ liệu.

Kiểu `any` là tên định danh phụ (**alias**) của `interface{}` rỗng. Nó phát huy vai trò ở ranh giới thực sự tiếp nhận nhiều hình thái dữ liệu đa dạng, tương tự như Chương 14 khi tiếp nhận đối tượng đích để kiểm tra qua cơ chế phản chiếu. Tuy nhiên, khai báo `func UniqueAny(values []any) []any` sẽ ép buộc phần hiện thực phải chuyển mạch kiểu (**type switch**), khẳng định kiểu (**type assertion**) hoặc chấp nhận phát sinh lỗi lúc chạy mới biết cách so sánh. Kiểu tổng quát không làm biến mất kiểu dữ liệu; nó yêu cầu trình biên dịch kiểm tra tính hợp lệ của đối số kiểu đối với ràng buộc ngay trước khi mã nguồn được phép thực thi.

Có một ngoại lệ cần kiểm tra ngay: từ Go1.20, `any` và một số kiểu chứa interface có thể thỏa mãn ràng buộc `comparable`, dù không thuộc tập kiểu strictly comparable. Vì thế `Unique([]any{[]byte{1}})` biên dịch nhưng panic khi dùng slice động làm khóa map; chỉ một phần tử cũng đủ. Constraint cho phép viết phép toán trong thân hàm, không cam đoan mọi giá trị động đều so sánh an toàn. Test `TestUniqueInterfaceKeyCanPanic` bắt panic có chủ đích này; không chuyển nó thành hợp đồng trả error mà implementation không có.

Ngay cả phép so sánh không panic cũng chưa định nghĩa cách gộp mọi dữ liệu nghiệp vụ: `NaN != NaN`, nên hai NaN vẫn là hai phần tử trong kết quả Unique. Hãy chọn chính sách riêng nếu miền dữ liệu muốn gộp NaN. Các phép thử tên miền/cổng chỉ dùng string/int; chúng không chứng minh hàm là bộ chuẩn hóa dữ liệu phổ quát.

Constraint là danh sách phép được phép, không phải giấy phép bao hết type

Có những thao tác đòi hỏi so sánh thứ tự hơn kém thay vì so sánh bằng. Chúng ta hoàn toàn có thể tự định nghĩa một ràng buộc thu hẹp phù hợp với đúng miền nghiệp vụ của bài toán, thay vì sao chép một ràng buộc quá rộng lớn chỉ để hàm trông có vẻ "tái sử dụng được".

```
type Measurable interface {
    ~int | ~int64 | ~float64
}

func Max[T Measurable](left, right T) T {
    if left > right {
        return left
    }
    return right
}

type Milliseconds int64

slowest := Max(Milliseconds(120), Milliseconds(80))
```

Max ở đây dùng đúng toán tử > chứ không định nghĩa thứ tự toàn phần cho số thực. Với NaN, phép so sánh ấy là false: `Max(NaN, 1)` trả 1 nhưng `Max(1, NaN)` trả NaN. Nếu số đo có thể chứa NaN, cần xác thực đầu vào hoặc quy định rõ cách xử lý; không suy từ constraint rằng kết quả độc lập thứ tự hai đối số.

Toán tử hợp nhất | chỉ ra rằng một đối số kiểu phải thuộc về một trong các thành phần liệt kê. Ký tự ngã ~ mở rộng thành phần đó cho mọi kiểu tự định nghĩa (**defined type**) có kiểu nền tảng tương ứng. Nếu thiếu `~int64`, kiểu `Milliseconds` sẽ không thỏa mãn ràng buộc cho dù phép so sánh `left > right` hoàn toàn sáng tỏ về mặt ngữ nghĩa. Ngược lại, ký tự ~ không thực hiện ép kiểu ngầm định giữa hai giá trị đã mang kiểu cụ thể: kiểu `int` và `int64` vẫn hoàn toàn khác biệt, vì vậy lời gọi `Max(int(1), int64(2))` không thể tìm ra một tham số `T` duy nhất để suy luận kiểu. Riêng các hằng số chưa định kiểu (như số 1) có thể tự động tiếp nhận kiểu dữ liệu từ đối số còn lại.

Một ràng buộc chứa thành phần kiểu như `~int | ~int64` được gọi là giao diện tổng quát (**general interface**): nó được dùng riêng để giới hạn tham số kiểu, hoàn toàn không phải một giá trị giao diện để truyền qua lại trong chương trình. Một giao diện cơ bản (**basic interface**) có thể được dùng làm kiểu của biến: nó chỉ khai báo hoặc nhúng các phương thức, và có thể để rỗng như `any`. Ràng buộc còn có thể sử dụng các thành phần kiểu để giới hạn tập hợp kiểu dữ liệu và các phép toán mà trình biên dịch cho phép. Việc đặt cả hai khái niệm này dưới cùng một từ khóa "interface" rất dễ làm mờ nhạt đi hai vai trò kỹ thuật riêng biệt.

> **Dừng để dự đoán:** Xét đoạn mã sau:

```
slowest := Max(Milliseconds(120), Milliseconds(80))
var raw int64 = slowest
```

> 1. Trình biên dịch thực hiện suy luận kiểu (type inference) cho `T` là `int64` hay `Milliseconds`?

> 2. Biến `slowest` có kiểu dữ liệu tĩnh là gì?

> 3. Lệnh gán `var raw int64 = slowest` có biên dịch thành công không, hay bị compiler báo lỗi?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Quá trình suy luận kiểu (type unification) chọn `T` chính xác là `Milliseconds` chứ không hề thu gọn (decay) về underlying type `int64`. Do cả hai đối số truyền vào đều mang kiểu `Milliseconds`, hàm generic bảo toàn thông tin kiểu của đối số và trả về kết quả mang đúng kiểu tĩnh `Milliseconds` thay vì ép về kiểu nền hay bọc trong interface rỗng.

Vì lý do đó, lệnh gán `var raw int64 = slowest` bị trình biên dịch từ chối ngay lập tức với lỗi: `cannot use slowest (variable of type Milliseconds) as int64 value in variable declaration`. Dù

`Milliseconds` có underlying type là `int64`, hệ thống kiểu của Go không bao giờ cho phép ép kiểu ngầm định giữa hai defined type khác nhau. Muốn gán giá trị này sang một biến `int64`, lập trình viên bắt buộc phải chuyển đổi tường minh bằng cú pháp `int64(slowest)`. Đây chính là giá trị bảo toàn thông tin kiểu (type preservation) ở thời điểm biên dịch của Generic: nó loại bỏ nguy cơ gán nhầm đơn vị đo lường mà không đòi hỏi chi phí đánh đổi qua `any` và type assertion ở runtime.

Generic named type giữ invariant cùng dữ liệu

Khi cấu trúc dữ liệu và các thao tác tính toán gắn liền với nhau, kiểu định danh tổng quát (**generic named type**) có thể làm giao diện lập trình trở nên tinh gọn và mạch lạc hơn. Tập hợp `Set` dưới đây chỉ chấp nhận các kiểu dữ liệu thỏa mãn `comparable` vì khóa của bảng băm trong Go bắt buộc phải đáp ứng ràng buộc này. Do ràng buộc được đặt ngay tại phần khai báo của kiểu, mọi phương thức gắn liền đều tự động kế thừa bất biến này thay vì phải khai báo lặp lại.

```
type Set[T comparable] map[T]struct{}

func (set Set[T]) Has(value T) bool {
    _, found := set[value]
    return found
}

ports := Set[int]{443: {}, 8080: {}}
```

Hai kiểu `Set[string]` và `Set[int]` là các kiểu được khởi tạo cụ thể riêng biệt. Bên gọi không thể vô tình gọi nhầm `Set[string].Has(443)`. Đây không phải lời khẳng định rằng mã nguồn tổng quát luôn chạy nhanh hơn mã nguồn dùng giao diện; hiệu năng phụ thuộc mật thiết vào khối lượng công việc, trình biên dịch và phiên bản Go, vì vậy cần phải đo đạc thực nghiệm nếu hiệu năng là căn cứ lựa chọn. Trong trường hợp này, lợi ích kỹ thuật đã sáng tỏ ngay trước khi đo chuẩn: giao diện lập trình xác định chính xác giá trị nào được phép đưa vào tập hợp.

Phiên bản Go 1.27 cho phép phương thức tự khai báo tham số kiểu riêng biệt của nó. Đặc tính này phát huy giá trị lớn khi thao tác xử lý thuộc về không gian tên của kiểu dữ liệu nhưng kết quả trả về lại làm biến đổi kiểu phần tử.

```
type Batch[E any] []E

func (batch Batch[E]) Map[F any](fn func(E) F) Batch[F] {
    result := make(Batch[F], len(batch))
    for i, value := range batch {
        result[i] = fn(value)
    }
    return result
}

labels := Batch[int]{2, 4}.Map(func(port int) string {
    return fmt.Sprintf("port=%d", port)
})
```

Đây là phương thức tổng quát của Go 1.27, khác biệt với phương thức thông thường khai báo trên kiểu tiếp nhận tổng quát như `Set[T].Has`. Phương thức tổng quát được khởi tạo cụ thể tại thời điểm sử dụng. Nó không biến phương thức giao diện thành phương thức tổng quát: phương thức trong giao diện không được phép tự khai báo tham số kiểu, và phương thức tổng quát không phải là cách để hiện thực hóa một phương thức giao diện tổng quát. Nếu bên sử dụng chỉ cần gọi `Map` trên một đối tượng mảng `Batch` cụ thể, phương thức giúp tổ chức mã nguồn ngăn nắp; nếu bên sử dụng cần một hành vi có thể thay thế linh hoạt, một giao diện nhỏ với chữ ký hàm cụ thể sẽ tường minh hơn rất nhiều.

Compiler có thể chia sẻ hoặc chuyên biệt hóa mã: không biến điều đó thành lý do chọn API

Xét theo ngữ nghĩa ngôn ngữ, một hàm tổng quát được khởi tạo cụ thể sau khi đối số kiểu thỏa mãn đầy đủ ràng buộc. Đặc tả kỹ thuật không cam kết cách thức trình biên dịch sinh mã máy, kích thước tệp thực thi hay số lượng cấp phát bộ nhớ. Bộ công cụ Go hiện tại sử dụng các cấu trúc nội bộ như biểu diễn hình thái kiểu dữ liệu và từ điển tra cứu; những thông tin này hỗ trợ kỹ sư khi đọc mã hợp ngữ hoặc mã nguồn của trình biên dịch, nhưng hoàn toàn không phải khế ước kỹ thuật dành cho tầng ứng dụng và có thể thay đổi tùy theo phiên bản, kiến trúc phần cứng, cơ chế tối ưu hóa theo hồ sơ đo lường và cấu trúc thân hàm.

Vì lẽ đó, không nên lựa chọn kiểu tổng quát chỉ vì niềm tin ngây thơ rằng nó luôn chạy nhanh hơn giao diện, và cũng đừng loại bỏ nó chỉ vì e ngại các cấu trúc từ điển tra cứu nội bộ của trình biên dịch. Hãy chọn kiểu tổng quát khi các thao tác và ràng buộc kỹ thuật giúp giao diện lập trình trở nên chuẩn xác hơn. Khi chi phí hiệu năng là căn cứ mang tính quyết định, hãy đo lường hai phương án hiện thực dưới cùng một phiên bản Go, cùng môi trường máy chủ, khối lượng tải và khế ước kiểm thử hiệu năng; sau đó tiến hành phân tích lượng cấp phát bộ nhớ và hồ sơ thực thi thay vì suy đoán mơ hồ từ các chi tiết nội bộ.

Interface thay vì generics khi câu hỏi là “ai làm được việc này?”

Một công cụ thăm dò qua giao thức mạng, một công cụ thăm dò tầng truyền vận và một đối tượng giả lập dùng trong kiểm thử có thể trả về kết quả qua các phương án hiện thực hoàn toàn khác nhau. Bên gọi không cần nắm giữ kiểu dữ liệu cụ thể của chúng; bên gọi chỉ cần kích hoạt một hành vi đã được định nghĩa. Giao diện ghi nhận khế ước kỹ thuật đó ngay ở thời điểm biên dịch.

```
type Checker interface {
    Check(context.Context) error
}

type httpChecker struct{}

func (httpChecker) Check(context.Context) error { return nil }

func RunOne(ctx context.Context, checker Checker) error {
    return checker.Check(ctx)
}

var _ Checker = httpChecker{}
```

Dòng lệnh cuối cùng thực hiện kiểm tra tính hợp chuẩn ở thời điểm biên dịch: nếu cấu trúc `httpChecker` thiếu mất phương thức `Check`, gói mã nguồn sẽ không thể biên dịch. Khi chương trình vận hành, một giá trị giao diện mang kiểu tĩnh là `Checker`, trong khi giá trị cụ thể chứa bên trong có thể mang kiểu động khác nhau ở từng lần gọi. Đó là lý do giao diện phát huy giá trị cốt lõi cho tính đa hình theo hành vi, hoàn toàn không phải vì nó có thể "tiếp nhận mọi loại kiểu dữ liệu".

Khi mã nguồn cần xác định trường hợp cụ thể nào đang ẩn sau giá trị giao diện, phép khẳng định kiểu (**type assertion**) và chuyển mạch kiểu (**type switch**) là các công cụ trực tiếp và an toàn nhất. Chúng hoàn toàn không đòi hỏi cơ chế phản chiếu tốn kém.

```
func describe(value any) string {
    switch typed := value.(type) {
    case string:
        return "name=" + typed
    case int:
        return "port"
    }
    return "unsupported"
}
```

Cú pháp `value.(int)` không có cờ kiểm tra sẽ lập tức phát sinh lỗi hoảng loạn (`panic`) khi phép khẳng định kiểu thất bại. Ngược lại, dạng hai giá trị `port, ok := value.(int)` sẽ trả về giá trị mặc định của `int` cùng cờ

`false`, nhờ đó bên gọi có thể chủ động quyết định chính sách xử lý lỗi phù hợp. Cấu trúc chuyển mạch kiểu chỉ thực hiện so sánh kiểu động với các trường hợp đã biết trước; các công cụ phản chiếu kiểu và thẻ cấu trúc từ Chương 14 chỉ nên xuất hiện khi siêu dữ liệu lúc thực thi chính là bản chất bài toán (như công cụ giải mã hoặc phân tích lược đồ). Tuyệt đối không lạm dụng cơ chế phản chiếu chỉ để thay thế cho một khối chuyển mạch kiểu có một vài trường hợp rõ ràng.

Typed nil: type động vẫn còn, dù pointer bên trong nil

Đoán output trước khi chạy đoạn này:

```
type ProbeError struct {
    Target string
}

func (err *ProbeError) Error() string {
    return "probe failed"
}

var problem *ProbeError = nil
var result error = problem

fmt.Println(problem == nil)
fmt.Println(result == nil)
```

Kết quả in ra lần lượt là `true`, sau đó là `false`. Biến `problem` là một con trỏ `nil` (`nil` **pointer**). Khi gán biến này vào kiểu giao diện `error`, giá trị giao diện vẫn lưu giữ kiểu động là `*ProbeError`; và giá trị động thuộc kiểu đó là một con trỏ rỗng.

Không cần phải suy đoán cách bố trí bộ nhớ nội bộ để giải thích hiện tượng này. Theo ngữ nghĩa chuẩn tắc của giao diện trong Go, biến `result` lưu giữ kiểu động `*ProbeError` và giá trị động rỗng của kiểu đó, do đó giá trị giao diện này hoàn toàn không phải là một giao diện `nil` (`nil` **interface**). Cách biểu diễn hai từ máy mà bộ thực thi Go trên một kiến trúc phần cứng sử dụng chỉ là chi tiết cài đặt; tuyệt đối không coi chi tiết đó là khế ước kỹ thuật và không bao giờ dựa vào kích thước hay các trường nội bộ của giao diện trong mã nguồn vận hành thực tế.

Bug thường gặp nằm ở đường return:

```
func checkTarget(ok bool) error {
    var problem *ProbeError
    if !ok {
        problem = &ProbeError{Target: "api"}
    }
    return problem // ok == true vẫn trả error khác nil
}
```

Hãy trả về trực tiếp giá trị `nil` tường minh ở nhánh thực thi thành công, hoặc chỉ khởi tạo đối tượng lỗi cụ thể khi thực sự xảy ra sự cố. Không cần phải dùng đến cơ chế phản chiếu để suy đoán con trỏ rỗng bên trong mọi giao diện lỗi: bên gọi chỉ cần tin tưởng vào khế ước `err == nil`, trong khi các phương thức của kiểu lỗi cụ thể phải tự phân định rõ đối tượng tiếp nhận có hợp lệ khi nhận giá trị rỗng hay không. Bài thực hành `labs/part19-type-information/typednil` cố tình lưu giữ khiếm khuyết này để người học tự tay khắc phục và kiểm chứng bằng ca kiểm thử.

Bài thực hành: Ba ranh giới, ba cách tự kiểm tra

Lab `labs/part19-type-information` bắt đầu ở test/tag exercise và contract, không ở implementation có sẵn.

```
cd labs/part19-type-information
go test -tags exercise ./exercise
go test ./fixed
go test -race ./fixed
go test -tags typednilexercise ./typednil
go test ./typednil/fixed
go vet ./...
```

Ba nhiệm vụ thực hành lần lượt yêu cầu người học xây dựng hàm `Unique[T comparable]` xuất phát từ ca kiểm thử, phản biện thiết kế `Load[T any]` khi chưa xác định rõ định dạng, quyền sở hữu dữ liệu và quy trình giải mã, đồng thời khắc phục khiếm khuyết con trỏ rỗng có kiểu để nhánh thành công trả về đúng giá trị lỗi `nil` thực sự. Chỉ sau khi đã hoàn thành các bước tự làm, người học mới nên tham khảo tài liệu hướng dẫn và mã nguồn đối

chiếu; chúng là tài liệu tham khảo để đối chiếu mạch tư duy logic, không phải khuôn mẫu duy nhất cho việc đặt tên.

Điểm dừng nhận thức của chương không phải là "hễ nhìn thấy cặp ngoặc vuông là áp dụng kiểu tổng quát". Hãy luôn tự đặt câu hỏi: thao tác tính toán cần bảo toàn thông tin kiểu nào, ràng buộc nào thực sự cho phép thực thi thao tác đó, hành vi nào nên được trừu tượng hóa thành giao diện, và siêu dữ liệu nào chỉ có thể biết được lúc chương trình vận hành. Khi trả lời thấu đáo được bốn câu hỏi đó, tầng trừu tượng sẽ không còn giống một màn ảo thuật khó lường mà sẽ trở thành một khế ước kỹ thuật trong sáng và có thể kiểm chứng độc lập.

Chương 20 — Dự án tổng kết: opsprobe từ mã nguồn đến vận hành

Ở Chương 1, chúng ta sử dụng một kết quả HTTP giả định để làm quen với cách đọc một chương trình Go. Các chương tiếp theo lần lượt bổ sung yêu cầu mạng thực tế, thời hạn kết thúc (**deadline**), cơ chế xử lý đồng thời, lưu trữ bền vững và các tín hiệu vận hành. Chương này kết nối toàn bộ những cấu phần đó vào một công cụ thăm dò điểm cuối hoàn chỉnh: khi một lượt thăm dò thất bại, bên gọi cần biết kết quả thuộc về ranh giới kỹ thuật nào và dữ liệu nào đã được lưu trữ an toàn.

Dự án `projects/opsprobe/` là một mô hình thực nghiệm mẫu hoàn chỉnh gồm logic nghiệp vụ cốt lõi, nhóm tiến trình xử lý giới hạn (**worker pool**), cơ sở dữ liệu SQLite, hệ thống đo lường, truy vết phân tán và giao diện lập trình HTTP. Các bài kiểm thử tái lập chính xác những khế ước kỹ thuật cụ thể, bao gồm cả tình huống điển hình khi bỏ quên việc đóng luồng phản hồi (**response body**). Việc sở hữu một bản mã nguồn chạy được không đồng nghĩa công cụ đã sẵn sàng cho mọi môi trường vận hành thực tế; các phân tích về bảo mật và triển khai dưới đây chỉ rõ những giả định kỹ thuật bắt buộc phải duy trì.

Mô hình tư duy cốt lõi của chương là: một hệ thống Go trưởng thành không phải là tập hợp các bộ khung cồng kềnh, mà là sự khớp nối chuẩn xác giữa các ranh giới kỹ thuật thu hẹp: ngữ nghĩa giá trị, khế ước xử lý lỗi, điều tiết áp suất đồng thời, tính nguyên tử của trạng thái, tín hiệu quan sát trung thực và kỷ luật phát hành có trách nhiệm.

Bản đồ kiến trúc và hướng phụ thuộc

Mã nguồn của dự án nằm tại thư mục `projects/opsprobe/`. Các package tách những trách nhiệm sau:

```
projects/opsprobe/
├── cmd/opsprobe/          # CLI oneshot và HTTP daemon
├── internal/
│   ├── probe/             # Bounded concurrency pool, Outcome
│   ├── store/             # SQLite, transaction nguyên tử
│   ├── telemetry/         # slog JSON, Prometheus, OTel
│   └── httpapi/           # REST API, backpressure, health
├── incident/              # Kịch bản sự cố rò rỉ socket TCP
├── deploy/                # Dockerfile, Kubernetes, Terraform
└── scenarios_test.go      # Kiểm thử 6 kịch bản sự cố
```

Bảng 22 — Phân tách trách nhiệm và ranh giới contract trong opsprobe

Gói mã nguồn	Trách nhiệm vận hành	Khế ước bảo vệ
`probe`	Kiểm tra điểm cuối, phân loại kết quả.	Giới hạn tiến trình làm việc, phân định kết quả rõ ràng, giải phóng luồng dữ liệu.
`store`	Lưu trữ siêu dữ liệu và kết quả thăm dò.	Giao dịch nguyên tử qua gói `database/sql`.
`telemetry`	Ghi nhận nhật ký, chỉ số đo lường và vết phân tán.	Không suy đoán cảm tính, đo lường chuẩn xác nhãn kết quả.
`httpapi`	Tiếp nhận yêu cầu, điều phối tải, trả về dữ liệu JSON.	Điều tiết áp suất ngược mã 429, kiểm tra tính sẵn sàng, rút cạn kết nối an toàn.
`cmd`	Điểm khởi đầu cấu hình toàn cục (**Composition Root**).	Phân tích tham số dòng lệnh, đón bắt tín hiệu hệ điều hành, tắt tiến trình an toàn.

Hai gói `probe` và `store` hoàn toàn không phụ thuộc vào `httpapi` hay `cmd`, do đó có thể kiểm thử độc lập mà không cần khởi động toàn bộ ứng dụng. Điều đó không biến mọi bài kiểm thử thành các phép tính toán thuần túy: kiểm thử vào ra dữ liệu của `probe` sử dụng máy chủ HTTP cục bộ, trong khi kiểm thử của `store` đòi hỏi cơ sở dữ liệu thực sự. Chiều hướng phụ thuộc giúp phân định ranh giới rõ ràng; các bộ dữ liệu kiểm thử (**fixtures**) vẫn phải tái lập trung thực đúng hành vi cần kiểm chứng.

Năm trụ cột kỹ thuật của một hệ thống đáng tin

1. Phân loại kết quả và thời hạn ngữ cảnh

Trong vận hành, biết một thao tác thất bại là chưa đủ; ta cần biết nó thất bại vì lý do gì. Trả về một giá trị boolean `false` hay một chuỗi lỗi mơ hồ sẽ xóa sạch thông tin quý giá. Trong `internal/probe/probe.go`, mỗi lần probe được phân loại rành mạch thành một trong bốn `Outcome`:

```
type Outcome string

const (
    OutcomeSuccess Outcome = "success"
    OutcomeFailure Outcome = "failure"
    OutcomeTimeout Outcome = "timeout"
    OutcomeCancel  Outcome = "cancel"
)
```

Khi gửi request, ta gắn deadline chặt chẽ cho từng target:

```
targetTimeout := p.defaultTimeout
if target.Timeout > 0 {
    targetTimeout = target.Timeout
}
probeCtx, cancel := context.WithTimeout(ctx, targetTimeout)
defer cancel()

req, err := http.NewRequestWithContext(
    probeCtx, method, target.URL, nil)
```

Việc phân định ranh giới giữa `OutcomeTimeout` và `OutcomeCancel` giúp bên gọi lưu giữ nguyên nhân kết thúc đúng theo khế ước kỹ thuật của bài thực hành. Cơ chế này không tự động phân loại được nguyên nhân gốc rễ từ tầng mạng hay chứng minh mọi hành động hủy bỏ đều xuất phát từ việc dừng ứng dụng; kỹ sư cần đối chiếu với vòng đời tiến trình, thời hạn kết thúc và các dữ liệu quan sát bổ trợ.

Mỗi kết quả probe ghi nhận thời lượng bằng mili-giây (`DurationMs`) phục vụ bảng điều khiển và nano-giây (`DurationNs`) để tránh mất thông tin do làm tròn số; đơn vị nano-giây không chứng minh đồng hồ đo đạt độ chính xác tuyệt đối đến nano-giây. Thời điểm ghi nhận đợt chạy trong cơ sở dữ liệu

phân biệt rõ `StartedAt` (bắt đầu thực thi) và `CompletedAt` (kết thúc toàn bộ tiến trình làm việc), tránh nhầm lẫn độ trễ một yêu cầu đơn lẻ với thời gian hoàn tất toàn bộ lô công việc.

2. Giới hạn đồng thời và áp suất ngược

Khi danh sách mục tiêu thăm dò tăng vọt từ 10 lên 10.000, việc khởi tạo tức thời `go p.ProbeSingle(...)` cho từng mục tiêu có thể gây áp lực nghẽn nghiêm trọng lên bộ nhớ, ổ cắm mạng và dịch vụ đích. Mức độ tác động phụ thuộc mật thiết vào khối lượng công việc và hạn ngạch của môi trường; không thể vội vàng suy đoán hệ điều hành chắc chắn sẽ sụp đổ chỉ dựa trên số lượng tiến trình goroutine thuần túy.

`opsprobe` sử dụng mô hình worker pool với số lượng goroutine cố định:

```
numWorkers := p.concurrency
if numWorkers > n {
    numWorkers = n
}

var wg sync.WaitGroup
for w := 0; w < numWorkers; w++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for j := range jobs {
            if ctx.Err() != nil {
                results[j.index] = Result{
                    TargetID: j.target.ID,
                    Outcome:  OutcomeCancel,
                }
                continue
            }
            results[j.index] = p.ProbeSingle(ctx, j.target)
        }
    }()
}
wg.Wait()
```

Đoạn trên lược bỏ instrumentation và một số field kết quả. Trong source, producer đưa toàn bộ job vào kênh đệm rồi đóng kênh trước khi khởi động worker. `WorkerObserver.WorkerStarted/WorkerStopped` bao quanh

từng lượt `ProbeSingle`, nên `opsprobe_active_workers` đo thao tác thăm dò đang chạy, không phải tổng số goroutine còn sống hay worker đang chờ việc. Lượt job bị bỏ qua vì context đã hủy không tăng gauge này. `wg.Wait()` là điểm bàn giao kết quả sau khi mọi worker kết thúc; hook quan sát không thay cho phép join đó.

Tại giao diện HTTP API, một kênh đệm vận hành như cờ hiệu (**semaphore**) để giới hạn số lượng lượt chạy đồng thời theo cấu hình. Khi cạn kiệt vị trí xử lý, chính sách của bài thực hành sẽ chủ động từ chối tiếp nhận công việc mới bằng mã trạng thái `429 Too Many Requests`; giới hạn này là định mức của ví dụ mẫu, chưa phải năng lực chịu tải đã được kiểm chứng an toàn cho môi trường thực tế:

```
select {
case a.runSem <- struct{}{}:
    defer func() { <-a.runSem }()
default:
    a.tel.RunsTotal.WithLabelValues(
        "rejected_backpressure").Inc()
    a.writeJSON(
        w, http.StatusTooManyRequests,
        map[string]string{
            "error": "too many runs, backpressure applied",
        },
    )
    return
}
```

3. Giao dịch nguyên tử trên SQLite: Hoặc toàn bộ, hoặc không có gì

Dữ liệu lưu trữ chỉ có giá trị khi nó phản ánh đúng trạng thái nhất quán của thế giới thực. Nếu một đợt chạy gồm 10 target, hệ thống không được phép rơi vào trạng thái: lưu thành công 5 target đầu rồi crash, bỏ lại 5 target sau và làm lệch báo cáo tổng hợp.

Trong `internal/store/store.go`, bản ghi đợt chạy (`runs`) và các kết quả probe (`probe_results`) được ghi trong một transaction qua `database/sql`. Đoạn dưới là mã giả về thứ tự thao tác; dấu `...` lược các

đối số nên không phải mã Go có thể sao chép để biên dịch. Truy vấn, đối số và nhánh lỗi đầy đủ nằm trong source và được kiểm thử bằng SQLite thật:

```
tx, err := s.db.BeginTx(ctx, nil)
if err != nil {
    return fmt.Errorf("begin transaction: %w", err)
}
defer tx.Rollback() // An toàn: no-op nếu Commit đã thành công

// 1. Chèn bản ghi đợt chạy (runs)
_, err = tx.ExecContext(ctx, insertRunQuery, run.ID, ...)
if err != nil {
    return fmt.Errorf("insert run: %w", err)
}

// 2. Chèn từng kết quả probe bằng prepared statement
stmt, err := tx.PrepareContext(ctx, insertResultQuery)
```

```
if err != nil {
    return err
}
defer stmt.Close()

for _, r := range results {
    _, err = stmt.ExecContext(ctx, run.ID, r.TargetID, ...)
    if err != nil {
        return err // defer tx.Rollback() sẽ hủy toàn bộ
    }
}

return tx.Commit()
```

Để kiểm chứng tính toàn vẹn của transaction một cách tất định (deterministic) mà không dựa vào thời điểm ngẫu nhiên, schema cơ sở dữ liệu được trang bị ràng buộc `CHECK (status_code >= 0)`. Trong kiểm thử tự động, ta cố tình truyền một kết quả có `status_code = -1` ở bản ghi thứ hai sau khi bản ghi cha đã được chèn. Database lập tức từ chối, transaction rollback toàn bộ, và lệnh đếm số dòng xác nhận cả hai bảng đều không lưu lại bất kỳ dữ liệu rác nào.

4. Vòng đời tài nguyên mạng: Thu hồi có giới hạn và điều kiện tái sử dụng

Bên gọi bắt buộc phải luôn đóng luồng dữ liệu `resp.Body`. Việc đọc dữ liệu đến khi cạn dòng (**EOF**) có thể hỗ trợ một số trường hợp tái sử dụng kết nối mạng, nhưng thao tác `Close` hay rút cạn dòng dữ liệu (**drain**) không bảo đảm kết nối chắc chắn sẽ được tái sử dụng: tầng truyền vận (**Transport**), giao thức mạng, máy chủ và trạng thái kết nối tức thời đều tham gia quyết định việc này. Ngược lại, việc rút cạn dòng dữ liệu không giới hạn đòi hỏi phải cân nhắc thận trọng khi dịch vụ đích có thể trả về phần thân phản hồi khổng lồ hoặc không bao giờ kết thúc.

`opsprobe` thiết lập chính sách bounded drain có kiểm soát:

```
const MaxDrainBytes = 16384 // 16 KiB

func drainResponseBody(
    body io.ReadCloser, maxBytes int64,
) (bool, error) {
    if body == nil {
        return false, nil
    }
    defer body.Close()
    if maxBytes <= 0 {
        return false, nil
    }
    if maxBytes == int64(^uint64(0)>>1) {
        return false, errors.New(
            "drain limit leaves no room for sentinel byte")
    }
    lr := &io.LimitedReader{R: body, N: maxBytes + 1}
    _, err := io.Copy(io.Discard, lr)
    if err != nil {
        return false, err
    }
    // Body nhỏ đã chạm EOF; không phải guarantee reuse.
    return lr.N > 0, nil
}
```

Quy tắc kỹ thuật ở đây rất rõ ràng:

Một là, đóng phần thân phản hồi trên mọi nhánh kết thúc: hoàn tất đầy đủ trách nhiệm với luồng dữ liệu phản hồi theo đúng khế ước kỹ thuật. Thao tác `Close` không đồng nghĩa với việc mỗi lần gọi đều phải đóng hẳn ổ cắm mạng TCP; tầng truyền vận có thể giữ lại kết nối để tái sử dụng.

Hai là, ấn định hạn mức 16 KiB: Đây là chính sách kỹ thuật của bài thực hành, không phải con số thống kê dung lượng phản hồi của các bộ khung bên ngoài. Hàm bổ trợ đọc thêm tối đa một byte để nhận biết phần thân có vượt hạn mức hay không, sử dụng `io.Discard` nhằm loại bỏ dữ liệu thay vì lưu giữ toàn bộ vào bộ nhớ RAM.

Ba là, nhận biết thời điểm cạn dòng dữ liệu: việc đọc tối đa `MaxDrainBytes + 1` giúp phân biệt rõ ràng phần thân nhỏ đã đọc trọn vẹn với phần thân vượt quá hạn mức cho phép. Thao tác rút cạn chỉ được coi là thành công khi phép đọc không phát sinh lỗi và biến đếm `lr.N > 0`; lỗi phát sinh trước khi chạm mốc kết thúc dòng không được phép diễn giải là thành công. Đây là quyết định phân bổ hạn mức của hàm bổ trợ, không phải cờ xác nhận đủ điều kiện tái sử dụng bên trong tầng truyền vận. Khả năng tái sử dụng kết nối HTTP/1.x còn phụ thuộc chặt chẽ vào quy cách đóng gói dữ liệu, máy chủ đích, sự cố đường truyền và trạng thái nội bộ của tầng truyền vận.

Bốn là, đưa ra quyết định đánh đổi kỹ thuật có chủ đích: Nếu phần thân phản hồi vượt quá 16 KiB, biến đếm `lr.N` sẽ nhận giá trị bằng 0. Bài thực hành ghi nhận phần thân chưa được đọc hết và lập tức đóng luồng dữ liệu để giải phóng tài nguyên; không đưa ra kết luận võ đoán rằng tầng truyền vận sẽ hay không thể tái sử dụng một kết nối cụ thể.

Năm là, thời lượng thăm dò phản ánh trọn vẹn toàn bộ vòng đời thao tác: Đồng hồ đo lường thời gian bắt đầu từ trước khi phát yêu cầu mạng cho đến khi quy trình dọn dẹp và rút cạn dòng dữ liệu kết thúc. Nếu mục tiêu trả về tiêu đề thành công 200 nhưng luồng dữ liệu phần thân bị tắc nghẽn kéo dài vượt quá thời hạn, lượt thăm dò sẽ kết luận chính xác là `OutcomeTimeout` thay vì báo nhầm là `OutcomeSuccess`.

5. Ranh giới bảo mật, hợp đồng JSON và truy vết phân tán

Một công cụ vận hành chỉ an toàn khi các ranh giới ngoại vi được xác định rành mạch:

Dự án `opsprobe` đặt giả định rằng người vận hành và danh bạ mục tiêu là các thành phần đáng tin cậy. Việc kiểm tra cú pháp địa chỉ URL không thể ngăn chặn triệt để lỗ hổng giả mạo yêu cầu từ phía máy chủ (SSRF). Nếu tiếp nhận dữ liệu đầu vào không tin cậy, hệ thống bắt buộc phải thiết lập chính sách kiểm soát mục tiêu và lưu lượng đi ra, kiểm soát phân giải tên miền và chuyển hướng theo mô hình đe dọa; không thể mặc định ngăn chặn mọi dải mạng nội bộ vì một công cụ vận hành nội bộ có thể cần truy cập chính các điểm cuối đó. Danh sách cho phép hoặc máy chủ ủy nhiệm trung gian cũng cần có chủ thể quản lý và cấu hình được kiểm chứng.

Về khế ước JSON tương thích đa nền tảng: Trường `timeout_ms` trong dữ liệu yêu cầu sử dụng kiểu số nguyên mili-giây (`0 <= timeout_ms <= 60000`), bảo đảm tính tương thích cao thay vì phân tích cú pháp chuỗi thời gian đặc thù của riêng Go.

Về cơ chế phân tán ngữ cảnh chuẩn W3C: Các yêu cầu thăm dò gửi ra ngoài được chèn thêm tiêu đề `traceparent` tương ứng với khoảng thời gian con (**child span**) hiện hành. Để kết nối liền mạch chuỗi truy vết, dịch vụ tiếp nhận bắt buộc phải đọc ngữ cảnh đó và cấu hình cơ chế lấy mẫu cùng việc xuất dữ liệu phù hợp; việc gửi đi một tiêu đề không tự nó chứng minh rằng chuỗi quan sát xuyên suốt giữa các dịch vụ đã hoàn chỉnh.

Bài tập chẩn đoán sự cố: Bão cạn kiệt Socket

Thư mục `projects/opsprobe/incident/` mô phỏng một sự cố kinh điển trong môi trường microservices.

Kịch bản và Triệu chứng lâm sàng

> **Ghi chú phương pháp luận:** Đây là kịch bản giả định mô phỏng tình huống sự cố thực tế để đặt ra bài toán chẩn đoán cho kỹ sư.

Hệ thống probe giả định được triển khai để kiểm tra sức khỏe các microservice nội bộ. Khi số lượng request tăng lên, hệ thống giám sát ghi nhận tỷ lệ `OutcomeTimeout` tăng vọt và độ trễ probe tăng cao. Tuy nhiên, khi kỹ sư thực hiện một lệnh cURL độc lập trực tiếp tới dịch vụ đích, dịch vụ vẫn phản hồi rất nhanh trong vài phần nghìn giây. Kiểm tra tài nguyên hệ điều hành bằng `ss -s` cho thấy số lượng socket TCP mở tăng liên tục theo từng lượt gọi mà không được thu hồi, tiệm cận giới hạn file descriptors (`ulimit -n`). Đồng thời, khi kiểm tra bằng `net/http/httptrace`, hook `GotConnInfo.Reused` luôn trả về `false`, cho thấy `http.Transport` không tái sử dụng được kết nối rảnh nào giữa các lượt probe.

> **Dừng để dự đoán:** Vì sao kết quả cURL nhanh chỉ là bằng chứng hỗ trợ thu hẹp giả thuyết chứ chưa đủ để bác bỏ hoàn toàn lỗi hạ tầng? Kỹ sư cần thiết kế phép thử nào với `httptrace` để xác minh vòng đời kết nối của client trước khi can thiệp vào cấu hình hệ thống?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Kết quả cURL phản hồi nhanh chỉ cho thấy dịch vụ đích có khả năng phục vụ một kết nối đơn lẻ tại thời điểm thử nghiệm. Nó giúp thu hẹp nghi vấn về việc máy chủ đích bị treo hoàn toàn, nhưng chưa đủ để loại trừ các yếu tố hạ tầng phức tạp như giới hạn kết nối đồng thời trên tường lửa, chính sách cân bằng tải hay tình trạng bão hòa hàng đợi socket.

Để cô lập nguyên nhân, kỹ sư dùng `net/http/httptrace` và trường `info.Reused`. Theo tài liệu `net/http`, nếu body chưa được đọc đến EOF và đóng, Transport có thể không tái sử dụng kết nối HTTP/1.x. Trong fixture cố ý bỏ body, ta quan sát kết nối mới; đó không phải bảo đảm rằng mọi request tiếp theo trong mọi cấu hình đều phải mở TCP mới hay chắc chắn cạn tài nguyên. HTTP/2 còn có vòng đời stream khác; không ngoại suy kết quả này sang mọi giao thức.

Bằng chứng mã nguồn và Đo đạc thực nghiệm

Kiểm tra `incident/incident.go` (đoạn hàm `BuggyProbe`):

```
resp, err := client.Do(req)
if err != nil {
    return 0, false, err
}
// BUG: Quên gọi resp.Body.Close()!
return resp.StatusCode, false, nil
```

Trong kịch bản giao thức HTTP/1.x của bài thực hành, hàm xử lý trả về kết quả mà không đọc trọn vẹn hoặc không đóng phần thân phản hồi. Kết nối đang lưu giữ luồng dữ liệu chưa hoàn tất sẽ không thể tái sử dụng cho các yêu cầu khác; việc gia tăng tải trong tình trạng này sẽ làm bùng nổ số lượng kết nối mở. Không áp dụng kết luận này một cách máy móc cho cơ chế dồn kênh phức hợp (**multiplexing**) của giao thức HTTP/2.

Bằng chứng đo đạc thực nghiệm

Trong bài kiểm thử 20 yêu cầu tuần tự tới máy chủ kiểm thử ở `incident/incident_test.go`, công cụ `httptrace` ghi nhận các số đếm thực nghiệm sau. Dữ liệu này mô tả chính xác ca kiểm thử cụ thể, không dự đoán tỷ lệ tái sử dụng kết nối cho mọi máy chủ hay mọi khối lượng tải:

Chỉ số thực nghiệm	BuggyProbe (Bỏ quên Close)	FixedProbe (Close + Drain 16 KiB)
Kết nối mới (`Reused == false`)	**20**	**1** (trong bài kiểm thử này)
Kết nối tái sử dụng (`Reused == true`)	**0**	**19** (trong bài kiểm thử này)
Kết quả vận hành	Mỗi yêu cầu mở ổ cắm mạng mới	Tái sử dụng ổ cắm mạng trong nhóm

Bài kiểm thử `TestIncident_BoundedDrainOversizedBody` xác nhận rằng với phần thân 32 KiB (vượt quá hạn mức 16 KiB), bài thực hành sẽ trả về `reusedEligible == false` và lập tức đóng phần thân phản hồi. Cơ chế rút cạn có giới hạn chỉ kiểm soát lượng dữ liệu mà máy khách chủ động đọc bỏ; nó không phải là cơ chế phòng chống tràn bộ nhớ đệm hay bảo đảm tuyệt đối rằng một kết nối mạng sẽ được tái sử dụng.

Consumer đã rời đi, worker còn nợ ai kết quả?

Các bài kiểm thử trước đó bảo vệ một tập hợp các mục tiêu đã xác định từ trước. Bây giờ chúng ta thay đổi bài toán: bên sản xuất công việc tiếp tục nhận việc trong khi dịch vụ phụ thuộc phản hồi chậm, hàng đợi bị lấp đầy và bên tiêu thụ ngừng đọc kết quả. Việc bổ sung thêm tiến trình làm việc có thể khiến dịch vụ phụ thuộc chịu tải nặng hơn; việc mở rộng bộ đệm chỉ kéo dài thêm khoản nợ dồn ứ. Phòng thực nghiệm `projects/opsprobe/failurelab` bao bọc hàm `ProbeSingle` hiện có bằng một hàng đợi tiếp nhận hữu hạn mà không cần viết lại nhóm tiến trình hay giao diện lập trình thực tế của dự án.

Hãy khảo sát ca kiểm thử `TestBoundedAdmission` trước khi xem mã nguồn. Bộ thử nghiệm giữ lại hai yêu cầu tại điểm chặn (**barrier**) của máy chủ HTTP cục bộ. Hai công việc tiếp theo được đưa vào hàng đợi; 100 lần nộp thêm sau đó bắt buộc phải nhận lỗi `ErrFull` mà không làm phát sinh thêm bất kỳ tiến trình goroutine nào. Khi giải phóng điểm chặn và tiêu thụ bốn kết quả: số lượng công việc đang chạy và số lượng trong hàng đợi đều trở về 0, đỉnh công việc đang chạy không vượt quá 2. Những con số này đại diện cho khế ước về sức chứa của bộ thử nghiệm, không phải kết quả đo chuẩn phần cứng hay lời khuyên “môi trường thực tế chỉ nên chạy hai tiến trình”.

Tiếp theo, hãy thực thi `TestConsumerStopsContract`. Lượt thăm dò đã hoàn tất, điểm kiểm tra ngay trước lệnh gửi xác nhận tiến trình làm việc đang chuẩn bị bàn giao kết quả; bên tiêu thụ cố tình không đọc dữ liệu đầu ra. Lệnh hủy bỏ (**cancel**) bắt buộc phải cho phép tiến trình trả về và bộ điều phối đóng kênh `Done`. Phiên bản khiếm khuyết đã bỏ qua nhánh xử lý hủy bỏ tại lệnh gửi nên không thể kết thúc trong thời hạn của bài kiểm thử. Khung kiểm thử được trang bị lối thoát khẩn cấp riêng để thu hồi tài nguyên sau khi bộ phán đoán bắt lỗi; phiên bản lỗi tuyệt đối không được phép để lại một tiến trình goroutine bị mắc kẹt bên trong toàn bộ tập kiểm thử.

```
cd projects/opsprobe
go test -count=1 -v ./failurelab
$env:RELIABILITY_MUTANT='uncancellable_send'
go test -count=1 -run TestConsumerStopsContract ./failurelab
Remove-Item Env:RELIABILITY_MUTANT
```

Lệnh thứ hai bắt buộc phải báo lỗi đỏ. Trước khi xem lời giải, hãy chỉ rõ điểm tắc nghẽn khi không còn bên tiếp nhận và lý do tại sao ngữ cảnh trên yêu cầu HTTP không thể giải phóng điểm tắc nghẽn đó. Đây là bước tích hợp khế ước gửi dữ liệu qua kênh từ Chương 9, hoàn toàn không phải là kỹ thuật tinh chỉnh bộ điều phối tiến trình.

Giải thích: Quá trình trao đổi HTTP đã hoàn tất nên thời hạn kết thúc tại `ProbeSingle` không còn quản lý tiến trình làm việc nữa. Tiến trình vẫn đang chờ gửi kết quả; chỉ có cấu trúc `select` lắng nghe ngữ cảnh của bộ thực thi mới mở ra lối thoát an toàn. Bên gọi bắt buộc phải phát tín hiệu hủy bỏ rồi chờ kênh `Done` kết thúc khi quyết định ngừng tiêu thụ dữ liệu. Phiên bản khắc phục không bảo đảm bên tiêu thụ sẽ xử lý mọi kết quả sau khi đã hủy bỏ; đó là chính sách hủy bỏ những lượt bàn giao chưa kịp hoàn tất, một điều kiện cần phải chấp nhận trước khi sử dụng các giao diện truyền tải dữ liệu dạng luồng (**streaming**).

Lấy tín hiệu đủ sức phân biệt nguyên nhân

Ca kiểm thử `TestLocalFaultsAndSignals` lần lượt tiêm vào các trạng thái lỗi: tạm giữ và giải phóng, hết thời hạn, lỗi máy chủ 503, phần thân bị ngắt giữa chừng dù tiêu đề báo 200, và ngắt kết nối trước khi nhận phản hồi. Kỹ thuật tạm giữ và giải phóng tạo ra một phản hồi chậm trễ được kiểm soát bằng điểm chặn, không dựa vào việc tạm dừng thời gian (**sleep**) để suy đoán máy chủ đã nhận yêu cầu. Hết thời hạn là một lỗi thất bại được chờ đợi thực tế. Với mỗi trường hợp, bài kiểm thử đọc kết quả phân loại, một dòng nhật ký tương ứng, mẫu biểu đồ phân bố và khoảng thời gian `opsprobe.probe` từ các thành phần đo lường có sẵn. Nó hoàn toàn không gọi tới hệ thống thu thập tập trung, mạng Internet hay dịch vụ thực tế.

Bảng 23 — Chọn phép đo phân biệt thay vì đổi cấu hình theo triệu chứng

Triệu chứng vận hành	Các giả thuyết cạnh tranh	Bằng chứng kỹ thuật cần thu thập
Độ trễ cao	Dịch vụ phụ thuộc giữ phản hồi hay tiến trình chờ lượt chạy?	Điểm chặn máy chủ và khoảng truy vết HTTP; dùng phân tích vết mạng và lịch trình từ Chương 10 để kiểm tra thời gian chờ.
Hàng đợi bị đầy	Tải gửi đến vượt năng lực xử lý hay tiến trình không thể thoát?	Tỷ lệ từ chối, số lượng đang chạy và hàng đợi; ngừng nộp việc, hủy bỏ và thu hồi đúng tiến trình.
Bên gọi bị quá hạn	Dịch vụ phụ thuộc đã dừng hay chỉ phía máy khách thôi chờ?	Kênh `Done` phía máy khách và tín hiệu kết thúc độc lập của hàm xử lý phía nhận.
Bộ nhớ Heap tăng	Cấp phát tạm thời, lượng tồn đọng hợp lệ hay giữ tài nguyên sai?	Khối lượng tải và hàng đợi; hồ sơ bộ nhớ sau giai đoạn lắng dịu từ Chương 10 thay vì một bản chụp tức thời.
Thời gian cao, CPU thấp	Chờ dữ liệu vào ra hoặc đồng bộ hay chi phí CPU bị nhìn thiếu?	Phân tích vết thời gian chờ cùng hồ sơ CPU của đúng khối lượng tải; không coi mức sử dụng CPU thấp là bằng chứng tuyệt đối.

Bài kiểm thử `TestCancelDoesNotProveDependencyStopped` thiết lập một hàm xử lý cố tình không kiểm tra ngữ cảnh. Phía máy khách hủy bỏ và kết thúc thành công bộ thực thi, nhưng hàm xử lý vẫn chưa dừng lại cho đến khi bộ thử nghiệm giải phóng nó. Một thông báo quá hạn gửi về cho bên gọi không phải là lời xác nhận rằng dịch vụ phía dưới đã hủy bỏ hoàn toàn các hiệu ứng phụ. Đối với các thao tác có làm biến đổi dữ liệu, hãy quay lại bài toán định danh và khoảng thời gian gửi lặp ở Chương 13 và Chương 25 trước khi quyết định bổ sung cơ chế thử lại.

Bộ thực thi tính toán thời hạn kết thúc ngay từ thời điểm tiếp nhận, do đó thời gian nằm chờ trong hàng đợi được tính trực tiếp vào ngân sách thời gian; thời hạn riêng của từng mục tiêu có thể thu hẹp ngân sách này hơn nữa. Nếu công việc tới được tiến trình xử lý khi ngân sách thời gian đã cạn kiệt, yêu cầu sẽ không được cấp thêm một hạn mức mới để che giấu độ trễ chờ đợi. Chính sách ở đây là chủ động ngắt bỏ tải ngay khi hàng đợi bị lấp đầy và không tự động thử lại. Hàng đợi chỉ lưu giữ tối đa sức chứa mục tiêu

và số lượng tiến trình cố định; bên gọi vẫn bắt buộc phải giới hạn kích thước của từng dữ liệu đầu vào ngay tại ranh giới của ứng dụng thực tế.

Lượt thực thi với cờ `RUN_BOUNDED_LOAD=1` gửi chính xác 64 lượt thử nghiệm trong tối đa 10 giây, sử dụng bốn tiến trình làm việc và hàng đợi có kích thước 16. Lệnh này ghi nhận thông lượng, các mốc thời lượng p50/p95 của lượt thăm dò, số lượng từng loại kết quả, đỉnh điểm công việc đang chạy và trong hàng đợi, cùng các bản chụp trạng thái tiến trình goroutine và bộ nhớ heap. Biểu đồ phân bố `failurelab_job_seconds` còn tính gộp cả thời gian chờ trong hàng đợi; hai định nghĩa độ trễ này tuyệt đối không được phép trộn lẫn. Dữ liệu đầu ra thực tế và câu lệnh tái lập được lưu giữ trong tài liệu bằng chứng của bài thực hành. Việc so sánh thông lượng giữa các máy tính khác nhau không phải là chuẩn mực chân lý; dữ liệu heap chưa được dọn dẹp sau thu gom rác cũng không phải là bằng chứng rò rỉ bộ nhớ. Hãy áp dụng phòng thực nghiệm bộ nhớ từ Chương 10 để kiểm chứng giả thuyết đó thay vì tự ý dựng lên một công cụ phát hiện rò rỉ dựa trên các con số tùy tiện.

Đóng gói, điều phối và chuyển giao có trách nhiệm

Để đưa `opsprobe` ra môi trường production, ta áp dụng toàn bộ các nguyên tắc đã học ở Chương 17 và 18:

Một là, tệp cấu hình đóng gói vùng chứa nhiều giai đoạn: bài thực hành biên dịch tệp nhị phân với cờ `CGO_ENABLED=0`, sau đó sử dụng ảnh cơ sở tối giản `gcr.io/distroless/static-debian12:nonroot` và vận hành dưới tài khoản không có đặc quyền quản trị (`USER 65532:65532`). Với các gói phụ thuộc thuần Go của bài thực hành, giải pháp này không đòi hỏi môi trường thực thi ngôn ngữ C; tuy nhiên không thể suy diễn rộng thành lời cam đoan rằng mọi chương trình đều không có phụ thuộc bên ngoài tệp nhị phân.

Hai là, bài thực hành triển khai một Pod đơn lẻ và một tệp cơ sở dữ liệu SQLite, với số bản sao `replicas: 1`, ổ lưu trữ `ReadWriteOnce` (RWO) và chiến lược triển khai `Recreate`. Chế độ RWO giới hạn quyền ghi theo từng nút máy chủ, không phải theo từng Pod; nhiều Pod chạy trên cùng một nút máy chủ vẫn có thể cùng gắn ổ lưu trữ đó. Bản thân SQLite có cơ chế khóa

cho nhiều kết nối hoặc tiến trình, vì vậy không phải cứ hai tiến trình cùng truy cập là dữ liệu lập tức mất tính nhất quán. Rủi ro chỉ thực sự bùng nổ khi chia sẻ tệp qua hệ thống tệp mạng có ngữ nghĩa khóa và đồng bộ hóa không tương thích. Chiến lược `Recreate` hạn chế việc chồng lấn các phiên bản trong quá trình luân chuyển thông thường, hoàn toàn không thay thế được khóa phân tán hay cơ chế cô lập khi nút máy chủ gặp sự cố. Nếu đòi hỏi nhiều tiến trình cùng ghi từ các nút máy chủ khác nhau, hãy ưu tiên cơ sở dữ liệu theo mô hình máy khách - máy chủ (như PostgreSQL), hoặc thiết kế một tiến trình sở hữu duy nhất cho SQLite đi kèm khế ước chuyển đổi dự phòng rõ ràng.

Ba là, nạp cấu hình lúc khởi động qua đối tượng ConfigMap: Mức độ đồng thời, thời hạn tối đa, giới hạn áp suất ngược và cấp độ nhật ký được nạp từ tệp `deploy/k8s/configmap.yaml` vào các biến môi trường (`OPSPROBE_*`). Việc chỉnh sửa ConfigMap không tự động cập nhật giá trị biến môi trường của các tiến trình đang vận hành; hệ thống cần khởi tạo lại Pod hoặc thiết kế cơ chế nạp lại cấu hình độc lập.

Bốn là, ghim chặt nội dung ảnh container trong đường ống CI/CD: chính sách của dự án yêu cầu bản kê ảnh container phải sử dụng mã băm nội dung (`image: ghcr.io/...@sha256:...`) đã được đường ống kiểm chứng độc lập. Cách làm này loại bỏ hoàn toàn sự phụ thuộc vào nhãn phiên bản (như `:latest`) vốn có thể bị trôi sang ảnh khác; bản thân mã băm nội dung không tự chứng minh ảnh container là đáng tin cậy, tương thích đúng nền tảng phần cứng hay còn được kho lưu trữ duy trì.

Lệnh kiểm thử và vận hành hệ thống

Có thể trực tiếp chạy và kiểm chứng toàn bộ năng lực của `opsprobe` trong terminal:

```
cd projects/opsprobe

# 1. Chạy test suite có race detector
go test -race ./...

# 2. Chạy CLI kiểm tra URL đơn lẻ
go run ./cmd/opsprobe `
  --oneshot-url=https://go.dev --timeout=2s

# 3. Khởi chạy HTTP daemon server
go run ./cmd/opsprobe --addr=127.0.0.1:8080 `
  --db=opsprobe.db --concurrency=4
```

Trong cửa sổ khác, gửi yêu cầu probe và thu thập metrics:

```
$target = '{"id":"t1","url":"http://127.0.0.1:8080/livez"}'
curl -X POST http://127.0.0.1:8080/runs `
  -H "Content-Type: application/json" `
  -d "{`"targets`":[$target]}"
curl http://127.0.0.1:8080/metrics
```

Cột mốc hoàn thành dự án tổng kết: Chốt mốc chuẩn hệ thống

Dự án tổng kết xâu chuỗi các bằng chứng kỹ thuật thu hẹp: ngữ nghĩa giá trị và hiện tượng trùng tên vùng nhớ, đồng bộ hóa tiến trình goroutine, vòng đời phần thân phản hồi, tính nguyên tử của giao dịch và định danh bất biến của sản phẩm đóng gói. Mỗi bằng chứng đều có phạm vi giới hạn riêng; một giao dịch cơ sở dữ liệu không thể giải quyết mọi hiệu ứng phụ bên ngoài, quy trình tắt ứng dụng có thời hạn tối đa và có thể không kịp xử lý hết mọi yêu cầu dang dở, trong khi mã băm nội dung không tự chứng minh logic bên trong sản phẩm là hoàn toàn đúng đắn. Khi chuyển giao sang môi trường vận hành thực tế, hãy luôn duy trì các giới hạn đó trong ca kiểm thử và hệ thống quan sát.

Khi mở rộng công cụ, hãy luôn duy trì các khế ước kỹ thuật này làm tiêu chí đánh giá mã nguồn: dữ liệu đầu vào nào được tin cậy, tài nguyên nào có chủ thể chịu trách nhiệm, hiệu ứng phụ nào có thể lặp lại an toàn, và quan sát nào xác nhận kết quả cuối cùng. Chương 21 sẽ áp dụng chính những câu hỏi cốt lõi này cho một bộ điều khiển (**controller**).

Việc hoàn thành dự án `opsprobe` chốt lại baseline vững chắc của cuốn sách sống (**living textbook**), đồng thời mở ra những bài toán hệ thống ở quy mô hạ tầng cao hơn: khi hệ thống không chỉ thăm dò thụ động mà cần liên tục tự điều hòa, dung hòa sai lệch giữa trạng thái mong muốn và thực tế để tự phục hồi.

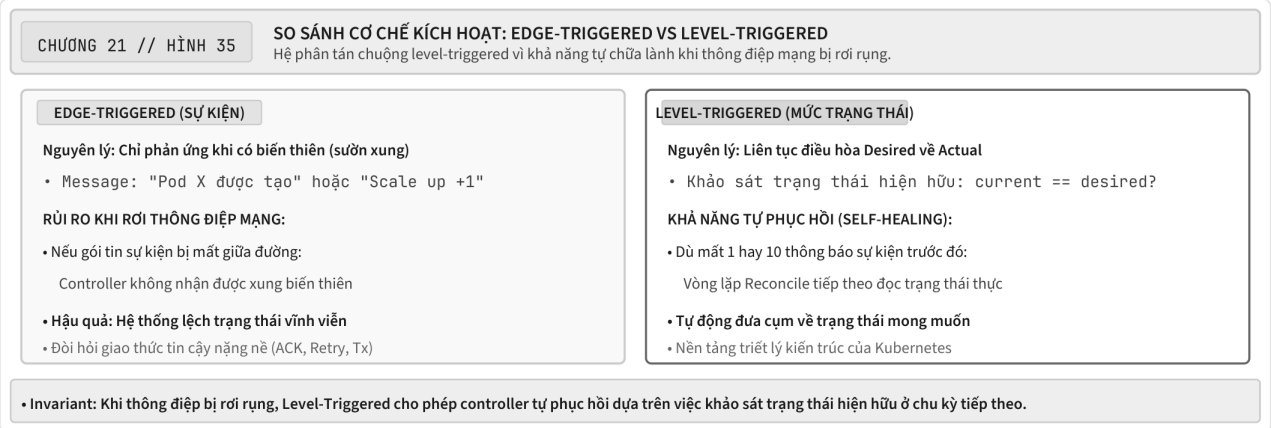
Chương 21 — Vòng lặp điều hòa và Controller Pattern

Trong Chương 20, dự án `opsprobe` đã trang bị cho ta một hệ thống thăm dò trạng thái hoàn chỉnh: kiểm tra định kỳ, phân loại kết quả rành mạch (`OutcomeSuccess`, `OutcomeFailure`, `OutcomeTimeout`), bảo toàn dữ liệu bằng giao dịch nguyên tử SQLite, và đo lường độ trễ qua Prometheus. Nhưng khi một dịch vụ phụ thuộc sập nguồn khiến dashboard chuyển màu đỏ rực, câu hỏi tiếp theo của một kỹ sư vận hành không còn là "làm sao để phát hiện?", mà là: "làm sao để hệ thống tự khôi phục trạng thái lành lặn mà không cần con người thức dậy lúc nửa đêm bấm nút restart?".

Chương này chuyển từ quan sát sang điều hòa: controller đọc trạng thái, tính sai lệch, thực hiện một thao tác rồi quan sát lại. Kubernetes dùng mẫu này cho nhiều controller; các công cụ khác có lịch chạy và mô hình quản lý state riêng, không nên gộp tất cả thành cùng một vòng lặp liên tục.

Kích hoạt theo mức và Kích hoạt theo cạnh

Để hiểu vòng lặp điều hòa trong controller của chương này, ta phân biệt hai cách thiết kế: kích hoạt theo cạnh (edge-triggered) và kích hoạt theo mức (level-triggered). Đây không phải lựa chọn bắt buộc của mọi hệ thống phân tán đáng tin cậy.



Hình 35 — Khi một thông điệp bị mất, controller dựa vào trạng thái có thể đọc lại observation và tiếp tục điều hòa. Khả năng hội tụ vẫn phụ thuộc việc quan sát và điều chỉnh thành công.

Trong mô hình kích hoạt theo cạnh (edge-triggered), phía tiêu thụ phản ứng theo các lần chuyển đổi trạng thái. Nếu mất sự kiện mà không có cơ chế phát lại, đồng bộ lại hoặc quan sát bổ sung, bên tiêu thụ có thể không nhận biết được trạng thái đã thay đổi và giữ nguyên kết luận cũ. Đây là lý do bộ điều khiển cần khả năng phục hồi từ quan sát hiện tại, thay vì chỉ dựa vào một chuỗi sự kiện được giả định là không bao giờ thất lạc.

Trong mô hình hướng trạng thái của bài thực hành, bộ điều khiển so sánh quan sát hiện tại với trạng thái mong muốn thay vì thực thi từng sự kiện như một mệnh lệnh. Mức mong muốn là 3 trong khi hiện tại là 1 sẽ dẫn tới đề xuất tạo thêm 2. Bản quan sát có thể cũ hoặc thiếu sót, nên chưa đủ để kết luận có thể tạo ngay: bộ phận thực thi (`actuator`) vẫn cần định danh, tiền điều kiện và quyền hạn phù hợp.

Ngay cả khi bộ điều khiển bị tắt đột ngột lúc đang khởi động bản sao thứ hai, ở lần thức dậy kế tiếp, nó lại tiếp tục so sánh và nhận ra sai lệch còn tồn tại. Nó có thể yêu cầu hành động tiếp, nhưng chỉ một quan sát mới tiếp theo mới xác nhận được sự hội tụ. Đặc tính hướng về trạng thái mong muốn này gọi là sự hội tụ trạng thái (state convergence).

Bốn pha của Vòng lặp điều hòa

Ta dùng bốn pha dưới đây làm mô hình đọc control loop của chương, không phải quy trình bắt buộc cho mọi controller:

Bước	Tên gọi	Nhiệm vụ kỹ thuật trong Go
1	**Observe**	Đọc trạng thái thực tế từ hệ thống hoặc store.
2	**Diff**	So sánh trạng thái thực tế với trạng thái mong muốn.
3	**Act**	Thực thi hành động khắc phục để triệt tiêu sai lệch.
4	**Schedule / Observe again**	Chờ sự kiện, resync hoặc lịch thử lại để đọc một quan sát mới; không suy diễn trạng thái từ kết quả lời gọi.

Trong Go, ta đóng gói hành vi này thành một interface nhỏ gọn:

```
type Result struct {
    Requeue      bool
    RequeueAfter time.Duration
}

type Reconciler interface {
    Reconcile(
        ctx context.Context, key string,
    ) (Result, error)
}
```

Quy ước này thể hiện rõ ranh giới trách nhiệm: `Reconciler` chỉ tập trung vào logic điều hòa của một tài nguyên duy nhất được định danh bởi `key`. Việc quản lý hàng đợi, phân phối worker và kiểm soát tần suất thử lại do tầng `Controller` bên ngoài đảm nhiệm.

Hàng đợi công việc có kiểm soát tốc độ

Một sai lầm thường gặp của lập trình viên Go là sử dụng ngay một kênh không đệm (`chan string`) để làm hàng đợi cho bộ điều khiển. Trên môi trường vận hành thực tế, kênh truyền thô nhanh chóng bộc lộ ba lỗ hổng nghiêm trọng:

Một là, thiếu cơ chế gộp khóa trùng lặp: trong kịch bản một bên tiêu thụ nhận và xử lý đủ 50 phần tử cùng khóa từ kênh mà không gộp, nó sẽ gọi hàm điều hòa 50 lần. Đây là cách dựng hàng đợi sơ sài, không tận dụng được đặc tính của bộ điều khiển; các lượt xử lý cũng có thể đọc được các quan sát khác nhau. Hàng đợi có trạng thái đánh dấu cần xử lý (`dirty`) và đang xử lý (`processing`) giúp gộp lịch thực thi, chứ không biến mọi tác dụng phụ thành xử lý chính xác một lần (`exactly-once`).

Hai là, xử lý cùng khóa song song: hai tiến trình xử lý (`worker`) có thể cùng đọc một quan sát thấy thiếu bản sao rồi đều phát yêu cầu tạo thêm. Tuần tự hóa theo khóa trong cùng một tiến trình giúp giảm xung đột đó, nhưng không thể phối hợp giữa nhiều tiến trình khác nhau hay ngăn chặn quan sát cũ; hệ thống vẫn cần quyền sở hữu (`ownership`) và tiền điều kiện ở bộ thực

thi. Tránh gọi mọi thao tác trùng lặp là hiện tượng phân đôi não (`split-brain`), vốn chỉ xảy ra khi nhiều bên cùng tự nhận mình giữ quyền điều phối tối cao.

Ba là, bão thử lại (`retry storm`): nếu thành phần phụ thuộc liên tục gặp lỗi mà khóa bị đẩy lại ngay mà không có ngân sách hay khoảng chờ giãn cách (`backoff`), tiến trình xử lý có thể tạo ra vòng lặp thử lại dồn dập, tiêu hao CPU và gia tăng áp lực lên dịch vụ phụ thuộc. Đây là hiện tượng bận thử lại (`busy retry`), khác với kỹ thuật khóa quay vòng (`spin-lock`). Chính sách thử lại luôn cần giới hạn tốc độ và hỗ trợ cơ chế hủy bỏ (`cancellation`).

Để nhìn rõ ba bài toán đó, lab dùng một `WorkQueue` tối giản. Nó mô phỏng các ý tưởng `dirty`, `processing` và đưa lại key sau `Done`, chứ không phải bản sao chính xác của `client-go/util/workqueue`.

```
type WorkQueue struct {
    mu           sync.Mutex
    cond         *sync.Cond
    queue        []string
    dirty        map[string]struct{}
    processing   map[string]struct{}
    failures     map[string]int
    shuttingDown bool
    config       RateLimiterConfig
}
```

Cơ chế phối hợp giữa `dirty` và `processing` tạo nên tính năng then chốt của hàng đợi:

```
func (q *WorkQueue) Add(item string) {
    q.mu.Lock()
    defer q.mu.Unlock()

    if q.shuttingDown {
        return
    }
    // 1. Deduplication: Nếu item đã nằm trong dirty, bỏ qua
    if _, exists := q.dirty[item]; exists {
        return
    }
    q.dirty[item] = struct{}{}

    // 2. Nếu item đang xử lý, không đẩy vào slice
    // để tránh hai worker cùng reconcile một key
    if _, isProcessing := q.processing[item]; isProcessing {
        return
    }

    q.queue = append(q.queue, item)
    q.cond.Signal()
}
```

Khi một worker hoàn tất lượt xử lý và gọi `Done(item)`:

```
func (q *WorkQueue) Done(item string) {
    q.mu.Lock()
    defer q.mu.Unlock()

    delete(q.processing, item)

    // Nếu trong lúc worker đang chạy, có thêm sự kiện mới
    // cho cùng key này, item lập tức được đưa lại vào queue
    if _, exists := q.dirty[item]; exists {
        q.queue = append(q.queue, item)
        q.cond.Signal()
    }
}
```

Trong mô hình này, một key không được hai worker xử lý đồng thời. Nếu `Add` xuất hiện khi key đang xử lý, `dirty` ghi nhận rằng cần có một lượt sau

`Done`. Đây là cơ chế gộp lịch xử lý, không phải cam kết exactly-once: worker có thể lỗi, tiến trình có thể dừng, và trạng thái có thẩm quyền vẫn phải được đọc lại ở lần reconcile sau.

> **Dừng để dự đoán:** Giả sử hàng đợi đang rỗng và Worker 1 vừa lấy key `"pod-1"` ra để chạy `Reconcile`. Trong lúc hàm reconcile đang thực thi dở dang, liên tiếp có hai sự kiện `Add("pod-1")` mới xuất hiện. Hàng đợi phối hợp `dirty` và `processing` như thế nào để đảm bảo worker thứ hai không nhảy vào can thiệp, và chuyện gì xảy ra khi Worker 1 gọi `Done("pod-1")`?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Bảng dưới đây mô tả sự dịch chuyển trạng thái nội bộ của hàng đợi qua từng thời điểm:

Thời điểm	Thao tác	`queue`	`dirty`	`processing`	Ý nghĩa vận hành
T0	Ban đầu	`[]`	`{}`	`{}`	Hàng đợi rỗng.
T1	`Add("pod-1")`	`["pod-1"]`	`{"pod-1"}`	`{}`	Đưa vào slice và đánh dấu dirty.
T1′	Worker 1 gọi `Get()`	`[]`	`{}`	`{"pod-1"}`	Lấy ra xử lý: xóa khỏi dirty, đưa vào processing.
T2	`Add("pod-1")` (lần 1)	`[]`	`{"pod-1"}`	`{"pod-1"}`	Thấy đang processing: chỉ đánh dấu dirty, không thêm vào slice.
T2′	`Add("pod-1")` (lần 2)	`[]`	`{"pod-1"}`	`{"pod-1"}`	Thấy đã trong dirty: gộp lại (deduplicate), bỏ qua.
T3	Worker 1 gọi `Done("pod-1")`	`["pod-1"]`	`{"pod-1"}`	`{}`	Xóa khỏi processing; thấy dirty còn key: đưa lại vào slice.
T4	Lượt `Get()` kế tiếp	`[]`	`{}`	`{"pod-1"}`	Reconcile lại để quan sát trạng thái mới nhất.

Cơ chế này hiện thực hóa mô hình điều hòa dựa trên trạng thái (level-triggered), không phải một hệ thống hàng đợi tin nhắn đảm bảo chuyển phát từng sự kiện riêng lẻ. Việc gộp sự kiện (coalescing) bảo đảm rằng dù có bao nhiêu thay đổi dồn dập trong lúc xử lý, hàng đợi chỉ lên lịch thêm đúng một lượt điều hòa kế tiếp sau khi tiến trình xử lý hiện tại hoàn tất, đồng thời ngăn chặn việc hai tiến trình cùng xử lý một khóa song song.

Quan trọng hơn, hàng đợi trong bộ nhớ không đưa ra cam kết xử lý chính xác một lần (exactly-once): nếu tiến trình bị dừng đột ngột, toàn bộ dữ liệu trong slice và map sẽ biến mất và bộ điều khiển phụ thuộc hoàn toàn vào chu kỳ quan sát kế tiếp để tái lập bức tranh trạng thái có thẩm quyền từ nguồn chân lý.

Kiểm soát giãn cách lũy thừa khi có sự cố

Khi `Reconcile` thất bại, thay vì đưa key trở lại hàng đợi ngay lập tức, ta áp dụng thuật toán **Exponential Backoff** thông qua phương thức `AddRateLimited`:

```
func (q *WorkQueue) AddRateLimited(item string) bool {
    q.mu.Lock()
    defer q.mu.Unlock()

    if q.shuttingDown {
        return false
    }
    failures := q.failures[item] + 1
    q.failures[item] = failures

    if failures > q.config.MaxRetries {
        return false // Vượt quá số lần thử lại tối đa
    }

    delay := retryDelay(
        q.config.BaseDelay, q.config.MaxDelay, failures)

    time.AfterFunc(delay, func() {
        q.Add(item)
    })
    return true
}
```

Hàm `retryDelay` trong source tăng gấp đôi nhưng kiểm tra `delay > max/2` trước khi nhân và bão hòa tại `max`. Cắt ngưỡng sau một phép nhân đã tràn là quá muộn: thời lượng có thể trở thành âm hoặc 0. Test `TestRetryDelaySaturatesBeforeOverflow` kiểm cả lần thất bại thứ

65 và thời lượng sát giới hạn số nguyên, không phụ thuộc scheduler hay độ trễ thực tế.

Với `BaseDelay=50ms`, lịch thử lại của mô hình này tăng theo 50ms, 100ms, 200ms, 400ms, rồi chạm trần `MaxDelay`. Khoảng giãn cách giúp giới hạn nhịp thử lại dồn dập khi hệ thống đích gặp lỗi; nó không loại bỏ mọi đợt thử lại đồng thời giữa nhiều khóa hay nhiều bộ điều khiển. Khi hàm điều hòa thành công, bộ điều khiển gọi `q.Forget(item)` để xóa bộ đếm thất bại.

Tính lũy đẳng: hành động có thể thử lại, quan sát được đọc lại

Vì một tài nguyên có thể được đưa vào hàng đợi nhiều lần do sự kiện lặp lại, đồng bộ lại định kỳ hoặc thử lại sau lỗi, logic điều hòa phải được thiết kế để an toàn khi gọi lại nhiều lần. Công thức toán học `f(f(x)) = f(x)` chỉ là một trực giác về trạng thái thuần túy; nó không phản ánh trọn vẹn khế ước vận hành thực tế, nơi lời gọi ra bên ngoài có thể thành công nhưng gói phản hồi bị thất lạc trên đường truyền, còn bộ đệm nhớ tạm có thể mang dữ liệu cũ.

Khi quan sát mới đã khớp với trạng thái mong muốn, hàm điều hòa không cần thực hiện thêm thao tác nào (no-op). Khi cần can thiệp, bộ thực thi phải có tiền điều kiện hoặc khóa lũy đẳng phù hợp với hệ thống đích. Sau khi phương thức `Act` trả về `nil`, bộ điều khiển mới chỉ biết lời gọi đã kết thúc không có lỗi; nó chưa thể khẳng định hệ thống bên ngoài đã hội tụ hoàn toàn. Sự xác nhận chỉ đến từ lần quan sát kế tiếp.

Hãy xem xét một Reconciler tự chữa lành cụ thể:

```
func (r *SelfHealingReconciler) Reconcile(
    ctx context.Context, key string,
) (Result, error) {
    state, exists := r.getObservedState(key)
    if !exists {
        return Result{}, nil // Không tồn tại: No-op
    }

    // 1. Phân tích sai lệch giữa thực tế và mong muốn
    isDegraded := !state.Healthy ||
        state.Replicas < r.desiredReplicas

    // 2. Đã đạt trạng thái mong muốn thì không đổi thêm
    if !isDegraded {
```

```
        return Result{}, nil
    }

    // 3. Thực thi hành động tự chữa lành (Act)
    if err := r.healTarget(ctx, key); err != nil {
        return Result{}, fmt.Errorf("heal %s: %w", key, err)
    }

    // 4. Không tự sửa actual state.
    // Event hoặc resync sẽ đưa key quay lại
    // khi lớp quan sát thấy trạng thái bên ngoài đã đổi.
    return Result{}, nil
}
```

Nếu `payment-service` đang có 1 bản sao trong khi yêu cầu là 3, hàm gọi bộ thực thi với `context.Context`. Một bài kiểm tra cố ý để bộ thực thi trả `nil` nhưng giữ quan sát cũ vẫn cho thấy mục tiêu chưa hoạt động bình thường; hàm điều hòa không được gán `Healthy=true` chỉ để làm bài kiểm thử đạt kết quả tốt. Khi quan sát sau đó phản ánh đủ 3 bản sao khỏe mạnh, lượt điều hòa kế tiếp mới không cần làm gì thêm (no-op). Bài thực hành cũng chặn bộ thực thi bằng kênh truyền rồi thay đổi quan sát trong lúc hành động đang chạy, để chứng minh một hành động cũ không được ghi đè quan sát mới.

Dừng có kiểm soát cho bộ điều khiển

Bộ điều khiển quản lý nhiều goroutine xử lý công việc. Khi context bị hủy, bộ điều khiển đóng hàng đợi để đánh thức các worker rồi chờ chúng thoát. Công việc đang chạy có thể trả lỗi do việc hủy theo yêu cầu (cancellation), không nhất thiết hoàn tất nghiệp vụ; chính sách xả cạn hàng đợi (drain) đến khi hoàn tất cần được thiết kế riêng:

```
func (c *Controller) Run(ctx context.Context) error {
    var wg sync.WaitGroup

    // Đánh thức và đóng hàng đợi khi context bị hủy
    go func() {
        <-ctx.Done()
        c.queue.ShutDown()
    }()

    for i := 0; i < c.concurrency; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for c.processNextWorkItem(ctx) {
            }
        }()
    }

    wg.Wait()
    return ctx.Err()
}
```

Phương thức `queue.ShutDown()` đánh thức goroutine đang chờ `cond.Wait()`. Worker nhận `shutdown == true` khi hàng đợi đã cạn, kết thúc vòng lặp rồi thông báo qua `wg.Done()`. Nó không tự hoàn tất thao tác bên ngoài; bộ thực thi phải tôn trọng `ctx`, và tác dụng phụ đã gửi cần tính lũy đẳng hoặc quan sát lại. Lab chưa lưu và dừng các timer `AfterFunc`: callback đến muộn sẽ bị `Add` từ chối sau shutdown, nhưng timer có thể còn giữ closure tới lúc hết hạn. Join worker không chứng minh mọi timer đã được thu hồi; không coi queue mô phỏng này là thư viện vận hành hoàn chỉnh.

Bài thực hành: Kiểm chứng Controller và WorkQueue

Toàn bộ kiến trúc trên được đóng gói tại thư mục thực hành `labs/part21-reconciliation-controller`:

```
cd labs/part21-reconciliation-controller
go test -v ./...     # Chạy toàn bộ test suite
go test -race ./...  # Kiểm tra race condition
go vet ./...        # Phân tích tĩnh cú pháp
```

Bộ kiểm thử dùng kênh truyền để xác nhận bộ thực thi nhận được tín hiệu hủy context, một hành động thành công không tự ý suy diễn quan sát, và quan sát mới xuất hiện khi hành động đang chạy không bị ghi đè. Bài kiểm tra giãn cách hiện chỉ xác nhận có nhiều lượt thử trong cửa sổ kiểm thử; nó chưa đo lường hoặc chứng minh chính xác các khoảng giãn cách tăng theo cấp số nhân.

Bước phát triển tiếp theo

Chương này xây dựng mô hình điều hòa tối giản: quan sát, so sánh sai lệch, thực thi hành động, rồi quan sát lại. Kết hợp với tín hiệu từ `opsprobe` (Chương 20), nó cho ta một nơi để thấy rõ vì sao kết quả hành động và trạng thái quan sát thực tế là hai bằng chứng kỹ thuật hoàn toàn độc lập.

Đây là nền tảng trực tiếp cho Chương 22, nơi cùng các câu hỏi ấy gặp List/Watch, bộ nhớ đệm và hàng đợi công việc của `client-go`; Chương 23 mới thêm API riêng, trạng thái, quyền sở hữu và cơ chế dọn dẹp trước khi xóa (finalizer) của một operator. Hàng đợi trong chương này là mô hình tái lập để nắm bắt cơ chế, không phải bản thay thế cho mã nguồn chính thức của Kubernetes.

Chương 22 — Từ watch đến một controller Kubernetes thật

Chương 21 sử dụng một hàng đợi tối giản để làm rõ sự khác biệt giữa kết quả hành động và trạng thái quan sát thực tế. Khi kết nối tới Kubernetes API, ta phải bổ sung cơ chế phục hồi luồng dữ liệu, đồng bộ hóa bộ nhớ đệm, điều phối theo khóa và xử lý xung đột. Một kênh truyền có thể là một thành phần trong thiết kế, nhưng bản thân nó không tự cung cấp các khế ước đó.

Chẳng hạn, trong kịch bản có 500 bản cập nhật dồn đến trong một giây, việc lưu giữ toàn bộ dữ liệu cũ để xử lý có thể khiến tiến trình hành động theo quan sát đã lỗi thời. Tải thực tế và hậu quả phải được đo lường trong môi trường cụ thể; ví dụ này nhằm đặt ra bài toán gộp khóa và đọc lại trạng thái.

Chương này sử dụng thư viện `client-go` v0.37.0 để dựng một bộ điều khiển nhỏ và kiểm tra các ranh giới kỹ thuật đó. Bài thực hành không thay thế cho thiết kế phân quyền RBAC toàn diện, cơ chế phục hồi thảm họa hay kiểm chứng trên cụm máy chủ vận hành thực tế.

---

1. Bản chất: Tín hiệu thông báo không phải là Chân lý

Câu hỏi trung tâm của chương này là:

> **Khi nhận được một sự kiện từ Kubernetes API Server, ta có nên tin vào dữ liệu đính kèm bên trong sự kiện đó để ra quyết định điều hòa hay không?**

Theo chính sách của bộ điều khiển trong chương, sự kiện chỉ là tín hiệu nhắc nhở để xếp khóa vào hàng đợi, chứ không phải bản chụp trạng thái được lưu giữ tới lúc ra quyết định. Hàm xử lý sự kiện vẫn có thể dùng dữ liệu đính kèm để lọc hoặc trích xuất siêu dữ liệu; tiến trình xử lý luôn đọc lại quan sát thực tế trước khi điều hòa.

Trong các hệ thống phân tán quy mô lớn, sự kiện thông báo (notification hint) chỉ là một tín hiệu cho biết tài nguyên tại một namespace có thể vừa

thay đổi. Bản thân sự kiện không phải là trạng thái thẩm quyền (authoritative state).

API Server là ranh giới thẩm quyền cho các đối tượng Kubernetes; bộ nhớ đệm của Informer chỉ là bản sao cục bộ giúp bộ điều khiển hoạt động hiệu quả, không phải nguồn chân lý duy nhất và có thể mang độ trễ. Tiến trình xử lý đọc bộ đệm để tránh bám vào dữ liệu sự kiện đã cũ, nhưng các thao tác ghi hoặc đọc ngay sau khi ghi luôn cần chính sách rõ ràng về việc đọc trực tiếp từ API hay từ bộ đệm.

Bẫy tư duy: Xử lý theo payload của sự kiện

Xét kịch bản điều phối sau: tại thời điểm t_0, đối tượng `Pod` mang tên `web-app` có trạng thái `Pending` và API Server phát ra sự kiện E_1`(Pending)`. Đến thời điểm t_1, Kubelet khởi động xong container khiến Pod chuyển sang `Running`, và API Server phát sự kiện E_2`(Running)`. Do độ trễ mạng hoặc hàng đợi bận rộn, tiến trình trong bộ điều khiển nhận E_1 chậm mất 3 giây. Nếu tiến trình dùng ngay dữ liệu bên trong E_1, nó sẽ lầm tưởng Pod vẫn đang ở trạng thái chờ và ra lệnh hủy Pod để tạo lại. Hành động này vô tình tiêu hủy một Pod đang vận hành ổn định.

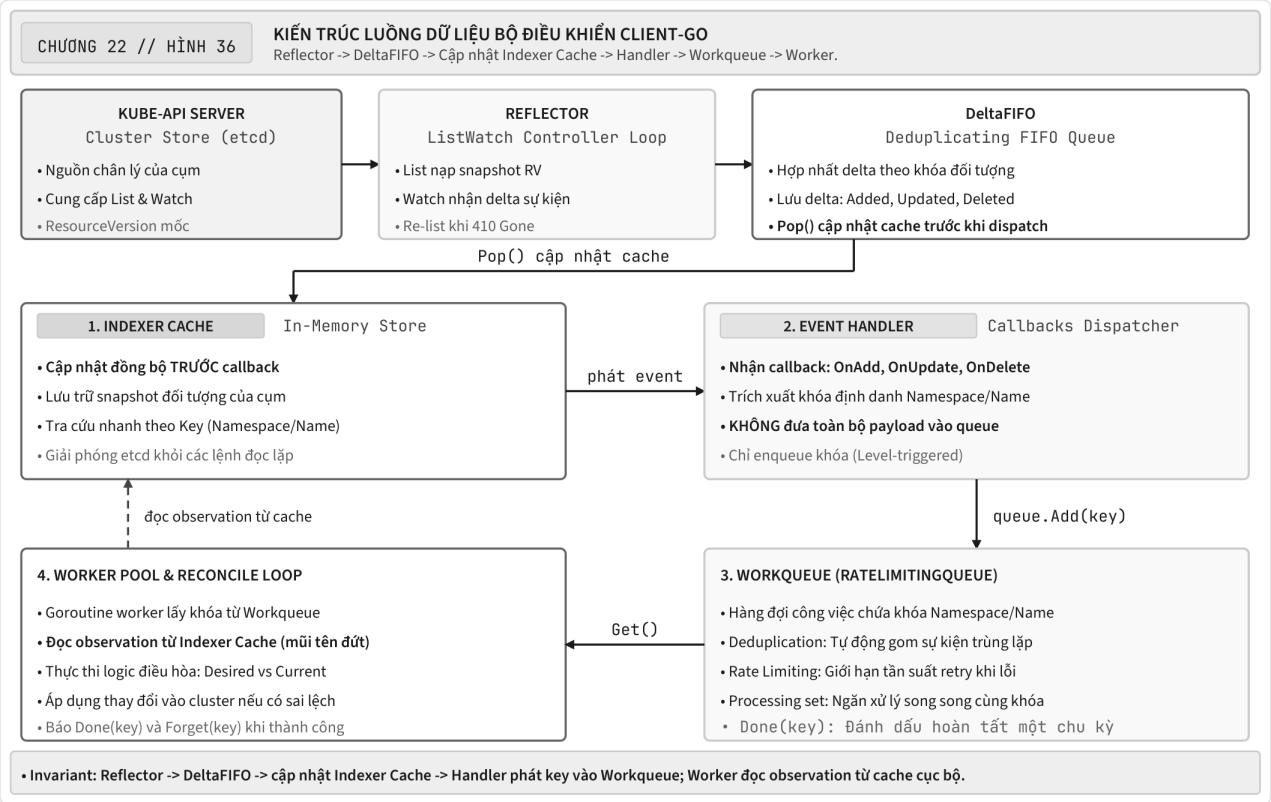
Chính sách áp dụng trong bài thực hành:

> **Quy tắc hàng đợi:** Đẩy khóa `namespace/name` vào hàng đợi, sau đó đọc quan sát hiện hành trong bộ đệm của Informer khi tiến trình xử lý. Bộ đệm chỉ là quan sát cục bộ và có thể trễ so với API Server; không thể coi đó là trạng thái mới nhất tuyệt đối của toàn cụm.

---

2. Kiến trúc toàn cảnh của một client-go Controller

Luồng xử lý chính đi từ API Server qua `Reflector` và `DeltaFIFO` tới `SharedInformer`. Bộ đệm được cập nhật trước khi hàm xử lý đẩy khóa vào hàng đợi; khi thực thi, tiến trình đọc lại quan sát cục bộ thay vì lưu giữ dữ liệu sự kiện cũ.



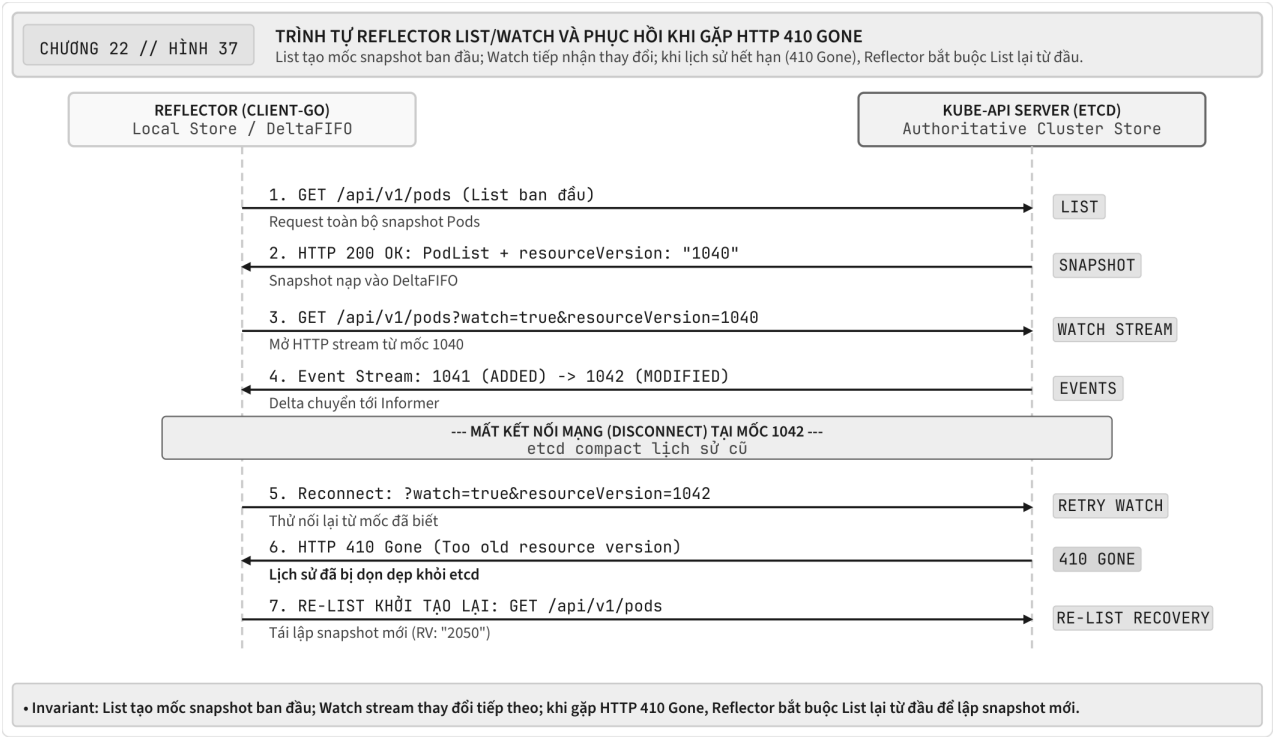
Hình 36 — Luồng sự kiện của bộ điều khiển. Đường nét đứt biểu thị bộ đệm cục bộ: dữ liệu này có thể có độ trễ so với API Server.

Bốn thành phần then chốt trong chuỗi cung ứng dữ liệu

Thành phần Informer	Vai trò kỹ thuật	Tương tác trong hệ thống
`Reflector`	Mở kết nối List/Watch tới API Server	Đẩy các biến động tài nguyên vào bộ đệm `DeltaFIFO`.
`Indexer (Local Store)`	Bộ nhớ đệm in-memory có đồng bộ truy cập	Phục vụ lookup local và giảm read tới API server; chi phí cần đo theo index và workload.
`SharedInformer`	Phân phối sự kiện từ `DeltaFIFO`	Đảm bảo local cache cập nhật trước khi event handler kích hoạt.
`Typed Rate-Limited WorkQueue`	Hàng đợi công việc chuyên dụng	Gộp key trùng và kiểm soát thử lại; không thay thế đồng bộ dữ liệu dùng chung.

3. Giao thức List/Watch và vai trò của resourceVersion

Thao tác `List` cung cấp bản chụp trạng thái (`snapshot`) kèm số phiên bản tài nguyên (`resourceVersion`); Watch tiếp nhận các thay đổi theo mốc được API hỗ trợ. Đây là khế ước kỹ thuật của Kubernetes API, không phải quy tắc cấm hệ thống thăm dò định kỳ hay sử dụng phương thức truyền tải khác.



Hình 37 — List tạo mốc quan sát cho Watch; khi nối lại không thể tiếp tục vì lịch sử đã hết, Reflector phải List lại.

Cơ chế hoạt động

Thứ nhất là luồng List truyền thống: Reflector lấy danh sách đối tượng kèm số phiên bản tài nguyên không trong suốt (`resourceVersion opaque`). Nó đưa bản chụp vào kho lưu trữ `DeltaFIFO`; chu trình xử lý của Informer sẽ cập nhật Indexer, chứ không phải Reflector ghi thẳng vào Indexer. Sơ đồ minh họa đường đi này. Phiên bản client-go được ghim còn hỗ trợ cơ chế watch-list khi máy chủ và cấu hình cho phép; ta không nên suy diễn rằng mọi lần khởi tạo đều phải phát lệnh List riêng lẻ.

Thứ hai là luồng `Watch` từ mốc đã nhận, chẳng hạn `?watch=true&resourceVersion=1040`. API gửi luồng thay đổi phát sinh sau mốc đó chừng nào lịch sử sự kiện còn được lưu giữ. Với HTTP/1.1, kết nối có thể dùng kiểu truyền tải phân đoạn (`chunked transfer`); HTTP/2

không áp dụng cơ chế đóng khung đó. Lưu ý rằng `resourceVersion` không phải độ lệch (`offset`) của etcd mà client có quyền tự diễn giải.

Thứ ba là tự phục hồi sau sự cố mạng: Nếu đường truyền bị đứt ở sự kiện `1042`, Reflector thử kết nối lại từ phiên bản đã biết. Đây là cơ chế bắt kịp thay đổi khi lịch sử còn giữ được, không phải lời hứa rằng mọi lần nối lại đều không cần List.

Thứ tư là xử lý lỗi HTTP 410 Gone: Nếu kết nối bị gián đoạn quá lâu khiến etcd đã dọn dẹp các bản ghi lịch sử (compact log), API Server sẽ trả về mã lỗi `410 Gone (Too old resource version)`. Khi đó, Reflector hiểu rằng khoảng trống dữ liệu không thể bù đắp, nó sẽ tự động kích hoạt một chu kỳ `List` mới từ đầu để tái lập snapshot chuẩn.

---

4. Xử lý xóa và bí ẩn của Tombstone

Trong vòng đời của một tài nguyên, sự kiện xóa (`OnDelete`) ẩn chứa một cạm bẫy kỹ thuật kinh điển mà hầu hết kỹ sư mới tiếp cận `client-go` đều vấp phải: **Tombstone** (Bia mộ dữ liệu).

Khi không nhận được sự kiện xóa nhưng một lần `List` lại cho thấy khóa đã biến mất, `DeltaFIFO` có thể tạo đối tượng `DeletedFinalStateUnknown`. Trường `Obj` lưu giữ quan sát cuối cùng được ghi nhận, có thể đã cũ chứ không phải bản chụp tại thời điểm xóa thực tế. Hàm xử lý cần chấp nhận cả đối tượng thông thường lẫn bia mộ dữ liệu (`tombstone`), đồng thời không suy diễn định danh mới từ tên gọi cũ.

Nếu ta ép kiểu trực tiếp:

```
// CẢNH BÁO: Panic runtime nếu đối tượng rơi vào Tombstone!
pod := obj.(*corev1.Pod)
```

Chương trình sẽ văng lỗi `panic: interface conversion: interface {} is cache.DeletedFinalStateUnknown, not *v1.Pod`.

Giải pháp an toàn tiêu chuẩn:

Thư viện `client-go` cung cấp hàm chuẩn hóa `cache.DeletionHandlingMetaNamespaceKeyFunc`:

```
func (c *Controller) handleDelete(obj any) {
    key, err := cache.DeletionHandlingMetaNamespaceKeyFunc(obj)
    if err != nil {
        runtime.HandleError(err)
        return
    }
    c.queue.Add(key)
}
```

Hàm `DeletionHandlingMetaNamespaceKeyFunc` hỗ trợ trích xuất khóa cho cả đối tượng thông thường lẫn bia mộ dữ liệu `DeletedFinalStateUnknown`. Bên gọi vẫn phải xử lý lỗi nếu đối tượng thiếu siêu dữ liệu phù hợp; hàm trợ giúp này không tự đảm bảo mọi dữ liệu truyền vào đều an toàn.

Khóa định danh chỉ mang thông tin `namespace/name`, do đó không thể phân biệt một đối tượng cũ đã bị xóa với một đối tượng mới được tạo cùng tên. Nếu tác vụ dọn dẹp bên ngoài Kubernetes đòi hỏi định danh bền vững, hãy ghi nhận `metadata.uid` khi đối tượng còn tồn tại và ràng buộc tài nguyên ngoại vi với UID đó; tránh suy diễn UID từ một bia mộ chỉ còn sót lại khóa.

---

5. Hàng đợi WorkQueue: Ba tập hợp và Kiểm soát tốc độ

Kênh truyền chỉ cung cấp cơ chế gửi nhận và thứ tự theo ngữ nghĩa của ngôn ngữ Go; bản thân nó không có tập hợp đánh dấu chờ (`dirty set`), tập hợp đang xử lý (`processing set`) hay chính sách thử lại. Trong bản hiện thực của client-go v0.37.0, `workqueue.Typed[T]` phối hợp hàng đợi cùng hai tập hợp khóa sau:

Tập hợp	Ý nghĩa kỹ thuật	Trách nhiệm
`queue []T`	Danh sách thứ tự chờ	Xác định key nào sẽ được worker lấy ra xử lý kế tiếp.
`dirty set[T]`	Tập hợp các key bị bẩn	Lưu các key có sự kiện phát sinh nhưng chưa hoàn tất điều hòa. Giúp **gộp trùng lặp (deduplication)**.
`processing set[T]`	Tập hợp key đang xử lý	Cơ chế queue để cùng key không được phân phối song song. Đây không thay thế race detector hay synchronization cho state do Reconciler sở hữu.

Quy trình điều phối của WorkQueue

`Add(key)` trước hết đánh dấu key trong `dirty`; key đã bẩn thì không được xếp thêm một bản sao. Nếu key đang ở `processing`, sự kiện mới chờ lượt xử lý hiện tại kết thúc. Nếu không, key được đưa vào `queue`. `Get()` lấy key ra, xóa dấu `dirty` và đánh dấu nó đang xử lý.

Khi worker xử lý xong, nó bắt buộc phải gọi `queue.Done(key)`. Lúc này WorkQueue sẽ xóa key khỏi tập `processing`. Nếu trong quá trình worker đang chạy mà có sự kiện mới tới (key đã được đánh dấu vào `dirty`), `Done()` sẽ tự động đưa key đó trở lại `queue` để xử lý tiếp!

Cơ chế Rate Limiting: Quên hay Phạt?

Một tiến trình xử lý sau khi hoàn tất một lượt điều hòa (`reconcile`) sẽ đứng trước hai tình huống:

Thứ nhất, nếu thành công: Gọi `c.queue.Forget(key)` để xóa sạch lịch sử số lần thất bại, đưa bộ đếm backoff về mức 0, và gọi `c.queue.Done(key)`.

Thứ hai, nếu gặp thất bại tạm thời: Không gọi `Forget`, mà gọi `c.queue.AddRateLimited(key)`. Thời gian chờ do `RateLimiter` đã cấu hình quyết định. Bài thực hành sử dụng `DefaultTypedItemBasedRateLimiter` của client-go v0.37.0: giãn thời gian thử lại theo hàm mũ dựa theo khóa, có mức trần, không tự thêm biến thiên ngẫu nhiên (`jitter`) hay cơ chế thùng thẻ bài (`token bucket`) toàn

cục. Đây là cơ chế kiểm soát tốc độ thử lại, không đảm bảo loại bỏ hoàn toàn mọi sự cố dây chuyền.

---

6. Hiện thực Controller hoàn chỉnh trong Go

Đoạn dưới là biến thể sư phạm sử dụng interface `Reconciler`, không phải bản chép nguyên source lab. Trong `labs/part22-client-go-controller/controller.go`, điểm mở rộng là callback `ReconcileFn`; khi không cấp callback, `reconcile` ghi kết quả vào `results` và `Results()` trả bản sao. Nhánh kiểm `DeletionTimestamp` dưới đây cũng không có trong lab đó. Cả hai dùng API generic của client-go v0.37.0. Test lab chứng minh implementation trong repository, không tự chứng minh biến thể in sách; khi đối chiếu, phải giữ rõ ranh giới này.

```
type Reconciler interface {
    Reconcile(ctx context.Context, key string) error
}

type Controller struct {
    indexer    cache.Indexer
    queue      workqueue.TypedRateLimitingInterface[string]
    informer   cache.SharedIndexInformer
    reconciler Reconciler
}
```

Hàm khởi chạy Run bảo đảm khế ước đồng bộ hóa bộ nhớ đệm trước khi phân phối công việc cho các worker:

```
func (c *Controller) Run(
    ctx context.Context, workers int,
) error {
    defer runtime.HandleCrash()
    defer c.queue.ShutDown()

    // Khế ước bắt buộc: Đợi local cache đồng bộ hoàn tất
    if c.informer != nil {
        synced := cache.WaitForNamedCacheSyncWithContext(
            ctx, c.informer.HasSynced,
        )
        if !synced {
            return fmt.Errorf("hết hạn chờ sync cache")
        }
    }
```

```
    var wg sync.WaitGroup
    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for c.processNextItem(ctx) {
            }
        }()
    }

    <-ctx.Done()
    c.queue.ShutDown()
    wg.Wait()
    return nil
}
```

Vòng lặp điều phối của từng Worker

Mỗi tiến trình xử lý (`worker`) lấy một khóa ra khỏi hàng đợi và bọc trong khối bảo vệ:

```
func (c *Controller) processNextItem(ctx context.Context) bool {
    key, shutdown := c.queue.Get()
    if shutdown {
        return false
    }
    defer c.queue.Done(key)

    err := c.reconcileHandler(ctx, key)
    if err == nil {
        // Thành công: Xóa lịch sử retry để giải phóng bộ nhớ
        c.queue.Forget(key)
        return true
    }

    // Thất bại: Thử lại có kiểm soát tốc độ (tối đa 5 lần)
```

```
    if c.queue.NumRequeues(key) < 5 {
        c.queue.AddRateLimited(key)
        return true
    }

    // Đã vượt quá ngưỡng thử lại: Hủy bỏ và ghi log lỗi
    c.queue.Forget(key)
    runtime.HandleError(
        fmt.Errorf(
            "bỏ cuộc sau 5 lần thử key %q: %w", key, err,
        ),
    )
    return true
}
```

Logic hòa giải: Đọc từ Cache thay vì Payload sự kiện

```
func (c *Controller) reconcileHandler(
    ctx context.Context, key string,
) error {
    // 1. Truy vấn snapshot quan sát hiện có từ local indexer
    obj, exists, err := c.indexer.GetByKey(key)
    if err != nil {
        return fmt.Errorf("lỗi đọc cache key %s: %w", key, err)
    }

    if !exists {
        // Không thấy tài nguyên trong cache cục bộ.
        return c.reconciler.Reconcile(ctx, key)
    }
```

```
    pod, ok := obj.(*corev1.Pod)
    if !ok {
        return fmt.Errorf("sai kiểu *corev1.Pod: %T", obj)
    }

    // Không xử lý pod đang trong tiến trình bị xóa dở
    if pod.DeletionTimestamp != nil {
        return nil
    }

    return c.reconciler.Reconcile(ctx, key)
}
```

---

7. Bằng chứng kiểm thử: Kiểm chứng 6 hành vi then chốt

Bộ kiểm thử tại `labs/part22-client-go-controller/controller_test.go` kiểm tra sáu khế ước kỹ thuật của bài thực hành. Các bài kiểm tra này không mô phỏng mọi điều kiện biên trên cụm máy chủ vận hành thực tế:

```
=== RUN   TestEventDeduplication
--- PASS: TestEventDeduplication (0.00s)
=== RUN   TestCacheNewerThanEvent
--- PASS: TestCacheNewerThanEvent (0.00s)
=== RUN   TestRateLimitedRetryDecoupledFromDomain
--- PASS: TestRateLimitedRetryDecoupledFromDomain (0.00s)
=== RUN   TestSafeDeletionAndTombstone
--- PASS: TestSafeDeletionAndTombstone (0.00s)
=== RUN   TestCleanCancellationShutdown
--- PASS: TestCleanCancellationShutdown (0.05s)
=== RUN   TestCacheNotSyncedGuard
--- PASS: TestCacheNotSyncedGuard (0.02s)
PASS
ok      part22-client-go-controller   3.829s
```

1. Bằng chứng gộp trùng sự kiện (TestEventDeduplication)

Bài kiểm tra thêm mười lần cùng khóa của đối tượng `Pod` mang tên `production/web-proxy` vào hàng đợi trước khi tiến trình xử lý bắt đầu. Hàng đợi chỉ chứa đúng một phần tử và sau một lần xử lý, bộ đếm số lần điều hòa bằng 1. Kết quả chứng minh khả năng gộp các khóa đang chờ trong lịch chạy này, chứ không hàm ý giảm 90% tải hay cam kết xử lý chính xác một lần khi sự kiện dồn đến trong lúc đang thực thi.

2. Đọc trạng thái mới từ Cache thay vì Payload cũ (TestCacheNewerThanEvent)

Bài kiểm tra đưa đối tượng `Pod` có `ResourceVersion: "1"` vào hàng đợi, rồi cập nhật bộ chỉ mục `Indexer` thành `"3"` trước khi tiến trình xử lý tiếp nhận. Kết quả quan sát ghi nhận giá trị `"3"`, xác nhận luồng xử lý luôn đọc lại bộ đệm thay vì giữ nguyên dữ liệu sự kiện cũ. Bài kiểm tra này không nhằm chứng minh bộ đệm luôn hoàn toàn đồng bộ với API Server.

3. Tách biệt lỗi tạm thời và Lũy thừa thử lại (TestRateLimitedRetryDecoupledFromDomain)

Bài kiểm tra `TestRateLimitedRetryDecoupledFromDomain` giả lập hai lần điều hòa thất bại và lần thứ ba thành công. Các câu khẳng định kiểm tra hai lượt thử lại, ba lượt triệu gọi và không làm rơi mất yêu cầu; nhánh thành công trong mã nguồn gọi `Forget(key)` để xóa lịch sử thử lại của khóa.

4. An toàn trước Tombstone (TestSafeDeletionAndTombstone)

Bài kiểm tra truyền đối tượng `cache.DeletedFinalStateUnknown` qua hàm đưa vào hàng đợi, xác nhận khóa `production/tombstone-pod` được tiếp nhận và ghi nhận hành động xóa trong kết quả. Bài thực hành không thực hiện cũng như không chứng minh việc dọn dẹp tài nguyên ngoại vi.

5. Dừng sạch tài nguyên (TestCleanCancellationShutdown)

Bài kiểm tra kích hoạt ba tiến trình xử lý, thêm năm khóa rồi phát tín hiệu `cancel()`. Câu khẳng định yêu cầu hàm `ctrl.Run` phải kết thúc trong vòng hai giây; khoảng dừng 50 ms là thời gian chờ trước khi hủy theo yêu cầu, không phải chỉ số đo lường tắt hệ thống. Mã nguồn chủ động đóng hàng đợi và chờ các tiến trình thoát, nhưng bài kiểm tra này không kiểm kê toàn bộ goroutine hay cam kết độ trễ dừng hệ thống trên môi trường sản xuất.

6. Khế ước bảo vệ Cache Chưa Đồng Bộ (TestCacheNotSyncedGuard)

Trong bài thực hành, nếu hàm `WaitForNamedCacheSyncWithContext` trả về `false`, bộ điều khiển sẽ từ chối khởi động các tiến trình xử lý và trả về lỗi đồng bộ hóa bộ đệm. Cổng kiểm soát này chặn luồng xử lý trước khi việc đồng bộ hoàn tất trong vòng đời hiện tại; việc đồng bộ xong không bảo đảm bộ đệm sẽ không bao giờ bị cũ hay phân quyền RBAC sẽ không đổi về sau.

---

8. Kiểm soát đồng thời lạc quan (OCC) và Lỗi 409 Conflict

Khi controller quyết định cập nhật trạng thái `Pod` lên máy chủ API, nó gửi một lệnh HTTP `PUT` hoặc `PATCH`. Trong môi trường phân tán, một controller khác hoặc người dùng thông qua `kubectl` có thể đã sửa đổi Pod đó trước ta một phần nghìn giây.

Với thao tác `Update` dưới đây, `resourceVersion` được dùng làm tiền điều kiện kiểm tra: phiên bản cũ sẽ khiến máy chủ API trả về mã lỗi `409 Conflict`. Không phải mọi lệnh PATCH đều áp dụng cùng điều kiện; một bản vá có thể bỏ qua `resourceVersion` hoặc sử dụng cơ chế xử lý xung đột và quyền sở hữu trường khác. Hãy đọc kỹ khế ước của thao tác ghi đang áp dụng, tránh suy diễn cơ chế đồng thời lạc quan (OCC) chỉ thuần túy dựa vào phương thức HTTP.

```
Operation cannot be fulfilled on pods "payment-api":
the object has been modified; please apply your changes
to the latest version and try again
```

Mẫu hình chuẩn: retry.RetryOnConflict

Tuyệt đối không coi 409 là một lỗi hệ thống nghiêm trọng khiến controller sập nguồn. Đây là một hiện tượng bình thường trong hệ phân tán. Cách xử lý chuẩn xác bằng `k8s.io/client-go/util/retry`:

```
func updatePodAnnotation(
    ctx context.Context,
    client kubernetes.Interface,
    namespace, name string,
) error {
    return retry.RetryOnConflict(
        retry.DefaultRetry,
        func() error {
            // 1. Đọc lại bản ghi hiện tại từ API Server
            latest, err := client.CoreV1().Pods(namespace).Get(
                ctx, name, metav1.GetOptions{},
            )
            if err != nil {
                return err
            }
```

```
            // 2. Thực hiện sửa đổi trên bản sao mới
            if latest.Annotations == nil {
                latest.Annotations = make(map[string]string)
            }
            now := time.Now().UTC().Format(time.RFC3339)
            latest.Annotations["reconciled-at"] = now

            // 3. Ghi đè lại API Server
            _, err = client.CoreV1().Pods(namespace).Update(
                ctx, latest, metav1.UpdateOptions{},
            )
            return err
        },
    )
}
```

Mẫu hình này tự động thử lại với backoff ngắn, đọc lại snapshot hiện tại của đối tượng từ API Server, áp dụng thay đổi và lưu lại.

---

9. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Đọc API trực tiếp ở mọi lượt mà không có budget.	Tăng tải và độ trễ tùy workload.	Dùng cache khi freshness cho phép; dùng API reader có chủ đích cho precondition hay read-after-write.
Quên Done(key).	Key còn trong processing; Add mới có thể đánh dirty nhưng không được xếp lại cho tới Done.	Đặt defer Done sau Get thành công.
Gọi Forget(key) khi Reconcile gặp lỗi.	Làm mất lịch sử thử lại của key. Backoff bị xóa bỏ, dẫn đến bão thử lại liên tục.	Chỉ gọi `Forget(key)` khi điều hòa thành công hoặc khi quyết định từ bỏ sau nhiều lần thử.
Chạy worker pool trước khi đồng bộ xong cache.	Nếu diễn giải object chưa có trong cache thành đã bị xóa, worker có thể cleanup nhầm.	Gate worker bằng cache sync; ngay cả sau sync, cleanup vẫn cần bằng chứng identity/ownership riêng.

Bài thực hành: Thiết kế Controller và WorkQueue an toàn

Thử thách 1: Tích hợp Metric đo đạc độ trễ và chiều sâu hàng đợi

Yêu cầu: Hãy thiết kế hai metric Prometheus để giám sát sức khỏe của controller: metric Gauge `controller_workqueue_depth` đo số lượng key đang chờ xử lý trong `queue`, và metric Histogram `controller_reconcile_duration_seconds` đo thời gian thực thi của một chu kỳ `Reconcile`.

Chỉ dẫn triển khai: Đặt điểm đo `time.Now()` trước khi gọi `c.reconciler.Reconcile` và cập nhật metric trong lệnh `defer`.

Thử thách 2: Xử lý Pod bị cô lập (Orphaned Pod Cleanup)

Yêu cầu: Khi cache không còn object, key chỉ chứa namespace/name. Viết nhánh từ chối xóa file ngoài cụm nếu không có record durable ràng buộc UID và file. Chứng minh trường hợp object mới cùng tên không bị cleanup nhầm. Đây là bài tập nhận diện thiếu bằng chứng, không phải yêu cầu xóa cho bằng được.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Lời giải Thử thách 1: Đo lường quan sát controller

Khai báo các bộ đo Prometheus:

```
var (
    queueDepth = prometheus.NewGaugeVec(
        prometheus.GaugeOpts{
            Name: "controller_workqueue_depth",
            Help: "Số lượng phần tử đang chờ trong workqueue",
        },
        []string{"controller"},
    )
    reconcileDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "controller_reconcile_duration_seconds",
            Help:    "Thời gian thực thi hàm Reconcile",
            Buckets: prometheus.DefBuckets,
        },
        []string{"controller", "status"},
    )
)
```

Thu thập số liệu đo lường trong vòng lặp xử lý:

```
func (c *Controller) processNextItemWithMetrics(
    ctx context.Context,
) bool {
    key, shutdown := c.queue.Get()
    if shutdown {
        return false
    }
    defer c.queue.Done(key)

    start := time.Now()
    err := c.reconcileHandler(ctx, key)
    duration := time.Since(start).Seconds()

    status := "success"
    if err != nil {
        status = "error"
    }
    reconcileDuration.WithLabelValues(
        "pod_controller", status,
    ).Observe(duration)
    // ... xử lý retry tiếp theo
    return true
}
```

Lời giải Thử thách 2: Xử lý dọn dẹp tài nguyên cô lập

```
func (r *CleanUpReconciler) Reconcile(
    ctx context.Context, key string,
) error {
    namespace, name, err := cache.SplitMetaNamespaceKey(key)
    if err != nil {
        return fmt.Errorf("key sai %q: %w", key, err)
    }

    obj, exists, err := r.indexer.GetByKey(key)
    if err != nil {
        return err
    }

    if !exists {
```

```
        // Key chỉ có namespace/name, không có UID.
        // Không xóa ngoại vi theo tên.
        // Một Pod cùng tên có thể đã được tạo lại.
        return fmt.Errorf(
            "pod %s/%s absent: cleanup needs recorded UID",
            namespace, name,
        )
    }

    // Pod vẫn tồn tại -> Điều hòa trạng thái bình thường
    _ = obj.(*corev1.Pod)
    return nil
}
```

Vì thế, đừng bao giờ đặt các tác vụ dọn dẹp nguy hiểm bên ngoài hệ thống trong nhánh `!exists` chỉ dựa vào khóa định danh. Thiết kế trên môi trường vận hành thực tế nên dùng cơ chế dọn dẹp trước khi xóa (finalizer) khi đối tượng vẫn còn mang `metadata.uid`, hoặc sử dụng một bản ghi bền vững đã liên kết UID với tài nguyên ngoại vi. Khi chỉ còn lại `namespace/name`, lựa chọn an toàn là giữ nguyên và chuyển tình huống vào quy trình đối soát thủ công.

Cơ chế `List/Watch`, bộ đệm nhớ tạm và hàng đợi công việc cung cấp các khế ước kỹ thuật chuyên biệt cho bộ điều khiển. Chương 23 sẽ bổ sung API tùy biến, quyền sở hữu và cơ chế finalizer nhằm xác định chuẩn xác thời điểm tài nguyên ngoài cụm được phép dọn dẹp an toàn.

Chương 23 — Từ controller đến operator: API riêng và vòng đời tài nguyên

Chương 22 đã thiết lập nền tảng của một bộ điều khiển (`controller`) sử dụng `client-go`, `SharedInformer` và `TypedRateLimitingInterface` để điều hòa các tài nguyên lõi như Pod. Tuy nhiên, khi hệ thống mở rộng, nhu cầu quản trị hạ tầng vượt khỏi phạm vi các tài nguyên cơ bản có sẵn của Kubernetes.

Một hệ thống phức tạp thường bao gồm nhiều thành phần phụ thuộc lẫn nhau: cụm cơ sở dữ liệu phân tán có lịch sao lưu định kỳ, dịch vụ nghiệp vụ cần cấu hình ingress kèm chứng chỉ bảo mật và tài khoản IAM. Phương thức quản lý thủ công qua nhiều tệp cấu hình YAML tĩnh dễ gây sai lệch cấu hình và phụ thuộc vào thao tác can thiệp trực tiếp của con người. Mô hình Operator giải quyết hạn chế này bằng cách đóng gói các quy trình vận hành miền (domain operational knowledge) vào chính mã nguồn Go của bộ điều khiển, tự động hóa vòng đời tài nguyên thông qua định nghĩa tài nguyên tùy biến (Custom Resource Definition - CRD).

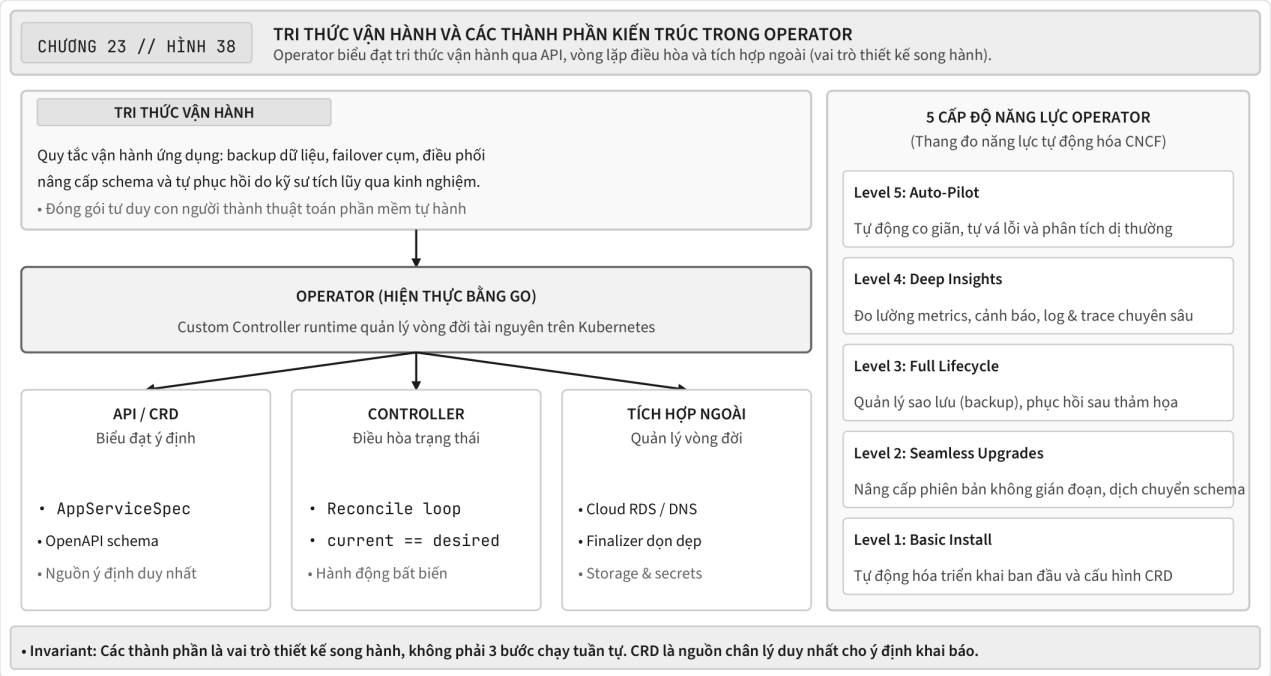
---

1. Operator là gì và Khi nào cần Operator?

Câu hỏi trung tâm của chương này là:

> **Sự khác biệt cốt lõi giữa một bộ điều khiển Kubernetes thông thường (`Controller`) và một bộ vận hành Kubernetes (`Operator`) là gì?**

Bộ vận hành (`Operator`) sử dụng bộ điều khiển để tự động hóa vòng đời của một ứng dụng hay nghiệp vụ chuyên biệt (`domain`), thường đi kèm một định nghĩa tài nguyên tùy biến (`CRD`). Bộ điều khiển nói chung cũng có thể làm việc với tài nguyên tùy biến; không có quy tắc nào bắt buộc nó chỉ được hiểu các tài nguyên có sẵn của hệ thống. Điểm khác biệt nằm ở tri thức vận hành mà logic điều hòa biểu đạt, chứ không phải một kiểu bộ điều khiển đặc biệt do Go hay Kubernetes nhận diện.



Hình 38 — Bộ vận hành (`Operator`) biểu đạt tri thức vận hành qua API, vòng lặp điều hòa và các tích hợp quản lý tài nguyên. Các thành phần này là vai trò thiết kế, không phải ba bước chạy tuần tự.

Thay vì bắt người dùng phải tự tạo `Deployment`, gắn `ConfigMap`, tạo `Secret` và mở `Service`, Operator cung cấp một tài nguyên duy nhất gọn gàng:

```
apiVersion: apps.example.com/v1alpha1
kind: AppService
metadata:
  name: payment-api
spec:
  image: "payment:v2.4.0"
  replicas: 3
  port: 8080
```

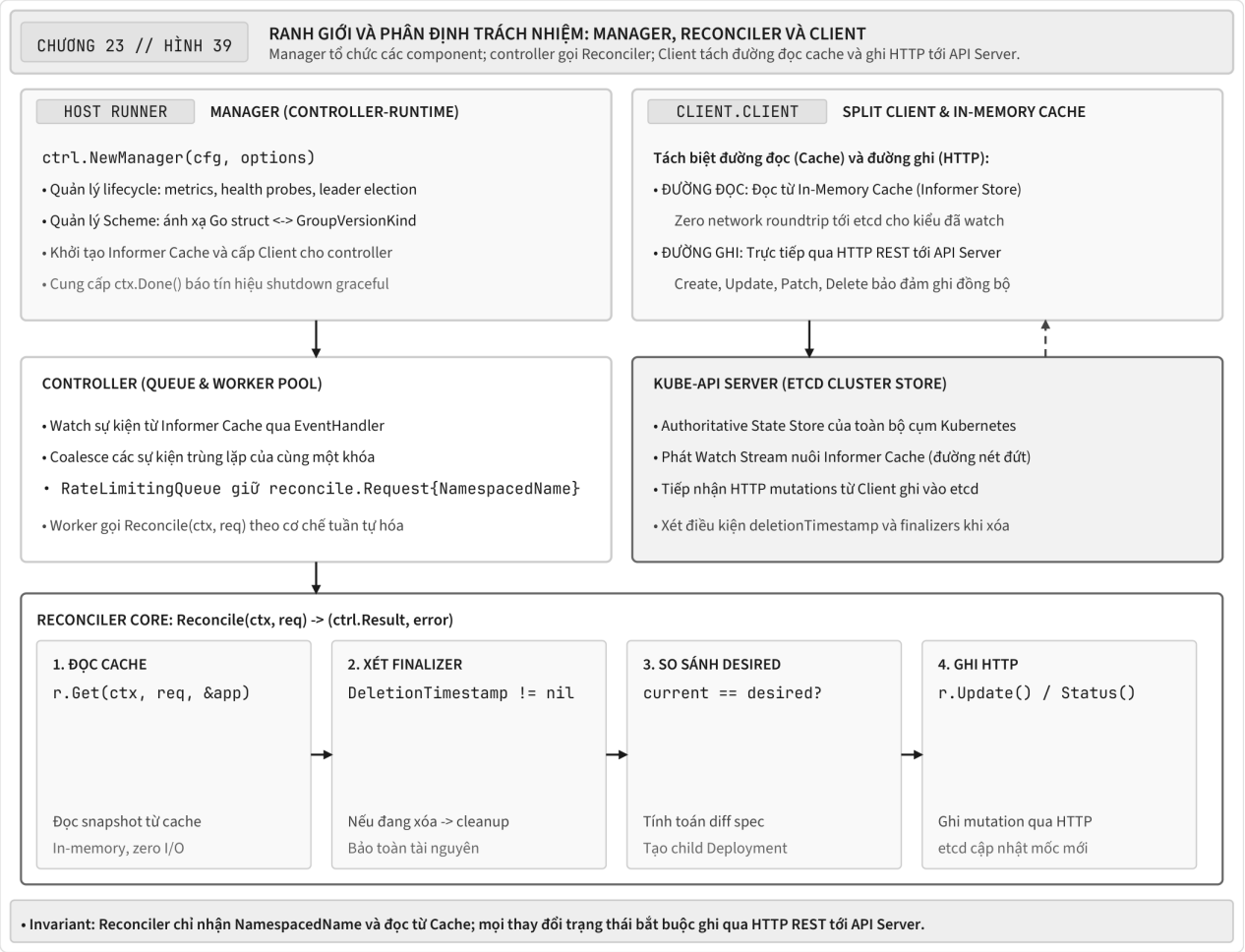
Khi ta nộp khai báo trên, bộ vận hành (`Operator`) sẽ tự động sinh ra đối tượng `Deployment` tương ứng, gắn nhãn, cấu hình health check, theo dõi sức khỏe và cập nhật trạng thái thực tế.

---

2. Kiến trúc của controller-runtime

Xây dựng bộ vận hành (`Operator`) bằng `client-go` thuần túy đòi hỏi phải tự kết nối các thành phần quản lý bộ đệm, hàng đợi và vòng đời tài nguyên. Thư viện `sigs.k8s.io/controller-runtime` cung cấp lớp kiến trúc tổ chức chung cho các thành phần này; mức độ tinh gọn mã nguồn thực tế phụ thuộc vào yêu cầu của từng operator.

Sơ đồ dưới đây tách trách nhiệm quản lý tiến trình, xử lý khóa và truy cập tài nguyên:



Hình 39 — Manager tổ chức các thành phần kiến trúc; bộ điều khiển gọi bộ điều hòa (`Reconciler`) cho một khóa tài nguyên. Client sử dụng bộ nhớ đệm cho các luồng đọc được cấu hình và giao thức HTTP cho luồng ghi trực tiếp; sơ đồ lược bỏ các luồng đọc không thông qua bộ đệm.

Bốn trụ cột của controller-runtime

Trụ cột `controller-runtime`	Vai trò kiến trúc	Tương tác trong hệ thống
`Manager`	Quản lý vòng đời tiến trình Operator	Quản lý bộ đệm chung, leader election, server metrics và probe.
`Scheme`	Đăng ký ánh xạ kiểu dữ liệu	Ánh xạ giữa Go struct (`v1alpha1.AppService`) và `GroupVersionKind`.
`Client` với cache	Phân tách đường đọc và ghi theo cấu hình	Get/List có thể đọc cache; kiểu bị DisableFor, metadata discovery hay cache chưa khởi tạo vẫn có đường HTTP. Writes đi tới API.
`Reconciler`	Hiện thực logic điều hòa cốt lõi	Nhận `ctrl.Request` chứa `NamespacedName`, hội tụ trạng thái.

---

3. Bản thiết kế CRD: Ranh giới rạch ròi giữa Spec và Status

Với API của bài thực hành, ta phân định rõ trường `Spec` (mô tả ý định người dùng) và trường `Status` (phản ánh quan sát thực tế của bộ điều khiển). Đây là hợp đồng của API; quyền ghi thực tế còn phụ thuộc vào RBAC và cấu hình subresource:

Khối dữ liệu	Ý nghĩa	Ai có quyền ghi?	Trách nhiệm
Spec	Trạng thái mong muốn (Desired State).	Người dùng, Helm, CI/CD pipeline, GitOps (ArgoCD).	Khai báo hệ thống cần đạt được điều gì.
Status	Trạng thái quan sát được.	Controller được policy giao trách nhiệm; RBAC quyết định quyền ghi thực tế.	Báo observation và generation đã quan sát.

Quy tắc chống trượt (Anti-Drift Rule)

> **Chính sách của bài thực hành:** Hàm điều hòa không sửa `Spec` để ghi nhận kết quả quan sát hoặc tự ý ghi đè ý định của người dùng. Mọi quan sát phải được ghi vào `Status`; một bộ điều khiển chỉ can thiệp vào `Spec` khi đó là khế ước vận hành đã được quy định tường minh.

Nếu người dùng cấu hình `replicas: 3`, mà Reconciler thấy tải cao rồi tự ý sửa `spec.replicas = 5`, hệ thống sẽ rơi vào cuộc chiến bất tận giữa Operator và công cụ GitOps (ArgoCD liên tục kéo về 3, Operator liên tục đẩy lên 5).

Mọi kết quả quan sát (số replica đang sẵn sàng, phiên bản thực tế đang chạy, điều kiện lỗi) bắt buộc phải ghi vào trường `Status`.

Phân lập Status Subresource

Để bảo vệ ranh giới này, Kubernetes cung cấp cơ chế tài nguyên phụ trạng thái (**Status Subresource** - `/status` endpoint). Khi cập nhật trạng thái, ta phải gọi:

```
// ĐÚNG: Chỉ sửa Status, không đổi metadata.generation
if err := r.Status().Update(ctx, appService); err != nil {
    return ctrl.Result{}, err
}
```

Với định nghĩa `CRD` kích hoạt tài nguyên phụ trạng thái (`status subresource`), thao tác ghi vào tài nguyên chính sẽ bỏ qua các thay đổi trong Status. Giá trị Generation chỉ tăng lên khi phần dữ liệu thuộc quy tắc thế hệ của spec thay đổi, không phải cứ gọi Update là tăng. Một thao tác ghi tạo ra sự kiện cũng chưa chứng minh vòng lặp vô tận: bộ lọc predicate, nội dung thay đổi và logic điều hòa đều có ảnh hưởng. Luôn đọc trạng thái qua đường dẫn quy chuẩn và xử lý xung đột ghi một cách cẩn trọng.

---

4. Quản lý tài nguyên con: OwnerReferences và Garbage Collection

Khi `AppService` sinh ra một `Deployment`, làm sao để Kubernetes biết hai đối tượng này có quan hệ cha - con?

Câu trả lời là **OwnerReference** (Tham chiếu chủ sở hữu). Thư viện `controller-runtime` cung cấp hàm chuẩn hóa:

```
if err := controllerutil.SetControllerReference(
    appService, newDeployment, r.Scheme,
); err != nil {
    return fmt.Errorf("không thể gán owner reference: %w", err)
}
```

OwnerReference giúp quản lý vòng đời thế nào

Thứ nhất là xóa theo tầng bậc (`cascading deletion`) theo chính sách: bộ thu gom rác sử dụng các tham chiếu sở hữu (`ownerReferences`) hợp lệ để xác định tài nguyên phụ thuộc, nhưng chính sách lan truyền (`propagation policy`), quy tắc cùng không gian tên hoặc phạm vi toàn cụm sẽ quyết định cách thức xóa cụ thể. Tránh suy diễn rằng tài nguyên con luôn bị xóa ngay lập tức. Về ranh giới không gian tên, OwnerReference là siêu dữ liệu sở hữu có quy tắc phạm vi chặt chẽ chứ không phải liên kết cha con tùy ý. Đối với tài nguyên phụ thuộc nằm trong không gian tên, nó có thể tham chiếu chủ sở hữu trong cùng không gian tên hoặc chủ sở hữu cấp toàn cụm; nếu trỏ tới chủ sở hữu ở không gian tên khác, tham chiếu sẽ không hợp lệ và bị coi như không tồn tại, khiến tài nguyên con có thể bị thu hồi khi mọi chủ sở hữu hợp lệ khác không còn. Ngược lại, tài nguyên phụ thuộc cấp toàn cụm chỉ được phép tham chiếu chủ sở hữu cấp toàn cụm; nếu trỏ tới chủ sở hữu trong một không gian tên cụ thể, tham chiếu sẽ không thể phân giải, và từ Kubernetes v1.20+ bộ thu gom rác sẽ ghi nhận cảnh báo `OwnerRefInvalidNamespace`.

Thứ hai là khả năng theo dõi sự kiện ngược dòng (`Watch Events`): Bộ điều khiển (`Controller`) có thể cấu hình `Watches(&appsv1.Deployment{}, handler.EnqueueRequestForOwner(...))`. Bất cứ khi nào ai đó sửa đổi hoặc xóa Deployment con, sự kiện sẽ tự động ánh xạ ngược về `AppService` cha để Reconciler thức dậy sửa chữa.

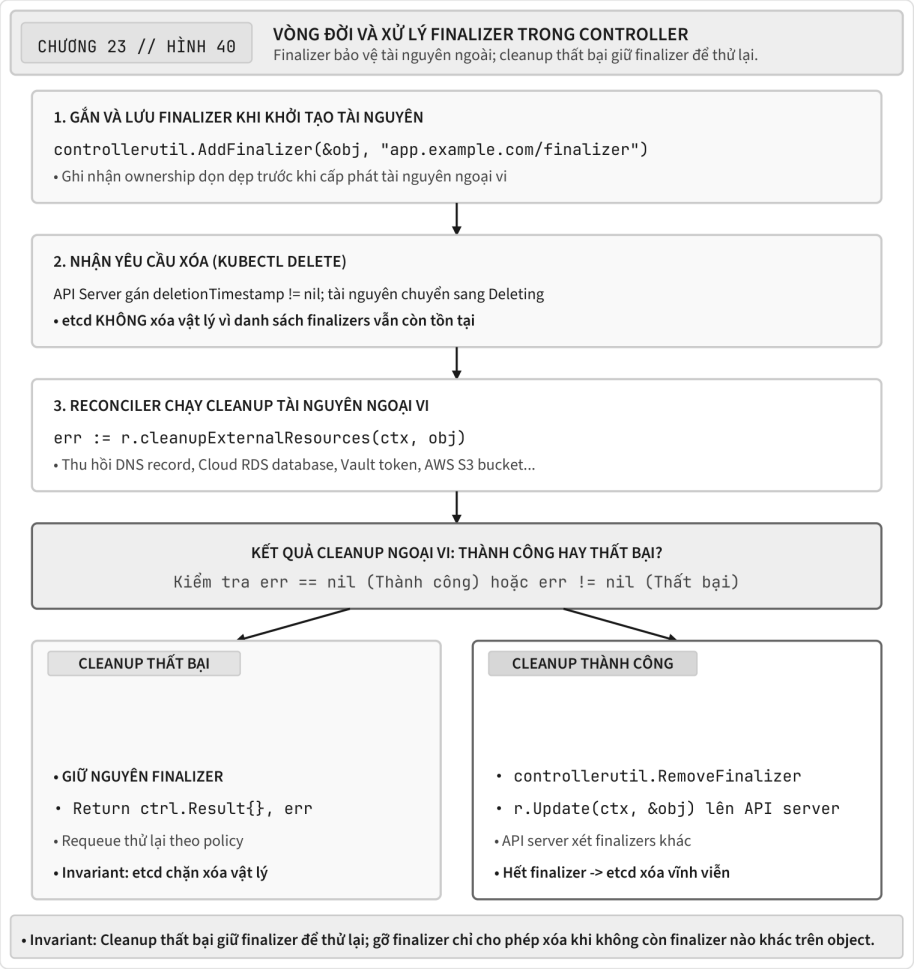
---

5. Finalizer: Cổng gác vòng đời và Dọn dẹp tài nguyên ngoại vi

Nếu `OwnerReference` giải quyết xuất sắc việc dọn dẹp các tài nguyên nội bộ trong Kubernetes, thì điều gì sẽ xảy ra nếu ứng dụng tạo ra các tài nguyên bên ngoài cụm như một cơ sở dữ liệu Amazon RDS, một bản ghi DNS trên Cloudflare, hay một hàng đợi tin nhắn AWS SQS?

Nếu đối tượng tài nguyên tùy biến (`AppService`) bị xóa trước khi quá trình dọn dẹp hoàn tất, bộ điều khiển có thể mất dữ liệu cần thiết để xác định tài nguyên ngoài cụm. Trong kịch bản này, cơ chế finalizer sẽ chặn việc hoàn tất xóa cho tới khi tiến trình dọn dẹp xác nhận đã hoàn thành an toàn; hệ thống vẫn cần cơ chế điều tra hoặc đối soát độc lập để xử lý các sự cố phát sinh ngoài luồng chạy bình thường.

Đây chính là sứ mệnh của cơ chế chốt vòng đời (**Finalizer**).



Hình 40 — Việc dọn dẹp thất bại sẽ giữ lại finalizer để có thể thử lại. Dọn dẹp thành công chỉ cho phép gỡ bỏ finalizer của bộ điều khiển này; máy chủ API vẫn cần xem xét toàn bộ các điều kiện xóa khác.

Máy chủ API chỉ hoàn tất việc xóa khi mọi điều kiện trong vòng đời tài nguyên, bao gồm toàn bộ finalizer còn lại, đều đã thỏa mãn. Gỡ bỏ finalizer của bộ điều khiển này không đồng nghĩa với việc đối tượng sẽ biến mất ngay lập tức; bộ điều khiển không trực tiếp can thiệp vào etcd để xóa dữ liệu.

Cạm bẫy Finalizer Deadlock

Tác vụ dọn dẹp luôn phải an toàn khi gọi lại nhiều lần. Một phản hồi báo không tìm thấy (404 Not Found) có thể được coi là đã hoàn tất nếu API và định danh chứng minh chính xác rằng tài nguyên đích thực sự không còn tồn tại; không tùy tiện bỏ qua mọi lỗi 404, vì nó cũng có thể xuất phát từ việc sai điểm cuối (`endpoint`), sai thẩm quyền hoặc sai định danh.

Nếu coi mọi mã lỗi 404 khi dọn dẹp đều là sự cố cần thử lại, bộ điều khiển có thể giữ mãi chốt `finalizer` dù tài nguyên bên ngoài đã biến mất. Đối tượng có thể bị kẹt ở trạng thái `Terminating` cho tới khi logic được sửa đổi hoặc có sự can thiệp thủ công. Chỉ coi việc không tìm thấy tài nguyên là dọn dẹp thành công khi đã xác nhận đúng định danh và khế ước API, không gộp mọi lỗi bên ngoài vào cùng một nhánh xử lý thô thiển.

---

6. Hiện thực Operator hoàn chỉnh trong Go

Dưới đây là phần lõi của bộ điều hòa (`Reconciler`) trong `labs/part23-controller-runtime-operator/`:

Định nghĩa cấu trúc điều hòa với khả năng tiêm phụ thuộc (Dependency Injection) cho bộ dọn dẹp ngoại vi:

```
const AppServiceFinalizer = "apps.example.com/finalizer"

type ExternalCleaner interface {
    Cleanup(
        ctx context.Context, app *appsv1alpha1.AppService,
    ) error
}

type AppServiceReconciler struct {
    client.Client
    Scheme          *runtime.Scheme
    ExternalCleaner ExternalCleaner
}
```

Vòng lặp điều hòa trung tâm

Hàm `Reconcile` điều phối toàn bộ vòng đời: từ kiểm tra xóa, tạo tài nguyên con, đến sửa chữa sai lệch cấu hình:

```
func (r *AppServiceReconciler) Reconcile(
    ctx context.Context, req ctrl.Request,
) (ctrl.Result, error) {
    appService := &appsv1alpha1.AppService{}
    err := r.Get(ctx, req.NamespacedName, appService)
    if err != nil {
        if errors.IsNotFound(err) {
            return ctrl.Result{}, nil
        }
        return ctrl.Result{}, err
    }

    // 1. Xử lý Finalizer: Kiểm soát vòng đời xóa
    if !appService.DeletionTimestamp.IsZero() {
        return r.handleDeletion(ctx, appService)
    }

    // Đảm bảo đối tượng luôn có Finalizer bảo vệ
    if !controllerutil.ContainsFinalizer(
```

```
        appService, AppServiceFinalizer,
    ) {
        controllerutil.AddFinalizer(
            appService, AppServiceFinalizer,
        )
        return ctrl.Result{}, r.Update(ctx, appService)
    }

    // 2. Điều hòa Deployment con và sửa sai lệch (Drift)
    if err := r.reconcileDeployment(
        ctx, appService,
    ); err != nil {
        return ctrl.Result{}, err
    }

    // 3. Cập nhật Status Subresource
    return ctrl.Result{}, r.updateStatus(ctx, appService)
}
```

Ngữ nghĩa trả về của Result và Error trong controller-runtime

Trong `controller-runtime` v0.25.1, giá trị trả về của `Reconcile(ctx, req)` quyết định hành vi tiếp theo của hàng đợi theo bốn nhánh rành mạch:

Nhánh thứ nhất, trả về `(ctrl.Result{}, err)` với `err != nil`: Nếu lỗi không phải `reconcile.TerminalError`, bộ điều khiển ghi nhật ký lỗi và đưa lại yêu cầu vào hàng đợi với thuật toán kiểm soát tốc độ thử lại (`AddRateLimited`). Nếu vô tình trả về đồng thời `err != nil` và cờ `Requeue` hoặc `RequeueAfter`, bộ điều khiển sẽ ghi nhật ký cảnh báo và bỏ qua hai cờ thử lại để ưu tiên xử lý lỗi.

Nhánh thứ hai, trả về `(ctrl.Result{RequeueAfter: d}, nil)` với `d > 0`: Bộ điều khiển xóa lịch sử lỗi của yêu cầu (`Forget`) và lên lịch đưa yêu cầu trở lại hàng đợi sau khoảng thời gian `d` (`AddAfter`).

Nhánh thứ ba, trả về `(ctrl.Result{Requeue: true}, nil)`: Bộ điều khiển đưa yêu cầu trở lại hàng đợi có kiểm soát tốc độ qua bộ giới hạn tốc độ (rate limiter). Trong các phiên bản controller-runtime mới, cách trả về này dần được thay thế bằng lỗi hoặc `RequeueAfter` tường minh.

Nhánh thứ tư, trả về `(ctrl.Result{}, nil)`: lượt này không yêu cầu xếp lại và lịch sử lỗi được xóa (`Forget`). Key vẫn có thể đã được đánh dirty trong lúc xử lý, có lịch chờ từ trước hoặc được nguồn sự kiện khác đưa vào queue; không suy rằng chỉ watch mới mới đánh thức worker.

Xử lý Xóa an toàn và Dọn dẹp ngoại vi

```
func (r *AppServiceReconciler) handleDeletion(
    ctx context.Context, app *appsv1alpha1.AppService,
) (ctrl.Result, error) {
    hasFin := controllerutil.ContainsFinalizer(
        app, AppServiceFinalizer,
    )
    if hasFin {
        if r.ExternalCleaner == nil {
            return ctrl.Result{}, fmt.Errorf(
                "external cleaner is required",
            )
        }
        err := r.ExternalCleaner.Cleanup(ctx, app)
        if err != nil {
```

```
            return ctrl.Result{}, fmt.Errorf(
                "cleanup lỗi: %w", err,
            )
        }
        // Dọn dẹp xong: gỡ finalizer của controller này
        controllerutil.RemoveFinalizer(
            app, AppServiceFinalizer,
        )
        if err := r.Update(ctx, app); err != nil {
            return ctrl.Result{}, err
        }
    }
    return ctrl.Result{}, nil
}
```

Tự động nhận diện và sửa chữa sai lệch (Drift Correction)

Nếu ai đó dùng `kubectl scale` để sửa số bản sao của đối tượng `Deployment` con từ 3 xuống 1, hàm điều hòa phát hiện sự không đồng nhất giữa `appService.Spec.Replicas` và `deployment.Spec.Replicas`, từ đó tự động ép trạng thái về đúng bản thiết kế ban đầu:

```
func (r *AppServiceReconciler) syncDeploymentDrift(
    app *appsv1alpha1.AppService, dep *appsv1.Deployment,
) bool {
    drifted := false
    if dep.Spec.Replicas == nil ||
        *dep.Spec.Replicas != app.Spec.Replicas {
        dep.Spec.Replicas = &app.Spec.Replicas
        drifted = true
    }
    if len(dep.Spec.Template.Spec.Containers) > 0 {
        c := &dep.Spec.Template.Spec.Containers[0]
        if c.Image != app.Spec.Image {
            c.Image = app.Spec.Image
            drifted = true
        }
    }
    return drifted
}
```

---

7. Bằng chứng kiểm thử: Chứng minh 5 quy luật Operator

Bộ kiểm thử tại `labs/part23-controller-runtime-operator/controllers/appservice_controller_test.go` sử dụng `fake.NewClientBuilder()` để xác nhận khế ước logic của hàm điều hòa. Công cụ này không mô phỏng toàn diện máy chủ API: việc kiểm tra tính hợp lệ dữ liệu, thế hệ tài nguyên và ngữ nghĩa của tài nguyên phụ không hoàn toàn giống cụm máy chủ thật. Khi cần xác minh tương tác trực tiếp với máy chủ API, ta sử dụng `envtest` hoặc bài kiểm tra tích hợp cụm riêng biệt. Vì vậy bài thực hành ghi nhận `ObservedGeneration` vào trường status để bên tiêu thụ biết trạng thái đã phản ánh đúng bản spec nào, chứ không dùng fake client để tuyên bố đã kiểm chứng trọn vẹn vòng đời của generation.

```
=== RUN   TestReconcileCreateOwnedDeployment
--- PASS: TestReconcileCreateOwnedDeployment (0.51s)
=== RUN   TestReconcileDriftCorrection
--- PASS: TestReconcileDriftCorrection (0.01s)
=== RUN   TestReconcileFinalizerExecution
--- PASS: TestReconcileFinalizerExecution (0.01s)
=== RUN   TestReconcileStatusSubresource
--- PASS: TestReconcileStatusSubresource (0.01s)
=== RUN   TestNotFoundResourceNoOp
--- PASS: TestNotFoundResourceNoOp (0.01s)
PASS
ok      part23-controller-runtime-operator/controllers   1.875s
```

1. Khởi tạo tài nguyên con kèm OwnerReference (TestReconcileCreateOwnedDeployment)

Khi tạo mới một `AppService`, bộ điều hòa tự động sinh một đối tượng `Deployment` mang tên tương ứng. Bằng chứng kiểm thử xác nhận `dep.OwnerReferences[0].Name == "cache-service"`, và `app.Finalizers` lập tức được bổ sung khóa `apps.example.com/finalizer`.

2. Tự phục hồi sai lệch cấu hình (TestReconcileDriftCorrection)

Lab sửa desired deployment rồi gọi Reconcile để kiểm tra logic khôi phục những field controller sở hữu. Test không đo tốc độ hội tụ của cluster thật; cache, event delivery, retry và API error vẫn ảnh hưởng thời gian sửa drift.

3. Vòng đời Finalizer (TestReconcileFinalizerExecution)

Gán `DeletionTimestamp` vào đối tượng. Bộ điều hòa gọi `ExternalCleaner.Cleanup()` rồi mới gỡ `finalizer`; thiếu bộ dọn dẹp là lỗi và `finalizer` phải giữ lại. Test chỉ chứng minh contract của lab; việc object thực sự biến mất là trách nhiệm API server/garbage collection và không được suy ra từ fake client.

4. Cập nhật Status Subresource độc lập (TestReconcileStatusSubresource)

Test kiểm chính sách hẹp của lab: `AvailableReplicas: 3` với ba replica mong muốn dẫn tới `Phase = "Ready"`, cập nhật qua `r.Status().Update()` mà không đổi `Spec`. Chính sách này chưa kiểm `Deployment.Status.ObservedGeneration`, `UpdatedReplicas` hoặc rollout mới đã hoàn tất; nhãn Ready không đủ chứng minh ảnh mới đang phục vụ. `AppService.Status.ObservedGeneration` chỉ ghi thế hệ AppService được lượt này đọc. Test mới từ chối Deployment cùng tên nhưng không thuộc quyền điều khiển; hai fixture drift/status cũ được gắn owner reference để đúng tình huống child sở hữu, không coi name là ownership.

5. An toàn khi tài nguyên biến mất (TestNotFoundResourceNoOp)

Gửi yêu cầu điều hòa cho một tài nguyên không hề tồn tại. Hàm điều hòa trả về `(ctrl.Result{}, nil)` một cách nhẹ nhàng, không gây `panic` hay sinh lỗi rác trong log hệ thống.

Đặt cùng assertion trước API server thật

Máy khách giả lập (`fake client`) ở trên cho phép kiểm tra hàm điều hòa đã gọi đúng luồng ghi và xử lý chính xác các đối tượng mẫu trong bộ kiểm thử. Nó không phải là một máy chủ API thu nhỏ. Bài kiểm tra `TestFakeDoesNotProveGeneration` trong thư mục `integration` thay đổi `spec` nhưng thế hệ tài nguyên (`generation`) vẫn giữ mức 1 trên bản giả lập của `controller-runtime` v0.25.1. Đừng tự ý nâng con số ấy lên 2 bằng hàm trợ giúp kiểm thử rồi ngộ nhận đó là bằng chứng Kubernetes tự động tăng thế hệ tài nguyên (`generation`).

Thử nghiệm một khế ước kỹ thuật cụ thể: tài nguyên phụ trạng thái (`status`) phải tách biệt rạch ròi luồng ghi mong muốn khỏi luồng ghi quan sát thực tế. Bài kiểm tra `TestRealAPIContract` nạp CRD có lược đồ cấu trúc và `subresources.status` vào một mặt phẳng điều khiển `envtest` độc lập. Nó không đọc tệp cấu hình kubeconfig cá nhân và không can thiệp vào cụm máy chủ từ Chương 17. Sau khi tạo, trạng thái tự khai bị loại bỏ; lệnh cập nhật gốc `Update` không thể ghi đè trường `status`; phương thức `Status().Update` không thể sửa đổi trường `spec`. Việc cập nhật `spec` sẽ

nâng thế hệ tài nguyên (`generation`) từ 1 lên 2, trong khi `observedGeneration` vẫn giữ mức 1 cho tới khi bộ điều khiển báo cáo quan sát mới. Cuối cùng, bên ghi trạng thái dùng phiên bản tài nguyên cũ phải nhận mã lỗi xung đột thay vì ghi đè lên quan sát mới.

```
# Từ module labs/part23-controller-runtime-operator
export KUBEBUILDER_ASSETS=/path/to/k8s-1.37-assets
go test -tags integration -count=1 -v ./integration
```

Đường thực thi này cần Linux/WSL và các binary `kube-apiserver`, `etcd` tương thích Kubernetes 1.37.x; dependency của lab đã ghim API v0.37.0. Môi trường Windows hiện tại chưa có assets đó, nên kiểm định API thật là `NOT_RUN`, không phải PASS. Test fake chạy được và việc biên dịch test cho Linux chỉ kiểm tra mã dùng đúng API thư viện. Khi tái lập trên Linux đủ prerequisite, hãy đọc output của chính test, không dùng kết quả compile làm chứng nhận integration.

> **Dừng để dự đoán:** Nếu thay thế `Status().Update` bằng `Update`, bài kiểm tra nào sẽ bắt được lỗi dù yêu cầu không trả về mã lỗi? Chạy lại với cờ `RELIABILITY_MUTANT=root_status_write` trên môi trường có đủ tài nguyên nhị phân; câu khẳng định của luồng trạng thái sẽ chuyển sang báo lỗi. Đây là biến thể đột biến kiểm thử được chuẩn bị cho môi trường tích hợp.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Điểm cuối gốc có thể chấp nhận yêu cầu nhưng lược bỏ trạng thái, đồng thời chấp nhận phần spec mà mã nguồn vô tình đính kèm. Bài kiểm tra đọc lại đối tượng bằng client không thông qua bộ đệm, đối soát toàn diện cả giai đoạn, ảnh container và thế hệ tài nguyên. Nếu chỉ kiểm tra `err == nil`, ta sẽ bỏ sót hoàn toàn việc ghi nhầm điểm cuối. Khi gặp xung đột thực tế, hàm điều hòa phải đọc lại quan sát mới rồi tính toán lại các thay đổi; việc lặp lại thao tác ghi trên cùng đối tượng cũ không phải là giải pháp xử lý xung đột.

Môi trường `envtest` chỉ kích hoạt máy chủ API và etcd, không chạy kubelet, bộ điều phối scheduler hay bộ thu gom rác. Do đó, bài kiểm tra này

không chứng minh Pod đã sẵn sàng, tham chiếu sở hữu đã thu hồi tài nguyên con hay chốt chặn finalizer đã dọn dẹp hạ tầng đám mây. Việc bóc tách khế ước API khỏi vòng lặp điều hòa và khối lượng công việc thực tế giúp ta nhận diện chính xác các khoảng trống bằng chứng kỹ thuật.

8. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Ghi observation vào Spec dù API quy định user sở hữu Spec.	Có thể xung đột với GitOps và gây reconcile lặp.	Giữ observation ở Status; xác định field ownership khi có chức năng sửa Spec.
Dùng `Update` cho Status khi CRD bật status subresource.	Status có thể bị bỏ qua; generation không phải cứ Update là tăng.	Dùng `Status().Update` cho status và kiểm tra conflict/observedGeneration theo API.
Cleanup coi mọi not-found là lỗi hoặc mọi 404 là thành công.	Có thể giữ finalizer không cần thiết hoặc bỏ cleanup sai đích.	Xác minh resource identity và contract API trước khi coi not-found là hoàn tất.
Không thiết kế ownership cho tài nguyên con.	Garbage collector không suy ra quan hệ cha/con từ tên; tài nguyên có thể còn lại khi cha bị xóa.	Nếu policy giao cleanup cho garbage collector, gắn owner reference hợp lệ; kiểm tra scope và deletion policy, không gắn owner tùy tiện cho tài nguyên chia sẻ.

---

9. Bài tập thực hành thiết kế Operator

Thử thách 1: Tự động khởi tạo và đồng bộ ConfigMap đi kèm Deployment

Yêu cầu kỹ thuật: Mở rộng `AppServiceReconciler` để mỗi khi điều hòa một `AppService`, nó tự động tạo và đồng bộ một `ConfigMap` con (`<app-name>-config`) chứa file cấu hình ứng dụng `app.json` với nội dung `{"port": <Spec.Port>}`. `ConfigMap` này phải được gắn `OwnerReference` trỏ về `AppService`. Nếu `ConfigMap` đã tồn tại nhưng dữ liệu bị lệch (drift), reconciler phải cập nhật lại nội dung.

Thử thách 2: Báo cáo Condition chuẩn mực trong Status

Yêu cầu kỹ thuật: Kubernetes khuyến nghị sử dụng slice `[]metav1.Condition` để thể hiện trạng thái chi tiết của tài nguyên. Hãy bổ sung hàm trợ giúp để cập nhật Condition `Type="DeploymentReady"` với `Status="True"` khi số lượng Pod sẵn sàng bằng số lượng replica mong muốn, hoặc `Status="False"` kèm Reason `"ReplicasUnavailable"` khi chưa đủ.

---

10. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Đồng bộ ConfigMap con và Xử lý Trôi cấu hình

```
func (r *AppServiceReconciler) reconcileConfigMap(
    ctx context.Context, app *appsv1alpha1.AppService,
) error {
    desiredJSON := fmt.Sprintf(`{"port": %d}`, app.Spec.Port)
    cm := &corev1.ConfigMap{
        ObjectMeta: metav1.ObjectMeta{
            Name:      app.Name + "-config",
            Namespace: app.Namespace,
        },
        Data: map[string]string{
            "app.json": desiredJSON,
        },
    }

    // Gán quan hệ cha con để garbage collector tự động dọn dẹp
    if err := controllerutil.SetControllerReference(
        app, cm, r.Scheme,
    ); err != nil {
        return err
    }

    found := &corev1.ConfigMap{}
    key := client.ObjectKeyFromObject(cm)
```

```
    err := r.Get(ctx, key, found)
    if errors.IsNotFound(err) {
        return r.Create(ctx, cm)
    }
    if err != nil {
        return err
    }

    // Kiểm tra quyền sở hữu, từ chối ghi đè nếu owner khác
    if !metav1.IsControlledBy(found, app) {
        return fmt.Errorf(
            "configmap %s exists but not owned by %s",
            found.Name, app.Name,
        )
    }

    // Phát hiện và sửa trôi cấu hình (configuration drift)
    if found.Data["app.json"] != cm.Data["app.json"] {
        found.Data = cm.Data
        return r.Update(ctx, found)
    }
    return nil
}
```

Phân tích ranh giới điều hòa (Reconcile Boundaries)

Trong một reconciler chuẩn mực, việc kiểm soát và bảo vệ ranh giới tài nguyên gồm ba kỷ luật cốt lõi:

Thứ nhất là kiểm tra quyền sở hữu (ownership verification). Khi `r.Get` tìm thấy `ConfigMap` đã tồn tại trên cụm, bộ điều hòa không được vội vàng cập nhật dữ liệu. Tài nguyên này có thể do một tiến trình khác tạo ra hoặc do người quản trị cấu hình thủ công. Hàm sử dụng `metav1.IsControlledBy(found, app)` để xác minh quyền điều khiển. Nếu `ConfigMap` không thuộc sở hữu của `AppService` hiện tại, chính sách an toàn nhất là từ chối can thiệp và trả về lỗi tường minh, tuyệt đối không ghi đè lên tài nguyên của chủ sở hữu khác.

Thứ hai là khắc phục hiện tượng trôi dạt cấu hình (drift correction). Với đối tượng `ConfigMap` đã xác thực đúng quyền sở hữu, việc chỉ kiểm tra trạng thái không tìm thấy khi khởi tạo ban đầu là chưa đủ. Nếu ai đó chỉnh sửa

`ConfigMap` trực tiếp hoặc trường `Spec.Port` của `AppService` được cập nhật sau đó, luồng xử lý chỉ tạo mới sẽ bỏ sót sai lệch dữ liệu. Do đó, hàm điều hòa đối chiếu nội dung thực tế qua phép so sánh `found.Data["app.json"] != cm.Data["app.json"]`, rồi gọi `Update(ctx, found)` khi có khác biệt để bảo đảm tính nhất quán sau cùng (eventual consistency).

Thứ ba là tác động tới ứng dụng trong Pod. Cập nhật `ConfigMap` không tự restart Pod. Với volume thông thường, dữ liệu mới được chiếu xuống sau một khoảng trễ; ứng dụng có thể theo dõi và nạp lại cấu hình mà không restart. Biến môi trường `envFrom` và mount qua `subPath` không tự nhận dữ liệu mới theo cách đó. Nếu chọn policy rollout, Operator có thể gắn mã băm cấu hình vào annotation của Pod template rồi cập nhật Deployment; thay đổi template kích hoạt triển khai theo strategy đã cấu hình. Đây là một lựa chọn, không phải cơ chế duy nhất, và việc gửi update thành công chưa chứng minh rollout hoàn tất.

Lời giải Thử thách 2: Quản lý Condition bằng meta.SetStatusCondition

Sử dụng thư viện chuẩn `k8s.io/apimachinery/pkg/api/meta`:

```
import "k8s.io/apimachinery/pkg/api/meta"

func (r *AppServiceReconciler) updateConditions(
    app *appsv1alpha1.AppService, isReady bool,
) {
    condition := metav1.Condition{
        Type:               "DeploymentReady",
        Status:             metav1.ConditionFalse,
        Reason:             "ReplicasUnavailable",
        Message:            "Chưa đủ bản sao sẵn sàng",
        LastTransitionTime: metav1.Now(),
    }

    if isReady {
        condition.Status = metav1.ConditionTrue
        condition.Reason = "DeploymentAvailable"
        condition.Message = "Tất cả bản sao đều khỏe mạnh"
    }

    meta.SetStatusCondition(
        &app.Status.Conditions, condition,
    )
}
```

Hàm `meta.SetStatusCondition` tự động kiểm tra: nếu điều kiện đã tồn tại và không thay đổi trạng thái, nó sẽ giữ nguyên trường `LastTransitionTime` cũ, giúp hệ thống theo dõi chính xác thời điểm chuyển giao trạng thái.

Tài nguyên tùy biến, thư viện `controller-runtime` và cơ chế `finalizer` cung cấp cho ta các khối kiến trúc cốt lõi để xây dựng một operator hoàn chỉnh. Bài thực hành không nhằm chứng minh ta đã giải quyết mọi trường hợp phức tạp trên môi trường sản xuất; khối lượng công việc thực tế còn đòi hỏi tiến hóa lược đồ dữ liệu, quyền sở hữu, chính sách thử lại, phân quyền RBAC và cơ chế phục hồi riêng. Chương 24 sẽ chuyển các ranh giới này sang AWS SDK for Go v2 và thông tin xác thực tạm thời.

Chương 24 — Tự động hóa AWS bằng Go mà không biến credential thành bí mật dài hạn

Trong hành trình xây dựng các công cụ vận hành và nền tảng hạ tầng, giao tiếp với các dịch vụ điện toán đám mây là nhiệm vụ thiết yếu của kỹ sư DevOps/SRE. Cho dù kỹ sư viết công cụ dọn dẹp snapshot định kỳ, sao lưu cơ sở dữ liệu lên Amazon S3, hay điều phối máy chủ qua EC2, ta đều phải trả lời một câu hỏi bảo mật cốt lõi:

> **Làm thế nào để chương trình Go giao tiếp an toàn với AWS API mà không bao giờ nhúng Access Key dài hạn vào mã nguồn, file cấu hình hay biến môi trường?**

Để lộ cặp khóa `AWS_ACCESS_KEY_ID` và `AWS_SECRET_ACCESS_KEY` dài hạn trong repo hoặc log có thể cho phép sử dụng quyền của khóa cho đến khi bị thu hồi. Vì vậy chương này ưu tiên credential tạm thời và quyền tối thiểu; không gán thứ hạng nguyên nhân sự cố khi không có bộ dữ liệu tương ứng.

Chương 21 đến Chương 23 đã xây dựng mô hình điều hòa: quan sát trạng thái, tính sai lệch, hành động và quan sát lại để đảm bảo tính lũy đẳng trong Kubernetes. Khi mở rộng sang đám mây, các nguyên lý ấy vẫn giữ nguyên giá trị: việc tự động hóa AWS API (như dọn dẹp snapshot hay lưu trữ backup) cũng là một dạng điều hòa với hệ thống ngoại vi, nơi tính lũy đẳng nghiệp vụ, ranh giới lỗi và quyền hạn tạm thời quyết định độ tin cậy của hệ thống. Chương này trang bị cho kỹ sư tư duy thiết kế hệ thống tự động hóa đám mây dựa trên thư viện chính thức AWS SDK for Go v2: từ cơ chế cấp quyền động ngắn hạn (Temporary Credentials), xác định Region, ngăn xếp middleware Smithy, ký chữ ký số SigV4, đến duyệt dữ liệu lớn qua Paginator và phân loại lỗi chuẩn mực.

---

1. Chuỗi định danh ngầm định và Quyền tạm thời

Ưu tiên trên môi trường vận hành thực tế là sử dụng thông tin xác thực tạm thời từ vai trò IAM hoặc cơ chế OIDC; điều đó không biến mọi nguồn trong chuỗi phân giải mặc định thành thông tin xác thực tạm thời. Với `config.LoadDefaultConfig`, các nguồn nền tảng gồm thông tin xác thực tĩnh từ biến môi trường, tệp cấu hình chia sẻ, thông tin xác thực container (ECS) và vai trò IAM qua dịch vụ EC2 IMDS; cấu hình SSO hay danh tính web sẽ chọn nhà cung cấp tương ứng. Chuỗi phân giải vẫn có thể tiếp nhận thông tin xác thực tĩnh từ môi trường hoặc tệp chia sẻ, đặc biệt trên máy phát triển và trong các bài kiểm thử.

Các khóa tạm thời này có thời hạn hiệu lực hữu hạn: thời lượng phiên của IAM Role có thể cấu hình linh hoạt từ 15 phút tới 12 giờ tùy theo cấu hình vai trò, trong khi các cơ chế phiên liên kết (chained roles) hay `AssumeRoleWithWebIdentity` áp dụng giới hạn riêng. Mỗi bộ thông tin xác thực tạm thời luôn gắn liền với một Session Token (`AWS_SESSION_TOKEN`). Cần đặc biệt lưu ý: nếu khóa tạm thời bị rò rỉ trong thời gian còn hiệu lực, kẻ tấn công vẫn có thể sử dụng hợp lệ cho đến khi hết hạn hoặc cho đến khi quản trị viên chủ động thu hồi phiên (thông qua IAM revocation policy, cập nhật inline policy, hoặc vô hiệu hóa IAM role).



Hình 41 — Cấu hình quyết định provider được chọn; đây là bản đồ nguồn, không phải thứ tự ưu tiên bất biến. Credential có thể là static hoặc tạm thời. Cache lấy credential khi có lượt Retrieve cần dữ liệu hợp lệ.

Cơ chế bộ đệm và xác thực đồng bộ của SDK (`aws.CredentialsCache`)

Trong mã nguồn SDK được ghim, quy trình phân giải thông tin xác thực rẽ nhánh linh hoạt theo cấu hình chứ không phải một danh sách ưu tiên bất biến: hồ sơ cấu hình (profile) được chọn tường minh qua biến môi trường hoặc tệp cấu hình, thông tin xác thực từ môi trường và Web Identity đều có điều kiện kích hoạt riêng; hồ sơ cấu hình có thể sử dụng hồ sơ nguồn, SSO hay quy trình xác thực trước khi chuyển tiếp tới container hoặc IMDS. Hãy tham khảo `config/resolve_credentials.go` cùng các tùy chọn của bên gọi khi cần giải thích nguyên nhân một nhà cung cấp được lựa chọn. Tránh suy diễn rằng thông tin xác thực luôn là loại tạm thời chỉ thuần túy dựa vào việc triệu gọi `LoadDefaultConfig`.

Đối với các khối lượng công việc container hóa và điều phối đám mây, SDK phân định rõ ba cơ chế cấp phát định danh độc lập thay vì gộp chung mọi môi trường:

Một là, cơ chế gắn vai trò IAM cho tài khoản dịch vụ Kubernetes (IRSA): Khối Pod được ánh xạ mã thông báo danh tính web OIDC (thông qua biến môi trường `AWS_WEB_IDENTITY_TOKEN_FILE` và `AWS_ROLE_ARN`), kích hoạt nhà cung cấp định danh gọi tới dịch vụ AWS STS thông qua giao diện lập trình `AssumeRoleWithWebIdentity`.

Hai là, cơ chế vai trò cho tác vụ ECS: Nhà cung cấp thông tin xác thực vùng chứa kết nối trực tiếp tới điểm cuối xác thực của tiến trình ECS Agent cục bộ thông qua biến môi trường `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI`.

Ba là, cơ chế định danh Pod trên EKS: Nhà cung cấp thông tin xác thực vùng chứa tương tác trực tiếp với tiến trình hỗ trợ danh tính trên nút máy chủ thông qua biến môi trường `AWS_CONTAINER_CREDENTIALS_FULL_URI` và mã thông báo xác thực tại `AWS_CONTAINER_AUTHORIZATION_TOKEN_FILE`.

SDK bao bọc provider trong `aws.CredentialsCache` để tránh lấy khóa lặp lại trước mỗi request. Khi có lượt `Retrieve` cần dữ liệu và khóa chưa có hoặc đã hết hạn, cache mới gọi provider; nó không có vòng quét định kỳ tự làm mới khi không có người gọi. `ExpiryWindow > 0` điều chỉnh mốc hết hạn lưu trong cache để lần gọi sau làm mới sớm; window không dương được bỏ qua. Không hiểu "theo nhu cầu" thành "không tạo goroutine": source SDK v1.36.3 dùng singleflight để các caller cùng chờ một lượt lấy khóa, chạy lượt đó trong goroutine ngắn hạn. Context bị hủy có thể kết thúc một caller mà không hủy lượt lấy khóa dùng chung. Test có điểm bắt tay kiểm caller đã trả lỗi hủy, provider vẫn chạy rồi hoàn tất cho caller khác; đây là hành vi của phiên bản được ghim, không phải bảo đảm không còn việc nền khi caller trả về.

Xác định Region và Ranh giới Dịch vụ Vùng

Khi nạp qua `config.LoadDefaultConfig`, Region có thể đến từ `config.WithRegion("ap-southeast-1")`, `AWS_REGION`/`AWS_DEFAULT_REGION` hoặc profile trong `~/.aws/config`; giá trị tường minh có ưu tiên cao hơn. SDK không có Region mặc định. Lấy Region từ EC2 IMDS cần bật tùy chọn `config.WithEC2IMDSRegion()`; không nhầm việc credential provider tự dùng IMDS với việc Region tự được suy ra từ IMDS.

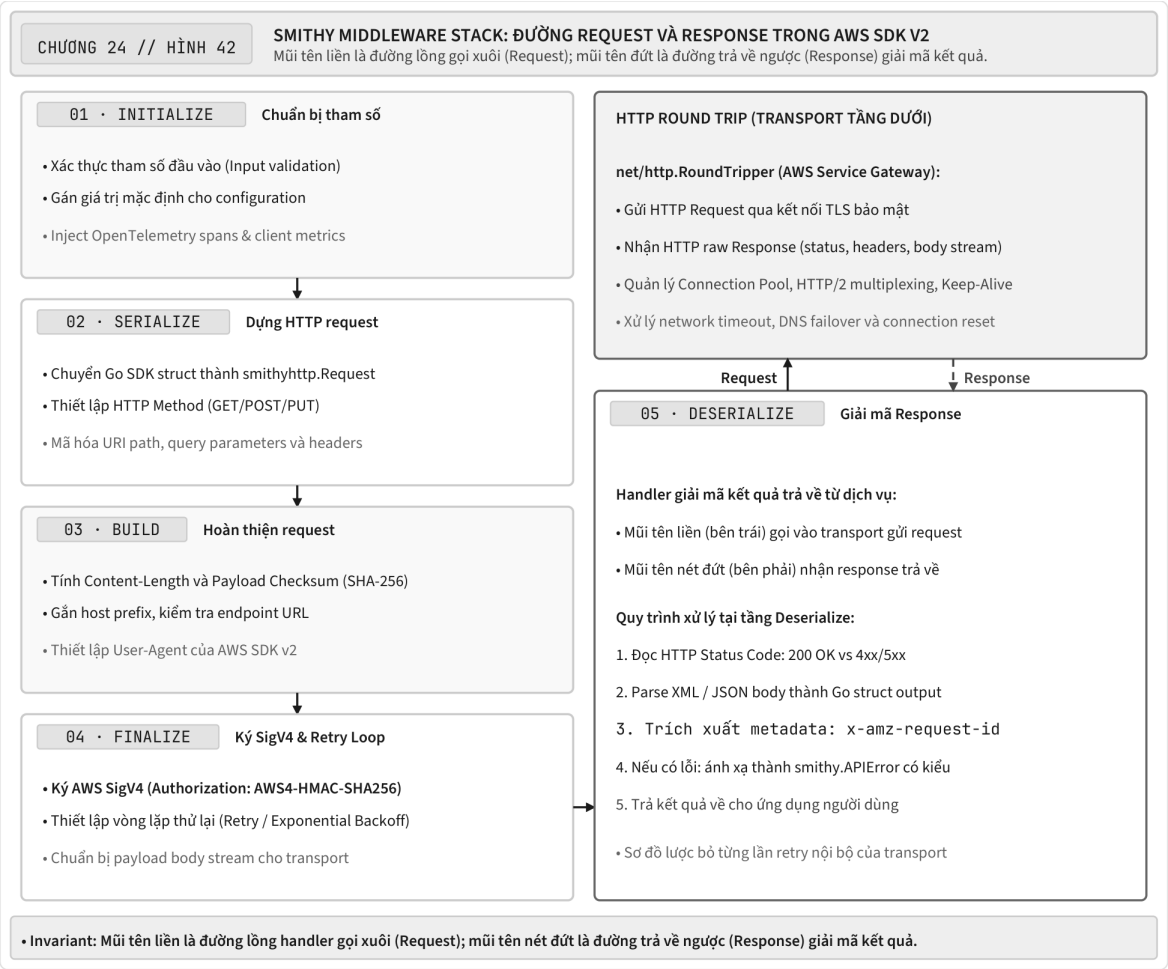
Khác với các dịch vụ mang tính toàn cầu (global services như IAM), hầu hết dịch vụ AWS (như Amazon S3, DynamoDB, EC2) đều vận hành theo từng region vật lý riêng biệt. Nếu không xác định được Region từ các nguồn cấu hình trên, SDK v2 sẽ trả về lỗi khi khởi tạo cuộc gọi đến dịch vụ vùng. Đặc biệt với Amazon S3, dù tên bucket là duy nhất trên phạm vi toàn cầu, mỗi bucket vẫn cư trú tại một region xác định; gửi request tới bucket ở region khác mà không cấu hình cross-region có thể dẫn đến lỗi điều hướng HTTP 301 hoặc lỗi ký chữ ký số SigV4.

---

2. Luồng thực thi của AWS SDK v2 và Smithy Middleware

AWS SDK for Go v2 sử dụng Smithy, một ngôn ngữ mô hình hóa dịch vụ và bộ công cụ sinh SDK. Smithy không phải là một giao thức truyền tải thay thế cho HTTP, và cũng không phải cơ chế ký SigV4 thay cho TLS; ngăn xếp trung gian (middleware stack) là phần tổ chức luồng triệu gọi của SDK.

Một thao tác như `s3.PutObject` đi qua năm bước của **Middleware Stack**. Mỗi bước triệu gọi hàm xử lý bên trong; kết quả phản hồi sau đó đi ngược trở ra ngoài. Vì vậy Deserialize xử lý gói phản hồi sau hàm xử lý, còn Initialize xử lý trước khi yêu cầu được phát đi:



Hình 42 — Mũi tên liền biểu diễn đường gọi vào handler, mũi tên đứt biểu diễn response trở về Deserialize. Middleware cụ thể và vị trí xử lý checksum phụ thuộc operation cùng cấu hình; sơ đồ không mô tả từng lần retry.

Chữ ký số SigV4 (Signature Version 4)

Trong luồng triệu gọi chữ ký SigV4, phần mềm trung gian ký ở pha hoàn tất (**Finalize**) sẽ kiến tạo yêu cầu chuẩn hóa (`canonical request`) từ phương thức HTTP, đường dẫn, tham số truy vấn, các tiêu đề được ký và mã băm của nội dung tải trọng theo chế độ ký. Đây là đại diện của yêu cầu cho thuật toán xác thực, không phải chữ ký của từng gói tin mạng.

SDK sử dụng khóa ký suy dẫn từ khóa bí mật, ngày tháng, vùng và dịch vụ để tạo chữ ký SigV4 cho yêu cầu chuẩn hóa. Máy chủ sẽ đối soát toàn bộ các thành phần được ký; không phải mọi tiêu đề đều được đưa vào danh sách ký

(`SignedHeaders`) và một số chế độ cho phép tải trọng không ký. Sửa đổi phần được bao phủ có thể gây lỗi xác minh, nhưng không suy ra một mã lỗi HTTP cố định cho mọi dịch vụ. Giao thức TLS vẫn luôn cần thiết để bảo vệ đường truyền; chữ ký SigV4 không thay thế cho TLS.

Cơ chế SigV4 xác thực yêu cầu gửi tới AWS và bảo vệ tính toàn vẹn của các thành phần được ký; nó không tự động mã hóa bí mật nội dung, không chống tấn công phát lại trong mọi ngữ cảnh, và không tự động ký toàn bộ tiêu đề. Giao thức TLS chịu trách nhiệm bảo vệ kênh truyền, còn bên gọi phải nắm rõ dịch vụ và tùy chọn ký của lời gọi mình sử dụng.

---

3. Quản lý dữ liệu lớn bằng Paginator

Nếu chỉ gọi một trang của API liệt kê, công cụ có thể bỏ sót object ở các trang sau:

```
// CẢNH BÁO: Chỉ lấy tối đa 1.000 file đầu tiên!
resp, err := client.ListObjectsV2(ctx, &s3.ListObjectsV2Input{
    Bucket: aws.String("big-data-bucket"),
})
```

Một bucket trên S3 có thể chứa 10 triệu đối tượng. API `ListObjectsV2` mặc định chỉ trả về tối đa 1.000 file mỗi lần gọi và trả về mã thông báo tiếp tục (`NextContinuationToken`).

Nếu ta tự viết vòng lặp `for` với token thủ công, mã nguồn sẽ trở nên cồng kềnh và dễ gặp lỗi vòng lặp vô tận nếu xử lý sai điều kiện ngắt.

Mẫu hình chuẩn: SDK Paginator

Hàm khởi tạo `NewListObjectsV2Paginator` quản lý mã đánh dấu tiếp tục (**continuation token**) và trả về từng trang dữ liệu riêng biệt. Nó không cam kết tổng mức sử dụng bộ nhớ sẽ luôn là $O(1)$ đối với mọi dữ liệu đầu vào: kích thước từng trang và cách thức bên gọi lưu giữ kết quả vẫn bắt buộc phải có định mức rõ ràng.

```
paginator := s3.NewListObjectsV2Paginator(
    client, &s3.ListObjectsV2Input{
        Bucket: aws.String("big-data-bucket"),
    },
)

for paginator.HasMorePages() {
    page, err := paginator.NextPage(ctx)
    if err != nil {
        return fmt.Errorf("lỗi đọc trang: %w", err)
    }
    for _, obj := range page.Contents {
        // Xử lý từng file mà không làm đầy RAM
        processFile(*obj.Key)
    }
}
```

Bộ phân trang chỉ lưu giữ trang dữ liệu đang xử lý tại thời điểm tức thời; mức tiêu thụ bộ nhớ của bên gọi phụ thuộc hoàn toàn vào cách xử lý tiếp theo. Hàm `ListAllKeys` trong bài thực hành cố tình gom toàn bộ kết quả vào mảng chuỗi `[]string`, do đó sẽ tiêu tốn bộ nhớ ở mức $O(n)$ theo tổng số lượng khóa. Với các tác vụ quét dữ liệu quy mô lớn, hãy xử lý trực tiếp từng mảng `page.Contents` hoặc sử dụng hàm phản hồi (**callback**) thay vì tích lũy toàn bộ kết quả vào bộ nhớ.

---

4. Phân loại lỗi và Thử lại: smithy.APIError

Khi một lệnh gọi AWS thất bại, ta không thể chỉ so sánh chuỗi lỗi bằng `strings.Contains(err.Error(), "404")`. AWS trả về lỗi có cấu trúc chuẩn mực thông qua interface `smithy.APIError`.

Cơ chế thử lại ở tầng truyền vận của SDK (`aws.Retryer`, mặc định tối đa 3 lần với thuật toán lùi lũy thừa kèm độ trễ ngẫu nhiên cho các lỗi tạm thời) hoàn toàn không bảo đảm tính lũy đẳng nghiệp vụ (**business idempotency**). Nếu một yêu cầu ghi bị ngắt kết nối mạng sau khi máy chủ AWS đã tiếp nhận và thực thi thành công, việc SDK tự động thử lại có thể gây ra tác dụng phụ lặp lại ngoài ý muốn. Một số giao diện API của AWS cung cấp mã định danh lũy đẳng tường minh (như `ClientToken` trong EC2) để máy chủ nhận diện

yêu cầu gửi lặp; tuy nhiên đối với thao tác `PutObject` trên Amazon S3, ngữ nghĩa phụ thuộc chặt chẽ vào cấu hình kho lưu trữ và các điều kiện tiên quyết. Khi kho lưu trữ bật tính năng quản lý phiên bản (**Versioning**), nhiều lượt gửi dữ liệu với cùng một khóa sẽ tạo ra các phiên bản độc lập chứ không ghi đè tại chỗ, do đó việc dùng chung một khóa không bảo đảm tạo ra một hiệu ứng phụ duy nhất. Khi bài toán đòi hỏi tạo mới mà không ghi đè, hệ thống có thể cần yêu cầu có điều kiện (như `If-None-Match: *` khi dịch vụ hỗ trợ), nhưng đây không phải giải pháp vạn năng cho mọi tình huống. Tính lũy đẳng nghiệp vụ bắt buộc phải được thiết kế dựa trên đúng khế ước kỹ thuật của từng thao tác và dịch vụ cụ thể. Trong chính sách của bài thực hành, lỗi từ chối truy cập `AccessDenied` tuyệt đối không được phép thử lại như lỗi tạm thời. Các lỗi điều tiết lưu lượng hoặc dịch vụ tạm thời không khả dụng có thể áp dụng thuật toán lùi lũy thừa, nhưng quyết định thử lại còn phụ thuộc vào bản chất thao tác, định mức tài nguyên và tính lũy đẳng. Đối với sự cố quá thời gian sau một thao tác ghi, máy chủ có thể đã hoàn tất hiệu ứng phụ; tuyệt đối không thử lại một cách mù quáng chỉ vì tên lỗi chứa chuỗi `RequestTimeout`. Hãy sử dụng bộ phân loại lỗi kết hợp chặt chẽ với khế ước dịch vụ và bộ thử lại đã được cấu hình cẩn trọng.

```
func ClassifyError(err error) (bool, string) {
    if err == nil {
        return false, ""
    }

    // 1. Kiểm tra lỗi nghiệp vụ cụ thể của dịch vụ S3
    var noSuchKey *types.NoSuchKey
    if errors.As(err, &noSuchKey) {
        return false, "NoSuchKey"
    }
    var noSuchBucket *types.NoSuchBucket
    if errors.As(err, &noSuchBucket) {
        return false, "NoSuchBucket"
    }
```

```
    // 2. Kiểm tra lỗi giao tiếp chung qua smithy.APIError
    var apiErr smithy.APIError
    if errors.As(err, &apiErr) {
        code := apiErr.ErrorCode()
        switch code {
        case "SlowDown", "ThrottlingException",
            "TooManyRequestsException", "RequestTimeout",
            "ServiceUnavailable":
            return true, code
        default:
            return false, code
        }
    }
    return false, "UnknownError"
}
```

`true` ở đây chỉ đánh dấu ứng viên lỗi tạm thời, không phải quyền thử lại mọi thao tác ghi. Bộ thử lại tiêu chuẩn mặc định tối đa ba **attempt**, gồm lượt đầu và tối đa hai lượt thử lại, không phải ba lần thử lại ngoài lượt đầu. Cấu hình có thể thay đổi; ngân sách và khả năng lặp tác dụng phụ vẫn do từng thao tác quyết định.

---

5. Hiện thực Storage Client và Middleware tùy biến

Dưới đây là cấu trúc mã nguồn trích xuất từ dự án mẫu `labs/part24-aws-sdk-go-v2/`. Trong khi mã nguồn triển khai trên môi trường thực tế sẽ sử dụng `config.LoadDefaultConfig(ctx)` để gọi trực tiếp tới các điểm cuối dịch vụ đám mây, dự án thực hành minh họa kiến trúc bằng máy chủ kiểm thử `httptest.Server` giả lập giao thức Amazon S3 nhằm kiểm chứng hành vi nội tại của SDK (ngăn xếp trung gian, nhà cung cấp định danh, bộ phân trang, phân loại lỗi) hoàn toàn cô lập, không đòi hỏi tài khoản đám mây thật hay phát sinh chi phí vận hành:

1. Provider cấp khóa tạm thời tự xoay vòng

Nhà cung cấp định danh giả lập dưới đây trả về thông tin xác thực có thời hạn hiệu lực để kiểm tra cơ chế lưu đệm của SDK. Mã nguồn không gọi dịch vụ STS và không mô phỏng quy trình xác thực hay phân quyền của dịch vụ này:

```
type DynamicCredentialProvider struct {
    retrieveCount atomic.Int64
    ttl           time.Duration
    roleArn       string
}

func (p *DynamicCredentialProvider) Retrieve(
    ctx context.Context,
) (aws.Credentials, error) {
    select {
    case <-ctx.Done():
        return aws.Credentials{}, ctx.Err()
    default:
```

```
    }
    count := p.retrieveCount.Add(1)
    now := time.Now()

    return aws.Credentials{
        AccessKeyID:     fmt.Sprintf("ASIA-TEMP-KEY-%d", count),
        SecretAccessKey: fmt.Sprintf("SECRET-TOKEN-%d", count),
        SessionToken:    fmt.Sprintf("SESSION-TOKEN-%d", count),
        Source:          "DynamicCredentialProvider",
        CanExpire:       true,
        Expires:         now.Add(p.ttl),
    }, nil
}
```

2. Can thiệp vào Smithy Middleware để thêm Header Audit

Để kiểm tra việc chèn một tiêu đề của ứng dụng vào yêu cầu gửi ra ngoài, chúng ta thiết lập phần mềm trung gian ở giai đoạn `Build`. Bài kiểm thử chỉ thẩm định tiêu đề tại máy chủ cục bộ; không suy diễn rằng dịch vụ nhật ký giám sát đám mây sẽ lưu trữ các tiêu đề tùy biến đó:

```
type AuditHeaderMiddleware struct {
    Origin string
}

func (m *AuditHeaderMiddleware) ID() string {
    return "AuditHeaderMiddleware"
}

func (m *AuditHeaderMiddleware) HandleBuild(
    ctx context.Context,
    in middleware.BuildInput,
    next middleware.BuildHandler,
) (middleware.BuildOutput, middleware.Metadata, error) {
    req, ok := in.Request.(*smithyhttp.Request)
    if ok && req != nil {
        req.Header.Set("X-Audit-Origin", m.Origin)
    }
    return next.HandleBuild(ctx, in)
}
```

Khi khởi tạo client, ta đăng ký middleware này vào pipeline:

```
s3Client := s3.NewFromConfig(cfg, func(o *s3.Options) {
    o.APIOptions = append(
        o.APIOptions,
        func(stack *middleware.Stack) error {
            return stack.Build.Add(
                &AuditHeaderMiddleware{
                    Origin: "automated-backup-worker",
                },
                middleware.After,
            )
        },
    )
})
```

6. Bằng chứng kiểm thử: Kiểm chứng 5 hành vi then chốt

Toàn bộ tập kiểm thử tại `labs/part24-aws-sdk-go-v2/storage_test.go` vận hành độc lập bằng máy chủ giả lập `httptest.Server` (phân loại mức kiểm chứng: giả lập kiểm chứng đối với tầng truyền vận HTTP và kiểm thử đơn vị đối với logic nhà cung cấp, bộ phân trang và phần mềm trung gian), mô phỏng cấu trúc phản hồi XML của Amazon S3 và kiểm chứng 5 hành vi kiến trúc then chốt mà không cần tài nguyên đám mây thực tế:

```
=== RUN   TestCredentialsCacheReuseAndExpiredRefresh
--- PASS: TestCredentialsCacheReuseAndExpiredRefresh (0.11s)
=== RUN   TestCustomSmithyMiddlewareAuditHeader
--- PASS: TestCustomSmithyMiddlewareAuditHeader (0.01s)
=== RUN   TestS3OperationsPutAndGet
--- PASS: TestS3OperationsPutAndGet (0.00s)
=== RUN   TestPaginatorListAllKeys
--- PASS: TestPaginatorListAllKeys (0.00s)
=== RUN   TestErrorClassification
--- PASS: TestErrorClassification (0.00s)
PASS
ok      part24-aws-sdk-go-v2   2.054s
```

1. Tự động làm mới khóa hết hạn (TestCredentialsCacheReuseAndExpiredRefresh)

Test bọc provider giả lập có TTL 100 ms trong `aws.NewCredentialsCache` với cấu hình mặc định. Ở lần gọi thứ hai khi credential còn hạn, cache trả về ngay kết quả đã lưu mà không gọi lại provider. Chỉ sau khi credential hết hạn, lần gọi tiếp theo mới kích hoạt cache gọi lại provider để cấp phát credential mới. Đây là unit test cục bộ với fake provider nhằm kiểm chứng cơ chế bộ đệm của SDK; test không gửi request tới AWS STS thật và không chứng minh credential được refresh thành công khi mạng hay dịch vụ IAM từ xa gặp sự cố.

2. Tiêm Header qua Smithy Middleware (TestCustomSmithyMiddlewareAuditHeader)

Máy chủ HTTP cục bộ bắt gói tin `PutObject` và kiểm tra header `X-Audit-Origin`. Giá trị nhận được chính xác là `"automated-backup-worker"`, chứng minh middleware đã can thiệp thành công vào luồng gửi tin của SDK.

3. Đọc ghi luồng dữ liệu an toàn (TestS3OperationsPutAndGet)

Kiểm chứng tính toàn vẹn của chuỗi dữ liệu nhị phân khi upload và download qua S3 Client.

4. Thu thập toàn bộ danh sách file qua Paginator (TestPaginatorListAllKeys)

Máy chủ giả lập trả về trang 1 kèm `IsTruncated: true` và `NextContinuationToken`, sau đó trả về trang 2 với `IsTruncated: false`. Hàm `ListAllKeys` tự động gọi 2 lần HTTP và thu thập đầy đủ 3 file mà không cần người dùng tự quản lý token phân trang.

5. Phân loại lỗi chính xác (TestErrorClassification)

Kiểm chứng hàm phân loại bóc tách chính xác các lỗi nghiệp vụ mô hình hóa của S3 (`types.NoSuchKey` và `types.NoSuchBucket`) thông qua `errors.As` (đều xác định `isRetryable = false`), đồng thời phân biệt chính xác mã lỗi tạm thời `SlowDown` (đánh dấu `isRetryable = true`) và lỗi phân quyền `AccessDenied` (đánh dấu `isRetryable = false`) của interface `smithy.APIError`.

---

7. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Ghi cứng khóa truy cập trong mã nguồn hoặc ảnh AMI/container.	Nguy cơ lộ thông tin xác thực và toàn bộ đặc quyền liên quan.	Lựa chọn nhà cung cấp thông tin xác thực tạm thời phù hợp với môi trường; hàm `LoadDefaultConfig` vẫn có thể nạp khóa tĩnh nếu cấu hình yêu cầu.
Không đóng luồng phản hồi khi gọi `s3.GetObject`.	Phần thân và tài nguyên tầng truyền vận bị giữ lại, gây nghẽn tài nguyên khi lặp nhiều yêu cầu.	Khai báo `defer resp.Body.Close()` ngay sau khi gọi thành công; việc tái sử dụng kết nối đòi hỏi điều kiện riêng.
Tự viết vòng phân trang mà không quản trị mã thẻ và ngân sách.	Nguy cơ lặp trang vô hạn, thất thoát dữ liệu hoặc chiếm dụng bộ nhớ nghiêm trọng.	Ưu tiên bộ phân trang tích hợp của SDK; luôn thẩm định ngữ cảnh, lỗi, số trang và dung lượng dữ liệu lưu giữ.
Thử lại một cách mù quáng với lỗi từ chối truy cập `AccessDenied`.	Gây tắc nghẽn hàng đợi và kích hoạt các cảnh báo an ninh bất thường trong hệ thống giám sát.	Sử dụng `errors.As(err, &apiErr)` để chỉ thử lại các mã lỗi tạm thời (`SlowDown`, `5xx`).

---

8. Bài tập thực hành thiết kế Cloud Tool

Thử thách 1: Tích hợp Context Timeout chặt chẽ cho thao tác đám mây

Yêu cầu: Xây dựng hàm `PutObjectWithTimeout` tiếp nhận tham số `timeout time.Duration` và truyền ngữ cảnh có thời hạn kết thúc tới SDK. Kiểm thử với máy chủ giả lập có độ trễ cao để quan sát bên gọi chủ động dừng chờ. Thao tác hủy bỏ không chứng minh các hiệu ứng phụ phía máy chủ được đảo ngược hay tài nguyên được giải phóng tức thì ở mọi tầng.

Thử thách 2: Thiết kế Waiter kiểm tra Bucket sẵn sàng

Yêu cầu: Sau khi tạo vùng lưu trữ, sử dụng `s3.NewBucketExistsWaiter` để chờ điều kiện tồn tại được quan sát qua lệnh `HeadBucket`. Thiết lập thời gian chờ tối đa 30 giây, khoảng lùi từ 2 đến 5 giây. Đây không phải là phép kiểm chứng phân giải tên miền toàn cầu hay mức độ sẵn sàng toàn diện của ứng dụng.

---

9. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Quản trị thời gian chết bằng Context

```
func (s *CloudStorage) PutObjectWithTimeout(
    parentCtx context.Context,
    bucket, key string,
    body io.Reader,
    timeout time.Duration,
) error {
    ctx, cancel := context.WithTimeout(parentCtx, timeout)
    defer cancel()

    err := s.PutObject(ctx, bucket, key, body)
    if err != nil {
        if errors.Is(ctx.Err(), context.DeadlineExceeded) {
            return fmt.Errorf(
                "upload %s/%s quá hạn sau %v: %w",
                bucket, key, timeout, err,
            )
        }
        return err
    }
    return nil
}
```

Lời giải Thử thách 2: Đồng bộ hóa bằng SDK Waiter

```
func WaitForBucketReady(
    ctx context.Context,
    client *s3.Client,
    bucket string,
) error {
    waiter := s3.NewBucketExistsWaiter(
        client,
        func(o *s3.BucketExistsWaiterOptions) {
            o.MaxDelay = 5 * time.Second
            o.MinDelay = 2 * time.Second
        },
    )

    // Chờ tối đa 30 giây
    maxWait := 30 * time.Second
    return waiter.Wait(
        ctx,
        &s3.HeadBucketInput{Bucket: aws.String(bucket)},
        maxWait,
    )
}
```

Bộ chờ `BucketExistsWaiter` lặp lại lệnh `HeadBucket` theo chính sách của SDK cho đến khi đạt trạng thái chấp nhận được, gặp lỗi hoặc vượt quá thời gian chờ tối đa `maxWait`. Nó chỉ xác nhận điều kiện tồn tại của vùng lưu trữ; không chứng minh phân giải tên miền, chính sách IAM hay ứng dụng đã sẵn sàng cho một quy trình vận hành lớn hơn. Vì vậy thời hạn chờ và điều kiện chấp nhận phải bám sát thao tác đang chờ.

Nhà cung cấp thông tin xác thực, ngăn xếp trung gian và cơ chế phân trang giải quyết các ranh giới kỹ thuật chuyên biệt. Trước khi triển khai trên hạ tầng AWS thực tế, ta cần kiểm tra vai trò IAM, chính sách phân quyền, ngân sách thử lại và tác dụng phụ của từng thao tác. Chương 25 sẽ chuyển sang tìm hiểu webhook và GitHub API, nơi định danh của một lần gửi tin không đồng nghĩa với định danh của một thao tác nghiệp vụ.

Chương 25 — Git và GitHub dưới góc nhìn của một hệ thống tự động hóa

Trong quy trình vận hành dựa trên Git (GitOps), Git là nguồn khai báo trạng thái mong muốn (**desired state**). Tuy nhiên, nó không bao quát toàn bộ trạng thái thực tế trên môi trường vận hành: cơ sở dữ liệu, các chuỗi bí mật nằm ngoài kho mã nguồn, trạng thái của bộ điều khiển và các dữ liệu quan sát đều có chủ thể quản lý riêng biệt. Chương này phân tích cách một chương trình tự động hóa (**bot**) tiếp nhận phiên bản mã nguồn và siêu dữ liệu từ nền tảng GitHub mà không nhầm lẫn giữa hai tầng dữ liệu đó.

Một cam kết mã nguồn được hợp nhất vào nhánh `main` có thể tự động kích hoạt công cụ điều phối triển khai ứng dụng lên cụm Kubernetes. Một nhãn phân loại được gắn vào yêu cầu kéo mã (**Pull Request**) có thể kích hoạt chương trình tự động cấp phát môi trường kiểm thử tạm thời trên hạ tầng đám mây.

Chương này phân tích hai khiếm khuyết thiết kế điển hình qua các ví dụ thực nghiệm: việc đánh đồng đối tượng Git cục bộ với dữ liệu GitHub qua giao diện lập trình, và việc xử lý thông điệp tự động (**webhook**) mà bỏ qua khâu xác thực, khử trùng lặp gửi tin hay khoảng thời gian gián đoạn do tiến trình bị sập đột ngột. Việc thẩm định các ranh giới kỹ thuật này giúp thiết kế hệ thống trở nên vững chắc và tin cậy hơn.

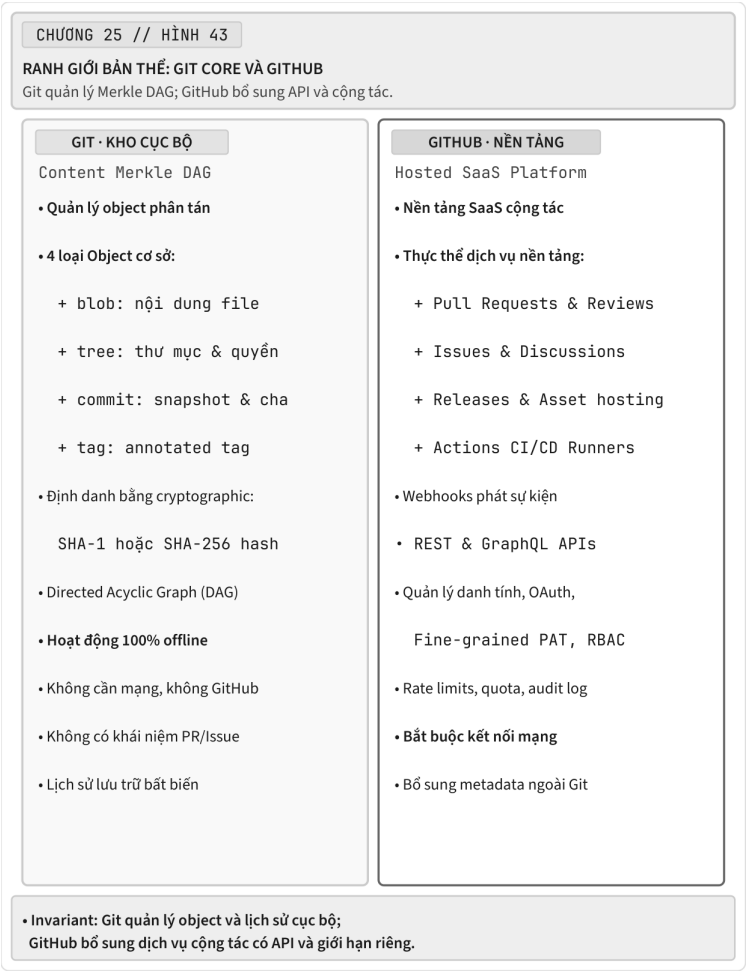
Nội dung chương đi sâu vào bản chất kỹ thuật của Git và nền tảng GitHub dưới lăng kính của một hệ thống hướng sự kiện, từ đó hướng dẫn xây dựng các công cụ tự động hóa chuẩn mực bằng Go.

---

1. Hai thế giới: Trạng thái Git cục bộ và Trạng thái GitHub

Câu hỏi trung tâm của chương này là:

> **Vì sao một hệ thống tự động hóa tin cậy phải tách bạch hoàn toàn giữa Trạng thái Git (Local Git State) và Trạng thái GitHub (GitHub Hosted State)?**



Hình 43 — Git quản lý object và lịch sử trong kho; GitHub bổ sung các dịch vụ cộng tác có API, quyền truy cập và giới hạn riêng.

Hai mô hình dữ liệu song hành

Git lưu commit, cây thư mục (`tree`) và khối dữ liệu (`blob`) trong một đồ thị định danh theo nội dung. Đọc object đã có không đòi hỏi mạng. Clone/fetch cần truy cập kho nguồn nhưng nguồn có thể là đường dẫn cục bộ; chỉ transport qua mạng mới cần kết nối tới máy chủ. Siêu dữ liệu GitHub như Pull Request và review không nằm trong cây mã nguồn của commit.

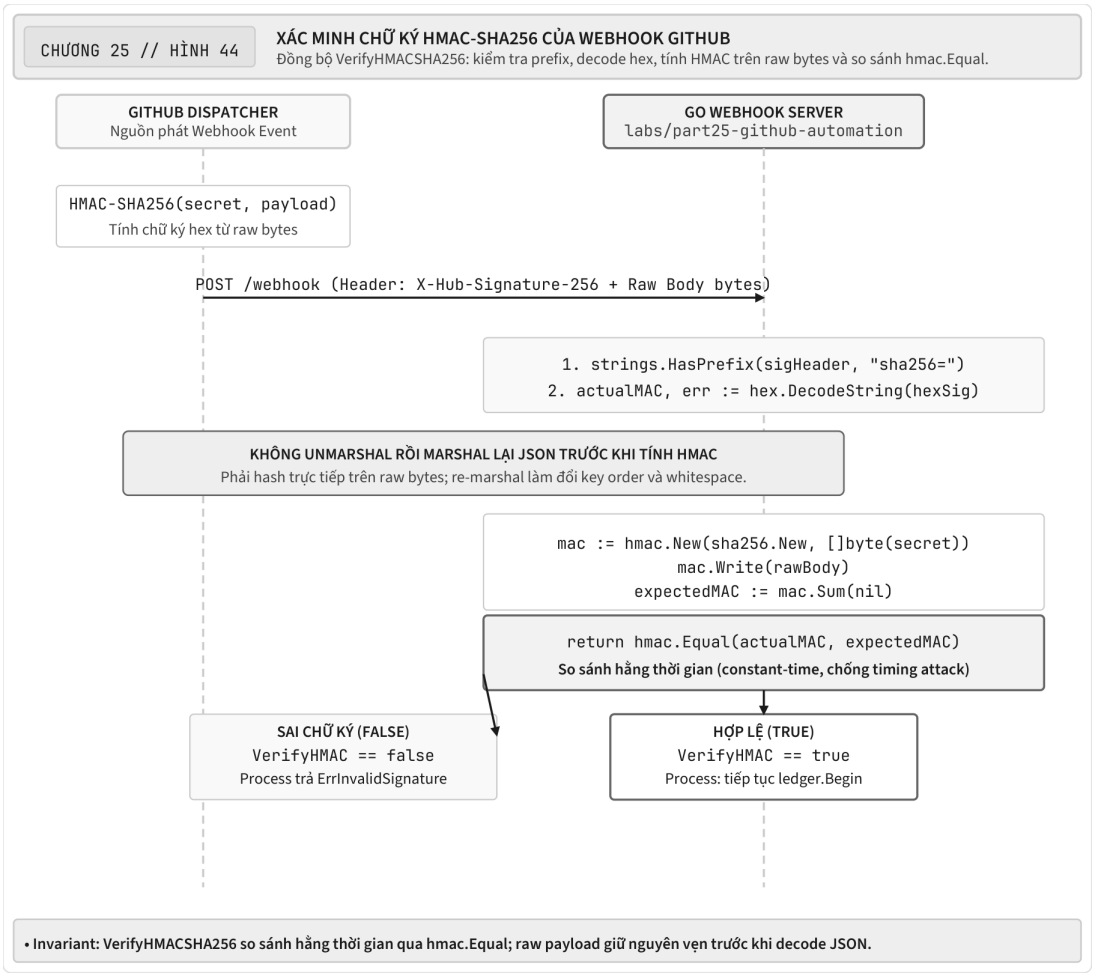
Siêu dữ liệu về Pull Request, nhãn phân loại và đánh giá xét duyệt không nằm trong cây Git của commit. Bài thực hành sử dụng `go-github/v92` cho REST API và `go-git/v5` cho đối tượng Git cục bộ. Một bot chỉ cần dùng một trong hai thư viện cũng hoàn toàn hợp lệ; hãy lựa chọn dựa trên dữ liệu và thao tác thực tế cần xử lý.

---

2. Bảo mật Webhook: Phòng ngừa Timing Side-Channel bằng HMAC-SHA256

Khi GitHub gửi một sự kiện (ví dụ `pull_request.opened`) đến máy chủ qua HTTP Webhook, làm sao ta biết chắc chắn thông điệp này thực sự do GitHub phát ra chứ không phải do một kẻ xấu giả mạo?

GitHub thực hiện ký toàn bộ nội dung thô của gói tin (**raw payload**) bằng chuỗi khóa bí mật thông qua thuật toán HMAC-SHA256, sau đó đính kèm vào tiêu đề yêu cầu: `X-Hub-Signature-256: sha256=<hex_digest>`



Hình 44 — Máy chủ tính toán chữ ký từ chính phần thân dữ liệu thô nhận được rồi so sánh với giá trị tiêu đề. Tuyệt đối không giải mã và tuần tự hóa lại dữ liệu JSON trước khi kiểm tra chữ ký.

Rủi ro rò rỉ thông tin qua thời gian thực thi (Timing Side-Channel)

Nếu ta dùng toán tử thông thường để so sánh chữ ký:

```
// CẢNH BÁO: Rò rỉ thông tin qua thời gian thực thi!
if actualSig == expectedSig { /* so sánh sai */ }
```

Toán tử so sánh chuỗi hoặc mảng byte mặc định trong hầu hết ngôn ngữ lập trình thường kết thúc sớm (early exit) ngay khi phát hiện byte không trùng khớp đầu tiên. Sự chênh lệch thời gian xử lý này vô tình tạo ra một kênh phụ (side-channel). Dù độ trễ biến thiên trên mạng Internet công cộng có thể làm nhiễu tín hiệu đo đạc, trong các môi trường nội bộ, mạng cục bộ tốc độ cao hoặc qua phương pháp phân tích thống kê trên lượng mẫu thử lớn, việc so sánh không hằng số thời gian vẫn có thể cung cấp manh mối để kẻ tấn công thu hẹp không gian tìm kiếm mã xác thực.

So sánh chữ ký an toàn với crypto/hmac.Equal hoặc crypto/subtle

Trong Go, ta bắt buộc phải dùng hàm `hmac.Equal` hoặc `subtle.ConstantTimeCompare`:

```
func VerifyHMACSHA256(
    payload []byte, signatureHeader string, secret []byte,
) bool {
    if len(secret) == 0 ||
        !strings.HasPrefix(signatureHeader, "sha256=") {
        return false
    }
    actualHex := strings.TrimPrefix(signatureHeader, "sha256=")
    actualSig, err := hex.DecodeString(actualHex)
    if err != nil {
        return false
    }

    mac := hmac.New(sha256.New, secret)
    mac.Write(payload)
    expectedSig := mac.Sum(nil)

    // So sánh hằng số thời gian (constant-time)
    return hmac.Equal(actualSig, expectedSig)
}
```

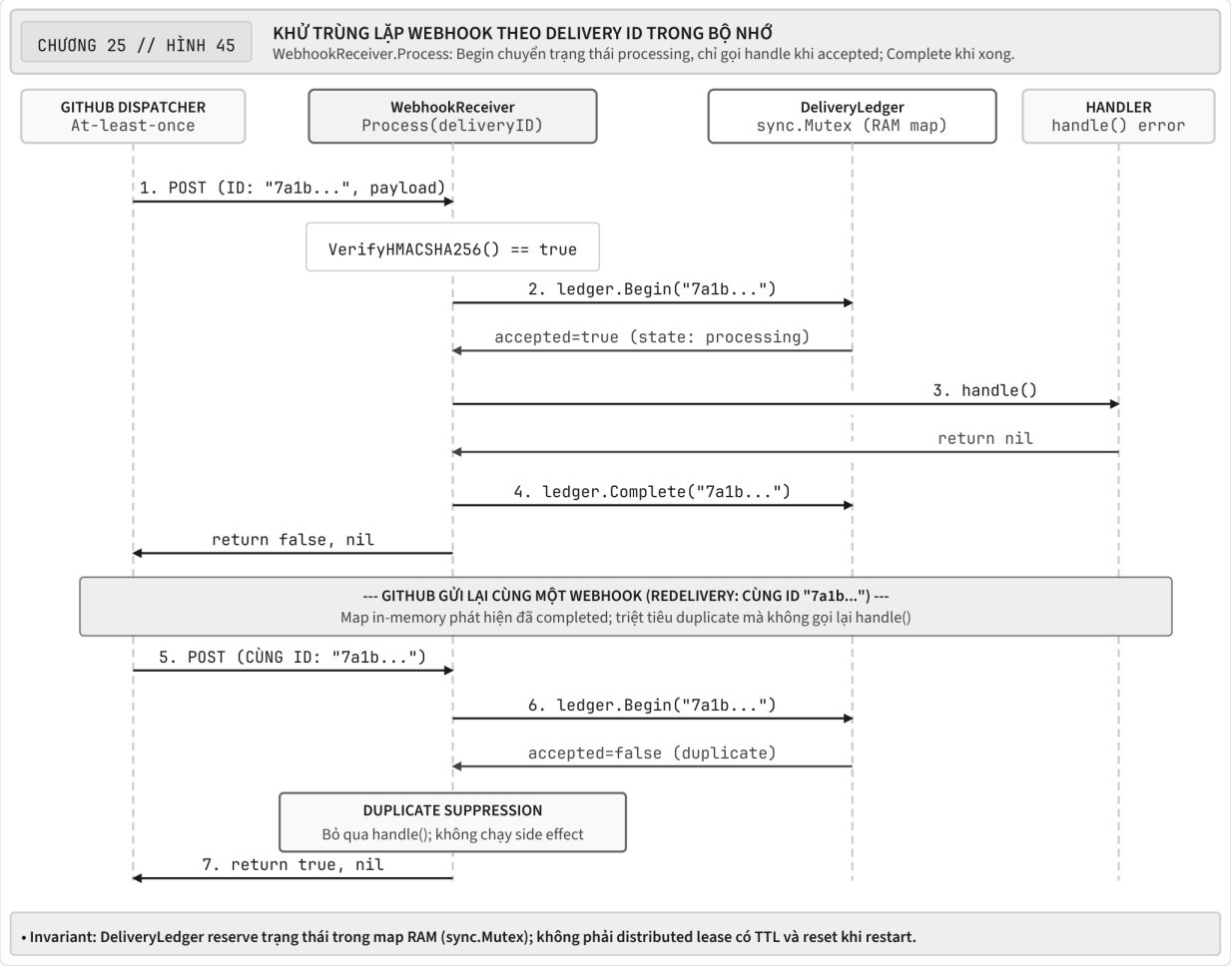
Hàm `hmac.Equal` thực hiện so sánh hai mã xác thực thông điệp mà không làm rò rỉ nội dung qua thời gian tính toán theo đúng khế ước kỹ thuật. Cơ chế này không chứng minh toàn bộ điểm cuối đạt thời gian thực thi hằng số hay loại bỏ mọi kênh rò rỉ phụ; bên gọi vẫn bắt buộc phải giới hạn kích thước tải trọng, xác thực chữ ký và bảo vệ chuỗi bí mật cẩn trọng.

---

3. Khế ước phân tán: Delivery ID và Tính Lũy Đẳng

Trong kiến trúc tích hợp webhook, một ngộ nhận phổ biến là cho rằng GitHub sẽ tự động thử lại mỗi khi máy chủ tiếp nhận trả về mã lỗi 5xx hoặc gặp sự cố quá thời gian chờ (timeout). Đối với các cấu hình webhook tiêu chuẩn, GitHub không tự động phát lại sự kiện khi điểm cuối tiếp nhận gặp lỗi. Việc gửi lại sự kiện chỉ diễn ra khi có thao tác kích hoạt thủ công từ người quản trị qua giao diện, lời gọi REST API từ mã nguồn tự động hóa, hoặc các công cụ nội bộ. Khi một lượt chuyển giao được kích hoạt lại, GitHub sẽ giữ nguyên giá trị định danh GUID trong tiêu đề `X-GitHub-Delivery`.

Mỗi lần phát sự kiện, GitHub đính kèm một mã UUID duy nhất tại header: `X-GitHub-Delivery: 7a1b2c3d-4e5f-6a7b-8c9d-0e1f2a3b4c5d`



Hình 45 — Bài thực hành nhận diện cùng một mã Delivery ID để tránh lặp lại tác dụng phụ trong luồng chạy được kiểm thử. Việc lưu trữ bền vững, xử lý sự cố dừng đột ngột và các lượt gửi tin đồng thời đòi hỏi chính sách và cơ chế riêng biệt.

Mã Delivery ID giúp nhận diện cùng một lượt gửi tin khi được phát lại. Tiêu đề này không nằm trong nội dung tải trọng được thuật toán HMAC ký, do đó riêng việc lưu giữ ID chưa đủ để ngăn chặn tấn công phát lại nếu kẻ tấn công giữ tải trọng và chữ ký hợp lệ rồi thay đổi ID. Sau bước xác thực, hệ thống cần ràng buộc chặt chẽ sự kiện, kho lưu trữ và định danh thao tác nghiệp vụ; trạng thái xử lý phải được lưu trữ bền vững nếu muốn bảo toàn kết quả qua các lần khởi động lại.

Mã Delivery ID chỉ giúp khử trùng lặp một lần chuyển giao; nó không tự biến thành khóa lũy đẳng của nghiệp vụ (như thao tác "triển khai commit X vào môi trường Y"). Hệ thống cần một máy trạng thái tối thiểu `processing` → `completed`: các lượt gửi lại trong lúc đang xử lý phải được giữ lại, lượt thực thi thất bại phải được đánh dấu lỗi để cho phép thử lại, và chỉ lượt đã hoàn tất mới được ngăn chặn thực thi lặp lại. Các thao tác biến đổi trạng thái nghiệp vụ đòi hỏi khóa thao tác riêng biệt, và trên môi trường sản xuất, cần kho dữ liệu phân tán bền vững có thời hạn tồn tại (TTL). Bài thực hành lưu giữ sổ ghi nhận trong bộ nhớ tạm (RAM), do đó các cam kết chỉ có hiệu lực trong vòng đời của tiến trình.

Trên môi trường production thực tế, một hệ thống tự động hóa chịu lỗi đòi hỏi phải kết hợp một trong các cơ chế sau:

Một là, kho lưu trữ phân tán bền vững: Lưu trữ mã định danh chuyển giao vào kho dữ liệu phân tán có cơ chế hết hạn theo thời gian (như Redis hoặc DynamoDB).

Hai là, áp dụng ràng buộc duy nhất trong cơ sở dữ liệu: Ghi nhận mã định danh chuyển giao vào bảng sự kiện với ràng buộc khóa duy nhất trong cùng một giao dịch xử lý nghiệp vụ.

Ba là, thiết lập định danh thao tác nghiệp vụ: Tạo mã định danh lũy đẳng gắn chặt với trạng thái cụ thể của tài nguyên đích thay vì chỉ dựa vào sự kiện chuyển giao của giao thức mạng.

Bốn là, thiết kế các thao tác biến đổi trạng thái có tính lũy đẳng dựa trên định danh và trạng thái quan sát thực tế. Việc khai báo trạng thái mong muốn giúp xác định đích đến nhưng không tự động biến mọi thao tác tạo

mới hay xóa bỏ thành lũy đẳng; các tiền điều kiện, quyền sở hữu và chính sách thử lại sau thất bại cục bộ vẫn cần được thiết kế chặt chẽ như đã phân tích ở các Chương 21 đến Chương 23.

Thiết kế bộ tiếp nhận có kiểm tra trùng lặp trong bộ nhớ:

```
type WebhookReceiver struct {
    secret []byte
    ledger *DeliveryLedger
}

func (r *WebhookReceiver) Process(
    deliveryID, signature string,
    payload []byte,
    handle func() error,
) (bool, error) {
    if deliveryID == "" {
        return false, ErrMissingDelivery
    }
    if !VerifyHMACSHA256(payload, signature, r.secret) {
        return false, ErrInvalidSignature
```

```
    }
    if handle == nil {
        return false, errors.New("webhook handler is required")
    }
    if _, accepted := r.ledger.Begin(deliveryID); !accepted {
        return true, nil
    }
    if err := handle(); err != nil {
        r.ledger.Fail(deliveryID)
        return false, err
    }
    r.ledger.Complete(deliveryID)
    return false, nil
}
```

Thao tác `Complete` diễn ra sau khi hàm xử lý `handle` hoàn tất; nếu phát sinh lỗi, hệ thống sẽ giải phóng lượt đặt chỗ để cho phép thử lại. Khi tiến trình còn đang hoạt động, sổ ghi nhận của bài thực hành ngăn chặn việc chuyển giao đồng thời hoặc việc chạy lại hàm xử lý với các lượt đã hoàn thành. Môi trường thực tế đòi hỏi trạng thái lưu trữ bền vững, cơ chế giành

quyền nguyên tử và quy trình phục hồi sự cố; một thời hạn thuê đơn lẻ không thể che lấp khoảng thời gian tiến trình bị sập đột ngột giữa lúc phát sinh hiệu ứng phụ ngoài cơ sở dữ liệu và thời điểm ghi nhận trạng thái hoàn tất.

Sổ ghi nhận trong bộ nhớ không tồn tại qua lần khởi động lại

Giữ nguyên bài thực hành webhook ở trên để kiểm tra thuật toán HMAC và sự cạnh tranh trong một tiến trình duy nhất. Bây giờ hãy đưa tình huống này qua ranh giới lỗi từng phân tích ở Chương 13: hàm xử lý đã tạo ra một thay đổi, nhưng tiến trình đột ngột tắt trước khi kịp gọi `Complete`. Sau khi khởi động lại, sổ ghi nhận trong bộ nhớ hoàn toàn trống rỗng; một lượt chuyển giao giống hệt có thể vượt qua cổng kiểm soát thêm một lần nữa. Đây không phải là khiếm khuyết của khóa đồng bộ hay chữ ký số HMAC. Khóa bảo vệ trạng thái đang vận hành, còn chữ ký xác nhận tính toàn vẹn của gói tin; không công cụ nào có khả năng lưu giữ cam kết nghiệp vụ sau khi tiến trình biến mất.

Trong bài thực hành hàng đợi gửi đi của Chương 13, mã `credit-001` là định danh thao tác nghiệp vụ, không tự động kế thừa từ mã chuyển giao của tầng truyền vận. Khi tiếp nhận thông điệp, trước hết phải xác thực tải trọng và ánh xạ nó sang thao tác nghiệp vụ được ứng dụng cho phép. Một mã chuyển giao mới cho cùng một thao tác nghiệp vụ tuyệt đối không được phép sinh ra hiệu ứng phụ mới; ngược lại, hai thao tác khác nhau không được gộp chung chỉ vì vô tình có cùng nội dung dữ liệu. Việc ánh xạ này là chính sách xử lý của chương trình tự động, chưa được các bộ thử nghiệm cũ chứng minh.

Hãy tự thiết lập bảng gồm ba điểm kiểm tra trong `TestRecoveryWindows`, sau đó thực thi ca kiểm thử. Tại điểm kiểm tra sau khi gửi, phía tiếp nhận đã lưu giao dịch thành công trong khi phía gửi vẫn ở trạng thái chờ. Lượt gửi lại mới bắt buộc phải có khả năng xử lý việc gửi trùng lặp. Bài kiểm thử `TestSendRetryAndReceiverConflict` kiểm tra hai lượt gửi và xác nhận chỉ một hiệu ứng duy nhất được ghi nhận, đồng thời chủ động từ chối dữ liệu xung đột ở phía tiếp nhận. Bằng chứng này sử dụng hai cơ sở dữ liệu cục bộ

và bên gửi gọi trực tiếp bên nhận; nó không khẳng định nền tảng GitHub cung cấp một khế ước lũy đẳng tương đương cho mọi điểm cuối API.

Một chương trình tự động hóa thực tế cần lựa chọn các hiệu ứng phụ có khả năng lặp lại an toàn, đọc lại trạng thái dựa trên định danh ổn định hoặc tận dụng cơ chế lũy đẳng của giao diện lập trình thực tế. Nếu một yêu cầu tạo bình luận gặp sự cố quá thời hạn sau khi máy chủ đã xử lý, không thể suy đoán ngây thơ rằng "gặp lỗi nên cứ gửi lại". Hàng đợi gửi đi chỉ lưu giữ các công việc chưa được xác nhận, không thể biến kết nối mạng thành một giao dịch nguyên tử. Chính sách thử lại đòi hỏi ngân sách thời gian, độ trễ lùi, giới hạn số lần và phương án đối soát; bài thực hành này chỉ kích hoạt thử lại do người vận hành hoặc ca kiểm thử yêu cầu, tuyệt đối không bổ sung vòng lặp thử lại vô hạn chạy ngầm.

4. Quản trị Rate Limit: Primary và Secondary

Khi chương trình tự động gọi giao diện lập trình của GitHub, hạn ngạch tốc độ là một định mức tài nguyên bắt buộc phải quan sát. Nó không phải là rủi ro duy nhất trên môi trường thực tế và không phải lúc nào cũng là điểm nghẽn lớn nhất:

Tuyệt đối không xem "5.000 requests/giờ" là con số phổ quát cố định. GitHub áp dụng hạn mức phân cấp tùy theo loại định danh và ngữ cảnh ủy quyền:

Loại định danh (Authentication Context)	Hạn mức Primary Quota	Dấu hiệu nhận diện
Chưa xác thực (Unauthenticated)	60 requests/giờ (tính theo địa chỉ IP)	Dễ bị nghẽn trong môi trường NAT/CI chung
PAT / OAuth User	Mức cơ bản 5.000 request/giờ theo user; app Enterprise đủ điều kiện có mức cao hơn.	Đọc X-RateLimit-Limit của response.
GitHub App user access token	Chung quota theo user; app thuộc Enterprise Cloud đủ điều kiện có thể là 15.000/giờ.	Không coi mỗi app là một quota user độc lập.
GitHub App: Installation (Non-Enterprise)	Base 5.000 req/h; nếu repos > 20: +50/h/repo; nếu org users > 20: +50/h/user; trần tối đa 12.500 req/h	Phù hợp nhất cho bot tự động hóa cấp tổ chức
GitHub App: Enterprise Cloud Installation	Trần tối đa 15.000 requests/giờ	Áp dụng cho tổ chức trên GitHub Enterprise Cloud
GITHUB_TOKEN trong GitHub Actions	1.000 requests/giờ cho mỗi repository (hoặc Enterprise rate nếu resource thuộc Enterprise Cloud)	Áp dụng cho runner tiêu chuẩn trong GitHub Actions
GitHub Enterprise Server (On-Premises)	Cấu hình độc lập bởi quản trị viên hệ thống	Tùy biến theo chính sách triển khai tự lưu trữ

Bên cạnh hạn ngạch cơ bản, GitHub còn áp dụng hạn ngạch thứ cấp, ví dụ như giới hạn số yêu cầu đồng thời hoặc tốc độ tạo nội dung mới. Khi vượt quá ngưỡng này, phản hồi trả về có thể là mã `403` hoặc `429`. Nếu phản hồi có tiêu đề `Retry-After`, hệ thống bắt buộc phải chờ đợi ít nhất khoảng thời gian đó; tiêu đề này không được cam kết sẽ luôn hiện diện. Khi thiếu tiêu đề trên, cần phân tích thông báo lỗi cùng các tiêu đề hạn ngạch khác, sau đó áp

dụng ngân sách chờ đợi và thử lại theo tài liệu hướng dẫn, tuyệt đối không lặp lại yêu cầu tức thì.

Thuật toán bóc tách Rate Limit trong Go

```
func ParseRateLimit(
    header http.Header, now time.Time,
) RateLimitStatus {
    status := RateLimitStatus{}
    if limit, err := strconv.Atoi(
        header.Get("X-RateLimit-Limit")); err == nil {
        status.Limit = limit
    }

    remStr := header.Get("X-RateLimit-Remaining")
    if remStr != "" {
        if rem, err := strconv.Atoi(remStr); err == nil {
            status.Remaining = rem
            if rem == 0 {
                status.IsExhausted = true
            }
        }
    }

    resetStr := header.Get("X-RateLimit-Reset")
    if resetStr != "" {
        epoch, err := strconv.ParseInt(resetStr, 10, 64)
        if err == nil {
            status.ResetAt = time.Unix(epoch, 0)
```

```
            if status.IsExhausted && status.ResetAt.After(now) {
                status.RetryAfter = status.ResetAt.Sub(now)
            }
        }
    }

    // Secondary rate limit dùng header Retry-After
    retryStr := header.Get("Retry-After")
    if retryStr != "" {
        sec, err := strconv.ParseInt(retryStr, 10, 64)
        if err == nil && sec >= 0 {
            delay := time.Duration(math.MaxInt64)
            if sec <= math.MaxInt64/int64(time.Second) {
                delay = time.Duration(sec) * time.Second
            }
            if delay > status.RetryAfter {
                status.RetryAfter = delay
            }
            status.IsExhausted = true
        }
    }
    return status
}
```

Lab đọc `Retry-After` dạng số giây theo hướng dẫn GitHub, bỏ qua giá trị âm/sai định dạng và bão hòa nếu duration không biểu diễn được. Khi đồng thời có primary reset và Retry-After, giữ khoảng chờ dài hơn; không ghi đè ngân sách một giờ thành một giây. Đây là parser metadata, không phải bộ cho phép retry: thiếu header không chứng minh còn quota, và dạng HTTP-date chưa được hỗ trợ. Caller vẫn phải đọc status/thông báo lỗi và chọn policy chờ theo API.

---

5. Tương tác GitHub API (`go-github/v92`) và Git trên bộ nhớ (`go-git/v5`)

Một công cụ tự động hóa toàn diện cần tương tác đồng thời với cả hai thế giới: giao tiếp với GitHub API thông qua thư viện `go-github/v92` do Google duy trì để truy xuất siêu dữ liệu, và thao tác cây thư mục Git cục bộ thông qua `go-git/v5`.

Tích hợp GitHub Client

Trong gói `labs/part25-github-automation/github_client.go`, ta đóng gói `github.Client` để hỗ trợ cấu hình linh hoạt endpoint mạng (cho phép trỏ tới máy chủ kiểm thử cục bộ `httptest.Server` hoặc máy chủ GitHub Enterprise):

```
type GitHubClient struct {
    client *github.Client
}

func NewGitHubClient(
    httpClient *http.Client, baseURL string,
) (*GitHubClient, error) {
    var opts []github.ClientOptionsFunc
    if httpClient != nil {
        opts = append(opts, github.WithHTTPClient(httpClient))
    }
    if baseURL != "" {
        if !strings.HasSuffix(baseURL, "/") {
            baseURL += "/"
        }
        opts = append(opts, github.WithURLs(&baseURL, nil))
    }
    gh, err := github.NewClient(opts...)
    if err != nil {
        return nil, fmt.Errorf("bad client options: %w", err)
```

```
    }
    return &GitHubClient{client: gh}, nil
}

func (c *GitHubClient) GetRepository(
    ctx context.Context, owner, repo string,
) (*github.Repository, *github.Rate, error) {
    repository, resp, err := c.client.Repositories.Get(
        ctx, owner, repo,
    )
    if err != nil {
        return nil, nil, err
    }
    var rate *github.Rate
    if resp != nil {
        rate = &resp.Rate
    }
    return repository, rate, nil
}
```

Thao tác Git trên bộ nhớ (In-memory Git) với go-git

Việc gọi trực tiếp công cụ dòng lệnh Git tạo ra sự phụ thuộc vào tệp thực thi bên ngoài và vòng đời của tiến trình con; thư mục làm việc đòi hỏi quyền sở hữu riêng biệt nếu vận hành đồng thời. Thư viện `go-git` là một giải pháp thay thế: ví dụ này sử dụng `memory.NewStorage()` cho kho lưu trữ đối tượng và `memfs.New()` cho cây thư mục làm việc trong bộ nhớ, không khẳng định rằng mọi trường hợp sử dụng dòng lệnh đều phát sinh xung đột:

```
func NewInMemGitRepository() (*InMemGitRepository, error) {
    fs := memfs.New()
    storer := memory.NewStorage()

    repo, err := git.Init(storer, fs)
    if err != nil {
        return nil, fmt.Errorf("lỗi init git mem: %w", err)
    }

    return &InMemGitRepository{repo: repo, fs: fs}, nil
}
```

Ghi file và tạo commit với chữ ký tác giả trong bộ nhớ:

```
func (r *InMemGitRepository) WriteFileAndCommit(
    filePath string, content []byte, author, msg string,
) (string, error) {
    wt, err := r.repo.Worktree()
    if err != nil {
        return "", err
    }

    f, err := r.fs.Create(filePath)
    if err != nil {
        return "", err
    }
    n, err := f.Write(content)
    if err == nil && n != len(content) {
        err = io.ErrShortWrite
    }
    if err != nil {
        _ = f.Close()
        return "", err
    }
```

```
    if err := f.Close(); err != nil {
        return "", err
    }

    if _, err := wt.Add(filePath); err != nil {
        return "", err
    }

    hash, err := wt.Commit(msg, &git.CommitOptions{
        Author: &object.Signature{
            Name:  author,
            Email: author + "@bot.local",
            When:  time.Now(),
        },
    })
    if err != nil {
        return "", err
    }
    return hash.String(), nil
}
```

Với các cấu trúc `memory.NewStorage()` và `memfs` của ví dụ, đối tượng Git và cây làm việc không được chủ động ghi xuống hệ thống tệp đĩa cứng. Thao tác tính mã băm, cấp phát bộ nhớ và thu gom rác vẫn phát sinh chi phí nhất định; hệ điều hành vẫn có thể hoán đổi bộ nhớ ra đĩa, vì vậy không thể suy đoán rằng "không có một byte nào chạm tới đĩa vật lý" hay cam kết một mức độ trễ cố định chỉ từ cấu hình chạy trong bộ nhớ.

---

6. Bằng chứng kiểm thử: Chứng minh 5 quy luật tự động hóa

Các bài kiểm thử trong `labs/part25-github-automation/` kiểm tra các trường hợp xác thực HMAC, sổ ghi nhận, bộ phân tích hạn ngạch và thao tác Git trong bộ nhớ; bộ thử nghiệm HTTP kiểm tra luồng sử dụng thư viện `go-github`. Chúng không chứng minh nền tảng GitHub thật luôn chuyển giao thành công, không loại bỏ nguy cơ tấn công phát lại khi kẻ xấu thay đổi mã tiêu đề, và không cam kết mọi kịch bản sự cố tiến trình đều an toàn:

```
=== RUN   TestWebhookHMACVerification
--- PASS: TestWebhookHMACVerification (0.00s)
=== RUN   TestWebhookDeliveryIdempotency
--- PASS: TestWebhookDeliveryIdempotency (0.00s)
=== RUN   TestRateLimitParsingPrimaryAndSecondary
--- PASS: TestRateLimitParsingPrimaryAndSecondary (0.00s)
=== RUN   TestInMemGitCommitAndHead
--- PASS: TestInMemGitCommitAndHead (0.00s)
=== RUN   TestGitHubClientIntegrationWithMockServer
--- PASS: TestGitHubClientIntegrationWithMockServer (0.00s)
PASS
ok      part25-github-automation   0.263s
```

1. Xác thực chữ ký an toàn (TestWebhookHMACVerification)

Kiểm chứng 4 ca biên: chữ ký đúng định dạng `sha256=` với secret hợp lệ; từ chối khi sửa đổi 1 byte trong payload; từ chối khi dùng sai Webhook Secret; và từ chối khi header sai định dạng.

2. Khử trùng lặp sự kiện (TestWebhookDeliveryIdempotency)

Gửi 2 webhook mang cùng một `X-GitHub-Delivery`. Lần đầu tiên xử lý thành công (`isDuplicate = false`). Lần thứ hai được nhận diện chính xác là sự kiện gửi lặp (`isDuplicate = true`), bảo vệ hệ thống không bị kích hoạt kép khi có redelivery từ UI hoặc qua REST API.

3. Bóc tách Rate Limit hai tầng (TestRateLimitParsingPrimaryAndSecondary)

Kiểm chứng cả hai tình huống: khi chạm trần Primary Quota (`Remaining: 0`), tính đúng thời gian cần chờ đến mốc `ResetAt`; khi chạm trần Secondary Quota (`Retry-After: 120`), chuyển đổi chính xác thành khoảng chờ 120 giây.

4. Vận hành Git trên RAM (TestInMemGitCommitAndHead)

Tạo liên tiếp 2 commit trong bộ nhớ RAM, kiểm tra mã băm SHA của commit thứ nhất và thứ hai, xác nhận con trỏ HEAD dịch chuyển chuẩn xác theo đồ thị DAG.

5. Tương tác GitHub Client (TestGitHubClientIntegrationWithMockServer)

Kiểm chứng việc khởi tạo client `google/go-github/v92`, định tuyến qua máy chủ giả lập `httptest.Server`, trích xuất chính xác cấu trúc repository và thông tin hạn mức `Rate` từ tiêu đề phản hồi.

---

7. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
So sánh chữ ký bằng == thay vì `hmac.Equal`.	Rò rỉ thông tin qua thời gian thực thi (timing side-channel).	Dùng `crypto/hmac.Equal` hoặc `crypto/subtle.ConstantTimeCompare`.
Bỏ qua X-GitHub-Delivery và xử lý mù quáng mọi webhook.	Gây trùng lặp hành vi (tạo 2 PR, merge 2 lần) khi có redelivery từ UI hoặc qua REST API.	Lưu `X-GitHub-Delivery` vào bộ đệm và bỏ qua các sự kiện trùng lặp.
Gọi API dồn mà không quan sát quota.	Có thể cạn primary hoặc secondary budget theo token/endpoint thực tế.	Đọc rate-limit headers; chọn budget chờ/retry theo workload. Mốc Remaining dưới 50 chỉ là ví dụ policy, không phải ngưỡng an toàn chung.
Dùng thư mục tạm chung không có o wnership/cleanup.	Có thể xung đột hoặc để lại dữ liệu nhạy cảm.	Dùng thư mục riêng và cleanup rõ, hoặc memory storage khi kích thước và lifecycle phù hợp; RAM không tự là ranh giới bảo mật.

---

8. Bài tập thực hành thiết kế Bot Tự động hóa

Thử thách 1: Lọc sự kiện theo nhánh (Branch Filtering Middleware)

Yêu cầu kỹ thuật: Một webhook `push` được gửi tới mỗi khi có commit mới. Hãy viết hàm lọc sự kiện chỉ cho phép xử lý nếu sự kiện nhắm vào nhánh `refs/heads/main`. Bỏ qua tất cả các sự kiện của nhánh tính năng (`feature/*`) mà không kích hoạt quy trình triển khai.

Thử thách 2: Bộ điều tốc thông minh (Smart Backoff with Jitter)

Yêu cầu kỹ thuật: Với fixture `Retry-After: 30`, tính thời gian chờ ít nhất 30 giây rồi thêm jitter 10–25%. Nếu nhiều worker cùng chờ, jitter có thể giảm nhịp gửi đồng loạt, không bảo đảm tránh secondary limit. Các tỷ lệ này là cấu hình bài tập, không phải ngưỡng GitHub hay security policy production.

---

9. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Lọc sự kiện theo nhánh đích

```
type PushPayload struct {
    Ref string `json:"ref"`
}

func ShouldDeployFromWebhook(payload []byte) bool {
    var push PushPayload
    if err := json.Unmarshal(payload, &push); err != nil {
        return false
    }

    // Chỉ kích hoạt tự động hóa trên nhánh chính (main)
    return push.Ref == "refs/heads/main"
}
```

Đây chỉ là bộ lọc tối thiểu phục vụ bài học đọc dữ liệu tải trọng, không phải chính sách áp dụng trên môi trường vận hành thực tế. Điểm cuối tiếp nhận thực tế bắt buộc phải xác thực chữ ký trước khi phân tích cú pháp, thiết lập danh sách cho phép cho cả tên sự kiện, hành động cụ thể, kho mã nguồn, chủ sở hữu, mã cài đặt ứng dụng và nhánh tham chiếu; sau đó liên kết chặt chẽ các giá trị đó với cấu hình triển khai. Việc chỉ kiểm tra nhánh `ref` đơn thuần sẽ tạo kẽ hở cho phép một kho lưu trữ hoặc lượt cài đặt nằm ngoài phạm vi kích hoạt các hành động nhạy cảm.

Lời giải Thử thách 2: Tính toán Backoff kết hợp Jitter

```
func CalculateBackoffWithJitter(
    baseDelay time.Duration,
    next func() float64,
) (time.Duration, error) {
    if baseDelay < 0 {
        return 0, errors.New("negative base backoff")
    }
    if next == nil {
        return 0, errors.New("jitter source is required")
    }
    value := next() // contract: 0 <= value < 1
    if math.IsNaN(value) || value < 0 || value >= 1 {
        return 0, errors.New("invalid jitter value")
    }
    fraction := 0.10 + value*0.15
    jitter := time.Duration(float64(baseDelay) * fraction)
    if baseDelay > time.Duration(math.MaxInt64)-jitter {
        return 0, errors.New("backoff duration overflows")
    }
    return baseDelay + jitter, nil
}
```

Độ trễ ngẫu nhiên chỉ làm giảm mức độ tương quan giữa các yêu cầu, không thể triệt tiêu hoàn toàn các đợt bùng nổ tải. Hàm xử lý tiếp nhận nguồn sinh số ngẫu nhiên qua cơ chế tiêm phụ thuộc; môi trường thực tế có thể sử dụng bộ sinh số ngẫu nhiên an toàn hoặc đối tượng sinh số có chủ thể quản lý riêng, trong khi bài kiểm thử truyền vào giá trị cố định để bảo đảm khả năng tái lập. Tuyệt đối không sử dụng bộ sinh số ngẫu nhiên toàn cục như một khế ước ngầm định.

Nắm vững cách vận hành kết hợp giữa xác thực HMAC-SHA256, tính lũy đẳng trong chuyển giao thông điệp, cơ chế gọi giao diện lập trình có kiểm soát hạn ngạch và thư viện Git vận hành trong bộ nhớ giúp xây dựng các hệ thống tự động hóa an toàn và vững chắc. Trong Chương 26, chúng ta sẽ mở rộng năng lực bảo vệ phần mềm lên cấp độ chuỗi cung ứng: thiết lập cổng kiểm soát chuỗi cung ứng phần mềm có thể kiểm chứng bằng Go với chữ ký số Cosign không cần khóa, mã băm bản kê OCI bất biến và công cụ quét lỗ hổng mã nguồn `govulncheck`.

Chương 26 — Chuỗi cung ứng phần mềm có thể kiểm chứng

Một kịch bản chuỗi cung ứng phần mềm điển hình có thể bao gồm mã nguồn nghiệp vụ, nhiều module phụ thuộc gián tiếp và ảnh nền container (base image) chứa sẵn các gói cài đặt của hệ điều hành. Số lượng thành phần thực tế phụ thuộc vào phương thức đóng gói: một ảnh nền tối giản `scratch` và một ảnh nền hệ điều hành đầy đủ có tập hợp gói phần mềm hoàn toàn khác nhau. Vì vậy danh mục thành phần (inventory) phải được kết xuất trực tiếp từ artifact thực tế, không thể ước lượng cảm tính từ số dòng mã nguồn Go của ứng dụng.

Thực tế này mở rộng bề mặt rủi ro của chuỗi cung ứng phần mềm (software supply chain).

Các sự cố an ninh thực tế như SolarWinds, Codecov hay các cuộc tấn công nhầm lẫn phụ thuộc (dependency confusion) và giả mạo định danh gói (typosquatting) chứng minh một ranh giới bảo mật quan trọng:

> **Một kho mã nguồn sạch, được duyệt kỹ lưỡng và vượt qua mọi bài kiểm thử đơn vị, vẫn có thể tạo ra một container image độc hại nếu quy trình đóng gói, gói phụ thuộc hoặc kho lưu trữ artifact bị xâm phạm.**

Ta cần kiểm tra ảnh container nào đã được xét, bằng chứng nào gắn nó với bộ dựng hoặc nguồn mã được phép, và phát hiện lỗ hổng nào phải chặn theo chính sách kiểm soát. Chữ ký và việc quét mã không chứng minh gói phát hành hoàn toàn sạch mã độc hay lỗ hổng; từng bộ kiểm chứng chỉ đưa ra kết luận trong phạm vi dữ liệu và gốc tin cậy của nó.

Chương này dùng một cổng chính sách Go để học nguyên tắc fail-closed: từ chối khi không xác minh được bằng chứng mà chính sách yêu cầu. Lab kiểm tra định dạng digest, chữ ký ECDSA cục bộ và dữ liệu mẫu về xuất xứ, lỗ hổng. Nó không tích hợp xác minh Cosign, không chạy `govulncheck` và không chứng nhận một quy trình đạt SLSA Build L3. Cấp độ SLSA còn đòi

bằng chứng về nền tảng dựng và cách bảo vệ provenance; một chuỗi `BuilderID` hay test xanh không đáp ứng những yêu cầu đó.

---

1. Mental Model: Chuỗi bảo chứng từ Mã nguồn đến Triển khai

Sơ đồ là thiết kế tham chiếu gồm sáu chặng, không phải những tích hợp lab đã chạy. Chữ ký cần gắn với khóa và định danh đã xác thực; provenance cần gắn với artifact thực. Cổng phải đối chiếu bằng chứng với chính sách của mình, không chỉ kiểm tra có một tệp mang tên "attestation":



Hình 46 — Cổng chỉ xác minh các bằng chứng mà chính sách yêu cầu. Mã băm nội dung, chữ ký và dữ liệu xuất xứ có vai trò khác nhau; đủ bằng chứng theo chính sách không chứng minh gói phát hành sạch mọi lỗi hoặc lỗ hổng.

Mỗi mắt xích cần bằng chứng phù hợp, nhưng không một tệp hay nhãn nào tự chứng minh toàn bộ chuỗi. `go.sum` ghi mã băm kiểm tra (**checksum**) module mà bộ công cụ biên dịch đã biết và kiểm tra, chứ không khóa hoàn toàn đồ thị như tệp khóa phiên bản. Dữ liệu xuất xứ (**provenance**) chỉ hữu ích sau khi bộ kiểm chứng xác minh đối tượng, danh tính và kỳ vọng của chính

sách. Mã băm nhận diện nội dung; cơ chế xác minh không khóa (**keyless verification**) của Cosign còn kiểm tra gốc tin cậy, định danh chứng chỉ và khẳng định mã băm. Bài lab dưới đây mô hình hóa chính sách đóng an toàn, không thực hiện các bước Cosign hoặc SLSA hoàn chỉnh đó.

---

2. Toàn vẹn Module: `go.sum` và Go Checksum Database

Trước khi ảnh container (`container image`) được đóng gói, chuỗi cung ứng bắt đầu ngay tại thời điểm Go tải các gói phụ thuộc về máy.

Bản chất kỹ thuật của `go.sum`

Một ngộ nhận kinh điển trong cộng đồng là coi `go.sum` như một tệp khóa phiên bản tương tự `package-lock.json` hay `yarn.lock`. Theo thiết kế, tệp `go.mod` mới là nơi quyết định phiên bản mô-đun thông qua giải thuật lựa chọn phiên bản tối thiểu (Minimal Version Selection - MVS). Ngược lại, `go.sum` là cơ sở dữ liệu xác thực nội dung mật mã (Content Validation Database). Nó lưu trữ các mã băm SHA-256 của toàn bộ mã nguồn mô-đun và tệp `go.mod` tương ứng nhằm phát hiện mọi hành vi can thiệp trái phép hoặc thay đổi nội dung sau thời điểm phát hành chính thức.

Mỗi bản ghi trong `go.sum` được cấu trúc thành hai dòng thông tin bổ trợ:

```
github.com/gin-gonic/gin v1.9.1 h1:4A06lVSJ...
github.com/gin-gonic/gin v1.9.1/go.mod h1:h1hp...
```

Dòng định dạng `h1:<base64-hash>` chứa chuỗi băm chuẩn tắc của cây thư mục mô-đun (theo giải thuật `dirhash.Hash1`): danh sách đường dẫn tệp được sắp xếp thứ tự, băm SHA-256 từng tệp, ghép thành danh sách định dạng chuẩn tắc rồi băm SHA-256 tổng thể và mã hóa base64. Dòng có hậu tố `/go.mod` lưu trữ mã băm chỉ tính riêng trên nội dung tệp khai báo phụ thuộc của mô-đun đó, cho phép công cụ Go kiểm tra cây phụ thuộc một cách nhanh chóng mà không bắt buộc phải tải toàn bộ mã nguồn của các mô-đun gián tiếp về máy.

Go Checksum Database (`sum.golang.org`)

Nếu một kẻ xấu xâm nhập được máy chủ Git của một thư viện bên thứ ba và tráo đổi nội dung của release tag `v1.2.0`, điều gì sẽ xảy ra?

Nếu máy cục bộ chưa từng tải bản release đó, làm sao Go biết mã băm `h1:` nào là chuẩn?

Với các mô-đun thuộc phạm vi kiểm tra của cơ sở dữ liệu mã băm và cấu hình mặc định phù hợp, bộ công cụ Go sẽ đối chiếu mã băm đã tải về với nhật ký minh bạch (**transparency log**) tại `sum.golang.org`. Nếu nội dung không trùng khớp, quá trình biên dịch sẽ lập tức báo lỗi kiểm tra mã băm. Cơ chế này bảo đảm tính toàn vẹn và nhất quán của nội dung tải về, hoàn toàn không chứng minh nội dung đó không chứa mã độc; các cấu hình như `GOPRIVATE`, `GONOSUMDB`, `GOSUMDB=off` và việc thay thế mã nguồn bằng chỉ thị `replace` sẽ tạo ra các ranh giới kiểm soát khác. Kỹ sư bắt buộc phải kiểm tra cấu hình biên dịch trước khi đưa ra kết luận rằng mô-đun đã đi qua cùng một quy trình thẩm định.

> Khuyến nghị: Luôn lưu trữ tệp `go.sum` cùng tệp `go.mod` vào hệ thống quản lý mã nguồn khi mô-đun có các gói phụ thuộc sinh ra tệp này. Tuyệt đối không xóa bỏ các mã băm đã được ghi nhận chỉ để tránh xung đột khi hợp nhất mã nguồn (**merge conflict**). Cơ sở dữ liệu mã băm vẫn có thể kiểm tra các mô-đun đủ điều kiện ngay cả khi chưa có mục ghi nhận cục bộ; do đó việc thiếu tệp `go.sum` không tự chứng minh rằng mã nguồn đã bị tráo đổi, nhưng sẽ làm mất đi bằng chứng mã băm độc lập mà dự án đã lưu giữ.

---

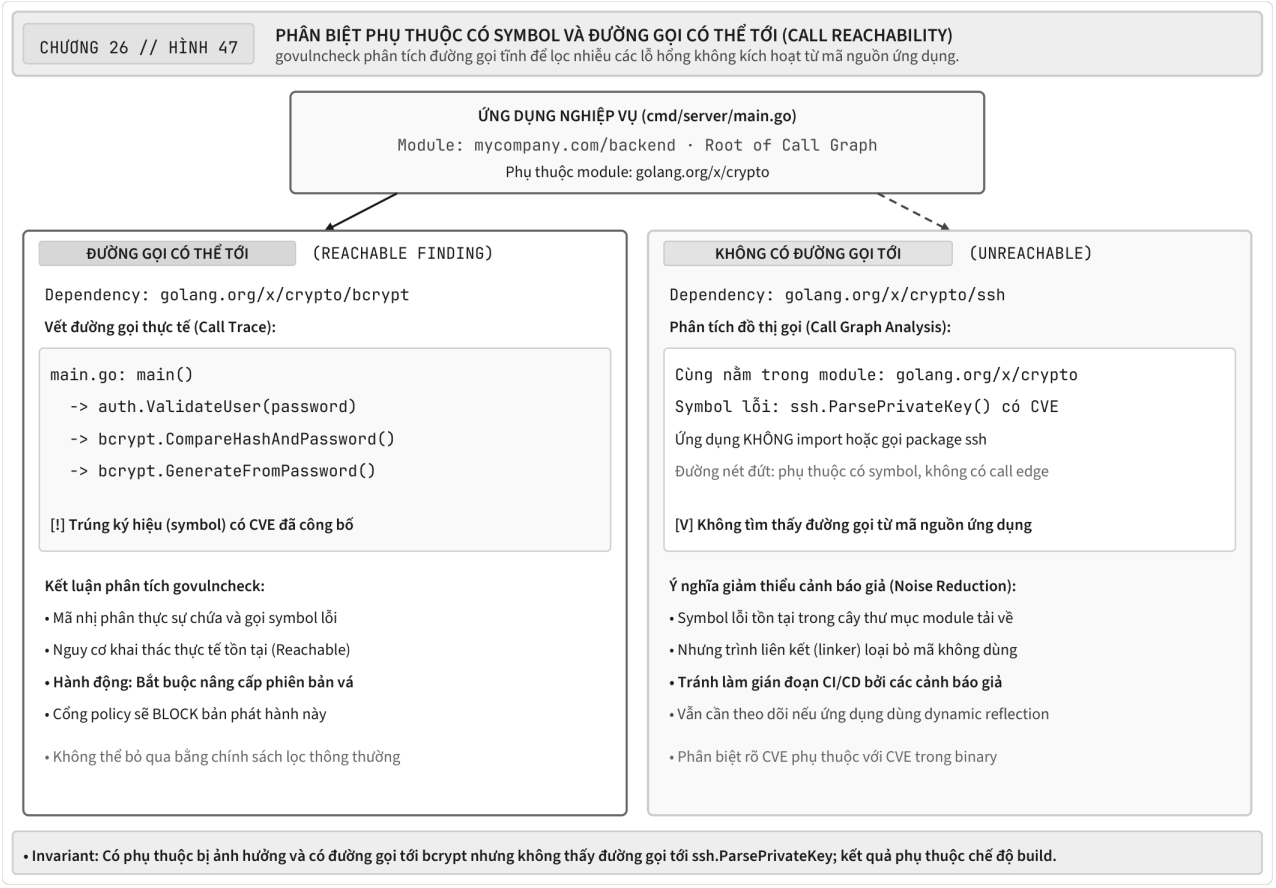
3. Quét lỗ hổng tĩnh vs Khả năng vươn tới ký hiệu (`govulncheck`)

Trong các đường ống tích hợp và triển khai liên tục (CI/CD) truyền thống, các công cụ quét vùng chứa (như Trivy, Grype, Snyk) thường đối chiếu danh sách gói phần mềm với cơ sở dữ liệu CVE. Cách tiếp cận này tạo ra một vấn nạn nghiêm trọng trong vận hành: hội chứng quá tải vì cảnh báo an ninh (**alert fatigue**).

Xét một tình huống: dự án dùng `golang.org/x/crypto`, và phiên bản đang dùng có cảnh báo lỗ hổng ở `ssh.ParsePrivateKey`, nhưng mã nguồn ứng dụng chỉ gọi `bcrypt.GenerateFromPassword`. Một công cụ quét chỉ nhìn danh mục gói phụ thuộc có thể báo lỗ hổng dù ký hiệu hàm không hề nằm trong đường dẫn gọi đã phân tích. Cảnh báo lỗ hổng và chặn phát hành là hai quyết định khác nhau; chính sách của đường ống CI/CD mới là nơi chọn điều kiện chặn. Đây không phải một CVE thật được gán cho phiên bản tùy ý.

Cơ chế phân tích đồ thị cuộc gọi của `govulncheck`

Ở chế độ mã nguồn (`source`), `govulncheck` còn phân tích khả năng vươn tới của ký hiệu, thay vì chỉ báo mô-đun có lỗ hổng. Phải đọc chế độ scan và coverage của kết quả đang dùng.



Hình 47 — Có phụ thuộc bị ảnh hưởng và tìm thấy đường gọi tới ký hiệu bị ảnh hưởng là hai mức bằng chứng khác nhau. Khả năng vươn tới theo phân tích tĩnh không chứng minh khai thác thực tế; không tìm thấy đường gọi cũng không chứng minh an toàn tuyệt đối.

Dữ liệu JSON của `govulncheck` là một luồng thông điệp (**stream message**) tuân theo lược đồ (**schema**) của phiên bản sử dụng, chứa định danh OSV và

phát hiện lỗ hổng kèm vết gọi (**trace**); không phải một đối tượng cố định chứa `Modules` và `Traces` viết hoa. Trong bài thực hành, `Severity` và `Reachable` là dữ liệu mẫu (**fixture**) đã được chuẩn hóa, không phải bóc tách trực tiếp từ đầu ra công cụ quét. Một phát hiện có vết gọi tới ký hiệu hàm chỉ là kết quả phân tích tĩnh về đường gọi có khả năng chạm tới, không chứng minh đường dẫn đó chắc chắn được kích hoạt hay khai thác thành công trong thực tế. Bộ tương thích (**adapter**) thực tế phải ghi rõ phiên bản/lược đồ, chế độ quét mã nguồn hay tệp nhị phân, độ bao phủ quét và quy tắc suy diễn chính sách từ các cảnh báo phát hiện.

Giới hạn phân tích cần lưu ý

Khả năng phân tích tĩnh của `govulncheck` tập trung chủ yếu vào mã nguồn Go. Kỹ thuật xấp xỉ đồ thị cuộc gọi tĩnh có giới hạn trước reflection, hàm assembly hay điều phối động, không phân tích các phụ thuộc liên kết động qua CGO runtime, không lần vết trong plugin tải động (`plugin.Open`), và không quét các thư viện hệ thống nhị phân của ảnh nền container (như OpenSSL hay glibc; những thành phần này đòi hỏi công cụ quét ảnh container chuyên dụng).

Thiết kế chính sách thông minh

`Reachable`, mức độ nghiêm trọng (**severity**), độ bao phủ của bộ quét và ngưỡng chấp nhận đều là dữ liệu đầu vào của chính sách. Một tổ chức có thể từ chối cảnh báo lỗ hổng mức cao hoặc nghiêm trọng nếu có thể vươn tới, tổ chức khác áp dụng ngoại lệ có thời hạn hoặc từ chối cả cảnh báo chưa chứng minh được đường gọi. Vì vậy `Reachable=false` không có nghĩa "an toàn"; trong bài thực hành nó chỉ dẫn tới trạng thái `WARN` theo chính sách minh họa và cần được xem xét lại trong chính sách thực tế.

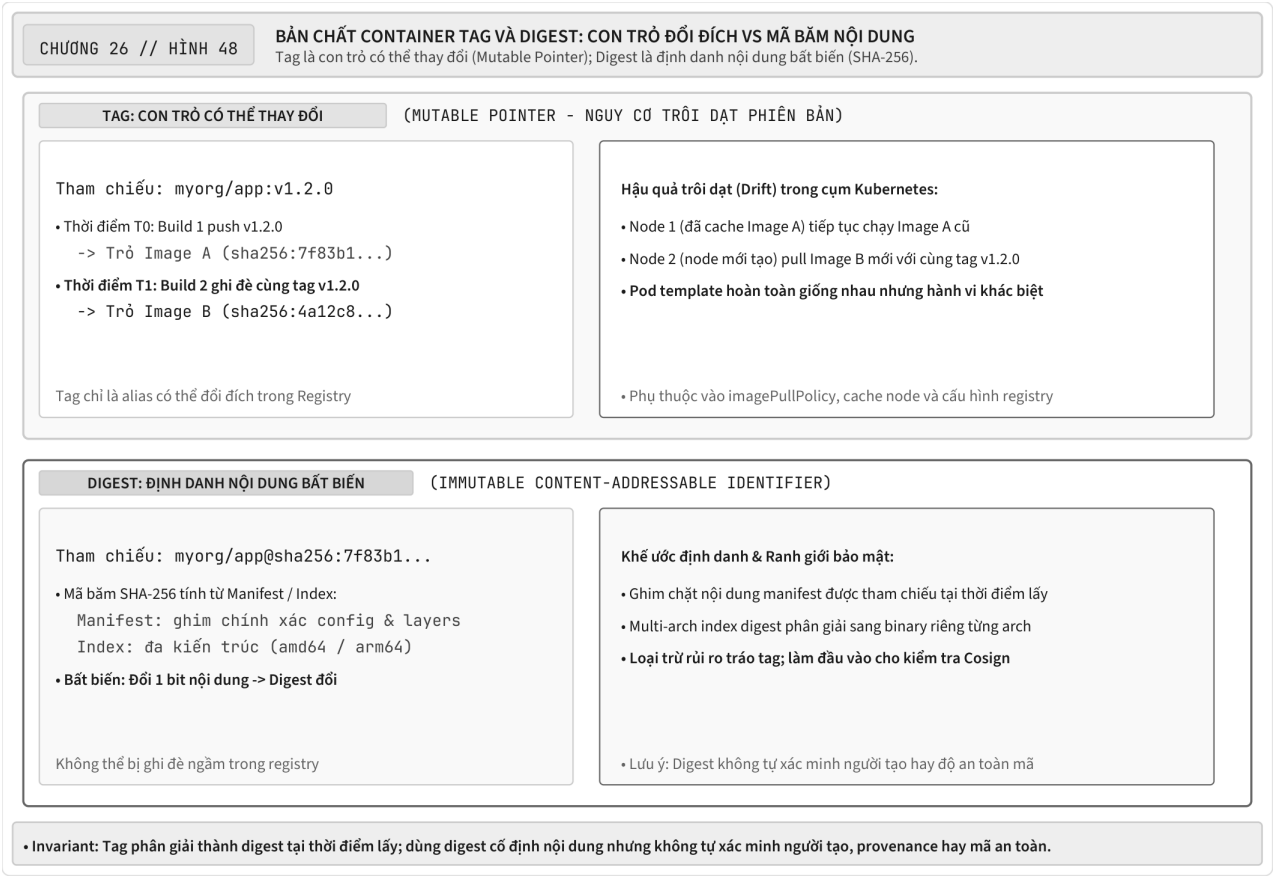
---

4. Bất biến OCI: Mã băm Digest vs Nhãn Tag có thể biến đổi

Khi ảnh container được đóng gói và đẩy lên kho lưu trữ bản kê tiêu chuẩn OCI (như Docker Hub, AWS ECR, GitHub Packages), chúng ta thường gắn nhãn định danh phiên bản như `:latest` hoặc `:v1.2.3`.

Tuy nhiên, trong quy chuẩn đóng gói vùng chứa:

> Thẻ nhãn (`tag`) là con trỏ có thể thay đổi, trong khi chuỗi băm nội dung (`digest`) là định danh bất biến đại diện cho chính xác từng byte của hiện vật xuất phẩm (`artifact`).



Hình 48 — Thẻ nhãn (`tag`) được phân giải thành mã băm (`digest`) tại một thời điểm. Dùng mã băm cố định nội dung được tham chiếu, không tự xác minh người tạo hay độ an toàn của nội dung ấy.

Nếu thẻ nhãn (`tag`) bị trỏ sang ảnh khác, một thao tác kéo ảnh (**pull**) tiếp theo có thể nhận nội dung khác dù khuôn mẫu Pod vẫn giữ nguyên chuỗi thẻ nhãn đó. Kết quả còn phụ thuộc `imagePullPolicy`, bộ nhớ đệm của nút tính toán và chính sách của kho lưu trữ. Việc ghim mã băm nội dung giúp ràng buộc chính xác dữ liệu đã xét, nhưng không tự chứng minh bên tạo dựng hay bảo đảm gói phát hành vô hại.

Chính sách của bài thực hành chọn quy tắc đầu tiên: Nguyên tắc cốt lõi: Từ chối mọi ảnh container không được định danh tường minh bằng mã băm SHA-256 dạng `sha256:<64_hex_chars>`.

---

5. Chữ ký số Sigstore/Cosign và Attestation SLSA

Làm sao chúng ta biết một ảnh container có mã băm `sha256:abc...` thực sự được tạo ra bởi quy trình CI chính thức của tổ chức chứ không phải do hacker tự biên dịch rồi đẩy lên?

Bộ công cụ Sigstore/Cosign là một lựa chọn cho việc ký và xác minh hiện vật xuất phẩm (`artifact`):

Một là, Ký không cần khóa tĩnh (**Keyless Signing**): Thay vì quản lý khóa bí mật dài hạn trên máy chủ CI, tiến trình thực thi tạo cặp khóa tạm thời cho mỗi lần ký và dùng mã xác thực OpenID Connect (OIDC) để xin chứng chỉ ngắn hạn từ Fulcio nhằm ràng buộc danh tính quy trình với khóa công khai.

Hai là, chứng chỉ ngắn hạn: Fulcio ràng buộc khóa công khai với danh tính OIDC theo chính sách của nhà cung cấp chứng chỉ (CA). `https://token.actions.githubusercontent.com` là bên phát hành (**issuer**), không phải danh tính riêng của quy trình. Bộ kiểm chứng phải xét cả danh tính chứng chỉ được phép, bên phát hành, thời hạn hiệu lực cùng bằng chứng xác minh tương ứng.

Ba là, Ký và xác minh khẳng định mã băm: Công cụ Cosign ký tải trọng chứa khẳng định (**claim**) về mã băm của ảnh; bộ kiểm chứng mặc định đối chiếu khẳng định đó cùng danh tính chứng chỉ theo chính sách. Quy trình này không đơn thuần chỉ là một lệnh gọi `ecdsa.VerifyASN1` trên chuỗi mã băm trong bài thực hành.

Bốn là, Bản chứng thực nguồn gốc SLSA (`SLSA Provenance`): Ngoài chữ ký số, đường ống CI còn tạo ra bản chứng thực xuất xứ (**provenance**) ghi rõ: mã băm commit nào, quy trình tự động nào (Builder ID) đã tạo ra gói phát hành.

---

6. Xây dựng Cổng chính sách Fail-Closed trong Go

Tại đây, toàn bộ các nguyên lý trên được hiện thực hóa vào một mô-đun Go theo mô hình chính sách kiểm chứng sư phạm.

Mô hình này không nhằm mục đích thay thế hay bao bọc toàn bộ mã nguồn của Cosign CLI hay SLSA verifier bên ngoài. Thay vào đó, nó tách bạch rõ ràng 3 khế ước giao tiếp (interfaces) cốt lõi của một hệ thống kiểm định chuỗi cung ứng hiện đại:

Khế ước giao tiếp	Nhiệm vụ kỹ thuật
`SignatureVerifier`	Chịu trách nhiệm xác thực chữ ký số mật mã của image (Cosign / Notary).
`ProvenanceVerifier`	Chịu trách nhiệm kiểm chứng xuất xứ bản build (SLSA Provenance / in-toto).
`VulnerabilityProvider`	Chịu trách nhiệm cung cấp dữ liệu lỗ hổng (govulncheck / scanner).

Khai báo các Interface và Mô hình dữ liệu chính sách

```
type Decision string

const (
    DecisionAllow Decision = "ALLOW"
    DecisionDeny  Decision = "DENY"
)

// 3 Interface cốt lõi của Cổng kiểm định chuỗi cung ứng:
type SignatureVerifier interface {
    VerifySignature(
        digest string, sig *SignatureVerification,
    ) error
}

type ProvenanceVerifier interface {
    VerifyProvenance(
        digest string, att *Attestation,
```

```
    ) error
}

type VulnerabilityProvider interface {
    GetVulnerabilities(
        digest string,
    ) ([]Vulnerability, error)
}

// Trong lab, đây là dữ liệu mẫu do test cung cấp.
type Vulnerability struct {
    ID        string `json:"id"`
    Package   string `json:"package"`
    Symbol    string `json:"symbol"`
    Severity  string `json:"severity"`
    Reachable bool   `json:"reachable"`
}
```

Metadata Xuất xứ, Chữ ký số và Bộ điều phối PolicyEngine

```
// Attestation đại diện cho metadata xuất xứ SLSA.
type Attestation struct {
    BuilderID     string `json:"builderId"`
    SubjectDigest string `json:"subjectDigest"`
}

// SignatureVerification chứa thông tin chữ ký ECDSA
// và định danh OIDC Issuer.
type SignatureVerification struct {
    PublicKey *ecdsa.PublicKey
    Signature []byte
    Issuer    string
}

type EvaluationResult struct {
    Decision   Decision `json:"decision"`
    Violations []string `json:"violations,omitempty"`
    Warnings   []string `json:"warnings,omitempty"`
}

// PolicyEngine điều phối việc kiểm định fail-closed
```

```
type PolicyEngine struct {
    TrustedBuilders []string
    TrustedIssuers  []string
    SigVerifier     SignatureVerifier
    ProvVerifier    ProvenanceVerifier
}

func NewPolicyEngine(builders, issuers []string) *PolicyEngine {
    sigV := &DefaultSignatureVerifier{
        TrustedIssuers: issuers,
    }
    provV := &DefaultProvenanceVerifier{
        TrustedBuilders: builders,
    }
    return &PolicyEngine{
        TrustedBuilders: builders,
        TrustedIssuers:  issuers,
        SigVerifier:     sigV,
        ProvVerifier:    provV,
    }
}
```

Kiểm tra định dạng OCI digest

```
// VerifyDigest chỉ kiểm tra định dạng chuỗi digest.
func VerifyDigest(digest string) error {
    if !digestRegex.MatchString(digest) {
        return errors.New(
            "invalid or mutable OCI digest: " +
                "must be sha256:64hex",
        )
    }
    return nil
}
```

Xác thực chữ ký mật mã ECDSA và OIDC Issuer

Đọc đoạn sau như một mô hình dùng dữ liệu mẫu đã tin cậy, không phải bộ xác minh danh tính thực thụ. Khóa công khai do bên gọi cung cấp; `Issuer` chỉ là chuỗi do bên gọi tự khai báo chứ chưa được ràng buộc với khóa bằng chứng chỉ số. `Attestation` cũng là cấu trúc chưa được xác thực. Một người tự tạo khóa, ký mã băm rồi sao chép bên phát hành/bên tạo dựng nằm trong danh sách trắng vẫn được mô hình chấp nhận. `TestModelAllowsSelfAssertedIdentity` giữ phản ví dụ này để ngăn việc hiểu nhầm kết quả kiểm thử đạt thành bằng chứng an toàn GitHub OIDC hay SLSA. Khi đưa vào cổng kiểm soát thực tế, phải thay các thành phần này bằng quy trình xác minh gốc tin cậy, định danh và xuất xứ đầy đủ; một phép so sánh chuỗi đơn thuần không thể lấp đầy lỗ hổng bảo mật này.

```
func (v *DefaultSignatureVerifier) VerifySignature(
    digest string,
    sig *SignatureVerification,
) error {
    if sig == nil || sig.PublicKey == nil {
        return errors.New(
            "missing cryptographic signature or public key",
        )
    }

    // So nhãn issuer trong fixture, chưa xác thực danh tính.
    trustedIssuer := false
    for _, ti := range v.TrustedIssuers {
        if sig.Issuer == ti {
            trustedIssuer = true
            break
        }
    }
    if !trustedIssuer {
        return fmt.Errorf(
            "untrusted OIDC signature issuer: %s",
```

```
            sig.Issuer,
        )
    }

    // Kiểm tra chữ ký bằng khóa được cung cấp.
    hash := sha256.Sum256([]byte(digest))
    if len(sig.Signature) == 0 {
        return errors.New("empty signature payload")
    }

    valid := ecdsa.VerifyASN1(
        sig.PublicKey, hash[:], sig.Signature,
    )
    if !valid {
        return errors.New(
            "signature verification failed: " +
                "cryptographic signature mismatch",
        )
    }
    return nil
}
```

Thực thi đánh giá toàn diện (Fail-Closed Evaluation)

`PolicyEngine` gọi verifier đã cấu hình, không tự bỏ qua interface để kiểm tra lại một cách khác. Thiếu verifier là lỗi, không phải dấu hiệu được phép đi tiếp. Phương thức chữ ký trong source có dạng:

```
func (e *PolicyEngine) VerifySignature(
    digest string,
    sig *SignatureVerification,
) error {
    if e.SigVerifier == nil {
        return errors.New("no signature verifier configured")
    }
    return e.SigVerifier.VerifySignature(digest, sig)
}
```

`VerifyProvenance` cũng ủy quyền và từ chối nếu chưa cấu hình verifier. `DefaultProvenanceVerifier` của lab chỉ so các trường fixture; các thông báo lỗi thật là `missing SLSA provenance attestation`, `provenance subject digest does not match artifact digest`

và `untrusted builder identity: %s`. Không dịch các chuỗi này khi đối chiếu output.

Hàm `Evaluate` dưới đây giữ đúng thứ tự và các nhánh trả về của `policy.go`. Hãy dự đoán: nếu chữ ký thất bại, provenance verifier có được gọi không? Tại sao một lần xác minh provenance thành công không thể bù cho chữ ký đã thất bại?

```
func (e *PolicyEngine) Evaluate(
    artifactDigest string,
    sig *SignatureVerification,
    att *Attestation,
    vulns []Vulnerability,
) EvaluationResult {
    res := EvaluationResult{Decision: DecisionAllow}
    if err := VerifyDigest(artifactDigest); err != nil {
        res.Decision = DecisionDeny
        res.Violations = append(res.Violations, err.Error())
        return res
    }
    err := e.VerifySignature(artifactDigest, sig)
    if err != nil {
        res.Decision = DecisionDeny
        res.Violations = append(res.Violations, err.Error())
        return res
    }
    err = e.VerifyProvenance(artifactDigest, att)
```

```
    if err != nil {
        res.Decision = DecisionDeny
        res.Violations = append(res.Violations, err.Error())
        return res
    }
    for _, v := range vulns {
        isCrit := strings.EqualFold(v.Severity, "CRITICAL")
        isHigh := strings.EqualFold(v.Severity, "HIGH")

        if isCrit || isHigh {
            if v.Reachable {
                // Fixture reachable: từ chối.
                res.Decision = DecisionDeny
                res.Violations = append(
                    res.Violations,
                    fmt.Sprintf(
                        "reachable %s vulnerability %s " +
                            "in %s (symbol: %s)",
```

```
                        v.Severity, v.ID, v.Package, v.Symbol,
                    ),
                )
            } else {
                // Fixture không reachable: cảnh báo.
                res.Warnings = append(
                    res.Warnings,
                    fmt.Sprintf(
                        "uncalled %s vulnerability %s " +
                            "in %s: tolerated under policy",
                        v.Severity, v.ID, v.Package,
                    ),
                )
            }
        }
    }
    return res
}
```

`TestPolicyGateStopsOnSignatureFailure` dùng verifier chữ ký trả lỗi và một provenance verifier ghi nhận việc được gọi. Test yêu cầu `DENY` và không được gọi verifier sau. Tạm bỏ `return res` ở nhánh lỗi chữ ký rồi chạy test: kết quả cuối có thể vẫn là `DENY`, nhưng contract dừng tuần tự đã bị phá. Chỉ assert quyết định cuối không bắt được lỗi đó.

7. Kiểm chứng Lab thực tế (`labs/part26-supply-chain-gate`)

Mã nguồn hoàn chỉnh của bài thực hành nằm tại thư mục `labs/part26-supply-chain-gate` (phân loại mức kiểm chứng: kiểm thử đơn vị đối với logic chính sách cổng đóng an toàn khi lỗi và chữ ký số ECDSA P-256 nội bộ, chứng minh khế ước an ninh của 7 kịch bản kiểm định mà không phụ thuộc vào hạ tầng mạng Sigstore bên ngoài). Khi chạy kiểm thử với cờ kiểm tra xung đột dữ liệu:

```
go test -v -race ./...
```

Transcript lịch sử sau ghi tên bảy test gốc; thời lượng không phải số đo cho lần chạy hiện tại và không chứng minh Sigstore hay SLSA đã được kiểm chứng trực tiếp:

```
=== RUN   TestPolicyGateAllowedWithValidSignature
--- PASS: TestPolicyGateAllowedWithValidSignature (0.00s)
=== RUN   TestModelAllowsSelfAssertedIdentity
--- PASS: TestModelAllowsSelfAssertedIdentity (0.00s)
=== RUN   TestPolicyGateDenyUnsignedImage
--- PASS: TestPolicyGateDenyUnsignedImage (0.00s)
=== RUN   TestPolicyGateDenyReachableVulnerability
--- PASS: TestPolicyGateDenyReachableVulnerability (0.00s)
=== RUN   TestPolicyGateAllowUncalledVulnerability
--- PASS: TestPolicyGateAllowUncalledVulnerability (0.00s)
=== RUN   TestPolicyGateFailClosedOnMutableTag
--- PASS: TestPolicyGateFailClosedOnMutableTag (0.00s)
=== RUN   TestPolicyGateStopsOnSignatureFailure
--- PASS: TestPolicyGateStopsOnSignatureFailure (0.00s)
PASS
ok      part26-supply-chain-gate   2.128s
```

Phân tích các kịch bản kiểm thử

Kịch bản fixture	Điều kiểm tra trong model	Quyết định (`Decision`)
Input được model chấp nhận	ECDSA tự tạo trong test; chuỗi issuer/builder khớp; không truyền finding. Không gọi GitHub OIDC hay scanner.	**ALLOW** theo policy model
Phản ví dụ tự khai danh tính (`TestModel...`)	Model chấp nhận key tự tạo và issuer/builder tự khai; chứng minh model chưa xác thực `OIDC`	`ALLOW` (nhấn mạnh ranh giới model)
Image không có chữ ký	Bị chặn đứng ngay tại Cửa 2	`DENY`
Lỗ hổng nghiêm trọng được fixture đánh dấu `Reachable = true`	Test cung cấp cờ này với tên `ssh.ParsePrivateKey`; không chạy scanner hay chứng minh ứng dụng gọi hàm	`DENY` theo policy model
Finding không được fixture đánh dấu reachable (`Reachable = false`)	`http2.Server.ServeConn` là tên symbol minh họa, không phải bằng chứng quan sát mọi đường chạy	`ALLOW` (kèm cảnh báo trong `Warnings`, theo policy lab)
Cố tình dùng tag thay vì digest	Truyền vào chuỗi `my-registry.io/app:v1.2.0`	Bị chặn ngay từ Cửa 1 → `DENY`
Dừng trước provenance sau lỗi chữ ký	Khi chữ ký sai, cổng dừng ngay tại Cửa 2 và không kiểm tra tiếp Cửa 3	`DENY` (fail-closed tuần tự)

8. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Triển khai bằng Docker tag (`:latest` hoặc `:v1.0.0`) thay vì sha256 digest.	Bị tấn công tráo đổi container image khi registry bị thỏa hiệp; pod scale up chạy phiên bản khác nhau.	Ghim (pin) mã băm bất biến `sha256:<hex>` trong Kubernetes Pod Spec.
Nhận public key và issuer do caller tự khai mà coi là identity đã xác thực.	Caller tự ký rồi khai issuer được phép; model vẫn có thể chấp nhận.	Dùng verifier thật kiểm tra trust roots, certificate identity/issuer và claim digest; so chuỗi không đủ.
Thiết kế Cổng dạng Fail-Open: Bỏ qua kiểm tra khi mạng tới OIDC/Registry bị timeout.	Khi mạng gặp sự cố, hệ thống tự động cho phép mọi image chưa được kiểm chứng đi thẳng vào production.	Luôn áp dụng nguyên tắc Fail-Closed: Bất kỳ lỗi mạng hay timeout nào cũng phải quy về `DENY`.
Bỏ `go.sum` vì xem nó là file rác.	Mất checksum đã ghi nhận trong project; không tự đồng nghĩa checksum database bị tắt.	Commit checksum khi có dependency; hiểu `GOSUMDB`/`GOPRIVATE`/`replace`. `go mod verify` kiểm tra cache, không quét mã độc.

---

9. Bài tập thực hành thiết kế Cổng bảo vệ

Thử thách 1: Ghim Base Image chuẩn (Golden Image Pinning)

Yêu cầu: Viết hàm kiểm tra một tham chiếu ảnh đã được tách từ lệnh `FROM`, chẳng hạn `registry.example/base@sha256:...`. Chỉ chấp nhận repository có trong catalog và digest hợp lệ khớp giá trị đã duyệt. Test phải từ chối tag, digest sai định dạng, repository lạ và chuỗi có nhiều dấu `@`. Bài tập không yêu cầu viết parser Dockerfile: stage alias, `ARG`, `--platform` và ảnh `scratch` cần chính sách riêng khi tích hợp thực tế.

Thử thách 2: Cơ chế Timeout Fail-Closed (Network Timeout Fallback)

Yêu cầu: Nhận một callback xác minh có hỗ trợ context. Thiếu callback, context đã hủy, lỗi xác minh hoặc callback trả `nil` sau khi context bị hủy đều phải dẫn tới `DENY`. Đặt ngân sách 2 giây cho callback; không tuyên bố context có thể cưỡng chế dừng một hàm không hợp tác. Viết test bằng callback cục bộ trước; các test này không gọi Rekor hay Fulcio.

---

10. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Xác thực Golden Base Image

```
type GoldenCatalog map[string]string

func VerifyBaseImage(
    imageRef string,
    catalog GoldenCatalog,
) (bool, error) {
    parts := strings.Split(imageRef, "@")
    if len(parts) != 2 || parts[0] == "" {
        return false, fmt.Errorf(
            "image must be pinned with digest: repo@sha256:hex",
        )
    }

    repo := parts[0]
    digest := parts[1]
    if err := VerifyDigest(digest); err != nil {
        return false, err
    }
```

```
    expectedDigest, exists := catalog[repo]
    if !exists {
        return false, fmt.Errorf(
            "untrusted base image repository: %s", repo,
        )
    }

    if digest != expectedDigest {
        return false, fmt.Errorf(
            "base image digest mismatch: expected %s, got %s",
            expectedDigest, digest,
        )
    }

    return true, nil
}
```

Lời giải Thử thách 2: Xử lý Timeout theo nguyên tắc Fail-Closed

```
func SafeRemoteVerify(
    ctx context.Context,
    verifyFunc func(context.Context) error,
) EvaluationResult {
    // Callback phải hợp tác với cancellation.
    timeoutCtx, cancel := context.WithTimeout(
        ctx, 2*time.Second,
    )
    defer cancel()

    if verifyFunc == nil {
        return deniedRemoteVerification(
            fmt.Errorf("no remote verifier configured"),
        )
    }
    if err := timeoutCtx.Err(); err != nil {
        return deniedRemoteVerification(err)
    }
    err := verifyFunc(timeoutCtx)
```

```
    if err == nil {
        err = timeoutCtx.Err()
    }
    if err != nil {
        return deniedRemoteVerification(err)
    }

    return EvaluationResult{Decision: DecisionAllow}
}

func deniedRemoteVerification(err error) EvaluationResult {
    return EvaluationResult{
        Decision: DecisionDeny,
        Violations: []string{
            fmt.Sprintf("fail-closed: aborted: %v", err),
        },
    }
}
```

Hai lời giải có source chạy được trong `contract_examples.go`. `TestBaseImageContract` kiểm tra cả catalog có digest sai; `TestRemoteVerificationRejectsNilAfterCancellation` cố tình

cho callback trả `nil` sau cancellation để bắt đường cho phép sai. Không assert một thời lượng cố định và không dùng goroutine bỏ mặc callback để tạo ảo giác có thể cưỡng chế timeout. Context chỉ thông báo hạn chót; nếu callback không trả về, wrapper này cũng không trả về. Kiểm tra `Err()` sau callback loại kết quả thành công khi cancellation đã được quan sát tại điểm kiểm tra, không tạo một thao tác nguyên tử với mọi cancellation xảy ra về sau.

Nguyên tắc đóng an toàn từ chối gói phát hành khi thiếu chứng cứ mà chính sách yêu cầu, kể cả lúc bộ kiểm chứng gặp sự cố gián đoạn dịch vụ. Cơ chế này không bảo đảm mọi gói phát hành được chấp nhận đều vô hại: gốc tin cậy, bên ký được phép, nội dung và chính quy tắc kiểm soát vẫn có thể sai sót hoặc bị xâm phạm. Tính an toàn và tính sẵn sàng của cổng kiểm soát là hai bài toán cần được kiểm chứng độc lập.

Quy trình kiểm soát chuỗi cung ứng phần mềm giúp đối chiếu tính toàn vẹn của mã nguồn, xuất xứ bản dựng và chữ ký số của ảnh container với chính sách đã chọn. Bằng chứng đó không chứng minh tệp nhị phân hoàn toàn vô hại: mã nguồn, bên ký hay gốc tin cậy vẫn có thể bị xâm phạm.

Khi vùng chứa thực sự vận hành, hệ thống cần bổ sung bằng chứng về hành vi lúc chạy: tiến trình nào mở kết nối mạng, đọc tệp cấu hình hay thực hiện các thao tác bất thường trong nhân hệ điều hành? Trong Chương 27, chúng ta sử dụng công nghệ eBPF kết hợp với Go để quan sát các sự kiện tại những điểm móc nối được chỉ định trong nhân hệ điều hành Linux. Đây không phải là khả năng bắt trọn mọi hành vi hay cơ chế quan sát miễn phí. Phạm vi điểm móc nối, cửa sổ thu thập, sự kiện bị đánh rơi và chi phí xử lý đều giới hạn phạm vi kết luận; kỹ sư bắt buộc phải đo lường độ trễ phụ trội với khối lượng tải thực tế trước khi triển khai trên môi trường vận hành.

Chương 27 — Quan sát Linux từ kernel bằng eBPF và Go

Cho đến thời điểm này của cuốn sách, chúng ta đã xây dựng các công cụ giám sát dựa trên ba trụ cột Observability truyền thống: Metrics (Prometheus), Logs và Tracing (OpenTelemetry).

Cơ chế thu thập dữ liệu quan sát (**instrumentation**) đặt trực tiếp trong ứng dụng đòi hỏi mã nguồn ứng dụng chủ động tạo chỉ số đo lường (**metrics**) và đoạn vết (**spans**) mang ngữ nghĩa nghiệp vụ. Nhưng chỉ số, nhật ký (**logs**) và vết thực thi (**traces**) không chỉ đến từ một nguồn: hệ điều hành, máy chủ ủy quyền (**proxy**) và các tầng hạ tầng cũng cung cấp tín hiệu độc lập. Câu hỏi đặt ra là: điểm quan sát nào vẫn phát hiện được hành vi bất thường khi bản thân ứng dụng không chủ động báo cáo?

Xét một kịch bản giả định: một tiến trình web bị khai thác lỗ hổng thực thi mã từ xa (RCE), mở vỏ lệnh `/bin/sh`, tải dữ liệu độc hại vào thư mục `/tmp` rồi thực thi mà hoàn toàn không phát sinh đoạn vết hay chỉ số nghiệp vụ nào từ mã ứng dụng. Cơ chế quan sát nội tại của ứng dụng sẽ bỏ lọt dấu vết đó; trong khi nhật ký hệ điều hành, máy chủ ủy quyền hoặc nhật ký kiểm toán (**audit log**) vẫn có thể ghi nhận. Do đó, chúng ta bổ sung một điểm quan sát ở tầng nhân (**kernel**) tại các điểm móc (**hooks**) được chỉ định, thay vì ngộ nhận eBPF có thể thấu suốt mọi hành vi của toàn hệ thống.

eBPF cho phép đặt điểm quan sát trực tiếp ở tầng nhân cho các điểm móc mà chính sách an ninh và phiên bản nhân Linux hỗ trợ; công nghệ này không nhìn thấy toàn bộ mọi hành vi và không thể thay thế nhật ký kiểm toán hay các cơ chế kiểm soát truy cập. Trước đây, việc can thiệp vào nhân hệ điều hành thường đòi hỏi mô-đun nhân Linux (LKM) viết bằng C, với bán kính ảnh hưởng sự cố (**blast radius**) rất lớn khi xảy ra lỗi nghiêm trọng. eBPF kết hợp với thư viện `cilium/ebpf` cho phép nạp mã trung gian (**bytecode**) thông qua bộ thẩm định nhân (**kernel verifier**), tuy nhiên khả năng gắn kết, đặc quyền thực thi và chi phí tài nguyên phụ trội (**overhead**) vẫn bắt buộc phải được kiểm chứng trên môi trường thực tế.

1. Mental Model: Chuỗi luân chuyển sự kiện Kernel - Go Userspace

Mô hình trung tâm cần tách hai thời điểm. Verifier kiểm tra chương trình khi nạp, không chạy lại như một bước trong đường đi của từng sự kiện:

Thời điểm	Đường xử lý của lab	Ranh giới
Nạp và gắn chương trình	Trình nạp nạp tệp đối tượng; bộ kiểm định nhân kiểm tra mã bytecode; trình nạp gắn chương trình vào điểm vết (`tracepoint`).	Chấp nhận chương trình không chứng minh policy quan sát đầy đủ hoặc overhead phù hợp.
Một lời gọi `execve` đi qua hook	Chương trình lấy trường sự kiện, reserve/submit record vào ring buffer; Go đọc record, giải mã ABI rồi áp dụng heuristic.	Record có thể mất; sự kiện đầu vào syscall không chứng minh exec thành công.

Với điểm móc `sys_enter_execve` trong bài lab, bản ghi dữ liệu chứa định danh tiến trình (PID), định danh người dùng (UID), định danh nhóm (GID), tên tiến trình gọi và đường dẫn tệp thực thi đọc từ đối số. Bộ đọc của `cilium/ebpf` ở phiên bản được ghim có thể lấy dữ liệu đã sẵn sàng mà không cần chờ qua cơ chế `epoll`. Thứ tự byte nhỏ (**little-endian**) là quy ước giao diện nhị phân (ABI) của tệp đối tượng `bpfel` đang biên dịch, không phải quy tắc cố định cho mọi nhân hệ điều hành hay chương trình eBPF. Ứng dụng Go chỉ phân tích những bản ghi thực sự nhận được; việc không nhận được bản ghi không đồng nghĩa hệ thống hoàn toàn không có hành vi khả nghi.

---

2. Kernel Verifier: Kiểm tra trước khi nạp

Tại sao nhân Linux (`Linux Kernel`) lại cho phép mã do người dùng viết chạy trực tiếp bên trong không gian bộ nhớ của nhân?

Câu trả lời nằm ở bộ kiểm định nhân (`kernel verifier`). Trước khi bất kỳ chương trình eBPF nào được nạp vào nhân Linux thông qua lời gọi hệ thống `SYS_BPF`, bộ kiểm định (`Verifier`) sẽ phân tích tĩnh toàn bộ mã trung gian bytecode và thực thi các quy tắc kiểm tra nghiêm ngặt:

Tiêu chuẩn an toàn Verifier	Cơ chế kiểm tra	Mục đích bảo vệ Kernel
Kiểm tra đường thực thi hữu hạn	Phân tích CFG, state và các loop mà kernel/version cho phép chứng minh an toàn.	Hạn chế đường chạy không an toàn; không biến mọi program được nạp thành không có overhead.
Kiểm soát truy cập bộ nhớ	Bắt buộc kiểm tra biên; truy cập userspace qua `bpf_probe_read_user_str()`.	Chống rò rỉ hoặc ghi đè trái phép lên không gian nhớ kernel.
Giới hạn stack của mô hình BPF đang xét	Tài liệu verifier mô tả frame 512 byte; call chain, program type và tính năng kernel còn có kiểm tra riêng.	Không dùng số này làm tổng stack budget cho mọi call chain hay version.
Theo dõi thanh ghi BPF và FP	`R10` là frame pointer chỉ đọc; verifier theo dõi kiểu và trạng thái thanh ghi BPF.	Đây là thanh ghi của máy BPF, không phải lời hứa giữ nguyên mọi thanh ghi vật lý của CPU.

> **Dừng để dự đoán:** Ranh giới giữa trình biên dịch C và bộ kiểm định nhân (**Kernel Verifier**). Xét đoạn minh họa tra cứu và cập nhật bộ đếm trong map dưới đây:

```
struct event *val = bpf_map_lookup_elem(&counters, &key);
val->count++;
```

Với các khai báo hỗ trợ phù hợp, Clang có thể biên dịch đoạn C này; nhưng đường truy cập con trỏ chưa kiểm tra kết quả tra cứu không đủ bằng chứng cho BPF verifier. Vì sao verifier đòi hỏi thêm một nhánh kiểm tra, và nhánh đó phải chứng minh điều gì trước `val->count++`?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Với các khai báo kiểu và map phù hợp, đoạn C có thể biên dịch dù chưa kiểm tra kết quả tra cứu. Clang kiểm tra và tối ưu mã theo mô hình ngôn ngữ C; việc biên dịch thành công không cung cấp bằng chứng mà BPF verifier yêu cầu về con trỏ trả về từ helper. `bpf_map_lookup_elem` có thể trả `NULL` khi không tìm thấy phần tử. Verifier theo dõi kết quả này dưới dạng `PTR_TO_MAP_VALUE_OR_NULL`, chưa cho phép truy cập như một con trỏ map value chắc chắn hợp lệ.

Do đó, bộ kiểm định nhân (`Kernel Verifier`) thực hiện phân tích đường đi trừu tượng và theo dõi trạng thái thanh ghi BPF chứa kết quả trả về dưới dạng con trỏ có thể mang giá trị `NULL`. Khi thanh ghi chưa được chứng minh an toàn, mọi chỉ thị đọc hoặc ghi bộ nhớ thông qua thanh ghi đó đều bị Verifier chặn đứng và từ chối nạp vì lỗi truy cập bộ nhớ không hợp lệ. Để vượt qua rào cản này, lập trình viên bắt buộc phải chèn một nhánh kiểm tra điều kiện tường minh ngay sau khi tra cứu:

```
struct event *val = bpf_map_lookup_elem(&counters, &key);
if (!val) {
    return 0; // Thoát an toàn nếu không tìm thấy key
}
val->count++;
```

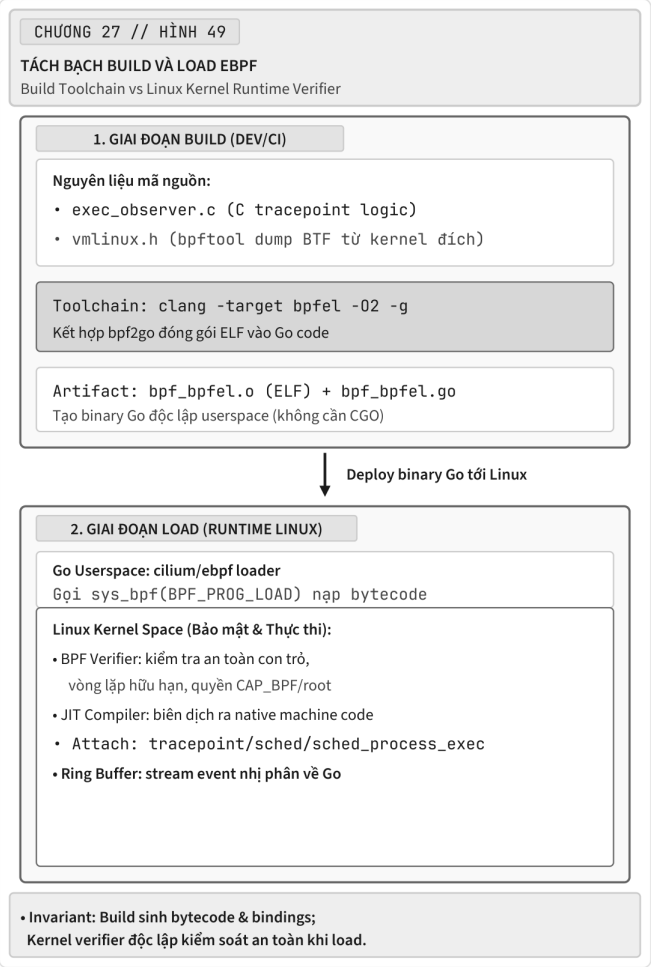
Nhánh kiểm tra `if (!val)` cung cấp cho Verifier bằng chứng cần thiết để chứng minh rằng trên mọi nhánh thực thi đi tới câu lệnh `val->count++`, con trỏ trong thanh ghi chắc chắn khác `NULL`. Tuy nhiên, việc vượt qua kiểm tra con trỏ `NULL` chỉ giải quyết riêng lỗi truy cập con trỏ chưa kiểm chứng; nó hoàn toàn không bảo đảm chương trình sẽ vượt qua tất cả các tiêu chuẩn kiểm định khác của bộ kiểm định (`Verifier`), chẳng hạn như giới hạn kích thước ngăn xếp (`stack`), tính hữu hạn của vòng lặp hay tính hợp lệ của các vùng đệm bộ nhớ truyền vào hàm trợ giúp.

---

3. Kiến trúc CGO-Free và Trình sinh mã `bpf2go`

Khi lập trình eBPF với Go, nhiều người lầm tưởng rằng bắt buộc phải cài đặt trình biên dịch Clang/LLVM cồng kềnh trên máy chủ sản xuất hoặc phải kích hoạt CGO.

Thư viện `github.com/cilium/ebpf` cung cấp đường nạp và quản lý object eBPF từ Go không phụ thuộc cgo ở phía userspace:



Hình 49 — Clang và bpf2go thuộc bộ công cụ biên dịch. Tệp nhị phân Go mang tệp đối tượng BPF đến môi trường chạy; nhân hệ điều hành vẫn thẩm định chương trình, quyền hạn và tính năng hỗ trợ trước khi nạp hoặc gắn kết thành công.

Quy trình biên dịch dưới đây đòi hỏi hệ điều hành Linux hỗ trợ định dạng BTF, công cụ `bpftool`, trình biên dịch Clang hỗ trợ đích biên dịch BPF cùng các tệp tiêu đề của thư viện `libbpf`. Thực thi lệnh từ thư mục bài thực hành; tệp tiêu đề `vmlinux.h` cung cấp các kiểu điểm theo dõi (**tracepoint**) mà tệp tiêu đề chuẩn của nhân không khai báo sẵn. Tệp tiêu đề này được trích xuất trực tiếp từ nhân hệ điều hành mục tiêu, hoàn toàn không phải mã nguồn Go viết thủ công:

```
bpftool btf dump file /sys/kernel/btf/vmlinux \
    format c > bpf/vmlinux.h
```

Chỉ thị trong `gen.go` dùng `bpf2go` của module đã ghim:

```
go run github.com/cilium/ebpf/cmd/bpf2go \
    -target bpfel -cc clang \
    bpf bpf/exec_observer.c -- \
    -I./bpf -O2 -g
```

Một là, giai đoạn phát triển: Kỹ sư viết mã C eBPF, sau đó chạy `go generate` với `bpf2go` và Clang trên máy phát triển để biên dịch mã nguồn C thành bytecode eBPF (định dạng ELF) cùng các tệp Go bindings tương ứng.

Hai là, tự động nhúng Bytecode: Công cụ `bpf2go` tự động sinh ra tệp Go chứa mã trung gian bytecode ELF đã được nhúng thẳng vào binary thông qua tính năng `//go:embed`.

Ba là, phía Go có thể nạp tệp đối tượng eBPF đã biên dịch sẵn thông qua các lời gọi hệ thống (**syscall**) của Linux mà không phụ thuộc cgo. Điều đó không xóa bỏ sự phụ thuộc vào môi trường máy chủ: nhân Linux phải hỗ trợ tính năng và điểm móc cần dùng, bên gọi phải có đặc quyền phù hợp, đồng thời dữ liệu BTF cùng các tài nguyên lúc biên dịch phải khớp với cách ứng dụng được đóng gói. Việc không cần môi trường thực thi C cho bộ nạp thuần Go không đồng nghĩa ứng dụng chạy được mà không cần các điều kiện tiên quyết trên máy chủ.

Bốn là, bài kiểm thử Go dùng nguồn đọc giả lập (`RecordReader`) để kiểm tra logic giải mã và điều phối kênh truyền mà không cần đặc quyền nạp BPF vào nhân. Bài lab mang tính mô hình hóa; việc tích hợp tệp đối tượng thực tế, bộ tương thích vùng đệm vòng và phân quyền trên máy chủ cần được cấu hình và kiểm chứng độc lập. Đối tượng `CollectionSpec` dựng trong kiểm thử không thay thế cho tệp ELF do trình biên dịch sinh ra.

---

4. Kênh truyền dữ liệu tốc độ cao: BPF Ring Buffer

Để chuyển dữ liệu từ tầng nhân lên không gian người dùng (**userspace**) trong Go, eBPF cung cấp cấu trúc vùng đệm vòng BPF (`BPF_MAP_TYPE_RINGBUF`, tức **BPF ring buffer**). Khác với mảng sự kiện hiệu năng (**perf event array**) thường phân chia vùng đệm riêng theo từng lõi CPU, vùng đệm vòng sử dụng vùng nhớ chia sẻ và bảo toàn thứ tự giữ chỗ bộ nhớ giữa các tiến trình tạo sự kiện. Cơ chế này đặc biệt hiệu quả cho các sự kiện mang tính tuần tự như fork, exec hay exit; tuy nhiên nó không biến bản ghi thành một đồng hồ toàn cục chính xác, cũng như không tự động xác định được toàn bộ trật tự nhân quả giữa các sự kiện xảy ra đồng thời trên nhiều lõi CPU.

Bộ đọc tại không gian người dùng ánh xạ vùng đệm vòng bằng lời gọi `mmap` để lấy các bản ghi; không cần một lời gọi hệ thống riêng lẻ cho từng bản ghi, nhưng vẫn phát sinh chi phí đồng bộ hóa, cơ chế đánh thức (**polling/wakeup**) và giải mã dữ liệu. Trong bài lab, chương trình giữ chỗ trước bộ nhớ trong vùng đệm rồi mới xác nhận ghi. Thao tác giữ chỗ có thể thất bại nếu vùng đệm bị đầy hoặc gặp nghẽn luồng xử lý; khi đó chương trình sẽ hủy bản ghi. Cần chủ động đếm số lượng bản ghi bị thất thoát nếu chính sách giám sát đòi hỏi phát hiện tín hiệu bị mất, tránh ngộ nhận việc bỏ qua bản ghi là hệ thống hoàn toàn ổn định.

---

5. Viết mã Kernel: Bắt sự kiện `sys_enter_execve`

Dưới đây là phần khai báo cấu trúc sự kiện và bản đồ Ring Buffer trong C eBPF (`labs/part27-ebpf-observer/bpf/exec_observer.c`):

```
// +build ignore
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

char __license[] SEC("license") = "Dual MIT/GPL";

struct exec_event {
    __u32 pid;
    __u32 uid;
    __u32 gid;
    char  comm[16];
    char  filename[128];
};

_Static_assert(sizeof(struct exec_event) == 156,
               "event ABI mismatch");

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024); // 256KB buffer
} events SEC(".maps");
```

Tiếp theo là hàm móc vào tracepoint `sys_enter_execve` để bắt thông tin mỗi khi có lời gọi thực thi nhị phân:

```
SEC("tracepoint/syscalls/sys_enter_execve")
int trace_execve(struct trace_event_raw_sys_enter *ctx) {
    struct exec_event *event;

    // Cấp phát trực tiếp trong ringbuffer
    event = bpf_ringbuf_reserve(
        &events, sizeof(struct exec_event), 0
    );
    if (!event) {
        return 0; // Buffer đầy hoặc bộ nhớ bận
    }

    // Xóa toàn record, kể cả phần đuôi chuỗi chưa được ghi
    __builtin_memset(event, 0, sizeof(*event));
    __u64 pid_tgid = bpf_get_current_pid_tgid();
    event->pid = (__u32)(pid_tgid >> 32);

    __u64 uid_gid = bpf_get_current_uid_gid();
    event->uid = (__u32)(uid_gid);
    event->gid = (__u32)(uid_gid >> 32);
```

```
    // Đọc tên của tiến trình gọi (calling task)
    if (bpf_get_current_comm(
        &event->comm, sizeof(event->comm)
    ) < 0) {
        bpf_ringbuf_discard(event, 0);
        return 0;
    }

    // Đọc đường dẫn file nhị phân đích từ tham số syscall
    const char *fn = (const char *)ctx->args[0];
    if (bpf_probe_read_user_str(
        &event->filename, sizeof(event->filename), fn
    ) < 0) {
        bpf_ringbuf_discard(event, 0);
        return 0;
    }

    // Gửi sự kiện lên Userspace
    bpf_ringbuf_submit(event, 0);
    return 0;
}
```

Đặc điểm kỹ thuật trong mã nguồn C eBPF

Điểm móc `sys_enter_execve` chỉ ghi nhận thời điểm lệnh gọi hệ thống bắt đầu được phát đi, hoàn toàn không chứng minh tiến trình thực thi thành công sau đó. Các lỗi không tìm thấy tệp hoặc thiếu quyền truy cập hoàn toàn có thể phát sinh ở những bước tiếp theo. Chương trình tại điểm móc có cơ hội khởi tạo bản ghi, nhưng các thao tác đặt chỗ bộ nhớ, đọc đối số và chuyển giao bản ghi vẫn có thể thất bại; hệ thống không cam kết mọi nỗ lực thực thi đều được ghi nhận trọn vẹn.

Bản ghi được cấp phát bộ nhớ toàn phần trước khi điền dữ liệu vào từng trường. Nếu hàm trợ giúp gặp lỗi khi đọc tên hoặc đường dẫn, chương trình sẽ chủ động hủy bỏ bản ghi thay vì xuất bản dữ liệu khiếm khuyết; nếu bộ đệm chuỗi quá ngắn, hàm trợ giúp có thể trả về chuỗi ký tự bị cắt ngắn. Bài thực hành hiện chưa thiết lập bộ đếm thất thoát hoặc cờ báo cắt ngắn. Định danh tiến trình ở đây là giá trị do hàm trợ giúp của nhân trả về, không bảo đảm trùng khớp với định danh nhìn thấy từ bên trong mọi không gian tên

của vùng chứa. Mốc thời gian của Go là thời điểm giải mã dữ liệu, không phải thời điểm nhân bắt đầu thực thi lệnh gọi hệ thống.

Hàm trợ giúp `bpf_get_current_comm` tại thời điểm này phản ánh tên của tiến trình đang phát lệnh gọi (như `bash`, `python` hoặc `containerd`), trong khi đường dẫn tệp nhị phân đích phải được trích xuất từ tham số đầu vào. Để phân giải định danh tiến trình tương thích với không gian người dùng, mã nguồn thực hiện dịch phải 32 bit từ giá trị 64-bit của hàm `bpf_get_current_pid_tgid()`, bởi trong nhân Linux định danh nhóm tiểu trình mới tương ứng với định danh tiến trình của người dùng. Cuối cùng, việc đọc đường dẫn chuỗi ký tự của người dùng bắt buộc phải thông qua hàm trợ giúp `bpf_probe_read_user_str` nhằm ngăn ngừa lỗi vi phạm trang nhớ khi con trỏ tham chiếu tới vùng địa chỉ chưa hợp lệ.

---

6. Xây dựng Trình giải mã nhị phân và Thu thập trong Go

Khi sự kiện từ nhân Linux được đẩy vào vùng đệm vòng (`Ring Buffer`), ứng dụng Go nhận được một mảng byte nhị phân thô (`[]byte`). Nhiệm vụ của chúng ta là giải mã mảng byte này thành struct Go với hiệu năng tối đa.

Mô hình dữ liệu và giải mã ABI (`event.go`)

```
package ebpfobserver

import (
    "bytes"
    "encoding/binary"
    "errors"
    "fmt"
    "time"
)

// EventPayloadSize đại diện cho kích thước struct C (156B).
const EventPayloadSize = 156

type ExecEvent struct {
    PID       uint32    `json:"pid"`
    UID       uint32    `json:"uid"`
    GID       uint32    `json:"gid"`
    Comm      string    `json:"comm"`
    Filename  string    `json:"filename"`
    Timestamp time.Time `json:"timestamp"`
}
```

Hàm giải mã bóc tách từng trường dữ liệu theo chuẩn little-endian:

```
// DecodeExecEvent giải mã mảng byte thô từ Ring Buffer.
func DecodeExecEvent(data []byte) (*ExecEvent, error) {
    if len(data) < EventPayloadSize {
        return nil, fmt.Errorf(
            "truncated event: want %d bytes, got %d",
            EventPayloadSize, len(data),
        )
    }

    pid := binary.LittleEndian.Uint32(data[0:4])
    uid := binary.LittleEndian.Uint32(data[4:8])
    gid := binary.LittleEndian.Uint32(data[8:12])

    commBytes := data[12:28]
    fileBytes := data[28:156]

    return &ExecEvent{
```

```
        PID:       pid,
        UID:       uid,
        GID:       gid,
        Comm:      parseCString(commBytes),
        Filename:  parseCString(fileBytes),
        Timestamp: time.Now().UTC(),
    }, nil
}

// parseCString cắt bỏ các byte null kết thúc chuỗi của C.
func parseCString(b []byte) string {
    idx := bytes.IndexByte(b, 0)
    if idx >= 0 {
        return string(b[:idx])
    }
    return string(b)
}
```

---

7. Điều phối luồng sự kiện và Phát hiện Bất thường An ninh (`observer.go`)

Để kiểm thử được trên mọi nền tảng mà không phụ thuộc vào quyền root Linux khi chạy unit test, chúng ta trừu tượng hóa nguồn đọc bằng interface `RecordReader`:

```
// RecordReader trừu tượng hóa nguồn đọc từ BPF Ring Buffer.
type RecordReader interface {
    Read() ([]byte, error)
    Close() error
}

type Observer struct {
    reader    RecordReader
    events    chan *ExecEvent
    errs      chan error
    done      chan struct{}
    closeOnce sync.Once
}
```

```
func NewObserver(
    reader RecordReader, bufferSize int,
) *Observer {
    if bufferSize <= 0 {
        bufferSize = 128
    }
    return &Observer{
        reader: reader,
        events: make(chan *ExecEvent, bufferSize),
        errs:   make(chan error, 16),
        done:   make(chan struct{}),
    }
}
```

Đoạn mã rút gọn dưới đây minh họa tiến trình goroutine xử lý; các hàm `readLoop`, `sendErr` và `sendEvent` biểu diễn các nhánh xử lý tương ứng trong bài thực hành, không phải các phương thức được khai báo độc lập trong mã nguồn chính. Khi chạy thử chương trình, hãy sử dụng tệp `observer.go` hoàn chỉnh, bao gồm cả tiến trình goroutine gọi `Close` khi ngữ cảnh bị hủy bỏ. Phương thức `Start` chỉ được phép gọi duy nhất một lần cho mỗi đối tượng quan sát:

```
func (o *Observer) Start(
    ctx context.Context,
) (<-chan *ExecEvent, <-chan error) {
    go func() {
        defer close(o.events)
        defer close(o.errs)
        o.readLoop(ctx)
    }()
    return o.events, o.errs
}
```

Vòng lặp đọc dữ liệu từ Ring Buffer và đẩy vào channel với khả năng hủy qua context:

```
func (o *Observer) readLoop(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return
        case <-o.done:
            return
        default:
            data, err := o.reader.Read()
            if err != nil {
                if errors.Is(err, io.EOF) ||
                    errors.Is(err, ErrObserverClosed) {
                    return
                }
```

```
                o.sendErr(ctx, err)
                continue
            }

            event, decodeErr := DecodeExecEvent(data)
            if decodeErr != nil {
                o.sendErr(ctx, decodeErr)
                continue
            }
            o.sendEvent(ctx, event)
        }
    }
}
```

Ngữ cảnh chỉ mở ra lối thoát tại cấu trúc `select`; nó không thể tự động can thiệp vào phương thức đọc dữ liệu của bộ đọc. Do đó, bài thực hành thiết lập một nhánh hủy bỏ chuyên biệt gọi `reader.Close()`. Khế ước kỹ thuật bắt buộc phải có là thao tác `Close` phải khiến một lệnh đọc đang bị tắc nghẽn lập tức trả về; bộ đọc giả lập trong bài kiểm thử mô phỏng chính xác trạng thái tắc nghẽn này và được giải phóng an toàn khi đóng. Nếu nguồn đọc dữ liệu khác không đáp ứng khế ước đó, chúng ta bắt buộc phải bao bọc hoặc thay đổi giao diện lập trình thay vì đưa ra lời hứa hão huyền về việc hủy bỏ tức thì.

Động cơ phát hiện bất thường an ninh (Heuristic Security Rules)

Khi nhận được sự kiện, chúng ta đối chiếu với các quy tắc an ninh phỏng đoán. Cần lưu ý: Đây là các quy tắc phỏng đoán (**heuristics**) hỗ trợ giám sát, không phải lá chắn bảo mật tuyệt đối, bởi kẻ tấn công nâng cao có thể lẩn tránh bằng cách gọi `execveat`, nạp nhị phân từ RAM qua `memfd_create`, hoặc dùng symlink:

```
type SecurityAlert struct {
    Event  *ExecEvent
    Rule   string
    Reason string
}

func checkBasicRules(e *ExecEvent) *SecurityAlert {
    fn := strings.ToLower(e.Filename)
    comm := strings.ToLower(e.Comm)

    // Luật 1: Đường dẫn nằm dưới thư mục tạm.
    if strings.HasPrefix(fn, "/tmp/") ||
        strings.HasPrefix(fn, "/dev/shm/") ||
        strings.HasPrefix(fn, "/var/tmp/") {
        return &SecurityAlert{
            Event:  e,
            Rule:   "SEC01_EXEC_FROM_TEMP_DIRECTORY",
            Reason: "process execution originated from " +
                "world-writable temporary directory",
```

```
        }
    }

    // Luật 2: Tên caller và đích khớp mẫu đáng điều tra.
    isShell := fn == "/bin/sh" || fn == "/bin/bash" ||
        fn == "/usr/bin/bash" || fn == "/bin/dash"
    isWeb := comm == "nginx" || comm == "node" ||
        comm == "python" || comm == "redis-server"
    if isShell && isWeb {
        return &SecurityAlert{
            Event:  e,
            Rule:   "SEC02_SHELL_FROM_WEBSERVER",
            Reason: "interactive shell spawned directly " +
                "from web service daemon process",
        }
    }
    return nil
}
```

Tiếp theo là quy tắc phát hiện các tiện ích mạng chạy bằng quyền root:

```
func DetectSecurityAnomalies(e *ExecEvent) *SecurityAlert {
    if e == nil {
        return nil
    }
    if alert := checkBasicRules(e); alert != nil {
        return alert
    }

    // Luật 3: Tiện ích dò quét mạng chạy với quyền root (UID 0)
    fn := strings.ToLower(e.Filename)
    isNet := strings.HasSuffix(fn, "/nc") ||
        strings.HasSuffix(fn, "/ncat") ||
        strings.HasSuffix(fn, "/nmap")
    if isNet && e.UID == 0 {
        return &SecurityAlert{
            Event:  e,
            Rule:   "SEC03_ROOT_NETWORK_RECON",
            Reason: "network exploration utility " +
                "executed with root privileges",
        }
    }
    return nil
}
```

`checkBasicRules` là cách tách đoạn trích cho dễ đọc, không phải hàm có sẵn trong lab; hai đoạn trên giữ các đường dẫn và chuỗi `Reason` của `DetectSecurityAnomalies` thật. Dù tên thông báo có chữ "executed" hay "interactive", dữ liệu hook chỉ cho biết một exec attempt. Lab không thu argv, không xác minh quyền ghi thực tế của thư mục, không chứng minh shell tương tác hay RCE. `Comm` là tên task có giới hạn và có thể thay đổi, không phải danh tính đã xác thực. Cảnh báo chỉ là đầu mối điều tra; đừng biến tên rule thành kết luận pháp chứng.

---

8. Kiểm chứng Lab thực tế (`labs/part27-ebpf-observer`)

Bộ kiểm thử tại `labs/part27-ebpf-observer/observer_test.go` vận hành độc lập (phân loại kiểm chứng: kiểm thử đơn vị và mô hình hóa đối với mô hình luồng kênh dữ liệu, giải mã giao diện nhị phân 156 byte, phát hiện bất thường theo suy nghiệm và kiểm tra cấu trúc đặc tả BPF mà không đòi hỏi đặc quyền quản trị của Linux). Chạy bộ kiểm thử hoàn chỉnh với cờ kiểm tra tương tranh dữ liệu:

```
go test -v -race ./...
```

Một transcript lịch sử của sáu test dưới đây chỉ minh họa tên test; thời gian không phải số đo cho lần chạy hiện tại hay cho kernel thật:

```
=== RUN   TestEncodeDecodeExecEvent
--- PASS: TestEncodeDecodeExecEvent (0.00s)
=== RUN   TestDecodeExecEventTruncated
--- PASS: TestDecodeExecEventTruncated (0.00s)
=== RUN   TestObserverStreaming
--- PASS: TestObserverStreaming (0.00s)
=== RUN   TestObserverContextCancellation
--- PASS: TestObserverContextCancellation (0.00s)
=== RUN   TestDetectSecurityAnomalies
--- PASS: TestDetectSecurityAnomalies (0.00s)
=== RUN   TestModeledCollectionSpec
--- PASS: TestModeledCollectionSpec (0.00s)
PASS
ok      part27-ebpf-observer   0.742s
```

Phân tích kết quả kiểm thử và Ranh giới kiểm chứng:

Bộ kiểm thử được phân loại ở cấp độ kiểm thử đơn vị kết hợp mô hình hóa. Biên dịch tệp đối tượng nhị phân là công đoạn đóng gói, đòi hỏi trình biên dịch và các tệp tiêu đề mà mã nguồn C sử dụng; công đoạn này không tự đòi hỏi đặc quyền nạp vào nhân hệ điều hành. Thao tác nạp và gắn điểm theo dõi đòi hỏi nhân Linux hỗ trợ điểm móc nối và đặc quyền phù hợp với phiên bản nhân, chủng loại chương trình cùng chính sách an ninh. Bài thực hành này chủ động bỏ qua bước nạp nhân thực tế khi chạy trên môi trường không hỗ trợ; việc thiếu tệp tiêu đề của nhân không phải là nguyên nhân phổ quát khiến tệp đối tượng đã biên dịch không thể nạp thành công.

Các test gốc kiểm tra decoder little-endian của ABI 156 byte, record thiếu dữ liệu, channel, cancellation, heuristic và `CollectionSpec` mô hình. Test cancellation nay chờ reader báo đã vào `Read` rồi mới hủy context; nếu hủy trước khi đọc, test có thể xanh mà chưa thử khả năng giải phóng lệnh đọc bị chặn. `TestSecurityRulesMatchDocumentedFixtureScope` kiểm tra thêm các nhánh `/var/tmp`, `/usr/bin/bash`, `/bin/dash` và `nmap` để đoạn trích không vô tình thu hẹp source thật. Không phát hiện race trên các đường đã chạy không chứng minh mọi lịch trình đều đúng. Không test nào ở đây chứng minh ELF đã được nạp, hook đã gắn, mọi sự kiện được thu đủ hoặc overhead phù hợp production.

---

9. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Dùng stack vượt giới hạn của kernel/program type.	Verifier có thể từ chối khi compiler hiện thực dữ liệu trên stack quá lớn; một khai báo bị tối ưu bỏ không tự tạo stack use.	Kiểm tra bytecode và verifier log; với event ring buffer của lab, reserve/submit tránh đặt toàn bộ event trên stack.
Lấy nhầm 32 bit thấp của `bpf_get_current_pid_tgid()`.	Thu được Thread ID (TID) thay vì Process ID (PID), khiến log hiển thị PID sai lệch hoàn toàn.	Luôn dịch bit phải 32 bit: `(__u32)(pid_tgid >> 32)` để lấy đúng PID của tiến trình.
Userspace xử lý chậm hơn tốc độ tạo record.	Ring buffer có thể hết chỗ, record bị mất.	Queue và worker có giới hạn chỉ hấp thụ tải trong một phạm vi; đo tốc độ, đếm mất record và chọn chính sách khi quá tải. Thêm buffer không bảo đảm hết mất dữ liệu.
Chạy ứng dụng thiếu quyền Linux capabilities.	Load/attach có thể trả `EPERM`.	Quyền cần thiết phụ thuộc kernel version, cấu hình LSM và loại program; kiểm tra policy của node. Không copy một bộ capability như công thức chung.

---

10. Bài tập thực hành thiết kế Observer eBPF

Thử thách 1: Bộ đếm tần suất thực thi tiến trình (Exec Rate Limiter)

Yêu cầu: Với fixture này, đếm exec attempt theo tên `Comm` trong cửa sổ cố định khoảng 1 giây, cảnh báo khi vượt 50. Đây là nhóm theo tên, không phải identity của một process; nhiều task có thể cùng Comm. Muốn đếm theo process cần PID/TGID cùng lifecycle và namespace phù hợp. Test phải phân biệt cửa sổ cố định với trượt; ngưỡng 50 chỉ là cấu hình bài tập.

Thử thách 2: Bóc tách tham số dòng lệnh từ Syscall (Argv Extraction)

Yêu cầu: `ctx->args[1]` trỏ tới mảng pointer `argv`. Thiết kế việc đọc có giới hạn từng pointer/chuỗi ở phía kernel và wire format với delimiter NUL. Ở Go, decode record thành `Args []string` mà không split hay trim khoảng trắng. Hook hiện tại chưa thu argv; đây là bài tập thiết kế thêm, không phải khả năng đã có.

---

11. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Bộ phát hiện tần suất thực thi

```
type ForkBombDetector struct {
    mu       sync.Mutex
    counts   map[string]int
    lastTick time.Time
}

func NewForkBombDetector() *ForkBombDetector {
    return &ForkBombDetector{
        counts:   make(map[string]int),
        lastTick: time.Now(),
    }
}

func (d *ForkBombDetector) Check(
    e *ExecEvent, threshold int,
```

```
) bool {
    d.mu.Lock()
    defer d.mu.Unlock()

    now := time.Now()
    if now.Sub(d.lastTick) >= time.Second {
        // Reset bộ đếm mỗi giây
        d.counts = make(map[string]int)
        d.lastTick = now
    }

    d.counts[e.Comm]++
    return d.counts[e.Comm] > threshold
}
```

Lời giải Thử thách 2: Bóc tách đối số dòng lệnh

```
type ExecEventExtended struct {
    ExecEvent
    Args []string `json:"args"`
}

func ParseNULSeparatedArgs(raw []byte) []string {
    if len(raw) == 0 {
        return nil
    }
    // Payload đã được validate, không gồm padding cuối buffer.
    if raw[len(raw)-1] == 0 {
        raw = raw[:len(raw)-1]
    }
    tokens := bytes.Split(raw, []byte{0})
    args := make([]string, len(tokens))
    for i, tok := range tokens {
        args[i] = string(tok) // Giữ cả argument rỗng.
    }
    return args
}
```

`argv` là mảng con trỏ tới các chuỗi C. Wire format phải ghi số đối số, số byte thực dùng, giới hạn thu thập và cờ cắt ngắn. Hàm trên giả định payload đã hợp lệ, mỗi đối số có byte NUL kết thúc và không kèm padding thừa. Chẳng hạn `a\x00\x00` biểu diễn hai đối số `"a"` và `""`; bỏ phần tử rỗng sẽ mất dữ liệu. Khoảng trắng cũng là dữ liệu, không dùng để split hay trim. Hook hiện tại chỉ lấy filename; lời giải decode không chứng minh kernel đã thu được argv.

---

eBPF bổ sung một điểm quan sát mạnh mẽ nhưng có giới hạn rõ ràng và cần được đo lường cẩn trọng; phương pháp suy nghiệm (**heuristic**) trong bài lab chỉ tạo tín hiệu cảnh báo phục vụ điều tra, không phải cơ chế ngăn chặn xâm nhập tuyệt đối. Trước khi áp dụng trên các máy chủ thực tế, hãy thẩm định kỹ phiên bản nhân hệ điều hành, đặc quyền hệ thống (**capabilities**), điểm móc được hỗ trợ, tỷ lệ thất thoát bản ghi và chính sách bảo mật dữ liệu của chính môi trường đó.

Chương 28 chuyển từ điểm quan sát kernel sang boundary công cụ Agent: ai được gọi, target nào được phép và bằng chứng nào cho phép một thay đổi. MCP chuẩn hóa giao tiếp, không thay thế authorization hay trách nhiệm vận hành. Chương 29 tiếp tục xét bằng chứng và giới hạn của tự động hóa.

Chương 28 — MCP và AIOps bằng Go: trao công cụ cho Agent mà không trao toàn quyền

Xét một tác tử (Agent) có công cụ đọc trạng thái sức khỏe dịch vụ (**health check**), xem nhật ký (**logs**) và đề xuất khởi động lại dịch vụ (**restart service**). Đọc dữ liệu và thay đổi hệ thống mang lại tác động hoàn toàn khác nhau; việc kết nối chúng qua một giao thức không đồng nghĩa hệ thống đã tự phân định quyền hạn. Chương này bắt đầu từ khế ước (**contract**) về quyền thực thi hành động, không dựa trên mức độ tự tin trong câu trả lời của mô hình trí tuệ nhân tạo.

Khi tool nối tới production, cần hỏi cụ thể:

> **Nếu trao cho Agent quyền truy cập vào một công cụ, làm sao ta bảo đảm tác tử không bị tấn công tiêm lời nhắc (Prompt Injection) để vô tình hoặc cố ý phá hủy cơ sở dữ liệu, rò rỉ token truy cập đám mây, hay làm sập toàn bộ hạ tầng?**

Giao thức bối cảnh mô hình (MCP, tức **Model Context Protocol**) chuẩn hóa ranh giới giao tiếp, nhưng không tự động cấp quyền hay làm cho công cụ trở nên an toàn. Chương này sử dụng thư viện chính thức `github.com/modelcontextprotocol/go-sdk/mcp` (phiên bản v1.8.0) để dựng một máy chủ mẫu mang tính sư phạm theo nguyên tắc đặc quyền tối thiểu, có sổ đăng ký mục tiêu (**target registry**) và dấu vết kiểm toán (**audit trail**). Chính sách ủy quyền (**authorization policy**), độ tin cậy của sổ đăng ký mục tiêu và bộ chấp hành (**actuator**) thực tế vẫn là trách nhiệm của cấu hình triển khai.

---

1. Mental Model: Ranh giới phòng thủ đa tầng từ Agent đến Hạ tầng

Mô hình trung tâm của lab là mỗi lời gọi phải đi qua các kiểm tra riêng. Danh tính do transport xác thực, không do argument tự khai; schema hợp lệ chưa đủ để cấp quyền thực thi:

Kiểm tra	Đối tượng thật của lab	Không suy ra
Giao thức/ transport	MCP SDK; stdio khi chạy server, transport trong memory khi test.	In-memory fixture không phải chứng minh authentication của HTTP deployment.
Schema	Struct và constraint của argument.	Chuỗi đúng schema chưa là target được phép hay instruction đáng tin.
Authoriza tion	Identity trong session context, role và action.	Caller tự ghi `operator` vào argument không tạo quyền.
Target và side effect	Target registry được tin, handler và actuator đã cấu hình.	Allowlist tên không bảo vệ nếu chính registry hay credential bị đổi trái phép.
Audit	Ghi cả quyết định cho phép/từ chối ở boundary tương ứng.	Log không chỉ xảy ra sau hành động thành công; không có bảo đảm bền vững từ interface logger.

Cơ chế truyền nhận qua dòng nhập/xuất chuẩn (**stdio**) tránh việc mở cổng lắng nghe HTTP trên máy chủ nhưng không ngăn chặn được các tiến trình cục bộ trên cùng máy có đặc quyền phù hợp. Lược đồ (**schema**) chỉ kiểm tra các ràng buộc dữ liệu đã khai báo; quyền hạn phải dựa trên định danh tin cậy cùng hành động và mục tiêu cụ thể. Sổ đăng ký mục tiêu của bài lab là cấu hình tĩnh đáng tin cậy, không chấp nhận bất kỳ URL tùy ý nào do tác tử tự ý cung cấp. Dấu vết kiểm toán chỉ thực sự phát huy tác dụng khi bộ ghi, kênh lưu trữ và phương án xử lý khi mất nhật ký được quy định bằng một khế ước rõ ràng.

---

2. Model Context Protocol: Thư viện Go SDK Chính thức

MCP là giao thức mở cho phép ứng dụng khách (**client**) trao đổi với máy chủ cung cấp công cụ và dữ liệu. Giao thức này định nghĩa định dạng thông điệp và vòng đời kết nối; độ phổ biến hay nhãn "tiêu chuẩn công nghiệp" không chứng minh chính sách bảo mật của một hệ thống triển khai là chính xác.

Ở tầng kiến trúc giao vận, MCP vận hành trên nền giao thức **JSON-RPC 2.0.** Máy chủ Go công bố danh sách các công cụ khả dụng kèm mô tả chức năng

và JSON Schema qua phương thức `tools/list`. Khi tác tử quyết định sử dụng một công cụ, nó phát thông điệp `tools/call` chứa tên công cụ và các tham số tương ứng.

Dự án thực hành tại `labs/part28-mcp-ops-tools/` ghim SDK v1.8.0. Theo ma trận tương thích của chính bộ công cụ phát triển (SDK), bản này hỗ trợ và thương lượng phiên bản giao thức (**revision**) mới nhất `2026-07-28` với các đối tác hỗ trợ, đồng thời vẫn tương thích ngược với các phiên bản cũ hơn. Vì phiên bản giao thức được thương lượng theo từng phiên kết nối, máy chủ không được tự suy diễn hành vi trên đường truyền chỉ từ số phiên bản của gói phụ thuộc. Thư viện SDK vẫn cho phép đăng ký hàm xử lý định kiểu rõ ràng, tự sinh JSON Schema từ thẻ cấu trúc Go và cung cấp `ReceivingMiddleware` cho các chính sách kiểm soát cục bộ.

Giao vận Stdio và Kết nối Cục bộ

Bài lab cung cấp kênh kết nối Stdio để ứng dụng khách khởi chạy tiến trình máy chủ và trao đổi qua `stdin`/`stdout`, cùng hàm `NewInMemoryTransports` phục vụ kiểm thử. Kênh Stdio không cần điểm cuối HTTP hay chứng chỉ TLS cho đường ống dẫn cục bộ, nhưng vẫn đòi hỏi kiểm soát tệp thực thi, quyền hạn tiến trình và cấu hình ứng dụng khách. Không thể áp đặt một độ trễ cố định hay ưu tiên phương thức truyền tải chung cho mọi hệ thống DevOps.

---

3. Cạm bẫy thiết kế: SSRF qua URL tùy ý và Giải pháp Target Allowlist

Một sai lầm rất phổ biến của các kỹ sư khi mới thiết kế MCP Server cho AIOps là cho phép Agent truyền trực tiếp một URL tùy ý:

```
{
  "name": "query_service_health",
  "arguments": {"target_url": "https://..."}
}
```

Đây là một khiếm khuyết an ninh nghiêm trọng. Các bộ xác thực JSON Schema chỉ kiểm tra định dạng hình thức của chuỗi ký tự mà không hiểu ngữ nghĩa của địa chỉ đích. Khi tác tử bị tấn công tiêm lời nhắc hoặc gặp

hiện tượng ảo giác, nó có thể gửi yêu cầu nhắm tới địa chỉ siêu dữ liệu nhạy cảm của điện toán đám mây (`http://169.254.169.254/latest/meta-data/...`), cổng quản trị nội bộ trên `localhost:9090`, hoặc các máy chủ cơ sở dữ liệu chưa mã hóa trong mạng riêng (VPC).

Để thu hẹp bề mặt tấn công SSRF do bên gọi tùy ý lựa chọn, tác tử tuyệt đối không được phép truyền trực tiếp địa chỉ URL mà chỉ được truyền mã định danh mục tiêu `target_id` đã được phê duyệt từ trước. Máy chủ thực hiện ánh xạ mã định danh này sang địa chỉ URL tương ứng từ danh bạ mục tiêu và lập tức từ chối các mã lạ. Điều này không tự chứng minh rằng địa chỉ URL trong danh bạ là hoàn toàn an toàn: một danh bạ bị nhập sai hoặc bị can thiệp trái phép vẫn là một ranh giới tin cậy riêng biệt, đòi hỏi phải thẩm định cấu hình, chính sách phân giải tên miền và lưu lượng mạng đi ra ngoài cùng thẩm quyền chỉnh sửa danh bạ.

---

4. Cấu hình Target Allowlist và Đăng ký Công cụ Typed Go

Dưới đây là cấu trúc định danh mục tiêu và mô hình tham số đầu vào được trích xuất từ `labs/part28-mcp-ops-tools/server.go`:

```
type HealthTarget struct {
    ID          string `json:"id"`
    ServiceName string `json:"serviceName"`
    URL         string `json:"url"`
}

// QueryHealthInput định nghĩa schema đầu vào
type QueryHealthInput struct {
    TargetID string `json:"target_id"`
}

// RestartServiceInput định nghĩa tham số mutating tool
type RestartServiceInput struct {
    ServiceName  string `json:"service_name"`
    ChangeTicket string `json:"change_ticket"`
}
```

Các trường struct có thể được bổ sung thêm tag `jsonschema` để cung cấp phần mô tả chi tiết cho từng tham số khi máy chủ MCP công bố danh sách công cụ tới tác tử qua lệnh `tools/list`.

---

5. Phân quyền vai trò theo Ngữ cảnh Phiên và Rào chắn Đột biến

Policy phải lấy quyền của caller từ nguồn tin cậy:

> **Vai trò của Agent phải được xác định qua Ngữ cảnh Phiên (Session Context), không để Agent tự khai báo vai trò trong tham số JSON của công cụ.**

Nếu để tác tử tự ý gửi `{"role": "admin"}` trong các tham số dữ liệu JSON, nguy cơ tiêm lời nhắc có thể dẫn tới việc tác tử mạo danh quản trị viên để chiếm đoạt quyền điều khiển toàn bộ hệ thống.

Gắn danh tính vào context.Context

```
type Role string

const (
    RoleObserver Role = "observer" // Chỉ đọc
    RoleOperator Role = "operator" // Đột biến có phiếu
    RoleAdmin    Role = "admin"    // Toàn quyền
)

type contextKey string
const roleContextKey contextKey = "mcp_caller_role"

func WithCallerRole(
    ctx context.Context, role Role,
) context.Context {
    return context.WithValue(ctx, roleContextKey, role)
}

func CallerRoleFromContext(ctx context.Context) Role {
    if r, ok := ctx.Value(roleContextKey).(Role); ok {
        return r
    }
    return RoleObserver // Mặc định đặc quyền tối thiểu
}
```

Một danh tính thật không tự sinh ra quyền tool

Bộ thử nghiệm phiên làm việc của chương giữ nguyên mức độ giả lập kiểm chứng đối với nguồn dữ liệu định danh. Nếu muốn khảo sát ranh giới mạng thực tế mà không cần đưa giao thức MCP vào mọi phép thử, hãy thực thi bài thực hành TLS từ Chương 11 trước: tiến trình đọc dù có chứng chỉ số đã được xác minh vẫn bị từ chối quyền đột biến; tiêu đề tự xưng là quản trị viên không thể thay đổi quyết định an ninh. Đó là bằng chứng về quá trình bắt tay và chính sách HTTP cục bộ, chưa phải là phép thẩm định ủy quyền toàn diện của giao thức truyền tải dạng luồng.

Khi triển khai hệ thống có cơ chế kết thúc TLS tại máy chủ ủy nhiệm, hàm xử lý có thể không còn giữ lại chuỗi chứng chỉ đã xác thực của máy khách ban đầu. Tuyệt đối không sao chép tiêu đề định danh từ mạng Internet vào ngữ cảnh phiên làm việc; hệ thống bắt buộc phải xác định máy chủ ủy nhiệm nào được tin cậy, bảo vệ đường truyền giữa máy chủ ủy nhiệm và dịch vụ nội bộ, đồng thời kiểm tra cơ chế chuyển giao danh tính của kiến trúc triển khai. Các phương thức truyền tải qua dòng dữ liệu chuẩn, xác thực OAuth hay xác thực hai chiều mTLS có cách thức thiết lập bên gọi khác nhau, nhưng đều có chung yêu cầu bắt buộc: ánh xạ bên gọi đã xác thực sang hành động và mục tiêu được phép. Chứng chỉ trong bài thực hành không làm thay đổi chính sách MCP và không khẳng định mọi máy chủ MCP đều bắt buộc phải dùng mTLS.

Tách bạch hai nhóm công cụ và Rào chắn Đột biến

Chính sách của bài thực hành cho phép vai trò quan sát viên gọi `query_service_health` trên các mục tiêu đã đăng ký và nghiêm cấm gọi `restart_service`. Đó không phải là đặc quyền tự do truy vấn tùy tiện: hệ thống thực tế đòi hỏi giới hạn phạm vi dữ liệu, mục tiêu cho phép, định mức tài nguyên và danh tính người gọi. Vai trò người vận hành vẫn bắt buộc phải đi qua bộ cấp phép thay đổi `ChangeAuthorizer` rồi mới tới bộ thực thi dịch vụ `ServiceActuator`; tên gọi của vai trò không tự nó mang đủ thẩm quyền để kích hoạt thay đổi.

`DefaultChangeAuthorizer` chỉ chấp nhận đúng `operator` hoặc `admin` rồi mới kiểm phiếu và dịch vụ. Một vai trò rỗng, viết sai hay chưa được định

nghĩa phải bị từ chối; kiểm riêng `role != observer` sẽ vô tình mở quyền cho các giá trị ấy. Đây là policy của lab, không phải role model do MCP chuẩn hóa.

```
type ServiceActuator interface {
    RestartService(
        ctx context.Context, serviceName string,
    ) error
}

type ChangeAuthorizer interface {
    AuthorizeChange(
        ctx context.Context, role Role, svc, ticket string,
    ) error
}
```

---

6. Xây dựng OpsServer trên nền tảng Official MCP Go SDK

Dưới đây là các cấu phần trọng tâm của máy chủ `OpsServer` trong tệp `labs/part28-mcp-ops-tools/server.go`. Các đoạn trích được rút gọn để đọc từng ranh giới: phần gắn khóa caller cho điều tốc và phép kiểm tra limiter trong handler không in lại ở đây. Source lab mới là cài đặt đầy đủ; việc biên dịch các đoạn trích không chứng minh policy đã bị lược bỏ vẫn hoạt động.

6.1. Khởi tạo máy chủ và ranh giới danh tính

Trong kiến trúc của bộ công cụ MCP Go, hàm `AddReceivingMiddleware` cho phép can thiệp vào mọi thông điệp triệu gọi từ xa gửi đến trước khi điều phối tới hàm xử lý cụ thể. Chúng ta sử dụng phần mềm trung gian này để bóc tách định danh bên gọi từ phiên làm việc và tiêm vai trò bảo mật vào ngữ cảnh `context.Context`. Bản thân `context.Context` không tự bảo đảm tính xác thực của danh tính; phần mềm trung gian trong bài thực hành lấy vai trò từ bảng ánh xạ phiên giả lập để phục vụ kiểm thử mô hình hóa. Trên môi trường vận hành thực tế, tầng truyền tải hoặc nhà cung cấp định danh bắt buộc phải xác thực danh tính bên gọi trước khi tiêm vai trò vào ngữ cảnh:

```
type OpsServer struct {
    mcpServer   *mcp.Server
    httpClient  *http.Client
    actuator    ServiceActuator
    authorizer  ChangeAuthorizer
    limiter     ToolRateLimiter
    targets     map[string]HealthTarget

    mu          sync.RWMutex
    sessionRole map[string]Role
    auditLog    []AuditRecord
}

func NewOpsServer(
    client *http.Client,
    actuator ServiceActuator,
    authorizer ChangeAuthorizer,
    limiter ToolRateLimiter,
    targets []HealthTarget,
) *OpsServer {
    if client == nil {
```

```
        client = &http.Client{Timeout: 5 * time.Second}
    }
    if limiter == nil {
        limiter = NewKeyedIntervalLimiter(10)
    }
    targetMap := make(map[string]HealthTarget, len(targets))
    for _, t := range targets {
        targetMap[t.ID] = t
    }

    server := mcp.NewServer(
        &mcp.Implementation{
            Name: "ops-mcp-server", Version: "v1.8.0",
        },
        &mcp.ServerOptions{
            Instructions: "AIOps server allowlist",
        },
    )

    ops := &OpsServer{
        mcpServer:   server,
```

```
        httpClient:  client,
        actuator:    actuator,
        authorizer:  authorizer,
        limiter:     limiter,
        targets:     targetMap,
        sessionRole: make(map[string]Role),
    }

    // Test mapping only; not production authentication.
    // Real transport authenticates before dispatch.
    server.AddReceivingMiddleware(
        func(next mcp.MethodHandler) mcp.MethodHandler {
            return func(
                ctx context.Context,
                method string,
                req mcp.Request,
            ) (mcp.Result, error) {
                role := CallerRoleFromContext(ctx)
                var sess mcp.Session
                if req != nil {
```

```
                    sess = req.GetSession()
                }
                if role == RoleObserver && sess != nil {
                    ops.mu.RLock()
                    id := sess.ID()
                    sessRole, found := ops.sessionRole[id]
                    ops.mu.RUnlock()
                    if found {
                        role = sessRole
                    }
                }
                ctx = WithCallerRole(ctx, role)
                return next(ctx, method, req)
            }
        },
    )

    ops.registerTools()
    return ops
}
```

6.2. Đăng ký Công cụ Kiểm tra Sức khỏe và Rào chắn SSRF

Công cụ `query_service_health` được đăng ký qua `mcp.AddTool`. SDK tự động giải mã tham số vào `QueryHealthInput` và chuyển giao quyền kiểm soát cho closure handler. Allowlist dưới đây chỉ chặn ID lạ; `http.Client` vẫn có thể theo redirect, và đích DNS có thể thay đổi. Lab không cài bộ lọc IP hoặc redirect: không dùng test ID lạ làm bằng chứng đã chặn mọi đường SSRF.

```
mcp.AddTool(s.mcpServer, &mcp.Tool{
    Name:        "query_service_health",
    Description: "Ktra sức khỏe qua target_id allowlist",
}, func(
    ctx context.Context,
    req *mcp.CallToolRequest,
    input QueryHealthInput,
) (*mcp.CallToolResult, any, error) {
    role := CallerRoleFromContext(ctx)
    target, found := s.targets[input.TargetID]
    if !found {
        errStr := fmt.Sprintf(
            "unapproved target_id %q: "+
                "blocked by SSRF allowlist policy",
            input.TargetID,
        )
        s.RecordAudit(
            role, "query_service_health",
            map[string]any{"target_id": input.TargetID},
            "DENY", errStr,
        )
        return &mcp.CallToolResult{
```

```
            IsError: true,
            Content: []mcp.Content{
                &mcp.TextContent{Text: errStr},
            },
        }, nil, nil
    }

    start := time.Now()
    httpReq, err := http.NewRequestWithContext(
        ctx, http.MethodGet, target.URL, nil,
    )
    if err != nil {
        errStr := fmt.Sprintf("req err: %v", err)
        s.RecordAudit(
            role, "query_service_health",
            map[string]any{"target_id": input.TargetID},
            "DENY", errStr,
        )
        return &mcp.CallToolResult{
            IsError: true,
            Content: []mcp.Content{
                &mcp.TextContent{Text: errStr},
```

```
            },
        }, nil, nil
    }

    resp, err := s.httpClient.Do(httpReq)
    if err != nil {
        errStr := fmt.Sprintf("health check failed: %v", err)
        s.RecordAudit(
            role, "query_service_health",
            map[string]any{"target_id": input.TargetID},
            "DENY", errStr,
        )
        return &mcp.CallToolResult{
            IsError: true,
            Content: []mcp.Content{
                &mcp.TextContent{Text: errStr},
            },
        }, nil, nil
    }
    defer resp.Body.Close()

    duration := time.Since(start).Round(time.Millisecond)
```

```
    health := "unhealthy"
    if resp.StatusCode >= http.StatusOK &&
        resp.StatusCode < http.StatusMultipleChoices {
        health = "healthy"
    }
    resText := fmt.Sprintf(
        "Service %s (%s) %s: HTTP %d in %v",
        target.ServiceName, target.ID, health,
        resp.StatusCode, duration,
    )
    s.RecordAudit(
        role, "query_service_health",
        map[string]any{"target_id": input.TargetID},
        "ALLOW", resText,
    )
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: resText},
        },
    }, nil, nil
})
```

6.3. Công cụ gây đột biến: dependency bắt buộc, không có đường mặc định cho phép

Với công cụ `restart_service`, `ChangeAuthorizer` và `ServiceActuator` là dependency bắt buộc. Thiếu một trong hai phải trả `DENY`; không được coi `nil` là "không cần kiểm tra" rồi báo restart thành công. Khi cả hai có mặt, yêu cầu phải qua `AuthorizeChange` trước khi kích hoạt `RestartService`:

```
mcp.AddTool(s.mcpServer, &mcp.Tool{
    Name:        "restart_service",
    Description: "Restart dịch vụ có điều kiện, cần ticket",
}, func(
    ctx context.Context,
    req *mcp.CallToolRequest,
    input RestartServiceInput,
) (*mcp.CallToolResult, any, error) {
    role := CallerRoleFromContext(ctx)
    args := map[string]any{
        "service_name":  input.ServiceName,
        "change_ticket": input.ChangeTicket,
    }
    deny := func(message string) (
        *mcp.CallToolResult, any, error,
    ) {
        s.RecordAudit(role, "restart_service", args,
            "DENY", message)
        return &mcp.CallToolResult{
```

```
            IsError: true,
            Content: []mcp.Content{
                &mcp.TextContent{Text: message},
            },
        }, nil, nil
    }

    if s.authorizer == nil || s.actuator == nil {
        return deny(
            "required authorization or actuator missing",
        )
    }
    if err := s.authorizer.AuthorizeChange(
        ctx, role, input.ServiceName, input.ChangeTicket,
    ); err != nil {
            s.RecordAudit(
                role, "restart_service", args,
                "DENY", err.Error(),
            )
```

```
            return &mcp.CallToolResult{
                IsError: true,
                Content: []mcp.Content{
                    &mcp.TextContent{Text: err.Error()},
                },
            }, nil, nil
    }

    if err := s.actuator.RestartService(
        ctx, input.ServiceName,
    ); err != nil {
            errStr := fmt.Sprintf("restart failed: %v", err)
            s.RecordAudit(
                role, "restart_service", args, "DENY", errStr,
            )
            return &mcp.CallToolResult{
                IsError: true,
                Content: []mcp.Content{
```

```
                    &mcp.TextContent{Text: errStr},
                },
            }, nil, nil
    }

    resText := fmt.Sprintf(
        "Service %s restarted under ticket %s",
        input.ServiceName, input.ChangeTicket,
    )
    s.RecordAudit(
        role, "restart_service", args, "ALLOW", resText,
    )
    return &mcp.CallToolResult{
        Content: []mcp.Content{
            &mcp.TextContent{Text: resText},
        },
    }, nil, nil
})
```

6.4. Ghi Nhật ký Kiểm toán Có Cấu trúc

Để theo dõi mọi hành vi của tác tử, máy chủ lưu vết các quyết định vào cấu trúc `AuditRecord`:

```
type AuditRecord struct {
    Timestamp time.Time      `json:"timestamp"`
    Caller    Role           `json:"caller"`
    ToolName  string         `json:"toolName"`
    Arguments map[string]any `json:"arguments"`
    Decision  string         `json:"decision"` // ALLOW/DENY
    Result    string         `json:"result"`
}

func (s *OpsServer) RecordAudit(
    caller Role, tool string, args map[string]any,
    decision, result string,
) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.auditLog = append(s.auditLog, AuditRecord{
        Timestamp: time.Now().UTC(),
        Caller:    caller,
        ToolName:  tool,
        Arguments: args,
        Decision:  decision,
        Result:    result,
    })
}
```

Nhật ký kiểm toán trong bộ nhớ RAM chỉ lưu giữ các bản ghi trong vòng đời vận hành của tiến trình. Việc lưu trữ bên ngoài tiến trình có thể gia tăng độ bền vững, nhưng các hệ thống lưu trữ phân tán không tự động bảo đảm tính toàn vẹn tuyệt đối. Hệ thống cần xác định rõ ai có quyền ghi hoặc xóa, thời hạn lưu giữ, cấu hình chống sửa đổi và phương thức phát hiện bản ghi bị thất thoát. Việc chuyển giao dữ liệu bất đồng bộ còn tiềm ẩn khoảng thời gian tiến trình bị sập trước khi bản ghi được xác nhận lưu trữ bền vững; chính sách bắt buộc phải xác định rõ có cho phép thao tác đột biến diễn ra khi hệ thống lưu trữ kiểm toán gặp sự cố hay không, đi kèm cơ chế đối soát kết quả rõ ràng.

---

7. Kiểm chứng Lab thực tế (`labs/part28-mcp-ops-tools`)

Mã nguồn hoàn chỉnh nằm tại `labs/part28-mcp-ops-tools` (phân loại kiểm chứng: kiểm thử đơn vị và giả lập kiểm chứng cho giao thức MCP chính thức, phân quyền theo ngữ cảnh phiên làm việc, rào chắn chống SSRF qua danh sách mục tiêu cho phép, thực thi yêu cầu HTTP tới máy chủ kiểm thử và kích hoạt bộ thực thi). Chạy kiểm thử:

```
go test -v -race ./...
```

Bộ kiểm thử xác thực 6 kịch bản an ninh thực chiến trên nền giao vận in-memory:

```
=== RUN   TestMCPListTools
--- PASS: TestMCPListTools (0.00s)
=== RUN   TestMCPQueryHealthRealHTTP
--- PASS: TestMCPQueryHealthRealHTTP (0.00s)
=== RUN   TestMCPSSRFDeniedForUnapprovedTarget
--- PASS: TestMCPSSRFDeniedForUnapprovedTarget (0.00s)
=== RUN   TestMCPOperatorRestartSuccess
--- PASS: TestMCPOperatorRestartSuccess (0.00s)
=== RUN   TestMCPObserverMutatingActionDenied
--- PASS: TestMCPObserverMutatingActionDenied (0.00s)
=== RUN   TestMCPMissingOrInvalidTicketDenied
--- PASS: TestMCPMissingOrInvalidTicketDenied (0.00s)
PASS
ok      part28-mcp-ops-tools   0.234s
```

Phân tích kết quả kiểm thử và Ranh giới xác thực:

Bộ kiểm thử khởi tạo client và server MCP qua kênh `mcp.NewInMemoryTransports()`. Kịch bản kiểm tra `tools/list` xác nhận schema JSON được sinh tự động từ struct tags `jsonschema`. Kịch bản kiểm tra sức khỏe gửi yêu cầu HTTP thực tế tới `httptest.Server`, đo thời gian phản hồi và ghi nhận trạng thái `ALLOW`.

Các bài kiểm thử xác nhận mục tiêu không xác định bị từ chối, vai trò quan sát viên không thể gọi thao tác khởi động lại, và phiếu phê duyệt sai không kích hoạt bộ thực thi trong các trường hợp đã kiểm tra. Bản ghi từ chối được kiểm chứng trực tiếp trong nhật ký bộ nhớ. Kết quả này không chứng minh

khả năng phòng chống trước mọi cuộc tấn công SSRF, tiêm lời nhắc hay nguy cơ thất thoát dữ liệu kiểm toán: danh bạ mục tiêu, nhà cung cấp định danh, tầng truyền vận và kho lưu trữ thực tế là các ranh giới kỹ thuật chưa được bộ thử nghiệm này bao quát.

---

8. Các cạm bẫy người học thường gặp (Learner Pitfalls)

Cạm bẫy thực tế	Hậu quả trên Production	Giải pháp phòng ngừa
Phó mặc an ninh cho JSON Schema của MCP.	Bị tấn công SSRF hoặc Prompt Injection đánh cắp token IAM đám mây.	Luôn coi tham số từ Agent là không tin cậy; kiểm tra ngữ nghĩa và IP trong Go handler.
Cho phép Agent gọi tool mutating mà không có mã phiếu phê duyệt.	Agent tự ý xóa hoặc khởi động lại các dịch vụ quan trọng khi hiểu nhầm bối cảnh.	Yêu cầu `change_ticket` và chỉ cấp quyền mutating cho vai trò Operator.
Mở HTTP không bảo vệ identity/kênh truyền.	Có thể bị nghe lén hoặc mạo danh.	Chọn Stdio khi phù hợp; với Streamable HTTP, dùng TLS và authorization theo spec/deployment. mTLS là một lựa chọn, không yêu cầu chung cho mọi MCP transport.
Không ghi nhật ký kiểm toán (Audit Trail).	Thiếu dữ liệu ở boundary tool để xác định caller và hành động, dù có thể còn log từ các tầng khác.	Ghi structured audit log phù hợp policy lưu trữ và bảo vệ dữ liệu.

---

9. Bài tập thực hành thiết kế MCP Server an toàn

Thử thách 1: Điều tốc phiên làm việc của Agent (Agent Rate Limiting)

Yêu cầu: Thiết kế phần mềm trung gian tiếp nhận cho lệnh gọi công cụ, gắn định mức tốc độ với bên gọi hoặc phiên làm việc đã xác thực. Bài tập ấn định hạn mức 10 yêu cầu/giây, không phải ngưỡng an toàn áp dụng cho môi trường vận hành thực tế. Kiểm thử cả trường hợp bên gọi sử dụng nhiều phiên làm việc đồng thời và dọn dẹp trạng thái bộ điều tốc; bộ công cụ không cung cấp hàm thực thi trực tiếp trong ví dụ này.

Thử thách 2: Cơ chế Phê duyệt Hai bước (Human-in-the-Loop Gate)

Yêu cầu: Đối với các hành động phá hủy nghiêm trọng (như xóa cơ sở dữ liệu hay thu hồi hạ tầng), hãy thiết kế cơ chế phê duyệt hai bước cho công cụ MCP: hành động ban đầu trả về trạng thái chờ kèm mã xác nhận tạm thời. Thao tác chỉ được thực thi khi nhận được lệnh xác nhận chứa mã thông báo hợp lệ trong thời hạn 5 phút. Cơ chế phải ràng buộc chặt chẽ mã xác nhận với hành động và mục tiêu đã phê duyệt, đồng thời bảo đảm mã xác nhận chỉ được dùng thành công đúng một lần duy nhất.

---

10. Hướng dẫn giải và Phân tích kiến trúc bài tập

Lời giải Thử thách 1: Bộ điều tốc theo caller đã xác thực

```
type ToolRateLimiter interface {
    Allow(key string, now time.Time) bool
}

// Giữ state theo session/caller đã xác thực.
// Không chia một lastCall toàn cục giữa các agent.
// Replicas need distributed limiting for a shared quota.
```

Lời giải Thử thách 2: Rào chắn Phê duyệt Hai bước

> **Dừng để dự đoán:** Giả sử Agent xin cấp token để khởi động lại dịch vụ cache staging, nhưng sau đó gửi lệnh xác nhận kèm theo hành động xóa database production. Nếu hệ thống chỉ kiểm tra thời hạn của token mà không so sánh hành động và mục tiêu, rủi ro an ninh nào sẽ phát sinh? Khi thiết kế thao tác xác nhận, ta nên xóa token ngay ở lần thử sai đầu tiên hay chỉ hủy token sau khi đã khớp thành công?

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Nếu mã xác nhận không gắn chặt với hành động và mục tiêu (`Action` và `Target`), hệ thống sẽ vấp phải lỗ hổng tráo đổi mục tiêu (action substitution). Một lời nhắc độc hại từ bên ngoài có thể lợi dụng sự phê duyệt của con người cho một thao tác vô hại để thực thi lệnh phá hủy trên hệ thống trọng yếu. Vì vậy, bản ghi phê duyệt phải đóng băng bộ đôi hành động và tài nguyên ngay từ thời điểm tạo token.

Về chính sách tiêu thụ token, ta lựa chọn chính sách vô hiệu hóa khi khớp thành công (single-use successful claim). Khi nhận đúng token, đúng hành động, đúng mục tiêu và còn trong thời hạn 5 phút, hàm sẽ xóa token khỏi bộ nhớ để chống tấn công phát lại (replay attack). Ngược lại, nếu chọn chính sách xóa token ngay cả khi tham số truyền vào không khớp, kẻ tấn công hoặc một tiến trình thử lại nhầm lẫn có thể cố tình gửi sai tham số để hủy hoại phiên phê duyệt hợp lệ, tạo ra nguy cơ từ chối dịch vụ (denial-of-service). Việc người dùng gửi một token không tồn tại hoàn toàn không gây ảnh hưởng đến các token hợp lệ khác. Nguy cơ dò đoán tham số khi giữ token được kiểm soát bằng độ hỗn loạn cao của chuỗi sinh ngẫu nhiên từ `crypto/rand` và thời gian hết hạn ngắn.

Cài đặt an toàn của `ApprovalManager` dưới mô hình bộ nhớ cục bộ:

Đoạn dưới là phác thảo của biến thể **chỉ tiêu thụ khi khớp thành công**, không phải source lab nguyên vẹn: nó lược constructor khởi tạo `pendings` và helper sinh token bằng `crypto/rand` có trả lỗi. Source `approval.go` chọn policy khác: một lần xác nhận có token tồn tại sẽ tiêu thụ token, kể cả khi gửi sai action/target. `TestApprovalTokenIsBoundAndOneTime` kiểm policy đó. Hai lựa chọn đánh đổi khả năng thử lại hợp lệ với nguy cơ dò/thử nhiều lần; không dùng test của policy này làm chứng cứ cho policy kia.

```
type PendingApproval struct {
    Action    string
    Target    string
    ExpiresAt time.Time
}

type ApprovalManager struct {
    mu       sync.Mutex
    pendings map[string]PendingApproval
}

func (m *ApprovalManager) RequestApproval(
    action, target string,
) string {
    m.mu.Lock()
    defer m.mu.Unlock()

    token := randomTokenFromCryptoRand()
    m.pendings[token] = PendingApproval{
        Action:    action,
        Target:    target,
```

```
        ExpiresAt: time.Now().Add(5 * time.Minute),
    }
    return token
}

func (m *ApprovalManager) Confirm(
    token, action, target string,
) bool {
    m.mu.Lock()
    defer m.mu.Unlock()

    item, exists := m.pendings[token]
    if !exists || !time.Now().Before(item.ExpiresAt) {
        return false
    }
    if item.Action != action || item.Target != target {
        return false
    }
    delete(m.pendings, token)
    return true
}
```

Toàn bộ thao tác kiểm tra và tiêu thụ mã xác nhận (`token`) phải diễn ra nguyên tử dưới cùng một khóa `m.mu.Lock()` nhằm ngăn chặn hai goroutine chạy đua cùng lúc với cùng một mã và bảo toàn liên kết chặt chẽ với hành động cùng mục tiêu (`Action` và `Target`). Trong mô hình này, việc chỉ xóa mã khi xác nhận thành công giúp duy trì tính hợp lệ của phiên phê duyệt trước các lần gửi sai tham số hoặc thử lại nhầm lẫn, đồng thời bảo đảm mỗi mã xác nhận chỉ được sử dụng đúng một lần.

Tuy nhiên, việc không xóa token khi kiểm tra thất bại đặt ra yêu cầu bắt buộc về chính sách dọn dẹp: những mã xác nhận đã hết hạn mà không bao giờ được sử dụng sẽ tích tụ vĩnh viễn trong bản đồ nếu không có cơ chế thu gom. Một hệ thống hoàn chỉnh đòi hỏi tiến trình chạy nền dọn dẹp định kỳ (periodic cleanup worker) hoặc cơ chế loại bỏ lười (lazy eviction) để quét và xóa các mục có `time.Now().After(item.ExpiresAt)`. Cấu trúc `ApprovalManager` trên đây là mô hình kiến trúc minh họa trong bộ nhớ cục bộ của một tiến trình, chưa phải giải pháp an toàn cho môi trường sản xuất. Môi trường thực tế đòi hỏi kho dữ liệu bền vững chịu lỗi qua các lần khởi động lại, xác thực danh tính con người độc lập khỏi tác tử AI (`agent`), và nhật ký kiểm toán không thể sửa đổi.

Một yêu cầu HTTP kết thúc thành công không đồng nghĩa dịch vụ đã thực sự khỏe mạnh. Bài lab áp dụng quy ước đơn giản: chỉ mã `2xx` được phân loại là `healthy`; mã `4xx`/`5xx` được trả về như trạng thái `unhealthy`, trong khi lỗi tầng truyền vận (**transport error**) mới tính là lỗi gọi công cụ. Hệ thống thực tế đòi hỏi khế ước kiểm tra trạng thái sức khỏe riêng biệt (chẳng hạn đầu kiểm tra sẵn sàng phục vụ như **readiness probe**, cấu trúc dữ liệu phản hồi và giới hạn thời gian chờ) thay vì chỉ suy diễn tình trạng sức khỏe từ mã trạng thái HTTP.

---

11. Điểm dừng: một tool an toàn là một boundary có bằng chứng

Chương 28 kết nối các ranh giới công cụ kỹ thuật đã học. Bảng dưới là bản đồ tra cứu, không phải chứng nhận đã làm chủ production; Chương 29 tiếp tục phân biệt đề xuất, bằng chứng, thẩm quyền và kết quả.

Chặng đường	Phạm vi kiến thức và kỹ năng	Thành tựu kỹ thuật cốt lõi
Phần I: Nền tảng Cơ học & Bộ nhớ Go (Ch. 00–06)	Cú pháp, value, pointer, slice, error, package và testing.	Dự đoán copy/aliasing, thiết kế package và đọc contract lỗi.
Phần II: Tương tranh & Độ ổn định Hệ thống (Ch. 07–13)	I/O stream, memory model, race detector, worker pool, GC, client trace và server lifecycle.	Tái lập race, thiết kế đường thoát, đo bottleneck và ranh giới shutdown.
Phần III: Dịch vụ Phân tán & Cơ sở Dữ liệu (Ch. 14–21)	Reflection, incident CLI, observability, container, CI/CD, generics và dự án opsprobe.	Đặt timeout, transaction boundary, type parameter và điều hòa hệ thống.
Phần IV: Kubernetes Operator & eBPF Nâng cao (Ch. 22–29)	Controller client-go, Operator CRD, AWS SDK v2, GitHub automation, Cosign, eBPF và MCP.	Xét retry, identity, quyền và evidence của từng boundary an toàn.

Khi đưa những ý tưởng này vào môi trường vận hành thực tế, hãy rà soát lại mô hình mối đe dọa (**threat model**), ranh giới tin cậy (**trust boundary**), khối lượng công việc thực tế và quy trình xử lý sự cố của từng hệ thống cụ thể. Các bài thực hành chỉ chứng minh những khế ước kỹ thuật trong phạm vi hẹp; chúng không thể thay thế việc xác thực danh tính thực tế, chính sách kiểm soát lưu lượng đi ra (**egress policy**), hệ thống kiểm toán bền bỉ hay quy trình duyệt vận hành của con người.

Chương 29 — Kỹ sư và bằng chứng trong kỷ nguyên Agent

Một tác tử AI (Agent) vừa đề xuất thay đổi thời gian chờ (**timeout**), tăng số lượng bản sao (**replicas**) và khởi động lại một dịch vụ (**restart service**). Nó đưa ra một đoạn giải thích mạch lạc, một bản vá mã nguồn hợp lệ và dòng thông báo "toàn bộ kiểm thử đều đạt". Nếu ta bấm đồng ý phê duyệt, điều gì đã thực sự được chứng minh? Rất có thể chỉ là bài kiểm thử trên một nhánh làm việc khác đã chạy thành công; có thể bản vá đang áp dụng cấu hình khác với môi trường thực tế; hoặc cũng có thể quyền khởi động lại chưa từng được cấp phát cho bên gọi này. Chất lượng của câu trả lời do AI sinh ra và quyền hạn tác động trực tiếp vào hệ thống hoàn toàn không phải là cùng một đại lượng.

Chương 28 đã hướng dẫn cách trao một công cụ có giới hạn rõ ràng. Chương này chuyển dịch góc nhìn từ phạm vi công cụ sang bản chất công việc của người kỹ sư: chuỗi kiểm chứng **đề xuất → bằng chứng → quyền hạn → hành động giới hạn → hậu điều kiện**. Tác tử AI có thể hỗ trợ đắc lực ở nhiều bước, nhưng một bước hoàn tất không tự động thay thế cho bước kế tiếp. Một mã băm đúng không chứng minh mã nguồn đúng; một ca kiểm thử đạt không tự cấp quyền triển khai; và một đợt triển khai thành công không chứng minh người dùng đã được phục vụ tốt hơn. Đây là mô hình tư duy cốt lõi để thiết kế quy trình kiểm soát và đánh giá khách quan những lời hứa hẹn về tương lai nghề nghiệp, chứ không phải một lời dự đoán xem chức danh nào sẽ trường tồn mãi mãi.

Đọc số liệu theo câu hỏi nó thực sự trả lời

Ngày đối chiếu lại nguồn của chương là 08/10/2026. "AI giúp lập trình nhanh hơn" thiếu ít nhất ba đối tượng: ai làm, làm việc gì, và nhanh hơn theo phép đo nào. Dữ liệu sau không dùng chung mẫu hay cùng loại outcome, nên không được lấy trung bình các phần trăm để tạo một con số về "năng suất AI".

Nghiên cứu của Peng và cộng sự, công bố tháng 02/2023 trên arXiv, phân nhóm ngẫu nhiên 95 lập trình viên tuyển qua Upwork từ ngày 15/05 đến ngày 20/06/2022. Công việc là viết HTTP server JavaScript. Có 70 người hoàn thành task và survey, 35 ở mỗi nhóm; phân tích thời gian trên nhóm hoàn thành cho mức giảm 55,8% khi được dùng GitHub Copilot, khoảng tin cậy 95% 21–89%. Mẫu phần lớn từ Ấn Độ và Pakistan. Kết quả thuộc nhiệm vụ chuẩn hóa và công cụ năm 2022, không đại diện toàn vòng đời service hay incident production. Tác giả có liên hệ Microsoft/GitHub; nghiên cứu không đo đầy đủ chất lượng bảo trì dài hạn.

METR công bố ngày 10/07/2025 một RCT với 16 người đóng góp giàu kinh nghiệm, hoàn thành 246 tác vụ trên kho mã nguồn mở quen thuộc. Mỗi tác vụ được phân ngẫu nhiên vào điều kiện cho phép hoặc không cho phép AI; công cụ thuộc giai đoạn tháng 02 đến tháng 06/2025, chủ yếu Cursor Pro với Claude 3.5/3.7 Sonnet. Ước lượng có điều chỉnh của nghiên cứu là thời gian tăng 19%, khoảng tin cậy 95% 2–39%; không phải chỉ lấy tỷ số hai thời gian trung bình thô. Đối tượng đo là contributor và task được tuyển, không đại diện lao động quốc gia. Kết quả không chứng minh AI vô dụng ở mọi bối cảnh.

Bản cập nhật METR ngày 24/02/2026 nêu vấn đề chọn người, chọn task và đo thời gian khi người tham gia dùng công cụ mới hoặc làm nhiều việc song song. Tác giả coi các ước lượng mới là không đáng tin để định lượng hiệu ứng hiện tại. Vì vậy cũng không được dùng con số chậm 19% của năm 2025 như định luật cho Agent năm 2026. “Chưa đo chắc” là một kết luận có ích: nó ngăn cả quảng cáo lẫn phản đối AI dựa trên một thí nghiệm đã đổi bối cảnh.

Go Developer Survey 2025 do Go team công bố ngày 21/01/2026, thu thập từ ngày 09/09 đến ngày 30/09/2025, dùng 5.379 phản hồi sau làm sạch. Mời qua kênh công khai và lời mời ngẫu nhiên trong VS Code/GoLand; đây không phải mẫu ngẫu nhiên của toàn thị trường lao động. Phản hồi chủ yếu từ Bắc Mỹ và châu Âu. CLI và API service là nhóm use case nổi bật, và có người dùng Go cho hạ tầng cloud. Điều đó cho bối cảnh ecosystem, không cho số việc làm Go ở Việt Nam. Nhận xét về AI là tự báo cáo, không phải phép đo tốc độ có nhóm đối chứng.

Ở lớp thị trường, BLS cập nhật Occupational Outlook Handbook ngày 27/08/2026, dự báo việc làm tại Mỹ của nhóm software developers, quality assurance analysts và testers tăng 10% trong 2025–2035. Đây là dự báo nghề nghiệp từ chương trình Employment Projections, không phải quan sát rằng việc làm đã tăng từng ấy, không tách Go, DevOps hay SRE và không mô tả Việt Nam. Openings còn gồm nhu cầu thay người rời nghề, không chỉ vị trí mới. Sách không có dữ liệu đủ mạnh để suy ra lương hay nhu cầu Go tại Việt Nam từ nhóm nghề rộng này.

Báo cáo DORA 2025 xem xét AI trong mối tương quan với năng lực của tổ chức: một công cụ có thể khuếch đại cả điểm mạnh lẫn điểm yếu đang có. Trang công bố thuộc chương trình nghiên cứu do Google Cloud dẫn dắt, có đối tác ngành. Ta dùng framing này như một gợi ý kiểm tra quy trình, không lấy nó làm bằng chứng nhân quả rằng mua công cụ sẽ làm SLO tốt hơn. Khi cần một con số từ survey, phải đọc mẫu, câu hỏi và phương pháp của chính số đó; chương không đưa phần trăm từ một báo cáo chỉ đọc phần giới thiệu.

Tự động hóa tác vụ không đồng nghĩa xóa bỏ một nghề nghiệp

Một chức danh là một tập hợp công việc, quyền hạn và trách nhiệm trong một tổ chức. Tạo tệp cấu hình YAML, tra cứu cú pháp API, viết mã kiểm thử mẫu lặp lại (**boilerplate test**), phân loại nhật ký, điều tra nguyên nhân gốc rễ và quyết định chấp nhận rủi ro là những tác vụ hoàn toàn khác nhau. Tự động hóa được một tác vụ có thể giảm bớt công gõ phím, tăng số lượng thay đổi được đề xuất, hoặc chuyển dịch quỹ thời gian sang khâu đánh giá (**review**). Điều đó không tự động đồng nghĩa tổ chức sẽ tăng hay giảm nhân sự; bài toán còn phụ thuộc vào chi phí công cụ, nhu cầu sản phẩm, chi phí phối hợp nội bộ, quy định tuân thủ và mô hình tổ chức doanh nghiệp. Phần này là góc nhìn suy luận về thiết kế công việc, không phải số liệu thống kê tuyển dụng.

Sự thật khó chịu là việc ghi nhớ nhiều lệnh CLI và cấu trúc YAML không tạo nên một lợi thế bền vững nếu tác vụ chỉ đơn thuần là chuyển một yêu cầu đã rõ ràng thành cú pháp quen thuộc. Khi công cụ tự động làm tốt phần việc đó, người kỹ sư buộc phải cạnh tranh ở những tầng năng lực sâu hơn: xác lập

khế ước kỹ thuật (**contract**), nhận diện dữ liệu bị thiếu hụt, giới hạn bán kính ảnh hưởng sự cố (**blast radius**), hiểu vì sao một phản ví dụ (**counterexample**) có thể bác bỏ một phương án giải quyết và kiểm chứng thay đổi trong môi trường vận hành thực tế. Không phải ai cũng tự nhiên chuyển dịch được sang phần việc phức tạp hơn; rèn luyện những năng lực đó đòi hỏi thực hành độc lập, chứ không chỉ dừng lại ở kỹ năng viết câu lệnh gợi ý (**prompt**).

Cơ hội phát triển cũng không chỉ gói gọn trong việc "viết mã nhanh hơn". Một nền tảng phát triển nội bộ (**internal developer platform**) có thể cắt giảm các thao tác trùng lặp thông qua API chuẩn hóa, khế ước triển khai và chính sách kiểm soát có thể tự động kiểm tra. Kỹ nghệ độ tin cậy hệ thống (SRE) có thể ứng dụng tự động hóa để thu thập vết thực thi (**traces**), đối chiếu cấu hình và khoanh vùng sự cố. Hạ tầng tính toán cho AI đòi hỏi kỹ sư am hiểu mạng truyền thông, hạn ngạch tài nguyên, định danh bảo mật, chi phí hạ tầng và luồng luân chuyển dữ liệu. An ninh thông tin và ứng phó sự cố đòi hỏi phân biệt rõ giữa một đề xuất hữu ích với một hành vi làm xóa dấu vết kiểm toán hoặc leo thang đặc quyền trái phép. Những định hướng đó mang lại giá trị thực khi tổ chức thực sự có bài toán cần giải quyết, chứ không bảo đảm cứ biết Go là mặc nhiên có việc làm.

Ở cấp độ khởi đầu (**entry-level**), chúng ta có thể đặt một kịch bản giả định: nếu phần lớn tác vụ đầu vào dành cho người mới từng là viết mã mẫu lặp lại (**boilerplate**), thì nay người học sẽ có ít cơ hội cọ xát qua việc tự tay viết từng dòng mã. Đây là rủi ro trong đào tạo cần được thiết kế lại phương pháp học, chứ không phải một khẳng định rằng toàn bộ vị trí cho kỹ sư mới vào nghề (**junior**) đã hoàn toàn biến mất. Một cách ứng phó hiệu quả là duy trì các bài tập thực hành cá nhân độc lập ở quy mô vừa phải, yêu cầu dự đoán hành vi trước khi chạy, tự đánh giá mã nguồn do AI sinh ra bằng các phản ví dụ và tự điều tra các tình huống lỗi chưa có lời giải sẵn. Một hồ sơ năng lực (**portfolio**) chất lượng nên chứng minh được khả năng phản biện và bác bỏ phương án sai, thay vì chỉ trưng bày nhiều kho mã nguồn do AI tạo dựng.

Go nằm ở đâu trong bó công việc ấy?

Các chương trước đã cho ví dụ kiểm chứng được: CLI gom dữ liệu vận hành, HTTP service, controller Kubernetes, cloud SDK, GitHub automation và tool server MCP. Go có static types, build/test tooling thống nhất và standard library phù hợp với nhiều bài toán I/O. Chúng giúp tạo vòng phản hồi cho cả người lẫn Agent: compiler bắt một lớp lỗi, test bắt những contract đã được mô tả, race detector tìm access racy trên đường chạy được thực thi. Không công cụ nào thay bài toán chọn contract.

Đây là lựa chọn theo workload. Một CLI thuần Go có thể giảm phụ thuộc runtime khi phân phối, nhưng cgo, chứng chỉ, timezone, file config và quyền OS vẫn có thể tạo dependency ngoài binary. Service có GC cần budget memory và latency. Interface nhỏ giúp thay implementation, nhưng không diễn đạt tự động ownership, deadline hoặc retry semantics. Simplicity làm một số bước review dễ hơn; nó cũng để lại công việc thiết kế bằng convention và test thay vì một hệ kiểu kiểm tra mọi invariant.

Nếu task chủ yếu là khám phá dữ liệu và notebook với library ML sẵn có, Python có thể hợp lý hơn. Nếu cần một subsystem có yêu cầu ownership hoặc kiểm soát memory rất chặt, Rust hay C/C++ có thể đáng cân nhắc tùy hệ thống và năng lực team. Browser UI có ecosystem khác. Một hệ thống có thể dùng Go ở control plane và ngôn ngữ khác ở data plane; boundary protocol và lifecycle quan trọng hơn tranh luận một ngôn ngữ phải làm tất cả. Chương 14 đã cho một ví dụ nhỏ về chi phí interop, không một bảng xếp hạng ngôn ngữ.

Khi nút thắt chuyển từ viết sang kiểm chứng

Phần này là góc nhìn suy luận thiết kế của tác giả, không phải một kết luận đo lường từ các nghiên cứu thực nghiệm trên. Xét một kịch bản giả định: tác tử AI tạo ra nhiều bản vá mã nguồn (**patches**) hơn trong cùng một khoảng thời gian, nhưng người đánh giá (**reviewer**) vẫn phải kiểm tra khế ước kỹ thuật, gói phụ thuộc, ca kiểm thử bị bỏ sót, quyền triển khai và tác động trực tiếp đến người dùng cuối. Nếu hàng đợi chờ duyệt phình to, việc rút ngắn thời gian gõ phím chưa chắc đã giúp rút ngắn tổng thời gian từ lúc nhận yêu cầu đến khi thay đổi được phê chuẩn vào nhánh chính. Việc chỉ đếm số lượng bản vá hay số dòng mã ở đầu vào sẽ bỏ sót hoàn toàn điểm nghẽn cốt lõi đó.

Muốn kiểm chứng lợi ích thực tế, hãy định nghĩa rõ tiêu chí hoàn thành trước: thay đổi đã qua khâu duyệt của đúng gói phát hành (**artifact**), chạy trên đúng mục tiêu (**target**) được phép và thỏa mãn đầy đủ hậu điều kiện. Hãy đo lường riêng biệt thời gian chờ duyệt, thời gian kiểm thử, khối lượng công việc phải làm lại sau khi bị từ chối (**rework**), chi phí bản quyền công cụ và số lượng sự cố phát sinh; đồng thời so sánh giữa các tác vụ có cùng độ phức tạp và yêu cầu chất lượng. Tuyệt đối không dùng cờ "kiểm thử thành công" do chính bên tạo bản vá tự khai báo làm thước đo độc lập. Một đề xuất sai bị bác bỏ kịp thời ngay từ đầu là một kết quả hữu ích cho hệ thống, không phải lỗi hao phí năng suất cần giấu giếm.

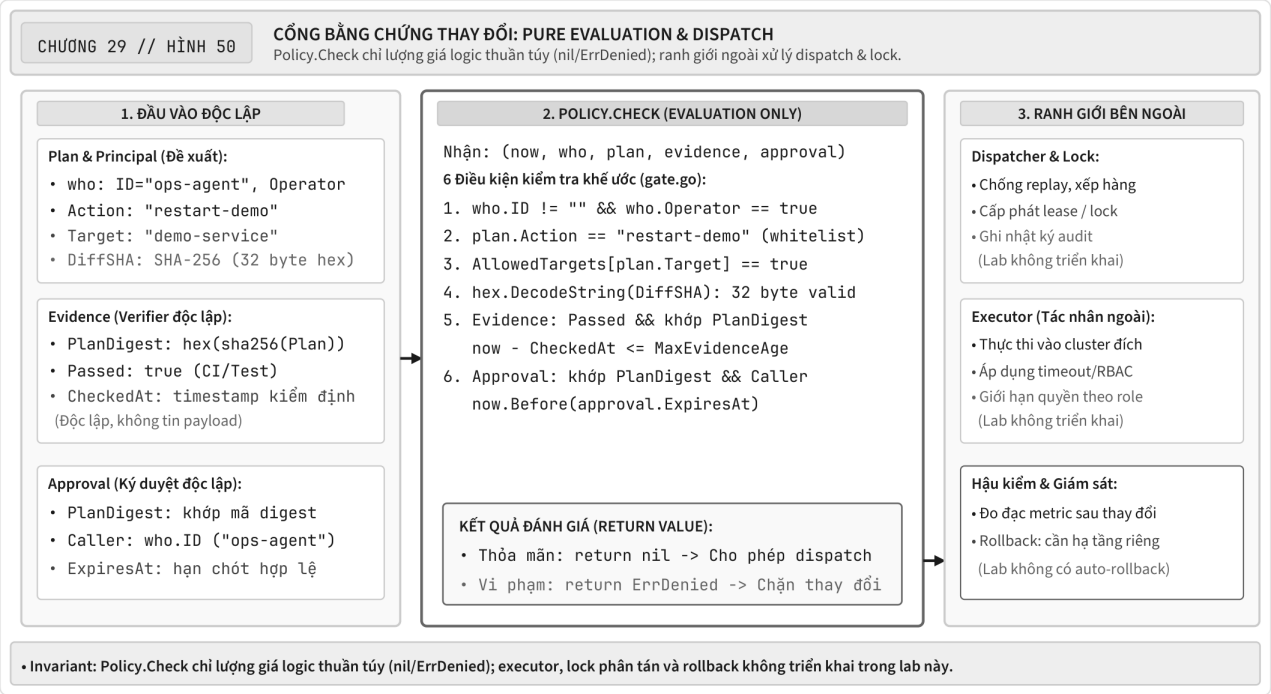
Bản triển khai báo xanh trên giao diện CI/CD chưa đồng nghĩa với kết quả mong đợi (**outcome**) đối với người dùng. Ví dụ: quá trình phát hành (**rollout**) không gây sập ứng dụng (**crash**), nhưng lại làm các yêu cầu phản hồi chậm hơn đáng kể hoặc trả về kết quả nghiệp vụ sai lệch: hậu điều kiện bắt buộc phải đánh giá đúng trải nghiệm người dùng thực tế, thay vì chỉ nhìn vào việc tiến trình có đang sống hay không. Chương 16 đã phân biệt các góc nhìn về khả năng quan sát; Chương 18 ràng buộc gói phát hành với quy trình thăng hạng môi trường. Tại đây, chúng ta đặt thêm câu hỏi về chi phí và điểm hoàn tất thực thụ của công việc, thay vì lặp lại một danh mục kiểm tra phát hành thuần túy.

Bài thực hành: Đánh giá phản biện ngắn

Một nhóm kỹ sư báo cáo: “Tác tử AI giúp tăng gấp đôi số lượng yêu cầu hợp nhất mã (Pull Request, tức PR) mỗi tuần”, nhưng hoàn toàn không có số liệu về tỷ lệ PR được chấp thuận, thời gian chờ duyệt hay sự cố phát sinh sau khi phát hành. Hãy chỉ ra ba chỉ số dữ liệu cần thu thập thêm trước khi đưa ra kết luận năng suất thực tế đã tăng. Không cần phủ nhận con số PR tăng lên; ta cần chỉ rõ con số đó chỉ đo lường nỗ lực đầu vào chứ chưa đo lường kết quả giá trị thực tế đầu ra.

Một cuộc điều tra trước khi cho phép restart

Xét một tình huống mô phỏng cục bộ dưới đây, không phải một sự cố xảy ra trên môi trường thực tế. Tác tử đọc nhật ký thấy lỗi quá thời gian chờ (**timeout**) rồi đề xuất khởi động lại dịch vụ `local-demo`. Nó viện dẫn bằng chứng kiểm thử đạt từ một phút trước đó. Nhưng trong chính phút đó, nội dung thay đổi (**diff**) đã bị sửa từ A sang B. Một chính sách kiểm soát chỉ kiểm tra hai cờ `testsPassed` và `approved` sẽ để phiên bản B đi qua bằng chính chứng cứ của phiên bản A. Khiếm khuyết ở đây không nằm ở việc mô hình AI viết sai câu lệnh; nó nằm ở chỗ hệ thống quản trị không ràng buộc bằng chứng kiểm định với đúng đối tượng đang được phê chuẩn.



Hình 50 — Đề xuất không mặc nhiên có quyền thực thi. Mỗi bước kiểm tra bắt buộc phải gắn chặt với đúng gói phát hành, mục tiêu áp dụng và danh tính bên gọi; các mũi tên diễn tả chính sách kiểm soát của kịch bản, không phải một giao thức mặc định bảo đảm an toàn tự động.

Hãy review policy lỗi này trước khi đọc lab:

```
if evidence.Passed && approved {
    return nil
}
return ErrDenied
```

Quy tắc đó thiếu mất định danh của bản thay đổi, thời hạn hiệu lực của bằng chứng, danh tính bên gọi (**caller**), mục tiêu tác động (**target**) và hành động cụ thể (**action**). Một cờ `Passed` do tác tử tự khai báo hoàn toàn không có giá trị như một bằng chứng độc lập. Việc thêm mã băm vào cùng gói dữ liệu tải trọng (**payload**) cũng không giải quyết được bài toán tin cậy: bên không đáng tin có thể ngụy tạo cả mã băm lẫn trạng thái kiểm thử thành công. Bộ kiểm chứng và kho lưu trữ phê duyệt bắt buộc phải nằm ngoài phạm vi ghi dữ liệu của tác tử. Chương 26 đã tách biệt định danh gói phát hành khỏi dữ liệu xuất xứ; Chương 28 đã tách biệt danh tính đã xác thực khỏi các tham số của công cụ. Ở đây chúng ta kết hợp hai ranh giới đó lại với nhau, chứ không xây dựng thêm một bộ khung công tác (**framework**) mới.

Lab: viết rào chắn từ contract, không từ lời giải

Mở tệp `labs/part29-change-evidence/gate_test.go` trước khi xem `gate.go`. Trong một bản sao thử nghiệm, tạm lược bỏ phần mã hiện thực tham chiếu rồi tự dựng giao diện lập trình (API) dựa theo ca kiểm thử. Một yêu cầu hợp lệ đòi hỏi: người vận hành (**operator**) đã được xác thực danh tính, hành động duy nhất được phép là `restart-demo`, mục tiêu cục bộ nằm trong danh sách cho phép (**allowlist**), mã băm SHA-256 hợp lệ của phần mã thay đổi, bằng chứng kiểm thử còn mới và đạt gắn liền với mã băm của kế hoạch thực thi, cùng một phê duyệt chưa hết hạn gắn với đúng bên gọi và kế hoạch đó. Thiếu bất kỳ điều kiện nào trong số này, cổng kiểm soát bắt buộc phải từ chối. Đây là chính sách mang tính sư phạm: giới hạn thời gian một phút chỉ là quy ước mẫu thử nghiệm, không phải tiêu chuẩn an ninh cứng nhắc cho môi trường sản xuất.

Mã băm tổng hợp (**digest**) bao gồm phiên bản mã hóa, hành động, mục tiêu tác động và mã băm của phần thay đổi mã nguồn. Phiên bản mã hóa bảo toàn ngữ nghĩa khi định dạng thay đổi; thuật toán SHA-256 chỉ giúp so khớp định danh chứ không tự kiểm tra tác động thực tế của mã thay đổi. Không thể lấy một chuỗi JSON có thứ tự khóa tùy ý rồi giả định mọi bên tạo dữ liệu đều sinh ra cùng một giá trị băm; bài lab sử dụng một cấu trúc dữ liệu với quy cách mã hóa cố định trong Go và duy trì khế ước đó xuyên suốt trong các bài kiểm thử. Trong hệ thống đa ngôn ngữ, việc định nghĩa một quy chuẩn mã hóa chuẩn tắc (**canonical encoding**) cần được xem như một giao thức kỹ thuật riêng biệt.

Chạy từ thư mục lab:

```
go test -v ./...
go vet ./...
go test -race ./...
```

Test phải từ chối diff đổi, target khác, action khác, caller khác, viewer, thiếu evidence, test lỗi, evidence cũ hoặc ở tương lai và approval hết hạn. Test deterministic kiểm tra cùng plan cho cùng digest và target đổi thì digest đổi.

Nó không chứng minh thuật toán hash không có collision; đó không phải điều một unit test nhỏ đo được.

Bản hiện thực tham chiếu trong bài lab không gọi vỏ lệnh hệ thống (**shell**), không thực sự khởi động lại dịch vụ và không nhận thông tin định danh xác thực (**credentials**) của môi trường sản xuất. Bằng chứng kiểm thử và phê duyệt ở đây là dữ liệu mẫu đáng tin cậy; trong hệ thống thực tế, nguồn gốc của chúng đòi hỏi cơ chế xác thực danh tính và chính sách lưu trữ bền vững. Việc phương thức `Check` trả về `nil` cũng chưa tự động ngăn việc một phê duyệt bị sử dụng lại hai lần, chưa khóa cứng gói phát hành để tránh bị tráo đổi giữa khâu kiểm tra và khâu thực thi, và chưa xác nhận trạng thái mục tiêu sau thay đổi hay cơ chế phục hồi (**rollback**). Những giới hạn này là chủ đích thiết kế: bài lab chỉ chứng minh khế ước kiểm tra cục bộ, không cấu thành một dịch vụ ủy quyền hoàn chỉnh sẵn sàng cho môi trường thực tế.

Bài thực hành: Khế ước phê duyệt dùng một lần

Một phê duyệt chỉ được phép sử dụng đúng một lần duy nhất. Hai tiến trình thực thi (**executors**) có thể đồng thời đọc được một phê duyệt hợp lệ. Hãy viết ca kiểm thử thất bại (**failing test**) cho hai tiến trình cùng tiêu thụ trước khi quyết định dùng khóa loại trừ tương hỗ (**mutex**) hay giao dịch cơ sở dữ liệu (**transaction**); chỉ duy nhất một bên được xác nhận quyền sử dụng thành công, bên còn lại phải bị từ chối. Nếu tiến trình thực thi bị sập sau khi đã giữ chỗ phê duyệt nhưng trước khi kịp hoàn tất hành động, chính sách hệ thống có cho phép thử lại không, và hành động đó có tính lũy kế an toàn (**idempotent**) hay không? Câu trả lời bắt buộc phải dựa trên phân tích hành vi thực tế của hệ thống, chứ không chỉ dựa vào việc đã đặt một khóa đồng bộ trong mã nguồn.

Gợi ý giải pháp (chỉ đọc sau khi đã tự làm)

Một cờ đánh dấu đã sử dụng lưu trong bộ nhớ RAM không giải quyết được bài toán giữa hai tiến trình độc lập. Khâu kiểm tra và khâu xác nhận quyền (**check and claim**) đòi hỏi một ranh giới nguyên tử (**atomic boundary**) trong kho lưu trữ dữ liệu đáng tin cậy; hành động diễn ra bên ngoài kho lưu trữ vẫn có thể để lại khoảng trống rủi ro khi tiến trình bị sập đột ngột. Hệ thống có thể cần máy trạng thái (**state machine**) với các pha rõ ràng, khóa định danh chống trùng lặp (**idempotency key**) hoặc quy trình điều tra hiện trạng trước khi cho phép thử lại. Mỗi quyết định thiết kế sẽ làm thay đổi kiểu hình lỗi (**failure mode**). Đây chính là tinh thần trách nhiệm của giao dịch ở Chương 13 và quy trình điều hòa trạng thái ở Chương 21, nay được áp dụng cho tự động hóa có sự tham gia của tác tử AI.

Năng lực nào cần luyện nếu không muốn chỉ làm người bấm duyệt?

Sự khác biệt giữa việc đánh giá mã nguồn (**code review**) thực thụ và việc chỉ đọc lướt qua nằm ở chỗ kỹ sư có khả năng thiết kế một phép thử phản biện để bác bỏ giả định. Với mã đồng thời (**concurrency**), hãy vẽ sơ đồ truy cập bộ nhớ và các cạnh quan hệ diễn ra trước (**happens-before**) thay vì mù quáng tin vào nhãn an toàn đa luồng (**thread-safe**). Với mạng máy tính, hãy theo dõi thời gian chờ qua từng chặng: phân giải DNS, thiết lập kết nối TCP, bắt tay mã hóa TLS, truyền nhận thân phản hồi; một mã trạng thái HTTP đơn thuần không thể phản ánh toàn bộ đường dẫn của một yêu cầu. Với Linux, hãy phân biệt rành mạch giữa tiến trình, bộ mô tả tệp, không gian tên (**namespaces**), hạn ngạch tài nguyên và dung lượng bộ nhớ được hoạch toán. Với hệ thống phân tán, hãy luôn đặt câu hỏi: trùng lặp dữ liệu, quan sát cũ kỹ, cơ chế thử lại và tác dụng phụ (**side effects**) xảy ra tại ranh giới nào. Đây là những năng lực kỹ thuật cốt lõi có thể chuyển giao giữa các công cụ, không hề phụ thuộc vào bất kỳ mô hình AI cụ thể nào.

Khả năng quan sát (**observability**) cũng không đơn thuần là dựng các bảng điều khiển đồ họa (**dashboards**). Một vết thực thi có đủ dữ liệu tương quan để phân biệt giữa thời gian chờ trong hàng đợi và thời gian chờ dịch vụ phụ thuộc hay không? Một chỉ số biến động do sự thay đổi của cơ cấu lưu lượng mạng hay do thay đổi trong mã nguồn? Tác tử AI có thể hỗ trợ viết câu truy

vấn và tóm tắt nhật ký; nhưng người kỹ sư vẫn là bên quyết định khế ước đo lường và bảo vệ dữ liệu nhạy cảm. Trong xử lý sự cố, tự động hóa tác vụ chỉ đọc dữ liệu có mức độ rủi ro hoàn toàn khác so với tự động hóa tác vụ xóa Pod hay chỉnh sửa tường lửa. Một hành động "khắc phục nhanh" thiếu kiểm soát có thể xóa sạch hiện trạng hiện trường cần phục vụ điều tra. Quyền hạn và chứng cứ bắt buộc phải tương xứng với mức độ tác động của hành động, không dựa trên vẻ tự tin trong văn bản của mô hình ngôn ngữ.

Một bài thực hành độc lập có thể đọng lại bằng một bộ hồ sơ kỹ thuật cô đọng: yêu cầu ta tự tay xác lập, phản ví dụ ta tìm ra, gói phát hành đúng phiên bản, ca kiểm thử ta đã chạy, phép đo lường có ranh giới rõ ràng, quyết định phân quyền và các hậu điều kiện ta trực tiếp quan sát. Đừng chỉ lưu lại những đoạn hội thoại dài với AI. Một hồ sơ kỹ thuật vững chắc cho phép người khác kiểm chứng lại toàn bộ lý do đằng sau mỗi quyết định, kể cả khi tác tử AI hay các thư viện đã thay đổi phiên bản. Đó chính là năng lực kiểm chứng có thể rèn luyện được, chứ không phải một ảo tưởng về sự miễn nhiễm trước những biến động của thị trường.

Kết thúc ở một quyết định có thể bảo vệ

Không có bất kỳ kết luận chắc chắn nào khẳng định "AI sẽ cướp mất nghề nghiệp" hay "kỹ sư giỏi sẽ hoàn toàn không bị ảnh hưởng" từ các tài liệu nghiên cứu hiện có. Các bằng chứng thực nghiệm cho thấy công cụ AI hỗ trợ tăng tốc một số tác vụ nhất định, nhưng lại làm chậm các tác vụ khác trong môi trường phức tạp, đồng thời việc đo lường năng suất khi công cụ liên tục thay đổi là bài toán nhiều thách thức phức tạp. Các dự báo việc làm chỉ mang tính diện rộng tại thị trường Hoa Kỳ, hoàn toàn không có một dự báo chuyên biệt nào đủ căn cứ khoa học cho thị trường kỹ sư Go hay SRE tại Việt Nam. Do đó, thông điệp cốt lõi của cuốn sách là một khuyến nghị có điều kiện: hãy tập trung học cách xây dựng và kiểm chứng hệ thống, tận dụng AI để mở rộng năng lực cá nhân nhưng luôn giữ quyền hạn, gói phát hành và kết quả đầu ra thành những đối tượng có thể kiểm toán minh bạch.

Khi có ai đó hỏi vì sao ta phê chuẩn một thay đổi trong hệ thống, câu trả lời thuyết phục không bao giờ là “tác tử AI đề xuất như thế” hay “toàn bộ kiểm thử đều báo xanh”. Câu trả lời đó phải là: dựa trên khế ước kỹ thuật nào, bằng chứng gắn liền với gói phát hành nào, trong điều kiện môi trường nào, do ai có thẩm quyền phê duyệt, và trạng thái hệ thống sau khi thực thi đã được quan sát, đo lường ra sao. Một cuốn bách khoa kỹ thuật thực chiến không dừng lại ở việc học thuộc thêm một thư viện. Nó dừng lại ở năng lực thấu suốt: biết rõ mình đang chứng minh điều gì, và điều gì vẫn còn là ẩn số chưa được kiểm chứng.

BACK MATTER — ATLAS MÃ NGUỒN 50 THƯ VIỆN GO DEVOPS & CLOUD

Điểm neo để đọc API và implementation đã ghim

Phụ bản này đứng sau Chương 29 và ngay trước Atlas Lỗi Go. Các nhận xét về implementation chỉ áp dụng cho version và commit ghi ở mỗi mục, không phải cam kết API cho mọi version. Khóa source chứng minh identity của checkout, không tự chứng minh mọi giải thích cơ chế; khi dùng một chi tiết để quyết định production, đọc code và test tại commit đó.

Phụ bản chọn 50 thư viện Go trong các lĩnh vực DevOps, cloud, networking, observability, IaC và hạ tầng Agent. Mỗi mục ghi version/commit được ghim để người đọc có thể đối chiếu. Mô tả API hoặc hợp đồng công khai không đồng nghĩa đã kiểm chứng mọi chi tiết implementation; câu nào nói đến cơ chế nội bộ phải có điểm neo source tương ứng ở bản đã ghim.

> Quy tắc kiểm chứng: không đoán version, tag, module path hay tên hàm. Một claim về implementation phải có source đã xác minh; cơ chế được neo vào type, struct hoặc file cụ thể. Mỗi mục phân tích cơ chế và ý nghĩa kỹ thuật, không dùng lời quảng bá thay cho bằng chứng.

PHẦN 1: NHÓM NỀN TẢNG TRỌNG YẾU (TIER S: 01–12)

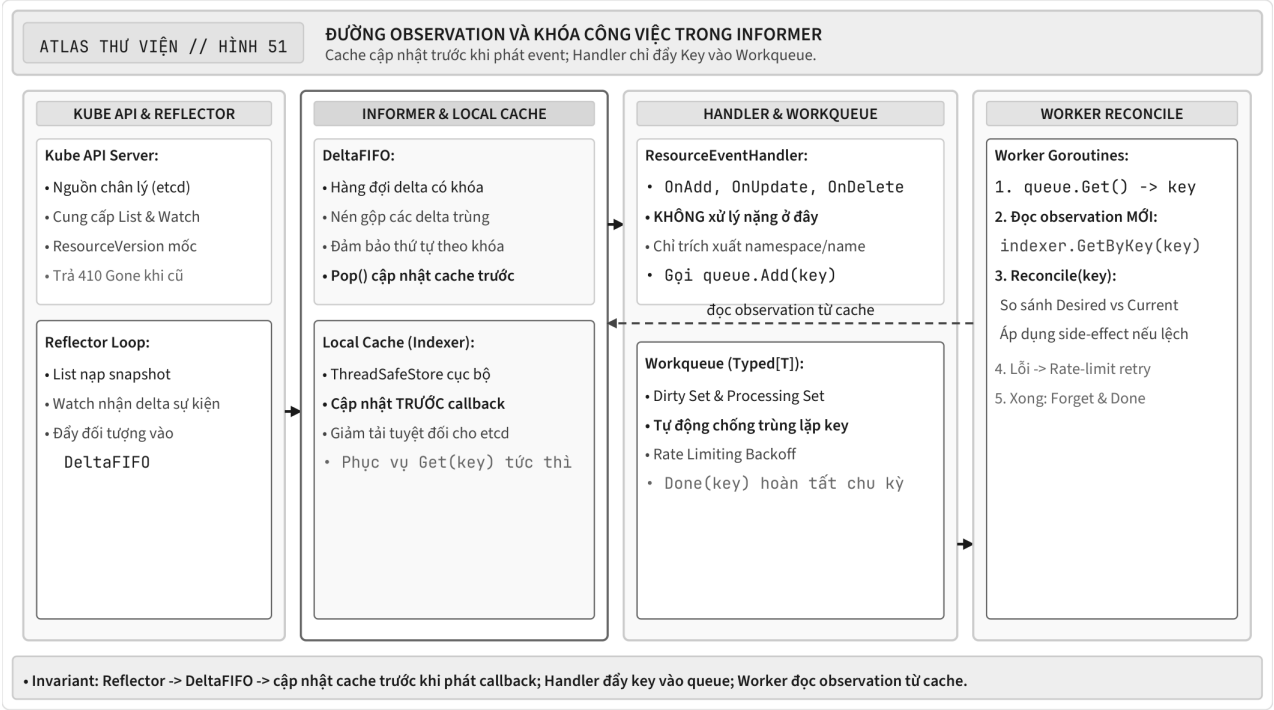
01. `k8s.io/client-go` **(v0.37.0 —** `28076445`**)**

Nếu controller lặp List toàn bộ Pod mỗi giây, nó tạo công việc tuần tự hóa và truyền snapshot lặp lại. Mức tải phụ thuộc số object, kích thước và API server; không suy ra CPU 100% hay cluster sập chỉ từ 500 Pod. Informer hỗ trợ duy trì observation local từ list/watch để nhiều consumer không phải tự polling toàn bộ tài nguyên theo cùng nhịp.

Reflector duy trì observation bằng list/watch và xử lý thay đổi resourceVersion cùng lỗi transport. Đường khởi tạo cụ thể có thể dùng List hoặc watch-list tùy version, capability và cấu hình. Watch là stream API

event, không phải trực tiếp theo dõi mọi thay đổi etcd; HTTP/2 cũng không dùng HTTP/1.1 chunked transfer encoding. Khi lịch sử không còn đủ hoặc stream lỗi, Reflector xử lý recovery theo source đã pin; không phải mỗi lần đứt stream đều bắt buộc List lại.

Trong đường dùng `DeltaFIFO` (`tools/cache/delta_fifo.go`), dữ liệu được gom theo khóa, không chuyển ngay tới handler người dùng. Các delta của một Pod có thể được giữ chung theo `namespace/name`. Vòng xử lý Informer rút delta rồi cập nhật cache trước khi phát event. `Indexer` ở `tools/cache/index.go` là interface; backing store tại `tools/cache/thread_safe_store.go` dùng `sync.RWMutex`. Lock ấy bảo vệ cấu trúc store, không làm object pointer trả về an toàn để sửa tại chỗ: caller phải coi object trong cache là chỉ đọc hoặc DeepCopy trước khi thay đổi. Đường queue được chọn còn phụ thuộc cấu hình/feature của phiên bản đã ghim.



Hình 51 — Informer cập nhật cache rồi phát event tới handler; handler của ứng dụng đưa khóa vào workqueue. Worker đọc observation hiện có khi điều hòa, không coi payload event là trạng thái cuối cùng.

Điểm neo kỹ thuật đáng giá nhất để học trong `client-go` nằm ở `util/workqueue/queue.go` (`type Typed[T]`). `TypedInterface[T]` là giao diện public; `TypedQueueConfig[T]` là cấu hình. Triển khai `Typed[T]` phối hợp hàng đợi có thể thay thế và hai tập khóa:

```
type Typed[T comparable] struct {
    queue      Queue[T]
    dirty      sets.Set[T]
    processing sets.Set[T]
    cond       *sync.Cond
    // Các trường lifecycle và metrics khác được lược bỏ.
}
```

Xét tình huống một worker đang bận xử lý khóa `"default/my-pod"` trong tập `processing`. Cùng lúc đó, 5 sự kiện cập nhật liên tiếp của Pod này được Informer gửi tới. `Add(item)` chỉ đánh dấu key trong `dirty` khi nó đang được xử lý, không đẩy thêm vào hàng đợi. Khi worker gọi `Done(item)`, nếu key vẫn `dirty`, nó được đẩy lại đúng một lần. Cơ chế khử trùng lặp này giới hạn việc xếp trùng một key; thứ tự và cấu trúc lưu trữ phụ thuộc triển khai `Queue[T]`. Worker thường lấy key rồi đọc snapshot hiện tại từ Indexer (snapshot này có thể trễ so với API server). Reconciliation lặp lại mới giúp hội tụ về trạng thái mong muốn.

Khi worker không chạy, kiểm tra lifecycle, error của Reflector và cache sync. Thiếu quyền list có thể ngăn initial population; thiếu quyền watch ảnh hưởng nhận cập nhật. Không suy ra `HasSynced` luôn `false` chỉ từ một lỗi watch, vì trạng thái sync còn phụ thuộc đường khởi tạo. Phải có cancellation/deadline và quan sát error thay vì chờ vô hạn.

---

02. `sigs.k8s.io/controller-runtime` (**v0.25.1** — `67b72c25`)

`controller-runtime` tổ chức cache, controller và lifecycle dùng chung, giảm phần wiring phải tự làm với `client-go`. `Reconcile(ctx, Request)` là contract điều hòa, không xóa trách nhiệm thiết kế retry, quyền, cleanup hay shutdown của ứng dụng. Không có một số dòng boilerplate cố định cho mọi controller.

Ở `pkg/manager/internal.go`, `controllerManager.Start` khởi động HTTP servers và webhooks trước caches, rồi mới các nhóm runnable phụ thuộc cache và leader election. Tín hiệu chỉ cancel context nếu caller đã thiết kế signal propagation. Shutdown đi qua stop procedure với timeout cấu hình; không hứa mọi worker luôn hoàn tất mọi side effect trước khi process dừng.

Ở commit này, đọc `pkg/client/client.go`, `Options.Cache` và `CacheOptions`, không tìm một `delegatingClient` trong file `split.go` của version cũ. Cấu hình rút gọn sau minh họa boundary đọc:

```
type CacheOptions struct {
    Reader       Reader
    DisableFor   []Object
    Unstructured bool
}
```

Client của manager thường dùng cache cho `Get`/`List` của type được cache. Nhưng `client.New` không có `Options.Cache.Reader` thì đọc API server; `DisableFor` và cấu hình unstructured còn có thể chọn đường trực tiếp. Lazy discovery/informer initialization cũng có thể phát sinh HTTP. Không suy ra một `Get` bất kỳ là "zero HTTP". Write đi tới API server; response thành công còn phải diễn giải theo operation/options, chẳng hạn dry-run không lưu thay đổi. Watch propagation và trạng thái workload thực tế là các boundary riêng, không được xác nhận chỉ bằng một write response.

`reconcile.Request` mang key thay vì snapshot object tại lúc phát event. Khi worker xử lý, object có thể đã đổi. Reconciler chọn reader phù hợp và đọc lại theo key; request không bắt buộc dùng cache và cũng không cung cấp freshness guarantee. Ví dụ dưới đây chọn `r.Get` qua client của reconciler:

```
var pod corev1.Pod
if err := r.Get(ctx, req.NamespacedName, &pod); err != nil {
    return ctrl.Result{}, client.IgnoreNotFound(err)
}
```

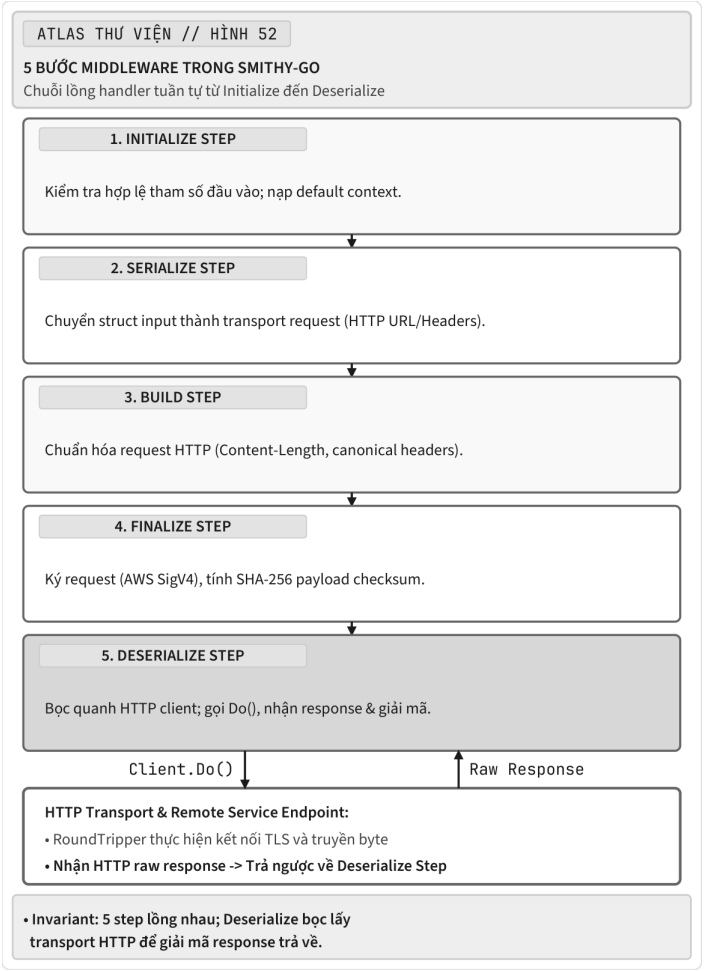
`client.IgnoreNotFound(err)` chỉ chuyển lỗi not-found thành `nil`; nó không làm các side effect trở nên lũy đẳng. Request theo key giúp reconciler đọc trạng thái khi xử lý, nhưng cache có thể stale. Muốn hội tụ an toàn cần thiết kế từng thao tác: kiểm tra ownership, dùng precondition khi ghi và quan sát lại sau lỗi hoặc retry. Khi cần độ mới của API server, chọn client đọc trực tiếp thay vì suy ra từ cache.

---

03. `github.com/aws/aws-sdk-go-v2` **(v1.47.0 —** `b189f382`**)**

Một lời gọi S3 được SDK serialize, resolve endpoint, ký khi operation yêu cầu và gửi qua HTTP. SigV4 dùng canonical request và credential; không phải mọi đường S3 đều băm toàn bộ body, vì có chế độ unsigned payload. Đọc `aws/signer/v4/middleware.go` và operation source để biết đường ký cụ thể.

SDK dùng `middleware.Stack` của module `github.com/aws/smithy-go`, không phải type khai báo trong `aws/middleware/stack.go`. Năm stage tổ chức request và response path như sau:



Hình 52 — Năm step tổ chức đường gọi vào bên trong. Deserialize gọi handler HTTP trước khi giải mã response; sơ đồ lược bỏ đường response đi ngược ra ngoài các step.

1. **Initialize:** Kiểm tra hợp lệ các tham số đầu vào và nạp giá trị mặc định vào ngữ cảnh.
2. **Serialize:** Đưa input operation vào request của transport; đường HTTP dùng `transport/http.Request` của smithy-go cho URL, method, headers và body.
3. **Build:** Chuẩn bị request, chẳng hạn content length và metadata phù hợp operation.
4. **Finalize:** Đường ký và retry được đặt ở đây trước transport; thứ tự middleware cụ thể phải đọc theo operation và cấu hình.

5. **Deserialize:** Đọc mã phản hồi HTTP, giải mã XML/JSON body thành struct kết quả, hoặc ánh xạ mã lỗi AWS (như `NoSuchKey`, `AccessDenied`) thành các struct lỗi cụ thể trong Go.

Xét scenario nhiều client gặp lỗi gần cùng lúc: backoff cố định theo cấp số nhân có thể làm các lần retry tiếp tục dồn thành đợt. `aws/retry/standard.go` ở version ghim dùng Full Jitter để phân tán thời điểm retry trong khoảng backoff. Jitter giảm đồng bộ thời gian, không tạo thêm capacity hay bảo đảm hạ tầng hồi phục; vẫn cần attempt budget, deadline và policy cho side effect.

Ở phiên bản được ghim `github.com/aws/aws-sdk-go-v2` v1.47.0, `retry.Standard` mặc định dùng rate limiter dạng quota với capacity 500 token. Khi `AWS_NEW_RETRIES_2026` không bằng `true`, cấu hình cũ tính `RetryCost` là 5 token cho retry thông thường và `RetryTimeoutCost` là 10 cho retry do timeout. Khi `AWS_NEW_RETRIES_2026=true`, `RetryCost` mặc định là 14 cho lỗi nhất thời, `ThrottlingRetryCost` là 5 cho lỗi throttling, còn `RetryTimeoutCost` không được dùng. Mỗi attempt thành công cộng `NoRetryIncrement` 1 token; retry thành công còn hoàn lại token đã lấy theo đường retry. Đây là hai bộ mặc định của cùng Standard retryer, không phải hai retry mode của SDK. Các cost, capacity và rate limiter đều có thể được cấu hình lại; khi quota không đủ, retry bị chặn. Vì thế không thể gán một chi phí token cố định cho mọi lỗi mạng hoặc mọi cấu hình AWS.

---

04. `github.com/prometheus/client_golang` **(v1.24.1 —** `d6087ee4`**)**

Nhiều goroutine cập nhật cùng counter có thể tranh chấp lock và cache line chứa state chia sẻ. Chi phí phụ thuộc workload, số writer, kiến trúc và đường đồng bộ; không hứa nó tăng theo một quy luật chung khi thêm core. Cần profile/benchmark trước khi chọn một primitive khác.

Trong `prometheus/counter.go` của version ghim, `type counter` giữ hai phần state và có hai đường cập nhật, không chỉ một vòng CAS cho mọi thao tác:

```
type counter struct {
    // valBits lưu giá trị float64 bằng bit representation
    valBits uint64
    valInt  uint64
    // ...
}
```

Bộ đếm tách giá trị ra làm hai thành phần: số nguyên (`valInt`) và số thực thập phân (`valBits`). Khi ta gọi hàm thông dụng nhất là `Inc()` (tương đương `Add(1)`), hàm nhận thấy giá trị tăng là số nguyên dương. Nó lập tức kích hoạt đường dẫn nhanh (fast path):

```
atomic.AddUint64(&c.valInt, 1)
```

`Inc` dùng `atomic.AddUint64`. Trong `Add`, nhánh integer kiểm tra `float64(uint64(v)) == v`; nếu không thỏa, chẳng hạn `0.15`, code cập nhật `valBits` bằng vòng CAS cho phép cộng float. Không gọi mọi giá trị nguyên là nhánh integer: khả năng biểu diễn còn quan trọng. Chi phí atomic vẫn phụ thuộc target, contention và cache coherence; không có số nanosecond cố định hay ranking "nhanh/chậm" cho mọi môi trường từ hai nhánh này.

Độ phức tạp tiếp theo nằm ở metric có nhãn động: `MetricVec` (`prometheus/vec.go`) dùng cơ chế tra cứu theo bộ giá trị nhãn và tái dùng metric đã tạo. Điều này không chứng minh mỗi lần tra cứu đều không cấp phát; chi phí thực tế cần đo với tập nhãn và đường gọi cụ thể.

Ở scrape path, `Gather()` thu thập metric families, còn handler chọn encoder theo format negotiation và ghi response. Gathering và encoding là hai bước khác nhau; chi phí cấp phát phụ thuộc số metric, tần suất scrape và encoder, không chỉ việc không dùng JSON.

---

05. `go.opentelemetry.io/otel` (v1.46.0 — 58db4c89)

Telemetry có overhead và cần budget. Export đồng bộ thêm thời gian chờ vào request path; queue không giới hạn có thể giữ quá nhiều memory khi backend lỗi. Không có tỷ lệ latency tăng gấp đôi phổ quát. Batch processor chọn queue, timeout, drop/block policy và shutdown behavior; đo overhead và loss theo workload thay vì coi instrumentation là miễn phí.

Ở version ghim, `batchSpanProcessor` trong `sdk/trace/batch_span_processor.go` dùng buffered channel có dung lượng giới hạn để chuyển span sang đường export. Đây là chi tiết implementation hữu ích để hiểu đầy queue và mất span, không phải lợi thế mặc định của channel so với mọi ring buffer:

```
type batchSpanProcessor struct {
    e             SpanExporter
    o             BatchSpanProcessorOptions
    queue         chan ReadOnlySpan
    dropped       atomic.Uint32
    // ...
}
```

Đoạn rút gọn dưới đây giữ nhánh enqueue/drop sau kiểm tra sampled span; source thật còn có context và instrumentation, không nên chép fragment này như một hàm nguyên vẹn của thư viện:

```
func (bsp *batchSpanProcessor) enqueueDrop(
    sd ReadOnlySpan,
) bool {
    select {
    case bsp.queue <- sd:
        return true
    default:
        bsp.dropped.Add(1)
        return false
    }
}
```

Trong đường không bật blocking, `select` có nhánh `default` làm enqueue không chờ khi queue đầy. Span có thể bị bỏ và bộ đếm `bsp.dropped` tăng.

Nhánh này chỉ giới hạn việc chờ dung lượng queue, không chứng minh cả lời gọi không có chi phí hoặc không tranh chấp. Bật blocking đổi trade-off; shutdown, exporter và timeout còn có contract riêng.

Ở đầu ra, một goroutine chạy nền (`bsp.processQueue`) lắng nghe hai tín hiệu: một timer chu kỳ (`batchTimeout`, mặc định 5 giây) hoặc kích thước lô gom tụ (`maxExportBatchSize`, mặc định 512 spans). Khi một trong hai điều kiện thỏa mãn, lô spans được rút ra và chuyển cho `Exporter.ExportSpans(ctx, batch)`.

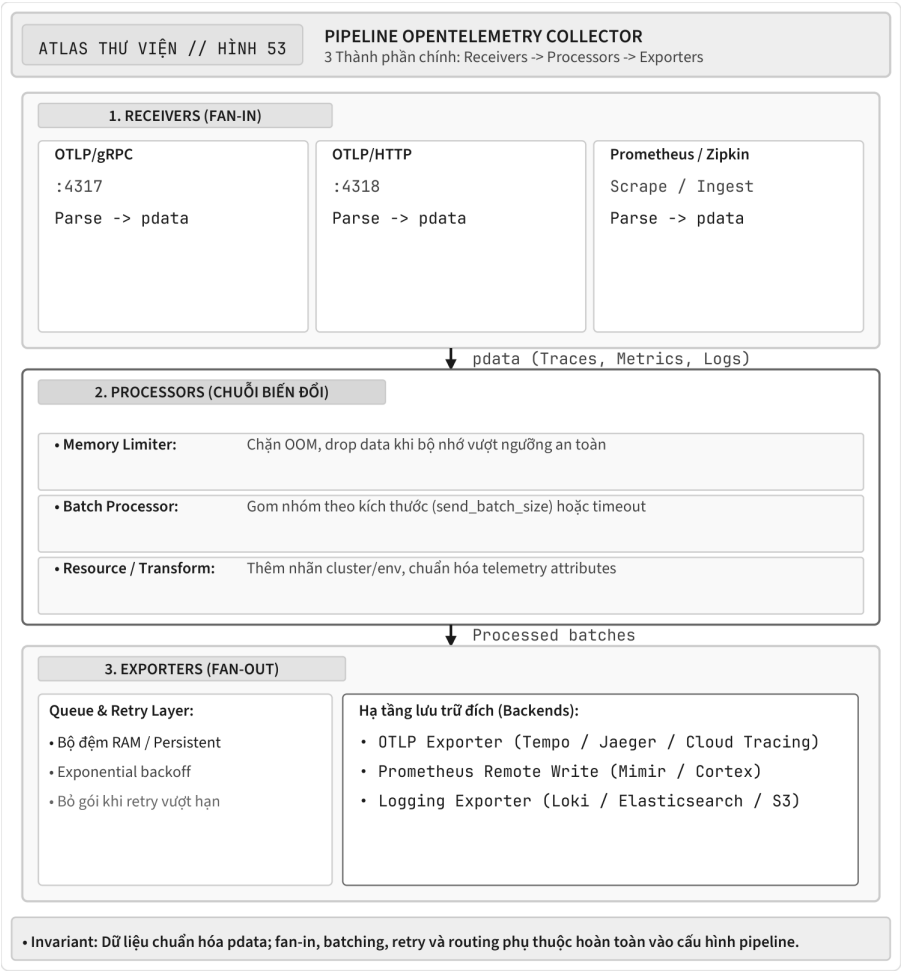
`TraceContext` (`propagation/trace_context.go`) extract/inject context theo W3C, trong đó có header `traceparent`. Consumer phải cấu hình và dùng propagator ở boundary thích hợp; đây không phải bảo đảm trace hoàn chỉnh. Khi tạo child span, TraceID nối cùng trace còn SpanID mới phân biệt span với parent. Truyền `context.Context` giúp mang quan hệ ấy qua các hàm, nhưng không tự tạo span, sample hoặc export dữ liệu.

---

06. `go.opentelemetry.io/collector` **(v0.161.0 —** `0bf928af`**)**

Nếu SDK đo telemetry trong tiến trình, Collector là đường tiếp nhận, xử lý và chuyển tiếp dữ liệu ở cấp hạ tầng. Một bản phân phối Collector có thể cấu hình receiver, processor và exporter để nối nhiều nguồn/đích; danh sách OTLP, Prometheus, Jaeger, Zipkin, Datadog, Elasticsearch hay S3 phụ thuộc component thực sự được đóng gói và cấu hình, không mặc định có trong core module.

Hợp đồng cấu hình Collector nối ba loại component trong pipeline:



Hình 53 — Pipeline nối receiver, các processor được cấu hình và exporter. Fan-in, fan-out, queue và retry phụ thuộc cách nối component cùng cấu hình cụ thể.

1. **Receiver:** Lắng nghe trên cổng mạng (gRPC/HTTP), chuyển đổi payload của các giao thức khác nhau về mô hình dữ liệu nội bộ chuẩn hóa mang tên `pdata` (`go.opentelemetry.io/collector/pdata`).
2. **Processor:** Áp dụng các quy tắc biến đổi: gom lô (`batch`), lọc dữ liệu (`filter`), hoặc lấy mẫu (`probabilistic_sampler`).
3. **Exporter:** Dịch `pdata` ngược lại định dạng của hệ thống đích và đẩy qua mạng.

`consumer.Capabilities` cho phép component khai báo nó có thể sửa dữ liệu. Đó là thông tin đầu vào cho việc xử lý chia sẻ dữ liệu ở fan-out, không phải một bảo đảm toàn pipeline không copy; khi thiết kế processor, caller phải tôn trọng hợp đồng ownership của component đang dùng.

Exporter helper có queue/retry policy tùy exporter và cấu hình. Persistent queue dùng storage extension khi được hỗ trợ và cấu hình; không phải mặc định mọi exporter đều có queue, cũng không phải RAM queue đầy sẽ tự spill xuống disk. Phải kiểm tra capacity, unit, overflow policy, retry timeout và storage của exporter đang dùng.

---

07. `github.com/moby/moby` **(v28.5.2 — 89c5e8fd)**

Trong đường Linux container đang xét, runtime tổ chức process với namespace, filesystem và cgroup theo cấu hình; nó dùng kernel của host, không tự có kernel riêng như VM. Không phải mọi container bật đủ cùng một tập namespace, và cgroup chỉ giới hạn những resource đã cấu hình. Moby điều phối các thành phần này; Docker trên host khác có boundary triển khai khác.

Khi mổ xẻ `daemon/daemon.go` (`type Daemon`), ta sẽ nhận ra Docker daemon không trực tiếp thực thi container. Nhiệm vụ chính của nó là quản lý trạng thái và phối hợp tài nguyên. Máy trạng thái vòng đời của một container được kiểm soát nghiêm ngặt tại `container/state.go` (`type State`):

```
type State struct {
    sync.Mutex
    Running           bool
    Paused            bool
    Restarting        bool
    OOMKilled         bool
    Dead              bool
    Pid               int
    ExitCodeValue     int
    ErrorMsg          string
    StartedAt         time.Time
    FinishedAt        time.Time
    // ...
}
```

Mutex của `State` được dùng làm lock chung với `Container`. Các cờ không luôn loại trừ nhau: paused process vẫn có thể Running; `Dead` không chỉ là tên khác của một process vừa nhận SIGKILL. Lock bảo vệ các state

transition theo implementation, không tự chứng minh toàn bộ lifecycle bên ngoài không có race.

Khía cạnh kỹ thuật ngoạn mục thứ hai của Moby là hệ thống tập tin phân lớp (Layered Filesystem) thông qua đồ thị driver lưu trữ `daemon/graphdriver/overlay2`. Khi ta khởi chạy một container từ image Ubuntu:

- Các layer của image được mount dưới dạng các thư mục chỉ đọc (`lowerdir`).
- Container được cấp một thư mục ghi duy nhất (`upperdir`).
- Kernel kết hợp chúng lại thành một thư mục ảo duy nhất (`merged`) bằng hệ thống tập tin `overlayfs`.

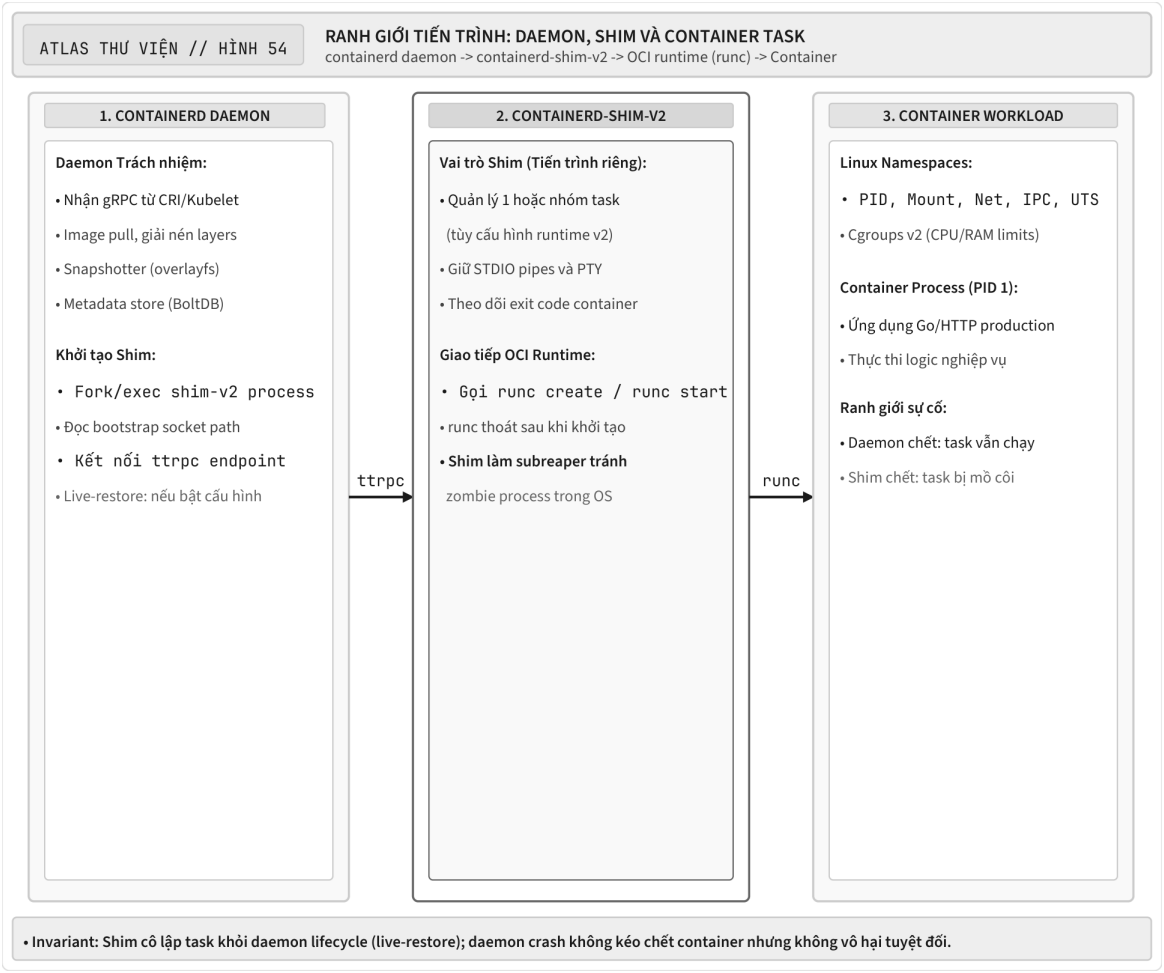
Với một file thường nằm trong layer chỉ đọc, OverlayFS có thể copy-up sang writable layer trước khi sửa, tùy operation và cấu hình filesystem. Không dùng `/etc/hosts` làm ví dụ mặc định vì runtime thường quản lý nó bằng mount riêng. Shared volume và bind mount có ownership khác image layer. Copy-on-write không chứng minh container isolation tuyệt đối hay một thời gian start cố định.

---

08. `github.com/containerd/containerd/v2` (v2.4.0 — `a7fe631d`)

Nếu daemon quản lý container gặp sự cố, vòng đời của task đang chạy có bắt buộc chấm dứt theo không? Câu trả lời phụ thuộc ranh giới giữa daemon và runtime shim, không phải một cam kết restart luôn êm cho mọi workload.

Câu trả lời nằm ở ranh giới daemon–shim của runtime v2; phía daemon được triển khai trong `core/runtime/v2/shim.go`. `containerd-shim-runc-v2` là một shim cụ thể, không đại diện mọi runtime.



Hình 54 — Daemon giao tiếp với shim qua endpoint; shim quản lý task qua runtime được chọn. Ranh giới tiến trình này không bảo đảm mọi failure của daemon đều vô hại với workload.

Daemon dùng shim process riêng để quản lý task. Ở commit được pin, `core/runtime/v2/shim.go` cho thấy containerd nạp shim, đọc bootstrap result và nối tới endpoint của shim. File này không chứng minh shim nào cũng trở thành subreaper, dùng đúng syscall `wait4` hay trực tiếp giữ mọi luồng I/O; muốn khẳng định các chi tiết ấy phải đọc implementation của shim được chọn.

Boundary giữa daemon và shim dùng endpoint có thể chọn ttrpc hoặc gRPC theo bootstrap result trong source đã pin; không có một transport hay mức tiết kiệm memory cố định cho mọi shim. Đường I/O, reaping và recovery phụ thuộc shim/runtime cụ thể.

Shim tách lifecycle task khỏi daemon containerd nên một số trường hợp daemon restart không buộc task dừng. Đây không phải bảo đảm workload không bị ảnh hưởng trong mọi failure; reattach, state storage và hoạt động

đang cần daemon có thể lỗi. Thời gian recovery cần đo trên version và cấu hình thật, không gán vài chục millisecond cho mọi host.

---

09. `github.com/hashicorp/terraform-plugin-framework` (v1.19.0 — `c7ac25e8`)

String Go có nhiều giá trị, trong đó chuỗi rỗng vẫn là giá trị hợp lệ; pointer có `nil` và các giá trị không `nil`. IaC cần biểu diễn thêm việc giá trị chưa biết tại plan time và null theo schema, thay vì dùng chuỗi rỗng cho tất cả trạng thái thiếu dữ liệu.

Hãy xem xét kịch bản sau: Kỹ sư viết một tệp cấu hình Terraform để tạo một mạng VPC mới, và trong cùng một kế hoạch đó, tạo một Subnet sử dụng thuộc tính `vpc_id = aws_vpc.main.id`. Khi chạy lệnh `terraform plan`, tài nguyên VPC chưa hề được tạo trên AWS, do đó `vpc_id` chưa hề tồn tại. Nếu Terraform gán giá trị đó bằng chuỗi rỗng `""` hoặc `nil`, các bước kiểm tra hợp lệ logic (validation) sẽ báo lỗi cú pháp sai, hoặc tệ hơn, provider sẽ gửi một request vô nghĩa lên cloud provider làm hỏng toàn bộ kế hoạch. Ngược lại, nếu coi nó là đã có giá trị, hệ thống sẽ hành xử sai.

Ở boundary dữ liệu Terraform, `terraform-plugin-framework` dùng các value biểu diễn được known, null và unknown, thay vì chỉ dùng một giá trị primitive Go không mang đủ trạng thái. Điều này không có nghĩa implementation không còn dùng kiểu Go bên trong. Các interface và kiểu value được xem tại `attr/value.go` và các kiểu liên quan:

```
type StringValue struct {
    state attr.ValueState
    value string
}
```

`types/basetypes/string_value.go` dùng `state` để phân biệt ba trạng thái:

- **Known (Đã biết):** Giá trị đã được xác định cụ thể (ví dụ `"us-east-1"`).

- **Null:** Giá trị vắng mặt theo type model; có thể đến từ cấu hình bỏ trống, biểu thức trả null hoặc logic provider. Không suy ra ý định người dùng chỉ từ `IsNull()`.
- **Unknown:** Giá trị chưa biết trong phase đang xét, chẳng hạn thuộc tính computed ở plan. Nó không chỉ áp dụng cho tài nguyên cloud hay luôn được giải quyết bởi một tài nguyên phụ thuộc.

`internal/fwserver/server.go` tổ chức các operation của framework; tầng server adapter cung cấp provider protocol tương ứng. Khi đọc `IsUnknown()` và `IsNull()`, câu hỏi cần trả lời là provider sẽ trì hoãn, giữ null hay tính một giá trị nào ở phase này. Gộp hai trạng thái thành chuỗi rỗng sẽ làm mất thông tin mà Terraform cần khi plan và apply.

---

10. `helm.sh/helm/v3` **(v3.22.0 —** `144ca65f`**)**

Một thay đổi kiến trúc của Helm 3 là bỏ daemon Tiller của Helm 2. Quyền Kubernetes của Tiller phụ thuộc ServiceAccount và RBAC được cấu hình; cấu hình quá rộng tạo rủi ro leo quyền cho người gửi lệnh tới Tiller.

Helm 3 bỏ Tiller và thực hiện action từ client. Quyền Kubernetes phụ thuộc cấu hình REST client và credential thực tế, có thể từ kubeconfig hoặc môi trường chạy service. Đây là thay đổi trust boundary, không phải lời hứa giải quyết mọi rủi ro quyền hạn; chart, hook và quyền client vẫn phải được review.

Không có Tiller thì release history được lưu ở đâu? Đây là đường cần lần theo khi điều tra rollback:

Một storage driver của Helm nằm ở `pkg/storage/driver/secrets.go` (`type Secrets`): khi chọn driver Secrets, release history được lưu trong Kubernetes Secret ở namespace triển khai, với nhãn nhận diện. Đây là đường mặc định phổ biến, không phải storage backend duy nhất Helm hỗ trợ:

```
name: sh.helm.release.v1.my-app.v1
labels:
  name: my-app
  owner: helm
  status: deployed
  version: "1"
```

Toàn bộ thông tin của bản phát hành (bao gồm source chart, file `values.yaml` đã hòa trộn, và toàn bộ chuỗi manifest YAML kết quả sinh ra từ `pkg/engine/engine.go`) được gom lại thành một struct `Release`, sau đó được tuần tự hóa JSON, nén bằng thuật toán gzip, mã hóa base64 và nhét trọn vẹn vào trường `data["release"]` của Secret.

Khi rollback, Helm đọc release history từ storage đã cấu hình và dùng action rollback để tạo revision mới, rồi cập nhật tài nguyên qua Kubernetes API. Đây không đơn thuần là giải nén một Secret, tính diff và Patch mọi tài nguyên. Helm 3 bỏ Tiller; quyền thực tế phụ thuộc credentials và RBAC của client đang gọi.

---

11. `github.com/go-git/go-git/v5` **(v5.19.2 —** `3eeb238d`**)**

Một image `scratch` không tự mang shell, git hay C runtime; các biến thể distroless có thành phần khác nhau và có thể mang library native. Nếu image không có executable git, gọi `exec.Command` không thể dùng nó. go-git là một lựa chọn library in-process; lựa chọn khác là đóng gói git phù hợp. Phải kiểm tra đúng image digest và dependency cần dùng, không suy ra từ nhãn “tối giản”.

`go-git` tách các thao tác object/packfile ở tầng plumbing khỏi các thao tác repository ở tầng porcelain. Phân tầng này giúp đọc đường đi của dữ liệu mà không cần gọi Git CLI:

Tầng plumbing (`plumbing/format/packfile/parser.go`, `packfile.go`) đọc packfile chứa các Git object, trong đó object delta tham chiếu một base object bằng offset hoặc object ID. Base không nhất thiết là "file của commit cũ"; đối tượng có thể là blob, tree, commit hoặc tag. Đường giải mã cần tìm base và áp dụng chuỗi delta để tái tạo object. Không suy ra toàn bộ object graph luôn được giữ trong RAM.

`plumbing/storer/storer.go` khai báo `EncodedObjectStorer`, tách thao tác với Git object khỏi backend lưu trữ. Hai lựa chọn cho ví dụ là:

- `storage/filesystem`: Lưu Git objects và references qua filesystem, thường ở `.git`; không suy từ tên backend rằng mọi tính năng Git CLI, như reflog, đều được hiện thực tương đương.
- `storage/memory`: Lưu trữ toàn bộ các object Git bên trong một mảng băm trên RAM (`storage/memory.Storage`).

```
r, err := git.Clone(memory.NewStorage(), nil, &git.CloneOptions{
    URL: "https://github.com/my-org/my-repo",
})
```

Đoạn code trên minh chứng cho tính linh hoạt của kiến trúc: Một controller GitOps có thể clone một repository, đọc nội dung file cấu hình YAML tại commit HEAD, tính toán diff, và kết thúc vòng lặp mà không lưu object storage của Git xuống ổ đĩa. `memory.Storage` tránh được chi phí filesystem I/O cho lưu trữ Git objects, nhưng vẫn tốn CPU cho hashing và giải nén delta, cùng với áp lực allocation/GC tương ứng với kích thước repository.

---

12. `golang.org/x/crypto/ssh` (v0.57.0 — `3f62bf11`)

Một `ssh.Client` có thể multiplex shell, SFTP và port-forwarding trên cùng một connection nếu các consumer dùng chung phiên ấy. Mở nhiều tab terminal độc lập thường tạo nhiều connection nếu không cấu hình chia sẻ. Channel của protocol cho phép nhiều luồng logic, không bảo đảm mọi ứng dụng SSH mặc định chỉ dùng một TCP socket.

Làm thế nào SSH có thể điều phối nhiều luồng dữ liệu khác nhau trên một socket mà không bị xáo trộn hoặc làm nghẽn lẫn nhau?

Mã nguồn tại `ssh/mux.go` (`type mux`) và `ssh/channel.go` (`type channel`) chứa đựng câu trả lời. Giao thức SSHv2 phân chia kết nối TCP duy nhất thành nhiều **kênh logic (logical channels)**. Mỗi khi ta gọi `client.NewSession()`, một gói tin `msgChannelOpen` được gửi qua mạng để đàm phán một kênh logic mới với ID riêng biệt. Bộ phân kênh `mux` lắng nghe luồng socket vật lý, đọc header của từng gói tin SSH và định tuyến các payload dữ liệu vào channel tương ứng.

Xét scenario một channel liên tục xuất dữ liệu trong khi consumer đọc chậm, còn channel khác dùng port forwarding. Cần phân biệt cửa sổ nhận riêng của channel với tài nguyên TCP dùng chung; không thể suy ra thông lượng hay thời gian chặn từ tên thao tác `cat /dev/urandom`.

SSH có flow control theo channel để receiver giới hạn lượng dữ liệu sender được gửi cho channel đó. Nó không loại bỏ contention của connection chung, TCP head-of-line blocking hay giới hạn memory của application; vẫn cần budget và xử lý consumer chậm.

- Mỗi kênh logic sở hữu một biến kích thước cửa sổ nhận dữ liệu độc lập (`myWindow uint32`).
- Bên gửi chỉ được phép truyền một lượng byte tối đa bằng đúng kích thước cửa sổ mà bên nhận đã cấp phép.
- Khi ứng dụng đọc qua reader nhận từ `session.StdoutPipe()`, nó giải phóng dữ liệu trong bộ đệm; mã nguồn trong `channel.go` gửi `msgChannelWindowAdjust` để cấp thêm hạn ngạch nhận byte cho bên gửi. Phải xử lý cả error mà `StdoutPipe()` trả về trước khi dùng reader.

Nếu consumer ngừng đọc và cửa sổ nhận của kênh đó hết, sender không thể tiếp tục gửi data cho riêng kênh ấy cho tới khi có điều chỉnh cửa sổ. Các kênh khác có cửa sổ riêng, nhưng vẫn chia sẻ cùng TCP connection và tài nguyên process; vì vậy không có bảo đảm chúng luôn "mượt mà" khi một consumer chậm.

PHẦN 2: NHÓM HỆ THỐNG CHUYÊN TRÁCH (TIER A: 13–30)

13. `github.com/open-policy-agent/opa` **(v1.20.2 —** `b2c26708`**)**

Một gateway cần budget cho authorization theo SLO và concurrency của chính nó. Không suy ra budget một millisecond chỉ từ request rate; đo policy, input, contention và end-to-end latency trước khi chọn cách evaluate.

Tại sao OPA có thể đánh giá các tập luật phức tạp viết bằng ngôn ngữ Rego nhanh hơn đáng kể so với phân tích cú pháp lại từ đầu?

OPA parse và compile policy/query thành các representation nội bộ rồi evaluate với input/data. Không mô tả mọi JSON data là một trie, cũng không khẳng định evaluation không thể chạy regex: policy có thể gọi builtin regex. Đọc đường evaluator và builtin của source đã pin khi cần hiểu chi phí thực tế.

Với policy/query ổn định được dùng lặp lại, cân nhắc `PrepareForEval` để tái dùng phần chuẩn bị. Tạo query mới cho mỗi request có thể lặp lại công việc parse/compile, nhưng lựa chọn phụ thuộc lifecycle và yêu cầu policy động. Prepared query không tự chứng minh mọi input đã được authorize đúng:

```
query, err := rego.New(
    rego.Query("data.authz.allow"),
    rego.Compiler(compiler),
).PrepareForEval(ctx)
```

`PrepareForEval(ctx)` tạo prepared query dùng lại cho các lần `Eval(ctx, rego.EvalInput(input))`. Evaluation vẫn thực hiện logic

policy và builtin, không chỉ đối soát con trỏ. Đo phần chuẩn bị riêng với evaluation khi benchmark; không gán một speedup chung.

---

14. `github.com/sigstore/cosign/v2` **(v2.6.5 — 3e82f50a)**

Khi kéo một container image `registry.internal/app:v1.2.0` về triển khai lên cụm Kubernetes sản xuất, làm sao ta có thể chứng minh với hệ thống kiểm toán rằng image này thực sự được sinh ra từ pipeline CI/CD chính thức của công ty chứ không phải do một hacker nội bộ sửa đổi đè lên registry?

Cosign hỗ trợ chữ ký tách rời gắn với image digest; ký số nói chung không bắt buộc sửa image. Cách lưu và tìm signature phụ thuộc artifact format và registry support của version được dùng.

Một đường lưu chữ ký kiểu tag dùng tên suy ra từ digest, chẳng hạn:

```
registry.internal/app:sha256-abc....sig
```

Chữ ký gắn với identity của image theo digest, không sửa các layer của image. Đường lưu và truy vấn còn tùy artifact format, OCI referrers và hỗ trợ registry ở bản Cosign đang dùng; ví dụ tag trên không phải định dạng bắt buộc cho mọi chữ ký.

Tính năng nổi bật của Cosign là chế độ **Ký Không Cần Quản Lý Khóa (Keyless Signing)**: Thay vì lưu trữ private key trên máy chủ CI/CD (nơi rất dễ bị lộ lọt), Cosign tích hợp với hai dịch vụ của Sigstore:

1. **Fulcio:** Cấp chứng chỉ ngắn hạn theo policy của CA và identity OIDC; đọc validity thực tế thay vì coi mọi chứng chỉ luôn sống đúng mười phút.
2. **Rekor:** Ghi nhận chữ ký vào một sổ cái nhật ký minh bạch bất biến (Transparency Log).

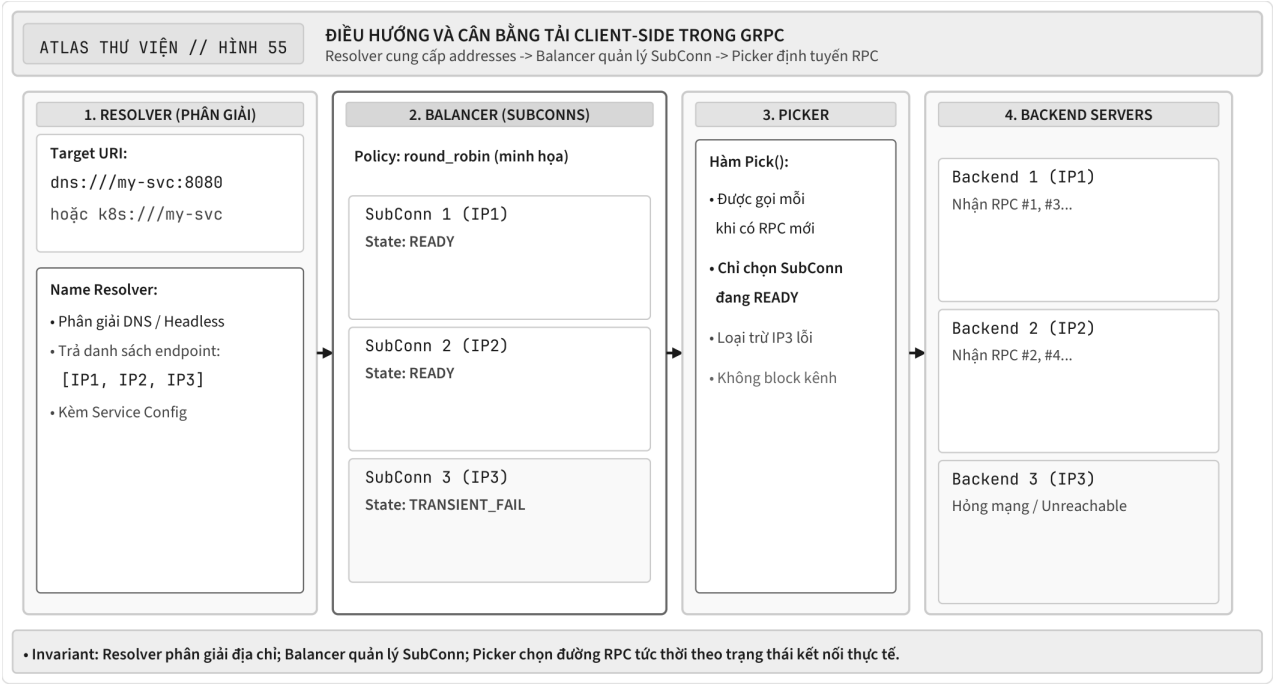
Trong keyless verification, verifier vẫn cần trust roots và policy identity/issuer, cùng các kiểm tra signature, certificate và transparency evidence của đường verify đang dùng. Một chữ ký hợp lệ không chứng minh image an toàn hay đến từ workflow được phép nếu identity chưa được đối chiếu.

15. `google.golang.org/grpc` (v1.84.0 — `e84aa5ab`)

Xét scenario có một gRPC client duy trì connection lâu tới Service có nhiều backend. Nếu connection đó được route vào một Pod, nhiều RPC trên nó có thể cùng tới Pod ấy, dù còn backend khác. Đây là ví dụ về granularity cân bằng tải, không phép đo CPU hay cam kết rằng backend chắc chắn sập.

Ranh giới là connection so với request: cân bằng L4 không tự phân phối từng RPC trên một connection tới backend khác. HTTP/1.1 cũng hỗ trợ keep-alive và connection reuse; vấn đề không riêng REST hay gRPC. Muốn phân phối RPC cần resolver biết các endpoint thích hợp và balancer policy phù hợp; resolver chỉ thấy một ClusterIP không tự nhìn thấy toàn bộ Pod.

Để cân bằng tải thực sự, `grpc-go` triển khai kiến trúc **Cân Bằng Tải Phía Máy Khách (Client-Side Load Balancing)** tại `clientconn.go` (`type ClientConn`) và `balancer_conn_wrappers.go`.



Hình 55 — Resolver cung cấp endpoint, balancer quản lý SubConn và picker chọn đường cho RPC. Ba backend cùng policy round_robin là cấu hình minh họa, không phải mặc định của mọi ClientConn.

Resolver cung cấp endpoint theo scheme và implementation. Với DNS headless service phù hợp, endpoint có thể là Pod IP; không hứa danh sách luôn mới hoặc đầy đủ. Policy `round_robin` có thể tạo SubConn tới endpoint và chọn đường ready cho RPC; nó không phải mặc định duy nhất.

SubConn là abstraction kết nối của balancer, không tên một type trong file transport cũ.

---

16. `google.golang.org/protobuf` **(v1.36.12 —** `cdd4c5f7`**)**

Protobuf dùng field number và wire type thay cho lặp tên field trên wire. Dung lượng và tốc độ so với JSON phải đo trên schema, value, encoder và workload; không có tỷ lệ 3–10 lần chung.

Để trả lời, hãy nhìn vào cách các byte được mã hóa trong `encoding/protowire/wire.go`. Protobuf không lặp tên trường như `"user_id":` hay `"is_active":` trên wire. Thay vào đó, mỗi trường dùng một tag nhị phân:

```
Tag = (Field Number << 3) | Wire Type
```

Ba bit cuối cùng xác định kiểu dây (`WireType`: Varint, 64-bit, Length-delimited, hoặc 32-bit), còn các bit phía trên chứa số thứ tự trường của struct.

Phép thuật tối ưu dung lượng của Protobuf nằm ở kỹ thuật mã hóa **Varint (Variable-length Quantity)**:

- Mỗi byte chỉ sử dụng 7 bit để lưu trữ dữ liệu số nguyên.
- Bit cao nhất (Most Significant Bit - MSB) là cờ báo hiệu: nếu MSB bằng 1, nghĩa là giá trị số còn kéo dài sang byte tiếp theo; nếu MSB bằng 0, đây là byte kết thúc của số đó.
- Value `5` dùng một byte varint, chưa tính tag. ZigZag áp dụng cho `sint32`/`sint64`; `int32`/`int64` âm không tự dùng ZigZag và có thể cần mười byte varint.

Runtime protobuf có các đường coder và metadata cho generated message, trong đó có thể dùng unsafe access ở implementation phù hợp. Reflection API vẫn tồn tại; không suy ra mọi decode chỉ là đọc memory hoặc đạt một giới hạn tốc độ phần cứng từ tên file `internal/impl/message.go`.

---

17. `github.com/google/go-containerregistry` (v0.22.1 — 8a72a424)

Xét scenario chỉ cần đọc metadata hoặc tìm một file trong image lớn. `docker pull` tải những layer cần mà local store chưa có; kích thước image đã giải nén không phải số byte phải truyền. Nếu công việc chưa cần layer content, một client đọc manifest/config riêng có thể tránh tải dư. Tìm file còn cần xét layer, whiteout và filesystem view, không chỉ thấy một path trong một tar bất kỳ.

`pkg/v1/image.go` định nghĩa interface `Image`; implementation `remote.Image` có thể trì hoãn việc đọc layer. Interface tự nó không bảo đảm mọi backend đều lazy:

```
type Image interface {
    // Rút gọn từ pkg/v1/image.go; còn các method khác.
    Manifest() (*Manifest, error)
    ConfigName() (Hash, error)
    RawConfigFile() ([]byte, error)
    Layers() ([]Layer, error)
    LayerByDigest(Hash) (Layer, error)
}
```

`remote.Image` cung cấp image handle với layer access trì hoãn; đọc metadata không đồng nghĩa tải toàn bộ layer. Số request có thể gồm auth, manifest/index và config theo operation; không khẳng định chỉ một GET hoặc một kích thước metadata cố định cho mọi image.

Layer cung cấp reader cho nội dung compressed hoặc uncompressed. Muốn dùng `tar.NewReader`, dùng luồng tar đã giải nén, không đưa gzip byte trực tiếp cho parser tar. Có thể dừng sớm và đóng reader, nhưng bandwidth tiết kiệm phụ thuộc vị trí file, compression và buffering; không có một tỷ lệ từ 20GB xuống vài trăm KB chung cho mọi scan.

---

18. `oras.land/oras-go/v2` (v2.6.2 — 105715ee)

OCI Distribution quy định API trao đổi manifest và blob, trong đó digest hỗ trợ định danh nội dung. Authentication, authorization, backup và phân phối nhiều vùng là khả năng hay policy của registry cụ thể, không phải tất cả đều có sẵn vì nó tuân theo OCI.

Tại sao chúng ta phải xây dựng các hệ thống lưu trữ riêng biệt cho Helm charts, tệp nhị phân WebAssembly (Wasm), tệp cấu hình Terraform, file chữ ký điện tử hay danh mục thành phần của phần mềm (SBOM - Software Bill of Materials), trong khi có thể lưu trữ toàn bộ chúng trực tiếp trên OCI Registry có sẵn của doanh nghiệp?

ORAS (OCI Registry As Storage) hiện thực hóa tầm nhìn đó thông qua mã nguồn tại `registry/remote/repository.go` và `content/oci/oci.go`. Trọng tâm kiến trúc của ORAS là interface `Target`:

```
type Target interface {
    content.Storage
    content.TagResolver
}
```

ORAS hỗ trợ đóng gói artifact với mediaType mô tả nội dung và liên kết qua `subject`. Ví dụ Wasm hay SBOM cần được thử với registry đích: hỗ trợ mediaType, manifest, referrers và policy có thể khác nhau. API cho phép diễn đạt một artifact không bảo đảm mọi registry chấp nhận nó.

`subject` và referrers hỗ trợ liên kết artifact. Lifecycle xóa/garbage collection phụ thuộc registry và policy; ORAS không tự bảo đảm xóa một image sẽ đồng bộ xóa mọi signature/SBOM liên quan.

---

19. `github.com/containernetworking/cni` **(v1.3.1 — `3f51e880`)**

Trong đường CNI thông thường của Pod không dùng hostNetwork, runtime chuẩn bị network namespace rồi plugin thiết lập network theo cấu hình. Không coi mọi Pod đều có namespace mới không interface: loopback, hostNetwork và plugin implementation tạo các trường hợp khác. CNI là contract giữa runtime và plugin, không một topology veth duy nhất.

Kubelet gắn Pod vào mạng lưới cluster bằng cách nào?

Đọc `pkg/skel/skel.go` với `PluginMainWithError`: CNI mô tả contract thực thi plugin process, không phải RPC qua gRPC. Cần phân biệt input điều khiển, cấu hình stdin và output/error mà runtime nhận.

Khi cần cấp phát mạng, container runtime gọi trực tiếp file thực thi của plugin CNI (ví dụ `/opt/cni/bin/bridge` hoặc `calico`) thông qua `pkg/invoke/raw_exec.go`. Mọi tham số điều khiển được truyền qua đúng hai kênh:

1. Các biến môi trường:

- `CNI_COMMAND`: Hành động cần làm (`ADD`, `DEL`, `CHECK`, hoặc `VERSION`). - `CNI_CONTAINERID`: Định danh của container. - `CNI_NETNS`: Đường dẫn tới file namespace mạng của Pod trong hệ thống (ví dụ `/proc/12345/ns/net`). - `CNI_IFNAME`: Tên interface mà runtime yêu cầu plugin tạo trong namespace; không phải một hằng số bắt buộc `eth0`.

2. Dữ liệu cấu hình mạng dạng JSON được đẩy trực tiếp qua luồng nhập chuẩn: `os.Stdin`.

Với bridge plugin, một đường triển khai có thể tạo veth pair, nối một đầu vào bridge host, đưa đầu kia vào namespace và đặt tên theo `CNI_IFNAME`, rồi dùng IPAM/configure routes. Plugin khác có thể dùng topology khác. Kết quả JSON và lifecycle command phải theo CNI spec; ví dụ veth không định nghĩa toàn bộ chuẩn.

---

20. `github.com/cilium/ebpf` **(v0.22.0 —** `e55144e1`**)**

Để quan sát syscall hay xử lý packet trong Linux, đã có nhiều cơ chế như audit, ptrace, ftrace, packet capture và kernel module, với điểm quan sát và chi phí khác nhau. eBPF bổ sung cách nạp chương trình vào hook được kernel hỗ trợ. Chọn nó theo dữ liệu cần thu, quyền, kernel và workload, không từ một so sánh chỉ có hai lựa chọn cực đoan.

Với object eBPF đã được biên dịch trước, `cilium/ebpf` cho phép Go loader nạp và gắn chương trình mà không cần cgo hoặc Clang trên máy đích. Máy đích vẫn cần Linux kernel, feature, quyền và resource limit tương thích; đường build object cần toolchain riêng.

`prog.go` (`type ProgramSpec`) và `map.go` (`type Map`) mô tả chương trình và map; các tầng nội bộ gọi Linux `bpf` syscall khi nạp hoặc thao tác tài nguyên. Không đồng nhất hai file API này với vị trí của lời gọi syscall. Sau

khi vượt qua kernel verifier, chương trình có thể được gắn vào hook phù hợp như XDP, TC hoặc kprobe.

Đường kernel → Go có thể dùng BPF ring buffer. `ringbuf/reader.go` (`type Reader`) tạo poller cho file descriptor của map; backend không-Windows tại `ringbuf/reader_other.go` dùng epoll để chờ dữ liệu. `Read` lấy từng record theo API, không phải lời hứa "batch read" cho mọi caller. Throughput và mức tiêu thụ tài nguyên phụ thuộc hook, kích thước event và workload; không suy ra speedup từ riêng cơ chế chờ.

---

21. `github.com/vishvananda/netlink` **(v1.3.1 — `17daef60`)**

Gọi ip bằng subprocess là một dependency vào executable, quoting/arguments và lifecycle process. Một CNI plugin có thể dùng library netlink để bỏ các lượt spawn đó. Chi phí phải đo theo số operation và workload; không tự suy ra bảng process bị quá tải chỉ vì code dùng CLI.

Với các thao tác link của ví dụ, `netlink` gửi message tới kernel Linux qua Netlink thay vì khởi chạy executable `ip`. Đọc `netlink_linux.go` và `link_linux.go`; không suy ra cả ứng dụng không còn subprocess hay mọi thao tác mạng đều là Netlink.

Ở tầng transport Netlink bên dưới, việc mở socket có dạng khái niệm:

```
fd, err := unix.Socket(
    unix.AF_NETLINK, unix.SOCK_RAW, unix.NETLINK_ROUTE,
)
```

Khi gọi `netlink.LinkAdd(&netlink.Veth{...})`, thư viện tạo Netlink request và thực thi qua `NETLINK_ROUTE`, không chạy shell. `link_linux.go` cho thấy `RTM_NEWLINK` và `req.Execute`; các chi tiết mở socket và syscall gửi nằm ở tầng transport bên dưới, không phải đoạn code chép nguyên từ `LinkAdd`.

Kernel trả kết quả hoặc lỗi theo Netlink protocol. Bỏ subprocess thay đổi đường đi và dependency, nhưng thời gian tạo veth và speedup so với CLI cần benchmark theo host, namespace và operation; không có con số vài phần mười millisecond hay hàng trăm lần chung.

22. `github.com/crossplane/crossplane-runtime` **(v1.20.11 — 84fc49a3)**

Kubernetes vốn được thiết kế để điều phối container trên một cụm máy chủ cục bộ. Nhưng triết lý điều hòa (Reconciliation loop) của Kubernetes xuất sắc đến mức người ta muốn dùng nó để quản lý toàn bộ thế giới điện toán đám mây: tạo database AWS RDS, cấp phát Google Cloud Storage, hay cấu hình Azure Virtual Network.

Làm thế nào để biến một API REST bất đồng bộ của AWS thành một đối tượng điều hòa tuần hoàn chuẩn mực của Kubernetes?

`crossplane-runtime` đặt hợp đồng với tài nguyên bên ngoài tại `pkg/reconciler/managed/reconciler.go`. `ExternalClient` là alias của `TypedExternalClient[resource.Managed]`; đoạn sau rút gọn interface generic ở commit đã pin:

```
type TypedExternalClient[T resource.Managed] interface {
    Observe(
        ctx context.Context, mg T,
    ) (ExternalObservation, error)
    Create(
        ctx context.Context, mg T,
    ) (ExternalCreation, error)
    Update(
        ctx context.Context, mg T,
    ) (ExternalUpdate, error)
    Delete(
        ctx context.Context, mg T,
    ) (ExternalDelete, error)
    Disconnect(ctx context.Context) error
}
```

Đọc bốn trách nhiệm sau trong reconciler cùng management policy; không coi đây là bốn lời gọi luôn chạy tuần tự ở mọi reconcile:

1. **Observe (Quan sát):** Reconciler định kỳ gọi `Observe()` để thăm dò trạng thái thực tế của tài nguyên trên cloud provider.
2. **Late initialization:** Provider có thể đưa giá trị quan sát vào field spec chưa được đặt, theo logic resource và management policy. Phải kiểm tra field

nào thuộc quyền provider; không tự suy rằng mọi default luôn giữ nguyên ý định người dùng.

3. **Reconcile State (Điều hòa sai lệch):** Nếu tài nguyên chưa tồn tại, nó gọi `Create()`. Nếu tài nguyên đã tồn tại nhưng cấu hình bị sai lệch (drift) so với file YAML khai báo, nó gọi `Update()` để kéo trạng thái cloud về đúng mong muốn.
4. **Delete & Finalizer:** Khi managed resource được yêu cầu xóa, policy và finalizer điều phối cleanup tài nguyên ngoài cụm. Xóa file YAML local không tự là API delete, và managed object không phải CRD definition. Management policy có thể chọn orphan thay vì xóa; đọc đường `ShouldDelete` cùng observe/delete để hiểu lúc finalizer được gỡ.

Crossplane có thể dùng Kubernetes làm control plane cho các tài nguyên mà provider và policy hỗ trợ. Không phải mọi dịch vụ cloud hay mọi field đều được quản lý theo cùng semantics; ownership, late initialization và deletion policy cần đọc theo managed resource.

---

23. `github.com/fluxcd/pkg/runtime` (**runtime/v0.114.0** — `a1797f9a`)

Khi một hệ thống GitOps tự động hóa triển khai phần mềm cho hàng trăm microservices, tình huống sự cố tồi tệ nhất là: Một commit cấu hình bị sai cú pháp, việc đồng bộ thất bại, nhưng người vận hành không hề hay biết và phải bỏ ra hàng giờ đồng hồ đào bới qua hàng chục nghìn dòng log của pod controller để tìm nguyên nhân.

Helpers trong `runtime/conditions/setter.go` giúp controller cập nhật condition theo cách nhất quán. Condition là thông tin controller báo về quá trình xử lý, không biến mọi tài nguyên thành một state machine có quan sát đầy đủ. Vẫn cần định nghĩa trạng thái nào có ý nghĩa và ai chịu trách nhiệm ghi status.

Helper condition của Flux dùng các kiểu condition cho những resource triển khai contract tương ứng; nó không áp đặt `.status.conditions` lên mọi Custom Resource trong Kubernetes. Ví dụ:

```
conditions.MarkTrue(
    obj, meta.ReadyCondition,
    "ReconciliationSucceeded",
    "Applied revision: %s", revision,
)
```

Nếu việc đồng bộ gặp sự cố (ví dụ lỗi xác thực SSH với GitHub), controller không chỉ ghi log ra màn hình console, mà gọi:

```
conditions.MarkFalse(
    obj, meta.ReadyCondition,
    "AuthenticationFailed",
    "Invalid SSH private key",
)
```

Helper cập nhật condition trên object trong memory. Controller còn phải ghi status qua Kubernetes API; helper không tự ghi etcd. Cột hiển thị của `kubectl get` phụ thuộc CRD printer columns; muốn xem Reason/Message có thể cần đọc status chi tiết.

`ObservedGeneration` ghi generation của spec mà writer status đã xử lý. So sánh với metadata.generation giúp phát hiện status chưa theo kịp spec; nó không chứng minh cache không stale hoặc toàn bộ reconciliation đã thành công.

---

24. `github.com/google/go-github/v92` **(v92.0.0 — 5149b4d7)**

Khi viết một con bot tự động hóa GitHub Actions hoặc công cụ dọn dẹp các pull request cũ trong một tổ chức doanh nghiệp có hàng nghìn repositories, ta sẽ phải đối mặt với bài toán phân trang (pagination) và giới hạn tần suất gọi API (Rate Limiting).

Với REST endpoint phân trang theo page trong ví dụ, GitHub chỉ đường tiếp theo bằng HTTP header `Link`, chẳng hạn

`<https://api.github.com/...page=2>; rel="next"`. Không lấy loop theo page này áp cho mọi endpoint hay cursor API.

`go-github` có đường offset pagination trong `github/github.go`. Nhiều hàm list nhận `ListOptions` và trả `*github.Response`; một số API dùng cursor hoặc dạng phân trang khác, nên ví dụ sau chỉ dành cho endpoint dùng page number:

```
opt := &github.PullRequestListOptions{
    ListOptions: github.ListOptions{PerPage: 100},
}
for {
    prs, resp, err := client.PullRequests.List(
        ctx, "my-org", "my-repo", opt,
    )
    if err != nil {
        return err
    }
    // Xử lý prs...
    if resp.NextPage == 0 {
        break // Đã duyệt hết tất cả các trang
    }
    opt.Page = resp.NextPage
}
```

Trong `github.go`, `Response.populatePageValues` đọc quan hệ của header `Link`. Loop theo page dưới đây dùng `resp.NextPage`, dừng khi không có page-number tiếp theo (`0`). Response còn có thông tin cursor cho các kiểu pagination khác. SDK parse header không thay việc kiểm tra error, quota, deadline hoặc sự thay đổi dữ liệu giữa các trang.

Struct `github.Response` công khai các trường `Rate: Rate{Limit, Remaining, Reset}`, cho phép các worker tự động điều chỉnh tốc độ gọi hoặc chủ động tạm dừng (`time.Sleep`) cho đến thời điểm `Reset` khi nhận thấy hạn ngạch gọi API của GitHub sắp cạn kiệt.

---

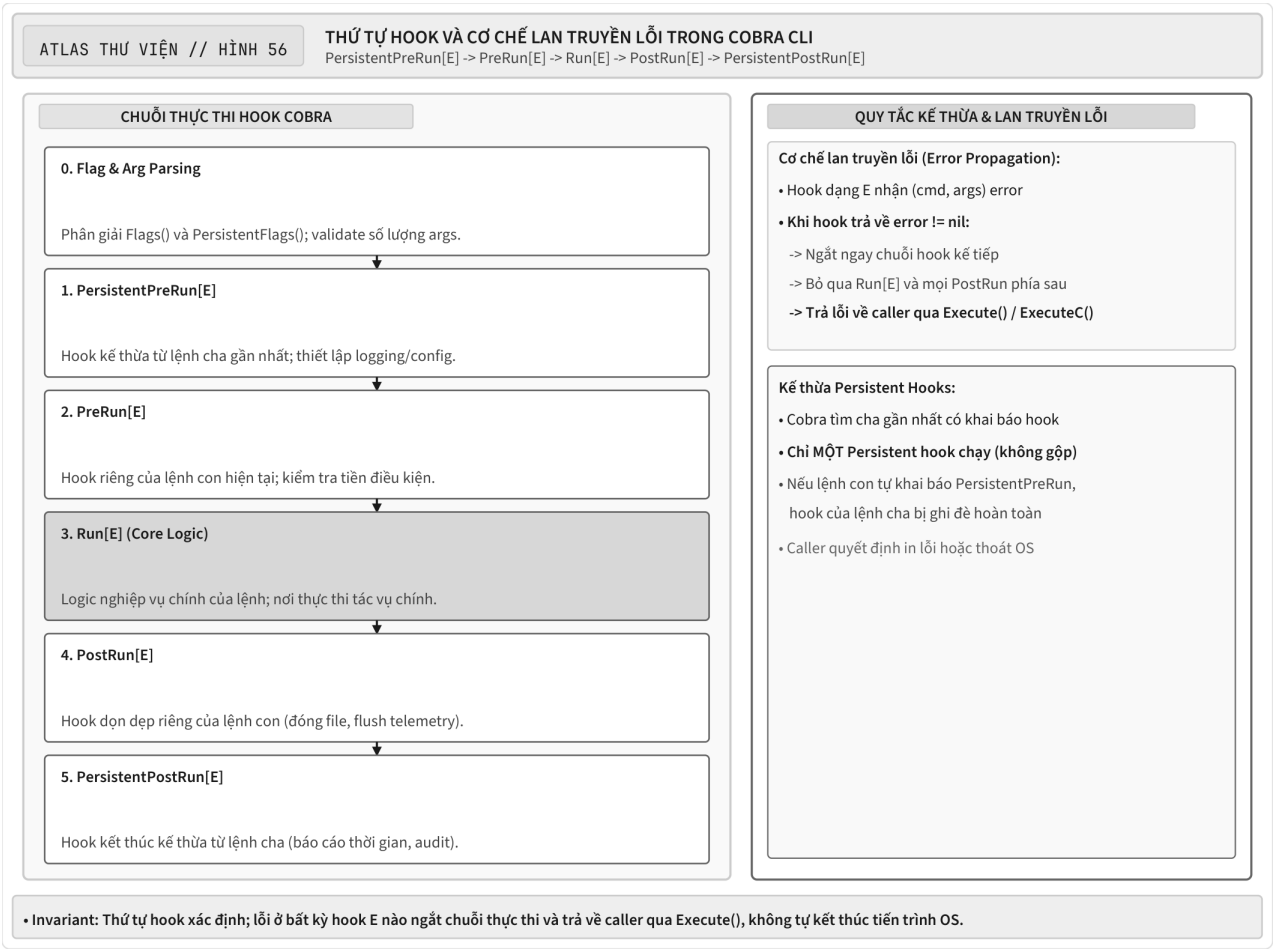
25. `github.com/spf13/cobra` (v1.10.2 — 88b30ab8)

Một số CLI Go như `kubectl`, `helm`, `gh` và `hugo` dùng Cobra để tổ chức lệnh phân cấp, flags và help. Chúng có các quy ước tương tự, không phải mọi CLI Go đều dùng cùng framework hay có hành vi giống nhau:

`cobra` trong `command.go` tổ chức command, flag và các điểm hook. Hãy đọc đường Execute thay vì suy từ trải nghiệm CLI rằng mọi lỗi đã được framework xử lý.

Cobra tổ chức toàn bộ ứng dụng CLI dưới dạng cây lệnh phân cấp (hierarchical command tree). Mỗi lệnh là một nút trên cây sở hữu con trỏ tới lệnh cha (`parent *Command`) và danh sách các lệnh con (`commands []*Command`).

Trong đường command execution của version ghim, các hook được chọn có thứ tự sau; hook có thể vắng, bản `E` có thể trả lỗi làm dừng đường chạy:



Hình 56 — Chỉ các hook được chọn mới chạy; hook có thể vắng. Lỗi từ hook E hoặc RunE ngắt đường chạy trước các hook phía sau.

Cobra phân biệt `Flags()` của lệnh với `PersistentFlags()` có thể kế thừa xuống cây lệnh. Lệnh con có thể khai báo flag trùng tên hoặc đổi behavior; đọc resolution thực tế thay vì mặc định một flag luôn có cùng nghĩa ở mọi cấp.

`PersistentPreRun` có thể đặt tác vụ khởi tạo chung ở lệnh cha. Cần kiểm tra cách Cobra chọn hook theo cây lệnh ở version đang dùng: hook của cha không phải cam kết sẽ luôn chạy nếu lệnh con khai báo hook tương ứng.

---

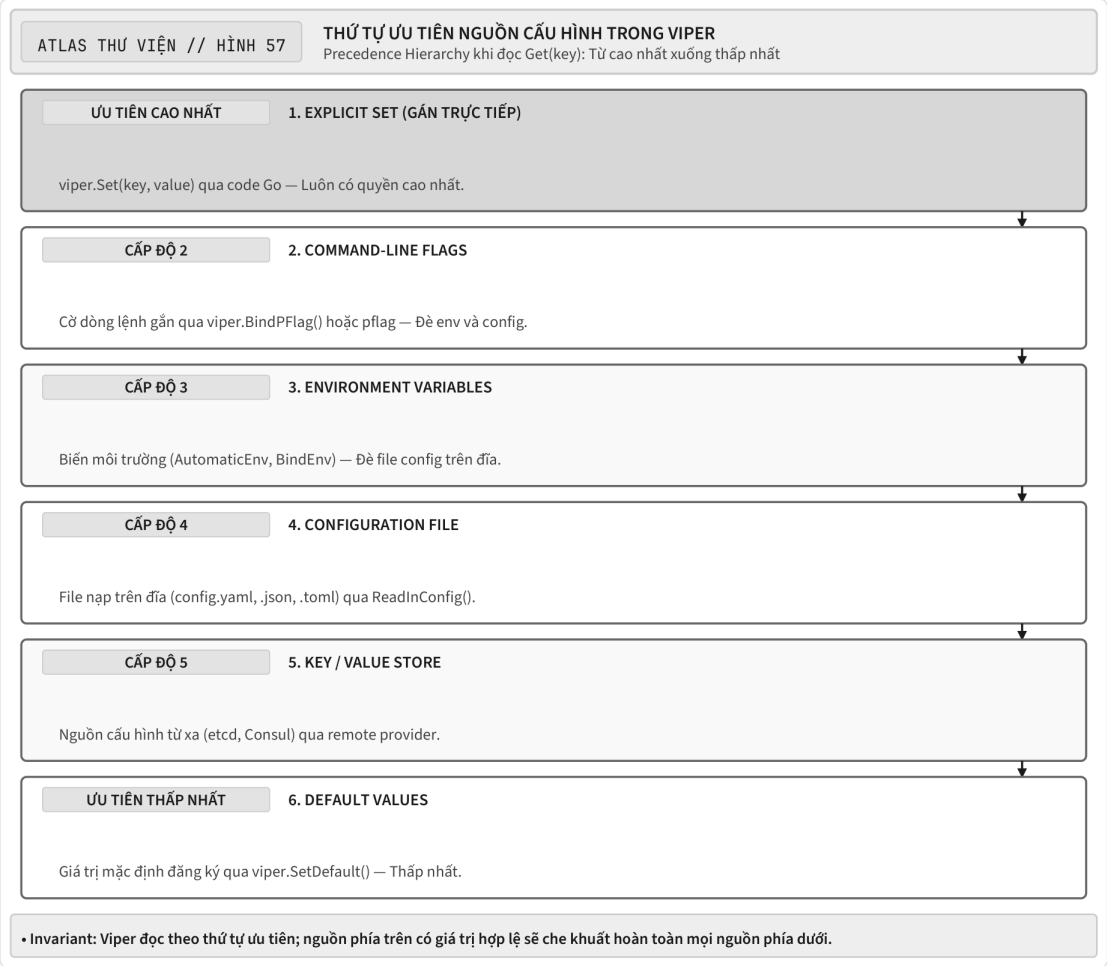
26. `github.com/spf13/viper` **(v1.21.0 — 394040ca)**

12-Factor App khuyến nghị tách cấu hình thay đổi theo deployment khỏi code; đó là hướng dẫn thiết kế, không phải guarantee của Viper. Trong ví dụ, các nguồn cấu hình có thứ tự ưu tiên cần được kiểm tra:

- Khi chạy trên máy tính cá nhân (local dev): Đọc cấu hình từ file `config.yaml`.
- Khi đóng gói vào Kubernetes: Nhận cấu hình ghi đè từ các biến môi trường (Environment Variables) hoặc cờ dòng lệnh CLI.
- Khi không có cấu hình ưu tiên cao hơn, Viper có thể dùng giá trị từ `SetDefault`. Giá trị mặc định vẫn cần validation theo contract ứng dụng; framework không làm nó tự trở nên an toàn.

Làm thế nào để kết hợp tất cả các nguồn cấu hình này mà không biến mã nguồn thành một mớ hỗn độn các câu lệnh `if-else`?

Viper ở version đã pin có thứ tự ưu tiên gồm explicit `Set`, flag đã bind, environment, config file, remote key/value store và default. Đây là quy tắc tìm value, không thay validation miền giá trị hay policy secret:



Hình 57 — Với một key, Viper ưu tiên nguồn phía trên khi nguồn ấy có giá trị hợp lệ theo cấu hình. Đây là thứ tự ưu tiên khi đọc, không phải lịch nạp file.

`GetInt` đọc theo precedence đó. `DATABASE_PORT` chỉ là key môi trường tương ứng khi đã cấu hình binding hoặc key replacer phù hợp; dấu chấm trong config key không tự luôn đổi thành gạch dưới. Nếu `Set` đã override key, nó đứng trên cả flag. Sau khi tìm và chuyển giá trị, application vẫn cần kiểm tra range và báo lỗi config phù hợp.

Cơ chế phân giải thứ tự ưu tiên này giúp ứng dụng triển khai linh hoạt trên nhiều môi trường mà không cần sửa đổi mã nguồn, chỉ cần điều chỉnh nguồn cấu hình theo cấp bậc tương ứng.

---

27. `github.com/fsnotify/fsnotify` (**v1.10.1** — `76b01a6e`)

Hot reload là một policy của ứng dụng, không phải tác dụng tự động của việc sửa ConfigMap. Với volume ConfigMap, kubelet có thể cập nhật file sau một khoảng trễ; biến môi trường và mount `subPath` có semantics khác. Ứng dụng còn phải nhận ra thay đổi, parse, validate và công bố config mới an toàn. Không giả định một callback fsnotify là đủ cho mọi cách cập nhật file.

`fsnotify` hiện thực hóa tính năng này bằng cách trừu tượng hóa các cơ chế thông báo sự kiện tập tin cấp thấp của từng hệ điều hành:

- Trên Linux: Sử dụng các hàm hệ thống `inotify_init1` và `inotify_add_watch` (`backend_inotify.go`).
- Trên Windows: Sử dụng hàm Win32 API `ReadDirectoryChangesW` (`backend_windows.go`).
- Trên macOS: backend kqueue ở version được pin; README vẫn ghi FSEvents chưa được hỗ trợ.

Tuy nhiên, có một cái bẫy chết người mà rất nhiều kỹ sư Go mắc phải khi triển khai hot reloading trên Kubernetes: **Cái bẫy Symlink của ConfigMap.**

Kubernetes không ghi đè trực tiếp nội dung vào file cấu hình đang mở. Thay vào đó, nó tạo một thư mục mới có tên là một chuỗi timestamp, mount dữ liệu vào đó, rồi thực hiện hoán đổi một liên kết mềm (symlink) mang tên `..data` trỏ sang thư mục mới, sau đó xóa thư mục cũ.

Nếu ứng dụng Go chỉ lắng nghe sự kiện `fsnotify.Write` trên file cấu hình:

```
if event.Op&fsnotify.Write == fsnotify.Write { ... }
```

Watch một file có thể mất hiệu lực khi file được thay atomically thay vì ghi tại chỗ. Với projected volume Kubernetes, cần hiểu việc đổi symlink và sự kiện của filesystem/OS. Theo dõi directory và xử lý rename/remove, đăng ký lại khi cần, rồi parse/validate config mới trước publication là một policy có thể kiểm tra. Không hứa hot reload tức thời hoặc không gián đoạn chỉ từ một callback fsnotify.

---

28. `go.uber.org/zap` (**v1.28.0** — `5b81b37b`)

Nếu ứng dụng xử lý tải cao, việc ghi lại log trên mỗi request có thể trở thành nguồn áp lực cấp phát đáng kể. Nếu sử dụng thư viện log thông thường dựa trên `fmt.Printf` hoặc các thư viện dùng `interface{}`:

```
log.Printf(
    "User %d transferred %f to user %d",
    fromID, amount, toID,
)
```

Đưa value vào interface không tự bảo đảm heap allocation. Escape analysis, call path và formatter quyết định allocation thực tế. Nếu logging là đường nóng, kiểm tra escape output và benchmark đúng field, formatter và destination trước khi kết luận GC pressure.

`zap` hướng tới logging có ít cấp phát ở các đường nóng; kết quả còn phụ thuộc encoder, field, output và phiên bản thư viện.

Cơ chế cốt lõi nằm ở cấu trúc `zap.Field` (`zapcore/field.go`):

```
type Field struct {
    Key       string
    Type      zapcore.FieldType
    Integer   int64
    String    string
    Interface any
}
```

Khi ghi log: `logger.Info("transfer", zap.Int64("from", fromID), zap.Float64("amount", amount))`, `zap.Int64` không hề đưa số nguyên vào `interface{}`! Nó gán thẳng giá trị vào trường `Integer int64` bên trong struct `Field`.

`zapcore/json_encoder.go` dùng bộ đệm tái sử dụng qua `sync.Pool` và ghi trực tiếp nhiều trường vào buffer. Điều đó giảm allocation trong những đường đi phù hợp, nhưng không cho phép suy ra một ngưỡng nano-giây hay `0 allocs/op` cho ứng dụng khác. Muốn so sánh với logger khác, cần benchmark cùng Go version, encoder, destination và tập field.

---

29. `go.uber.org/automaxprocs` (v1.6.0 — 1ea14c35)

Thư viện `automaxprocs` giải quyết bài toán lịch sử đối với các binary Go phiên bản cũ chạy trong container có CPU limit thấp. Trước Go 1.25, mặc định `GOMAXPROCS` không xét cgroup CPU bandwidth limit, nên một process có thể chọn số lượng song song gần số CPU host thay vì giới hạn CPU của container.

Edition dùng toolchain Go 1.27.1; các module lab có directive riêng cần kiểm tra khi build. Với Go từ 1.25 trên Linux, mặc định `GOMAXPROCS` có thể xét CPU logic, affinity và throughput limit của cgroup, rồi cập nhật định kỳ. Main module hoặc workspace khai báo Go 1.24 trở xuống có thể giữ mặc định tương thích cũ. `GODEBUG=containermaxprocs=0` tắt xét quota; `updatemaxprocs=0` tắt cập nhật. Đặt `GOMAXPROCS` thủ công bằng biến môi trường hoặc API cũng vô hiệu cập nhật tự động. CPU request của Kubernetes không phải quota này; `GOMAXPROCS` không thay việc kernel áp quota.

Vì vậy không thể kết luận rằng đặt `GOMAXPROCS` bằng quota sẽ loại bỏ throttling. Nó có thể giảm các đỉnh song song bất lợi cho một workload, nhưng GC, syscall, loại workload, quota phân số và chính sách scheduler đều còn ảnh hưởng đến độ trễ. Runtime Go 1.27 cũng có các quy tắc làm tròn và không hạ mặc định xuống dưới hai chỉ vì cgroup quota.

Một cách dùng lịch sử của `automaxprocs` là import side effect tại `maxprocs/maxprocs.go`:

```
import _ "go.uber.org/automaxprocs"
```

Khi được nạp, thư viện đọc cấu hình cgroup và gọi `runtime.GOMAXPROCS` theo chính sách của nó. Đây vẫn có thể hữu ích cho Go cũ, cho chính sách cấu hình rõ ràng, hoặc các cạnh môi trường cần kiểm chứng. Với Go 1.27 chạy mặc định, không nên xem nó là phụ thuộc bắt buộc chỉ để runtime hiểu Kubernetes CPU limit.

```
Quota = cfs_quota_us / cfs_period_us
```

Biểu thức này mô tả quota throughput của cgroup v1, không phải CPU request và cũng không phải cam kết throughput thực tế của ứng dụng. Hãy đo latency, CPU throttling và `runtime.GOMAXPROCS(0)` trên đúng workload trước khi quyết định giữ một override.

---

30. `github.com/hashicorp/go-retryablehttp` **(v0.7.8 — `e1f5485f`)**

Xét scenario một client retry sau response 503 rồi dần gặp lỗi thiếu file descriptor. Retry count không đủ để kết luận nguyên nhân; kiểm tra body có được đóng, số connection, goroutine và descriptor đang giữ. Đây là tình huống để luyện điều tra, không phải kết quả đo hay tần suất sự cố của thư viện.

Một nguyên nhân cần điều tra là vòng đời `response.Body` trong `net/http.Transport`. Caller phải luôn đóng `response.Body`; việc đọc đến EOF có thể là điều kiện hữu ích cho một số đường tái sử dụng kết nối, nhưng không phải bảo đảm tái sử dụng. Transport, giao thức, kích thước body, server và kết nối hiện có đều ảnh hưởng kết quả.

Nếu ta chỉ đơn thuần viết:

```
resp, err := client.Do(req)
if err != nil {
    return err
}
if resp.StatusCode >= 500 {
    resp.Body.Close() // Có thể không reuse, không tự là leak.
    // Thực hiện retry...
}
```

Kết nối có thể không đủ điều kiện để tái sử dụng khi body bị bỏ dở. Tùy tình trạng kết nối và Transport, client có thể đóng kết nối thay vì giữ nó trong idle pool. Dưới tải cao, một pattern retry sai hoặc giới hạn connection không phù hợp có thể làm số file descriptor tăng; cần đo `httptrace`, connection metrics và lỗi thực tế trước khi kết luận nguyên nhân.

Ở `client.go`, thư viện dùng helper bounded drain trước khi đóng body. Mã minh họa policy, không phải signature chép nguyên từ source:

```
func (c *Client) drainBody(body io.ReadCloser) {
    defer body.Close()
    _, _ = io.Copy(
        io.Discard, io.LimitReader(body, respReadLimit),
    )
}
```

Trước mỗi lần thử lại, thư viện có thể dùng `io.LimitReader` để giới hạn lượng dữ liệu bỏ đi trước khi đóng body. Đây là trade-off giữa lượng đọc thêm, bộ nhớ và khả năng giữ kết nối; nó không chứng minh TCP đã được "thanh tẩy" hoặc connection chắc chắn quay lại pool.

PHẦN 3: NHÓM MỞ RỘNG CHUYÊN SÂU (TIER B: 31–43)

31. `golang.org/x/sync` **(v0.23.0 —** `f75267d8`**)**

Xét scenario nhiều request cùng thấy cache miss cho một key rồi cùng truy vấn database. Các lời gọi chồng thời gian có thể tạo tải dư thừa. Số request, capacity và ảnh hưởng cần đo trên workload thật; tên cache stampede không cho một ngưỡng gây sập chung.

`singleflight.Group` trong `singleflight/singleflight.go` cho phép gộp các lời gọi cùng key đang diễn ra:

```
var g singleflight.Group
v, err, shared := g.Do("product_123", func() (any, error) {
    return fetchProductFromDB("product_123")
})
```

`singleflight.Group.Do` gộp những lời gọi cùng key đang chồng thời gian: một call chạy function, các caller trùng chờ và nhận cùng kết quả. Implementation Do dùng WaitGroup; DoChan có đường channel riêng. Sau khi call hoàn tất, call mới có thể chạy function lại; đây không phải cache và không bảo đảm 10.000 request bất kỳ chỉ tạo một query. Scope, cancellation, error sharing và Forget cần policy riêng.

`errgroup.WithContext` tạo group và derived context bị cancel khi hàm đầu tiên trả lỗi hoặc `Wait` kết thúc. Một zero `errgroup.Group` không tự có context để cancel. `semaphore.Weighted` hỗ trợ giới hạn concurrency theo trọng số; caller còn phải release phần đã acquire.

---

32. `golang.org/x/time` **(v0.16.0 — `fb013b3d`)**

Một implementation token bucket minh họa có thể dùng goroutine với ticker để nạp token vào channel. Khoảng tick là lựa chọn của ví dụ, không phải thành phần bắt buộc của thuật toán.

Nếu tạo một goroutine và timer cho mỗi limiter, tài nguyên nền tăng theo số limiter ngay cả khi nhiều limiter không có request. Cần đo chi phí ấy thay vì gắn một số người dùng với kết luận hết tài nguyên. `rate.Limiter` chọn cách tính lượng token theo thời gian đã trôi qua khi có thao tác.

Mã nguồn của `rate.Limiter` tại `rate/rate.go` (`type Limiter`) triển khai **Thuật Toán Token Bucket Không Cần Goroutine Chạy Nền**.

Bên trong struct `Limiter` hoàn toàn không có goroutine hay timer nào cả. Các trường trạng thái quan trọng của implementation hiện tại bao gồm: `sync.Mutex` bảo vệ toàn bộ struct, `limit Limit` (tốc độ nạp token), `burst int` (dung lượng tối đa), `tokens float64` (số token hiện có), `last time.Time` (thời điểm cập nhật cuối), và `lastEvent time.Time` (dùng cho Reserve). Đây là implementation detail của version hiện tại, không phải API contract.

Khi một request gọi vào hàm `AllowN(now, n)`:

1. Nó lấy thời điểm hiện tại `now`.
2. Nó tính khoảng thời gian trôi qua kể từ lần gọi cuối: `Δt = now - last`.
3. Nó tính lượng token khả dụng từ elapsed time và rate, có giới hạn burst.
4. Đường reserve quyết định có chấp nhận không; không phải mọi lần `AllowN` thất bại đều ghi lại `last` và trừ token. `Wait`/reservation còn có semantics và timer riêng, khác đường `Allow` không chờ.

Implementation tính token theo khoảng thời gian khi gọi thao tác, không có goroutine/timer nền riêng cho từng `Limiter`. Nhưng `Wait` có thể tạo timer cho caller cần chờ; operation vẫn tốn CPU, lock và state. Số instance phù hợp phải theo workload và ngân sách, không suy ra khả năng scale không có chi phí.

---

33. `github.com/hashicorp/go-plugin` (v1.8.0 — `155dcddc`)

Package `plugin` có giới hạn tương thích và platform được tài liệu hóa. Host và plugin cần toolchain/build configuration cùng các dependency chung tương thích; khác biệt có thể gây lỗi nạp hoặc runtime failure. Không phải cứ lệch một flag là chắc chắn `panic` ngay. Out-of-process plugin đổi boundary tương thích sang protocol và lifecycle IPC, với chi phí và failure mode riêng.

`go-plugin` tổ chức host và plugin thành các process giao tiếp qua IPC. Cần phân biệt handshake nhận diện executable/protocol với authentication; cookie không bảo vệ trước một plugin độc hại.

Với đường plugin dùng `go-plugin`, host giao tiếp với **tiến trình plugin riêng** qua RPC. Không suy rộng cơ chế này thành mọi loại plugin và extension của Terraform.

Quá trình bắt tay diễn ra như sau:

1. Host khởi chạy process plugin bằng API tạo process của OS, với environment có magic cookie và cấu hình handshake. Không mô tả đường Windows là fork.
2. Tiến trình plugin khởi động, mở một điểm lắng nghe cục bộ (Unix Domain Socket hoặc port TCP ngẫu nhiên tùy cấu hình), và in ra stdout dòng thông báo sẵn sàng kèm theo địa chỉ kết nối.
3. Plugin kiểm tra magic cookie để tránh chạy nhầm ngoài host; host parse dòng địa chỉ và protocol trên stdout để kết nối. Cookie không xác thực peer. Transport `net/rpc` hay gRPC và cấu hình TLS có trust boundary riêng.

Lợi ích của thiết kế này bao gồm: Plugin có thể được viết bằng ngôn ngữ khác nhau và biên dịch độc lập, miễn là phù hợp với transport và protocol được cấu hình (gRPC path hỗ trợ cross-language tốt hơn net/rpc path). Về **cách ly sự cố (Fault Isolation)**: `panic` xảy ra trong tiến trình con plugin không trực tiếp gây `panic` cho tiến trình mẹ, và đây chính là lợi ích thực sự của kiến trúc out-of-process. Tuy nhiên plugin vẫn có thể làm host treo (hung RPC), tiêu hao CPU/RAM/I/O của máy chủ, hoặc làm hỏng tài nguyên chia sẻ bên ngoài; không nên hiểu là cách ly hoàn toàn.

---

34. `github.com/hashicorp/hcl/v2` **(v2.25.0 —** `00057cf0`**)**

Tại sao HashiCorp không dùng YAML hay JSON để viết cấu hình cho Terraform mà lại kỳ công sáng tạo ra một ngôn ngữ riêng mang tên HCL (HashiCorp Configuration Language)?

HCL native syntax hỗ trợ block, attribute và expression; HCL còn có JSON syntax. Việc dùng HCL cho Terraform cho phép tách parse cấu trúc khỏi evaluate biểu thức theo context. YAML hay JSON là format dữ liệu khác; khả năng biểu thức còn phụ thuộc application dùng chúng, không phải một tuyên bố rằng YAML không thể mô hình hóa cấu hình phức tạp.

Mã nguồn của `hcl/v2` được cấu trúc thành hai tầng tách bạch rõ rệt: Tầng Cú Pháp Cấu Trúc (`hclsyntax/parser.go`) và Tầng Đánh Giá Ngữ Cảnh (`eval_context.go`).

HCL có thể parse cấu hình trước khi có giá trị cho các biến trong biểu thức. `EvalContext` cung cấp biến và hàm khi caller quyết định evaluate biểu thức; HCL không tự xây hay lập lịch đồ thị phụ thuộc của Terraform. Terraform là application dùng HCL và quyết định lúc nào các giá trị tham chiếu sẵn sàng.

---

35. `github.com/hashicorp/terraform-plugin-go` **(v0.31.0 —** `09a1181b`**)**

Nếu `terraform-plugin-framework` (thư viện số 09) là giao diện cấp cao thân thiện dành cho lập trình viên, thì `terraform-plugin-go` là tầng nền móng cấp thấp (Low-Level RPC Protocol) giao tiếp trực tiếp với Terraform Core.

Package `tfprotov6` định nghĩa hợp đồng RPC của Terraform Provider Protocol v6. Các giá trị động có thể mang dữ liệu MessagePack hoặc JSON theo protocol. Không quy toàn bộ dữ liệu trạng thái thành một định dạng nhị phân duy nhất khi đọc contract này.

`tftypes.Value` biểu diễn type, null và unknown khi trao đổi dữ liệu với Terraform. Caller cần giữ schema và trạng thái ấy qua conversion; có type model không tự bảo đảm mọi adapter của application đều không làm mất thông tin.

---

36. `github.com/prometheus/common` (**v0.71.0** — `9a4aff03`)

Một scrape workload nhiều target có thể tốn CPU ở parser. Chi phí phụ thuộc số sample, nhãn, format và input; không suy ra trần CPU từ một số target giả định. Đọc parser của expfmt để hiểu grammar, rồi benchmark đúng dữ liệu nếu lựa chọn parser là một quyết định performance.

`expfmt/text_parse.go` triển khai parser trạng thái cho Prometheus text exposition format.

Bộ giải mã đọc luồng đầu vào và nhận diện tên metric, bộ nhãn, giá trị cùng timestamp theo grammar. Không suy ra số cấp phát, lợi thế performance hoặc mức phổ biến của format từ riêng cấu trúc parser; nếu throughput là tiêu chí chọn format, đo trên dữ liệu thật.

---

37. `modernc.org/sqlite` (**v1.59.0** — `c96a4e6c`)

Driver dùng cgo cần C toolchain phù hợp khi build và có thêm allocator/lifetime boundary. Cross-build cần native target support. Chi phí crossing phụ thuộc operation và môi trường; không có số nanosecond chung nếu không có benchmark cùng hardware, payload, Go version và cấu hình.

Driver `modernc.org/sqlite` tại version đã pin không dùng cgo ở đường driver. Đây là lựa chọn triển khai; cần kiểm tra target được hỗ trợ và các dependency của chính version được build.

Đây là driver SQLite không dùng cgo trong đường build được tài liệu hóa, không phải lời hứa tương thích và hiệu năng cho mọi target. Cơ chế chuyển

dịch C sang Go, biểu diễn con trỏ và allocator là chi tiết implementation cần đọc đúng source ở version được ghim; không dùng suy đoán về bố trí bộ nhớ để chọn driver.

Tránh cgo có thể đơn giản hóa build trên target mà driver hỗ trợ. Không kết luận driver Go luôn dùng nhiều memory hơn hay luôn chậm hơn bản C: so sánh cần cùng SQLite version, query, concurrency, database, hardware và cấu hình. Chọn driver theo compatibility, lifecycle và chi phí vận hành trước; nếu performance quyết định lựa chọn, benchmark đúng workload.

---

38. `go.opentelemetry.io/contrib` (**v1.46.0** — `c4c6248e`)

OpenTelemetry Go cốt lõi (mục 05) cung cấp API/SDK. Các module contrib cung cấp integration cho thư viện chuẩn như `net/http` và cho thư viện khác như client gRPC. Cần chọn module và version tương thích với SDK; đây không phải một bộ wrapper tự bao phủ mọi dependency của ứng dụng.

`otelhttp` cung cấp wrapper cho `http.Handler` trong `instrumentation/net/http/otelhttp/handler.go`.

Khi bọc một `http.Handler` tiêu chuẩn bằng `otelhttp.NewHandler(handler, "my-operation")`, middleware thực hiện một chu trình hoàn chỉnh:

1. Trích xuất context từ request header theo propagator đã cấu hình. Đọc `traceparent` cần propagator hỗ trợ W3C Trace Context; không tự suy ra chỉ từ wrapper.
2. Khởi tạo một Server Span mới và nhúng vào `r.Context()`.
3. Dùng `request.NewRespWriterWrapper` cùng `httpsnoop.Wrap` để ghi nhận status code và số byte phản hồi, đồng thời giữ các interface tùy chọn của writer gốc. `http.ResponseWriter` chuẩn không có hàm đọc lại status code sau khi ghi.
4. Khi handler kết thúc, middleware đặt span status theo semantic convention, ghi response attributes và kết thúc span. Cách diễn giải mã 5xx thuộc semantic convention được dùng, không phải một lệnh `RecordError` vô điều kiện trong handler.

Wrapper tạo điểm instrumentation cho đường request đã bọc. Trace propagation, sampling, exporter và middleware khác vẫn quyết định dữ liệu cuối; một dòng bọc handler không chứng minh trace xuyên service đã đầy đủ.

---

39. `github.com/aquasecurity/trivy` **(v0.74.0 —** `e1fd17a0`**)**

Trivy phát hiện package/version từ artifact rồi đối chiếu nguồn vulnerability phù hợp. Scan latency phụ thuộc image, cache, scanner được bật, database và I/O; không có cam kết vài giây cho mọi image.

Mã nguồn tại `pkg/fanal/artifact/artifact.go` hé lộ quy trình bóc tách từng lớp (layer by layer) của Trivy.

Artifact analysis xử lý filesystem/layer và các analyzer đã chọn. Các file metadata thường có ích gồm:

- Hệ điều hành Linux: `/var/lib/dpkg/status` (Debian/Ubuntu), `/lib/apk/db/installed` (Alpine), hoặc cơ sở dữ liệu RPM.
- Ứng dụng ngôn ngữ: `go.sum`, `package-lock.json`, `pom.xml`, `requirements.txt`.

Kết quả phụ thuộc analyzer và dữ liệu package có trong artifact; không bảo đảm suy ra mọi version chính xác từ mọi file. Trivy còn có các scanner khác như secret và misconfiguration, nên không mô tả mọi scan là chỉ đọc manifest trong RAM. Vấn đề cache, database freshness và scanner coverage cần ghi trong scan contract.

---

40. `github.com/in-toto/in-toto-golang` **(v0.11.0 —** `36d782ff`**)**

Trong một quy trình CI/CD hiện đại, mã nguồn trải qua nhiều bước: Lập trình viên commit -> CI checkout -> Biên dịch nhị phân -> Chạy unit test -> Đóng gói container image. Kẻ tấn công có thể không tấn công được vào Git, nhưng có thể can thiệp vào máy chủ build để hoán đổi file nhị phân ngay sau khi biên dịch xong và trước khi đóng gói.

`in-toto` cung cấp model metadata và verifier để kiểm tra các bước mà layout yêu cầu; đọc `in_toto/model.go` (`type Link`) để biết dữ liệu nào thực sự được ghi.

Mỗi bước trong pipeline CI/CD được ghi nhận lại bằng một tài liệu bằng chứng gọi là **Link Metadata**:

- **Materials:** Tên và digest của những đầu vào được recorder thu thập theo cấu hình; SHA-256 là một thuật toán thường dùng, không phải mọi file đều mặc nhiên được ghi.
- **Products:** Tên và digest của đầu ra được thu thập. Artifact ngoài tập này không được bảo vệ chỉ vì bước có một Link.
- Metadata có thể được ký bởi recorder với key được layout tin. Chữ ký chỉ ràng buộc dữ liệu đã ghi với key, không chứng minh recorder không bị can thiệp.

Verifier đối chiếu signatures và rules của layout với materials/products đã được ghi. Coverage phụ thuộc những file và bước được policy yêu cầu, cùng trust của key và recorder; không hứa phát hiện mọi tráo đổi nếu một artifact hoặc boundary chưa được ghi nhận.

---

41. `github.com/theupdateframework/go-tuf/v2` **(v2.4.2 — `f5edbde3`)**

Khi máy chủ tự động tải các bản cập nhật phần mềm hoặc chữ ký container từ xa, nó phải đối mặt với nhiều hình thức tấn công tinh vi: Kẻ tấn công có thể giả mạo máy chủ cập nhật, hoặc nguy hiểm hơn, thực hiện cuộc tấn công đóng băng thời gian (Freeze/Replay Attack): liên tục gửi lại một bản cập nhật cũ đã có lỗ hổng bảo mật nhưng chữ ký vẫn còn hợp lệ.

TUF tách trách nhiệm metadata giữa các role để giới hạn trust và kiểm tra update theo version, expiration và signature:

1. **Root Role:** Xác định các key và threshold được tin cậy cho metadata. Giữ root key offline là khuyến nghị bảo vệ quan trọng; không phải bằng chứng rằng mọi deployment thực tế đều làm vậy.
2. **Targets Role:** Ký xác nhận danh sách các tệp tin cập nhật cùng mã hash và dung lượng chính xác.

3. **Snapshot Role:** Ghi version và có thể ghi hash/length của metadata targets và delegated roles cần dùng, nhằm kiểm tra một tập metadata nhất quán. Nó không kiểm kê mọi file của hệ thống.
4. **Timestamp Role:** Tham chiếu snapshot metadata và có expiration để giúp client phát hiện freeze. Thời hạn là policy triển khai; timestamp không phải chứng thực liên tục giờ hệ thống của server.

TUF client kiểm tra metadata version, expiration, role signatures và các quan hệ theo policy/spec; expiration hỗ trợ chống freeze, không thay mọi cơ chế chống rollback. Không suy ra mọi hệ ký artifact như Sigstore đều dùng cùng update protocol TUF ở từng operation.

---

42. `cloud.google.com/go` **(v0.123.0 —** `4e837358`**)**

Với download lớn trên WAN, retry từ byte đầu có thể lặp nhiều công việc. Range read/resume có thể hữu ích, nhưng phải kiểm tra generation/identity của object và contract API; không ghép các byte từ hai version khác nhau thành một file rồi gọi đó là download thành công.

Thư viện Cloud Storage của Google giải quyết bài toán phục hồi dữ liệu tại `storage/reader.go` (`type Reader`).

`Reader` có API `io.ReadCloser`. Trong đường HTTP của `storage/http_client.go`, `httpReader` giữ số byte đã đọc và callback reopen; lỗi retryable có thể mở lại range theo offset và generation. Không phải mọi lỗi hoặc transport đều cùng behavior. Đường decompressive transcoding còn có trường hợp phải đọc bỏ byte đã thấy; kiểm tra source và generation condition thay vì hứa mọi lần resume đều không lặp dữ liệu truyền.

Metadata server là một nguồn credential trong môi trường Google Cloud được cấu hình phù hợp. Việc ứng dụng nhận identity nào phụ thuộc môi trường và cấu hình xác thực, kể cả đường Workload Identity; không suy ra từ riêng `storage.Reader` hay `compute/metadata` rằng mọi container đều có token đúng quyền.

---

43. `github.com/Azure/azure-sdk-for-go/sdk/azcore` **(sdk/azcore/v1.23.1 —** `d86ae78b`**)**

Azure SDK for Go dùng policy pipeline để tổ chức các bước như authentication, retry và transport. Với client ARM đang xét, xem `sdk/azcore/runtime/pipeline.go`; không suy ra rằng mọi lời gọi HTTP trong một ứng dụng đều đi qua pipeline này.

Một pipeline của Azure là một chuỗi các `Policy` được thực thi tuần tự:

- **Authentication Policy:** Dùng `TokenCredential` mà caller cấp; Managed Identity chỉ là một cách cung cấp credential, không mặc định cho mọi client.
- **Telemetry Policy:** Gắn metadata nhận diện SDK/request theo cấu hình; không khẳng định Azure portal trực tiếp dùng trường đó cho mọi request.
- **Retry Policy:** Áp dụng retry theo policy cho response/error phù hợp, có xét thông tin server như `Retry-After` khi đường thực thi hỗ trợ.

Client có thể thêm policy theo các điểm mở rộng mà SDK hỗ trợ. Thứ tự, per-call/per-retry scope và dịch vụ dùng client ấy phải được kiểm tra; không suy ra một cấu hình client thay đổi mọi dịch vụ Azure trong process.

PHẦN 4: HẠ TẦNG AI AGENT & ĐIỀU PHỐI HIỆN ĐẠI (FRONTIER: 44–50)

44. `github.com/modelcontextprotocol/go-sdk` **(v1.8.0 —** `3f3b699b`**)**

MCP định nghĩa cách client và server trao đổi tool/resource qua protocol; ứng dụng vẫn phải xác định server nào được tin, dữ liệu nào được phép đọc và hành động nào caller được phép yêu cầu.

Go SDK tổ chức server trong `mcp/server.go` và transports trong package `mcp`. Giao thức dùng JSON-RPC với stdio hoặc HTTP transport phù hợp version; Streamable HTTP không đồng nghĩa transport SSE cũ. SDK không tự biến một tool thành hành động được phép.

JSON Schema là kiểm tra cấu trúc, không phải bộ phát hiện prompt injection. Một argument có thể đúng schema nhưng chứa yêu cầu không được phép; chính sách handler phải kiểm tra nghĩa và quyền.

JSON Schema kiểm tra cấu trúc và constraint đã khai báo, không phân biệt mọi nội dung chuỗi an toàn với instruction độc hại. Handler vẫn cần semantic validation, authenticated identity, authorization và giới hạn egress/side effect. Process sandbox là một lớp có trade-off riêng; CNI chỉ thiết lập network, không tự là security sandbox cho Agent.

---

45. `github.com/google/adk-go` **(HEAD-main —** `f9ce16ef`**)**

ADK tổ chức model call, tool và session thành một vòng thực thi nhiều bước. Đọc đường runner ở commit ghim để biết state nào được giữ và lúc nào một tool được gọi; không suy ra quyền tự trị từ tên framework.

Ở ADK, đọc package `agent` và `tool/functiontool`. Wrapper function tool dùng config name/description và input/output types hoặc schema được cung cấp; không giả định nó tự parse mọi docstring Go thành quyền hay tool semantics.

Session, runner và tool call tạo các boundary lifecycle khác nhau. Khi triển khai, phải kiểm tra budget cho model calls, token, thời gian và side effect bằng API thực có ở version được pin; không dựa vào một field `MaxSteps` chưa được đối chiếu. Framework không tự cấp quyền production hay bảo đảm task hoàn tất.

---

46. `github.com/microsoft/agent-framework-go` **(HEAD-main —** `5fea5266`**)**

Ở commit đã ghim, hãy đọc feature và giới hạn của Go implementation trước khi chọn nó cho ứng dụng; nhãn enterprise không thay thế test compatibility và failure recovery.

README của commit đã pin mô tả các package `agent`, `provider`, `tool` và `workflow`. Đây là Go implementation đang public preview, không có feature parity mặc định với các phiên bản trên `.NET` hoặc Python. Middleware, workflow và checkpoint là các abstraction để đọc tiếp; không quy một `runtime.go` tưởng tượng thành bảo đảm tự planning, authorization hay Redis/CosmosDB session isolation.

---

47. `github.com/cloudwego/eino` **(v0.9.21 — ba04fde8)**

`eino` của CloudWeGo cung cấp các component và orchestration cho ứng dụng LLM trong Go. So sánh overhead với framework Python cần workload và phép đo tương đương, không suy ra từ ngôn ngữ implementation.

`compose/graph.go` mô hình hóa graph và mode chạy. Directed graph không mặc nhiên là DAG: source phân biệt Pregel mode có thể chứa cycle với DAG mode và validation tương ứng.

Mỗi thành phần trong đường ống xử lý (gồm bộ thu hồi dữ liệu RAG `Retriever`, mẫu câu lệnh `PromptTemplate`, mô hình ngôn ngữ `ChatModel`, và công cụ `Tool`) được coi là một đỉnh (**Node**) trên đồ thị, kết nối với nhau bằng các cạnh (**Edges**) truyền dữ liệu định kiểu chặt chẽ.

Một stream abstraction có thể phục vụ nhiều consumer, nhưng fan-out cần policy backpressure, cancellation và bộ đệm. Nếu một nhánh hiển thị token cho người dùng trước khi nhánh moderation kết luận, đó là kiểm tra sau phát hành, không phải bảo đảm chặn nội dung trước khi lộ ra. Với yêu cầu chặn trước, cần buffer/gate theo đơn vị nội dung đã chọn và chấp nhận độ trễ tương ứng; không coi streaming song song là safety boundary tự có.

---

48. `trpc.group/trpc-go/trpc-agent-go` **(v1.11.2 — 5a0030b6)**

Một Agent phụ thuộc provider có thể gặp throttling, timeout hoặc lỗi transport. Tần suất và latency cần đo ở provider, model và thời điểm thật; không có con số ba mươi giây chung cho mọi API.

Khi dùng `trpc-agent-go`, kiểm tra các package agent/tool và middleware của version thực tế. Hai yêu cầu thiết kế sau là policy cần triển khai và test, không claim rằng framework tự bật sẵn chúng:

- **Circuit Breaker (scenario):** Chọn threshold và cửa sổ quan sát để chuyển sang Open, giới hạn request thử lại và định nghĩa half-open. Fallback cần model/semantics phù hợp; không có ngưỡng 20% hay local model dự phòng tự động cho mọi hệ thống.

- **Metrics & Tracing:** Thiết kế correlation, đo token và thời gian theo provider response, rồi kiểm tra export và privacy. Có span không chứng minh billing được đo chính xác.

---

49. `github.com/kagent-dev/kagent` **(HEAD-main —** `375fe73a`**)**

Nếu muốn trao quyền cho một AI Agent tự động điều tra nguyên nhân sự cố trong cụm máy chủ Kubernetes, làm thế nào để ngăn chặn con AI đó vô tình thực hiện một lệnh tai hại như xóa nhầm namespace `production`?

README ở commit này mô tả `Agent`, `ModelConfig`, `ToolServer` và controller/engine/UI/CLI. Kubernetes-native resource là cách tổ chức lifecycle, không bảo đảm Agent không thể xóa namespace.

Không suy ra từ chữ task rằng mọi invocation là một CRD `Task` hoặc luôn tạo một sandbox Pod mới. Nếu ứng dụng chọn sandbox-per-task, đó là deployment policy phải đối chiếu controller source và cấu hình, cùng identity/egress của tool server thực sự chạy hành động.

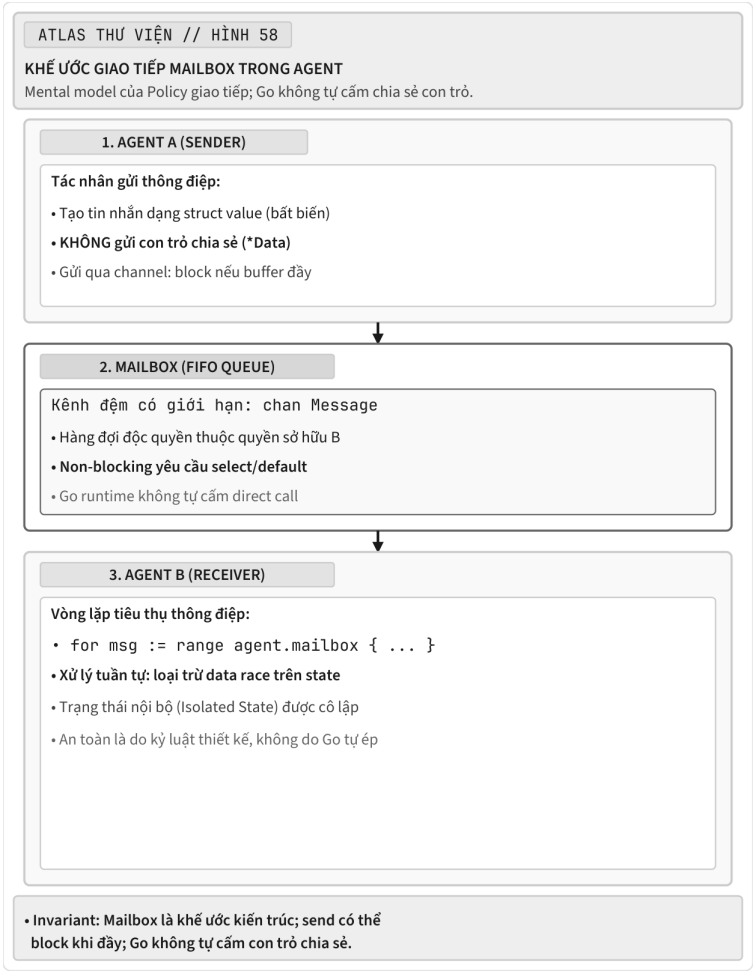
Trong scenario này, sandbox chỉ được cấp quyền đọc cần thiết và có owner dọn tài nguyên. Nhưng ServiceAccount không đại diện toàn bộ sandbox: token mount, network egress, host mounts, security context và quyền bên ngoài Kubernetes vẫn phải được kiểm tra. Pod không có quyền sửa qua Kubernetes API vẫn có thể tạo side effect trên một endpoint khác nếu egress và credential cho phép.

---

50. `github.com/agentscope-ai/agentscope-go` **(HEAD-main —** `8f82bd22`**)**

Khi nhiều Agent cùng sửa state có pointer dùng chung, chương trình vẫn cần synchronization và ownership như các chương concurrency đã dạy. Race hoặc deadlock không tất yếu chỉ vì có nhiều Agent; chúng phụ thuộc access và giao thức phối hợp.

`agentscope-go` là một lựa chọn tổ chức Agent, message và lifecycle. Khi đọc source đã pin, phải phân biệt abstraction Actor với những mechanism implementation thực sự có; tên framework không chứng minh isolation hoặc mọi failure đã được giải quyết.



Hình 58 — Mailbox là mental model của policy giao tiếp. Go và framework không tự cấm direct call hay chia sẻ pointer giữa các agent.

Một mô hình actor có thể đặt state và mailbox riêng cho từng Agent, nhưng Go không tự cấm direct call hay chia sẻ pointer. Thiết kế ứng dụng phải quy định giao tiếp và ownership, rồi kiểm tra các access thực tế. Không suy ra an toàn chỉ từ tên pattern.

README mô tả agent, tools, workflow, contexts và channels. Diagram mailbox phía trên là mental model thiết kế ứng dụng, không khẳng định mỗi Agent trong framework luôn là một actor process cô lập. Budget vẫn cần xét model calls, memory, network và lifecycle của từng implementation.

---

5. TỔNG KẾT & QUY TẮC BẢO TOÀN KIẾN TRÚC

Sau các chương, sách đặt Atlas mã nguồn 50 thư viện Go DevOps và cloud để nối kiến thức với implementation thực tế. Phụ lục A (Atlas Lỗi Go) nằm sau Atlas mã nguồn và luôn là nội dung cuối cùng của sách.

Atlas này không tồn tại độc lập mà là điểm tựa thực tế cho các chương trước đó trong cuốn sách:

- Informer và controller-runtime liên hệ **Chương 17, 21–23**; Chương 16 dạy observability, không phải Kubernetes client API.
- Kỹ thuật `sync/atomic` trong `client_golang` và `zap` minh họa cho **Chương 08** (Một race bắt đầu từ đâu) và **Chương 10** (Khi chương trình chậm hoặc phình).
- Vòng lặp xử lý `io.ReadCloser` và drain body trong `aws-sdk-go-v2` và `go-retryablehttp` neo chặt vào **Chương 11** (Lần theo một request HTTP) và **Chương 20** (OpsProbe Capstone).
- Tinh chỉnh CFS quota của `automaxprocs` và netlink socket giải thích tường tận các ranh giới hệ điều hành được trình bày tại **Chương 15** (Từ incident đến công cụ).

Commit ở heading là điểm neo của từng bài trong Atlas. `library_sources/lock.json` có thể đã được cập nhật tới version mới hơn; hãy đối chiếu full SHA trong hồ sơ nguồn rồi đọc đúng commit, không lấy checkout mới nhất thay thế âm thầm cho bản được giải thích. Identity không bảo đảm một binary hay cấu hình khác có cùng behavior. Đọc API contract, implementation và test như ba lớp bằng chứng khác nhau; scenario/policy là yêu cầu thiết kế, không phải lời hứa cho mọi version tương lai.

PHỤ LỤC A — ATLAS LỖI GO

Đọc lỗi từ triệu chứng đến nguyên nhân

Phụ lục này là tài liệu tra cứu kỹ thuật và phản xạ chẩn đoán nhanh, là điểm hội tụ tra cứu cuối cùng cho các lỗi Go và lỗi vận hành xuất hiện xuyên suốt cuốn sách theo nguyên tắc Grayscale-first. Mỗi mục gồm: mã định danh chẩn đoán (bao gồm chuỗi lỗi thực tế `EXACT`, giá trị sentinel chuẩn `SENTINEL`, trạng thái nền tảng `STATUS`, hoặc nhóm hiện tượng chẩn đoán `FAMILY`); bản chất cơ chế trong 1 câu; điểm cảnh báo hiểu lầm (`!`) nếu có; hành động kiểm tra đầu tiên (→); cùng tham chiếu chương (`[ChX,Y]`).

Bảng phân nhóm Taxonomy

`EXACT` là diagnostic hoặc phần diagnostic thực có, có thể thay tên và số bằng placeholder; không bao gồm prefix đường dẫn/vị trí do tool thêm. Các heading lược thuật ngữ, ghép nhiều chuỗi hay ghi tên tool để định vị được phân loại `FAMILY`, không giả làm byte output để copy so khớp. Wording còn phụ thuộc toolchain, OS, driver và version; kiểm tra output thật trước khi kết luận nguyên nhân.

Nhóm	Tên nhóm	Phạm vi chẩn đoán	Mã lỗi
A	Compiler & Type System	Lỗi biên dịch, hệ thống kiểu, interface, scope và syntax	A01–A14
B	Runtime & Panic	Crash process, nil pointer dereference, bounds, panic	B01–B11
C	Error Values & I/O	Sentinel errors, stream truncated, encoding/decoding	C01–C08
D	Context & Cancellation	Hủy tác vụ, timeout, deadline cascade, rò rỉ context	D01–D04
E	Filesystem & Process	File I/O, quyền hạn Linux, binary PATH, signals OS	E01–E07
F	Network / HTTP / TLS	DNS resolution, TCP handshake, reset socket, TLS/x509	F01–F11
G	Database	Connection pool, locking SQLite, transaction lifecycle	G01–G06
H	Concurrency	Deadlock toàn cục, data race warning, goroutine leak	H01–H05
I	Modules / Test / Toolchain	go.mod resolution, go vet static analysis, go test timeout	I01–I08
J	Container / Kubernetes / CI-CD	OOMKilled, CrashLoopBackOff, probe failure, gate reject	J01–J11

Bản đồ phản xạ 30 giây

Triệu chứng nhận diện	Hướng tra	Triệu chứng nhận diện	Hướng tra
Không biên dịch được mã nguồn?	**Nhóm A**	Sự cố DNS / TCP / TLS / HTTP?	**Nhóm F**
Biên dịch được nhưng panic / crash?	**Nhóm B**	Database nghẽn / lỗi transaction?	**Nhóm G**
Hàm trả về `err != nil` / I/O stream?	**Nhóm C**	Data race / deadlock / leak goroutine?	**Nhóm H**
Tác vụ bị timeout / cancel?	**Nhóm D**	Lỗi go build / test / vet / module?	**Nhóm I**
Lỗi file / permission / process?	**Nhóm E**	Pod crash / OOM / probe fail / gate?	**Nhóm J**

A — COMPILER & TYPE SYSTEM

A01 `undefined: x`

Không tìm thấy identifier trong scope hiện tại. Tham chiếu tên không được export từ package khác có thể phát diagnostic khác, chẳng hạn `name not exported`; đừng so khớp chúng như cùng một chuỗi.

→ Kiểm tra lỗi chính tả tên biến/hàm, scope khai báo, hoặc viết hoa chữ cái đầu nếu gọi từ package khác. **Ch1,3,5**

A02 `no new variables on left side of :=`

Vế trái `:=` không có biến mới nào khác _ trong block đang xét; dùng lại toàn bộ biến cũ hoặc chỉ có _ đều không thỏa điều kiện khai báo ngắn.

→ Đổi toán tử sang phép gán thường =, hoặc bổ sung ít nhất một biến mới vào vế trái. **Ch1,2**

A03 `"fmt" imported and not used`

Diagnostic này được compiler Go 1.27.1 phát cho import `fmt` không được dùng; tên package khác làm chuỗi thay đổi. Dot import cũng cần dùng identifier của package, còn blank import khai báo chủ đích chỉ chạy initialization.

→ Xóa dòng import thừa, dùng blank identifier `_ "pkg"` nếu cần chạy hàm `init()`, hoặc chạy `goimports`. **Ch1,5**

A04 `declared and not used: x`

Biến cục bộ được khai báo nhưng không có bất kỳ dòng code nào tiếp theo đọc hay sử dụng giá trị của nó.

→ Sử dụng biến vào logic, gán vào blank identifier `_ = x` nếu đang debug dở, hoặc loại bỏ khai báo thừa. **Ch1,2**

A05 `cannot use x (variable of type T1) as T2 value in argument`

Đối số không gán được cho kiểu tham số theo quy tắc assignability. Không suy ra mọi cặp kiểu khác tên đều phải conversion: hằng số biểu diễn được, interface satisfaction và một số kiểu không có tên có quy tắc riêng.

→ Thực hiện ép kiểu tường minh `T2(x)` nếu hai kiểu tương thích, hoặc sửa signature của hàm nhận. **Ch2,3,14**

A06 `assignment mismatch`

- `1 variable but f returns 2 values`
- `2 variables but f returns 1 value`

Số lượng biến nhận ở vế trái không khớp chính xác với số lượng giá trị trả về của hàm ở vế phải.

→ Kiểm tra signature của hàm gọi và hứng đủ tất cả các giá trị trả về (dùng _ để bỏ qua nếu cần). **Ch1,4**

A07 `missing return`

Hàm có kết quả nhưng thân hàm không kết thúc bằng terminating statement theo Go spec. Compiler không chứng minh mọi điều kiện nghiệp vụ luôn đúng; một vòng `for {}` không có break cũng có thể là terminating statement.

→ Kiểm tra return và các nhánh theo quy tắc terminating statement; không thêm panic chỉ để che một kết quả nghiệp vụ chưa được thiết kế. **Ch1,4**

A08 `invalid operation: operator not defined on struct`

Sử dụng toán tử so sánh (==, !=) trên struct chứa ít nhất một field không thể so sánh (như slice, map, func).

→ So sánh từng field có thể so sánh, hoặc tự viết hàm helper so sánh nghiệp vụ chuyên biệt. **Ch2,3**

A09 `cannot assign to struct field in map`

Không thể gán trực tiếp `m["k"].Field` vì map index expression không addressable theo Go spec; đừng suy ra quy tắc này từ một địa chỉ vật lý của implementation.

→ Lấy struct ra biến tạm, cập nhật field rồi gán ngược lại vào map, hoặc dùng map chứa pointer `map[K]*V`. **Ch2,3**

A10 `cannot take the address of x`

Toán tử lấy địa chỉ & được áp dụng lên unaddressable value (giá trị trả về của hàm, hằng số, map index expression).

→ Gán giá trị đó vào một biến tạm cục bộ trước khi thực hiện lấy địa chỉ `&temp`. **Ch2,14**

A11 `constant overflows int`

Giá trị hằng số vượt quá giới hạn biểu diễn bit của kiểu số nguyên đích trong kiến trúc hiện tại.

→ Chỉ định kiểu số nguyên có độ rộng bit lớn hơn (`int64`, `uint64`), hoặc sử dụng package `math/big`. **Ch1,2**

A12 `syntax error: unexpected newline, expecting comma or }`

Thiếu dấu phẩy , ở phần tử cuối cùng của composite literal hoặc danh sách tham số nhiều dòng (multi-line).

→ Bổ sung dấu phẩy , vào cuối dòng ngay trước dấu đóng ngoặc nhọn } hoặc ngoặc đơn). **Ch1,3**

A13 `invalid type assertion: x.(T)`

Thực hiện cú pháp type assertion trên một biến không phải kiểu interface (`non-interface type`).

→ Chỉ áp dụng type assertion trên biến kiểu interface; kiểm tra lại kiểu dữ liệu thực tế của biến x. **Ch3,14,19**

A14 `x does not implement I (missing method M)`
• `method M has pointer receiver`

Type cụ thể không thỏa mãn interface vì thiếu method hoặc sai khác receiver (pointer receiver vs value receiver).

→ Kiểm tra receiver của method `M`: nếu là pointer receiver `*T`, đối tượng truyền vào phải là địa chỉ `&val`. **Ch3,5,19**

B — RUNTIME & PANIC

B01 `panic: runtime error: invalid memory address or nil pointer dereference`

Dereference con trỏ `nil` hoặc gọi method qua `nil` interface có thể gây `panic`. Method có pointer receiver vẫn có thể nhận `nil` và xử lý nó; chỉ lời gọi trên receiver `nil` chưa đủ kết luận.

> ! Nil dereference có thể làm process crash; đừng dùng `recover` để che lỗi mà không xác định contract và recovery boundary.

→ Thêm kiểm tra `if ptr == nil` trước khi truy cập, hoặc rà soát constructor/hàm khởi tạo chưa gán con trỏ. **Ch2,3,12**

B02 `panic: runtime error: index out of range [x] with length y`

Truy cập phần tử của slice hoặc mảng tại chỉ số âm hoặc lớn hơn hay bằng độ dài hiện tại `len`.

→ Kiểm tra `len(s)` trước khi truy cập trực tiếp qua index, hoặc ưu tiên duyệt bằng vòng lặp `for range`. **Ch2,7**

B03 `panic: runtime error: slice bounds out of range [:x] with capacity y`

Biểu thức cắt slice `s[low:high]` có cận âm, cận trên vượt `cap(s)` hoặc cận dưới lớn hơn cận trên.

→ Với slice hai cận, kiểm tra `0 <= low <= high <= cap(s)`; string dùng `len` làm cận trên, không có `cap`. **Ch2,7**

B04 `panic: runtime error: makeslice: len out of range`

Diagnostic này chỉ đường cấp phát slice có length không hợp lệ hoặc vượt giới hạn implementation. Capacity không hợp lệ và `len > cap` có thể phát `makeslice: cap out of range`; allocation thất bại vì OOM có đường lỗi khác.

→ Kiểm tra kích thước buffer không âm và nằm trong ngân sách; buffer độ dài 0 có thể hợp lệ theo contract. **Ch2,7**

B05 `panic: assignment to entry in nil map`

Thực hiện thao tác gán giá trị vào một key của map chưa được khởi tạo (`nil map`).

→ Khởi tạo map bằng hàm `make(map[K]V)` hoặc map literal `{}` trước khi ghi dữ liệu. **Ch2,3**

B06 `panic: send on closed channel`

Thực hiện hành vi gửi dữ liệu vào một channel đã bị gọi lệnh `close()`.

> ! Bên đóng phải biết mọi sender đã ngừng gửi. `sync.Once` chỉ tránh đóng hai lần; nó không chặn việc gửi sau khi đóng.

→ Gom quyền đóng về goroutine điều phối, chờ các sender kết thúc rồi mới `close`. Producer duy nhất có thể tự đóng sau lần gửi cuối. **Ch8,9,12**

B07 `panic: close of closed channel`

Gọi lệnh `close()` lần thứ hai trên cùng một channel đã được đóng trước đó.

→ Sử dụng `sync.Once` để bảo đảm đóng channel duy nhất một lần, hoặc kiểm soát lifecycle đóng tập trung. **Ch8,9**

B08 `panic: interface conversion: interface {} is T1, not T2`

Ép kiểu dạng assertion `v := x.(T2)` thất bại trong runtime khi giá trị thực tế bên trong là `T1` hoặc `nil`.

→ Dùng `v, ok := x.(T2)` khi type động chưa được bảo đảm bởi contract. Nếu dùng assertion một giá trị, cần invariant rõ và hiểu rằng vi phạm sẽ `panic`. **Ch3,14,19**

B09 `fatal error: concurrent map read and map write`

Nhiều goroutine cùng truy cập đọc và ghi đồng thời vào một biến kiểu `map` chuẩn mà không có cơ chế khóa đồng bộ.

> ! Đây là lỗi fatal từ kiểm tra nội tại của runtime, không thể chặn bằng `recover`; khác với `-race`, theo dõi truy cập không đồng bộ trên những đường chạy được thực thi. Cả hai không bảo đảm phát hiện mọi race trong mọi lịch chạy.

→ Bảo vệ map bằng `sync.RWMutex`, dùng `sync.Map` khi phù hợp, hoặc tuần tự hóa qua channel; kiểm thử với `go test -race`. **Ch8,9,20**

B10 `fatal error: stack overflow`

Hàm gọi đệ quy vô tận hoặc chuỗi lồng hàm quá sâu khiến kích thước stack của goroutine tăng trưởng liên tục vượt quá giới hạn tối đa do Go runtime quy định (`maxstacksize`).

→ Bổ sung điều kiện dừng (base case) chuẩn xác cho hàm đệ quy, hoặc chuyển đổi giải thuật sang dạng lặp (iterative). **Ch10**

B11 `fatal error: sync: unlock of unlocked mutex`

Go 1.27.1 phát lỗi fatal khi gọi `Unlock()` trên `sync.Mutex` hiện không được khóa, kể cả khi mutex đã được khóa rồi mở trước đó. `recover` không cứu được lỗi này; `sync.RWMutex` có diagnostic riêng.

→ Kiểm tra cấu trúc đặt `defer mu.Unlock()` ngay sau `mu.Lock()`, tránh gọi unlock thủ công rải rác nhiều nhánh return. **Ch8,12**

C — ERROR VALUES & I/O

C01 `io.EOF`

Tín hiệu hoàn tất stream đọc dữ liệu (đã đọc hết toàn bộ byte sẵn có, không còn byte nào phía sau).

! `io.EOF` thường đánh dấu hết stream; EOF xuất hiện sớm vẫn có thể vi phạm contract của format hoặc operation.

→ Xử lý phần `n > 0` trước khi xét error; chỉ coi EOF là kết thúc thành công nếu operation đã nhận đủ dữ liệu theo contract. **Ch7,11,20**

C02 `io.ErrUnexpectedEOF`

Stream dữ liệu bị ngắt đột ngột trước khi đọc đủ số byte dự kiến (ví dụ trong `io.ReadFull` hoặc fixed-size header).

→ Kiểm tra kết nối mạng có bị ngắt giữa chừng hoặc payload truyền tải có bị cắt cụt (truncated) hay không. **Ch7,11,20**

C03 `io.ErrShortWrite`

Hàm `Write()` ghi được ít byte hơn buffer cung cấp nhưng không trả về lỗi cụ thể từ hệ thống bên dưới.

→ Kiểm tra `n` và `err`; không retry vô hạn khi writer không tiến triển. Nếu policy cho phép ghi tiếp, cần budget và bảo toàn phần chưa ghi. **Ch7**

C04 `target document exceeds byte limit`

Dữ liệu stream đi vào vượt quá ngưỡng kích thước tối đa cho phép theo chính sách an toàn (giá trị sentinel `ErrDocumentTooLarge`).

→ Bảo vệ bộ nhớ khỏi cạn kiệt (OOM); từ chối xử lý tiếp và trả về lỗi payload vượt giới hạn cho caller. **Ch7,20**

C05 `json: cannot unmarshal string into Go struct field of type int`

Kiểu dữ liệu trong JSON payload không tương thích với định nghĩa kiểu của struct field đích trong Go.

→ Đối chiếu schema JSON nguồn, điều chỉnh kiểu dữ liệu struct field hoặc triển khai `json.Unmarshaler` tùy biến. **Ch7,14,20**

C06 `unexpected end of JSON input`

Chuỗi JSON đưa vào `json.Unmarshal` bị rỗng hoặc bị ngắt cụt giữa chừng cú pháp.

! Phân biệt rõ: `json.Unmarshal` trả về lỗi cú pháp này khi buffer rỗng; trong khi `json.Decoder.Decode()` trên stream rỗng trả về `io.EOF`, và chỉ trả về unexpected end hoặc `io.ErrUnexpectedEOF` khi stream bị cắt cụt khi đang parse dở token.

→ Kiểm tra dữ liệu thực đọc được và lỗi decode; không dùng riêng `Content-Length` làm bằng chứng body đầy đủ. Giới hạn byte và xử lý stream rỗng theo contract API. **Ch7,11,20**

C07 `sql: no rows in result set`

Không có hàng cho truy vấn một kết quả; `QueryRowContext` trả `*sql.Row` và trì hoãn lỗi `sql.ErrNoRows` tới `Scan`, không trả cặp row/error trực tiếp.

→ Dùng `errors.Is(err, sql.ErrNoRows)` để phân biệt trường hợp không có dữ liệu với lỗi truy vấn database thực sự. **Ch13,20**

C08 `sql: transaction has already been committed or rolled back`

Gọi lệnh `Commit()` hoặc `Rollback()` trên một transaction cơ sở dữ liệu (`*sql.Tx`) đã kết thúc trước đó (`sql.ErrTxDone`).

→ Rà soát pattern `defer tx.Rollback()`; nếu `Commit()` đã thành công thì rollback sau đó trả về lỗi này và an toàn để bỏ qua. **Ch13**

D — CONTEXT & CANCELLATION

D01 `context canceled`

Context bị hủy bởi `cancel()`, context cha bị hủy hoặc lifecycle của operation kết thúc.

! Operation return không tự cancel child do nó tạo. Kiểm tra ai sở hữu CancelFunc; parent sống lâu và timer chưa hết có thể giữ child ngoài nhu cầu. `go vet` sạch không chứng minh mọi đường bàn giao cancel đều đúng. [Ch12]

! `http.Server.Shutdown` không tự hủy context của active request. Muốn signal dừng process hủy handler phải truyền application context có chủ đích; policy ấy có thể làm gián đoạn graceful drain.

→ Cho operation quan sát cancellation, dọn tài nguyên rồi trả kết quả theo contract caller. `OutcomeCancel` là phân loại của opsprobe; HTTP 499 là quy ước phi chuẩn của một số hệ thống, không phải status bắt buộc của Go server. **Ch4,11,12,20**

D02 `context deadline exceeded`

Tác vụ không hoàn thành trong khoảng thời gian timeout hoặc trước mốc thời gian deadline đã ấn định (`context.DeadlineExceeded`).

> ! Deadline có thể giải phóng quan hệ context/timer, nhưng không giết goroutine không quan sát context. Đối chiếu tín hiệu hoàn tất với stack chờ, không chỉ `ctx.Err()`. [Ch9,10,12]

> ! Client bị timeout không có nghĩa là downstream service đã ngừng xử lý; kết nối có thể vẫn đang chạy ngầm gây nghẽn.

→ Xác định nơi khởi tạo deadline (`WithTimeout`), đo lường độ trễ từng phân đoạn và điều chỉnh timeout hợp lý. **Ch4,11,12,20**

D03 Client Disconnect During Body Read

Client ngắt kết nối khi handler đang đọc `req.Body`. Lỗi đọc tùy đường kết nối và trạng thái body; không bảo đảm luôn là `context.Canceled`. Incoming request context bị hủy khi kết nối đóng, request bị hủy trên HTTP/2 hoặc `ServeHTTP` kết thúc.

> ! Hủy incoming context không chứng minh công việc dùng context tách rời như `Background` đã kết thúc. Tìm đường tạo child và owner của công việc chạy nền. [Ch12]

→ Kiểm tra `r.Context().Done()` hoặc `errors.Is(err, context.Canceled)` trong các tác vụ đọc stream để kịp thời giải phóng CPU và buffer. **Ch11,12,20**

D04 Server Graceful Shutdown Timeout

Phương thức `server.Shutdown(ctx)` chạm mốc timeout của context truyền vào trước khi toàn bộ các kết nối HTTP đang hoạt động được đóng mềm mại, trả về `context.DeadlineExceeded`.

> ! Khi drain hết ngân sách, mở goroutine dump tìm handler/downstream chưa trả về và đường join của owner. Gọi cancel không đồng nghĩa đã chờ công việc dừng. [Ch9,10,12]

→ Điều tra request còn chạy và ngân sách drain. Context truyền cho `Shutdown` giới hạn thời gian chờ, không tự trở thành context của handler; kiểm tra policy hủy request riêng. **Ch12,20**

E — FILESYSTEM & PROCESS

E01 `no such file or directory`

Đường dẫn tập tin hoặc thư mục đích không tồn tại trên filesystem của hệ điều hành (`os.ErrNotExist`).

→ Kiểm tra đường dẫn tuyệt đối/tương đối, working directory thực tế của process, hoặc mount path trong container. **Ch7,15,17**

E02 `permission denied`

Process không có đủ quyền đọc, ghi, hoặc thực thi tập tin/thư mục đích (`os.ErrPermission`).

→ Kiểm tra file mode permission (`chmod`), UID/GID của container user (non-root), hoặc chính sách SELinux/AppArmor. **Ch15,17**

E03 `file already exists`

Cố gắng tạo mới tập tin với cờ độc quyền (`os.O_EXCL`) hoặc tạo thư mục (`os.Mkdir`) khi đường dẫn đã tồn tại (`os.ErrExist`).

→ Sử dụng `os.IsExist(err)` để xử lý phân nhánh ghi đè hoặc bỏ qua nếu tập tin đã sẵn sàng. **Ch7,15**

E04 `read-only file system`

Cố gắng ghi dữ liệu vào một filesystem được mount ở chế độ chỉ đọc (lỗi hệ thống `EROFS`, ví dụ trong distroless hoặc container bảo mật cao).

→ Chuyển đường dẫn ghi file tạm sang thư mục được mount volume riêng biệt (như `/tmp` kiểu `emptyDir`). **Ch17,18**

E05 `executable file not found in $PATH`

Hàm `exec.LookPath` hoặc `exec.Command` không tìm thấy file thực thi tương ứng trong các thư mục của biến môi trường `$PATH` (`exec.ErrNotFound`).

→ Kiểm tra working directory, mount và base image thực tế; image distroless thường không có shell/ls, nhưng biến thể debug khác. Dùng đường dẫn phù hợp hoặc công cụ chẩn đoán riêng. **Ch15,17**

E06 `signal: terminated / signal: killed`

Process bị dừng đột ngột bởi tín hiệu hệ điều hành: SIGTERM khi dừng dịch vụ có phối hợp, hoặc SIGKILL khi bị cưỡng chế tiêu diệt (kill -9, hết hạn grace period, hoặc do Linux kernel OOM Killer).

> ! SIGKILL có thể đến từ nhiều nguyên nhân (admin kill, timeout của bộ điều phối, cgroup limit); không tự động suy diễn mọi SIGKILL đều là OOM.

→ Kiểm tra dmesg/journalctl hoặc Kubernetes termination reason để xác định process bị kill bởi OOM Killer hay do lệnh quản trị bên ngoài. **Ch10,12,17**

E07 `failed to load bpf program: operation not permitted / permission denied (eBPF Verifier Error)`

- `failed to load bpf program: operation not permitted`
- `failed to load bpf program: permission denied: ... (verifier log)`

Load bị từ chối có thể do quyền, chính sách host hoặc bytecode không đạt kiểm tra verifier. Capability cần thiết tùy kernel và loại program; không suy ra nguyên nhân chỉ từ một chuỗi permission denied.

> ! Đọc error và verifier log nếu có. Verifier phân tích program trước khi cho chạy; không phải mọi lỗi permission đều có log verifier.

→ Kiểm tra feature, hook, policy và quyền tối thiểu trước khi cấp thêm quyền. Sửa bytecode theo log cụ thể; không có quy tắc rằng mọi pointer đều phải đi qua `bpf_probe_read_*`. **Ch27**

F — NETWORK / HTTP / TLS

F01 `dial tcp: lookup host: no such host`

Hệ thống DNS resolver không thể phân giải tên miền mục tiêu thành địa chỉ IP hợp lệ (`net.DNSError`).

→ Kiểm tra cấu hình DNS cục bộ (`/etc/resolv.conf`), service discovery của cụm Kubernetes (CoreDNS), hoặc lỗi chính tả host. **Ch11,16,20**

F02 `dial tcp: connect: connection refused`

Connect bị từ chối; thường do không có listener ở đích, nhưng firewall hay thiết bị trung gian cũng có thể chủ động reject. Lỗi này không chứng minh host đích đang hoạt động bình thường.

→ Xác nhận service mục tiêu đã khởi chạy, bind đúng địa chỉ (`0.0.0.0` thay vì `127.0.0.1`), và port đích chính xác. **Ch4,11,16,20**

F03 `i/o timeout / dial tcp: i/o timeout`

Dial không hoàn tất trong ngân sách kết nối, hoặc đọc/ghi vượt deadline tương ứng. `net.Dialer.Timeout` giới hạn dial, không đặt timeout chung cho mọi lần đọc/ghi sau đó.

→ Kiểm tra tường lửa/security group có đang chặn gói tin SYN không, routing mạng, hoặc server mục tiêu bị treo cứng. **Ch11,16,20**

F04 `read: connection reset by peer`

TCP stack báo connection reset. RST có thể đến từ peer hoặc thiết bị trung gian; riêng error string không xác định nguồn gây reset.

> ! Process restart, policy timeout hoặc middlebox là các hướng điều tra, không phải nguyên nhân đã được chứng minh. Crash cũng không luôn tạo RST.

→ Kiểm tra log của server phía đối tác xem có sự cố `panic`, OOM, hay timeout của load balancer trung gian không. **Ch11,12,20**

F05 `write: broken pipe`

Lời gọi ghi bị báo EPIPE ở đường socket đang xét. TCP cho phép half-close; nhận FIN chỉ báo đóng chiều gửi của peer, không tự chứng minh mọi lần ghi tiếp phải lỗi.

→ Bắt lỗi ghi socket, ngừng gửi dữ liệu lên kết nối đã chết và dọn dẹp các tài nguyên liên đới của handler. **Ch11,12,20**

F06 `http: server closed idle connection`

Transport phát hiện server đóng một HTTP/1.x connection mà nó đang coi là idle. Idle timeout là một khả năng, không phải nguyên nhân duy nhất.

→ Kiểm tra policy server/proxy và thời điểm reuse. Retry chỉ khi request body và side effect cho phép; đặt `IdleConnTimeout` thấp hơn server có thể giảm một race, không loại bỏ mọi close. **Ch11,20**

F07 `x509: certificate signed by unknown authority`

Bắt tay TLS thất bại vì chứng chỉ số của máy chủ không được ký bởi Certificate Authority (CA) có trong trust store của client.

> ! `InsecureSkipVerify` bỏ kiểm tra certificate chain và hostname mặc định; không dùng để chữa lỗi production. Custom verification cần contract riêng; TLS encryption không vì cờ này mà tự biến mất.

→ Thêm Root CA nội bộ vào trust store của hệ điều hành/container hoặc nạp CA tùy biến qua `tls.Config.RootCAs`. **Ch11,17,20**

F08 `x509: certificate has expired or is not yet valid`

Chứng chỉ số TLS của máy chủ đã quá hạn sử dụng, hoặc đồng hồ hệ thống trên máy chạy Go bị lệch thời gian (time drift).

→ Kiểm tra thời hạn chứng chỉ bằng `openssl x509`, gia hạn chứng chỉ hoặc đồng bộ lại đồng hồ hệ điều hành qua NTP. **Ch11,17**

F09 `x509: certificate is valid for X, not Y`

Tên miền truy cập không khớp với danh sách Subject Alternative Names (SAN) được cấp trong chứng chỉ số TLS.

→ Cấp lại chứng chỉ với trường SAN bao quát đúng tên miền hoặc địa chỉ IP đang sử dụng để kết nối. **Ch11,17**

F10 Unread Response Body (Lost Connection Reuse)

Với HTTP/1.x, đóng body trước EOF có thể làm mất khả năng tái sử dụng kết nối. HTTP/2 có lifecycle stream riêng; không suy ra cứ đóng body sớm là toàn bộ socket phải đóng.

> ! EOF của `io.LimitReader` có thể chỉ báo hết ngân sách, không phải hết body gốc. `ReuseEligible` là cờ của helper trong lab, không phải cam kết từ Transport.

→ Đóng body; nếu chọn drain, giới hạn byte và thời gian. EOF hỗ trợ một số đường HTTP/1.x reuse, không bảo đảm reuse ở mọi Transport/protocol. **Ch11,20**

F11 `RequestTimeTooSkewed`

- `RequestTimeTooSkewed: The difference between the request time and the current time is too large.`

Diagnostic này có thể gặp từ Amazon S3 khi thời gian request lệch ngoài cửa sổ được service chấp nhận. Không áp một ngưỡng phút chung cho mọi service hoặc mọi kiểu ký AWS; đọc response và contract API thực tế.

> ! Thường gặp trong container hoặc máy ảo khi tiến trình đồng bộ thời gian NTP bị lỗi hoặc đóng băng sau khi suspend.

→ Đồng bộ lại đồng hồ hệ điều hành thông qua dịch vụ chrony hoặc NTP daemon (`chronyc makestep`). **Ch24**

G — DATABASE

G01 `sql: database is closed`

Thực hiện thao tác trên `*sql.DB` đã đóng. Đây không phải `sql.ErrConnDone`: sentinel ấy dành cho `*sql.Conn` đã trả lại pool hoặc đóng.

→ Xác định owner của pool và ai gọi Close quá sớm. Tái dùng pool lâu dài khi phù hợp; một process có thể có nhiều DB hoặc thay pool có phối hợp, không cần quy tắc duy nhất một lần suốt đời process. **Ch13,20**

G02 `driver: bad connection`

Driver báo connection không dùng được (`driver.ErrBadConn`); `database/sql` có thể retry theo contract. Đây khác với chờ pool đạt `SetMaxOpenConns`.

> ! Chờ pool có thể kết thúc khi connection được trả lại, context bị hủy/hết hạn hoặc DB đóng. Không có error string chuẩn mang tên "connection pool exhausted".

→ Điều tra connection lỗi và thống kê pool riêng. Chỉ đổi `SetMaxOpenConns` sau khi xét capacity database; context budget giới hạn chờ khi driver hỗ trợ, không sửa connection hỏng. **Ch13,20**

G03 Unclosed sql.Rows (Connection Pool Starvation)

Không đóng `Rows` khi dừng duyệt sớm có thể giữ connection ngoài pool. Đọc tới EOF có thể tự đóng Rows theo contract, nhưng caller vẫn nên đóng tường minh và kiểm tra `rows.Err()`.

> ! Không có diagnostic chuẩn mang nghĩa "quên close". Connection bị giữ có thể làm các query sau chờ pool; xem `DB.Stats`, cancellation và lifecycle thực tế, không suy ra mọi truy vấn đều treo tới timeout.

→ Sau khi `QueryContext` thành công, đặt `defer rows.Close()` gần acquisition khi cùng hàm sở hữu Rows. Nếu trả Rows cho caller, phải chuyển rõ quyền đóng. **Ch13,20**

G04 `sqlite: database is locked / busy`

SQLite gặp xung đột ghi đồng thời từ nhiều transaction hoặc goroutine trên cùng một tập tin cơ sở dữ liệu (`busy / locked`).

→ Cân nhắc WAL, busy timeout hoặc tuần tự hóa ghi theo workload. Cú pháp DSN/pragma tùy driver; các tên `_journal_mode` và `_busy_timeout` không phải contract chung của `database/sql`. **Ch13,20**

G05 `UNIQUE constraint failed`

Cố gắng chèn hoặc cập nhật bản ghi có giá trị trùng lặp trên cột được đánh chỉ mục duy nhất (PRIMARY KEY hoặc UNIQUE).

→ Giữ UNIQUE constraint làm rào chắn nguyên tử và xử lý conflict theo nghiệp vụ, có thể dùng upsert phù hợp. Check tồn tại rồi ghi vẫn có race; không thay constraint bằng check-then-act. **Ch13,20**

G06 `FOREIGN KEY constraint failed`

Thao tác insert hoặc delete vi phạm tính toàn vẹn quan hệ khóa ngoại giữa bảng cha và bảng con.

→ Kiểm tra foreign key, transaction, deferred constraint và cascade của database thật; không áp một thứ tự insert/delete duy nhất cho mọi schema. **Ch13,20**

H — CONCURRENCY

H01 `fatal error: all goroutines are asleep - deadlock!`

Runtime phát hiện không còn công việc có thể tiến triển theo các điều kiện kiểm tra nội tại của version đang chạy.

> ! Goroutine chờ I/O hoặc timer không tự đồng nghĩa deadlock. Runtime không phát hiện mọi vòng chờ cục bộ; goroutine khác còn hoạt động có thể che một nhóm đang kẹt.

→ Thiết kế đồ thị phụ thuộc của lock/channel; luôn tuân thủ thứ tự acquisition lock nhất quán và tránh chờ đợi vòng tròn. **Ch8,9,12**

H02 `WARNING: DATA RACE`

Go race detector (`go test -race` / `go run -race`) phát hiện các truy cập xung đột tới cùng vùng nhớ, có ít nhất một lần ghi và thiếu quan hệ đồng bộ cần thiết trên đường chạy được quan sát.

> ! Không bỏ qua data race. Kết quả có thể không nhất quán hoặc gây crash; một lần chạy không crash không chứng minh chương trình đúng.

→ Bảo vệ vùng nhớ dùng chung bằng `sync.Mutex`, chuyển sang toán tử `sync/atomic`, hoặc tái cấu trúc truyền dữ liệu qua channel. **Ch8,9,20**

H03 Goroutine Leak

Goroutine không kết thúc vì không còn đường thoát khỏi chờ channel, lock hoặc I/O. Context chỉ hữu ích nếu operation thực sự quan sát cancellation.

> ! Loop nền vẫn chạy định kỳ cũng có thể vượt lifecycle của owner. `Ticker.Stop` không đóng channel hoặc kết thúc reader; với cơ chế timer mới từ Go 1.23, ticker không còn tham chiếu có thể được GC thu hồi, không phải mọi lần thiếu Stop đều là leak. [Ch9]

→ So số goroutine qua các chu kỳ với dump/profile, xác nhận receive/loop/callback thiếu đường thoát trong source rồi chờ `done` sau sửa. Heap ổn không loại trừ lỗi vòng đời. **Ch9,10,12**

> ! Runtime không tự thu hồi một goroutine chỉ vì nó bị block; những reference còn sống có thể giữ dữ liệu. Go 1.27 có profile `goroutineleak` cho một lớp chờ không thể được đánh thức, không phải bộ phát hiện mọi leak hay cơ chế GC goroutine.

→ Xác định owner và đường thoát; truyền context/deadline tới API có hỗ trợ. Buffer chỉ đổi thời điểm block, không tự sửa leak. **Ch8,9,12,20**

H04 `panic: sync: negative WaitGroup counter`

Phương thức `wg.Add()` nhận giá trị âm làm counter nhỏ hơn 0, hoặc phương thức `wg.Done()` bị gọi nhiều lần hơn số lần `Add()`.

→ Kiểm tra số lần `Done()` và tăng counter trước khi khởi chạy worker trong mẫu đang học. `Add` dương khi counter bằng 0 phải xảy ra trước `Wait`; không để worker mới tự tăng sau khi caller đã có thể `Wait`. **Ch8,9**

H05 `panic: sync: WaitGroup misuse: Add called concurrently with Wait`

Diagnostic này không có nghĩa mọi `Add` đồng thời với `Wait` đều sai. Theo contract, `Add` dương khi counter đang bằng 0 phải xảy ra trước `Wait`; khi counter đã dương, quy tắc khác áp dụng.

→ Trong mẫu worker đang học, đăng ký task trước khi khởi chạy; nếu tái sử dụng WaitGroup, chỉ thêm lứa mới sau khi mọi `Wait` của lứa cũ đã trả về. **Ch8,9**

I — MODULES / TEST / TOOLCHAIN

I01 `go: cannot find main module`

Lệnh `go build` hoặc `go test` được thực thi tại một thư mục không chứa file `go.mod` và không thuộc không gian làm việc `go.work`.

→ Di chuyển terminal vào đúng thư mục con chứa module cần thao tác (`working-directory`), hoặc khởi tạo module bằng `go mod init`. **Ch1,5,18**

I02 `go: module X: git fetch: authentication required`

Go module resolver không thể tải mã nguồn của module private do thiếu thông tin xác thực Git qua SSH hoặc Personal Access Token.

→ Cấu hình biến môi trường `GOPRIVATE=github.com/myorg/*` và thiết lập Git credential helper hoặc SSH key tương ứng. **Ch5,18**

I03 `go vet: composite literal uses unkeyed fields`

Khai báo struct literal không chỉ định tên field tường minh (ví dụ viết `Point{1, 2}` thay vì `Point{X: 1, Y: 2}`).

→ Bổ sung tên field vào struct literal để tránh lỗi vỡ mã nguồn khi struct được thêm trường mới trong tương lai. **Ch3,6**

I04 `go vet: unreachable code`

Đoạn mã nằm ở vị trí không bao giờ có thể được thực thi tới (ngay sau lệnh `return`, `panic`, hoặc vòng lặp vô tận không có break).

→ Loại bỏ đoạn mã chết (dead code) không cần thiết hoặc điều chỉnh lại cấu trúc điều hướng control-flow. **Ch1,4**

I05 `go test: no Go files in directory`

Thư mục chỉ định trong lệnh test không chứa bất kỳ tập tin mã nguồn Go (`*.go`) hợp lệ nào thuộc package cần kiểm thử.

→ Kiểm tra lại đường dẫn package hoặc bổ sung tập tin kiểm thử `*_test.go` tương ứng vào thư mục. **Ch6,18**

I06 `go test: build constraints exclude all Go files`

Tất cả các file mã nguồn Go trong thư mục đều bị loại trừ bởi build tags chỉ định ở đầu file (ví dụ `//go:build integration`).

→ Truyền cờ build tags tương ứng khi thực thi lệnh kiểm thử, ví dụ: `go test -tags=integration ./...`. **Ch6,18**

I07 `go test: timed out after Xm`

Toàn bộ suite kiểm thử hoặc một test cụ thể chạy vượt quá giới hạn thời gian cho phép (mặc định 10 phút hoặc cờ `-timeout`).

→ Điều tra các test bị treo do deadlock, rò rỉ goroutine hoặc sleep quá lâu; chạy `go test -v -timeout=30s` để cô lập test lỗi. **Ch6,8,20**

I08 `API rate limit exceeded`

- `API rate limit exceeded for user ID <id>`

GitHub REST API có thể trả HTTP 403 hoặc 429 khi vượt primary hay secondary rate limit. Theo tài liệu ngày 29-09-2026, mức cơ bản là 5.000 request/giờ cho user được xác thực và 60 cho IP không xác thực; loại token, App, endpoint và Enterprise có ngoại lệ. Dùng header thực tế, không gán mức ấy cho mọi token.

! Thường gặp khi các bot tự động hóa hoặc CI/CD pipeline gửi yêu cầu liên tục trong vòng lặp mà không kiểm tra hạn mức còn lại.

→ Bóc tách header `X-RateLimit-Reset` hoặc `Retry-After` để tạm dừng tiến trình (sleep with jitter) cho đến khi quota được khôi phục trước khi gửi yêu cầu tiếp theo. **Ch25**

J — CONTAINER / KUBERNETES / CI-CD

J01 CrashLoopBackOff

Container liên tục kết thúc rồi được restart theo policy, và kubelet đang áp backoff. Exit 0 dưới `restartPolicy: Always`, failure hay liveness kill đều có thể dẫn tới vòng restart; không chỉ `panic` hoặc exit code dương.

! Đây là waiting reason của container được phản ánh trong trạng thái Pod, không phải chuỗi lỗi của Go runtime hay một Pod phase riêng.

→ Kiểm tra log của container trước khi crash bằng lệnh `kubectl logs <pod> --previous`, rà soát biến môi trường và config bị thiếu. **Ch17,18,21**

J02 OOMKilled (Exit Code 137)

Container runtime/kubelet báo container bị OOM kill. Có thể liên quan cgroup memory limit hoặc áp lực OOM của host; cần đọc reason và sự kiện, không suy từ RSS của một process đơn lẻ.

! Exit code 137 thường biểu thị SIGKILL và không riêng OOM. SIGKILL không thể bị handler chặn, nên process không chạy `defer` để dọn tài nguyên.

→ Kiểm tra rò rỉ bộ nhớ qua heap profile (`pprof`), điều chỉnh thuật toán caching hoặc nâng hạn mức `limits.memory` cho pod. **Ch10,17,21**

J03 ImagePullBackOff / ErrImagePull

Waiting reason của container, được phản ánh trong Pod status, khi kéo image lỗi hoặc đang backoff. Sai tag, thiếu credential và lỗi mạng là các hướng điều tra, không phải nguyên nhân đã được xác nhận chỉ từ reason.

! Kubelet trên node phối hợp với container runtime để kéo image; trạng thái được báo về API. Lỗi này xảy ra trước khi container Go khởi chạy.

→ Kiểm tra tên image và tag, cấu hình `imagePullSecrets` cho service account, hoặc kiểm tra kết nối mạng egress của cụm. **Ch17,18**

J04 CreateContainerConfigError

Trạng thái lỗi cấu hình (`Waiting Reason`) khi Kubernetes không tìm thấy ConfigMap hoặc Secret được tham chiếu trong cấu hình Pod.

! Kubelet gặp lỗi khi chuẩn bị cấu hình container; không phải diagnostic của Go hay riêng thao tác chọn node của scheduler.

→ Chạy `kubectl describe pod <pod>` để xác định chính xác tên ConfigMap/Secret đang bị thiếu và tạo bổ sung vào namespace. **Ch17,21**

J05 Readiness probe failed

- `HTTP probe failed with statuscode: 503`

Probe readiness thất bại theo cấu hình; khi đạt failure threshold, Pod Ready có thể thành `false`. EndpointSlice có thể vẫn giữ endpoint với `ready=false`; behavior Service còn phụ thuộc `publishNotReadyAddresses`.

! Readiness failure không tự restart container và cũng không chứng minh process đang chạy bình thường; nó chỉ phản ánh kết quả của probe đã chọn.

→ Kiểm tra tình trạng kết nối đến các dependency hạ tầng (database, cache) và rà soát logic kiểm tra sẵn sàng của handler. **Ch12,17,20,21**

J06 Liveness probe failed

Liveness probe thất bại; đạt failure threshold dẫn tới xử lý restart container theo lifecycle và restart policy. Nếu `failureThreshold=1`, một lần lỗi đã đủ đạt ngưỡng; vẫn phải xét grace period và policy, không suy ra restart tức thời chỉ từ event.

> ! Tránh dùng lỗi database xa làm liveness failure nếu restart không chữa được nó; policy sai có thể gây cascading restart.

→ Chọn probe mà restart có khả năng chữa failure. Thường tránh đưa dependency xa vào liveness nếu nó gây restart dây chuyền; đây là khuyến nghị thiết kế, không phải giới hạn API chỉ cho kiểm tra nội tại. **Ch12,17,21**

J07 Admission Gate Rejected

- `unsigned artifact or digest mismatch`

Trạng thái từ chối (`Admission / Policy Status`) từ Kubernetes Validating Admission Webhook hoặc Promotion Gate của CI/CD khi artifact container image không có chữ ký cosign hợp lệ hoặc digest SHA-256 không khớp với bản ghi phát hành.

> ! Đây là rào chắn chính sách ở tầng CI/CD delivery hoặc Kubernetes admission webhook, không phải lỗi từ runtime của ứng dụng.

→ Bảo đảm image được build và ký số thông qua pipeline CI chính thức có attestation/provenance hợp lệ trước khi promote. **Ch18**

J08 `Operation cannot be fulfilled: the object has been modified`

- `Operation cannot be fulfilled on <resource>: the object has been modified; please apply your changes to the latest version and try again`

API server trả HTTP 409 Conflict khi update gửi `metadata.resourceVersion` không còn khớp version hiện hành. Đối chiếu API semantics; không cần giả định client thấy hay thao tác trực tiếp với bản ghi etcd.

> ! Đây là cơ chế kiểm soát đồng thời lạc quan (Optimistic Concurrency Control) của Kubernetes nhằm ngăn chặn ghi đè mất dữ liệu giữa các client cạnh tranh.

→ Sử dụng `k8s.io/client-go/util/retry.RetryOnConflict` để đọc lại snapshot mới nhất từ API Server và áp dụng thay đổi trước khi thử lại. **Ch22**

J09 `no kind is registered for the type in scheme`

- `no kind is registered for the type <Type> in scheme <Scheme>`

Lỗi cấu hình runtime (`Kubernetes Scheme Error`) khi một Go struct được truyền vào Controller Client hoặc Reconciler nhưng kiểu dữ liệu này chưa được đăng ký vào bảng `runtime.Scheme` thông qua hàm `AddToScheme`.

> ! Xảy ra phổ biến khi khởi tạo Operator với Custom Resource Definition (CRD) nhưng quên nạp `v1alpha1.AddToScheme(mgr.GetScheme())`.

→ Đăng ký SchemeBuilder của CRD vào Manager Scheme trước khi khởi động Controller. **Ch23**

J10 Denied by Supply Chain Policy: reachable vulnerability or untrusted builder

- `reachable <SEVERITY> vulnerability <ID> in <package> (symbol: <symbol>)`
- `untrusted builder identity: <builder>`

Trạng thái chặn theo policy khi công cụ phân tích tìm đường gọi có thể tới ký hiệu liên quan lỗ hổng, hoặc identity của builder không được chấp nhận.

> ! Reachability tĩnh không chứng minh đường ấy đã chạy, khai thác thành công hay có mã độc. Phân biệt kết quả phân tích với trace thực nghiệm.

→ Cập nhật phiên bản dependency đã vá lỗi, loại bỏ việc gọi ký hiệu chứa lỗ hổng, hoặc kiểm tra lại builder ID trong CI workflow. **Ch26**

J11 MCP Tool Execution Denied: unauthorized mutation or SSRF boundary violation

- `mcp: role <role> not authorized for mutating tool restart_service`
- `unapproved target_id <id>: blocked by SSRF allowlist policy`

Trạng thái từ chối theo policy của lab Chương 28 khi role không được phép thay đổi hoặc `target_id` không nằm trong danh bạ. Đây là message ứng dụng, không phải diagnostic chuẩn MCP. Lab không nhận URL từ caller và không kiểm tra từng IP sau DNS/redirect; không trình bày nó như bộ chặn mọi địa chỉ private/metadata.

> ! Schema kiểm tra cấu trúc và constraint đã khai báo, không thay authorization, kiểm soát destination hay phòng prompt injection. Handler, gateway hoặc lớp mạng phải thực thi policy phù hợp; không phải yêu cầu bắt buộc dùng Go.

→ Kiểm tra role của Agent trong phiên MCP, cấu hình whitelist URL cho công cụ mạng, hoặc cấp token ủy quyền (Change Request ID) trước khi gọi các tool gây đột biến. **Ch28**